

The Complete HTML Reference Book

Every element and attribute from HTML 1
(1991) to the HTML Living Standard, with CSS

HTML 1 • HTML 2.0 • HTML 3.2 • HTML 4.01 • XHTML • HTML5 •
Living Standard

Elements • Attributes • Global attributes • Events • Input types •
CSS • Entities • Guides

Compiled from the HTML Living Standard, MDN Web Docs,
htmlreference.io, codeshack.io and the W3Schools HTML and CSS
tutorial.

Edition of September 2026

Contents

How to use this book	30
Part 1: The History of HTML	33
A short history of HTML	34
Part 2: HTML Fundamentals	43
HTML fundamentals	44
Part 3: HTML Version by Version	54
How to read this part	55
HTML 1.0 (1991-1993)	56
HTML 2.0 (RFC 1866, 1995)	65
HTML 3.2 (W3C, January 1997)	83
HTML 4.0 and 4.01 (1997-1999)	110
HTML5 (WHATWG 2004 onwards, W3C Recommendation 2014) . .	147
The HTML Living Standard (2014-today)	202
Part 4: Global Attributes	209
Global attributes	210
accesskey	212
autocapitalize	213
autocorrect	214

autofocus	215
class	216
contenteditable	218
data-*	220
dir	222
draggable	223
enterkeyhint	224
exportparts	225
hidden	226
id	228
inert	229
inputmode	230
is	231
itemid, itemprop, itemref, itemscope, itemtype	232
lang	234
nonce	236
part	237
popover	238
role	240
aria-*	241
slot	243
spellcheck	244
style	245
tabindex	247
title	249
translate	251

virtualkeyboardpolicy 252

writingsuggestions 253

xml:lang, xml:base, xml:space 254

Part 5: Event Handler Attributes 255

Event handler attributes 256

onabort 261

onauxclick 262

onbeforeinput 263

onbeforematch 264

onbeforetoggle 265

onblur 266

oncancel 267

oncanplay 268

oncanplaythrough 269

onchange 270

onclick 271

onclose 272

oncontextmenu 273

oncopy, oncut, onpaste 274

oncuechange 275

ondblclick 276

ondrag, ondragend, ondragenter, ondragleave, ondragover,
ondragstart, ondrop 277

ondurationchange 278

onemptied 279

onended	280
onerror	281
onfocus	282
onformdata	283
oninput	284
oninvalid	285
onkeydown	286
onkeypress	287
onkeyup	288
onload	289
onloadeddata	290
onloadedmetadata	291
onloadstart	292
onmousedown, onmouseup	293
onmouseenter, onmouseleave	294
onmousemove	295
onmouseover, onmouseout	296
onpause, onplay, onplaying	297
onpointerdown, onpointerup, onpointermove, onpointerover, onpointerout, onpointerenter, onpointerleave, onpointercancel, ongotpointercapture, onlostpointercapture	298
onprogress	300
onratechange	301
onreset	302
onresize	303
onscroll	304
onscrollend	305

onsecuritypolicyviolation 306

onseeked, onseeking 307

onselect 308

onselectionchange 309

onslotchange 310

onstalled 311

onsubmit 312

onsuspend 313

ontimeupdate 314

ontoggle 315

ontouchstart, ontouchmove, ontouchend, ontouchcancel 316

onvolumechange 317

onwaiting 318

onwheel 319

onanimationstart, onanimationend, onanimationiteration,
onanimationcancel 320

ontransitionstart, ontransitionend, ontransitionrun, ontransitioncancel₃₂₁

oncontextlost, oncontextrestored 322

Window events on <body>: onafterprint, onbeforeprint,
onbeforeunload, onhashchange, onlanguagechange, onmessage,
onmessageerror, onoffline, ononline, onpagehide, onpagereveal,
onpageshow, onpageswap, onpopstate, onrejectionhandled,
onstorage, onunhandledrejection, onunload 323

Part 6: Element Reference **325**

Element index **326**

Elements by category **334**

<a> Anchor (hyperlink)	336
<abbr> Abbreviation	350
<acronym> Acronym (obsolete)	355
<address> Contact address	360
<applet> Java applet (obsolete)	364
<area> Image map area	370
<article> Self-contained article	379
<aside> Aside (sidebar / tangential content)	385
<audio> Embedded audio	390
 Bring attention to (bold)	399
<base> Document base URL	403
<basefont> Base font (obsolete)	408
<bdi> Bidirectional isolate	413
<bdo> Bidirectional text override	418
<bgsound> Background sound (non-standard, obsolete)	423
<big> Bigger text (obsolete)	428
<blink> Blinking text (non-standard, obsolete)	431
<blockquote> Block quotation	435
<body> Document body	440
 Line break	452
<button> Button	457
<canvas> Graphics canvas	467
<caption> Table caption	474
<center> Centred block (obsolete)	479
<cite> Citation (title of a work)	483
<code> Inline code	487

<col> Table column	491
<colgroup> Table column group	497
<content> Shadow DOM insertion point (obsolete)	502
<data> Machine-readable value	507
<datalist> Predefined options for an input	512
<dd> Description details	517
 Deleted text	522
<details> Disclosure widget	527
<dfn> Definition term	532
<dialog> Dialog box	537
<dir> Directory list (obsolete)	544
<div> Generic block container	549
<dl> Description list	555
<dt> Description term	561
 Emphasis	564
<embed> External content	568
<fencedframe> Fenced frame (experimental)	574
<fieldset> Form field group	579
<figcaption> Figure caption	585
<figure> Figure with optional caption	589
 Font (obsolete)	593
<footer> Footer	598
<form> Form	605
<frame> Frame (obsolete)	615
<frameset> Frameset (obsolete)	622
<h1> Headings h1 to h6	627

<head> Document head	633
<header> Header	639
<hgroup> Heading group	644
<hr> Thematic break (horizontal rule)	648
<html> Document root	654
<i> Idiomatic text (italic)	661
<iframe> Inline frame	665
<image> Image (non-standard alias, obsolete)	678
 Image	683
<input> Input control	696
<ins> Inserted text	716
<isindex> Search index prompt (obsolete)	721
<kbd> Keyboard input	726
<keygen> Key-pair generator (obsolete)	729
<label> Form control label	734
<legend> Fieldset caption	740
List item	745
<link> External resource link	751
<listing> Listing (obsolete)	764
<main> Main content	767
<map> Image map	772
<mark> Highlighted (marked) text	777
<marquee> Scrolling text (obsolete)	781
<math> MathML root	788
<menu> Menu (toolbar list)	794
<menuitem> Menu item (obsolete)	799

<meta> Metadata	804
<meter> Scalar measurement (gauge)	813
<multicol> Multi-column text (non-standard, obsolete)	819
<nav> Navigation section	824
<nextid> Next identifier (HTML 1 editor hint)	829
<nobr> No line break (non-standard, obsolete)	834
<noembed> Fallback for embed (obsolete)	837
<noframes> Fallback for frames (obsolete)	840
<noscript> Fallback when scripting is disabled	843
<object> External object	847
Ordered list	854
<optgroup> Option group	860
<option> Option	865
<output> Calculation or user-action result	870
<p> Paragraph	875
<param> Object parameter (deprecated)	881
<picture> Picture (responsive image container)	886
<plaintext> Plain text (obsolete)	892
<portal> Portal (experimental, abandoned)	896
<pre> Pre-formatted text	901
<progress> Progress indicator	907
<q> Inline quotation	912
<rb> Ruby base (deprecated)	917
<rp> Ruby fallback parenthesis	921
<rt> Ruby text (annotation)	924
<rtc> Ruby text container (deprecated)	927

<ruby> Ruby annotation	931
<s> Strikethrough (no longer accurate)	935
<samp> Sample output	939
<script> Script	942
<search> Search section	952
<section> Generic section	956
<select> Drop-down / list box	961
<selectedcontent> Selected option display (customisable select)	968
<shadow> Shadow root insertion point (obsolete)	973
<slot> Shadow DOM slot	977
<small> Side comment (fine print)	984
<source> Media or image source	988
<spacer> Spacer (non-standard, obsolete)	994
 Generic inline container	999
<strike> Strikethrough (obsolete)	1003
 Strong importance	1007
<style> Embedded style sheet	1011
<sub> Subscript	1017
<summary> Disclosure summary	1020
<sup> Superscript	1024
<svg> Scalable Vector Graphics root	1027
<table> Table	1034
<tbody> Table body	1045
<td> Table data cell	1051
<template> Content template	1061
<textarea> Multi-line text input	1067

<tfoot> Table footer	1077
<th> Table header cell	1082
<thead> Table header	1090
<time> Date / time	1096
<title> Document title	1101
<tr> Table row	1105
<track> Text track for media	1112
<tt> Teletype text (obsolete)	1118
<u> Unarticulated annotation (underline)	1121
Unordered list	1125
<var> Variable	1131
<video> Embedded video	1134
<wbr> Word break opportunity	1144
<xmp> Example text (obsolete)	1148

Part 7: The input Element in Depth 1152

Input types overview 1153

input type=text	1155
input type=password	1156
input type=checkbox	1157
input type=radio	1158
input type=submit	1159
input type=reset	1160
input type=button	1161
input type=hidden	1162
input type=image	1163

input type=file	1164
input type=email	1165
input type=url	1166
input type=tel	1167
input type=search	1168
input type=number	1169
input type=range	1170
input type=color	1171
input type=date	1172
input type=time	1173
input type=datetime-local	1174
input type=month	1175
input type=week	1176

Part 8: Obsolete and Deprecated Features **1178**

Obsolete elements **1179**

Obsolete attributes	1181
-------------------------------	------

Part 9: Attribute Reference **1184**

Alphabetical attribute index **1185**

abbr	1196
accept	1198
accept-charset	1200
action	1201
align	1203

alink	1225
allow	1226
allowfullscreen	1228
allowpaymentrequest	1229
alpha	1230
alt	1231
archive	1237
aria-label	1239
aria-labelledby	1240
as	1241
async	1242
attributionsrc	1243
autocomplete	1246
autofocus	1250
autoplay	1256
axis	1258
background	1260
balance	1261
behavior	1262
bgcolor	1263
blocking	1275
border	1279
bottommargin	1284
capture	1285
cellpadding	1286
cellspacing	1287

challenge	1288
char	1289
charoff	1298
charset	1307
checked	1312
cite	1314
classid	1318
clear	1319
closedby	1320
code	1321
codebase	1322
codetype	1324
color	1325
colorspace	1329
cols	1330
colspan	1333
command	1335
commandfor	1338
compact	1339
content	1345
controls	1346
controlslist	1348
coords	1350
credentialless	1352
crossorigin	1353
csp	1359

data	1360
datetime	1361
declare	1364
decoding	1365
default	1366
defer	1368
dir	1369
direction	1372
dirname	1373
disabled	1375
disablepictureinpicture	1386
disableremoteplayback	1387
display	1390
download	1391
elementtiming	1393
enctype	1394
etc.	1395
event	1396
face	1397
fetchpriority	1399
fill	1403
for	1404
form	1407
formaction	1418
formenctype	1420
formmethod	1422

formnovalidate	1424
formtarget	1426
frame	1428
frameborder	1429
framespacing	1432
gutter	1433
headers	1434
height	1436
high	1453
href	1454
hreflang	1459
hspace	1461
http-equiv	1466
icon	1467
id	1468
imagesizes	1469
imagesrcset	1470
incremental	1471
inputmode	1472
integrity	1473
ismap	1475
itemprop	1476
keytype	1477
kind	1478
label	1479
lang	1485

language	1486
leftmargin	1487
link	1488
list	1489
loading	1490
longdesc	1492
loop	1495
low	1500
lowsrc	1501
manifest	1502
marginheight	1503
marginwidth	1505
max	1507
maxlength	1510
media	1512
method	1517
min	1518
minlength	1520
moz-opaque	1522
mozactionhint	1523
multiple	1524
muted	1526
n	1528
name	1529
nohref	1549
nomodule	1550

nonce	1551
noresize	1553
noshade	1554
novalidate	1555
nowrap	1556
object	1559
onbeforeunload	1560
onhashchange	1561
onlanguagechange	1562
onload	1563
onpopstate	1564
onresize	1565
onstorage	1566
onunload	1567
open	1568
optimum	1570
orient	1571
pattern	1572
ping	1573
placeholder	1575
playsinline	1577
popovertarget	1578
popovertargetaction	1580
poster	1582
preload	1583
preserveAspectRatio	1585

profile	1586
prompt	1587
property	1588
radiogroup	1589
readonly	1590
referrerpolicy	1592
rel	1600
required	1605
results	1608
rev	1609
reversed	1611
rightmargin	1612
role	1613
rows	1614
rowspan	1616
rules	1618
sandbox	1619
scheme	1620
scope	1621
scoped	1623
scrollamount	1624
scrolldelay	1625
scrolling	1626
select	1628
selected	1629
shadowrootclonable	1630

shadowrootdelegatesfocus	1631
shadowrootmode	1632
shadowrootserializable	1633
shape	1634
size	1636
sizes	1643
span	1646
spellcheck	1648
src	1649
srcdoc	1663
srclang	1664
srcset	1665
standby	1667
start	1668
step	1669
stroke	1670
stroke-width	1671
summary	1672
target	1673
text	1679
title	1680
topmargin	1687
truespeed	1688
type	1689
typemustmatch	1707
usemap	1708

valign 1711

value 1720

valuetype 1729

version 1730

viewBox 1731

vlink 1732

volume 1733

vspace 1734

webkitdirectory 1739

width 1740

wrap 1762

xmlns 1764

Part 10: Value References **1767**

Link relationship types (rel) **1768**

Standard meta names **1771**

Autocomplete tokens **1774**

Common MIME types **1777**

URL schemes used in href **1781**

Named colours **1782**

Character entity reference **1785**

Part 11: CSS Reference

1793

CSS fundamentals

1794

CSS selectors

1802

*	1805
element	1806
.class	1807
#id	1808
[attr]	1809
[attr="value"]	1810
[attr~="word"]	1811
[attr ="en"]	1812
[attr^="https"]	1813
[attr\$=".pdf"]	1814
[attr*="example"]	1815
[attr="v" i]	1816
A B	1817
A > B	1818
A + B	1819
A ~ B	1820
A, B	1821
:hover	1822
:focus / :focus-visible / :focus-within	1823
:active	1824
:visited / :link	1825
:first-child / :last-child / :only-child	1826

:nth-child(an+b) / :nth-last-child()	1827
:first-of-type / :last-of-type / :nth-of-type()	1828
:not(selector)	1829
:is(a, b) / :where(a, b)	1830
:has(selector)	1831
:empty	1832
:root	1833
:checked / :indeterminate	1834
:disabled / :enabled / :read-only / :read-write	1835
:required / :optional	1836
:valid / :invalid / :user-invalid / :in-range / :out-of-range / :placeholder-shown	1837
:target	1838
:lang(ar)	1839
:dir rtl)	1840
:popover-open / :modal / :fullscreen / :open	1841
::before / ::after	1842
::first-line / ::first-letter	1843
::marker	1844
::placeholder	1845
::selection	1846
::backdrop	1847
::part(name) / ::slotted(sel)	1848
::file-selector-button	1849
::cue	1850

Fonts and text	1851
Box model	1877
Backgrounds	1895
Layout	1902
Motion	1918
Interaction	1923

Styling HTML elements: cookbook 1935

Lists and tables with CSS 1939

Part 12: HTML and CSS Course 1953

About this course 1954

Lesson 1: Introduction and environment setup	1956
Lesson 2: Your first web page	1958
Lesson 3: Essentials - elements, attributes and nesting	1960
Lesson 4: Document structure	1962
Lesson 5: Text content	1964
Lesson 6: Lists and tables	1966
Lesson 7: Links and navigation	1968
Lesson 8: Images and media	1970
Lesson 9: Forms and validation	1972
Lesson 10: Reusable components	1974
Lesson 11: Styling essentials - the three ways to add CSS	1975
Lesson 12: Selectors and specificity	1977
Lesson 13: Cascade and inheritance	1979

Lesson 14: Typography	1980
Lesson 15: Colours and backgrounds	1982
Lesson 16: The box model	1983
Lesson 17: Layout fundamentals - normal flow	1986
Lesson 18: Positioning and z-index	1987
Lesson 19: Flexbox	1988
Lesson 20: CSS Grid	1991
Lesson 21: Layout patterns	1994
Lesson 22: Responsive design	1995
Lesson 23: Responsive components	1997
Lesson 24: Custom properties (CSS variables)	1998
Lesson 25: calc() and CSS functions	1999
Lesson 26: Transitions	2001
Lesson 27: Transforms	2003
Lesson 28: Animations	2004
Lesson 29: Effects and filters	2005
Lesson 30: Accessibility	2006
Lesson 31: Design systems	2008
Lesson 32: Performance	2010
Lesson 33: Debugging and testing	2011
Lesson 34: Tooling and automation (optional)	2013
Lesson 35: Project - a complete landing page	2014
Lesson 36: Quick quiz and answers	2019

Part 13: Practical Guides	2021
----------------------------------	-------------

Building a complete page step by step	2022
--	-------------

Lists in depth	2025
Tables in depth	2028
Forms in depth	2030
Semantic layout and accessibility	2033
Media: images, audio and video	2035
SEO essentials in HTML	2037
Migrating old HTML (2, 3.2, 4) to HTML5	2038
Part 14: Glossary and Index	2040
Glossary	2041
Glossary in depth	2045
Accessibility (a11y)	2046
ARIA	2047
Attribute	2048
Block-level element	2049
Boolean attribute	2050
Browser	2051
Cascade	2052
Character reference / entity	2053
Child / parent / sibling / ancestor / descendant	2054

Class	2055
Content model	2056
CSS	2057
Deprecated	2058
DOCTYPE	2059
DOM	2060
Element	2061
Empty / void element	2062
Flow content	2063
Focus	2064
Fragment	2065
Global attribute	2066
HTML	2067
HTTP / HTTPS	2068
Hyperlink	2069
Id	2070
Inline element	2071
JavaScript	2072
Landmark	2073
Living Standard	2074
MDN	2075
Metadata	2076
MIME type	2077
Nesting	2078
Obsolete	2079
Phrasing content	2080

Pseudo-class / pseudo-element	2081
Quirks mode	2082
Rendering	2083
Responsive design	2084
Screen reader	2085
Sectioning content	2086
Selector	2087
Semantic HTML	2088
Specificity	2089
Tag	2090
URL	2091
UTF-8	2092
Validator	2093
Viewport	2094
W3C	2095
WCAG	2096
WHATWG	2097
XHTML	2098

Alphabetical index

2099

How to use this book

This book is a complete desk reference for the HTML language, from the very first version published by Tim Berners-Lee in 1991 up to the HTML Living Standard that browsers implement today. It is written for people who are learning HTML for the first time and for experienced developers who need to look something up quickly.

How the book is organised

- Part 1 - History: what changed in HTML 1, 2, 3.2, 4.01, XHTML and HTML5.
- Part 2 - Fundamentals: syntax, document structure, nesting rules, content categories, comments, entities and the rules that apply to every tag.
- Part 3 - Global attributes: attributes such as id, class, style, title, hidden, lang and data-* that can be placed on any element.
- Part 4 - Event handler attributes: onclick, onload, onchange and every other on* attribute.
- Part 5 - Element reference: one full entry for every HTML element, alphabetically ordered. Each entry contains a description, syntax, content model, a table of every attribute, one or more complete examples, accessibility notes, version history and common mistakes.
- Part 6 - The input element in depth: every value of the type attribute with its own entry.
- Part 7 - Obsolete and deprecated features: elements and attributes from HTML 2, 3.2 and 4 that you may meet in old pages, with the modern replacement for each.

- Part 8 - Attribute reference: every attribute in the language listed alphabetically with the elements that accept it.
- Part 9 - Value references: link relationship types, MIME types, meta names, autocomplete tokens, colour names, URL schemes and the character entity table.
- Part 10 - Practical guides: forms, tables, lists, media, semantic layout, accessibility, SEO.
- Part 11 - Glossary and alphabetical index.

How to find something quickly

- In the HTML edition, type any word into the search box at the top: a tag name, an attribute name, a value, or a plain English word such as 'italic' or 'bottom of page'.
- In the PDF edition, use the table of contents at the front, the alphabetical index at the back, or your PDF reader's search function (Ctrl+F).
- Every element entry has the same layout, so once you know where the attribute table is in one entry you know where it is in all of them.

Conventions used in this book

Notation	Meaning
<tag>	An HTML element written with its tag name, for example <footer>.
attribute	An attribute name, written in lower case, for example href.
attribute="value"	An attribute together with an example value.
Void element	An element that has no closing tag and no content, for example .

Notation	Meaning
Deprecated	Still works in browsers but should not be used in new pages.
Obsolete	Removed from the standard; browsers may still render it for old pages.
HTML 2 / 3.2 / 4.01 / 5	The version of the specification in which a feature first appeared.

All examples in this book are complete and valid. You can copy any of them into a file named `index.html` and open it in a browser.

Part 1

The History of HTML

From Tim Berners-Lee's 18 tags to the Living Standard.

A short history of HTML

HTML (HyperText Markup Language) is the language used to describe the structure of pages on the World Wide Web. It has existed for more than three decades and has gone through several official versions. Knowing the history helps you understand why some tags exist, why others are marked obsolete, and why modern HTML looks the way it does.

HTML 1.0 (1991 - 1993): the beginning

In 1989 Tim Berners-Lee, a scientist at CERN in Switzerland, proposed a system for sharing documents between researchers. In 1991 he published a short document called 'HTML Tags' that described 18 elements. There was no official 'HTML 1.0' specification; the name is used informally for this first generation of the language. In 1993 Berners-Lee and Dan Connolly published an Internet Draft called 'Hypertext Markup Language (HTML)' which is often called HTML 1.0.

The first version was extremely simple. It had no images at first, no tables, no forms and no styling of any kind. Its purpose was to mark headings, paragraphs, lists and, most importantly, hyperlinks.

Element	Purpose in HTML 1
<title>	The document title.
<a>	The anchor: a hyperlink to another document. The most important invention of HTML.
<h1> to <h6>	Six levels of heading.
<p>	A paragraph break (in the earliest version it was a separator, not a container).

Element	Purpose in HTML 1
, ,	Unordered and ordered lists.
<dl>, <dt>, <dd>	Glossary (definition) lists.
<menu>, <dir>	Compact list variants (later removed).
<address>	Author contact information.
<pre>, <listing>, <xmp>, <plaintext>	Pre-formatted and literal text blocks.
<isindex>	A searchable index prompt (later removed).
<nextid>	Used by editors to generate identifiers (later removed).
	Added in 1993 by Marc Andreessen for the Mosaic browser.

HTML 2.0 (1995): the first official standard

HTML 2.0 was published in November 1995 as RFC 1866 by the Internet Engineering Task Force (IETF). It gathered together everything that browsers of the time already supported and made it an official standard. It was written by Tim Berners-Lee and Dan Connolly.

New in HTML 2.0:

- Forms: <form>, <input>, <select>, <option>, <textarea>. For the first time a page could send information back to a server.
- Images with the element and its src, alt and align attributes.
- Character formatting: , <i>, <tt>, , , <code>, <samp>, <kbd>, <var>, <cite>.

- The <head> and <body> structure and <meta>, <base>, <link>, <isindex> in the head.
- The <blockquote>,
, <hr> elements.
- Character entities such as &, <, > and the ISO Latin-1 set (´ and so on).
- The DOCTYPE declaration: <!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">

HTML 2.0 had no tables and no way to control colours or fonts. Pages were plain grey backgrounds with black text and blue links.

HTML 3.0 (1995, never finished) and HTML 3.2 (1997)

HTML 3.0 was an ambitious draft by Dave Raggett that proposed tables, mathematical formulas, figures and text flow around images. It was too large for browsers to implement and was abandoned. Meanwhile Netscape and Microsoft added their own tags (such as , <center>, <blink> and <marquee>), starting the 'browser wars'.

The newly formed World Wide Web Consortium (W3C) responded with HTML 3.2, published in January 1997. Code-named 'Wilbur', it standardised what browsers actually did rather than what was ideal. HTML 3.2 is remembered as the version that mixed presentation into the markup.

New in HTML 3.2:

- Tables: <table>, <tr>, <td>, <th>, <caption>.
- Java applets: <applet>, <param>.

- Presentational elements: `<font>`, `<center>`, `<big>`, `<small>`, `<strike>`, `<u>`, `<sub>`, `<sup>`.
- Client-side image maps: `<map>`, `<area>`.
- `<div>` as a generic block container, with the `align` attribute.
- `<script>` and `<style>` were reserved for future use.
- Presentational attributes everywhere: `bgcolor`, `text`, `link`, `vlink`, `alink` on `<body>`; `width`, `border`, `cellpadding`, `cellspacing` on `<table>`; `align` on almost everything.

HTML 4.0 (1997) and HTML 4.01 (1999): separation of structure and style

HTML 4.0 was published in December 1997 and revised as HTML 4.01 in December 1999. It is the version that introduced the idea that HTML should describe structure and meaning while Cascading Style Sheets (CSS) should describe appearance. Most presentational elements of HTML 3.2 were officially deprecated.

HTML 4.01 came in three variants, chosen with the DOCTYPE:

Variant	DOCTYPE	Meaning
Strict	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd"></code>	No deprecated elements, no frames.

Variant	DOCTYPE	Meaning
Transitional	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"></code>	Deprecated elements allowed, to ease migration.
Frameset	<code><!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN" "http://www.w3.org/TR/html4/frameset.dtd"></code>	Same as Transitional but the body is replaced by a <code><frameset></code> .

New in HTML 4.01:

- Style sheets: the `<style>` element, the style attribute, the class and id attributes on all elements.
- Scripting: the `<script>` and `<noscript>` elements and the `on*` event attributes.
- Frames: `<frameset>`, `<frame>`, `<noframes>` and the inline `<iframe>`.
- Richer tables: `<thead>`, `<tbody>`, `<tfoot>`, `<colgroup>`, `<col>`, plus scope, headers, abbr, axis.
- Richer forms: `<button>`, `<label>`, `<fieldset>`, `<legend>`, `<optgroup>`, accesskey, tabindex, disabled, readonly.
- Internationalisation: lang and dir attributes, `<bdo>`, right-to-left text, Unicode support.
- Accessibility: the title attribute everywhere, alt required on `<img>`, longdesc, `<abbr>`, `<acronym>`.

- Generic <object> element for embedded content.
- New phrase elements: <ins>, , <q>, .

XHTML 1.0 (2000) and XHTML 1.1 (2001)

XHTML was HTML 4.01 rewritten as an XML application. The elements were identical, but the syntax rules of XML applied: every tag must be closed (
 instead of
), all tag and attribute names must be lower case, attribute values must always be quoted, and a single error makes the whole page fail to display. XHTML 2.0 was planned as a completely new language but was abandoned in 2009 because it was not compatible with existing web pages.

HTML5 (2008 - 2014) and the HTML Living Standard (2019 - today)

In 2004 a group of browser makers (Apple, Mozilla and Opera) formed the WHATWG (Web Hypertext Application Technology Working Group) to evolve HTML in a backward-compatible way. Their work became HTML5. The W3C adopted it in 2007 and published HTML5 as a Recommendation in October 2014, followed by HTML 5.1 (2016) and HTML 5.2 (2017).

In 2019 the W3C and WHATWG agreed that there would be only one HTML specification, the WHATWG 'HTML Living Standard', which is updated continuously and has no version number. When people say 'HTML5' today they mean this living standard.

New in HTML5:

- A simple DOCTYPE: <!DOCTYPE html>

- Semantic sectioning elements: `<header>`, `<footer>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<figure>`, `<figcaption>`.
- Native audio and video: `<audio>`, `<video>`, `<source>`, `<track>`.
- Graphics: `<canvas>`, and inline `<svg>` and `<math>`.
- New form controls: input types email, url, tel, number, range, date, time, color, search; the placeholder, required, pattern, autofocus and autocomplete attributes; `<datalist>`, `<output>`, `<progress>`, `<meter>`.
- Interactive elements: `<details>`, `<summary>`, `<dialog>`.
- Text-level elements: `<mark>`, `<time>`, `<data>`, `<wbr>`, `<bdi>`, `<ruby>`, `<rt>`, `<rp>`.
- The `<template>`, `<slot>` and `<picture>` elements; the `srcset` and `sizes` attributes; lazy loading.
- Custom data attributes (`data-*`), `contenteditable`, `draggable`, `hidden`, `spellcheck`, `translate`.
- Removal of presentational elements: `<font>`, `<center>`, `<big>`, `<strike>`, `<tt>`, frames, `<applet>`.
- Precise parsing rules so that every browser builds the same document tree from the same HTML, even when the HTML contains mistakes.

Timeline summary

Year	Version	Published by	Key additions
1991	HTML Tags (HTML 1)	Tim Berners-Lee, CERN	Headings, paragraphs, lists, links

Year	Version	Published by	Key additions
1993	HTML 1.0 draft	IETF draft	 added by Mosaic
1995	HTML 2.0	IETF RFC 1866	Forms, images, character formatting
1997	HTML 3.2	W3C	Tables, applets, , <center>, presentation al attributes
1997	HTML 4.0	W3C	CSS, scripting, frames, richer tables and forms
1999	HTML 4.01	W3C	Corrections to 4.0; Strict, Transitional, Frameset
2000	XHTML 1.0	W3C	HTML 4.01 in XML syntax
2008	HTML5 first draft	WHATWG / W3C	Semantic elements, audio, video, canvas

Year	Version	Published by	Key additions
2014	HTML5 Recommendation	W3C	Official HTML5
2016 / 2017	HTML 5.1 / 5.2	W3C	<picture>, <dialog>, minor additions
2019 - today	HTML Living Standard	WHATWG	Single continuously updated specification

HTML Fundamentals

Syntax, structure, nesting rules, content categories and entities.

HTML fundamentals

What HTML is

HTML is a markup language, not a programming language. It cannot calculate, make decisions or loop. Its only job is to wrap pieces of content in tags that say what each piece is: 'this is a heading', 'this is a paragraph', 'this is a link to another page', 'this is the footer'. The browser reads these tags, builds a tree of objects called the DOM (Document Object Model), and draws the page. CSS (Cascading Style Sheets) then controls how it looks and JavaScript controls how it behaves.

Anatomy of an element

```
<p class="intro">Hello, world!</p>
```

				--- closing tag
				+----- content
				+----- end of opening tag
				+----- attribute name="value"
				+----- opening tag with tag name

- The opening tag starts with < followed by the tag name and ends with >.
- Attributes go inside the opening tag, after the tag name, separated by spaces. Each attribute has a name and usually a value in quotes: name="value".
- The content is everything between the opening and closing tag. It can be text, other elements or both.
- The closing tag is the same as the opening tag but with a slash after the <: </p>.
- The whole thing, from < to >, is called an element.

Void elements (tags with no closing tag)

A small group of elements never have content and therefore never have a closing tag. Writing a closing tag for them is an error. In HTML5 you may write them as `<br>` or, in XML style, as `<br />`; both are accepted.

Void element	Purpose
<code><area></code>	Clickable region inside an image map
<code><base></code>	Base URL for the document
<code>
</code>	Line break
<code><col></code>	Table column properties
<code><embed></code>	External application content
<code><hr></code>	Thematic break (horizontal rule)
<code></code>	Image
<code><input></code>	Form control
<code><link></code>	External resource link (usually CSS)
<code><meta></code>	Metadata
<code><source></code>	Media or image source
<code><track></code>	Text track for media
<code><wbr></code>	Word break opportunity

The structure of a document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title shown in the browser tab</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
```

```

    <h1>Site name</h1>
    <nav><a href="/">Home</a> <a href="/about">About</a></nav>
</header>
<main>
    <h2>Page heading</h2>
    <p>Page content.</p>
</main>
<footer>
    <p>&copy; 2026 Site name</p>
</footer>
<script src="app.js"></script>
</body>
</html>

```

Line	What it does
<!DOCTYPE html>	Tells the browser to use the modern standards rendering mode. Always the first line. Not a tag; it has no closing tag.
<html lang="en">	The root element. Everything else goes inside. lang states the language of the page.
<head>	Information about the page that is not displayed: character set, title, links to CSS, metadata.
<meta charset="utf-8">	Declares the character encoding. Must come within the first 1024 bytes. UTF-8 supports every language.
<meta name="viewport" ...>	Makes the page scale correctly on phones.
<title>	Required. Shown in the browser tab, bookmarks and search results.
<body>	Everything the visitor sees.
<script>	JavaScript. Usually placed at the end of the body or given the defer attribute so it does not block rendering.

Attributes

Attributes give extra information about an element. They are written in the opening tag only. Rules:

- Names are case-insensitive but by convention are written in lower case.
- Values should be quoted with double quotes. Single quotes are also allowed. Quotes may be omitted only if the value contains no spaces or special characters, but it is safer to always quote.
- A boolean attribute is either present or absent; its value does not matter. `<input disabled>`, `<input disabled="">` and `<input disabled="disabled">` all mean the same thing. Writing `disabled="false"` still means disabled! To turn it off, remove the attribute.
- An attribute may appear only once on an element.
- Enumerated attributes accept a fixed list of keywords, for example `dir="ltr"` or `dir="rtl"`.

```
<a href="https://example.com" target="_blank" rel="noopener">Visit</a>
<input type="checkbox" checked disabled>
<div id="main" class="container wide" data-user-id="42"></div>
```

Nesting

Elements must be properly nested: the element opened last must be closed first. `<p><em>right</em></p>` is correct; `<p><em>wrong</p></em>` is an error.

Not every element may go inside every other. The rules are described by content categories (see below) and are listed in the 'Permitted parents' and 'Permitted content' rows of each element entry. The most common mistakes:

- A `<p>` cannot contain block-level elements such as `<div>`, `<ul>`, `<table>` or another `<p>`. The browser will silently

close the paragraph.

- An `<a>` cannot contain another `<a>` or any interactive element such as `<button>`.
- A `<button>` cannot contain interactive content.
- `<li>` must be a direct child of `<ul>`, `<ol>` or `<menu>`. `<tr>` must be inside `<table>`, `<thead>`, `<tbody>` or `<tfoot>`. `<td>` and `<th>` must be inside `<tr>`. `<option>` must be inside `<select>`, `<optgroup>` or `<datalist>`.
- `<header>` and `<footer>` cannot contain another `<header>` or `<footer>`.
- Heading elements must not be placed inside another heading.

Content categories

HTML5 groups elements into categories that define where they may appear. An element may belong to several categories.

Category	Meaning	Examples
Metadata content	Sets up the presentation or behaviour of the rest of the page; lives in <code><head></code> .	<code><base></code> , <code><link></code> , <code><meta></code> , <code><noscript></code> , <code><script></code> , <code><style></code> , <code><title></code>
Flow content	Almost everything that may appear in <code><body></code> .	<code><p></code> , <code><div></code> , <code><h1></code> , <code></code> , <code><table></code> , <code><a></code> , <code></code> , plain text ...
Sectioning content	Defines the scope of headings and footers.	<code><article></code> , <code><aside></code> , <code><nav></code> , <code><section></code>
Heading content	Section headings.	<code><h1></code> to <code><h6></code> , <code><hgroup></code>

Category	Meaning	Examples
Phrasing content	Text and the elements that mark up text within a paragraph.	<code><a></code> , <code></code> , <code></code> , <code></code> , <code></code> , <code><input></code> , <code><code></code> , plain text ...
Embedded content	Imports another resource into the page.	<code><audio></code> , <code><canvas></code> , <code><embed></code> , <code><iframe></code> , <code></code> , <code><math></code> , <code><object></code> , <code><picture></code> , <code><svg></code> , <code><video></code>
Interactive content	Designed for user interaction.	<code><a href></code> , <code><button></code> , <code><details></code> , <code><input></code> , <code><label></code> , <code><select></code> , <code><textarea></code> , <code><video controls></code>
Palpable content	Content that is neither empty nor hidden, so that an element has something visible.	Most flow and phrasing content
Form-associated content	Elements that have a form owner.	<code><button></code> , <code><fieldset></code> , <code><input></code> , <code><label></code> , <code><meter></code> , <code><object></code> , <code><output></code> , <code><select></code> , <code><textarea></code>

Block-level and inline elements

Before HTML5 elements were described as either block-level or inline. The terms are still very useful because they match the default CSS display of each element.

	Block-level (display: block)	Inline (display: inline)
Starts on a new line?	Yes	No; flows with the text
Takes full width?	Yes, of its container	Only as wide as its content

	Block-level (display: block)	Inline (display: inline)
Can set width/height?	Yes	No (unless display is changed)
Can contain?	Other blocks and inline elements	Only other inline elements and text
Examples	<code><div>, <p>, <h1>-<h6>, , , , <table>, <form>, <header>, <footer>, <section>, <article>, <nav>, <main>, <aside>, <blockquote>, <pre>, <hr>, <figure></code>	<code>, <a>, , , , <i>, , <code>,
, <input>, <label>, <button>, <select>, <textarea>, <abbr>, <cite>, <q>, <small>, <sub>, <sup>, <time>, <mark></code>

Comments

```
<!-- This is a comment. The browser ignores it. -->
<!--
  Comments can span
  several lines.
-->
```

Comments may not contain the sequence `--` inside them and may not be nested. Remember that visitors can read comments by viewing the page source, so never put passwords or private information in them.

Character references (entities)

Some characters have special meaning in HTML and must be written as character references when you want them to

appear as text. Every reference starts with & and ends with ;.

Character	Named reference	Numeric reference	Why it is needed
<	<	<	Starts a tag
>	>	>	Ends a tag
&	&	&	Starts a reference
"	"	"	Delimits attribute values
'	'	'	Delimits attribute values
non-breaking space	 	 	A space that will not wrap or collapse
(c)	©	©	Convenience
EUR	€	€	Convenience

Numeric references may be decimal (€) or hexadecimal (€). With UTF-8 encoding you can type most characters directly and only <, > and & really need references. The complete entity table is in Part 9.

Whitespace

In normal text the browser collapses any run of spaces, tabs and line breaks into a single space, and removes spaces at the start and end of a block. To preserve whitespace exactly, use the <pre> element or the CSS property white-space: pre. To

force a line break use `<br>`. To insert a space that cannot collapse use ` `;

Case sensitivity and quoting

- Tag and attribute names are not case-sensitive in HTML (`<DIV>` equals `<div>`), but lower case is the universal convention and is required in XHTML.
- Attribute values may be case-sensitive depending on the attribute: `id` and `class` values are case-sensitive; enumerated keywords such as `type="TEXT"` are not.
- Always quote attribute values. It avoids a whole category of bugs.

Character encoding

Always save your files as UTF-8 and declare it with `<meta charset="utf-8">` as the first thing inside `<head>`. UTF-8 can represent every character of every language, including Arabic, and is the only encoding recommended by the HTML standard. If the encoding is wrong you will see question marks or strange symbols instead of your text.

Right-to-left languages

For Arabic, Hebrew, Persian and Urdu pages set `dir="rtl"` and the correct `lang` on the `<html>` element. The browser then aligns text to the right, mirrors lists and tables and handles mixed left-to-right words correctly.

```
<html lang="ar" dir="rtl">
```

Validation

A validator is a tool that checks your HTML for errors. Browsers are forgiving and will display broken HTML, but the result may differ between browsers and may be inaccessible. Use the free W3C validator at <https://validator.w3.org/> or the Nu HTML Checker at <https://validator.nu/>.

The ten most common beginner mistakes

- Forgetting `<!DOCTYPE html>`, which puts the browser in 'quirks mode' and changes layout behaviour.
- Forgetting `<meta charset="utf-8">`, which breaks non-English characters.
- Forgetting to close tags, or closing them in the wrong order.
- Putting block elements inside `<p>`.
- Using `<br>` repeatedly to create space instead of CSS margins.
- Using `<table>` for page layout instead of CSS.
- Using `<b>` and `<i>` for emphasis where `<strong>` and `<em>` carry the meaning.
- Omitting the alt attribute on images.
- Using `<div>` for everything where `<header>`, `<nav>`, `<main>`, `<footer>` describe the content better.
- Skipping heading levels (jumping from `<h1>` to `<h4>`).

HTML Version by Version

Which elements each version of HTML introduced, from HTML 1 to the Living Standard.

How to read this part

Each chapter lists the elements whose history mentions that version of HTML, with a one-paragraph summary. An element can appear in more than one chapter (for example when HTML 4 changed it and HTML5 changed it again). Elements marked OBSOLETE are documented for reading old pages only; use the modern replacement named in the element's full entry in Part 6.

HTML 1.0 (1991-1993)

The first HTML, described in Tim Berners-Lee's 'HTML Tags' document (1991) and the 1993 IETF draft by Berners-Lee and Dan Connolly. It had about 20 elements: headings, paragraphs, anchors, lists, addresses, preformatted text, plus a few editor hints. There were no images (the `<img>` tag was added by Mosaic in 1993), no tables and no forms.

Elements in this chapter: 22

Element	Meaning	Status today
<code><a></code>	Anchor (hyperlink)	current
<code><address></code>	Contact address	current
<code>
</code>	Line break	current
<code><dd></code>	Description details	current
<code><dir></code>	Directory list (obsolete)	obsolete
<code><dl></code>	Description list	current
<code><dt></code>	Description term	current
<code><h1></code>	Headings h1 to h6	current
<code><isindex></code>	Search index prompt (obsolete)	obsolete
<code></code>	List item	current
<code><listing></code>	Listing (obsolete)	obsolete
<code><menu></code>	Menu (toolbar list)	current
<code><nextid></code>	Next identifier (HTML 1 editor hint)	obsolete
<code></code>	Ordered list	current
<code><p></code>	Paragraph	current

Element	Meaning	Status today
<code><plaintext></code>	Plain text (obsolete)	obsolete
<code><pre></code>	Pre-formatted text	current
<code><rb></code>	Ruby base (deprecated)	deprecated
<code><ruby></code>	Ruby annotation	current
<code><title></code>	Document title	current
<code></code>	Unordered list	current
<code><xmp></code>	Example text (obsolete)	obsolete

<a> — Anchor (hyperlink)

Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<a>` element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and `<a>` was one of the very first elements created by Tim Berners-Lee in 1991.

```
<a href="URL">link text</a>
```

<address> — Contact address

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

```
<address>contact information</address>
```

**
 — Line break**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

```
<br>
```

<dd> — Description details

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dd>` element provides the description, definition or value for the preceding term (`<dt>`) in a description list (`<dl>`). One term may have several `<dd>` elements, and several terms may share one `<dd>`. Browsers indent `<dd>` from the left by default.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dir> — Directory list (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

```
<dir><li>item</li></dir>
```

<dl> — Description list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`).

Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dt> — Description term

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

```
<dt>Term</dt>
```

<h1> — Headings h1 to h6

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<h1>` to `<h6>` elements represent six levels of section headings. `<h1>` is the highest (most important) level and `<h6>` the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```

<isindex> — Search index prompt (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<isindex>` element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a `<form>` with an `<input type="search">`.

```
<isindex prompt="Search: ">
```

** — List item**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

```
<ul>
  <li>Item</li>
</ul>
```

<listing> — Listing (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The `<listing>` element, from the 1991 tag set, displayed its content in a monospace font exactly as written, like `<pre>` but originally at a smaller size and without interpreting markup. It was deprecated already in HTML 2.0 and removed in HTML 3.2. Use `<pre>`, optionally with `<code>`.

```
<listing>text</listing>
```

<menu> — Menu (toolbar list)

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<menu>` element is a semantic alternative to `<ul>` for a list of commands or actions, such as a toolbar of buttons. Each command is an `<li>` containing a `<button>` or similar control. Browsers render it identically to `<ul>`.

```
<menu>
  <li><button>Action</button></li>
</menu>
```

<nextid> — Next identifier (HTML 1 editor hint)

Status: OBSOLETE | Versions: HTML 1 / HTML 2.0 (RFC 1866); removed in HTML 3.2

The `<nextid>` element was used by the very first HTML editors (the NeXT-based editor written at CERN) to remember the next automatically generated anchor name, so that the editor could continue numbering anchors (z1, z2, z3...) when a document was reopened. It carried a single attribute, `n`, holding that name. It had no visible effect and was placed in the `<head>`.

```
<nextid n="z12">
```

** — Ordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

<p> — Paragraph

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <p> element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where <p> was a separator between paragraphs rather than a container.

```
<p>Text of the paragraph.</p>
```

<plaintext> — Plain text (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The <plaintext> element told the browser to treat everything after it as plain text, without interpreting any further tags, until the end of the file. It could not be closed. It was one of the 1991 elements, was deprecated in HTML 2.0 and is obsolete. Use <pre> for pre-formatted text, escaping < and & as entities, or serve the file as text/plain.

```
<plaintext>
```

<pre> — Pre-formatted text

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <pre> element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

```
<pre>
  text with
    preserved   spacing
</pre>
```

<rb> — Ruby base (deprecated)

Status: DEPRECATED | Versions: XHTML 1.1 Ruby, HTML 5.1; removed from the Living Standard

The `<rb>` element explicitly marked the base text component of a ruby annotation, the text that the `<rt>` annotation is placed above. The Living Standard removed it because the base text can simply be written directly inside `<ruby>`. Browsers still support it.

```
<ruby><rb>□</rb><rt>kan</rt></ruby>
```

<ruby> — Ruby annotation

Status: CURRENT | Versions: XHTML 1.1 (2001), HTML5

The `<ruby>` element represents small annotations rendered above, below or beside base text, most often used to show the pronunciation of East Asian characters. The name comes from a small typographic size called 'ruby' used by British printers. The base text is written directly inside `<ruby>`, each annotation in an `<rt>`, and optional fallback parentheses in `<rp>`.

```
<ruby>base<rp>(</rp><rt>annotation</rt><rp>)</rp></ruby>
```

<title> — Document title

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<title>` element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one `<title>`, inside `<head>`.

```
<title>Page title</title>
```

** — Unordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ul>` element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an `<li>`. If the order matters, use `<ol>`.

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

<xmp> — Example text (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The `<xmp>` element (example) displayed its content literally, without interpreting any markup, in a monospace font, so that authors could show HTML source without escaping `<` and `&`. It was one of the 1991 elements, deprecated in HTML 2.0 and removed in HTML 3.2, although browsers still support it for old pages. Use `<pre>` with escaped entities.

```
<xmp><p>shown literally</p></xmp>
```


HTML 2.0 (RFC 1866, 1995)

HTML 2.0 was the first official standard, published by the IETF in November 1995. It documented what browsers already supported: images, forms (input, select, textarea), the `<head>/<body>` split and character entities for Latin-1.

Elements in this chapter: 44

Element	Meaning	Status today
<code><a></code>	Anchor (hyperlink)	current
<code><address></code>	Contact address	current
<code></code>	Bring attention to (bold)	current
<code><base></code>	Document base URL	current
<code><blockquote></code>	Block quotation	current
<code><body></code>	Document body	current
<code>
</code>	Line break	current
<code><cite></code>	Citation (title of a work)	current
<code><code></code>	Inline code	current
<code><dd></code>	Description details	current
<code><dir></code>	Directory list (obsolete)	obsolete
<code><dl></code>	Description list	current
<code><dt></code>	Description term	current
<code></code>	Emphasis	current
<code><form></code>	Form	current
<code><h1></code>	Headings h1 to h6	current

Element	Meaning	Status today
<head>	Document head	current
<hr>	Thematic break (horizontal rule)	current
<html>	Document root	current
<i>	Idiomatic text (italic)	current
	Image	current
<input>	Input control	current
<isindex>	Search index prompt (obsolete)	obsolete
<kbd>	Keyboard input	current
	List item	current
<link>	External resource link	current
<listing>	Listing (obsolete)	obsolete
<menu>	Menu (toolbar list)	current
<meta>	Metadata	current
<nextid>	Next identifier (HTML 1 editor hint)	obsolete
	Ordered list	current
<option>	Option	current
<p>	Paragraph	current
<plaintext>	Plain text (obsolete)	obsolete
<pre>	Pre-formatted text	current
<samp>	Sample output	current

Element	Meaning	Status today
<select>	Drop-down / list box	current
	Strong importance	current
<textarea>	Multi-line text input	current
<title>	Document title	current
<tt>	Teletype text (obsolete)	obsolete
	Unordered list	current
<var>	Variable	current
<xmp>	Example text (obsolete)	obsolete

<a> — Anchor (hyperlink)

Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <a> element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and <a> was one of the very first elements created by Tim Berners-Lee in 1991.

```
<a href="URL">link text</a>
```

<address> — Contact address

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

```
<address>contact information</address>
```

** — Bring attention to (bold)**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

```
<b>text</b>
```

<base> — Document base URL

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<base>` element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one `<base>` element per document and it must be placed in `<head>`, before any element that uses a URL.

```
<base href="URL" target="_blank">
```

<blockquote> — Block quotation

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<blockquote>` element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional `cite` attribute holds the URL of the source; a visible citation should be given with the `<cite>` element, usually in a `<footer>` inside the `blockquote` or in a following paragraph.

```
<blockquote cite="URL">quoted text</blockquote>
```

<body> — Document body

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<body>` element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one `<body>` in a document and it is the second child of `<html>`, after `<head>`.

```
<body>...</body>
```

**
 — Line break**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

```
<br>
```

<cite> — Citation (title of a work)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<cite>` element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

```
<cite>Title of Work</cite>
```

<code> — Inline code

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

```
<code>snippet</code>
```

<dd> — Description details

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dd>` element provides the description, definition or value for the preceding term (`<dt>`) in a description list (`<dl>`). One term may have several `<dd>` elements, and several terms may share one `<dd>`. Browsers indent `<dd>` from the left by default.

```
<dl>  
  <dt>Term</dt>  
  <dd>Description</dd>  
</dl>
```

<dir> — Directory list (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

```
<dir><li>item</li></dir>
```

<dl> — Description list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`). Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dt> — Description term

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

```
<dt>Term</dt>
```

** — Emphasis**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

```
<em>text</em>
```

<form> — Form

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<form>` element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the action attribute using the HTTP method in the method attribute.

```
<form action="URL" method="post">
  ...controls...
  <button>Submit</button>
</form>
```

<h1> — Headings h1 to h6

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<h1>` to `<h6>` elements represent six levels of section headings. `<h1>` is the highest (most important) level and `<h6>` the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```


<head> — Document head

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<head>` element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and `<meta>` tags for search engines and social networks. Nothing inside `<head>` is displayed on the page itself, except the title, which appears in the browser tab.

```
<head>
  <meta charset="utf-8">
  <title>Title</title>
</head>
```

<hr> — Thematic break (horizontal rule)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<hr>` element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

```
<hr>
```

<html> — Document root

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<html>` element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: `<head>` followed by `<body>`. It comes immediately after the `<!DOCTYPE html>` declaration.

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

<i> — Idiomatic text (italic)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<i>` element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

```
<i>text</i>
```

** — Image**

Status: CURRENT | Versions: Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<img>` element embeds an image in the document. It is a void element; the image file is identified by the `src` attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The `alt` attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

```

```

<input> — Input control

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<input>` element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the `type` attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

```
<input type="text" name="field" id="field">
```

<isindex> — Search index prompt (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<isindex>` element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a `<form>` with an `<input type="search">`.

```
<isindex prompt="Search: ">
```

<kbd> — Keyboard input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<kbd>` element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting `<kbd>` elements inside a `<kbd>` represents individual keys in a key combination; wrapping `<kbd>` in `<samp>` represents input echoed by the system.

```
<kbd>Ctrl</kbd> + <kbd>S</kbd>
```

** — List item**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

```
<ul>
  <li>Item</li>
</ul>
```

<link> — External resource link

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <link> element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (rel="stylesheet"), but it is also used for site icons (rel="icon"), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in <head>, though body-ok link types such as stylesheet may appear in <body>.

```
<link rel="stylesheet" href="style.css">
```

<listing> — Listing (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The <listing> element, from the 1991 tag set, displayed its content in a monospace font exactly as written, like <pre> but originally at a smaller size and without interpreting markup. It was deprecated already in HTML 2.0 and removed in HTML 3.2. Use <pre>, optionally with <code>.

```
<listing>text</listing>
```

<menu> — Menu (toolbar list)

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The <menu> element is a semantic alternative to for a list of commands or actions, such as a toolbar of buttons. Each command is an containing a <button> or similar control. Browsers render it identically to .

```
<menu>
  <li><button>Action</button></li>
</menu>
```

<meta> — Metadata

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<meta>` element represents metadata about the document that cannot be expressed by `<title>`, `<base>`, `<link>`, `<style>` or `<script>`. It is a void element placed in `<head>`. Depending on which attributes are present it does one of four things: declares the character encoding (charset); provides document-level metadata as a name/content pair (name); simulates an HTTP response header (http-equiv); or supplies microdata (itemprop).

```
<meta charset="utf-8">
<meta name="description" content="...">
```

<nextid> — Next identifier (HTML 1 editor hint)

Status: OBSOLETE | Versions: HTML 1 / HTML 2.0 (RFC 1866); removed in HTML 3.2

The `<nextid>` element was used by the very first HTML editors (the NeXT-based editor written at CERN) to remember the next automatically generated anchor name, so that the editor could continue numbering anchors (z1, z2, z3...) when a document was reopened. It carried a single attribute, `n`, holding that name. It had no visible effect and was placed in the `<head>`.

```
<nextid n="z12">
```

** — Ordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

<option> — Option

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<option>` element defines one item in a `<select>` drop-down list, an `<optgroup>`, or a `<datalist>` of suggestions. Its text content is what the user sees; the `value` attribute is what is submitted with the form. If `value` is omitted the text content is submitted instead.

```
<option value="v">Text</option>
```

<p> — Paragraph

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

```
<p>Text of the paragraph.</p>
```

<plaintext> — Plain text (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The `<plaintext>` element told the browser to treat everything after it as plain text, without interpreting any further tags, until the end of the file. It could not be closed. It was one of the 1991 elements, was deprecated in HTML 2.0 and is obsolete. Use `<pre>` for pre-formatted text, escaping `<` and `&` as entities, or serve the file as `text/plain`.

```
<plaintext>
```

<pre> — Pre-formatted text

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<pre>` element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

```
<pre>
  text with
    preserved   spacing
</pre>
```

<samp> — Sample output

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

```
<samp>program output</samp>
```

<select> — Drop-down / list box

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<select>` element creates a control for choosing one option (or, with multiple, several) from a list. It contains `<option>` elements, optionally grouped in `<optgroup>`. By default it appears as a drop-down menu; with the `multiple` attribute or a size greater than 1 it appears as a scrolling list box. The name attribute and the value of the chosen option(s) are submitted with the form.

```
<select name="n">
  <option value="1">One</option>
</select>
```

** — Strong importance**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

```
<strong>important text</strong>
```

<textarea> — Multi-line text input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<textarea>` element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike `<input>`, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

```
<textarea name="n" rows="4" cols="40">initial text</textarea>
```


<title> — Document title

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <title> element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one <title>, inside <head>.

```
<title>Page title</title>
```

<tt> — Teletype text (obsolete)

Status: OBSOLETE | Versions: HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5

The <tt> element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements <code>, <kbd>, <samp> or <var>, or CSS font-family: monospace on a .

```
<tt>text</tt>
```

 — Unordered list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an . If the order matters, use .

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

<var> — Variable

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<var>` element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with `<code>`, `<kbd>` and `<samp>`.

```
<var>x</var>
```

<xmp> — Example text (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

The `<xmp>` element (example) displayed its content literally, without interpreting any markup, in a monospace font, so that authors could show HTML source without escaping `<` and `&`. It was one of the 1991 elements, deprecated in HTML 2.0 and removed in HTML 3.2, although browsers still support it for old pages. Use `<pre>` with escaped entities.

```
<xmp><p>shown literally</p></xmp>
```

HTML 3.2 (W3C, January 1997)

HTML 3.2 (code name 'Wilbur') was the first W3C Recommendation. It absorbed the popular Netscape and Internet Explorer extensions: tables, applets, text flow around images, subscripts and superscripts, and the presentational `<font>`, `<center>` and `<u>` elements together with attributes such as `bgcolor` and `align`. Most of that presentational markup is obsolete today, replaced by CSS.

Elements in this chapter: 64

Element	Meaning	Status today
<code><a></code>	Anchor (hyperlink)	current
<code><address></code>	Contact address	current
<code><applet></code>	Java applet (obsolete)	obsolete
<code><area></code>	Image map area	current
<code></code>	Bring attention to (bold)	current
<code><base></code>	Document base URL	current
<code><basefont></code>	Base font (obsolete)	obsolete
<code><big></code>	Bigger text (obsolete)	obsolete
<code><blockquote></code>	Block quotation	current
<code><body></code>	Document body	current
<code>
</code>	Line break	current
<code><caption></code>	Table caption	current
<code><center></code>	Centred block (obsolete)	obsolete

Element	Meaning	Status today
<cite>	Citation (title of a work)	current
<code>	Inline code	current
<dd>	Description details	current
<dfn>	Definition term	current
<dir>	Directory list (obsolete)	obsolete
<div>	Generic block container	current
<dl>	Description list	current
<dt>	Description term	current
	Emphasis	current
	Font (obsolete)	obsolete
<form>	Form	current
<h1>	Headings h1 to h6	current
<head>	Document head	current
<hr>	Thematic break (horizontal rule)	current
<html>	Document root	current
<i>	Idiomatic text (italic)	current
	Image	current
<input>	Input control	current
<isindex>	Search index prompt (obsolete)	obsolete
<kbd>	Keyboard input	current
	List item	current

Element	Meaning	Status today
<link>	External resource link	current
<map>	Image map	current
<menu>	Menu (toolbar list)	current
<meta>	Metadata	current
<nextid>	Next identifier (HTML 1 editor hint)	obsolete
	Ordered list	current
<option>	Option	current
<p>	Paragraph	current
<param>	Object parameter (deprecated)	deprecated
<pre>	Pre-formatted text	current
<s>	Strikethrough (no longer accurate)	current
<samp>	Sample output	current
<script>	Script	current
<select>	Drop-down / list box	current
<small>	Side comment (fine print)	current
<strike>	Strikethrough (obsolete)	obsolete
	Strong importance	current
<style>	Embedded style sheet	current
<sub>	Subscript	current

Element	Meaning	Status today
<sup>	Superscript	current
<table>	Table	current
<td>	Table data cell	current
<textarea>	Multi-line text input	current
<th>	Table header cell	current
<title>	Document title	current
<tr>	Table row	current
<tt>	Teletype text (obsolete)	obsolete
<u>	Unarticulated annotation (underline)	current
	Unordered list	current
<var>	Variable	current

<a> — Anchor (hyperlink)

Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <a> element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and <a> was one of the very first elements created by Tim Berners-Lee in 1991.

```
<a href="URL">link text</a>
```

<address> — Contact address

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

```
<address>contact information</address>
```

<applet> — Java applet (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<applet>` element embedded a Java applet in the page. Applets were small Java programs that ran inside the browser with the help of a plug-in. Java applets were popular in the late 1990s but were slow, insecure and not supported on mobile devices. HTML 4.01 deprecated `<applet>` in favour of `<object>`, and HTML5 removed it entirely. No modern browser runs applets.

```
<applet code="MyApplet.class" width="300" height="200"></applet>
```

<area> — Image map area

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<area>` element defines a clickable region inside an image map. An image map is an image (`<img usemap="#name">`) linked to a `<map name="name">` element that contains one or more `<area>` elements. Each area has a shape and coordinates, and behaves like a hyperlink.

```
<area shape="rect" coords="x1,y1,x2,y2" href="URL" alt="text">
```

** — Bring attention to (bold)**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

```
<b>text</b>
```

<base> — Document base URL

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<base>` element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one `<base>` element per document and it must be placed in `<head>`, before any element that uses a URL.

```
<base href="URL" target="_blank">
```

<basefont> — Base font (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<basefont>` element set the default font size, colour and face for the whole document. It was a presentational element from HTML 3.2 and was deprecated in HTML 4.01 in favour of CSS. HTML5 removed it. Use CSS on the body element instead.

```
<basefont size="3" color="black" face="Arial">
```


<big> — Bigger text (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01; obsolete in HTML5

The `<big>` element rendered its content one font size larger than the surrounding text. It was purely presentational and was removed in HTML5. Its counterpart `<small>` survived because it was given a new meaning (side comments and fine print). Use CSS font-size instead of `<big>`.

```
<big>text</big>
```

<blockquote> — Block quotation

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<blockquote>` element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional `cite` attribute holds the URL of the source; a visible citation should be given with the `<cite>` element, usually in a `<footer>` inside the `blockquote` or in a following paragraph.

```
<blockquote cite="URL">quoted text</blockquote>
```

<body> — Document body

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<body>` element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one `<body>` in a document and it is the second child of `<html>`, after `<head>`.

```
<body>...</body>
```

**
 — Line break**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

```
<br>
```

<caption> — Table caption

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<caption>` element gives a table a title or explanation. It must be the first child of the `<table>` element, and a table may have only one. By default it is displayed centred above the table; the CSS `caption-side` property can move it below.

```
<table>
  <caption>Title</caption>
  ...
</table>
```

<center> — Centred block (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<center>` element horizontally centred its content. It was introduced by Netscape, standardised in HTML 3.2, deprecated in HTML 4.01 and removed in HTML5. It is equivalent to `<div align="center">`. Use CSS instead: `text-align: center` for inline content, or `margin: 0 auto` with a width for blocks.

```
<center>content</center>
```

<cite> — Citation (title of a work)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<cite>` element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

```
<cite>Title of Work</cite>
```

<code> — Inline code

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

```
<code>snippet</code>
```

<dd> — Description details

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dd>` element provides the description, definition or value for the preceding term (`<dt>`) in a description list (`<dl>`). One term may have several `<dd>` elements, and several terms may share one `<dd>`. Browsers indent `<dd>` from the left by default.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dfn> — Definition term

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<dfn>` element marks the term that is being defined in the surrounding sentence or paragraph. The nearest paragraph, description list group or section is the definition. Browsers render it in italics. If the element has a `title` attribute, that is the term being defined; if it contains an `<abbr>` with a `title`, the abbreviation's expansion is the term; otherwise the text content is the term.

```
<dfn>term</dfn>
```

<dir> — Directory list (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

```
<dir><li>item</li></dir>
```

<div> — Generic block container

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<div>` element (division) is a generic container for flow content. It has no meaning of its own and no effect on the content or layout until it is styled with CSS (using `class` or `id`) or manipulated with JavaScript. Use it to group elements for styling purposes, for layout (flexbox, grid), or when no other element is more appropriate.

```
<div class="name">...</div>
```

<dl> — Description list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`).

Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dt> — Description term

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

```
<dt>Term</dt>
```

** — Emphasis**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

```
<em>text</em>
```

** — Font (obsolete)**

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<font>` element set the size, colour and typeface of its text. It was the main way to style text before CSS, was deprecated in HTML 4.01 and removed in HTML5. Browsers still render it for old pages. Every use of `<font>` can be replaced by CSS: `font-size`, `color` and `font-family`.

```
<font size="4" color="red" face="Arial">text</font>
```

<form> — Form

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<form>` element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the `action` attribute using the HTTP method in the `method` attribute.

```
<form action="URL" method="post">
  ...controls...
  <button>Submit</button>
</form>
```

<h1> — Headings h1 to h6

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<h1>` to `<h6>` elements represent six levels of section headings. `<h1>` is the highest (most important) level and `<h6>` the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```

<head> — Document head

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<head>` element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and `<meta>` tags for search engines and social networks. Nothing inside `<head>` is displayed on the page itself, except the title, which appears in the browser tab.

```
<head>
  <meta charset="utf-8">
  <title>Title</title>
</head>
```

<hr> — Thematic break (horizontal rule)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<hr>` element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

```
<hr>
```

<html> — Document root

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <html> element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: <head> followed by <body>. It comes immediately after the <!DOCTYPE html> declaration.

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

<i> — Idiomatic text (italic)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <i> element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

```
<i>text</i>
```

 — Image

Status: CURRENT | Versions: Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The element embeds an image in the document. It is a void element; the image file is identified by the src attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The alt attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

```

```


<input> — Input control

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<input>` element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the `type` attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

```
<input type="text" name="field" id="field">
```

<isindex> — Search index prompt (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<isindex>` element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with `?keywords` in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a `<form>` with an `<input type="search">`.

```
<isindex prompt="Search: ">
```

<kbd> — Keyboard input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<kbd>` element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting `<kbd>` elements inside a `<kbd>` represents individual keys in a key combination; wrapping `<kbd>` in `<samp>` represents input echoed by the system.

```
<kbd>Ctrl</kbd> + <kbd>S</kbd>
```

** — List item**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

```
<ul>
  <li>Item</li>
</ul>
```

<link> — External resource link

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<link>` element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (`rel="stylesheet"`), but it is also used for site icons (`rel="icon"`), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in `<head>`, though body-ok link types such as `stylesheet` may appear in `<body>`.

```
<link rel="stylesheet" href="style.css">
```

<map> — Image map

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<map>` element, together with `<area>` elements, defines a client-side image map: an image with several clickable regions each linking to a different destination. The map is connected to an `<img>` through the `img`'s `usemap` attribute, which must be `#` followed by the map's name.

```
<map name="n">
  <area ...>
</map>
<img usemap="#n" ...>
```

<menu> — Menu (toolbar list)

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<menu>` element is a semantic alternative to `<ul>` for a list of commands or actions, such as a toolbar of buttons. Each command is an `<li>` containing a `<button>` or similar control. Browsers render it identically to `<ul>`.

```
<menu>
  <li><button>Action</button></li>
</menu>
```

<meta> — Metadata

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<meta>` element represents metadata about the document that cannot be expressed by `<title>`, `<base>`, `<link>`, `<style>` or `<script>`. It is a void element placed in `<head>`. Depending on which attributes are present it does one of four things: declares the character encoding (charset); provides document-level metadata as a name/content pair (name); simulates an HTTP response header (http-equiv); or supplies microdata (itemprop).

```
<meta charset="utf-8">
<meta name="description" content="...">
```

<nextid> — Next identifier (HTML 1 editor hint)

Status: OBSOLETE | Versions: HTML 1 / HTML 2.0 (RFC 1866); removed in HTML 3.2

The `<nextid>` element was used by the very first HTML editors (the NeXT-based editor written at CERN) to remember the next automatically generated anchor name, so that the editor could continue numbering anchors (z1, z2, z3...) when a document was reopened. It carried a single attribute, `n`, holding that name. It had no visible effect and was placed in the `<head>`.

```
<nextid n="z12">
```

** — Ordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

<option> — Option

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<option>` element defines one item in a `<select>` drop-down list, an `<optgroup>`, or a `<datalist>` of suggestions. Its text content is what the user sees; the `value` attribute is what is submitted with the form. If `value` is omitted the text content is submitted instead.

```
<option value="v">Text</option>
```

<p> — Paragraph

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

```
<p>Text of the paragraph.</p>
```

<param> — Object parameter (deprecated)

Status: DEPRECATED | Versions: HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard

The `<param>` element defined a parameter for an `<object>` element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

```
<object data="movie.swf">
  <param name="quality" value="high">
</object>
```

<pre> — Pre-formatted text

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<pre>` element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

```
<pre>
  text with
    preserved   spacing
</pre>
```

<s> — Strikethrough (no longer accurate)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<s>` element renders text with a line through it and represents content that is no longer accurate or no longer relevant, such as an old price next to the new one, or a sold-out item. In HTML 4 it was a purely presentational strike-through synonym for `<strike>` and was deprecated; HTML5 gave it this meaning and made it valid again while removing `<strike>`.

```
<s>outdated text</s>
```

<samp> — Sample output

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

```
<samp>program output</samp>
```

<script> — Script

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<script>` element embeds executable code, almost always JavaScript, or links to an external script file with the `src` attribute. It can also hold data blocks (JSON, templates, import maps, speculation rules) that the browser does not execute when type is set to a non-script MIME type.

```
<script src="app.js" defer></script>
<script>
  // inline code
</script>
```

<select> — Drop-down / list box

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<select>` element creates a control for choosing one option (or, with multiple, several) from a list. It contains `<option>` elements, optionally grouped in `<optgroup>`. By default it appears as a drop-down menu; with the `multiple` attribute or a size greater than 1 it appears as a scrolling list box. The name attribute and the value of the chosen option(s) are submitted with the form.

```
<select name="n">
  <option value="1">One</option>
</select>
```

<small> — Side comment (fine print)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5 (redefined)

The `<small>` element represents side comments and small print: disclaimers, caveats, legal restrictions, copyright notices, attribution. It renders the text one size smaller. In HTML 3.2 and 4 it was purely presentational ('smaller text'); HTML5 kept it and gave it this meaning, while its counterpart `<big>` was removed.

```
<small>fine print</small>
```

<strike> — Strikethrough (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<strike>` element drew a line through its text. It was purely presentational, was deprecated in HTML 4.01 and removed in HTML5. Use `<del>` for removed content, `<s>` for content that is no longer accurate, or CSS text-decoration: line-through for pure styling.

```
<strike>text</strike>
```


** — Strong importance**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

```
<strong>important text</strong>
```

<style> — Embedded style sheet

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<style>` element contains CSS rules that apply to the document. It belongs in `<head>`, though browsers tolerate it in `<body>`. A document may have any number of `<style>` elements; they are applied in order together with any linked style sheets. For anything larger than a few rules, an external file loaded with `<link rel="stylesheet">` is better because it can be cached and shared between pages.

```
<style>
  selector { property: value; }
</style>
```

<sub> — Subscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sub>` element renders text as a subscript: smaller and below the baseline. Use it only where the subscript has a typographic meaning, such as chemical formulas (H₂O), mathematical variable indices (x₁) or footnote markers in some styles. For purely visual lowering, use CSS `vertical-align`.

```
H<sub>2</sub>0
```

<sup> — Superscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sup>` element renders text as a superscript: smaller and above the baseline. Use it where the superscript has meaning: exponents (x^2), ordinal suffixes (1st), footnote references, and abbreviations like M^{lle}. For visual raising only, use CSS.

```
x<sup>2</sup>
```

<table> — Table

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<table>` element represents tabular data: information arranged in a two-dimensional grid of rows and columns, where the position of each cell gives it meaning (this value belongs to this row and this column). Examples: price lists, timetables, statistics, comparison charts, spreadsheets.

```
<table>
  <caption>Title</caption>
  <thead><tr><th>Head</th></tr></thead>
  <tbody><tr><td>Data</td></tr></tbody>
</table>
```

<td> — Table data cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<td>` element defines a cell of a table that contains data. It must be inside a `<tr>`. Each `<td>` occupies one column by default; `colspan` makes it span several columns and `rowspan` several rows. The `headers` attribute can list the ids of the `<th>` cells that describe it, which is needed only for complex tables where `scope` is not enough.

```
<tr>
  <td>Data</td>
</tr>
```

<textarea> — Multi-line text input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<textarea>` element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike `<input>`, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

```
<textarea name="n" rows="4" cols="40">initial text</textarea>
```

<th> — Table header cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<th>` element defines a header cell in a table: a cell that labels a column or a row rather than holding data. Browsers render it bold and centred. The `scope` attribute states whether the header applies to its column (`col`), its row (`row`), a group of columns (`colgroup`) or a group of rows (`rowgroup`). Browsers can usually guess for simple tables, but stating it explicitly is best practice because screen readers use the header text when reading each data cell.

```
<th scope="col">Header</th>
```

<title> — Document title

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<title>` element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one `<title>`, inside `<head>`.

```
<title>Page title</title>
```

<tr> — Table row

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<tr>` element defines a row of cells in a table. It contains `<th>` (header) and/or `<td>` (data) cells and is placed inside `<table>`, `<thead>`, `<tbody>` or `<tfoot>`. Rows are the fundamental unit of table construction: you build a table one row at a time, and the columns emerge from the cells in each row.

```
<tr>
  <td>Cell 1</td>
  <td>Cell 2</td>
</tr>
```

<tt> — Teletype text (obsolete)

Status: OBSOLETE | Versions: HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5

The `<tt>` element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements `<code>`, `<kbd>`, `<samp>` or `<var>`, or CSS `font-family: monospace` on a `<span>`.

```
<tt>text</tt>
```

<u> — Unarticulated annotation (underline)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<u>` element represents a span of inline text that should be rendered in a way that indicates a non-textual annotation: most commonly, marking misspelled words (as a word processor does with a wavy red line) or labelling proper names in Chinese text. Browsers render it with a solid underline. In HTML 4 `<u>` simply meant 'underline' and was deprecated; HTML5 redefined it.

```
<u>text</u>
```

** — Unordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ul>` element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an `<li>`. If the order matters, use `<ol>`.

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

<var> — Variable

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<var>` element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with `<code>`, `<kbd>` and `<samp>`.

```
<var>x</var>
```

HTML 4.0 and 4.01 (1997-1999)

HTML 4 separated structure from presentation, introduced style sheets, scripting, frames, richer tables (thead, tbody, tfoot, colgroup), richer forms (button, fieldset, legend, label, optgroup), internationalisation (dir, lang, bdo) and accessibility features. It came in Strict, Transitional and Frameset flavours. XHTML 1.0 (2000) was HTML 4.01 re-expressed in XML.

Elements in this chapter: 86

Element	Meaning	Status today
<a>	Anchor (hyperlink)	current
<abbr>	Abbreviation	current
<acronym>	Acronym (obsolete)	obsolete
<address>	Contact address	current
<applet>	Java applet (obsolete)	obsolete
<area>	Image map area	current
	Bring attention to (bold)	current
<base>	Document base URL	current
<basefont>	Base font (obsolete)	obsolete
<bdo>	Bidirectional text override	current
<big>	Bigger text (obsolete)	obsolete
<blockquote>	Block quotation	current

Element	Meaning	Status today
<body>	Document body	current
 	Line break	current
<button>	Button	current
<caption>	Table caption	current
<center>	Centred block (obsolete)	obsolete
<cite>	Citation (title of a work)	current
<code>	Inline code	current
<col>	Table column	current
<colgroup>	Table column group	current
<dd>	Description details	current
	Deleted text	current
<dfn>	Definition term	current
<dir>	Directory list (obsolete)	obsolete
<div>	Generic block container	current
<dl>	Description list	current
<dt>	Description term	current
	Emphasis	current
<fieldset>	Form field group	current
	Font (obsolete)	obsolete
<form>	Form	current
<frame>	Frame (obsolete)	obsolete

Element	Meaning	Status today
<frameset>	Frameset (obsolete)	obsolete
<h1>	Headings h1 to h6	current
<head>	Document head	current
<hr>	Thematic break (horizontal rule)	current
<html>	Document root	current
<i>	Idiomatic text (italic)	current
<iframe>	Inline frame	current
	Image	current
<input>	Input control	current
<ins>	Inserted text	current
<isindex>	Search index prompt (obsolete)	obsolete
<kbd>	Keyboard input	current
<label>	Form control label	current
<legend>	Fieldset caption	current
	List item	current
<link>	External resource link	current
<map>	Image map	current
<menu>	Menu (toolbar list)	current
<meta>	Metadata	current
<noframes>	Fallback for frames (obsolete)	obsolete

Element	Meaning	Status today
<noscript>	Fallback when scripting is disabled	current
<object>	External object	current
	Ordered list	current
<optgroup>	Option group	current
<option>	Option	current
<p>	Paragraph	current
<param>	Object parameter (deprecated)	deprecated
<pre>	Pre-formatted text	current
<q>	Inline quotation	current
<s>	Strikethrough (no longer accurate)	current
<samp>	Sample output	current
<script>	Script	current
<select>	Drop-down / list box	current
<small>	Side comment (fine print)	current
	Generic inline container	current
<strike>	Strikethrough (obsolete)	obsolete
	Strong importance	current
<style>	Embedded style sheet	current
<sub>	Subscript	current

Element	Meaning	Status today
<sup>	Superscript	current
<table>	Table	current
<tbody>	Table body	current
<td>	Table data cell	current
<textarea>	Multi-line text input	current
<tfoot>	Table footer	current
<th>	Table header cell	current
<thead>	Table header	current
<title>	Document title	current
<tr>	Table row	current
<tt>	Teletype text (obsolete)	obsolete
<u>	Unarticulated annotation (underline)	current
	Unordered list	current
<var>	Variable	current

<a> — Anchor (hyperlink)

Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<a>` element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and `<a>` was one of the very first elements created by Tim Berners-Lee in 1991.

```
<a href="URL">link text</a>
```

<abbr> — Abbreviation

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<abbr>` element marks an abbreviation or acronym. The optional title attribute holds the full expansion, which most browsers show as a tooltip when the mouse hovers over the text.

```
<abbr title="expansion">ABBR</abbr>
```

<acronym> — Acronym (obsolete)

Status: OBSOLETE | Versions: HTML 4.01; obsolete in HTML5

The `<acronym>` element marked an acronym: an abbreviation pronounced as a word, such as NASA or laser. HTML5 removed it because the distinction from `<abbr>` was confusing and browsers treated the two identically.

```
<acronym title="expansion">NASA</acronym>
```

<address> — Contact address

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

```
<address>contact information</address>
```

<applet> — Java applet (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<applet>` element embedded a Java applet in the page. Applets were small Java programs that ran inside the browser with the help of a plug-in. Java applets were popular in the late 1990s but were slow, insecure and not supported on mobile devices. HTML 4.01 deprecated `<applet>` in favour of `<object>`, and HTML5 removed it entirely. No modern browser runs applets.

```
<applet code="MyApplet.class" width="300" height="200"></applet>
```

<area> — Image map area

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<area>` element defines a clickable region inside an image map. An image map is an image (`<img usemap="#name">`) linked to a `<map name="name">` element that contains one or more `<area>` elements. Each area has a shape and coordinates, and behaves like a hyperlink.

```
<area shape="rect" coords="x1,y1,x2,y2" href="URL" alt="text">
```

** — Bring attention to (bold)**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

```
<b>text</b>
```

<base> — Document base URL

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<base>` element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one `<base>` element per document and it must be placed in `<head>`, before any element that uses a URL.

```
<base href="URL" target="_blank">
```

<basefont> — Base font (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<basefont>` element set the default font size, colour and face for the whole document. It was a presentational element from HTML 3.2 and was deprecated in HTML 4.01 in favour of CSS. HTML5 removed it. Use CSS on the body element instead.

```
<basefont size="3" color="black" face="Arial">
```

<bdo> — Bidirectional text override

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<bdo>` element overrides the current text direction and forces its content to be displayed in the direction given by the `dir` attribute, character by character, ignoring the Unicode bidirectional algorithm. The `dir` attribute is required.

```
<bdo dir="rtl">text</bdo>
```

<big> — Bigger text (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01; obsolete in HTML5

The `<big>` element rendered its content one font size larger than the surrounding text. It was purely presentational and was removed in HTML5. Its counterpart `<small>` survived because it was given a new meaning (side comments and fine print). Use CSS font-size instead of `<big>`.

```
<big>text</big>
```

<blockquote> — Block quotation

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<blockquote>` element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional `cite` attribute holds the URL of the source; a visible citation should be given with the `<cite>` element, usually in a `<footer>` inside the `blockquote` or in a following paragraph.

```
<blockquote cite="URL">quoted text</blockquote>
```

<body> — Document body

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<body>` element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one `<body>` in a document and it is the second child of `<html>`, after `<head>`.

```
<body>...</body>
```

**
 — Line break**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

```
<br>
```

<button> — Button

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<button>` element represents a clickable button that the user can activate with a mouse, keyboard, finger or voice to perform an action. Unlike `<input type="button">`, a `<button>` may contain rich content: text, images, icons and other phrasing content.

```
<button type="button">label</button>
```

<caption> — Table caption

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<caption>` element gives a table a title or explanation. It must be the first child of the `<table>` element, and a table may have only one. By default it is displayed centred above the table; the CSS `caption-side` property can move it below.

```
<table>
  <caption>Title</caption>
  ...
</table>
```

<center> — Centred block (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<center>` element horizontally centred its content. It was introduced by Netscape, standardised in HTML 3.2, deprecated in HTML 4.01 and removed in HTML5. It is equivalent to `<div align="center">`. Use CSS instead: `text-align: center` for inline content, or `margin: 0 auto` with a width for blocks.

```
<center>content</center>
```

<cite> — Citation (title of a work)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<cite>` element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

```
<cite>Title of Work</cite>
```


<code> — Inline code

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

```
<code>snippet</code>
```

<col> — Table column

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<col>` element defines one or more columns in a table so that attributes and styles can be applied to whole columns at once. It is a void element and lives inside a `<colgroup>`. The `span` attribute makes one `<col>` stand for several consecutive columns.

```
<colgroup>
  <col span="2" style="background: #eee">
</colgroup>
```

<colgroup> — Table column group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<colgroup>` element groups one or more columns of a table for the purpose of styling and structure. It is placed inside `<table>` after the optional `<caption>` and before any `<thead>`, `<tbody>`, `<tfoot>` or `<tr>`. It either has a `span` attribute (and no children) or contains `<col>` elements.

```
<colgroup span="2"></colgroup>
<!-- or -->
<colgroup><col><col></colgroup>
```

<dd> — Description details

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The <dd> element provides the description, definition or value for the preceding term (<dt>) in a description list (<dl>). One term may have several <dd> elements, and several terms may share one <dd>. Browsers indent <dd> from the left by default.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

 — Deleted text

Status: CURRENT | Versions: HTML 4.01, HTML5

The element marks text that has been deleted from a document, typically to show changes between versions ('track changes'). Browsers render it with a line through it. It is usually paired with <ins>, which marks inserted text.

```
<del datetime="YYYY-MM-DD" cite="URL">deleted text</del>
```

<dfn> — Definition term

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The <dfn> element marks the term that is being defined in the surrounding sentence or paragraph. The nearest paragraph, description list group or section is the definition. Browsers render it in italics. If the element has a title attribute, that is the term being defined; if it contains an <abbr> with a title, the abbreviation's expansion is the term; otherwise the text content is the term.

```
<dfn>term</dfn>
```

<dir> — Directory list (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

```
<dir><li>item</li></dir>
```

<div> — Generic block container

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<div>` element (division) is a generic container for flow content. It has no meaning of its own and no effect on the content or layout until it is styled with CSS (using `class` or `id`) or manipulated with JavaScript. Use it to group elements for styling purposes, for layout (flexbox, grid), or when no other element is more appropriate.

```
<div class="name">...</div>
```

<dl> — Description list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`).

Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dt> — Description term

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

```
<dt>Term</dt>
```

** — Emphasis**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

```
<em>text</em>
```

<fieldset> — Form field group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<fieldset>` element groups related controls and labels inside a form, for example all the fields of a postal address, or a set of radio buttons that answer one question. The optional `<legend>` element, which must be the first child, gives the group a caption. Browsers draw a border around the group with the legend placed on the border.

```
<fieldset>
  <legend>Caption</legend>
  ...controls...
</fieldset>
```

** — Font (obsolete)**

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<font>` element set the size, colour and typeface of its text. It was the main way to style text before CSS, was deprecated in HTML 4.01 and removed in HTML5. Browsers still render it for old pages. Every use of `<font>` can be replaced by CSS: `font-size`, `color` and `font-family`.

```
<font size="4" color="red" face="Arial">text</font>
```

<form> — Form

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<form>` element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the `action` attribute using the HTTP method in the `method` attribute.

```
<form action="URL" method="post">
  ...controls...
  <button>Submit</button>
</form>
```

<frame> — Frame (obsolete)

Status: OBSOLETE | Versions: Netscape 2 (1996), HTML 4.01 Frameset; obsolete in HTML5

The `<frame>` element defined one rectangular sub-window inside a `<frameset>`, each displaying a separate HTML document. Frames were used in the late 1990s to keep a navigation menu fixed while the content scrolled. They caused many problems: bookmarks and the back button did not work as expected, search engines could not index the pages properly, printing was unreliable and screen readers struggled. HTML5 removed `<frame>` and `<frameset>`. Use CSS for layout and `<iframe>` when you truly need to embed another document.

```
<frameset cols="200,*">
  <frame src="menu.html" name="menu">
  <frame src="main.html" name="main">
</frameset>
```

<frameset> — Frameset (obsolete)

Status: OBSOLETE | Versions: Netscape 2, HTML 4.01 Frameset; obsolete in HTML5

The `<frameset>` element replaced `<body>` in a frames document and divided the window into rows and columns, each filled by a `<frame>` or a nested `<frameset>`. It required the HTML 4.01 Frameset DOCTYPE. It is obsolete in HTML5. See `<frame>` for the reasons and the modern replacement.

```
<frameset rows="80,*">...</frameset>
```

<h1> — Headings h1 to h6

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<h1>` to `<h6>` elements represent six levels of section headings. `<h1>` is the highest (most important) level and `<h6>` the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```

<head> — Document head

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<head>` element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and `<meta>` tags for search engines and social networks. Nothing inside `<head>` is displayed on the page itself, except the title, which appears in the browser tab.

```
<head>
  <meta charset="utf-8">
  <title>Title</title>
</head>
```

<hr> — Thematic break (horizontal rule)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<hr>` element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

```
<hr>
```

<html> — Document root

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<html>` element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: `<head>` followed by `<body>`. It comes immediately after the `<!DOCTYPE html>` declaration.

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

<i> — Idiomatic text (italic)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<i>` element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

```
<i>text</i>
```

<iframe> — Inline frame

Status: CURRENT | Versions: Internet Explorer 3 (1996), HTML 4.01, HTML5

The `<iframe>` element embeds another HTML page inside the current page, in a rectangular area called a nested browsing context. Each `iframe` has its own document, its own window object and its own history. Common uses are embedding videos from YouTube, maps, advertisements, payment forms, social media widgets and third-party tools.

```
<iframe src="URL" title="description" width="600" height="400"></
iframe>
```


** — Image**

Status: CURRENT | Versions: Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<img>` element embeds an image in the document. It is a void element; the image file is identified by the `src` attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The `alt` attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

```

```

<input> — Input control

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<input>` element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the `type` attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

```
<input type="text" name="field" id="field">
```

<ins> — Inserted text

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<ins>` element marks text that has been added to a document, to show changes between versions. Browsers underline it by default. It is the counterpart of `<del>`. Like `<del>` it accepts optional `cite` and `datetime` attributes that record why and when the change was made.

```
<ins datetime="YYYY-MM-DD">inserted text</ins>
```

<isindex> — Search index prompt (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<isindex>` element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a `<form>` with an `<input type="search">`.

```
<isindex prompt="Search: ">
```

<kbd> — Keyboard input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<kbd>` element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting `<kbd>` elements inside a `<kbd>` represents individual keys in a key combination; wrapping `<kbd>` in `<samp>` represents input echoed by the system.

```
<kbd>Ctrl</kbd> + <kbd>S</kbd>
```

<label> — Form control label

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<label>` element gives a caption to a form control. Associating a label with a control has two big benefits: clicking the label focuses or toggles the control (a much larger click target for checkboxes and radio buttons), and screen readers announce the label when the control gets focus.

```
<label for="id">Text</label> <input id="id">
<!-- or -->
<label>Text <input></label>
```

<legend> — Fieldset caption

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<legend>` element provides a caption for its parent `<fieldset>`. It must be the first child of the fieldset. Browsers draw it on top of the fieldset's border. Screen readers announce the legend together with the label of each control inside the group, which is why fieldset/legend is the recommended way to caption groups of radio buttons or checkboxes.

```
<fieldset>
  <legend>Caption</legend>
  ...
</fieldset>
```

** — List item**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

```
<ul>
  <li>Item</li>
</ul>
```

<link> — External resource link

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<link>` element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (`rel="stylesheet"`), but it is also used for site icons (`rel="icon"`), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in `<head>`, though body-ok link types such as `stylesheet` may appear in `<body>`.

```
<link rel="stylesheet" href="style.css">
```

<map> — Image map

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<map>` element, together with `<area>` elements, defines a client-side image map: an image with several clickable regions each linking to a different destination. The map is connected to an `<img>` through the `img`'s `usemap` attribute, which must be `#` followed by the map's name.

```
<map name="n">  
  <area ...>  
</map>  
<img usemap="#n" ...>
```

<menu> — Menu (toolbar list)

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<menu>` element is a semantic alternative to `<ul>` for a list of commands or actions, such as a toolbar of buttons. Each command is an `<li>` containing a `<button>` or similar control. Browsers render it identically to `<ul>`.

```
<menu>
  <li><button>Action</button></li>
</menu>
```

<meta> — Metadata

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<meta>` element represents metadata about the document that cannot be expressed by `<title>`, `<base>`, `<link>`, `<style>` or `<script>`. It is a void element placed in `<head>`. Depending on which attributes are present it does one of four things: declares the character encoding (charset); provides document-level metadata as a name/content pair (name); simulates an HTTP response header (http-equiv); or supplies microdata (itemprop).

```
<meta charset="utf-8">
<meta name="description" content="...">
```

<noframes> — Fallback for frames (obsolete)

Status: OBSOLETE | Versions: HTML 4.01 Frameset; obsolete in HTML5

The `<noframes>` element contained content to display in browsers that did not support frames. It became obsolete together with `<frameset>` and `<frame>` in HTML5.

```
<noframes><body>fallback</body></noframes>
```

<noscript> — Fallback when scripting is disabled

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<noscript>` element defines content to be shown only if scripting is disabled in the browser or the browser does not support scripting. When scripting is enabled the content is ignored entirely (in fact the parser treats it as raw text). It may be used in `<head>` (containing only `<link>`, `<style>` and `<meta>`) and in `<body>`.

```
<noscript>fallback content</noscript>
```

<object> — External object

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<object>` element represents an external resource, treated as an image, a nested browsing context (like an `iframe`) or a resource handled by a plug-in, depending on its type. It was introduced in HTML 4.01 as a universal replacement for `<img>`, `<applet>` and `<embed>`. Unlike `<embed>`, it accepts fallback content between its tags, shown if the resource cannot be displayed, and it may contain `<param>` elements (now obsolete) that pass parameters to plug-ins.

```
<object data="URL" type="MIME" width="W" height="H">fallback</object>
```

** — Ordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

<optgroup> — Option group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<optgroup>` element groups related `<option>` elements inside a `<select>` under a non-selectable heading given by the `label` attribute. Groups cannot be nested. Disabling an `optgroup` disables all its options.

```
<select>
  <optgroup label="Group">
    <option>...</option>
  </optgroup>
</select>
```

<option> — Option

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<option>` element defines one item in a `<select>` drop-down list, an `<optgroup>`, or a `<datalist>` of suggestions. Its text content is what the user sees; the `value` attribute is what is submitted with the form. If `value` is omitted the text content is submitted instead.

```
<option value="v">Text</option>
```

<p> — Paragraph

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

```
<p>Text of the paragraph.</p>
```

<param> — Object parameter (deprecated)

Status: DEPRECATED | Versions: HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard

The `<param>` element defined a parameter for an `<object>` element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

```
<object data="movie.swf">  
  <param name="quality" value="high">  
</object>
```


<pre> — Pre-formatted text

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<pre>` element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

```
<pre>
  text with
    preserved   spacing
</pre>
```

<q> — Inline quotation

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<q>` element indicates a short inline quotation that sits within a sentence. Browsers automatically add quotation marks around it, choosing the correct style for the language declared with `lang` (curly double quotes in English, guillemets in French, and so on), so you should not type quotation marks yourself. For long quotations that need their own paragraph use `<blockquote>`.

```
<q cite="URL">quoted words</q>
```

<s> — Strikethrough (no longer accurate)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<s>` element renders text with a line through it and represents content that is no longer accurate or no longer relevant, such as an old price next to the new one, or a sold-out item. In HTML 4 it was a purely presentational strike-through synonym for `<strike>` and was deprecated; HTML5 gave it this meaning and made it valid again while removing `<strike>`.

```
<s>outdated text</s>
```

<samp> — Sample output

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

```
<samp>program output</samp>
```

<script> — Script

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<script>` element embeds executable code, almost always JavaScript, or links to an external script file with the `src` attribute. It can also hold data blocks (JSON, templates, import maps, speculation rules) that the browser does not execute when type is set to a non-script MIME type.

```
<script src="app.js" defer></script>
<script>
  // inline code
</script>
```

<select> — Drop-down / list box

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<select>` element creates a control for choosing one option (or, with multiple, several) from a list. It contains `<option>` elements, optionally grouped in `<optgroup>`. By default it appears as a drop-down menu; with the `multiple` attribute or a size greater than 1 it appears as a scrolling list box. The name attribute and the value of the chosen option(s) are submitted with the form.

```
<select name="n">
  <option value="1">One</option>
</select>
```

<small> — Side comment (fine print)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5 (redefined)

The `<small>` element represents side comments and small print: disclaimers, caveats, legal restrictions, copyright notices, attribution. It renders the text one size smaller. In HTML 3.2 and 4 it was purely presentational ('smaller text'); HTML5 kept it and gave it this meaning, while its counterpart `<big>` was removed.

```
<small>fine print</small>
```

** — Generic inline container**

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<span>` element is a generic inline container for phrasing content. Like `<div>`, it has no meaning of its own and no visual effect; it exists to be a hook for CSS (via class or id) or JavaScript, or to carry attributes such as lang or dir on a part of a sentence. It should be used only when no other element is more appropriate: `<em>`, `<strong>`, `<mark>`, `<time>`, `<abbr>`, `<code>` and the other text-level elements all carry meaning that `<span>` does not.

```
<span class="name">text</span>
```

<strike> — Strikethrough (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<strike>` element drew a line through its text. It was purely presentational, was deprecated in HTML 4.01 and removed in HTML5. Use `<del>` for removed content, `<s>` for content that is no longer accurate, or CSS text-decoration: line-through for pure styling.

```
<strike>text</strike>
```

** — Strong importance**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

```
<strong>important text</strong>
```

<style> — Embedded style sheet

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<style>` element contains CSS rules that apply to the document. It belongs in `<head>`, though browsers tolerate it in `<body>`. A document may have any number of `<style>` elements; they are applied in order together with any linked style sheets. For anything larger than a few rules, an external file loaded with `<link rel="stylesheet">` is better because it can be cached and shared between pages.

```
<style>
  selector { property: value; }
</style>
```

<sub> — Subscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sub>` element renders text as a subscript: smaller and below the baseline. Use it only where the subscript has a typographic meaning, such as chemical formulas (H₂O), mathematical variable indices (x₁) or footnote markers in some styles. For purely visual lowering, use CSS `vertical-align`.

```
H<sub>2</sub>0
```

<sup> — Superscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sup>` element renders text as a superscript: smaller and above the baseline. Use it where the superscript has meaning: exponents (x²), ordinal suffixes (1st), footnote references, and abbreviations like M^{lle}. For visual raising only, use CSS.

```
x<sup>2</sup>
```

<table> — Table

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<table>` element represents tabular data: information arranged in a two-dimensional grid of rows and columns, where the position of each cell gives it meaning (this value belongs to this row and this column). Examples: price lists, timetables, statistics, comparison charts, spreadsheets.

```
<table>
  <caption>Title</caption>
  <thead><tr><th>Head</th></tr></thead>
  <tbody><tr><td>Data</td></tr></tbody>
</table>
```

<tbody> — Table body

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<tbody>` element groups the main data rows of a table, as distinct from the header rows in `<thead>` and the summary rows in `<tfoot>`. A table may have several `<tbody>` elements to group rows into sections (for example one per category), which can then be styled separately. When you write `<tr>` directly inside `<table>` without a `<tbody>`, the browser creates one automatically, so a `tbody` is always present in the DOM.

```
<tbody>
  <tr>...</tr>
</tbody>
```

<td> — Table data cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<td>` element defines a cell of a table that contains data. It must be inside a `<tr>`. Each `<td>` occupies one column by default; `colspan` makes it span several columns and `rowspan` several rows. The `headers` attribute can list the ids of the `<th>` cells that describe it, which is needed only for complex tables where `scope` is not enough.

```
<tr>
  <td>Data</td>
</tr>
```

<textarea> — Multi-line text input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<textarea>` element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike `<input>`, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

```
<textarea name="n" rows="4" cols="40">initial text</textarea>
```

<tfoot> — Table footer

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<tfoot>` element groups the rows that summarise the columns of a table: totals, averages, counts. In HTML5 it is placed after the `<tbody>` elements (HTML 4.01 required it before them so that the browser could render the footer before receiving all data). When printing a long table, browsers can repeat the `tfoot` at the bottom of every page.

```
<tfoot>
  <tr><th>Total</th><td>260</td></tr>
</tfoot>
```

<th> — Table header cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<th>` element defines a header cell in a table: a cell that labels a column or a row rather than holding data. Browsers render it bold and centred. The `scope` attribute states whether the header applies to its column (`col`), its row (`row`), a group of columns (`colgroup`) or a group of rows (`rowgroup`). Browsers can usually guess for simple tables, but stating it explicitly is best practice because screen readers use the header text when reading each data cell.

```
<th scope="col">Header</th>
```

<thead> — Table header

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<thead>` element groups the rows that hold the column headers of a table. It comes after any `<caption>` and `<colgroup>` and before `<tbody>` and `<tfoot>`. Browsers repeat the `thead` at the top of each printed page of a long table, and CSS `position: sticky` on its cells keeps the headers visible while scrolling. A table may have at most one `thead`.

```
<thead>
  <tr><th>Column</th></tr>
</thead>
```


<title> — Document title

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<title>` element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one `<title>`, inside `<head>`.

```
<title>Page title</title>
```

<tr> — Table row

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<tr>` element defines a row of cells in a table. It contains `<th>` (header) and/or `<td>` (data) cells and is placed inside `<table>`, `<thead>`, `<tbody>` or `<tfoot>`. Rows are the fundamental unit of table construction: you build a table one row at a time, and the columns emerge from the cells in each row.

```
<tr>
  <td>Cell 1</td>
  <td>Cell 2</td>
</tr>
```

<tt> — Teletype text (obsolete)

Status: OBSOLETE | Versions: HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5

The `<tt>` element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements `<code>`, `<kbd>`, `<samp>` or `<var>`, or CSS font-family: monospace on a `<span>`.

```
<tt>text</tt>
```

<u> — Unarticulated annotation (underline)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<u>` element represents a span of inline text that should be rendered in a way that indicates a non-textual annotation: most commonly, marking misspelled words (as a word processor does with a wavy red line) or labelling proper names in Chinese text. Browsers render it with a solid underline. In HTML 4 `<u>` simply meant 'underline' and was deprecated; HTML5 redefined it.

```
<u>text</u>
```

** — Unordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ul>` element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an `<li>`. If the order matters, use `<ol>`.

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

<var> — Variable

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<var>` element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with `<code>`, `<kbd>` and `<samp>`.

```
<var>x</var>
```

HTML5 (WHATWG 2004 onwards, W3C Recommendation 2014)

HTML5 added semantic sectioning (header, footer, nav, main, article, section, aside), native audio and video, canvas, new form controls and input types with built-in validation, and it defined precisely how browsers must parse even broken markup. It also removed the presentational and frame elements.

Elements in this chapter: 127

Element	Meaning	Status today
<a>	Anchor (hyperlink)	current
<abbr>	Abbreviation	current
<acronym>	Acronym (obsolete)	obsolete
<address>	Contact address	current
<applet>	Java applet (obsolete)	obsolete
<area>	Image map area	current
<article>	Self-contained article	current
<aside>	Aside (sidebar / tangential content)	current
<audio>	Embedded audio	current
	Bring attention to (bold)	current
<base>	Document base URL	current
<basefont>	Base font (obsolete)	obsolete

Element	Meaning	Status today
<bdi>	Bidirectional isolate	current
<bdo>	Bidirectional text override	current
<big>	Bigger text (obsolete)	obsolete
<blockquote>	Block quotation	current
<body>	Document body	current
 	Line break	current
<button>	Button	current
<canvas>	Graphics canvas	current
<caption>	Table caption	current
<center>	Centred block (obsolete)	obsolete
<cite>	Citation (title of a work)	current
<code>	Inline code	current
<col>	Table column	current
<colgroup>	Table column group	current
<data>	Machine-readable value	current
<datalist>	Predefined options for an input	current
<dd>	Description details	current
	Deleted text	current
<details>	Disclosure widget	current

Element	Meaning	Status today
<dfn>	Definition term	current
<dialog>	Dialog box	current
<dir>	Directory list (obsolete)	obsolete
<div>	Generic block container	current
<dl>	Description list	current
<dt>	Description term	current
	Emphasis	current
<embed>	External content	current
<fieldset>	Form field group	current
<figcaption>	Figure caption	current
<figure>	Figure with optional caption	current
	Font (obsolete)	obsolete
<footer>	Footer	current
<form>	Form	current
<frame>	Frame (obsolete)	obsolete
<frameset>	Frameset (obsolete)	obsolete
<h1>	Headings h1 to h6	current
<head>	Document head	current
<header>	Header	current
<hgroup>	Heading group	current
<hr>	Thematic break (horizontal rule)	current
<html>	Document root	current

Element	Meaning	Status today
<i>	Idiomatic text (italic)	current
<iframe>	Inline frame	current
	Image	current
<input>	Input control	current
<ins>	Inserted text	current
<isindex>	Search index prompt (obsolete)	obsolete
<kbd>	Keyboard input	current
<keygen>	Key-pair generator (obsolete)	obsolete
<label>	Form control label	current
<legend>	Fieldset caption	current
	List item	current
<link>	External resource link	current
<main>	Main content	current
<map>	Image map	current
<mark>	Highlighted (marked) text	current
<marquee>	Scrolling text (obsolete)	obsolete
<math>	MathML root	current
<menu>	Menu (toolbar list)	current
<menuitem>	Menu item (obsolete)	obsolete
<meta>	Metadata	current

Element	Meaning	Status today
<meter>	Scalar measurement (gauge)	current
<nav>	Navigation section	current
<noframes>	Fallback for frames (obsolete)	obsolete
<noscript>	Fallback when scripting is disabled	current
<object>	External object	current
	Ordered list	current
<optgroup>	Option group	current
<option>	Option	current
<output>	Calculation or user-action result	current
<p>	Paragraph	current
<param>	Object parameter (deprecated)	deprecated
<picture>	Picture (responsive image container)	current
<pre>	Pre-formatted text	current
<progress>	Progress indicator	current
<q>	Inline quotation	current
<rb>	Ruby base (deprecated)	deprecated
<rp>	Ruby fallback parenthesis	current

Element	Meaning	Status today
<rt>	Ruby text (annotation)	current
<rtc>	Ruby text container (deprecated)	deprecated
<ruby>	Ruby annotation	current
<s>	Strikethrough (no longer accurate)	current
<samp>	Sample output	current
<script>	Script	current
<section>	Generic section	current
<select>	Drop-down / list box	current
<slot>	Shadow DOM slot	current
<small>	Side comment (fine print)	current
<source>	Media or image source	current
	Generic inline container	current
<strike>	Strikethrough (obsolete)	obsolete
	Strong importance	current
<style>	Embedded style sheet	current
<sub>	Subscript	current
<summary>	Disclosure summary	current

Element	Meaning	Status today
<sup>	Superscript	current
<svg>	Scalable Vector Graphics root	current
<table>	Table	current
<tbody>	Table body	current
<td>	Table data cell	current
<template>	Content template	current
<textarea>	Multi-line text input	current
<tfoot>	Table footer	current
<th>	Table header cell	current
<thead>	Table header	current
<time>	Date / time	current
<title>	Document title	current
<tr>	Table row	current
<track>	Text track for media	current
<tt>	Teletype text (obsolete)	obsolete
<u>	Unarticulated annotation (underline)	current
	Unordered list	current
<var>	Variable	current
<video>	Embedded video	current
<wbr>	Word break opportunity	current

<a> — Anchor (hyperlink)

Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<a>` element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and `<a>` was one of the very first elements created by Tim Berners-Lee in 1991.

```
<a href="URL">link text</a>
```

<abbr> — Abbreviation

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<abbr>` element marks an abbreviation or acronym. The optional title attribute holds the full expansion, which most browsers show as a tooltip when the mouse hovers over the text.

```
<abbr title="expansion">ABBR</abbr>
```

<acronym> — Acronym (obsolete)

Status: OBSOLETE | Versions: HTML 4.01; obsolete in HTML5

The `<acronym>` element marked an acronym: an abbreviation pronounced as a word, such as NASA or laser. HTML5 removed it because the distinction from `<abbr>` was confusing and browsers treated the two identically.

```
<acronym title="expansion">NASA</acronym>
```

<address> — Contact address

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

```
<address>contact information</address>
```

<applet> — Java applet (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<applet>` element embedded a Java applet in the page. Applets were small Java programs that ran inside the browser with the help of a plug-in. Java applets were popular in the late 1990s but were slow, insecure and not supported on mobile devices. HTML 4.01 deprecated `<applet>` in favour of `<object>`, and HTML5 removed it entirely. No modern browser runs applets.

```
<applet code="MyApplet.class" width="300" height="200"></applet>
```

<area> — Image map area

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<area>` element defines a clickable region inside an image map. An image map is an image (`<img usemap="#name">`) linked to a `<map name="name">` element that contains one or more `<area>` elements. Each area has a shape and coordinates, and behaves like a hyperlink.

```
<area shape="rect" coords="x1,y1,x2,y2" href="URL" alt="text">
```

<article> — Self-contained article

Status: CURRENT | Versions: HTML5

The `<article>` element represents a complete, self-contained piece of content that would make sense on its own if it were distributed independently, for example in an RSS feed. Typical uses are a blog post, a news story, a forum post, a product card, a user comment or an interactive widget.

```
<article>...</article>
```

<aside> — Aside (sidebar / tangential content)

Status: CURRENT | Versions: HTML5

The `<aside>` element represents content that is only indirectly related to the main content around it. It is often displayed as a sidebar or a call-out box: pull quotes, related links, advertising, author biographies, glossary definitions, or a list of other articles.

```
<aside>...</aside>
```

<audio> — Embedded audio

Status: CURRENT | Versions: HTML5

The `<audio>` element embeds sound content: music, podcasts, sound effects or recorded speech. The source may be given with the `src` attribute or with one or more `<source>` child elements so that the browser can choose a format it supports. Content placed inside the element after the `<source>` elements is shown only by browsers that do not support `<audio>` at all.

```
<audio src="file.mp3" controls></audio>
```

** — Bring attention to (bold)**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

```
<b>text</b>
```

<base> — Document base URL

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<base>` element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one `<base>` element per document and it must be placed in `<head>`, before any element that uses a URL.

```
<base href="URL" target="_blank">
```

<basefont> — Base font (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<basefont>` element set the default font size, colour and face for the whole document. It was a presentational element from HTML 3.2 and was deprecated in HTML 4.01 in favour of CSS. HTML5 removed it. Use CSS on the body element instead.

```
<basefont size="3" color="black" face="Arial">
```

<bdi> — Bidirectional isolate

Status: CURRENT | Versions: HTML5

The `<bdi>` element isolates a span of text that might be written in a different direction from the text around it, so that its direction does not affect the surrounding text. It is essential when you insert user-generated text of unknown direction, such as user names, into a sentence.

```
<bdi>text of unknown direction</bdi>
```

<bdo> — Bidirectional text override

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<bdo>` element overrides the current text direction and forces its content to be displayed in the direction given by the `dir` attribute, character by character, ignoring the Unicode bidirectional algorithm. The `dir` attribute is required.

```
<bdo dir="rtl">text</bdo>
```

<big> — Bigger text (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01; obsolete in HTML5

The `<big>` element rendered its content one font size larger than the surrounding text. It was purely presentational and was removed in HTML5. Its counterpart `<small>` survived because it was given a new meaning (side comments and fine print). Use CSS font-size instead of `<big>`.

```
<big>text</big>
```

<blockquote> — Block quotation

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<blockquote>` element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional `cite` attribute holds the URL of the source; a visible citation should be given with the `<cite>` element, usually in a `<footer>` inside the `blockquote` or in a following paragraph.

```
<blockquote cite="URL">quoted text</blockquote>
```

<body> — Document body

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<body>` element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one `<body>` in a document and it is the second child of `<html>`, after `<head>`.

```
<body>...</body>
```

**
 — Line break**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

```
<br>
```

<button> — Button

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<button>` element represents a clickable button that the user can activate with a mouse, keyboard, finger or voice to perform an action. Unlike `<input type="button">`, a `<button>` may contain rich content: text, images, icons and other phrasing content.

```
<button type="button">label</button>
```

<canvas> — Graphics canvas

Status: CURRENT | Versions: HTML5

The `<canvas>` element provides a blank, resolution-dependent bitmap surface on which JavaScript can draw graphics: shapes, text, images, animations, charts, games and image processing. The element itself does nothing; all drawing is done through the 2D context (`getContext('2d')`) or a WebGL / WebGPU context.

```
<canvas id="c" width="400" height="300">fallback</canvas>
```

<caption> — Table caption

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<caption>` element gives a table a title or explanation. It must be the first child of the `<table>` element, and a table may have only one. By default it is displayed centred above the table; the CSS `caption-side` property can move it below.

```
<table>
  <caption>Title</caption>
  ...
</table>
```


<center> — Centred block (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<center>` element horizontally centred its content. It was introduced by Netscape, standardised in HTML 3.2, deprecated in HTML 4.01 and removed in HTML5. It is equivalent to `<div align="center">`. Use CSS instead: `text-align: center` for inline content, or `margin: 0 auto` with a width for blocks.

```
<center>content</center>
```

<cite> — Citation (title of a work)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<cite>` element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

```
<cite>Title of Work</cite>
```

<code> — Inline code

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

```
<code>snippet</code>
```

<col> — Table column

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<col>` element defines one or more columns in a table so that attributes and styles can be applied to whole columns at once. It is a void element and lives inside a `<colgroup>`. The `span` attribute makes one `<col>` stand for several consecutive columns.

```
<colgroup>
  <col span="2" style="background: #eee">
</colgroup>
```

<colgroup> — Table column group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<colgroup>` element groups one or more columns of a table for the purpose of styling and structure. It is placed inside `<table>` after the optional `<caption>` and before any `<thead>`, `<tbody>`, `<tfoot>` or `<tr>`. It either has a `span` attribute (and no children) or contains `<col>` elements.

```
<colgroup span="2"></colgroup>
<!-- or -->
<colgroup><col><col></colgroup>
```

<data> — Machine-readable value

Status: CURRENT | Versions: HTML5

The `<data>` element links a piece of human-readable content with a machine-readable version of the same value held in the required `value` attribute. It is useful for product codes, numeric values written in words, sorting keys and similar cases where a script or search engine needs the exact value while the visitor sees a friendly form.

```
<data value="machine value">human text</data>
```

<datalist> — Predefined options for an input

Status: CURRENT | Versions: HTML5

The `<datalist>` element contains a set of `<option>` elements that represent suggested values for an `<input>`. The input is linked to the list with its `list` attribute, which must equal the datalist's id. The user can pick a suggestion or type any other value, which makes it a combination of a text box and a drop-down list (a 'combobox').

```
<input list="id">
<datalist id="id">
  <option value="...">
</datalist>
```

<dd> — Description details

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dd>` element provides the description, definition or value for the preceding term (`<dt>`) in a description list (`<dl>`). One term may have several `<dd>` elements, and several terms may share one `<dd>`. Browsers indent `<dd>` from the left by default.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

** — Deleted text**

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<del>` element marks text that has been deleted from a document, typically to show changes between versions ('track changes'). Browsers render it with a line through it. It is usually paired with `<ins>`, which marks inserted text.

```
<del datetime="YYYY-MM-DD" cite="URL">deleted text</del>
```

<details> — Disclosure widget

Status: CURRENT | Versions: HTML5 (5.1)

The `<details>` element creates a disclosure widget: a box that can be opened and closed by the user to show or hide its content, with no JavaScript at all. The first child should be a `<summary>` element, which provides the always-visible label and the clickable arrow. If there is no `<summary>` the browser shows the word 'Details'.

```
<details>
  <summary>Label</summary>
  content
</details>
```

<dfn> — Definition term

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<dfn>` element marks the term that is being defined in the surrounding sentence or paragraph. The nearest paragraph, description list group or section is the definition. Browsers render it in italics. If the element has a `title` attribute, that is the term being defined; if it contains an `<abbr>` with a `title`, the abbreviation's expansion is the term; otherwise the text content is the term.

```
<dfn>term</dfn>
```

<dialog> — Dialog box

Status: CURRENT | Versions: HTML 5.2 / Living Standard

The `<dialog>` element represents a dialog box or other interactive component such as a confirmation prompt, an alert, a subwindow or a modal overlay. It is hidden by default and is shown with the `open` attribute or, preferably, with the JavaScript methods `show()` (non-modal) and `showModal()` (modal). A modal dialog blocks interaction with the rest of the page, is displayed on top of everything else, and can be closed with the Escape key.

```
<dialog id="d">...</dialog>
<script>document.getElementById('d').showModal()</script>
```

<dir> — Directory list (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

```
<dir><li>item</li></dir>
```

<div> — Generic block container

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<div>` element (division) is a generic container for flow content. It has no meaning of its own and no effect on the content or layout until it is styled with CSS (using `class` or `id`) or manipulated with JavaScript. Use it to group elements for styling purposes, for layout (flexbox, grid), or when no other element is more appropriate.

```
<div class="name">...</div>
```

<dl> — Description list

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`).

Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

<dt> — Description term

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

```
<dt>Term</dt>
```

** — Emphasis**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

```
<em>text</em>
```

<embed> — External content

Status: CURRENT | Versions: Netscape extension (1995), standardised in HTML5

The `<embed>` element embeds external content at a specified point in the document. The content is provided by an external application or plug-in, or is a natively supported type such as an image, video, PDF or HTML page. It is a void element with no fallback content.

```
<embed src="file.pdf" type="application/pdf" width="600" height="400">
```

<fieldset> — Form field group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<fieldset>` element groups related controls and labels inside a form, for example all the fields of a postal address, or a set of radio buttons that answer one question. The optional `<legend>` element, which must be the first child, gives the group a caption. Browsers draw a border around the group with the legend placed on the border.

```
<fieldset>
  <legend>Caption</legend>
  ...controls...
</fieldset>
```

<figcaption> — Figure caption

Status: CURRENT | Versions: HTML5

The `<figcaption>` element provides a caption or legend for the content of its parent `<figure>` element. It must be either the first or the last child of the `<figure>`. The caption is programmatically associated with the figure, which gives the figure an accessible name.

```
<figure>
  <img ...>
  <figcaption>Caption</figcaption>
</figure>
```

<figure> — Figure with optional caption

Status: CURRENT | Versions: HTML5

The `<figure>` element represents self-contained content that is referenced from the main flow but could be moved elsewhere (to an appendix, a sidebar or another page) without affecting the meaning of the text: an illustration, a diagram, a photo, a code listing, a chart, a poem, a table. An optional `<figcaption>` gives it a caption.

```
<figure>
  content
  <figcaption>Caption</figcaption>
</figure>
```

** — Font (obsolete)**

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<font>` element set the size, colour and typeface of its text. It was the main way to style text before CSS, was deprecated in HTML 4.01 and removed in HTML5. Browsers still render it for old pages. Every use of `<font>` can be replaced by CSS: `font-size`, `color` and `font-family`.

```
<font size="4" color="red" face="Arial">text</font>
```

<footer> — Footer

Status: CURRENT | Versions: HTML5

The `<footer>` element represents the footer of its nearest ancestor sectioning content or sectioning root: the page (`<body>`), an `<article>`, a `<section>`, an `<aside>` or a `<nav>`. A footer typically contains information about the section it belongs to: who wrote it, copyright data, links to related documents, contact details, a site map, or navigation back to the top.

```
<footer>...</footer>
```


<form> — Form

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<form>` element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the action attribute using the HTTP method in the method attribute.

```
<form action="URL" method="post">
  ...controls...
  <button>Submit</button>
</form>
```

<frame> — Frame (obsolete)

Status: OBSOLETE | Versions: Netscape 2 (1996), HTML 4.01 Frameset; obsolete in HTML5

The `<frame>` element defined one rectangular sub-window inside a `<frameset>`, each displaying a separate HTML document. Frames were used in the late 1990s to keep a navigation menu fixed while the content scrolled. They caused many problems: bookmarks and the back button did not work as expected, search engines could not index the pages properly, printing was unreliable and screen readers struggled. HTML5 removed `<frame>` and `<frameset>`. Use CSS for layout and `<iframe>` when you truly need to embed another document.

```
<frameset cols="200,*">
  <frame src="menu.html" name="menu">
  <frame src="main.html" name="main">
</frameset>
```

<frameset> — Frameset (obsolete)

Status: OBSOLETE | Versions: Netscape 2, HTML 4.01 Frameset; obsolete in HTML5

The `<frameset>` element replaced `<body>` in a frames document and divided the window into rows and columns, each filled by a `<frame>` or a nested `<frameset>`. It required the HTML 4.01 Frameset DOCTYPE. It is obsolete in HTML5. See `<frame>` for the reasons and the modern replacement.

```
<frameset rows="80,*">...</frameset>
```

<h1> — Headings h1 to h6

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<h1>` to `<h6>` elements represent six levels of section headings. `<h1>` is the highest (most important) level and `<h6>` the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```

<head> — Document head

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<head>` element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and `<meta>` tags for search engines and social networks. Nothing inside `<head>` is displayed on the page itself, except the title, which appears in the browser tab.

```
<head>
  <meta charset="utf-8">
  <title>Title</title>
</head>
```

<header> — Header

Status: CURRENT | Versions: HTML5

The `<header>` element represents introductory content for its nearest sectioning ancestor or for the page: typically a group of headings, a logo, a search form, the author's name, and the main navigation. A page usually has one header at the top of `<body>`, but articles and sections may have their own headers too.

```
<header>...</header>
```

<hgroup> — Heading group

Status: CURRENT | Versions: HTML5 (removed from W3C HTML 5.0, kept in the Living Standard with a new definition)

The `<hgroup>` element groups a heading (`<h1>`-`<h6>`) with one or more `<p>` elements that contain a subtitle, alternative title, tagline or other secondary text. It tells the browser that the paragraphs belong to the heading rather than being ordinary content.

```
<hgroup>
  <h1>Title</h1>
  <p>Subtitle</p>
</hgroup>
```

<hr> — Thematic break (horizontal rule)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<hr>` element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

```
<hr>
```

<html> — Document root

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<html>` element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: `<head>` followed by `<body>`. It comes immediately after the `<!DOCTYPE html>` declaration.

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

<i> — Idiomatic text (italic)

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<i>` element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

```
<i>text</i>
```

<iframe> — Inline frame

Status: CURRENT | Versions: Internet Explorer 3 (1996), HTML 4.01, HTML5

The `<iframe>` element embeds another HTML page inside the current page, in a rectangular area called a nested browsing context. Each `iframe` has its own document, its own window object and its own history. Common uses are embedding videos from YouTube, maps, advertisements, payment forms, social media widgets and third-party tools.

```
<iframe src="URL" title="description" width="600" height="400"></
iframe>
```

** — Image**

Status: CURRENT | Versions: Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<img>` element embeds an image in the document. It is a void element; the image file is identified by the `src` attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The `alt` attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

```

```

<input> — Input control

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<input>` element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the `type` attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

```
<input type="text" name="field" id="field">
```

<ins> — Inserted text

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<ins>` element marks text that has been added to a document, to show changes between versions. Browsers underline it by default. It is the counterpart of `<del>`. Like `<del>` it accepts optional `cite` and `datetime` attributes that record why and when the change was made.

```
<ins datetime="YYYY-MM-DD">inserted text</ins>
```

<isindex> — Search index prompt (obsolete)

Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<isindex>` element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a `<form>` with an `<input type="search">`.

```
<isindex prompt="Search: ">
```

<kbd> — Keyboard input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<kbd>` element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting `<kbd>` elements inside a `<kbd>` represents individual keys in a key combination; wrapping `<kbd>` in `<samp>` represents input echoed by the system.

```
<kbd>Ctrl</kbd> + <kbd>S</kbd>
```

<keygen> — Key-pair generator (obsolete)

Status: OBSOLETE | Versions: Netscape, briefly in HTML5; removed

The `<keygen>` element generated a public/private key pair when a form was submitted, for client certificate enrolment. It came from Netscape, was added to HTML5 and then removed from the standard in 2017 and from all browsers. There is no direct replacement; use the Web Crypto API.

```
<keygen name="key" challenge="...">
```

<label> — Form control label

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<label>` element gives a caption to a form control. Associating a label with a control has two big benefits: clicking the label focuses or toggles the control (a much larger click target for checkboxes and radio buttons), and screen readers announce the label when the control gets focus.

```
<label for="id">Text</label> <input id="id">
<!-- or -->
<label>Text <input></label>
```

<legend> — Fieldset caption

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<legend>` element provides a caption for its parent `<fieldset>`. It must be the first child of the fieldset. Browsers draw it on top of the fieldset's border. Screen readers announce the legend together with the label of each control inside the group, which is why `fieldset/legend` is the recommended way to caption groups of radio buttons or checkboxes.

```
<fieldset>
  <legend>Caption</legend>
  ...
</fieldset>
```

** — List item**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

```
<ul>
  <li>Item</li>
</ul>
```

<link> — External resource link

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<link>` element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (`rel="stylesheet"`), but it is also used for site icons (`rel="icon"`), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in `<head>`, though body-ok link types such as stylesheet may appear in `<body>`.

```
<link rel="stylesheet" href="style.css">
```

<main> — Main content

Status: CURRENT | Versions: HTML 5.1 (2016)

The `<main>` element represents the dominant content of the `<body>`: the content that is directly related to, or expands upon, the central topic of the page or the central function of the application. It excludes content repeated across pages such as the site header, navigation, sidebars, search box, logo and footer.

```
<main>...</main>
```


<map> — Image map

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<map>` element, together with `<area>` elements, defines a client-side image map: an image with several clickable regions each linking to a different destination. The map is connected to an `<img>` through the `img`'s `usemap` attribute, which must be `#` followed by the map's name.

```
<map name="n">
  <area ...>
</map>
<img usemap="#n" ...>
```

<mark> — Highlighted (marked) text

Status: CURRENT | Versions: HTML5

The `<mark>` element represents text that is marked or highlighted for reference because of its relevance in the current context. Typical uses: highlighting the search terms in a page of results, or drawing attention to a passage in a quotation that the current text discusses. Browsers render it with a yellow background, like a highlighter pen.

```
<mark>text</mark>
```

<marquee> — Scrolling text (obsolete)

Status: OBSOLETE | Versions: Internet Explorer 2 (1995), never standard; obsolete in HTML5 (but its behaviour is specified for compatibility)

The `<marquee>` element scrolled its content horizontally or vertically across the screen. It was a Microsoft extension that became infamous alongside Netscape's `<blink>`. HTML5 lists it as obsolete but, because so many old pages use it, browsers still implement it. Moving text is hard to read and distracting; if you need an animation, use CSS animations with a pause control and respect the `prefers-reduced-motion` media query.

```
<marquee>text</marquee>
```

<math> — MathML root

Status: CURRENT | Versions: HTML5 (inline MathML)

The `<math>` element is the top-level element of MathML (Mathematical Markup Language), which can be written directly inside HTML5 documents to display mathematical formulas. Inside `<math>` you use MathML elements such as `<mi>` (identifier), `<mn>` (number), `<mo>` (operator), `<mrow>` (group), `<mfrac>` (fraction), `<msup>` (superscript), `<msqrt>` (square root) and `<mtable>`. All modern browsers support MathML Core.

```
<math>
  <mi>x</mi><mo>=</mo><mn>2</mn>
</math>
```

<menu> — Menu (toolbar list)

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<menu>` element is a semantic alternative to `<ul>` for a list of commands or actions, such as a toolbar of buttons. Each command is an `<li>` containing a `<button>` or similar control. Browsers render it identically to `<ul>`.

```
<menu>
  <li><button>Action</button></li>
</menu>
```

<menuitem> — Menu item (obsolete)

Status: OBSOLETE | Versions: HTML5 draft (Firefox only); removed 2017

The `<menuitem>` element represented a command in a `<menu type="context">`, letting a page add custom items to the browser's right-click menu. Only Firefox implemented it; it was removed from the standard in 2017 and from Firefox in 2020. Build custom menus with lists, buttons and JavaScript (or the popover attribute) instead.

```
<menuitem label="Refresh" onclick="...">
```

<meta> — Metadata

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<meta>` element represents metadata about the document that cannot be expressed by `<title>`, `<base>`, `<link>`, `<style>` or `<script>`. It is a void element placed in `<head>`. Depending on which attributes are present it does one of four things: declares the character encoding (charset); provides document-level metadata as a name/content pair (name); simulates an HTTP response header (http-equiv); or supplies microdata (itemprop).

```
<meta charset="utf-8">
<meta name="description" content="...">
```

<meter> — Scalar measurement (gauge)

Status: CURRENT | Versions: HTML5

The `<meter>` element represents a scalar measurement within a known range, or a fractional value: disk usage, the relevance of a search result, password strength, a score out of ten, temperature within safe limits. Browsers render it as a gauge bar whose colour changes depending on whether the value is in the optimum, suboptimal or bad region defined by low, high and optimum.

```
<meter value="0.6">60%</meter>
```

<nav> — Navigation section

Status: CURRENT | Versions: HTML5

The `<nav>` element represents a section of the page whose purpose is to provide navigation links, either within the current document or to other documents. Typical uses are the main site menu, a table of contents, breadcrumbs, pagination and 'previous/next' links.

```
<nav>
  <ul>
    <li><a href="...">Link</a></li>
  </ul>
</nav>
```

<noframes> — Fallback for frames (obsolete)

Status: OBSOLETE | Versions: HTML 4.01 Frameset; obsolete in HTML5

The `<noframes>` element contained content to display in browsers that did not support frames. It became obsolete together with `<frameset>` and `<frame>` in HTML5.

```
<noframes><body>fallback</body></noframes>
```

<noscript> — Fallback when scripting is disabled

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<noscript>` element defines content to be shown only if scripting is disabled in the browser or the browser does not support scripting. When scripting is enabled the content is ignored entirely (in fact the parser treats it as raw text). It may be used in `<head>` (containing only `<link>`, `<style>` and `<meta>`) and in `<body>`.

```
<noscript>fallback content</noscript>
```

<object> — External object

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<object>` element represents an external resource, treated as an image, a nested browsing context (like an `iframe`) or a resource handled by a plug-in, depending on its type. It was introduced in HTML 4.01 as a universal replacement for `<img>`, `<applet>` and `<embed>`. Unlike `<embed>`, it accepts fallback content between its tags, shown if the resource cannot be displayed, and it may contain `<param>` elements (now obsolete) that pass parameters to plug-ins.

```
<object data="URL" type="MIME" width="W" height="H">fallback</object>
```

** — Ordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

<optgroup> — Option group

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<optgroup>` element groups related `<option>` elements inside a `<select>` under a non-selectable heading given by the `label` attribute. Groups cannot be nested. Disabling an `optgroup` disables all its options.

```
<select>
  <optgroup label="Group">
    <option>...</option>
  </optgroup>
</select>
```

<option> — Option

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<option>` element defines one item in a `<select>` drop-down list, an `<optgroup>`, or a `<datalist>` of suggestions. Its text content is what the user sees; the `value` attribute is what is submitted with the form. If `value` is omitted the text content is submitted instead.

```
<option value="v">Text</option>
```

<output> — Calculation or user-action result

Status: CURRENT | Versions: HTML5

The `<output>` element is a container into which a site or app can inject the result of a calculation or the outcome of a user action, for example the total price in a cart, the current value of a range slider, or a status message. It is form-associated and labelable, and it is an ARIA live region by default (`role="status"`), which means screen readers announce changes to its content automatically.

```
<output name="result" for="a b">0</output>
```

<p> — Paragraph

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

```
<p>Text of the paragraph.</p>
```

<param> — Object parameter (deprecated)

Status: DEPRECATED | Versions: HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard

The `<param>` element defined a parameter for an `<object>` element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

```
<object data="movie.swf">  
  <param name="quality" value="high">  
</object>
```

<picture> — Picture (responsive image container)

Status: CURRENT | Versions: HTML 5.1 (2016)

The `<picture>` element contains zero or more `<source>` elements and exactly one `<img>` element, and lets the browser choose the most suitable image from several candidates. The browser examines each `<source>` in order and uses the first whose media query matches and whose type it supports; if none match it falls back to the `<img>`. The `<img>` element is required: it is what is actually displayed, and it carries the alt text, width, height, loading and other attributes.

```
<picture>
  <source srcset="..." media="..." type="...">
  
</picture>
```

<pre> — Pre-formatted text

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<pre>` element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

```
<pre>
  text with
    preserved   spacing
</pre>
```


<progress> — Progress indicator

Status: CURRENT | Versions: HTML5

The `<progress>` element displays the completion progress of a task, such as a file upload, a download, a multi-step form or a loading operation. It is rendered as a progress bar. With a value attribute it shows a determinate bar (a known percentage); without value it shows an indeterminate, animated bar meaning 'working, but I do not know how long it will take'.

```
<progress value="30" max="100">30%</progress>
```

<q> — Inline quotation

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<q>` element indicates a short inline quotation that sits within a sentence. Browsers automatically add quotation marks around it, choosing the correct style for the language declared with `lang` (curly double quotes in English, guillemets in French, and so on), so you should not type quotation marks yourself. For long quotations that need their own paragraph use `<blockquote>`.

```
<q cite="URL">quoted words</q>
```

<rb> — Ruby base (deprecated)

Status: DEPRECATED | Versions: XHTML 1.1 Ruby, HTML 5.1; removed from the Living Standard

The `<rb>` element explicitly marked the base text component of a ruby annotation, the text that the `<rt>` annotation is placed above. The Living Standard removed it because the base text can simply be written directly inside `<ruby>`. Browsers still support it.

```
<ruby><rb>□</rb><rt>kan</rt></ruby>
```

<rp> — Ruby fallback parenthesis

Status: CURRENT | Versions: HTML5

The `<rp>` element provides fallback parentheses for browsers that do not support ruby annotations. Browsers that support ruby hide the `<rp>` content; older browsers display it, so the annotation appears in brackets after the base text instead of above it. Place one `<rp>` before and one after each `<rt>`.

```
<ruby>□<rp>(</rp><rt>kan</rt><rp>)</rp></ruby>
```

<rt> — Ruby text (annotation)

Status: CURRENT | Versions: HTML5

The `<rt>` element specifies the ruby text component of a ruby annotation: the small text placed above (or beside) the base characters to show pronunciation, translation or transliteration. It is used for East Asian typography (furigana in Japanese, pinyin/zhuyin in Chinese) and increasingly for other languages. It must be inside a `<ruby>` element.

```
<ruby>base<rt>annotation</rt></ruby>
```

<rtc> — Ruby text container (deprecated)

Status: DEPRECATED | Versions: HTML 5.1; removed from the Living Standard

The `<rtc>` element grouped several `<rt>` annotations so that a base text could have two layers of annotation (for example pronunciation above and meaning below). Only Firefox implemented it, and the Living Standard removed it. Nest two `<ruby>` elements for double-sided annotations.

```
<ruby>base<rtc><rt>a</rt></rtc><rtc><rt>b</rt></rtc></ruby>
```

<ruby> — Ruby annotation

Status: CURRENT | Versions: XHTML 1.1 (2001), HTML5

The `<ruby>` element represents small annotations rendered above, below or beside base text, most often used to show the pronunciation of East Asian characters. The name comes from a small typographic size called 'ruby' used by British printers. The base text is written directly inside `<ruby>`, each annotation in an `<rt>`, and optional fallback parentheses in `<rp>`.

```
<ruby>base<rp>(</rp><rt>annotation</rt><rp>)</rp></ruby>
```

<s> — Strikethrough (no longer accurate)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The `<s>` element renders text with a line through it and represents content that is no longer accurate or no longer relevant, such as an old price next to the new one, or a sold-out item. In HTML 4 it was a purely presentational strike-through synonym for `<strike>` and was deprecated; HTML5 gave it this meaning and made it valid again while removing `<strike>`.

```
<s>outdated text</s>
```

<samp> — Sample output

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

```
<samp>program output</samp>
```

<script> — Script

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<script>` element embeds executable code, almost always JavaScript, or links to an external script file with the `src` attribute. It can also hold data blocks (JSON, templates, import maps, speculation rules) that the browser does not execute when type is set to a non-script MIME type.

```
<script src="app.js" defer></script>
<script>
  // inline code
</script>
```

<section> — Generic section

Status: CURRENT | Versions: HTML5

The `<section>` element represents a generic, thematic grouping of content, typically with a heading. Examples: the chapters of an article, the tabs of a tabbed interface, the 'Introduction', 'Features' and 'Pricing' areas of a landing page. Each section should be identifiable, usually by an `<h2>`-`<h6>` heading as its first child.

```
<section>
  <h2>Heading</h2>
  ...
</section>
```

<select> — Drop-down / list box

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<select>` element creates a control for choosing one option (or, with multiple, several) from a list. It contains `<option>` elements, optionally grouped in `<optgroup>`. By default it appears as a drop-down menu; with the `multiple` attribute or a `size` greater than 1 it appears as a scrolling list box. The `name` attribute and the value of the chosen option(s) are submitted with the form.

```
<select name="n">
  <option value="1">One</option>
</select>
```

<slot> — Shadow DOM slot

Status: CURRENT | Versions: HTML5 / DOM Standard (2016)

The `<slot>` element is a placeholder inside a web component's shadow tree that is filled with content from the light DOM (the markup written by the user of the component). A named slot receives children that have a matching `slot` attribute; the unnamed default slot receives all other children. Content inside the `<slot>` element itself is fallback content shown when nothing is slotted.

```
<slot name="title">Default title</slot>
```

<small> — Side comment (fine print)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5 (redefined)

The `<small>` element represents side comments and small print: disclaimers, caveats, legal restrictions, copyright notices, attribution. It renders the text one size smaller. In HTML 3.2 and 4 it was purely presentational ('smaller text'); HTML5 kept it and gave it this meaning, while its counterpart `<big>` was removed.

```
<small>fine print</small>
```

<source> — Media or image source

Status: CURRENT | Versions: HTML5

The `<source>` element specifies one of several alternative resources for a `<picture>`, `<audio>` or `<video>` element. It is a void element. The browser examines the sources in order and uses the first one it can display: for `<picture>` it checks the media query and image type, for `<audio>`/`<video>` it checks the type (MIME type and codecs). Because it is used inside three different parents, the set of relevant attributes differs: `<picture>` uses `srcset`, `sizes`, `media`, `type`, `width` and `height`; media elements use `src`, `type` and (for video) `media`.

```
<source src="file" type="MIME"> <!-- in audio/video -->
<source srcset="file" type="MIME" media="query"> <!-- in picture -->
```

** — Generic inline container**

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<span>` element is a generic inline container for phrasing content. Like `<div>`, it has no meaning of its own and no visual effect; it exists to be a hook for CSS (via `class` or `id`) or JavaScript, or to carry attributes such as `lang` or `dir` on a part of a sentence. It should be used only when no other element is more appropriate: `<em>`, `<strong>`, `<mark>`, `<time>`, `<abbr>`, `<code>` and the other text-level elements all carry meaning that `<span>` does not.

```
<span class="name">text</span>
```

<strike> — Strikethrough (obsolete)

Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

The `<strike>` element drew a line through its text. It was purely presentational, was deprecated in HTML 4.01 and removed in HTML5. Use `<del>` for removed content, `<s>` for content that is no longer accurate, or CSS `text-decoration: line-through` for pure styling.

```
<strike>text</strike>
```

** — Strong importance**

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

```
<strong>important text</strong>
```

<style> — Embedded style sheet

Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

The `<style>` element contains CSS rules that apply to the document. It belongs in `<head>`, though browsers tolerate it in `<body>`. A document may have any number of `<style>` elements; they are applied in order together with any linked style sheets. For anything larger than a few rules, an external file loaded with `<link rel="stylesheet">` is better because it can be cached and shared between pages.

```
<style>
  selector { property: value; }
</style>
```

<sub> — Subscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sub>` element renders text as a subscript: smaller and below the baseline. Use it only where the subscript has a typographic meaning, such as chemical formulas (H₂O), mathematical variable indices (x₁) or footnote markers in some styles. For purely visual lowering, use CSS `vertical-align`.

```
H<sub>2</sub>0
```

<summary> — Disclosure summary

Status: CURRENT | Versions: HTML5 (5.1)

The `<summary>` element specifies the visible heading, label or caption of a `<details>` element. Clicking it (or pressing Enter/Space when it is focused) toggles the details open and closed. It must be the first child of `<details>`. Browsers show a small triangle marker before it, which can be styled or removed with the `::marker` pseudo-element or `list-style: none`.

```
<details>
  <summary>Label</summary>
  ...
</details>
```

<sup> — Superscript

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<sup>` element renders text as a superscript: smaller and above the baseline. Use it where the superscript has meaning: exponents (x²), ordinal suffixes (1st), footnote references, and abbreviations like M^{lle}. For visual raising only, use CSS.

```
x<sup>2</sup>
```


<svg> — Scalable Vector Graphics root

Status: CURRENT | Versions: HTML5 (inline SVG)

The `<svg>` element is the root of an inline SVG (Scalable Vector Graphics) image written directly in the HTML. SVG describes shapes with elements such as `<circle>`, `<rect>`, `<line>`, `<path>`, `<text>` and `<g>`, which scale to any size without losing sharpness, can be styled with CSS and animated with CSS or JavaScript. Inline SVG is ideal for icons, logos, charts and diagrams. SVG files can also be used through ``, but then CSS from the page cannot style their insides.

```
<svg viewBox="0 0 100 100" width="100" height="100">
  <circle cx="50" cy="50" r="40"/>
</svg>
```

<table> — Table

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<table>` element represents tabular data: information arranged in a two-dimensional grid of rows and columns, where the position of each cell gives it meaning (this value belongs to this row and this column). Examples: price lists, timetables, statistics, comparison charts, spreadsheets.

```
<table>
  <caption>Title</caption>
  <thead><tr><th>Head</th></tr></thead>
  <tbody><tr><td>Data</td></tr></tbody>
</table>
```

<tbody> — Table body

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<tbody>` element groups the main data rows of a table, as distinct from the header rows in `<thead>` and the summary rows in `<tfoot>`. A table may have several `<tbody>` elements to group rows into sections (for example one per category), which can then be styled separately. When you write `<tr>` directly inside `<table>` without a `<tbody>`, the browser creates one automatically, so a `tbody` is always present in the DOM.

```
<tbody>
  <tr>...</tr>
</tbody>
```

<td> — Table data cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<td>` element defines a cell of a table that contains data. It must be inside a `<tr>`. Each `<td>` occupies one column by default; `colspan` makes it span several columns and `rowspan` several rows. The `headers` attribute can list the ids of the `<th>` cells that describe it, which is needed only for complex tables where `scope` is not enough.

```
<tr>
  <td>Data</td>
</tr>
```

<template> — Content template

Status: CURRENT | Versions: HTML5 (2013)

The `<template>` element holds HTML that is not rendered when the page loads but can be cloned and inserted into the document later by JavaScript. The content is parsed into a separate document fragment (`template.content`): scripts inside do not run, images do not load and the markup is invisible. It is the standard way to define reusable markup for lists, cards and web components.

```
<template id="t">
  <li class="item"></li>
</template>
```

<textarea> — Multi-line text input

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<textarea>` element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike `<input>`, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

```
<textarea name="n" rows="4" cols="40">initial text</textarea>
```

<tfoot> — Table footer

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<tfoot>` element groups the rows that summarise the columns of a table: totals, averages, counts. In HTML5 it is placed after the `<tbody>` elements (HTML 4.01 required it before them so that the browser could render the footer before receiving all data). When printing a long table, browsers can repeat the tfoot at the bottom of every page.

```
<tfoot>
  <tr><th>Total</th><td>260</td></tr>
</tfoot>
```

<th> — Table header cell

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<th>` element defines a header cell in a table: a cell that labels a column or a row rather than holding data. Browsers render it bold and centred. The `scope` attribute states whether the header applies to its column (`col`), its row (`row`), a group of columns (`colgroup`) or a group of rows (`rowgroup`). Browsers can usually guess for simple tables, but stating it explicitly is best practice because screen readers use the header text when reading each data cell.

```
<th scope="col">Header</th>
```

<thead> — Table header

Status: CURRENT | Versions: HTML 4.01, HTML5

The `<thead>` element groups the rows that hold the column headers of a table. It comes after any `<caption>` and `<colgroup>` and before `<tbody>` and `<tfoot>`. Browsers repeat the `thead` at the top of each printed page of a long table, and CSS position: sticky on its cells keeps the headers visible while scrolling. A table may have at most one `thead`.

```
<thead>
  <tr><th>Column</th></tr>
</thead>
```

<time> — Date / time

Status: CURRENT | Versions: HTML5

The `<time>` element represents a specific period in time: a date, a time of day, a date and time, a duration, a week or a month. The optional `datetime` attribute holds the value in a machine-readable format so that browsers, search engines, calendars and scripts can understand it, while the visible content can be written in any human-friendly way ('11 September 2026', 'yesterday', '□□ □□□□□□'). If `datetime` is omitted, the text content itself must be in a valid format.

```
<time datetime="2026-09-11">11 September 2026</time>
```

<title> — Document title

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<title>` element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one `<title>`, inside `<head>`.

```
<title>Page title</title>
```

<tr> — Table row

Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

The `<tr>` element defines a row of cells in a table. It contains `<th>` (header) and/or `<td>` (data) cells and is placed inside `<table>`, `<thead>`, `<tbody>` or `<tfoot>`. Rows are the fundamental unit of table construction: you build a table one row at a time, and the columns emerge from the cells in each row.

```
<tr>
  <td>Cell 1</td>
  <td>Cell 2</td>
</tr>
```

<track> — Text track for media

Status: CURRENT | Versions: HTML5

The <track> element adds timed text tracks to <audio> or <video>: subtitles (translations), captions (dialogue plus sound descriptions for deaf and hard-of-hearing viewers), descriptions (spoken description of the video for blind users), chapters (navigation) and metadata (for scripts). Tracks are WebVTT files (.vtt). It is a void element placed after any <source> elements.

```
<track kind="captions" src="captions.vtt" srclang="en"
label="English" default>
```

<tt> — Teletype text (obsolete)

Status: OBSOLETE | Versions: HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5

The <tt> element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements <code>, <kbd>, <samp> or <var>, or CSS font-family: monospace on a .

```
<tt>text</tt>
```

<u> — Unarticulated annotation (underline)

Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

The <u> element represents a span of inline text that should be rendered in a way that indicates a non-textual annotation: most commonly, marking misspelled words (as a word processor does with a wavy red line) or labelling proper names in Chinese text. Browsers render it with a solid underline. In HTML 4 <u> simply meant 'underline' and was deprecated; HTML5 redefined it.

```
<u>text</u>
```

** — Unordered list**

Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<ul>` element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an `<li>`. If the order matters, use `<ol>`.

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

<var> — Variable

Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

The `<var>` element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with `<code>`, `<kbd>` and `<samp>`.

```
<var>x</var>
```

<video> — Embedded video

Status: CURRENT | Versions: HTML5

The `<video>` element embeds a video player in the document. The video file is given by `src` or by one or more `<source>` children (so that the browser can pick a supported format), followed by optional `<track>` elements for captions and subtitles, and finally fallback content for browsers without video support. Common formats are MP4 (H.264/AAC, universally supported), WebM (VP9/AV1, smaller) and, in Safari, HEVC.

```
<video src="clip.mp4" controls width="640" height="360"></video>
```


<wbr> — Word break opportunity

Status: CURRENT | Versions: Netscape / IE extension, standardised in HTML5

The `<wbr>` element marks a position inside a word where the browser may break the line if necessary, without inserting a hyphen or a visible character. It is useful for very long words, URLs and code identifiers that would otherwise overflow their container. If no break is needed, nothing is displayed. It is a void element.

Super<wbr>cali<wbr>fragilistic

The HTML Living Standard (2014-today)

Since 2019 the WHATWG Living Standard is the single HTML specification; there will be no 'HTML6'. Features are added continuously once two browser engines ship them: `<dialog>`, `<template>`, `<slot>`, `<picture>`, `<search>`, the popover attribute, lazy loading, customisable select and more.

Elements in this chapter: 13

Element	Meaning	Status today
<code><dialog></code>	Dialog box	current
<code><fencedframe></code>	Fenced frame (experimental)	experimental
<code><hgroup></code>	Heading group	current
<code><main></code>	Main content	current
<code><menuitem></code>	Menu item (obsolete)	obsolete
<code><param></code>	Object parameter (deprecated)	deprecated
<code><picture></code>	Picture (responsive image container)	current
<code><portal></code>	Portal (experimental, abandoned)	experimental
<code><rb></code>	Ruby base (deprecated)	deprecated
<code><rtc></code>	Ruby text container (deprecated)	deprecated
<code><search></code>	Search section	current

Element	Meaning	Status today
<code><selectedcontent></code>	Selected option display (customisable select)	experimental
<code><slot></code>	Shadow DOM slot	current

<dialog> — Dialog box

Status: CURRENT | Versions: HTML 5.2 / Living Standard

The `<dialog>` element represents a dialog box or other interactive component such as a confirmation prompt, an alert, a subwindow or a modal overlay. It is hidden by default and is shown with the `open` attribute or, preferably, with the JavaScript methods `show()` (non-modal) and `showModal()` (modal). A modal dialog blocks interaction with the rest of the page, is displayed on top of everything else, and can be closed with the Escape key.

```
<dialog id="d">...</dialog>
<script>document.getElementById('d').showModal()</script>
```

<fencedframe> — Fenced frame (experimental)

Status: EXPERIMENTAL | Versions: Experimental (Chromium, 2024)

The `<fencedframe>` element embeds another HTML page in a way that is even more isolated than an `<iframe>`: the embedded page cannot communicate with the embedding page, and the URL is chosen through privacy-preserving APIs rather than given directly. It is part of the Privacy Sandbox proposals for showing personalised advertisements without cross-site tracking. It is not yet a standard and is only available in Chromium browsers.

```
<fencedframe width="300" height="250"></fencedframe>
```

<hgroup> — Heading group

Status: CURRENT | Versions: HTML5 (removed from W3C HTML 5.0, kept in the Living Standard with a new definition)

The `<hgroup>` element groups a heading (`<h1>-<h6>`) with one or more `<p>` elements that contain a subtitle, alternative title, tagline or other secondary text. It tells the browser that the paragraphs belong to the heading rather than being ordinary content.

```
<hgroup>
  <h1>Title</h1>
  <p>Subtitle</p>
</hgroup>
```

<main> — Main content

Status: CURRENT | Versions: HTML 5.1 (2016)

The `<main>` element represents the dominant content of the `<body>`: the content that is directly related to, or expands upon, the central topic of the page or the central function of the application. It excludes content repeated across pages such as the site header, navigation, sidebars, search box, logo and footer.

```
<main>...</main>
```

<menuitem> — Menu item (obsolete)

Status: OBSOLETE | Versions: HTML5 draft (Firefox only); removed 2017

The `<menuitem>` element represented a command in a `<menu type="context">`, letting a page add custom items to the browser's right-click menu. Only Firefox implemented it; it was removed from the standard in 2017 and from Firefox in 2020. Build custom menus with lists, buttons and JavaScript (or the popover attribute) instead.

```
<menuitem label="Refresh" onclick="...">
```

<param> — Object parameter (deprecated)

Status: DEPRECATED | Versions: HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard

The `<param>` element defined a parameter for an `<object>` element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

```
<object data="movie.swf">
  <param name="quality" value="high">
</object>
```

<picture> — Picture (responsive image container)

Status: CURRENT | Versions: HTML 5.1 (2016)

The `<picture>` element contains zero or more `<source>` elements and exactly one `<img>` element, and lets the browser choose the most suitable image from several candidates. The browser examines each `<source>` in order and uses the first whose media query matches and whose type it supports; if none match it falls back to the `<img>`. The `<img>` element is required: it is what is actually displayed, and it carries the alt text, width, height, loading and other attributes.

```
<picture>
  <source srcset="..." media="..." type="...">
  
</picture>
```

<portal> — Portal (experimental, abandoned)

Status: EXPERIMENTAL | Versions: Chrome experiment (2019-2022); not standardised

The `<portal>` element was an experiment that embedded a preview of another page, like an `iframe`, with the ability to 'activate' it and navigate seamlessly into it with an animation. It was available only behind a flag in Chrome and the work has been superseded by the View Transitions API and prerendering with the Speculation Rules API. Do not use it.

```
<portal src="https://example.com"></portal>
```

<rb> — Ruby base (deprecated)

Status: DEPRECATED | Versions: XHTML 1.1 Ruby, HTML 5.1; removed from the Living Standard

The `<rb>` element explicitly marked the base text component of a ruby annotation, the text that the `<rt>` annotation is placed above. The Living Standard removed it because the base text can simply be written directly inside `<ruby>`. Browsers still support it.

```
<ruby><rb>□</rb><rt>kan</rt></ruby>
```

<rtc> — Ruby text container (deprecated)

Status: DEPRECATED | Versions: HTML 5.1; removed from the Living Standard

The `<rtc>` element grouped several `<rt>` annotations so that a base text could have two layers of annotation (for example pronunciation above and meaning below). Only Firefox implemented it, and the Living Standard removed it. Nest two `<ruby>` elements for double-sided annotations.

```
<ruby>base<rtc><rt>a</rt></rtc><rtc><rt>b</rt></rtc></ruby>
```

<search> — Search section

Status: CURRENT | Versions: Living Standard (2023)

The <search> element represents a part of the document whose contents are search or filtering controls: a site search form, a filter panel in a product list, or the controls of a find-in-page feature. It has no visual effect; it exposes the 'search' landmark to assistive technology. It replaces the old pattern <form role="search">. The form itself still goes inside it.

```
<search>
  <form>...</form>
</search>
```

<selectedcontent> — Selected option display (customisable select)

Status: EXPERIMENTAL | Versions: Living Standard (2025, experimental)

The <selectedcontent> element is part of the customisable <select> proposal. Placed inside the <button> that sits at the start of a <select>, it mirrors the content of the currently selected <option>, so that rich option content (icons, colour swatches, descriptions) is shown in the closed control as well as in the open list. The browser clones the selected option's children into <selectedcontent> whenever the selection changes.

```
<select>
  <button><selectedcontent></selectedcontent></button>
  <option>...</option>
</select>
```

<slot> — Shadow DOM slot

Status: CURRENT | Versions: HTML5 / DOM Standard (2016)

The `<slot>` element is a placeholder inside a web component's shadow tree that is filled with content from the light DOM (the markup written by the user of the component). A named slot receives children that have a matching slot attribute; the unnamed default slot receives all other children. Content inside the `<slot>` element itself is fallback content shown when nothing is slotted.

```
<slot name="title">Default title</slot>
```


Global Attributes

Attributes that can be used on any HTML element.

Global attributes

The attributes in this part are 'global': they may be specified on every HTML element, even those where they have no obvious effect. They are listed alphabetically. Event handler attributes (on*) are in Part 5.

Attribute	Values
accesskey	single character
autocapitalize	off none on sentences words characters
autocorrect	on off
autofocus	boolean
class	space-separated class names
contenteditable	true false plaintext-only (empty = true)
data-*	any text
dir	ltr rtl auto
draggable	true false
enterkeyhint	enter done go next previous search send
exportparts	comma-separated part names (optionally name:alias)
hidden	boolean until-found
id	unique identifier
inert	boolean
inputmode	none text decimal numeric tel search email url
is	custom element name
itemid, itemprop, itemref, itemscope, itemtype	various

Attribute	Values
lang	BCP 47 language tag
nonce	cryptographic number used once
part	space-separated part names
popover	auto manual hint (empty = auto)
role	ARIA role name
aria-*	various
slot	slot name
spellcheck	true false
style	CSS declarations
tabindex	integer
title	text
translate	yes no (empty = yes)
virtualkeyboardpolicy	auto manual
writingsuggestions	true false
xml:lang, xml:base, xml:space	various

Global attribute: accesskey

Values: single character | Since: HTML 4.01

Description

Defines a keyboard shortcut that activates or focuses the element. The key combination depends on the browser and operating system (usually Alt + key on Windows/Linux in Chrome and Edge, Alt + Shift + key in Firefox, Control + Option + key on macOS). Because these combinations often collide with browser and assistive-technology shortcuts, accesskey is rarely recommended; if used, document the shortcuts on the page.

Example

```
<button accesskey="s">Save (Alt+S)</button>
```

Where it can be used

On every HTML element, including <html>, <head> and <body>, and on elements inside <template> and shadow trees. Elements where the attribute has no practical effect (for example tabindex on <meta>) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: accesskey.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with element.getAttribute('accesskey') or, for most global attributes, a property of the same name.

Global attribute: autocapitalize

Values: off | none | on | sentences | words | characters | Since: Living Standard

Description

Controls whether and how text input is automatically capitalised on virtual keyboards (phones and tablets). It has no effect on physical keyboards. Inherited by descendant editable elements.

Example

```
<input name="username" autocapitalize="none">
<textarea autocapitalize="sentences"></textarea>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `autocapitalize`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('autocapitalize')` or, for most global attributes, a property of the same name.

Global attribute: autocorrect

Values: on | off | Since: Living Standard

Description

Controls whether the browser may automatically correct spelling in editable text. Originally a Safari extension, added to the Living Standard in 2024.

Example

```
<input name="code" autocorrect="off">
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `autocorrect`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('autocorrect')` or, for most global attributes, a property of the same name.

Global attribute: autofocus

Values: boolean | Since: HTML5 (global since 2019)

Description

Indicates that the element should receive focus as soon as the page loads (or the <dialog> it is in is shown). Only one element per document should have it. Use with care: moving focus automatically can disorient screen reader users and scrolls the page to the element.

Example

```
<input name="q" type="search" autofocus>
```

Where it can be used

On every HTML element, including <html>, <head> and <body>, and on elements inside <template> and shadow trees. Elements where the attribute has no practical effect (for example tabindex on <meta>) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: autofocus.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('autofocus')` or, for most global attributes, a property of the same name.

Global attribute: class

Values: space-separated class names | Since: HTML 4.01

Description

A list of the classes the element belongs to. Classes are the primary hook for CSS (selector `.name`) and for JavaScript (`document.querySelectorAll('.name')`). An element may have many classes and many elements may share a class. Class names are case-sensitive, should not start with a digit, and by convention use lowercase words joined with hyphens (kebab-case). Choose names that describe what the element is, not how it looks (product-card rather than red-box), so that the style can change without renaming.

Example

```
<div class="card featured">...</div>
<style>.card { border: 1px solid #ccc; } .featured { border-color:
gold; }</style>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: class.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('class')` or, for most global attributes, a property of the same name.

Global attribute: contenteditable

Values: true | false | plaintext-only | (empty = true) | Since: HTML5

Description

Makes the element's content editable by the user directly in the page, turning it into a basic rich-text editor. `plaintext-only` allows editing text but not formatting. The property is inherited: children of an editable element are editable unless they set `contenteditable="false"`. Combine with the `input` event and the `document.execCommand` or `Selection` APIs to build editors.

Example

```
<p contenteditable="true">Click here and edit this text.</p>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `contenteditable`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('contenteditable')` or, for most global

attributes, a property of the same name.

Global attribute: data-*

Values: any text | Since: HTML5

Description

Custom data attributes. Any attribute whose name starts with data- is reserved for the page author's own data and is guaranteed never to be given a meaning by browsers. The value is available in JavaScript through `element.dataset` (data-user-id becomes `dataset.userId`) and in CSS through `attr()`. Use them to attach information to elements without misusing class or id.

Example

```
<li data-product-id="42" data-price="120">Learn HTML</li>
<script>
  const li = document.querySelector('li');
  console.log(li.dataset.productId, li.dataset.price); // "42" "120"
</script>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: data-.*.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('data-*')` or, for most global attributes, a property of the same name.

Global attribute: dir

Values: ltr | rtl | auto | Since: HTML 4.01

Description

The text direction of the element's content. ltr = left to right (English, French...); rtl = right to left (Arabic, Hebrew, Persian, Urdu); auto = let the browser decide from the first strongly directional character, useful for user-generated content. Set it on <html> for the whole page. It affects text alignment, the side on which bullets and scrollbars appear, and the order of table columns.

Example

```
<html lang="ar" dir="rtl">
<p dir="ltr">This English paragraph inside an Arabic page reads left
to right.</p>
<input dir="auto">
```

Where it can be used

On every HTML element, including <html>, <head> and <body>, and on elements inside <template> and shadow trees. Elements where the attribute has no practical effect (for example tabindex on <meta>) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: dir.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with element.getAttribute('dir') or, for most global attributes, a property of the same name.

Global attribute: draggable

Values: true | false | Since: HTML5

Description

Whether the element can be dragged with the Drag and Drop API. Images and links are draggable by default; other elements need `draggable="true"`. The value must be written explicitly (it is not a boolean attribute). Dragging fires `dragstart`, `drag`, `dragend` on the source and `dragenter`, `dragover`, `drop` on the target.

Example

```
<div draggable="true" ondragstart="event.dataTransfer.setData('text',  
'hello')">Drag me</div>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `draggable`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('draggable')` or, for most global attributes, a property of the same name.

Global attribute: enterkeyhint

Values: enter | done | go | next | previous | search | send | Since: Living Standard

Description

A hint about which label or icon to show on the Enter key of virtual keyboards, matching the action that pressing Enter will perform in this field.

Example

```
<input type="search" enterkeyhint="search">
<textarea enterkeyhint="send"></textarea>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `enterkeyhint`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('enterkeyhint')` or, for most global attributes, a property of the same name.

Global attribute: exportparts

Values: comma-separated part names (optionally name:alias) | Since: CSS Shadow Parts

Description

In web components: re-exports parts from a nested shadow tree so that they can be styled from outside with the `::part()` pseudo-element.

Example

```
<my-dialog exportparts="header, footer:dialog-footer"></my-dialog>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `exportparts`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('exportparts')` or, for most global attributes, a property of the same name.

Global attribute: hidden

Values: boolean | until-found | Since: HTML5

Description

Indicates that the element is not (yet) relevant and should not be rendered. Browsers implement it as `display: none`, so any CSS that sets `display` on the element will override it (add `[hidden] { display: none !important }` to be safe). The value `until-found` keeps the content hidden but searchable with `find-in-page` and fragment links; the browser reveals it automatically when found. Do not use `hidden` to hide content only visually while keeping it for screen readers; use a `visually-hidden` CSS class instead.

Example

```
<p hidden>Not shown.</p>
<section hidden="until-found">Revealed when the user searches for
text inside it.</section>
<script>document.querySelector('p').hidden = false;</script>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `hidden`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('hidden')` or, for most global attributes, a property of the same name.

Global attribute: id

Values: unique identifier | Since: HTML 4.01

Description

A unique identifier for the element within the document. No two elements may share an id. It is used as the target of fragment links (`href="#id"`), by CSS (`#id`), by JavaScript (`getElementById`), by `<label for>`, by `aria-labelledby` and `aria-describedby`, by `<input list>`, by `<output for>`, by the `form` attribute and by `<map name>`. Ids are case-sensitive, must not contain spaces, and should start with a letter.

Example

```
<h2 id="pricing">Pricing</h2>
<a href="#pricing">Jump to pricing</a>
<label for="email">Email</label> <input id="email">
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `id`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('id')` or, for most global attributes, a property of the same name.

Global attribute: inert

Values: boolean | Since: Living Standard (2022)

Description

Makes the element and all its descendants inert: they cannot be focused, clicked, selected or found by find-in-page, and they are hidden from assistive technology. It is the tool for disabling everything behind a custom modal dialog or an open off-canvas menu. Native `<dialog> showModal()` applies inertness to the rest of the page automatically.

Example

```
<main inert>Page content that cannot be interacted with while the  
menu is open</main>  
<nav class="drawer">Menu</nav>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `inert`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('inert')` or, for most global attributes, a property of the same name.

Global attribute: inputmode

Values: none | text | decimal | numeric | tel | search | email | url | Since: HTML5

Description

A hint about which virtual keyboard to display when the element (an input, textarea or contenteditable) is focused. Unlike changing type, it does not change validation or the value: `inputmode="numeric"` on a text input shows a number pad but still accepts a postal code with a leading zero.

Example

```
<input name="zip" inputmode="numeric" pattern="[0-9]*">  
<input name="amount" inputmode="decimal">
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `inputmode`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('inputmode')` or, for most global attributes, a property of the same name.

Global attribute: is

Values: custom element name | Since: HTML5 / Custom Elements

Description

Declares that a built-in element should behave like a customised built-in element defined with `customElements.define(name, class, { extends: 'button' })`.
Not supported in Safari.

Example

```
<button is="fancy-button">Click</button>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `is`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('is')` or, for most global attributes, a property of the same name.

Global attribute: itemid, itemprop, itemref, itemscope, itemtype

Values: various | Since: HTML5 Microdata

Description

Microdata attributes that embed machine-readable structured data (for example schema.org vocabularies) into the page for search engines. `itemscope` creates an item; `itemtype` gives its type URL; `itemprop` names a property; `itemid` gives a global identifier; `itemref` includes properties located elsewhere. JSON-LD in a `<script type="application/ld+json">` is the more common approach today.

Example

```
<div itemscope itemtype="https://schema.org/Book">
  <span itemprop="name">Learn HTML</span> by
  <span itemprop="author">Sara</span>
</div>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `itemid`, `itemprop`, `itemref`, `itemscope`, `itemtype`.
- Put the value in double quotes unless it is a boolean attribute.

- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('itemid, itemprop, itemref, itemscope, itemtype')` or, for most global attributes, a property of the same name.

Global attribute: lang

Values: BCP 47 language tag | Since: HTML 4.01

Description

The language of the element's content. Language tags consist of a two-letter language code (en, ar, fr, de, es, zh, ja) optionally followed by a script (zh-Hans) and/or a region (en-GB, ar-EG, pt-BR). Set it on `<html>` for the page and on any element whose language differs. Browsers use it for hyphenation, quotation marks, font choice and spell checking; screen readers use it to switch voices; translation tools use it to decide what to translate. An empty value means the language is unknown.

Example

```
<html lang="en">
<p>The word <span lang="ar">كتاب</span> means book.</p>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `lang`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('lang')` or, for most global attributes, a property of the same name.

Global attribute: nonce

Values: cryptographic number used once | Since: CSP

Description

A random token generated by the server for each response and listed in the Content-Security-Policy header, so that only the inline `<script>` and `<style>` elements carrying that nonce are allowed to run. Browsers hide the value from scripts after parsing.

Example

```
<script nonce="r4nd0m">/* allowed by CSP 'nonce-r4nd0m' */</script>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `nonce`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('nonce')` or, for most global attributes, a property of the same name.

Global attribute: part

Values: space-separated part names | Since: CSS Shadow Parts

Description

In web components: names an element inside a shadow tree so that it can be styled from outside with the `::part(name)` pseudo-element.

Example

```
<!-- inside the shadow tree -->
<button part="confirm">OK</button>
<!-- outside -->
<style>my-dialog::part(confirm) { background: green; }</style>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `part`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('part')` or, for most global attributes, a property of the same name.

Global attribute: popover

Values: auto | manual | hint | (empty = auto) | Since: Living Standard (2023)

Description

Turns the element into a popover: hidden by default and displayed in the top layer (above everything else) when shown with a button that has `popovertarget`, or with `showPopover()/togglePopover()` in JavaScript. auto popovers close when the user clicks outside or presses Escape and only one is open at a time; manual popovers must be closed explicitly. Style the backdrop with `::backdrop` and the open state with `:popover-open`.

Example

```
<button popovertarget="menu">Menu</button>
<div id="menu" popover>
  <a href="/profile">Profile</a>
  <a href="/logout">Log out</a>
</div>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `popover`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('popover')` or, for most global attributes, a property of the same name.

Global attribute: role

Values: ARIA role name | Since: WAI-ARIA 1.0

Description

From WAI-ARIA: overrides or specifies the semantic role of the element for assistive technology, for example `role="button"`, `role="navigation"`, `role="alert"`, `role="tab"`. The first rule of ARIA is not to use it when a native element already has the right role: prefer `<button>` over `<div role="button">`. Use it for widgets HTML lacks (tabs, trees, comboboxes) and for landmarks in older browsers.

Example

```
<div role="alert">Your order was placed.</div>
<ul role="tablist"><li role="tab" aria-selected="true">Details</li></ul>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `role`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('role')` or, for most global attributes, a property of the same name.

Global attribute: aria-*

Values: various | Since: WAI-ARIA 1.0

Description

From WAI-ARIA: a family of attributes that describe the state and properties of elements for assistive technology. The most used: aria-label (accessible name from a string), aria-labelledby (name from another element), aria-describedby (description), aria-hidden (hide from AT), aria-expanded (open/closed), aria-current (current item), aria-live (announce changes), aria-controls, aria-selected, aria-checked, aria-disabled, aria-required, aria-invalid, aria-haspopup, aria-pressed, aria-busy, aria-modal.

Example

```
<button aria-label="Close" aria-expanded="false" aria-  
controls="panel">x</button>  
<div id="status" aria-live="polite"></div>
```

Where it can be used

On every HTML element, including <html>, <head> and <body>, and on elements inside <template> and shadow trees. Elements where the attribute has no practical effect (for example tabindex on <meta>) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: aria-*
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('aria-*')` or, for most global attributes, a property of the same name.

Global attribute: slot

Values: slot name | Since: DOM Standard

Description

Assigns the element to a named `<slot>` inside the shadow tree of its parent custom element.

Example

```
<user-card>
  <span slot="name">Ahmed</span>
</user-card>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `slot`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('slot')` or, for most global attributes, a property of the same name.

Global attribute: spellcheck

Values: true | false | Since: HTML5

Description

Whether the browser should check the spelling and grammar of the element's editable text. Browsers decide the default per element (usually on for `textarea` and `contenteditable`, off for `input`). Turn it off for codes, usernames and passwords. Note: some browsers send checked text to a remote service.

Example

```
<textarea spellcheck="true"></textarea>
<input name="coupon" spellcheck="false">
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `spellcheck`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('spellcheck')` or, for most global attributes, a property of the same name.

Global attribute: style

Values: CSS declarations | Since: HTML 4.01

Description

Inline CSS applied to this element only. Declarations are written as property: value pairs separated by semicolons, exactly as inside a style sheet rule but without the selector and braces. Inline styles have higher specificity than any selector and are hard to override, cannot use media queries or pseudo-classes, and are blocked by strict Content Security Policies, so prefer classes and a style sheet. Inline styles are convenient for quick tests and for values computed by JavaScript.

Example

```
<p style="color: red; font-size: 1.2em; margin-bottom: 0">Inline  
styled paragraph</p>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `style`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('style')` or, for most global attributes, a property of the same name.

Global attribute: tabindex

Values: integer | Since: HTML 4.01

Description

Controls whether the element can be focused with the keyboard (Tab key) and in what order. `tabindex="0"` makes a normally non-focusable element (such as a `div`) focusable, in the natural document order. `tabindex="-1"` makes it focusable by script (`element.focus()`) but not by Tab; use it for the target of a skip link or a container that receives focus programmatically. Positive values (1, 2, 3...) force a custom order before all other elements and should be avoided because they make the page confusing. Interactive elements (links, buttons, form controls) are focusable without `tabindex`.

Example

```
<div tabindex="0" role="button">Focusable custom control</div>
<main id="content" tabindex="-1">...</main>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `tabindex`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('tabindex')` or, for most global attributes, a property of the same name.

Global attribute: title

Values: text | Since: HTML 4.01

Description

Advisory information about the element, shown by most desktop browsers as a tooltip when the mouse rests on it. On `<abbr>` it holds the expansion; on `<link rel=stylesheet>` it names an alternate style sheet; on `<iframe>` it is the accessible name. Tooltips are not accessible to touch-screen or keyboard users and are read inconsistently by screen readers, so never put essential information only in title. Do not repeat the visible text in title; it produces double announcements.

Example

```
<abbr title="World Health Organization">WHO</abbr>  
<button title="Save (Ctrl+S)">Save</button>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `title`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.

- In JavaScript read it with `element.getAttribute('title')` or, for most global attributes, a property of the same name.

Global attribute: translate

Values: yes | no | (empty = yes) | Since: HTML5

Description

Whether the element's text and translatable attribute values should be translated when the page is localised by machine translation tools. Set `translate="no"` on brand names, code, and addresses.

Example

```
<p>Buy it at <span translate="no">Book Store</span> today.</p>
<code translate="no">getElementById</code>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `translate`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('translate')` or, for most global attributes, a property of the same name.

Global attribute: virtualkeyboardpolicy

Values: auto | manual | Since: VirtualKeyboard API

Description

For editable elements: whether the virtual keyboard opens automatically on focus (auto) or only when the script calls `navigator.virtualKeyboard.show()` (manual).

Example

```
<div contenteditable virtualkeyboardpolicy="manual"></div>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `virtualkeyboardpolicy`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('virtualkeyboardpolicy')` or, for most global attributes, a property of the same name.

Global attribute: **writingsuggestions**

Values: true | false | Since: Living Standard (2024)

Description

Whether the browser may offer writing suggestions (predictive text, autocomplete of sentences) in editable elements.

Example

```
<textarea writingsuggestions="false"></textarea>
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `writingsuggestions`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('writingsuggestions')` or, for most global attributes, a property of the same name.

Global attribute: `xml:lang`, `xml:base`, `xml:space`

Values: various | Since: XHTML 1.0

Description

XML attributes that only have meaning in XHTML documents served as XML. In HTML they are ignored (`xml:lang` must match `lang` if both are present).

Example

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
```

Where it can be used

On every HTML element, including `<html>`, `<head>` and `<body>`, and on elements inside `<template>` and shadow trees. Elements where the attribute has no practical effect (for example `tabindex` on `<meta>`) still accept it without a validation error, but such use should be avoided.

Checklist

- Write the name in lower case: `xml:lang`, `xml:base`, `xml:space`.
- Put the value in double quotes unless it is a boolean attribute.
- Do not repeat the same attribute twice on one start tag; browsers keep the first one only.
- In JavaScript read it with `element.getAttribute('xml:lang', 'xml:base', 'xml:space')` or, for most global attributes, a property of the same name.

Event Handler Attributes

Every on* attribute and the event that triggers it.

Event handler attributes

Every HTML element accepts event handler content attributes whose value is JavaScript code run when the event fires. The attribute name is 'on' followed by the event name. In modern code it is better to attach handlers with `element.addEventListener('click', fn)` in a script, which keeps behaviour out of the markup and allows several handlers per event, but the attributes remain valid and are useful for quick prototypes.

```
<!-- attribute style -->
<button onclick="alert('Hi')">Say hi</button>

<!-- script style (preferred) -->
<button id="hi">Say hi</button>
<script>
  document.getElementById('hi').addEventListener('click', () => alert(
    'Hi'));
</script>
```

Attribute	Event	Applies to
onabort	abort	img, audio, video, and others
onauxclick	auxclick	All elements
onbeforeinput	beforeinput	input, textarea, contenteditable
onbeforematch	beforematch	All elements
onbeforetoggle	beforetoggle	popover elements, details, dialog
onblur	blur	Focusable elements
oncancel	cancel	dialog, input type=file
oncanplay	canplay	audio, video
oncanplaythrough	canplaythrough	audio, video

Attribute	Event	Applies to
onchange	change	input, select, textarea
onclick	click	All elements
onclose	close	dialog, popover
oncontextmenu	contextmenu	All elements
oncopy, oncut, onpaste	copy, cut, paste	All elements
oncuechange	cuechange	track
ondblclick	dblclick	All elements
ondrag, ondragend, ondragenter, ondragleave, ondragover, ondragstart, ondrop	drag events	All elements
ondurationchange	durationchange	audio, video
onemptied	emptied	audio, video
onended	ended	audio, video
onerror	error	img, script, link, audio, video, body
onfocus	focus	Focusable elements
onformdata	formdata	form
oninput	input	input, select, textarea, contenteditable
oninvalid	invalid	Form controls
onkeydown	keydown	Focusable elements, document
onkeypress	keypress	-
onkeyup	keyup	Focusable elements
onload	load	img, script, link, iframe, body, object

Attribute	Event	Applies to
onloadeddata	loadeddata	audio, video
onloadedmetadata	loadedmetadata	audio, video
onloadstart	loadstart	audio, video
onmousedown, onmouseup	mousedown, mouseup	All elements
onmouseenter, onmouseleave	mouseenter, mouseleave	All elements
onmousemove	mousemove	All elements
onmouseover, onmouseout	mouseover, mouseout	All elements
onpause, onplay, onplaying	pause, play, playing	audio, video
onpointerdown, onpointerup, onpointermove, onpointerover, onpointerout, onpointerenter, onpointerleave, onpointercancel, ongotpointercapture, onlostpointercapture	pointer events	All elements
onprogress	progress	audio, video
onratechange	ratechange	audio, video
onreset	reset	form
onresize	resize	body, video
onscroll	scroll	Scrollable elements, body
onscrollend	scrollend	Scrollable elements, body
onsecuritypolicyviolation	securitypolicyviolation	All elements

Attribute	Event	Applies to
onseeked, onseeking	seeked, seeking	audio, video
onselect	select	input, textarea
onselectionchange	selectionchange	document, input, textarea
onslotchange	slotchange	slot
onstalled	stalled	audio, video
onsubmit	submit	form
onsuspend	suspend	audio, video
ontimeupdate	timeupdate	audio, video
ontoggle	toggle	details, popover elements
ontouchstart, ontouchmove, ontouchend, ontouchcancel	touch events	All elements
onvolumechange	volumechange	audio, video
onwaiting	waiting	audio, video
onwheel	wheel	All elements
onanimationstart, onanimationend, onanimationiteration, onanimationcancel	animation events	All elements
ontransitionstart, ontransitionend, ontransitionrun, ontransitioncancel	transition events	All elements
oncontextlost, oncontextrestored	contextlost, contextrestored	canvas

Attribute	Event	Applies to
Window events on <body>: onafterprint, onbeforeprint, onbeforeunload, onhashchange, onlanguagechange, onmessage, onmessageerror, onoffline, ononline, onpagehide, onpagereveal, onpageshow, onpageswap, onpopstate, onrejectionhandled, onstorage, onunhandledrejection, onunload	window events	body

Event attribute: onabort

Event name: abort | Applies to: img, audio, video, and others

Fires when

Loading of a resource was aborted.

Attribute form

```
<div onabort="console.log('abort fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('abort', function (event) {  
    console.log('abort fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onauxclick

Event name: auxclick | Applies to: All elements

Fires when

A non-primary mouse button (middle, right) was clicked.

Attribute form

```
<div onauxclick="console.log('auxclick fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('auxclick', function (event) {  
  console.log('auxclick fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onbeforeinput

Event name: beforeinput | Applies to: input, textarea, contenteditable

Fires when

Just before the value of an editable element changes; can be cancelled.

Attribute form

```
<div onbeforeinput="console.log('beforeinput fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('beforeinput', function (event) {  
    console.log('beforeinput fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onbeforematch

Event name: beforematch | Applies to: All elements

Fires when

Content hidden with `hidden=until-found` is about to be revealed by find-in-page.

Attribute form

```
<div onbeforematch="console.log('beforematch fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('beforematch', function (event) {  
    console.log('beforematch fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: **onbeforetoggle**

Event name: beforetoggle | Applies to: popover elements, details, dialog

Fires when

A popover or <details> is about to open or close; cancellable for popovers.

Attribute form

```
<div onbeforetoggle="console.log('beforetoggle fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('beforetoggle', function (event) {  
    console.log('beforetoggle fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onblur

Event name: blur | Applies to: Focusable elements

Fires when

The element lost focus. Does not bubble; use focusout for delegation.

Attribute form

```
<div onblur="console.log('blur fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('blur', function (event) {  
    console.log('blur fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: oncancel

Event name: cancel | Applies to: dialog, input type=file

Fires when

The user pressed Escape in a <dialog>, or cancelled a file picker.

Attribute form

```
<div oncancel="console.log('cancel fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('cancel', function (event) {  
    console.log('cancel fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: oncanplay

Event name: canplay | Applies to: audio, video

Fires when

Enough media data is available to start playing.

Attribute form

```
<div oncanplay="console.log('canplay fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('canplay', function (event) {  
    console.log('canplay fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: `oncanplaythrough`

Event name: `canplaythrough` | Applies to: audio, video

Fires when

The media can probably play to the end without buffering.

Attribute form

```
<div oncanplaythrough="console.log('canplaythrough fired', event)">...
</div>
```

Script form (recommended)

```
element.addEventListener('canplaythrough', function (event) {
    console.log('canplaythrough fired on', event.target);
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onchange

Event name: change | Applies to: input, select, textarea

Fires when

The value of a form control was committed: after a text field loses focus with a changed value, or immediately when a checkbox, radio, select or file input changes. Use input for live updates.

Attribute form

```
<div onchange="console.log('change fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('change', function (event) {  
    console.log('change fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onclick

Event name: click | Applies to: All elements

Fires when

The element was clicked with the primary button, tapped, or activated with Enter/Space on a button or link. The most used event attribute.

Attribute form

```
<div onclick="console.log('click fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('click', function (event) {  
    console.log('click fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onclose

Event name: close | Applies to: dialog, popover

Fires when

A <dialog> was closed, or a popover was hidden.

Attribute form

```
<div onclose="console.log('close fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('close', function (event) {  
  console.log('close fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: `oncontextmenu`

Event name: `contextmenu` | Applies to: All elements

Fires when

The user requested the context menu (right-click, long press, Shift+F10). Preventing default hides the browser menu.

Attribute form

```
<div oncontextmenu="console.log('contextmenu fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('contextmenu', function (event) {  
  console.log('contextmenu fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: oncopy, oncut, onpaste

Event name: copy, cut, paste | Applies to: All elements

Fires when

Clipboard operations on the element.

Attribute form

```
<div oncopy, oncut, onpaste="console.log('copy, cut, paste fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('copy, cut, paste', function (event) {  
    console.log('copy, cut, paste fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: **oncuechange**

Event name: `cuechange` | Applies to: `track`

Fires when

The active text track cues changed.

Attribute form

```
<div oncuechange="console.log('cuechange fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('cuechange', function (event) {  
  console.log('cuechange fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable `'event'` refers to the Event object and `'this'` refers to the element.
- Returning `false` from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: ondblclick

Event name: dblclick | Applies to: All elements

Fires when

Double click. Not fired on touch in most browsers.

Attribute form

```
<div ondblclick="console.log('dblclick fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('dblclick', function (event) {  
    console.log('dblclick fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: ondrag, ondragend, ondragenter, ondragleave, ondragover, ondragstart, ondrop

Event name: drag events | Applies to: All elements

Fires when

Stages of a drag-and-drop operation: dragstart/drag/dragend on the dragged element, dragenter/dragover/dragleave/drop on the drop target. Call `preventDefault` in dragover to allow dropping.

Attribute form

```
<div ondrag, ondragend, ondragenter, ondragleave, ondragover,
ondragstart, ondrop="console.log('drag events fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('drag events', function (event) {
  console.log('drag events fired on', event.target);
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: ondurationchange

Event name: durationchange | Applies to: audio, video

Fires when

The media duration attribute was updated.

Attribute form

```
<div ondurationchange="console.log('durationchange fired', event)">...  
</div>
```

Script form (recommended)

```
element.addEventListener('durationchange', function (event) {  
    console.log('durationchange fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onemptied

Event name: emptied | Applies to: audio, video

Fires when

The media element became empty (for example after a network error or `load()`).

Attribute form

```
<div onemptied="console.log('emptied fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('emptied', function (event) {  
    console.log('emptied fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onended

Event name: ended | Applies to: audio, video

Fires when

Playback reached the end.

Attribute form

```
<div onended="console.log('ended fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('ended', function (event) {  
  console.log('ended fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onerror

Event name: error | Applies to: img, script, link, audio, video, body

Fires when

A resource failed to load, or a script error occurred (on body/window).

Attribute form

```
<div onerror="console.log('error fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('error', function (event) {  
    console.log('error fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onfocus

Event name: focus | Applies to: Focusable elements

Fires when

The element received focus. Does not bubble; use focusin for delegation.

Attribute form

```
<div onfocus="console.log('focus fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('focus', function (event) {  
    console.log('focus fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onformdata

Event name: formdata | Applies to: form

Fires when

The form's data is being constructed for submission; lets scripts add entries.

Attribute form

```
<div onformdata="console.log('formdata fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('formdata', function (event) {  
    console.log('formdata fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: oninput

Event name: input | Applies to: input, select, textarea, contenteditable

Fires when

The value of an editable element changed, on every keystroke or change.

Attribute form

```
<div oninput="console.log('input fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('input', function (event) {  
    console.log('input fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: oninvalid

Event name: invalid | Applies to: Form controls

Fires when

A control failed validation on submission.

Attribute form

```
<div oninvalid="console.log('invalid fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('invalid', function (event) {  
    console.log('invalid fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onkeydown

Event name: `keydown` | Applies to: Focusable elements, document

Fires when

A key was pressed down. Repeats while held. Use `event.key`.

Attribute form

```
<div onkeydown="console.log('keydown fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('keydown', function (event) {  
  console.log('keydown fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable `'event'` refers to the Event object and `'this'` refers to the element.
- Returning `false` from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onkeypress

Event name: keypress | Applies to: -

Fires when

Deprecated. A character key was pressed. Use keydown.

Deprecated

Attribute form

```
<div onkeypress="console.log('keypress fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('keypress', function (event) {  
    console.log('keypress fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onkeyup

Event name: keyup | Applies to: Focusable elements

Fires when

A key was released.

Attribute form

```
<div onkeyup="console.log('keyup fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('keyup', function (event) {  
    console.log('keyup fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onload

Event name: load | Applies to: img, script, link, iframe, body, object

Fires when

The resource (image, script, style sheet, frame, or the whole page on body) finished loading.

Attribute form

```
<div onload="console.log('load fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('load', function (event) {  
    console.log('load fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onloadeddata

Event name: loadeddata | Applies to: audio, video

Fires when

The first frame of media has loaded.

Attribute form

```
<div onloadeddata="console.log('loadeddata fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('loadeddata', function (event) {  
    console.log('loadeddata fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onloadedmetadata

Event name: loadedmetadata | Applies to: audio, video

Fires when

Duration and dimensions are known.

Attribute form

```
<div onloadedmetadata="console.log('loadedmetadata fired', event)">...  
</div>
```

Script form (recommended)

```
element.addEventListener('loadedmetadata', function (event) {  
    console.log('loadedmetadata fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onloadstart

Event name: loadstart | Applies to: audio, video

Fires when

The browser started loading media.

Attribute form

```
<div onloadstart="console.log('loadstart fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('loadstart', function (event) {  
    console.log('loadstart fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onmousedown, onmouseup

Event name: mousedown, mouseup | Applies to: All elements

Fires when

A mouse button was pressed or released.

Attribute form

```
<div onmousedown, onmouseup="console.log('mousedown, mouseup fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('mousedown, mouseup', function (event) {  
    console.log('mousedown, mouseup fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onmouseenter, onmouseleave

Event name: mouseenter, mouseleave | Applies to: All elements

Fires when

The pointer entered or left the element (does not bubble, not fired for children).

Attribute form

```
<div onmouseenter, onmouseleave="console.log('mouseenter, mouseleave fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('mouseenter, mouseleave', function (event) {  
    console.log('mouseenter, mouseleave fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onmousemove

Event name: mousemove | Applies to: All elements

Fires when

The pointer moved over the element. Fires very often.

Attribute form

```
<div onmousemove="console.log('mousemove fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('mousemove', function (event) {  
    console.log('mousemove fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onmouseover, onmouseout

Event name: mouseover, mouseout | Applies to: All elements

Fires when

The pointer entered or left the element or one of its children (bubbles).

Attribute form

```
<div onmouseover, onmouseout="console.log('mouseover, mouseout fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('mouseover, mouseout', function (event) {  
    console.log('mouseover, mouseout fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onpause, onplay, onplaying

Event name: pause, play, playing | Applies to: audio, video

Fires when

Playback paused / play() called / actually started after buffering.

Attribute form

```
<div onpause, onplay, onplaying="console.log('pause, play, playing fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('pause, play, playing', function (event) {  
    console.log('pause, play, playing fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute:

onpointerdown, onpointerup, onpointermove, onpointerover, onpointerout, onpointerenter, onpointerleave, onpointercancel, ongotpointercapture, onlostpointercapture

Event name: pointer events | Applies to: All elements

Fires when

Unified events for mouse, touch and pen input. Prefer them over mouse and touch events.

Attribute form

```
<div onpointerdown, onpointerup, onpointermove, onpointerover,  
onpointerout, onpointerenter, onpointerleave, onpointercancel,  
ongotpointercapture, onlostpointercapture="console.log('pointer  
events fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('pointer events', function (event) {  
    console.log('pointer events fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).

- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onprogress

Event name: progress | Applies to: audio, video

Fires when

Periodic progress while downloading media.

Attribute form

```
<div onprogress="console.log('progress fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('progress', function (event) {  
    console.log('progress fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onratechange

Event name: ratechange | Applies to: audio, video

Fires when

Playback speed changed.

Attribute form

```
<div onratechange="console.log('ratechange fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('ratechange', function (event) {  
  console.log('ratechange fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onreset

Event name: reset | Applies to: form

Fires when

The form was reset.

Attribute form

```
<div onreset="console.log('reset fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('reset', function (event) {  
    console.log('reset fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onresize

Event name: `resize` | Applies to: `body`, `video`

Fires when

The window was resized (on `body`); or an element resized (video). Use `ResizeObserver` for elements.

Attribute form

```
<div onresize="console.log('resize fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('resize', function (event) {  
    console.log('resize fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable `'event'` refers to the Event object and `'this'` refers to the element.
- Returning `false` from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onscroll

Event name: scroll | Applies to: Scrollable elements, body

Fires when

The element or document scrolled.

Attribute form

```
<div onscroll="console.log('scroll fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('scroll', function (event) {  
    console.log('scroll fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onscrollend

Event name: scrollend | Applies to: Scrollable elements, body

Fires when

Scrolling finished.

Attribute form

```
<div onscrollend="console.log('scrollend fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('scrollend', function (event) {  
    console.log('scrollend fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onsecuritypolicyviolation

Event name: securitypolicyviolation | Applies to: All elements

Fires when

A Content Security Policy rule was violated.

Attribute form

```
<div onsecuritypolicyviolation="console.log('securitypolicyviolation fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('securitypolicyviolation', function (event) {  
    console.log('securitypolicyviolation fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onseeked, onseeking

Event name: seeked, seeking | Applies to: audio, video

Fires when

A seek operation finished / started.

Attribute form

```
<div onseeked, onseeking="console.log('seeked, seeking fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('seeked, seeking', function (event) {  
    console.log('seeked, seeking fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onselect

Event name: select | Applies to: input, textarea

Fires when

Text was selected inside an input or textarea.

Attribute form

```
<div onselect="console.log('select fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('select', function (event) {  
    console.log('select fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onselectionchange

Event name: selectionchange | Applies to: document, input, textarea

Fires when

The document selection changed.

Attribute form

```
<div onselectionchange="console.log('selectionchange fired', event)">
  ..</div>
```

Script form (recommended)

```
element.addEventListener('selectionchange', function (event) {
  console.log('selectionchange fired on', event.target);
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onslotchange

Event name: slotchange | Applies to: slot

Fires when

The nodes assigned to a slot changed.

Attribute form

```
<div onslotchange="console.log('slotchange fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('slotchange', function (event) {  
    console.log('slotchange fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onstalled

Event name: stalled | Applies to: audio, video

Fires when

The browser is trying to fetch media data but none is arriving.

Attribute form

```
<div onstalled="console.log('stalled fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('stalled', function (event) {  
    console.log('stalled fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onsubmit

Event name: submit | Applies to: form

Fires when

The form is about to be submitted; call `preventDefault()` to handle it with JavaScript.

Attribute form

```
<div onsubmit="console.log('submit fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('submit', function (event) {  
    console.log('submit fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onsuspend

Event name: suspend | Applies to: audio, video

Fires when

Media loading was intentionally suspended.

Attribute form

```
<div onsuspend="console.log('suspend fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('suspend', function (event) {  
    console.log('suspend fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: `ontimeupdate`

Event name: `timeupdate` | Applies to: audio, video

Fires when

The playback position changed (several times per second).

Attribute form

```
<div ontimeupdate="console.log('timeupdate fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('timeupdate', function (event) {  
  console.log('timeupdate fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable `'event'` refers to the Event object and `'this'` refers to the element.
- Returning `false` from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: ontoggle

Event name: toggle | Applies to: details, popover elements

Fires when

A <details> element or popover opened or closed.

Attribute form

```
<div ontoggle="console.log('toggle fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('toggle', function (event) {  
  console.log('toggle fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: `ontouchstart`, `ontouchmove`, `ontouchend`, `ontouchcancel`

Event name: touch events | Applies to: All elements

Fires when

Touch screen interactions. Prefer pointer events.

Attribute form

```
<div ontouchstart, ontouchmove, ontouchend, ontouchcancel="console.  
log('touch events fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('touch events', function (event) {  
    console.log('touch events fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: onvolumechange

Event name: volumechange | Applies to: audio, video

Fires when

Volume or muted state changed.

Attribute form

```
<div onvolumechange="console.log('volumechange fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('volumechange', function (event) {  
    console.log('volumechange fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onwaiting

Event name: waiting | Applies to: audio, video

Fires when

Playback stopped because the next frame is not available.

Attribute form

```
<div onwaiting="console.log('waiting fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('waiting', function (event) {  
    console.log('waiting fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: onwheel

Event name: wheel | Applies to: All elements

Fires when

A mouse wheel or trackpad scroll gesture.

Attribute form

```
<div onwheel="console.log('wheel fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('wheel', function (event) {  
    console.log('wheel fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: **onanimationstart,** **onanimationend,** **onanimationiteration,** **onanimationcancel**

Event name: animation events | Applies to: All elements

Fires when

CSS animation lifecycle.

Attribute form

```
<div onanimationstart, onanimationend, onanimationiteration,  
onanimationcancel="console.log('animation events fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('animation events', function (event) {  
    console.log('animation events fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: ontransitionstart, ontransitionend, ontransitionrun, ontransitioncancel

Event name: transition events | Applies to: All elements

Fires when

CSS transition lifecycle.

Attribute form

```
<div ontransitionstart, ontransitionend, ontransitionrun,  
ontransitioncancel="console.log('transition events fired', event)">...  
</div>
```

Script form (recommended)

```
element.addEventListener('transition events', function (event) {  
  console.log('transition events fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Event attribute: oncontextlost, oncontextrestored

Event name: contextlost, contextrestored | Applies to: canvas

Fires when

A canvas drawing context was lost or restored.

Attribute form

```
<div oncontextlost, oncontextrestored="console.log('contextlost, contextrestored fired', event)">...</div>
```

Script form (recommended)

```
element.addEventListener('contextlost, contextrestored', function (event) {
    console.log('contextlost, contextrestored fired on', event.target);
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as event.preventDefault()).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; addEventListener is unaffected.

Event attribute: Window events on <body>:

onafterprint, onbeforeprint, onbeforeunload, onhashchange, onlanguagechange, onmessage, onmessageerror, onoffline, ononline, onpagehide, onpagereveal, onpageshow, onpageswap, onpopstate, onrejectionhandled, onstorage, onunhandledrejection, onunload

Event name: window events | Applies to: body

Fires when

Events that fire on the window object and are exposed as attributes on <body> (and <frameset>). See the <body> entry.

Attribute form

<div Window events on <body>: onafterprint, onbeforeprint, onbeforeunload, onhashchange, onlanguagechange, onmessage, onmessageerror, onoffline, ononline, onpagehide, onpagereveal, onpageshow, onpageswap, onpopstate, onrejectionhandled, onstorage,

```
onunhandledrejection, onunload="console.log('window events fired',  
event)">...</div>
```

Script form (recommended)

```
element.addEventListener('window events', function (event) {  
    console.log('window events fired on', event.target);  
});
```

Notes

- Inside the attribute value, the variable 'event' refers to the Event object and 'this' refers to the element.
- Returning false from the attribute code cancels the default action (same as `event.preventDefault()`).
- The attribute value is compiled as the body of a function, so a syntax error is reported only when the event first fires.
- Content Security Policy (CSP) headers that forbid inline scripts also block attribute handlers; `addEventListener` is unaffected.

Element Reference

One complete entry for each of the 144 HTML elements, alphabetically.

Element index

Element	Meaning	Status	Since
<a>	Anchor (hyperlink)	current	HTML 1 (1991)
<abbr>	Abbreviation	current	HTML 4.01
<acronym>	Acronym (obsolete)	obsolete	HTML 4.01
<address>	Contact address	current	HTML 1
<applet>	Java applet (obsolete)	obsolete	HTML 3.2
<area>	Image map area	current	HTML 3.2
<article>	Self-contained article	current	HTML5
<aside>	Aside (sidebar / tangential content)	current	HTML5
<audio>	Embedded audio	current	HTML5
	Bring attention to (bold)	current	HTML 2.0
<base>	Document base URL	current	HTML 2.0
<basefont>	Base font (obsolete)	obsolete	HTML 3.2
<bdi>	Bidirectional isolate	current	HTML5
<bdo>	Bidirectional text override	current	HTML 4.01
<bgsound>	Background sound (non-standard, obsolete)	obsolete	Internet Explorer extension
<big>	Bigger text (obsolete)	obsolete	HTML 3.2
<blink>	Blinking text (non-standard, obsolete)	obsolete	Netscape extension
<blockquote>	Block quotation	current	HTML 2.0
<body>	Document body	current	HTML 2.0

Element	Meaning	Status	Since
 	Line break	current	HTML 1
<button>	Button	current	HTML 4.01
<canvas>	Graphics canvas	current	HTML5
<caption>	Table caption	current	HTML 3.2
<center>	Centred block (obsolete)	obsolete	HTML 3.2
<cite>	Citation (title of a work)	current	HTML 2.0
<code>	Inline code	current	HTML 2.0
<col>	Table column	current	HTML 4.01
<colgroup>	Table column group	current	HTML 4.01
<content>	Shadow DOM insertion point (obsolete)	obsolete	Shadow DOM v0 (Chrome)
<data>	Machine-readable value	current	HTML5
<datalist>	Predefined options for an input	current	HTML5
<dd>	Description details	current	HTML 1
	Deleted text	current	HTML 4.01
<details>	Disclosure widget	current	HTML5 (5.1)
<dfn>	Definition term	current	HTML 3.2
<dialog>	Dialog box	current	HTML 5.2 / Living Standard
<dir>	Directory list (obsolete)	obsolete	HTML 1
<div>	Generic block container	current	HTML 3.2
<dl>	Description list	current	HTML 1
<dt>	Description term	current	HTML 1
	Emphasis	current	HTML 2.0

Element	Meaning	Status	Since
<embed>	External content	current	Netscape extension (1995)
<fencedframe>	Fenced frame (experimental)	experimental	Experimental (Chromium)
<fieldset>	Form field group	current	HTML 4.01
<figcaption>	Figure caption	current	HTML5
<figure>	Figure with optional caption	current	HTML5
	Font (obsolete)	obsolete	HTML 3.2
<footer>	Footer	current	HTML5
<form>	Form	current	HTML 2.0
<frame>	Frame (obsolete)	obsolete	Netscape 2 (1996)
<frameset>	Frameset (obsolete)	obsolete	Netscape 2
<h1>	Headings h1 to h6	current	HTML 1
<head>	Document head	current	HTML 2.0
<header>	Header	current	HTML5
<hgroup>	Heading group	current	HTML5 (removed from W3C HTML 5.0)
<hr>	Thematic break (horizontal rule)	current	HTML 2.0
<html>	Document root	current	HTML 2.0
<i>	Idiomatic text (italic)	current	HTML 2.0
<iframe>	Inline frame	current	Internet Explorer 3 (1996)
<image>	Image (non-standard alias, obsolete)	obsolete	Never standard
	Image	current	Mosaic (1993)

Element	Meaning	Status	Since
<input>	Input control	current	HTML 2.0
<ins>	Inserted text	current	HTML 4.01
<isindex>	Search index prompt (obsolete)	obsolete	HTML 1
<kbd>	Keyboard input	current	HTML 2.0
<keygen>	Key-pair generator (obsolete)	obsolete	Netscape
<label>	Form control label	current	HTML 4.01
<legend>	Fieldset caption	current	HTML 4.01
	List item	current	HTML 1
<link>	External resource link	current	HTML 2.0
<listing>	Listing (obsolete)	obsolete	HTML 1
<main>	Main content	current	HTML 5.1 (2016)
<map>	Image map	current	HTML 3.2
<mark>	Highlighted (marked) text	current	HTML5
<marquee>	Scrolling text (obsolete)	obsolete	Internet Explorer 2 (1995)
<math>	MathML root	current	HTML5 (inline MathML)
<menu>	Menu (toolbar list)	current	HTML 1
<menuitem>	Menu item (obsolete)	obsolete	HTML5 draft (Firefox only)
<meta>	Metadata	current	HTML 2.0
<meter>	Scalar measurement (gauge)	current	HTML5
<multicol>	Multi-column text (non-standard, obsolete)	obsolete	Netscape 3 only
<nav>	Navigation section	current	HTML5

Element	Meaning	Status	Since
<nextid>	Next identifier (HTML 1 editor hint)	obsolete	HTML 1 / HTML 2.0 (RFC 1866)
<nobr>	No line break (non-standard, obsolete)	obsolete	Netscape / IE extension
<noembed>	Fallback for embed (obsolete)	obsolete	Netscape extension
<noframes>	Fallback for frames (obsolete)	obsolete	HTML 4.01 Frameset
<noscript>	Fallback when scripting is disabled	current	HTML 4.01
<object>	External object	current	HTML 4.01
	Ordered list	current	HTML 1
<optgroup>	Option group	current	HTML 4.01
<option>	Option	current	HTML 2.0
<output>	Calculation or user-action result	current	HTML5
<p>	Paragraph	current	HTML 1
<param>	Object parameter (deprecated)	deprecated	HTML 3.2
<picture>	Picture (responsive image container)	current	HTML 5.1 (2016)
<plaintext>	Plain text (obsolete)	obsolete	HTML 1
<portal>	Portal (experimental, abandoned)	experimental	Chrome experiment (2019-2022)
<pre>	Pre-formatted text	current	HTML 1
<progress>	Progress indicator	current	HTML5

Element	Meaning	Status	Since
<q>	Inline quotation	current	HTML 4.01
<rb>	Ruby base (deprecated)	deprecat ed	XHTML 1.1 Ruby
<rp>	Ruby fallback parenthesis	current	HTML5
<rt>	Ruby text (annotation)	current	HTML5
<rtc>	Ruby text container (deprecated)	deprecat ed	HTML 5.1
<ruby>	Ruby annotation	current	XHTML 1.1 (2001)
<s>	Strikethrough (no longer accurate)	current	HTML 3.2
<samp>	Sample output	current	HTML 2.0
<script>	Script	current	HTML 3.2 (reserved)
<search>	Search section	current	Living Standard (2023)
<section>	Generic section	current	HTML5
<select>	Drop-down / list box	current	HTML 2.0
<selected content>	Selected option display (customisable select)	experim ental	Living Standard (2025)
<shadow >	Shadow root insertion point (obsolete)	obsolete	Shadow DOM v0
<slot>	Shadow DOM slot	current	HTML5 / DOM Standard (2016)
<small>	Side comment (fine print)	current	HTML 3.2
<source>	Media or image source	current	HTML5
<spacer>	Spacer (non-standard, obsolete)	obsolete	Netscape 3-4 only
	Generic inline container	current	HTML 4.01
<strike>	Strikethrough (obsolete)	obsolete	HTML 3.2

Element	Meaning	Status	Since
	Strong importance	current	HTML 2.0
<style>	Embedded style sheet	current	HTML 3.2 (reserved)
<sub>	Subscript	current	HTML 3.2
<summary>	Disclosure summary	current	HTML5 (5.1)
<sup>	Superscript	current	HTML 3.2
<svg>	Scalable Vector Graphics root	current	HTML5 (inline SVG)
<table>	Table	current	HTML 3.2
<tbody>	Table body	current	HTML 4.01
<td>	Table data cell	current	HTML 3.2
<template>	Content template	current	HTML5 (2013)
<textarea>	Multi-line text input	current	HTML 2.0
<tfoot>	Table footer	current	HTML 4.01
<th>	Table header cell	current	HTML 3.2
<thead>	Table header	current	HTML 4.01
<time>	Date / time	current	HTML5
<title>	Document title	current	HTML 1
<tr>	Table row	current	HTML 3.2
<track>	Text track for media	current	HTML5
<tt>	Teletype text (obsolete)	obsolete	HTML 2.0
<u>	Unarticulated annotation (underline)	current	HTML 3.2
	Unordered list	current	HTML 1
<var>	Variable	current	HTML 2.0

Element	Meaning	Status	Since
<video>	Embedded video	current	HTML5
<wbr>	Word break opportunity	current	Netscape / IE extension
<xmp>	Example text (obsolete)	obsolete	HTML 1

Elements by category

Category	Elements
Content sectioning	<address>, <article>, <aside>, <footer>, <h1>, <header>, <hgroup>, <main>, <nav>, <search>, <section>
Demarcating edits	, <ins>
Document metadata	<base>, <head>, <link>, <meta>, <style>, <title>
Embedded content	<embed>, <fencedframe>, <iframe>, <object>, <param>, <portal>
Embedded content (MathML)	<math>
Embedded content (SVG)	<svg>
Forms	<button>, <datalist>, <fieldset>, <form>, <input>, <label>, <legend>, <meter>, <optgroup>, <option>, <output>, <progress>, <select>, <selectedcontent>, <textarea>
Image and multimedia	<area>, <audio>, , <map>, <picture>, <source>, <track>, <video>
Interactive elements	<details>, <dialog>, <summary>
Main root	<html>
Obsolete	<nextid>
Obsolete embedded content	<applet>, <bgsound>, <image>, <noembed>
Obsolete forms	<isindex>, <keygen>
Obsolete frames	<frame>, <frameset>, <noframes>
Obsolete interactive	<menuitem>

Category	Elements
Obsolete presentational	<basefont>, <big>, <blink>, <center>, , <marquee>, <multicol>, <nobr>, <spacer>, <strike>, <tt>
Obsolete text content	<dir>, <listing>, <plaintext>, <xmp>
Obsolete text-level	<acronym>
Obsolete web components	<content>, <shadow>
Scripting	<canvas>, <noscript>, <script>
Scripting / web components	<template>
Sectioning root	<body>
Table content	<caption>, <col>, <colgroup>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
Text content	<blockquote>, <dd>, <div>, <dl>, <dt>, <figcaption>, <figure>, <hr>, , <menu>, , <p>, <pre>,
Text-level semantics	<a>, <abbr>, , <bdi>, <bdo>, , <cite>, <code>, <data>, <dfn>, , <i>, <kbd>, <mark>, <q>, <s>, <samp>, <small>, , , <sub>, <sup>, <time>, <u>, <var>, <wbr>
Text-level semantics (ruby)	<rb>, <rp>, <rt>, <rtc>, <ruby>
Web components	<slot>

Quick card: <a>

Anchor (hyperlink)

The <a> element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and <a> was one of the very first elements created by Tim Berners-Lee in 1991.

At a glance

Status	CURRENT
Versions	HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	href, target, rel, download, hreflang, type, referrerpolicy, ping, attributionsrc, charset, coords, name, rev, shape
Related	<area>, <link>, <button>, <nav>

Minimal example

```
<p>Read the <a href="https://developer.mozilla.org/">MDN
documentation</a>.</p>
<p>See our <a href="about.html">About page</a>.</p>
<p>Jump to the <a href="#footer">bottom of this page</a>.</p>
```

Most common mistake

Nesting an <a> inside another <a>, or putting a <button> inside an <a>.

<a> — Anchor (hyperlink)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 1 (1991), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <a> element (anchor) creates a hyperlink to another web page, a file, a location within the same page, an email address, a telephone number, or any other resource that can be identified by a URL. Hyperlinks are the defining feature of the World Wide Web and <a> was one of the very first elements created by Tim Berners-Lee in 1991.

The content of the <a> element is what the visitor sees and clicks. It is usually text, but in HTML5 it may also be an image, a whole card of content, or any other flow content, as long as the link does not contain another link or an interactive element such as a button.

If the href attribute is missing the element is a placeholder: it is rendered as plain text, is not focusable and is not announced as a link by screen readers. This is useful, for example, for the current page in a navigation menu.

In HTML 4 the <a> element was also used to define named anchors () that other links could jump to. In HTML5 the name attribute is obsolete; use the id attribute on any element instead.

Syntax

```
<a href="URL">link text</a>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, interactive content (when href is present), palpable content
Permitted content	Transparent (whatever the parent allows), but it must not contain interactive content or another <a>
Permitted parents	Any element that accepts phrasing content or flow content
Tag omission	Neither the start tag nor the end tag may be omitted
DOM interface	HTMLAnchorElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
href	URL	The address the link points to. It may be absolute (https://example.com/page), relative (about.html, ../images/photo.jpg), a fragment (#section-2), or a special scheme (mailto:, tel:, sms:, javascript:). Without href the element is not a link.	HTML 1

Attribute	Values	Description	Since
target	_self _blank _parent _top frame-name	Where to open the linked document. _self (default) opens it in the same browsing context; _blank opens a new tab or window; _parent opens it in the parent frame; _top opens it in the full window, replacing all frames. A custom name targets an <iframe> with that name.	HTML 4.01
rel	space-separated link types	The relationship between the current document and the linked document. Common values: nofollow, noopener, noreferrer, external, alternate, author, bookmark, help, license, next, prev, search, tag. See the link types table in Part 9.	HTML 4.01
download	empty or filename	Instructs the browser to download the resource instead of navigating to it. If a value is given it is used as the suggested file name. Only works for same-origin URLs and blob:/data: URLs.	HTML5
hreflang	BCP 47 language tag	The language of the linked document, for example en, ar, fr-CA. Purely advisory.	HTML 4.01
type	MIME type	The media type of the linked resource, for example application/pdf. Advisory only.	HTML 4.01

Attribute	Values	Description	Since
referrerpolicy	no-referrer no-referrer-when-downgrade origin origin-when-cross-origin same-origin strict-origin strict-origin-when-cross-origin unsafe-url	How much referrer information is sent when following the link.	HTML5
ping	space-separated URLs	A list of URLs that receive a POST request with the body PING when the link is followed. Used for tracking.	HTML5
attribution src	empty or URL	Experimental: registers an attribution source for the Attribution Reporting API.	Living Standard
charset	character encoding	Obsolete. The character encoding of the linked resource. Use the HTTP Content-Type header instead.	HTML 4.01 (obsolete in HTML5)
coords	comma-separated numbers	Obsolete. Coordinates for use with shape in old image maps. Use <area> instead.	HTML 4.01 (obsolete in HTML5)
name	text	Obsolete. Defined a named anchor that could be linked to with #name. Use id instead.	HTML 1 (obsolete in HTML5)
rev	link types	Obsolete. Reverse relationship. Use rel instead.	HTML 4.01 (obsolete in HTML5)

Attribute	Values	Description	Since
shape	rect circle poly default	Obsolete. Shape of a clickable region. Use <area> instead.	HTML 4.01 (obsolete in HTML5)

<a>: attribute by attribute

Each attribute of <a> explained in detail, with a minimal example you can copy.

href

Values: URL | Since: HTML 1

The address the link points to. It may be absolute (<https://example.com/page>), relative ([about.html](#), [../images/photo.jpg](#)), a fragment (<#section-2>), or a special scheme (<mailto:>, <tel:>, <sms:>, <javascript:>). Without href the element is not a link.

Write href="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a href="https://example.com/page.html">...</a>
```

target

Values: `_self` | `_blank` | `_parent` | `_top` | frame-name | Since: HTML 4.01

Where to open the linked document. `_self` (default) opens it in the same browsing context; `_blank` opens a new tab or window; `_parent` opens it in the parent frame; `_top` opens it in the full window, replacing all frames. A custom name targets an `<iframe>` with that name.

Write target="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a target="_self">...</a>
```

rel

Values: space-separated link types | Since: HTML 4.01

The relationship between the current document and the linked document. Common values: nofollow, noopener, noreferrer, external, alternate, author, bookmark, help, license, next, prev, search, tag. See the link types table in Part 9.

Write `rel="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a rel="value">...</a>
```

download

Values: empty or filename | Since: HTML5

Instructs the browser to download the resource instead of navigating to it. If a value is given it is used as the suggested file name. Only works for same-origin URLs and blob:/data: URLs.

Write `download="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a download="empty">...</a>
```

hreflang

Values: BCP 47 language tag | Since: HTML 4.01

The language of the linked document, for example en, ar, fr-CA. Purely advisory.

Write `hreflang="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a hreflang="en">...</a>
```

type

Values: MIME type | Since: HTML 4.01

The media type of the linked resource, for example application/pdf. Advisory only.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a type="text/plain">...</a>
```

referrerpolicy

Values: no-referrer | no-referrer-when-downgrade | origin | origin-when-cross-origin | same-origin | strict-origin | strict-origin-when-cross-origin | unsafe-url | Since: HTML5

How much referrer information is sent when following the link.

Write referrerpolicy="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a referrerpolicy="no-referrer">...</a>
```

ping

Values: space-separated URLs | Since: HTML5

A list of URLs that receive a POST request with the body PING when the link is followed. Used for tracking.

Write ping="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a ping="space-separated URLs">...</a>
```

attributionsrc

Values: empty or URL | Since: Living Standard

Experimental: registers an attribution source for the Attribution Reporting API.

Write attributionsrc="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a attributionsrc="empty">...</a>
```


charset

Values: character encoding | Since: HTML 4.01 (obsolete in HTML5)

Obsolete. The character encoding of the linked resource. Use the HTTP Content-Type header instead.

Write `charset="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a charset="character encoding">...</a>
```

coords

Values: comma-separated numbers | Since: HTML 4.01 (obsolete in HTML5)

Obsolete. Coordinates for use with `shape` in old image maps. Use `<area>` instead.

Write `coords="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a coords="3">...</a>
```

name

Values: text | Since: HTML 1 (obsolete in HTML5)

Obsolete. Defined a named anchor that could be linked to with `#name`. Use `id` instead.

Write `name="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a name="value">...</a>
```

rev

Values: link types | Since: HTML 4.01 (obsolete in HTML5)

Obsolete. Reverse relationship. Use `rel` instead.

Write `rev="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a rev="link types">...</a>
```

shape

Values: rect | circle | poly | default | Since: HTML 4.01 (obsolete in HTML5)

Obsolete. Shape of a clickable region. Use <area> instead.

Write shape="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<a shape="rect">...</a>
```

<a>: worked examples

Basic links

```
<p>Read the <a href="https://developer.mozilla.org/">MDN
documentation</a>.</p>
<p>See our <a href="about.html">About page</a>.</p>
<p>Jump to the <a href="#footer">bottom of this page</a>.</p>
```

The first link is absolute, the second is relative to the current page, and the third is a fragment link that scrolls to the element whose id is footer.

Opening in a new tab safely

```
<a href="https://example.com" target="_blank" rel="noopener
noreferrer">
    Example (opens in a new tab)
</a>
```

When you use `target="_blank"` add `rel="noopener"` so the new page cannot access your page through `window.opener`. Modern browsers do this automatically, but adding it is still good practice. Tell the user that the link opens in a new tab.

Email, telephone and download links

```
<a href="mailto:info@example.com?subject=Hello">Email us</a>
<a href="tel:+201001234567">Call us: 010 0123 4567</a>
<a href="report.pdf" download="annual-report-2026.pdf">Download the
report (PDF)</a>
```

`mailto:` opens the visitor's email program; `tel:` dials on a phone; `download` forces a save dialog with the suggested file name.

A link wrapping a whole card (HTML5)

```
<a href="product.html" class="card">
    
    <h3>Running shoe</h3>
    <p>450 EGP</p>
</a>
```

Since HTML5 an anchor may contain block-level content. The entire card becomes clickable.

A placeholder link for the current page

```
<nav>
  <a>Home</a>
  <a href="shop.html">Shop</a>
  <a href="contact.html">Contact</a>
</nav>
```

Home has no href, so it is shown as text and is not focusable, which is correct for the page the visitor is already on. You could also add `aria-current="page"` to a real link instead.

Default browser CSS

```
a:-webkit-any-link { color: -webkit-link; cursor: pointer; text-
decoration: underline; }
```

Accessibility

- Link text must make sense on its own. Screen reader users often read a list of all links on a page; 'click here' or 'read more' tells them nothing. Prefer 'Download the 2026 price list'.
- Do not use the same link text for links that go to different places.
- When a link contains only an image, the image's alt text becomes the link text, so it must describe the destination.
- Links should look different from ordinary text (usually underlined) and have a visible focus outline.
- Do not use javascript: URLs; use a `<button>` for actions that are not navigation.

Common mistakes

- Nesting an `<a>` inside another `<a>`, or putting a `<button>` inside an `<a>`.
- Using an `<a>` without href as a button. Use `<button>` instead.

- Forgetting `rel="noopener"` with `target="_blank"` on old browsers.
- Using `onclick="location.href=..."` on a `<div>` instead of a real link, which breaks keyboard access, right-click and search engines.
- Writing the URL with spaces or with backslashes instead of forward slashes.

See also

`<area>`, `<link>`, `<button>`, `<nav>`

Quick card: <abbr>

Abbreviation

The `<abbr>` element marks an abbreviation or acronym. The optional title attribute holds the full expansion, which most browsers show as a tooltip when the mouse hovers over the text.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	title
Related	<code><dfn></code> , <code><acronym></code>

Minimal example

```
<p>The <abbr title="World Health Organization">WHO</abbr> was founded  
in 1948.</p>
```

Most common mistake

Using `<acronym>`, which is obsolete.

<abbr> — Abbreviation

Category: Text-level semantics | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<abbr>` element marks an abbreviation or acronym. The optional title attribute holds the full expansion, which most browsers show as a tooltip when the mouse hovers over the text.

Use `<abbr>` the first time an abbreviation appears in a document, ideally writing the expansion in the text as well, because tooltips are not available on touch screens and are not read by every screen reader.

HTML 4.01 also had an `<acronym>` element for pronounceable abbreviations such as NASA. It was removed in HTML5; use `<abbr>` for both.

Syntax

```
<abbr title="expansion">ABBR</abbr>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
title	text	The full expansion of the abbreviation. On <abbr> this attribute has a specific meaning and should contain nothing else.	HTML 4.01

<abbr>: attribute by attribute

Each attribute of <abbr> explained in detail, with a minimal example you can copy.

title

Values: text | Since: HTML 4.01

The full expansion of the abbreviation. On <abbr> this attribute has a specific meaning and should contain nothing else.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<abbr title="value">...</abbr>
```

<abbr>: worked examples

Abbreviation with tooltip

```
<p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
```

Hovering WHO shows 'World Health Organization'.

Defining an abbreviation with <dfn>

```
<p><dfn><abbr title="HyperText Markup Language">HTML</abbr></dfn> is the language of the web.</p>
```

Wrapping in <dfn> marks this as the place where the term is defined.

Styling the dotted underline

```
<style>
  abbr[title] { text-decoration: underline dotted; cursor: help; }
</style>
<p>Pages are served over <abbr title="HyperText Transfer Protocol Secure">HTTPS</abbr>.</p>
```

Most browsers already show a dotted underline; this rule makes it consistent.

Default browser CSS

```
abbr[title] { text-decoration: underline dotted; }
```

Accessibility

- Provide the expansion in plain text at least once; do not rely on the tooltip alone.
- Some screen readers can be set to read the title attribute, but it is off by default.

Common mistakes

- Using <acronym>, which is obsolete.
- Putting long explanations in title; it is for the expansion only.

See also

<dfn>, <acronym>

Quick card: <acronym>

Acronym (obsolete)

The <acronym> element marked an acronym: an abbreviation pronounced as a word, such as NASA or laser. HTML5 removed it because the distinction from <abbr> was confusing and browsers treated the two identically.

At a glance

Status	OBSOLETE
Versions	HTML 4.01; obsolete in HTML5
Category	Obsolete text-level
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	title
Related	<abbr>

Minimal example

```
<!-- HTML 4.01 -->
<acronym title="National Aeronautics and Space Administration">NASA</acronym>
```

```
<!-- HTML5 -->
<abbr title="National Aeronautics and Space Administration">NASA</abbr>
```

Most common mistake

Using it at all in new pages.

<acronym> — Acronym (obsolete)

Category: Obsolete text-level | Status: OBSOLETE | Versions: HTML 4.01; obsolete in HTML5

Description

The `<acronym>` element marked an acronym: an abbreviation pronounced as a word, such as NASA or laser. HTML5 removed it because the distinction from `<abbr>` was confusing and browsers treated the two identically.

Replace every `<acronym>` with `<abbr>`.

Syntax

```
<acronym title="expansion">NASA</acronym>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Phrasing content (historical)
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
title	text	The expansion of the acronym.	HTML 4.01

<acronym>: attribute by attribute

Each attribute of <acronym> explained in detail, with a minimal example you can copy.

title

Values: text | Since: HTML 4.01

The expansion of the acronym.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<acronym title="value">...</acronym>
```

<acronym>: worked examples

Old code and its replacement

```
<!-- HTML 4.01 -->
<acronym title="National Aeronautics and Space Administration">NASA</acronym>
```

```
<!-- HTML5 -->
<abbr title="National Aeronautics and Space Administration">NASA</abbr>
```

Default browser CSS

```
acronym { text-decoration: underline dotted; }
```

Accessibility

- Same as <abbr>.

Common mistakes

- Using it at all in new pages.

See also

<abbr>

Quick card: <address>

Contact address

The <address> element indicates that its content provides contact information for a person, people or an organisation. When placed inside an <article> it gives the contact details of the article's author; when placed in the page <footer> or <body> it gives the contact details for the whole document.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<footer>, <article>, <a>

Minimal example

```
<footer>
  <address>
    Written by <a href="mailto:jane@example.com">Jane Doe</a>.<br>
    Visit us at:<br>
    Example Ltd.<br>
    12 Nile Street<br>
    Giza, Egypt
  </address>
</footer>
```

Most common mistake

Using <address> for every postal address on a page (for example in an order form).

<address> — Contact address

Category: Content sectioning | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<address>` element indicates that its content provides contact information for a person, people or an organisation. When placed inside an `<article>` it gives the contact details of the article's author; when placed in the page `<footer>` or `<body>` it gives the contact details for the whole document.

The contact information may be any kind: physical address, email, telephone, URL, social media handle, geographic coordinates. Despite its name, `<address>` is not meant for arbitrary postal addresses (such as the shipping address in an order); it is only for contact details of the author or owner of the content.

Browsers display `<address>` in italics by default, exactly as they did in 1991.

Syntax

```
<address>contact information</address>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, palpable content
Permitted content	Flow content, but no heading content, no sectioning content, and no <code><header></code> , <code><footer></code> or nested <code><address></code>

Property	Value
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<address>: worked examples

Contact details in the page footer

```
<footer>
  <address>
    Written by <a href="mailto:jane@example.com">Jane Doe</a>.<br>
    Visit us at:<br>
    Example Ltd.<br>
    12 Nile Street<br>
    Giza, Egypt
  </address>
</footer>
```

Author contact inside an article

```
<article>
  <h2>Learning HTML</h2>
  <p>...</p>
  <address>Contact the author: <a href="tel:+201000000000">+20 100
000 0000</a></address>
</article>
```

Here the address applies to the article only.

Default browser CSS

```
address { display: block; font-style: italic; }
```

Accessibility

- Screen readers may announce the element as 'contact information', which helps users find it.

Common mistakes

- Using <address> for every postal address on a page (for example in an order form).
- Placing headings or sections inside it.

See also

<footer>, <article>, <a>

Quick card: <applet>

Java applet (obsolete)

The <applet> element embedded a Java applet in the page. Applets were small Java programs that ran inside the browser with the help of a plug-in. Java applets were popular in the late 1990s but were slow, insecure and not supported on mobile devices. HTML 4.01 deprecated <applet> in favour of <object>, and HTML5 removed it entirely. No modern browser runs applets.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete embedded content
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	code, codebase, archive, object, alt, name, width, align, hspace
Related	<object>, <embed>, <canvas>

Minimal example

```
<applet code="Clock.class" width="200" height="200">  
  <param name="timezone" value="Africa/Cairo">  
  Your browser does not support Java applets.  
</applet>
```

Most common mistake

Any use in a modern page.

<applet> — Java applet (obsolete)

Category: Obsolete embedded content | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The <applet> element embedded a Java applet in the page. Applets were small Java programs that ran inside the browser with the help of a plug-in. Java applets were popular in the late 1990s but were slow, insecure and not supported on mobile devices. HTML 4.01 deprecated <applet> in favour of <object>, and HTML5 removed it entirely. No modern browser runs applets.

Syntax

```
<applet code="MyApplet.class" width="300" height="200"></applet>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Embedded content (historical)
Permitted content	Zero or more <param> elements followed by fallback content
Permitted parents	Any element that accepts embedded content
Tag omission	Neither tag may be omitted
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
code	class file name	The name of the compiled applet class file.	HTML 3.2
codebase	URL	The base URL for the applet's class files.	HTML 3.2
archive	comma-separated URLs	JAR archives to preload.	HTML 4.01
object	file name	A serialised applet file.	HTML 4.01
alt	text	Alternative text for browsers without Java.	HTML 3.2
name	text	A name so that applets on the same page can find each other.	HTML 3.2
width / height	pixels	Size of the applet display area.	HTML 3.2
align	left right top middle bottom	Alignment relative to surrounding text.	HTML 3.2
hspace / vspace	pixels	Horizontal and vertical margins.	HTML 3.2

<applet>: attribute by attribute

Each attribute of <applet> explained in detail, with a minimal example you can copy.

code

Values: class file name | Since: HTML 3.2

The name of the compiled applet class file.

Write code="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet code="class file name">...</applet>
```

codebase

Values: URL | Since: HTML 3.2

The base URL for the applet's class files.

Write codebase="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet codebase="https://example.com/page.html">...</applet>
```

archive

Values: comma-separated URLs | Since: HTML 4.01

JAR archives to preload.

Write archive="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet archive="comma-separated URLs">...</applet>
```

object

Values: file name | Since: HTML 4.01

A serialised applet file.

Write object="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet object="file name">...</applet>
```

alt

Values: text | Since: HTML 3.2

Alternative text for browsers without Java.

Write alt="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet alt="value">...</applet>
```

name

Values: text | Since: HTML 3.2

A name so that applets on the same page can find each other.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet name="value">...</applet>
```

width / height

Values: pixels | Since: HTML 3.2

Size of the applet display area.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet width="3">...</applet>
```

align

Values: left | right | top | middle | bottom | Since: HTML 3.2

Alignment relative to surrounding text.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet align="left">...</applet>
```

hspace / vspace

Values: pixels | Since: HTML 3.2

Horizontal and vertical margins.

Write hspace="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<applet hspace="3">...</applet>
```


<applet>: worked examples

Historical example

```
<applet code="Clock.class" width="200" height="200">  
  <param name="timezone" value="Africa/Cairo">  
  Your browser does not support Java applets.  
</applet>
```

For comparison, modern pages implement such features with JavaScript and <canvas>.

Default browser CSS

```
applet { display: inline-block; }
```

Accessibility

- Applets were inaccessible to assistive technology.

Common mistakes

- Any use in a modern page.

See also

<object>, <embed>, <canvas>

Quick card: <area>

Image map area

The `<area>` element defines a clickable region inside an image map. An image map is an image (`<img usemap="#name">`) linked to a `<map name="name">` element that contains one or more `<area>` elements. Each area has a shape and coordinates, and behaves like a hyperlink.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Image and multimedia
Closing tag	None (void element)
Display	inline
Own attributes	alt, coords, shape, href, target, download, rel, ping, referrerpolicy, nohref, type
Related	<code><map></code> , <code></code> , <code><a></code>

Minimal example

```

<map name="office">
  <area shape="rect" coords="0,0,200,150" href="kitchen.html"
alt="Kitchen">
  <area shape="circle" coords="300,75,50" href="meeting.html"
alt="Meeting room">
  <area shape="poly" coords="0,150,200,150,100,300" href="desks.html"
alt="Desks">
  <area shape="default" href="office.html" alt="Whole office">
</map>
```

Most common mistake

Forgetting the # in usemap="#name".

<area> — Image map area

Category: Image and multimedia | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<area>` element defines a clickable region inside an image map. An image map is an image (`<img usemap="#name">`) linked to a `<map name="name">` element that contains one or more `<area>` elements. Each area has a shape and coordinates, and behaves like a hyperlink.

`<area>` is a void element: it has no content and no closing tag. It may only be used inside `<map>`.

Syntax

```
<area shape="rect" coords="x1,y1,x2,y2" href="URL" alt="text">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Flow content, phrasing content
Permitted content	None (void element)
Permitted parents	<code><map></code> (the <code><area></code> may be a descendant of <code><map></code> at any depth, as long as there is no intervening <code><a></code>)
Tag omission	No end tag
DOM interface	HTMLAreaElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
alt	text	Alternative text for the area. Required when href is present so that the link has an accessible name.	HTML 3.2
coords	comma-separated numbers	The coordinates of the shape in CSS pixels measured from the top-left corner of the image. rect: x1,y1,x2,y2 (top-left and bottom-right). circle: x,y,radius. poly: x1,y1,x2,y2,... (any number of points).	HTML 3.2
shape	rect circle poly default	The shape of the region. default covers the whole image.	HTML 3.2
href	URL	The destination of the link.	HTML 3.2
target	_self _blank _parent _top name	Where to open the destination.	HTML 4.01
download	empty or filename	Download the resource instead of navigating.	HTML5
rel	link types	Relationship of the linked resource.	HTML5
ping	URLs	URLs to ping when the link is followed.	HTML5
referrerpolicy	referrer policy keyword	Referrer policy for the navigation.	HTML5
noreferrer	boolean	Obsolete. Indicated the area had no link. Simply omit href instead.	HTML 3.2 (obsolete)

Attribute	Values	Description	Since
type	MIME type	Obsolete in HTML5. Type of the linked resource.	HTML 4.01

<area>: attribute by attribute

Each attribute of <area> explained in detail, with a minimal example you can copy.

alt

Values: text | Since: HTML 3.2

Alternative text for the area. Required when href is present so that the link has an accessible name.

Write alt="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area alt="value">
```

coords

Values: comma-separated numbers | Since: HTML 3.2

The coordinates of the shape in CSS pixels measured from the top-left corner of the image. rect: x1,y1,x2,y2 (top-left and bottom-right). circle: x,y,radius. poly: x1,y1,x2,y2,... (any number of points).

Write coords="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area coords="3">
```

shape

Values: rect | circle | poly | default | Since: HTML 3.2

The shape of the region. default covers the whole image.

Write shape="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area shape="rect">
```

href

Values: URL | Since: HTML 3.2

The destination of the link.

Write href="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area href="https://example.com/page.html">
```

target

Values: _self | _blank | _parent | _top | name | Since: HTML 4.01

Where to open the destination.

Write target="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area target="_self">
```

download

Values: empty or filename | Since: HTML5

Download the resource instead of navigating.

Write download="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area download="empty">
```

rel

Values: link types | Since: HTML5

Relationship of the linked resource.

Write rel="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area rel="link types">
```

ping

Values: URLs | Since: HTML5

URLs to ping when the link is followed.

Write ping="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area ping="https://example.com/page.html">
```


referrerpolicy

Values: referrer policy keyword | Since: HTML5

Referrer policy for the navigation.

Write `referrerpolicy="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area referrerpolicy="referrer policy keyword">
```

nohref

Values: boolean | Since: HTML 3.2 (obsolete)

Obsolete. Indicated the area had no link. Simply omit `href` instead.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<area nohref>
```

type

Values: MIME type | Since: HTML 4.01

Obsolete in HTML5. Type of the linked resource.

Write `type="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<area type="text/plain">
```

<area>: worked examples

A world map with three clickable regions

```

<map name="office">
  <area shape="rect" coords="0,0,200,150" href="kitchen.html"
alt="Kitchen">
  <area shape="circle" coords="300,75,50" href="meeting.html"
alt="Meeting room">
  <area shape="poly" coords="0,150,200,150,100,300" href="desks.html"
alt="Desks">
  <area shape="default" href="office.html" alt="Whole office">
</map>
```

The rectangle covers the top-left quarter, the circle has centre (300,75) and radius 50, and the polygon is a triangle. The default area catches clicks anywhere else.

Default browser CSS

```
area { display: inline; }
```

Accessibility

- Every <area> with href needs a meaningful alt.
- Image maps do not scale well on small screens; consider SVG with real links instead.

Common mistakes

- Forgetting the # in usemap="#name".
- Coordinates that do not match the displayed image size.
- Writing a closing </area> tag.

See also

<map>, , <a>

Quick card: <article>

Self-contained article

The <article> element represents a complete, self-contained piece of content that would make sense on its own if it were distributed independently, for example in an RSS feed. Typical uses are a blog post, a news story, a forum post, a product card, a user comment or an interactive widget.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<section>, <aside>, <header>, <footer>, <main>, <nav>

Minimal example

```
<article>
  <header>
    <h2>How I learned HTML</h2>
    <p>Posted <time datetime="2026-09-11">11 September 2026</time> by
Ahmed</p>
  </header>
  <p>It started with a single footer tag...</p>
  <footer>
    <p>Tags: html, learning</p>
  </footer>
</article>
```

Most common mistake

Using `<article>` for every box on the page; use `<section>` or `<div>` for content that is not self-contained.

<article> — Self-contained article

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The `<article>` element represents a complete, self-contained piece of content that would make sense on its own if it were distributed independently, for example in an RSS feed. Typical uses are a blog post, a news story, a forum post, a product card, a user comment or an interactive widget.

An `<article>` usually has its own heading, and may have its own `<header>`, `<footer>` and `<address>`. Articles can be nested: comments on a blog post are articles inside the post's article.

The test for choosing `<article>` rather than `<section>` or `<div>` is: 'Could this piece be copied to another site and still make sense?' If yes, it is an article.

Syntax

```
<article>...</article>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning content, palpable content
Permitted content	Flow content

Property	Value
Permitted parents	Any element that accepts flow content, but not inside <address>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<article>: worked examples

A blog post

```
<article>
  <header>
    <h2>How I learned HTML</h2>
    <p>Posted <time datetime="2026-09-11">11 September 2026</time> by
Ahmed</p>
  </header>
  <p>It started with a single footer tag...</p>
  <footer>
    <p>Tags: html, learning</p>
  </footer>
</article>
```

Nested articles for comments

```
<article>
  <h2>Post title</h2>
  <p>Post text.</p>
  <section>
    <h3>Comments</h3>
    <article>
      <p>Great post!</p>
      <footer>- Sara</footer>
    </article>
  </section>
</article>
```

Product cards in a shop

```
<article class="product">
  
  <h3>Learn HTML</h3>
  <p>120 EGP</p>
  <button>Add to cart</button>
</article>
```

Default browser CSS

```
article { display: block; }
```

Accessibility

- Screen readers expose <article> with the role 'article' and may let users jump between articles.
- Give each article a heading so it can be identified.

Common mistakes

- Using `<article>` for every box on the page; use `<section>` or `<div>` for content that is not self-contained.
- Omitting a heading inside the article.

See also

`<section>`, `<aside>`, `<header>`, `<footer>`, `<main>`, `<nav>`

Quick card: <aside>

Aside (sidebar / tangential content)

The <aside> element represents content that is only indirectly related to the main content around it. It is often displayed as a sidebar or a call-out box: pull quotes, related links, advertising, author biographies, glossary definitions, or a list of other articles.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<article>, <section>, <main>, <nav>

Minimal example

```
<main>
  <article>
    <h1>The Nile</h1>
    <p>The Nile is the longest river in Africa...</p>
  </article>
  <aside>
    <h2>Related</h2>
    <ul>
      <li><a href="aswan.html">Aswan Dam</a></li>
      <li><a href="delta.html">Nile Delta</a></li>
    </ul>
  </aside>
</main>
```

Most common mistake

Using `<aside>` for parenthetical text inside a paragraph; use parentheses or `<span>` for that.

<aside> — Aside (sidebar / tangential content)

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The <aside> element represents content that is only indirectly related to the main content around it. It is often displayed as a sidebar or a call-out box: pull quotes, related links, advertising, author biographies, glossary definitions, or a list of other articles.

Inside an <article>, an <aside> holds content related to that article. Directly inside <body> or <main>, it holds content related to the page as a whole, such as a site-wide sidebar.

Syntax

<aside>...</aside>

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning content, palpable content
Permitted content	Flow content
Permitted parents	Any element that accepts flow content, but not inside <address>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<aside>: worked examples

A sidebar with related links

```
<main>
  <article>
    <h1>The Nile</h1>
    <p>The Nile is the longest river in Africa...</p>
  </article>
  <aside>
    <h2>Related</h2>
    <ul>
      <li><a href="aswan.html">Aswan Dam</a></li>
      <li><a href="delta.html">Nile Delta</a></li>
    </ul>
  </aside>
</main>
```

A pull quote inside an article

```
<article>
  <p>Learning takes time, but every tag you learn is a tool you keep.
</p>
  <aside><q>Every tag you learn is a tool you keep.</q></aside>
  <p>...</p>
</article>
```

Default browser CSS

```
aside { display: block; }
```

Accessibility

- Exposed as a 'complementary' landmark so users can skip to or over it.

Common mistakes

- Using <aside> for parenthetical text inside a paragraph; use parentheses or for that.
- Putting the main content in an <aside>.

See also

<article>, <section>, <main>, <nav>

Quick card: <audio>

Embedded audio

The <audio> element embeds sound content: music, podcasts, sound effects or recorded speech. The source may be given with the src attribute or with one or more <source> child elements so that the browser can choose a format it supports. Content placed inside the element after the <source> elements is shown only by browsers that do not support <audio> at all.

At a glance

Status	CURRENT
Versions	HTML5
Category	Image and multimedia
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	src, controls, autoplay, loop, muted, preload, crossorigin, controlslist, disableremoteplayback
Related	<video>, <source>, <track>, <embed>, <object>

Minimal example

```
<audio src="song.mp3" controls>
  Your browser does not support the audio element.
  <a href="song.mp3">Download the song</a>.
</audio>
```

Most common mistake

Expecting autoplay with sound to work.

<audio> — Embedded audio

Category: Image and multimedia | Status: CURRENT | Versions: HTML5

Description

The `<audio>` element embeds sound content: music, podcasts, sound effects or recorded speech. The source may be given with the `src` attribute or with one or more `<source>` child elements so that the browser can choose a format it supports. Content placed inside the element after the `<source>` elements is shown only by browsers that do not support `<audio>` at all.

By default nothing is displayed. Add the `controls` attribute to show the browser's built-in play, pause, volume and progress controls, or control playback from JavaScript with the `HTMLMediaElement` methods `play()` and `pause()`.

Browsers block automatic playback with sound until the user has interacted with the page, so autoplay usually works only together with muted.

Syntax

```
<audio src="file.mp3" controls></audio>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content; interactive and palpable if controls is present

Property	Value
Permitted content	If src is present: zero or more <track> elements then transparent content with no media elements. If src is absent: zero or more <source>, then zero or more <track>, then transparent content with no media elements
Permitted parents	Any element that accepts embedded content
Tag omission	Neither tag may be omitted
DOM interface	HTMLAudioElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The URL of the audio file. Alternatively use <source> children.	HTML5
controls	boolean	Show the browser's playback controls.	HTML5
autoplay	boolean	Start playing as soon as enough data has loaded. Usually blocked unless muted.	HTML5
loop	boolean	Restart from the beginning when the end is reached.	HTML5
muted	boolean	Start with the sound muted.	HTML5
preload	none metadata auto	A hint about how much to download before playback: nothing, only metadata such as duration (default in most browsers), or the whole file.	HTML5

Attribute	Values	Description	Since
crossorigin	anonymous use-credentials	How to handle cross-origin requests, needed when the audio will be processed with the Web Audio API or canvas.	HTML5
controlslist	nodownload nofullscreen noremoteplayback	Hides specific built-in controls (non-standard, Chromium).	Living Standard
disableremoteplayback	boolean	Disables casting to remote devices.	Living Standard

<audio>: attribute by attribute

Each attribute of <audio> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML5

The URL of the audio file. Alternatively use <source> children.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<audio src="https://example.com/page.html">...</audio>
```

controls

Values: boolean | Since: HTML5

Show the browser's playback controls.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<audio controls>...</audio>
```

autoplay

Values: boolean | Since: HTML5

Start playing as soon as enough data has loaded. Usually blocked unless muted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<audio autoplay>...</audio>
```

loop

Values: boolean | Since: HTML5

Restart from the beginning when the end is reached.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<audio loop>...</audio>
```

muted

Values: boolean | Since: HTML5

Start with the sound muted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<audio muted>...</audio>
```

preload

Values: none | metadata | auto | Since: HTML5

A hint about how much to download before playback: nothing, only metadata such as duration (default in most browsers), or the whole file.

Write `preload="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<audio preload="none">...</audio>
```

crossorigin

Values: anonymous | use-credentials | Since: HTML5

How to handle cross-origin requests, needed when the audio will be processed with the Web Audio API or canvas.

Write `crossorigin="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<audio crossorigin="anonymous">...</audio>
```

controlslist

Values: nodownload | nofullscreen | noremoteplayback | Since: Living Standard

Hides specific built-in controls (non-standard, Chromium).

Write `controlslist="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<audio controlslist="nodownload">...</audio>
```

disableremoteplayback

Values: boolean | Since: Living Standard

Disables casting to remote devices.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<audio disableremoteplayback>...</audio>
```

<audio>: worked examples

Simple player

```
<audio src="song.mp3" controls>
  Your browser does not support the audio element.
  <a href="song.mp3">Download the song</a>.
</audio>
```

Multiple formats with <source>

```
<audio controls preload="metadata">
  <source src="podcast.opus" type="audio/ogg; codecs=opus">
  <source src="podcast.mp3" type="audio/mpeg">
  <p>Download the <a href="podcast.mp3">MP3</a>.</p>
</audio>
```

The browser uses the first source it can play. Opus is smaller; MP3 works everywhere.

Controlling playback with JavaScript

```
<audio id="beep" src="beep.wav"></audio>
<button onclick="document.getElementById('beep').play()">Play beep</button>
```

Background music that loops (muted by default)

```
<audio src="ambient.mp3" autoplay loop muted></audio>
```

The user must unmute it themselves.

Default browser CSS

```
audio:not([controls]) { display: none !important; }
```

Accessibility

- Provide a transcript for spoken content; <track> captions are supported only on <video> in most browsers.
- Always include controls or provide your own accessible buttons.
- Never autoplay sound; it is disorienting and can hide a screen reader's speech.

Common mistakes

- Expecting autoplay with sound to work.

- Using a single format that not all browsers support.
- Forgetting the type attribute on <source>, which forces the browser to download each file to test it.

See also

<video>, <source>, <track>, <embed>, <object>

Quick card: `<b>`

Bring attention to (bold)

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code></code> , <code></code> , <code><i></code> , <code><mark></code>

Minimal example

```
<p>This laptop has a <b>15-inch display</b>, <b>16 GB of RAM</b> and  
a <b>1 TB SSD</b>.</p>
```

Most common mistake

Using `<b>` for headings instead of `<h1>`-`<h6>`.

 — Bring attention to (bold)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<b>` element draws the reader's attention to a span of text without giving it extra importance, emphasis or an alternative voice. Browsers render it in bold. Typical uses are keywords in a summary, product names in a review, or the lead sentence of an article.

In HTML 4 `<b>` was a purely presentational element meaning 'bold'. HTML5 redefined it with the meaning above so that it remains valid, but the standard says to use `<b>` only as a last resort when no other element is more appropriate: `<strong>` for importance, `<em>` for emphasis, `<mark>` for relevance, `<h1>`-`<h6>` for headings, and CSS font-weight for pure styling.

Syntax

```
<b>text</b>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

: worked examples

Keywords in a product description

```
<p>This laptop has a <b>15-inch display</b>, <b>16 GB of RAM</b> and  
a <b>1 TB SSD</b>.</p>
```

The keywords stand out but are not more 'important' in the `<strong>` sense.

**Difference between and **

```
<p><strong>Warning:</strong> the <b>red</b> wire carries current.</p>
```

'Warning' is important (strong); 'red' is merely highlighted as a keyword (b).

Default browser CSS

```
b { font-weight: bold; }
```

Accessibility

- Screen readers do not announce `<b>` in any special way. Use `<strong>` when the boldness carries meaning.

Common mistakes

- Using `<b>` for headings instead of `<h1>-<h6>`.
- Using `<b>` for emphasis instead of `<em>` or `<strong>`.

See also

`<strong>`, `<em>`, `<i>`, `<mark>`

Quick card: <base>

Document base URL

The <base> element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one <base> element per document and it must be placed in <head>, before any element that uses a URL.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Document metadata
Closing tag	None (void element)
Display	none
Own attributes	href, target
Related	<head>, <link>, <a>

Minimal example

```
<head>
  <base href="https://cdn.example.com/assets/">
</head>
<body>
  
  <!-- loads https://cdn.example.com/assets/logo.png -->
</body>
```

Most common mistake

Placing <base> after a <link> or <script> that uses a relative URL; it does not apply retroactively.

<base> — Document base URL

Category: Document metadata | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <base> element specifies the base URL used to resolve all relative URLs in the document, and/or the default target for all links and forms. There may be only one <base> element per document and it must be placed in <head>, before any element that uses a URL.

It is a void element. At least one of href or target must be present.

Syntax

```
<base href="URL" target="_blank">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Metadata content
Permitted content	None (void element)
Permitted parents	<head> (only one allowed)
Tag omission	No end tag
DOM interface	HTMLBaseElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
href	URL	The base URL. Relative URLs such as images/logo.png are resolved against it.	HTML 2.0
target	_self _blank _parent _top name	The default browsing context for links and forms that do not specify their own target.	HTML 4.01

<base>: attribute by attribute

Each attribute of <base> explained in detail, with a minimal example you can copy.

href

Values: URL | Since: HTML 2.0

The base URL. Relative URLs such as images/logo.png are resolved against it.

Write href="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<base href="https://example.com/page.html">
```

target

Values: _self | _blank | _parent | _top | name | Since: HTML 4.01

The default browsing context for links and forms that do not specify their own target.

Write target="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<base target="_self">
```

<base>: worked examples

Making all relative links point to another server

```
<head>
  <base href="https://cdn.example.com/assets/">
</head>
<body>
  
  <!-- loads https://cdn.example.com/assets/logo.png -->
</body>
```

Opening every link in a new tab

```
<head>
  <base target="_blank">
</head>
```

Individual links can still override this with their own target.

Default browser CSS

```
base { display: none; }
```

Accessibility

- Opening all links in new tabs without warning is confusing; use with care.

Common mistakes

- Placing <base> after a <link> or <script> that uses a relative URL; it does not apply retroactively.
- Having more than one <base>.
- Forgetting that fragment links (#top) are also affected and will navigate to the base URL.

See also

<head>, <link>, <a>

Quick card: <basefont>

Base font (obsolete)

The <basefont> element set the default font size, colour and face for the whole document. It was a presentational element from HTML 3.2 and was deprecated in HTML 4.01 in favour of CSS. HTML5 removed it. Use CSS on the body element instead.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete presentational
Closing tag	None (void element)
Display	none
Own attributes	size, color, face
Related	, <body>

Minimal example

```
<!-- Old -->
<basefont size="4" face="Arial" color="#333">

<!-- New -->
<style>
  body { font: 18px Arial, sans-serif; color: #333; }
</style>
```

Most common mistake

Any use.

<basefont> — Base font (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The <basefont> element set the default font size, colour and face for the whole document. It was a presentational element from HTML 3.2 and was deprecated in HTML 4.01 in favour of CSS. HTML5 removed it. Use CSS on the body element instead.

Syntax

```
<basefont size="3" color="black" face="Arial">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Historical
Permitted content	None
Permitted parents	<head> or <body>
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
size	1 to 7	Default font size on the 1-7 HTML scale (3 = normal).	HTML 3.2
color	colour	Default text colour.	HTML 4.01
face	comma-separated font names	Default font family.	HTML 4.01

<basefont>: attribute by attribute

Each attribute of <basefont> explained in detail, with a minimal example you can copy.

size

Values: 1 to 7 | Since: HTML 3.2

Default font size on the 1-7 HTML scale (3 = normal).

Write size="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<basefont size="1 to 7">
```

color

Values: colour | Since: HTML 4.01

Default text colour.

Write color="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<basefont color="#336699">
```

face

Values: comma-separated font names | Since: HTML 4.01

Default font family.

Write face="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<basefont face="value">
```

<basefont>: worked examples

Replacement in CSS

```
<!-- Old -->
<basefont size="4" face="Arial" color="#333">

<!-- New -->
<style>
  body { font: 18px Arial, sans-serif; color: #333; }
</style>
```

Common mistakes

- Any use.

See also

, <body>

Quick card: <bdi>

Bidirectional isolate

The <bdi> element isolates a span of text that might be written in a different direction from the text around it, so that its direction does not affect the surrounding text. It is essential when you insert user-generated text of unknown direction, such as user names, into a sentence.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	dir
Related	<bdo>,

Minimal example

```
<ul>
  <li><bdi>Ahmed</bdi>: 120 points</li>
  <li><bdi>محمد</bdi>: 95 points</li>
  <li><bdi>Sara</bdi>: 80 points</li>
</ul>
```

Most common mistake

Using <bdo> when you want isolation; <bdo> forces a direction, <bdi> isolates.

<bdi> — Bidirectional isolate

Category: Text-level semantics | Status: CURRENT | Versions: HTML5

Description

The `<bdi>` element isolates a span of text that might be written in a different direction from the text around it, so that its direction does not affect the surrounding text. It is essential when you insert user-generated text of unknown direction, such as user names, into a sentence.

Without isolation, an Arabic or Hebrew name followed by a number in an English sentence can cause the number and punctuation to appear in the wrong place. `<bdi>` tells the bidirectional algorithm to treat the inner text as a separate unit.

It is equivalent to CSS `unicode-bidi: isolate`, and by default its direction is `auto` (detected from the content).

Syntax

```
<bdi>text of unknown direction</bdi>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
dir	ltr rtl auto	On <bdi> the default is auto, unlike other elements where dir is inherited.	HTML5

<bdi>: attribute by attribute

Each attribute of <bdi> explained in detail, with a minimal example you can copy.

dir

Values: ltr | rtl | auto | Since: HTML5

On <bdi> the default is auto, unlike other elements where dir is inherited.

Write dir="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bdi dir="ltr">...</bdi>
```


<bdi>: worked examples

User names in a leaderboard

```
<ul>
  <li><bdi>Ahmed</bdi>: 120 points</li>
  <li><bdi>محمد</bdi>: 95 points</li>
  <li><bdi>Sara</bdi>: 80 points</li>
</ul>
```

Without <bdi> the second line could render as ': 95 points محمد' with the colon misplaced.

Default browser CSS

```
bdi { unicode-bidi: isolate; }
```

Accessibility

- Improves reading order for screen readers as well as visual order.

Common mistakes

- Using <bdo> when you want isolation; <bdo> forces a direction, <bdi> isolates.

See also

<bdo>,

Quick card: <bdo>

Bidirectional text override

The <bdo> element overrides the current text direction and forces its content to be displayed in the direction given by the dir attribute, character by character, ignoring the Unicode bidirectional algorithm. The dir attribute is required.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	dir
Related	<bdi>

Minimal example

```
<p><bdo dir="rtl">This text is reversed</bdo></p>
```

Most common mistake

Omitting dir, which is required.

<bdo> — Bidirectional text override

Category: Text-level semantics | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<bdo>` element overrides the current text direction and forces its content to be displayed in the direction given by the `dir` attribute, character by character, ignoring the Unicode bidirectional algorithm. The `dir` attribute is required.

It is rarely needed; use it when you must display text backwards, for example to show how a right-to-left string is stored, or to fix a badly encoded document.

Syntax

```
<bdo dir="rtl">text</bdo>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
dir	ltr rtl	Required. The direction to force.	HTML 4.01

<bdo>: attribute by attribute

Each attribute of <bdo> explained in detail, with a minimal example you can copy.

dir

Values: ltr | rtl | Since: HTML 4.01

Required. The direction to force.

Write dir="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bdo dir="ltr">...</bdo>
```

<bdo>: worked examples

Displaying English backwards

```
<p><bdo dir="rtl">This text is reversed</bdo></p>
```

Renders as 'desrever si txet sihT'.

Default browser CSS

```
bdo { unicode-bidi: isolate-override; }
```

Accessibility

- Screen readers read the characters in the overridden order, which may be nonsense; use sparingly.

Common mistakes

- Omitting `dir`, which is required.
- Using `<bdo>` for isolation; use `<bdi>`.

See also

`<bdi>`

Quick card: <bgsound>

Background sound (non-standard, obsolete)

The <bgsound> element was an Internet Explorer extension that played a sound file automatically when the page loaded. It was never part of any HTML standard and is not supported by any modern browser. Use <audio> instead.

At a glance

Status	OBSOLETE
Versions	Internet Explorer extension, never standard; obsolete
Category	Obsolete embedded content
Closing tag	None (void element)
Display	none
Own attributes	src, loop, balance, volume
Related	<audio>

Minimal example

```
<audio src="music.mp3" autoplay loop muted></audio>
```

Most common mistake

Any use.

<bgsound> — Background sound (non-standard, obsolete)

Category: Obsolete embedded content | Status: OBSOLETE | Versions: Internet Explorer extension, never standard; obsolete

Description

The <bgsound> element was an Internet Explorer extension that played a sound file automatically when the page loaded. It was never part of any HTML standard and is not supported by any modern browser. Use <audio> instead.

Syntax

```
<bgsound src="music.mid" loop="infinite">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Historical
Permitted content	None
Permitted parents	<head> or <body>
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The sound file.	IE
loop	number infinite	Repeat count.	IE
balance	-10000 to 10000	Stereo balance.	IE
volume	-10000 to 0	Volume.	IE

<bgsound>: attribute by attribute

Each attribute of <bgsound> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: IE

The sound file.

Write `src="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bgsound src="https://example.com/page.html">
```

loop

Values: number | infinite | Since: IE

Repeat count.

Write `loop="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bgsound loop="3">
```

balance

Values: -10000 to 10000 | Since: IE

Stereo balance.

Write `balance="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bgsound balance="-10000 to 10000">
```

volume

Values: -10000 to 0 | Since: IE

Volume.

Write `volume="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<bgsound volume="-10000 to 0">
```

<bgsound>: worked examples

Replacement

```
<audio src="music.mp3" autoplay loop muted></audio>
```

Common mistakes

- Any use.

See also

[<audio>](#)

Quick card: **<big>**

Bigger text (obsolete)

The `<big>` element rendered its content one font size larger than the surrounding text. It was purely presentational and was removed in HTML5. Its counterpart `<small>` survived because it was given a new meaning (side comments and fine print). Use CSS font-size instead of `<big>`.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01; obsolete in HTML5
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><small></code> , <code></code>

Minimal example

```
<!-- Old -->
<p>Only <big>3 days</big> left!</p>
```

```
<!-- New -->
<p>Only <span style="font-size: larger">3 days</span> left!</p>
```

Most common mistake

Any use in new pages.

<big> — Bigger text (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01; obsolete in HTML5

Description

The `<big>` element rendered its content one font size larger than the surrounding text. It was purely presentational and was removed in HTML5. Its counterpart `<small>` survived because it was given a new meaning (side comments and fine print). Use CSS font-size instead of `<big>`.

Syntax

```
<big>text</big>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Phrasing content (historical)
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<big>: worked examples

Replacement

```
<!-- Old -->
<p>Only <big>3 days</big> left!</p>
```

```
<!-- New -->
<p>Only <span style="font-size: larger">3 days</span> left!</p>
```

Default browser CSS

```
big { font-size: larger; }
```

Common mistakes

- Any use in new pages.

See also

<small>,

Quick card: **<blink>**

Blinking text (non-standard, obsolete)

The `<blink>` element made its text flash on and off. It was a Netscape Navigator extension from 1994, was never part of any standard, and was widely considered the most annoying element ever created. All browsers removed support by 2013. Blinking content is also an accessibility hazard for people with photosensitive epilepsy and attention disorders.

At a glance

Status	OBSOLETE
Versions	Netscape extension, never standard; obsolete
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><marquee></code>

Minimal example

```
<blink>Sale!</blink>
```

Most common mistake

Any use.

<blink> — Blinking text (non-standard, obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: Netscape extension, never standard; obsolete

Description

The <blink> element made its text flash on and off. It was a Netscape Navigator extension from 1994, was never part of any standard, and was widely considered the most annoying element ever created. All browsers removed support by 2013. Blinking content is also an accessibility hazard for people with photosensitive epilepsy and attention disorders.

Syntax

```
<blink>text</blink>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Historical
Permitted content	Phrasing content
Permitted parents	Any
Tag omission	Neither tag may be omitted
DOM interface	HTMLUnknownElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<blink>: worked examples

Do not do this

```
<blink>Sale!</blink>
```

If you must animate, use a CSS animation with a slow, subtle effect and respect `prefers-reduced-motion`.

Accessibility

- Flashing content can trigger seizures. WCAG forbids flashing more than three times per second.

Common mistakes

- Any use.

See also

<marquee>

Quick card: <blockquote>

Block quotation

The <blockquote> element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional cite attribute holds the URL of the source; a visible citation should be given with the <cite> element, usually in a <footer> inside the blockquote or in a following paragraph.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	cite
Related	<q>, <cite>, <footer>

Minimal example

```
<blockquote cite="https://www.w3.org/People/Berners-Lee/">
  <p>The Web does not just connect machines, it connects people.</p>
  <footer>- <cite>Tim Berners-Lee</cite></footer>
</blockquote>
```

Most common mistake

Using <blockquote> just to indent text; use CSS margin for that.

<blockquote> — Block quotation

Category: Text content | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<blockquote>` element indicates that the enclosed text is an extended quotation from another source. Browsers indent it on both sides by default. The optional `cite` attribute holds the URL of the source; a visible citation should be given with the `<cite>` element, usually in a `<footer>` inside the `blockquote` or in a following paragraph.

For short quotations that sit inside a sentence use `<q>` instead.

Syntax

```
<blockquote cite="URL">quoted text</blockquote>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning root, palpable content
Permitted content	Flow content
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLQuoteElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
cite	URL	The URL of the source document or message. Not displayed by browsers; readable by scripts and tools.	HTML 4.01

<blockquote>: attribute by attribute

Each attribute of <blockquote> explained in detail, with a minimal example you can copy.

cite

Values: URL | Since: HTML 4.01

The URL of the source document or message. Not displayed by browsers; readable by scripts and tools.

Write cite="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<blockquote cite="https://example.com/page.html">...</blockquote>
```

<blockquote>: worked examples

Quotation with source

```
<blockquote cite="https://www.w3.org/People/Berners-Lee/">
  <p>The Web does not just connect machines, it connects people.</p>
  <footer>- <cite>Tim Berners-Lee</cite></footer>
</blockquote>
```

Custom styling

```
<style>
  blockquote {
    margin: 1em 0; padding: 0.5em 1em;
    border-left: 4px solid #c8a24a; background: #fdf8f0;
  }
</style>
<blockquote><p>Practice is the only tutorial that always works.</p></
blockquote>
```

Default browser CSS

```
blockquote { display: block; margin: 1em 40px; }
```

Accessibility

- Screen readers announce 'blockquote' so users know the text is quoted.

Common mistakes

- Using <blockquote> just to indent text; use CSS margin for that.
- Putting the author's name inside the quotation as if it were quoted.

See also

<q>, <cite>, <footer>

Quick card: <body>

Document body

The <body> element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one <body> in a document and it is the second child of <html>, after <head>.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Sectioning root
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	onafterprint, onbeforeunload, onhashchange, onlanguagechange, onload, onmessage, onoffline, onpagehide, onpopstate, onresize, onstorage, onunload, alink, background, bgcolor, link, text, vlink, bottommargin
Related	<html>, <head>, <main>, <header>, <footer>

Minimal example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Hello</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```


Most common mistake

Having two `<body>` elements.

<body> — Document body

Category: Sectioning root | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<body>` element contains all the content of the document that is displayed to the visitor: text, images, links, forms, media, and the scripts that operate on them. There is exactly one `<body>` in a document and it is the second child of `<html>`, after `<head>`.

In HTML 3.2 and 4.01 `<body>` carried presentational attributes (`bgsColor`, `text`, `link`, `vlink`, `alink`, `background`) that set page colours and background images. They are obsolete in HTML5; use CSS. The `<body>` element also accepts window event handler attributes such as `onload` and `onresize`, because events on the window are reflected onto the body.

The start and end tags of `<body>` may technically be omitted and the browser will create the element automatically, but always write them for clarity.

Syntax

```
<body>...</body>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Sectioning root

Property	Value
Permitted content	Flow content
Permitted parents	<html>, as its second child
Tag omission	Start tag may be omitted if the body is empty or starts with non-whitespace content that is not a comment, <script> or <style>; end tag may be omitted if not followed by a comment
DOM interface	HTMLBodyElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
onafterprint, onbeforeprint	script	Fired after and before the document is printed.	HTML5
onbeforeunload	script	Fired when the user is about to leave; can prompt for confirmation.	HTML5
onhashchange	script	Fired when the URL fragment (#...) changes.	HTML5
onlanguagechange	script	Fired when the preferred language changes.	HTML5
onload	script	Fired when the whole page, including images, has loaded.	HTML 4.01
onmessage, onmessageerror	script	Fired when a message is received via postMessage.	HTML5
onoffline, ononline	script	Fired when network connectivity is lost or regained.	HTML5

Attribute	Values	Description	Since
onpagehide, onpageshow	script	Fired when navigating away from or back to the page (including from the back-forward cache).	HTML5
onpopstate	script	Fired when the active history entry changes.	HTML5
onresize	script	Fired when the window is resized.	HTML 4.01
onstorage	script	Fired when localStorage or sessionStorage changes in another tab.	HTML5
onunload	script	Fired when the document is unloaded. Unreliable; prefer pagehide.	HTML 4.01
alink	colour	Obsolete. Colour of active (being clicked) links. Use CSS a:active.	HTML 3.2
background	URL	Obsolete. Background image. Use CSS background-image.	HTML 3.2
bgcolor	colour	Obsolete. Background colour. Use CSS background-color.	HTML 3.2
link	colour	Obsolete. Colour of unvisited links. Use CSS a:link.	HTML 3.2
text	colour	Obsolete. Text colour. Use CSS color.	HTML 3.2
vlink	colour	Obsolete. Colour of visited links. Use CSS a:visited.	HTML 3.2
bottommargin, leftmargin, rightmargin, topmargin	pixels	Obsolete, non-standard. Page margins. Use CSS margin.	IE/Netscape

<body>: attribute by attribute

Each attribute of <body> explained in detail, with a minimal example you can copy.

onafterprint, onbeforeprint

Values: script | Since: HTML5

Fired after and before the document is printed.

Write onafterprint="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onafterprint="script">...</body>
```

onbeforeunload

Values: script | Since: HTML5

Fired when the user is about to leave; can prompt for confirmation.

Write onbeforeunload="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onbeforeunload="script">...</body>
```

onhashchange

Values: script | Since: HTML5

Fired when the URL fragment (#...) changes.

Write onhashchange="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onhashchange="script">...</body>
```

onlanguagechange

Values: script | Since: HTML5

Fired when the preferred language changes.

Write onlanguagechange="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onlanguagechange="script">...</body>
```

onload

Values: script | Since: HTML 4.01

Fired when the whole page, including images, has loaded.

Write `onload="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onload="script">...</body>
```

onmessage, onmessageerror

Values: script | Since: HTML5

Fired when a message is received via `postMessage`.

Write `onmessage="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onmessage="script">...</body>
```

onoffline, ononline

Values: script | Since: HTML5

Fired when network connectivity is lost or regained.

Write `onoffline="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onoffline="script">...</body>
```

onpagehide, onpageshow

Values: script | Since: HTML5

Fired when navigating away from or back to the page (including from the back-forward cache).

Write `onpagehide="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onpagehide="script">...</body>
```

onpopstate

Values: script | Since: HTML5

Fired when the active history entry changes.

Write onpopstate="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onpopstate="script">...</body>
```

onresize

Values: script | Since: HTML 4.01

Fired when the window is resized.

Write onresize="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onresize="script">...</body>
```

onstorage

Values: script | Since: HTML5

Fired when localStorage or sessionStorage changes in another tab.

Write onstorage="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onstorage="script">...</body>
```

onunload

Values: script | Since: HTML 4.01

Fired when the document is unloaded. Unreliable; prefer pagehide.

Write onunload="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body onunload="script">...</body>
```

alink

Values: colour | Since: HTML 3.2

Obsolete. Colour of active (being clicked) links. Use CSS `a:active`.

Write `alink="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body alink="#336699">...</body>
```

background

Values: URL | Since: HTML 3.2

Obsolete. Background image. Use CSS `background-image`.

Write `background="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body background="https://example.com/page.html">...</body>
```

bgcolor

Values: colour | Since: HTML 3.2

Obsolete. Background colour. Use CSS `background-color`.

Write `bgcolor="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body bgcolor="#336699">...</body>
```

link

Values: colour | Since: HTML 3.2

Obsolete. Colour of unvisited links. Use CSS `a:link`.

Write `link="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body link="#336699">...</body>
```


text

Values: colour | Since: HTML 3.2

Obsolete. Text colour. Use CSS color.

Write text="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body text="#336699">...</body>
```

vlink

Values: colour | Since: HTML 3.2

Obsolete. Colour of visited links. Use CSS a:visited.

Write vlink="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body vlink="#336699">...</body>
```

bottommargin, leftmargin, rightmargin, topmargin

Values: pixels | Since: IE/Netscape

Obsolete, non-standard. Page margins. Use CSS margin.

Write bottommargin="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<body bottommargin="3">...</body>
```

<body>: worked examples

Minimal document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Hello</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

HTML 3.2 colours and their CSS replacement

```
<!-- HTML 3.2 -->
<body bgcolor="#ffffff" text="#000000" link="#0000ff" vlink="#800080">

<!-- HTML5 + CSS -->
<style>
  body { background: #fff; color: #000; }
  a:link { color: blue; }
  a:visited { color: purple; }
</style>
<body>
```

Running code when the page has loaded

```
<body onload="console.log('Page loaded')">
```

Prefer `addEventListener('DOMContentLoaded', ...)` in a script.

Default browser CSS

```
body { display: block; margin: 8px; }
```

Accessibility

- Set the page language on `<html>`, not `<body>`.
- Keep a logical reading order in the body; CSS can rearrange the visual layout but assistive technology follows the source order.

Common mistakes

- Having two `<body>` elements.

- Putting <title> or <meta> in the body.
- Using the obsolete colour attributes.

See also

<html>, <head>, <main>, <header>, <footer>

Quick card:

Line break

The
 element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	None (void element)
Display	inline
Own attributes	clear
Related	<p>, <wbr>, <hr>, <pre>

Minimal example

```
<p>  
  Example Ltd.<br>  
  12 Nile Street<br>  
  Giza, Egypt  
</p>
```

Most common mistake

Using
 for vertical spacing.

 — Line break

Category: Text-level semantics | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<br>` element produces a line break in text: the text after it starts on a new line. It is a void element with no closing tag. Use it only where the division of lines is part of the content, as in a postal address or a poem.

Do not use `<br>` to create space between paragraphs or blocks. Use separate `<p>` elements and CSS margins instead. A series of `<br><br><br>` to push content down is a classic beginner mistake that breaks on different screen sizes and confuses screen readers.

Syntax

`<br>`

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Flow content, phrasing content
Permitted content	None (void element)
Permitted parents	Any element that accepts phrasing content
Tag omission	No end tag
DOM interface	HTMLBRElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
clear	left right all none	Obsolete. Moved the following text below any floated images. Use CSS clear.	HTML 3.2

: attribute by attribute

Each attribute of
 explained in detail, with a minimal example you can copy.

clear

Values: left | right | all | none | Since: HTML 3.2

Obsolete. Moved the following text below any floated images.

Use CSS clear.

Write clear="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<br clear="left">
```


: worked examples

An address

```
<p>
  Example Ltd.<br>
  12 Nile Street<br>
  Giza, Egypt
</p>
```

A poem

```
<p>
  Roses are red,<br>
  Violets are blue,<br>
  HTML is easy,<br>
  And so are you.
</p>
```

Wrong and right

```
<!-- Wrong: spacing with br -->
<p>First</p><br><br>
<p>Second</p>

<!-- Right: spacing with CSS -->
<style>p { margin-bottom: 2em; }</style>
<p>First</p>
<p>Second</p>
```

Default browser CSS

```
br { display: inline; }
```

Accessibility

- Screen readers may announce each
, so many consecutive breaks are noisy.

Common mistakes

- Using
 for vertical spacing.
- Writing </br>, which is invalid (browsers treat it as
).
- Using
 to separate list items instead of /.

See also

<p>, <wbr>, <hr>, <pre>

Quick card: <button>

Button

The <button> element represents a clickable button that the user can activate with a mouse, keyboard, finger or voice to perform an action. Unlike <input type="button">, a <button> may contain rich content: text, images, icons and other phrasing content.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	type, name, value, disabled, autofocus, form, formaction, formenctype, formmethod, formnovalidate, formtarget, popovertarget, popovertargetaction, command, commandfor
Related	<input>, <form>, <a>, <label>

Minimal example

```
<form action="/save" method="post">
  <label>Name <input name="name"></label>
  <button type="submit">Save</button>
  <button type="reset">Clear</button>
  <button type="button" onclick="alert('Just a script')">Help</button>
</form>
```

Most common mistake

Forgetting type="button" and accidentally submitting the form.

<button> — Button

Category: Forms | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<button>` element represents a clickable button that the user can activate with a mouse, keyboard, finger or voice to perform an action. Unlike `<input type="button">`, a `<button>` may contain rich content: text, images, icons and other phrasing content.

Inside a `<form>`, a button submits the form by default (`type="submit"`). Set `type="button"` for buttons that only run JavaScript, and `type="reset"` to clear the form. Because the default is submit, forgetting `type="button"` is a frequent cause of forms submitting unexpectedly.

The button's form-related attributes (`formaction`, `formmethod` and so on) let one form have several submit buttons that send the data to different places or in different ways.

Syntax

```
<button type="button">label</button>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Flow content, phrasing content, interactive content, listed, labelable, submittable, form-associated, palpable
Permitted content	Phrasing content, but no interactive content and no descendant with the <code>tabindex</code> attribute

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLButtonElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
type	submit reset button	submit (default) sends the form; reset restores default values; button does nothing by itself.	HTML 4.01
name	text	The name sent with the form data when this button submits the form.	HTML 4.01
value	text	The value sent with the name.	HTML 4.01
disabled	boolean	The button cannot be clicked or focused and is not submitted.	HTML 4.01
autofocus	boolean	Focus this button when the page loads (global in HTML5).	HTML5
form	form id	Associates the button with a form elsewhere in the document.	HTML5
formaction	URL	Overrides the form's action for this button.	HTML5
formenctype	application/x-www-form-urlencoded multipart/form-data text/plain	Overrides the form's enctype.	HTML5

Attribute	Values	Description	Since
formmethod	get post dialog	Overrides the form's method.	HTML5
formnovalidate	boolean	Skips validation when this button submits.	HTML5
formtarget	_self _blank _parent _top name	Overrides the form's target.	HTML5
popovertarget	element id	The popover element this button shows, hides or toggles.	Living Standard
popovertargetaction	show hide toggle	What to do to the popover (default toggle).	Living Standard
command	show-modal close request-close show-popover hide-popover toggle-popover custom	Experimental: declarative action to perform on commandfor's target.	Living Standard
commandfor	element id	Experimental: the element the command applies to.	Living Standard

<button>: attribute by attribute

Each attribute of <button> explained in detail, with a minimal example you can copy.

type

Values: submit | reset | button | Since: HTML 4.01

submit (default) sends the form; reset restores default values; button does nothing by itself.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button type="submit">...</button>
```

name

Values: text | Since: HTML 4.01

The name sent with the form data when this button submits the form.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button name="value">...</button>
```

value

Values: text | Since: HTML 4.01

The value sent with the name.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button value="value">...</button>
```

disabled

Values: boolean | Since: HTML 4.01

The button cannot be clicked or focused and is not submitted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<button disabled>...</button>
```

autofocus

Values: boolean | Since: HTML5

Focus this button when the page loads (global in HTML5).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<button autofocus>...</button>
```

form

Values: form id | Since: HTML5

Associates the button with a form elsewhere in the document.

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button form="form id">...</button>
```

formaction

Values: URL | Since: HTML5

Overrides the form's action for this button.

Write `formaction="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button formaction="https://example.com/page.html">...</button>
```

formenctype

Values: application/x-www-form-urlencoded | multipart/form-data | text/plain
| Since: HTML5

Overrides the form's enctype.

Write `formenctype="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button enctype="application">...</button>
```

formmethod

Values: get | post | dialog | Since: HTML5

Overrides the form's method.

Write `formmethod="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button formmethod="get">...</button>
```

formnovalidate

Values: boolean | Since: HTML5

Skips validation when this button submits.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<button formnovalidate>...</button>
```

formtarget

Values: `_self` | `_blank` | `_parent` | `_top` | name | Since: HTML5

Overrides the form's target.

Write `formtarget="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button formtarget="_self">...</button>
```

popovertarget

Values: element id | Since: Living Standard

The popover element this button shows, hides or toggles.

Write `popovertarget="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button popovertarget="element id">...</button>
```

popovertargetaction

Values: `show` | `hide` | `toggle` | Since: Living Standard

What to do to the popover (default toggle).

Write `popovertargetaction="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button popovertargetaction="show">...</button>
```

command

Values: show-modal | close | request-close | show-popover | hide-popover | toggle-popover | custom | Since: Living Standard

Experimental: declarative action to perform on commandfor's target.

Write command="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button command="show-modal">...</button>
```

commandfor

Values: element id | Since: Living Standard

Experimental: the element the command applies to.

Write commandfor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<button commandfor="element id">...</button>
```


<button>: worked examples

Three button types in a form

```
<form action="/save" method="post">
  <label>Name <input name="name"></label>
  <button type="submit">Save</button>
  <button type="reset">Clear</button>
  <button type="button" onclick="alert('Just a script')">Help</button>
</form>
```

Button with an icon

```
<button type="button">
  
  Add to cart
</button>
```

The image is decorative (empty alt) because the text already labels the button.

Two submit buttons going to different URLs

```
<form action="/save" method="post">
  <input name="title">
  <button>Save draft</button>
  <button formaction="/publish">Publish</button>
</form>
```

Toggleing a popover without JavaScript

```
<button popovertarget="tip">Show tip</button>
<div id="tip" popover>Press Ctrl+S to save.</div>
```

Icon-only button with an accessible name

```
<button type="button" aria-label="Close">&times;</button>
```

Default browser CSS

```
button { display: inline-block; text-align: center; cursor: default;
padding: 1px 6px; }
```

Accessibility

- Buttons are focusable and activated with Enter or Space automatically; a <div onclick> is not.
- Every button needs a text label or aria-label.
- Use <button> for actions and <a> for navigation.

- Do not remove the focus outline without providing an alternative.

Common mistakes

- Forgetting `type="button"` and accidentally submitting the form.
- Nesting an `<a>` or another `<button>` inside.
- Using `<div>` or `<span>` styled as a button.
- Using `<input type="button">` when you need HTML content inside.

See also

`<input>`, `<form>`, `<a>`, `<label>`

Quick card: <canvas>

Graphics canvas

The <canvas> element provides a blank, resolution-dependent bitmap surface on which JavaScript can draw graphics: shapes, text, images, animations, charts, games and image processing. The element itself does nothing; all drawing is done through the 2D context (getContext('2d')) or a WebGL / WebGPU context.

At a glance

Status	CURRENT
Versions	HTML5
Category	Scripting
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	width, height, moz-opaque
Related	<svg>, , <script>

Minimal example

```
<canvas id="demo" width="300" height="150">
  A blue rectangle with the text Hello.
</canvas>
<script>
  const ctx = document.getElementById('demo').getContext('2d');
  ctx.fillStyle = '#1b6fd8';
  ctx.fillRect(10, 10, 120, 80);
  ctx.fillStyle = 'white';
  ctx.font = '20px sans-serif';
  ctx.fillText('Hello', 30, 60);
</script>
```

Most common mistake

Sizing the canvas with CSS instead of width/height attributes.

<canvas> — Graphics canvas

Category: Scripting | Status: CURRENT | Versions: HTML5

Description

The <canvas> element provides a blank, resolution-dependent bitmap surface on which JavaScript can draw graphics: shapes, text, images, animations, charts, games and image processing. The element itself does nothing; all drawing is done through the 2D context (getContext('2d')) or a WebGL / WebGPU context.

The width and height attributes set the size of the drawing surface in pixels (default 300 x 150). Setting the size with CSS instead scales the bitmap and makes it blurry; always set the attributes.

Content placed between the tags is fallback content shown by browsers that do not support canvas and read by screen readers, because the drawn pixels are invisible to assistive technology.

Syntax

```
<canvas id="c" width="400" height="300">fallback</canvas>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content, palpable content

Property	Value
Permitted content	Transparent, but no interactive content other than <a>, <button>, <input> of certain types and <select> with focusable descendants
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLCanvasElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
width	positive integer	Width of the bitmap in CSS pixels. Default 300.	HTML5
height	positive integer	Height of the bitmap in CSS pixels. Default 150.	HTML5
moz-opaque	boolean	Firefox only: hints that there is no transparency, for faster rendering.	Non-standard

<canvas>: attribute by attribute

Each attribute of <canvas> explained in detail, with a minimal example you can copy.

width

Values: positive integer | Since: HTML5

Width of the bitmap in CSS pixels. Default 300.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<canvas width="3">...</canvas>
```

height

Values: positive integer | Since: HTML5

Height of the bitmap in CSS pixels. Default 150.

Write height="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<canvas height="3">...</canvas>
```

moz-opaque

Values: boolean | Since: Non-standard

Firefox only: hints that there is no transparency, for faster rendering.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<canvas moz-opaque>...</canvas>
```

<canvas>: worked examples

Drawing a rectangle and text

```
<canvas id="demo" width="300" height="150">
  A blue rectangle with the text Hello.
</canvas>
<script>
  const ctx = document.getElementById('demo').getContext('2d');
  ctx.fillStyle = '#1b6fd8';
  ctx.fillRect(10, 10, 120, 80);
  ctx.fillStyle = 'white';
  ctx.font = '20px sans-serif';
  ctx.fillText('Hello', 30, 60);
</script>
```

Simple animation

```
<canvas id="ball" width="200" height="100"></canvas>
<script>
  const c = document.getElementById('ball'), ctx = c.getContext('2d');
  let x = 0;
  function frame() {
    ctx.clearRect(0, 0, c.width, c.height);
    ctx.beginPath(); ctx.arc(x, 50, 10, 0, Math.PI * 2); ctx.fill();
    x = (x + 2) % c.width;
    requestAnimationFrame(frame);
  }
  frame();
</script>
```

Default browser CSS

```
canvas { display: inline; }
```

Accessibility

- Canvas content is invisible to screen readers. Provide fallback content, an aria-label, or a text alternative next to it.
- For charts, provide the data as a table too.
- For diagrams and icons, SVG is usually more accessible than canvas.

Common mistakes

- Sizing the canvas with CSS instead of width/height attributes.
- Forgetting to clear before redrawing.
- Using canvas for content that should be real HTML text.

See also

`<svg>`, `<img>`, `<script>`

Quick card: <caption>

Table caption

The <caption> element gives a table a title or explanation. It must be the first child of the <table> element, and a table may have only one. By default it is displayed centred above the table; the CSS caption-side property can move it below.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-caption
Own attributes	align
Related	<table>, <figcaption>

Minimal example

```
<table>
  <caption>Monthly sales, 2026</caption>
  <tr><th>Month</th><th>Sales</th></tr>
  <tr><td>January</td><td>120</td></tr>
  <tr><td>February</td><td>150</td></tr>
</table>
```

Most common mistake

Placing <caption> after <tr>.

<caption> — Table caption

Category: Table content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The <caption> element gives a table a title or explanation. It must be the first child of the <table> element, and a table may have only one. By default it is displayed centred above the table; the CSS caption-side property can move it below.

A caption is important for accessibility: screen readers announce it when the user enters the table, so the user knows what the data is about before hearing the cells.

Syntax

```
<table>
  <caption>Title</caption>
  ...
</table>
```

Technical summary

Property	Value
Void element	No
Default display	table-caption
Content categories	None
Permitted content	Flow content, but no descendant <table>
Permitted parents	<table>, as its first child
Tag omission	End tag may be omitted if the caption is not followed by whitespace or a comment
DOM interface	HTMLTableCaptionElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	top bottom left right	Obsolete. Position of the caption. Use CSS caption-side and text-align.	HTML 3.2

<caption>: attribute by attribute

Each attribute of <caption> explained in detail, with a minimal example you can copy.

align

Values: top | bottom | left | right | Since: HTML 3.2

Obsolete. Position of the caption. Use CSS caption-side and text-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<caption align="top">...</caption>
```

<caption>: worked examples

Table with caption

```
<table>
  <caption>Monthly sales, 2026</caption>
  <tr><th>Month</th><th>Sales</th></tr>
  <tr><td>January</td><td>120</td></tr>
  <tr><td>February</td><td>150</td></tr>
</table>
```

Caption below the table

```
<style>
  table { caption-side: bottom; }
</style>
```

Default browser CSS

```
caption { display: table-caption; text-align: center; }
```

Accessibility

- Prefer <caption> over a heading above the table; it is programmatically linked to the table.
- If the table is inside a <figure>, use <figcaption> instead of <caption>, not both.

Common mistakes

- Placing <caption> after <tr>.
- Multiple captions.

See also

<table>, <figcaption>

Quick card: <center>

Centred block (obsolete)

The <center> element horizontally centred its content. It was introduced by Netscape, standardised in HTML 3.2, deprecated in HTML 4.01 and removed in HTML5. It is equivalent to <div align="center">. Use CSS instead: text-align: center for inline content, or margin: 0 auto with a width for blocks.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<div>

Minimal example

```
<!-- Old -->
<center><h1>Welcome</h1></center>

<!-- New: centre text -->
<h1 style="text-align: center">Welcome</h1>

<!-- New: centre a block -->
<div style="width: 600px; margin: 0 auto">...</div>
```

Most common mistake

Any use in new pages.

<center> — Centred block (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The <center> element horizontally centred its content. It was introduced by Netscape, standardised in HTML 3.2, deprecated in HTML 4.01 and removed in HTML5. It is equivalent to <div align="center">. Use CSS instead: text-align: center for inline content, or margin: 0 auto with a width for blocks.

Syntax

```
<center>content</center>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content (historical)
Permitted content	Flow content
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<center>: worked examples

Replacement

```
<!-- Old -->
<center><h1>Welcome</h1></center>

<!-- New: centre text -->
<h1 style="text-align: center">Welcome</h1>

<!-- New: centre a block -->
<div style="width: 600px; margin: 0 auto">...</div>
```

Default browser CSS

```
center { display: block; text-align: -webkit-center; }
```

Common mistakes

- Any use in new pages.

See also

<div>

Quick card: <cite>

Citation (title of a work)

The <cite> element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<blockquote>, <q>, <i>

Minimal example

```
<p>I am reading <cite>The Pragmatic Programmer</cite> this month.</p>
```

Most common mistake

Using <cite> for the URL of a source; that belongs in the cite attribute of <blockquote> or <q>.

<cite> — Citation (title of a work)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <cite> element marks the title of a creative work that is being referenced: a book, article, film, song, painting, play, poem, website, legal case, computer program, and so on. Browsers render it in italics.

In HTML 4 <cite> could also mark the name of a person being quoted. The HTML Living Standard restricts it to titles of works; a person's name is not a title. In practice many pages use it for author names inside <blockquote> footers, and validators accept it.

Syntax

<cite>Title of Work</cite>

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<cite>: worked examples

Book title

```
<p>I am reading <cite>The Pragmatic Programmer</cite> this month.</p>
```

Source of a quotation

```
<blockquote>
  <p>Any fool can write code that a computer can understand.</p>
  <footer>- Martin Fowler, <cite>Refactoring</cite></footer>
</blockquote>
```

Default browser CSS

```
cite { font-style: italic; }
```

Accessibility

- No special announcement; the meaning is mainly for search engines and styling.

Common mistakes

- Using <cite> for the URL of a source; that belongs in the cite attribute of <blockquote> or <q>.
- Using <i> instead of <cite> for titles.

See also

<blockquote>, <q>, <i>

Quick card: `<code>`

Inline code

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><pre></code> , <code><kbd></code> , <code><samp></code> , <code><var></code>

Minimal example

`<p>Use the <code><footer></code> element for the bottom of the page and save the file as <code>index.html</code>.</p>`

Most common mistake

Forgetting to escape `<` and `&` inside code.

<code> — Inline code

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<code>` element marks a fragment of computer code: an element name, a file name, a variable, a function, a command, or any other string a computer would recognise. Browsers render it in a monospace font.

For a multi-line block of code, wrap a `<code>` element in a `<pre>` element so that line breaks and indentation are preserved. The class attribute may be used to indicate the language, conventionally as `language-xxx`.

Syntax

```
<code>snippet</code>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<code>: worked examples

Inline

<p>Use the <code><footer></code> element for the bottom of the page and save the file as <code>index.html</code>.</p>

Note that < and > inside code must still be written as < and >.

Code block

```
<pre><code class="language-css">footer {
  text-align: center;
  color: gray;
}</code></pre>
```

Default browser CSS

```
code { font-family: monospace; }
```

Accessibility

- Some screen readers switch to reading punctuation when they encounter <code>.

Common mistakes

- Forgetting to escape < and & inside code.
- Using <code> alone for multi-line blocks (line breaks collapse without <pre>).

See also

<pre>, <kbd>, <samp>, <var>

Quick card: <col>

Table column

The `<col>` element defines one or more columns in a table so that attributes and styles can be applied to whole columns at once. It is a void element and lives inside a `<colgroup>`. The `span` attribute makes one `<col>` stand for several consecutive columns.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 4.01, HTML5 |
| Category | Table content |
| Closing tag | None (void element) |
| Display | table-column |
| Own attributes | span, align, bgcolor, char, charoff, valign, width |
| Related | <code><colgroup></code> , <code><table></code> |

Minimal example

```
<table>
  <colgroup>
    <col style="background: #f0f0f0">
      <col>
        <col style="width: 200px">
      </colgroup>
    <tr><th>Item</th><th>Qty</th><th>Notes</th></tr>
    <tr><td>Pen</td><td>3</td><td>Blue ink</td></tr>
  </table>
```

Most common mistake

Placing `<col>` outside `<colgroup>` (browsers insert an implicit `colgroup`, but be explicit).

<col> — Table column

Category: Table content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<col>` element defines one or more columns in a table so that attributes and styles can be applied to whole columns at once. It is a void element and lives inside a `<colgroup>`. The `span` attribute makes one `<col>` stand for several consecutive columns.

Only a few CSS properties apply to columns: background, border, width and visibility. Text colour and font cannot be set on a column; set them on the cells.

Syntax

```
<colgroup>
  <col span="2" style="background: #eee">
</colgroup>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | Yes (no closing tag) |
| Default display | table-column |
| Content categories | None |
| Permitted content | None (void element) |
| Permitted parents | <code><colgroup></code> that does not have a <code>span</code> attribute |
| Tag omission | No end tag |
| DOM interface | HTMLTableColElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|--|--|-----------|
| span | positive integer | How many consecutive columns this element represents. Default 1. | HTML 4.01 |
| align | left center right justify char | Obsolete. Horizontal alignment of the cells. Use CSS text-align. | HTML 4.01 |
| bgcolor | colour | Obsolete. Background colour. Use CSS. | HTML 4.01 |
| char | character | Obsolete. Alignment character (for align=char). | HTML 4.01 |
| charoff | number | Obsolete. Offset from the alignment character. | HTML 4.01 |
| valign | top middle bottom baseline | Obsolete. Vertical alignment. Use CSS vertical-align. | HTML 4.01 |
| width | pixels or percentage | Obsolete. Column width. Use CSS width. | HTML 4.01 |

<col>: attribute by attribute

Each attribute of <col> explained in detail, with a minimal example you can copy.

span

Values: positive integer | Since: HTML 4.01

How many consecutive columns this element represents.
Default 1.

Write span="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col span="3">
```

align

Values: left | center | right | justify | char | Since: HTML 4.01

Obsolete. Horizontal alignment of the cells. Use CSS text-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col align="left">
```

bgcolor

Values: colour | Since: HTML 4.01

Obsolete. Background colour. Use CSS.

Write bgcolor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col bgColor="#336699">
```

char

Values: character | Since: HTML 4.01

Obsolete. Alignment character (for align=char).

Write char="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col char="character">
```

charoff

Values: number | Since: HTML 4.01

Obsolete. Offset from the alignment character.

Write `charoff="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col charoff="3">
```

valign

Values: top | middle | bottom | baseline | Since: HTML 4.01

Obsolete. Vertical alignment. Use CSS `vertical-align`.

Write `valign="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col valign="top">
```

width

Values: pixels or percentage | Since: HTML 4.01

Obsolete. Column width. Use CSS `width`.

Write `width="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<col width="3">
```

<col>: worked examples

Shading the first column and widening the last

```
<table>
  <colgroup>
    <col style="background: #f0f0f0">
    <col>
    <col style="width: 200px">
  </colgroup>
  <tr><th>Item</th><th>Qty</th><th>Notes</th></tr>
  <tr><td>Pen</td><td>3</td><td>Blue ink</td></tr>
</table>
```

Default browser CSS

```
col { display: table-column; }
```

Accessibility

- Columns have no accessibility role; use `<th scope="col">` for column headers.

Common mistakes

- Placing `<col>` outside `<colgroup>` (browsers insert an implicit `colgroup`, but be explicit).
- Trying to set font or colour on `<col>`.

See also

`<colgroup>`, `<table>`

Quick card: <colgroup>

Table column group

The <colgroup> element groups one or more columns of a table for the purpose of styling and structure. It is placed inside <table> after the optional <caption> and before any <thead>, <tbody>, <tfoot> or <tr>. It either has a span attribute (and no children) or contains <col> elements.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 4.01, HTML5 |
| Category | Table content |
| Closing tag | Required unless noted under Tag omission |
| Display | table-column-group |
| Own attributes | span, align |
| Related | <col>, <table> |

Minimal example

```
<table>
  <colgroup style="background: #e8f4ff"><col><col></colgroup>
  <colgroup style="background: #fff4e8"><col></colgroup>
  <tr><th>First</th><th>Last</th><th>Age</th></tr>
  <tr><td>Ahmed</td><td>Ali</td><td>20</td></tr>
</table>
```

Most common mistake

Mixing a span attribute with <col> children.

<colgroup> — Table column group

Category: Table content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The <colgroup> element groups one or more columns of a table for the purpose of styling and structure. It is placed inside <table> after the optional <caption> and before any <thead>, <tbody>, <tfoot> or <tr>. It either has a span attribute (and no children) or contains <col> elements.

Syntax

```
<colgroup span="2"></colgroup>
<!-- or -->
<colgroup><col><col></colgroup>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | table-column-group |
| Content categories | None |
| Permitted content | If span is absent: zero or more <col> and <template>. If span is present: nothing |
| Permitted parents | <table>, after <caption> and before <thead>, <tbody>, <tfoot> and <tr> |
| Tag omission | Start tag may be omitted if the first thing inside is a <col>; end tag may be omitted if not followed by whitespace or a comment |
| DOM interface | HTMLTableColElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|---|------------------|--|-----------|
| span | positive integer | Number of columns in the group when there are no <col> children. | HTML 4.01 |
| align,
bgcolor,
char,
charoff,
valign,
width | various | Obsolete presentational attributes; see <col>. | HTML 4.01 |

<colgroup>: attribute by attribute

Each attribute of <colgroup> explained in detail, with a minimal example you can copy.

span

Values: positive integer | Since: HTML 4.01

Number of columns in the group when there are no <col> children.

Write `span="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<colgroup span="3">...</colgroup>
```

align, bgcolor, char, charoff, valign, width

Values: various | Since: HTML 4.01

Obsolete presentational attributes; see <col>.

Write `align="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<colgroup align="various">...</colgroup>
```

<colgroup>: worked examples

Two groups with different backgrounds

```
<table>
  <colgroup style="background: #e8f4ff"><col><col></colgroup>
  <colgroup style="background: #fff4e8"><col></colgroup>
  <tr><th>First</th><th>Last</th><th>Age</th></tr>
  <tr><td>Ahmed</td><td>Ali</td><td>20</td></tr>
</table>
```

Default browser CSS

```
colgroup { display: table-column-group; }
```

Common mistakes

- Mixing a span attribute with <col> children.
- Placing colgroup after tbody.

See also

<col>, <table>

Quick card: <content>

Shadow DOM insertion point (obsolete)

The <content> element was part of the first, experimental version of Shadow DOM (v0). It marked the point where light DOM children were inserted into a shadow tree. It was replaced by the standardised <slot> element and removed from browsers in 2019.

At a glance

| | |
|----------------|--|
| Status | OBSOLETE |
| Versions | Shadow DOM v0 (Chrome, 2013); obsolete |
| Category | Obsolete web components |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | select |
| Related | <slot>, <template>, <shadow> |

Minimal example

```
<!-- Old -->
<content select=".title"></content>
```

```
<!-- New -->
<slot name="title"></slot>
```

Most common mistake

Any use.

<content> — Shadow DOM insertion point (obsolete)

Category: Obsolete web components | Status: OBSOLETE | Versions: Shadow DOM v0 (Chrome, 2013); obsolete

Description

The <content> element was part of the first, experimental version of Shadow DOM (v0). It marked the point where light DOM children were inserted into a shadow tree. It was replaced by the standardised <slot> element and removed from browsers in 2019.

Syntax

```
<content select=".title"></content>
```

Technical summary

| Property | Value |
|--------------------|--------------------|
| Void element | No |
| Default display | inline |
| Content categories | Historical |
| Permitted content | Any |
| Permitted parents | Shadow root |
| Tag omission | Neither |
| DOM interface | HTMLUnknownElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|--------------|-------------------------------------|---------------|
| select | CSS selector | Which light DOM children to insert. | Shadow DOM v0 |

<content>: attribute by attribute

Each attribute of <content> explained in detail, with a minimal example you can copy.

select

Values: CSS selector | Since: Shadow DOM v0

Which light DOM children to insert.

Write `select="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<content select="CSS selector">...</content>
```

<content>: worked examples

Replacement

```
<!-- Old -->  
<content select=".title"></content>
```

```
<!-- New -->  
<slot name="title"></slot>
```

Common mistakes

- Any use.

See also

<slot>, <template>, <shadow>

Quick card: <data>

Machine-readable value

The <data> element links a piece of human-readable content with a machine-readable version of the same value held in the required value attribute. It is useful for product codes, numeric values written in words, sorting keys and similar cases where a script or search engine needs the exact value while the visitor sees a friendly form.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML5 |
| Category | Text-level semantics |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | value |
| Related | <time>, <meter> |

Minimal example

```
<ul>
  <li><data value="SKU-1001">Blue notebook</data></li>
  <li><data value="SKU-1002">Red notebook</data></li>
</ul>
```

Most common mistake

Using <data> for dates; use <time>.

<data> — Machine-readable value

Category: Text-level semantics | Status: CURRENT | Versions: HTML5

Description

The `<data>` element links a piece of human-readable content with a machine-readable version of the same value held in the required value attribute. It is useful for product codes, numeric values written in words, sorting keys and similar cases where a script or search engine needs the exact value while the visitor sees a friendly form.

If the value is a date or time, use `<time>` instead, which has the same purpose but a defined format.

Syntax

```
<data value="machine value">human text</data>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | inline |
| Content categories | Flow content, phrasing content, palpable content |
| Permitted content | Phrasing content |
| Permitted parents | Any element that accepts phrasing content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLDataElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|--------|---|-------|
| value | text | Required. The machine-readable form of the content. | HTML5 |

<data>: attribute by attribute

Each attribute of <data> explained in detail, with a minimal example you can copy.

value

Values: text | Since: HTML5

Required. The machine-readable form of the content.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<data value="value">...</data>
```

<data>: worked examples

Product codes

```
<ul>
  <li><data value="SKU-1001">Blue notebook</data></li>
  <li><data value="SKU-1002">Red notebook</data></li>
</ul>
```

Numbers written in words

```
<p>We sold <data value="1000000">one million</data> copies.</p>
```

Default browser CSS

```
data { display: inline; }
```

Accessibility

- The value attribute is not read by screen readers; the visible text must be understandable alone.

Common mistakes

- Using <data> for dates; use <time>.

See also

<time>, <meter>

Quick card: <datalist>

Predefined options for an input

The <datalist> element contains a set of <option> elements that represent suggested values for an <input>. The input is linked to the list with its list attribute, which must equal the datalist's id. The user can pick a suggestion or type any other value, which makes it a combination of a text box and a drop-down list (a 'combobox').

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML5 |
| Category | Forms |
| Closing tag | Required unless noted under Tag omission |
| Display | none |
| Own attributes | None (global attributes only) |
| Related | <input>, <option>, <select> |

Minimal example

```
<label for="browser">Favourite browser</label>
<input id="browser" name="browser" list="browsers">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

Most common mistake

Mismatched id and list values.

<datalist> — Predefined options for an input

Category: Forms | Status: CURRENT | Versions: HTML5

Description

The `<datalist>` element contains a set of `<option>` elements that represent suggested values for an `<input>`. The input is linked to the list with its `list` attribute, which must equal the datalist's id. The user can pick a suggestion or type any other value, which makes it a combination of a text box and a drop-down list (a 'combobox').

The datalist itself is never displayed. Browsers show the suggestions in their own way, usually as a drop-down that filters as the user types. It works with text, search, url, tel, email, number, range, color, date and time inputs, though support for non-text types varies.

Syntax

```
<input list="id">
<datalist id="id">
  <option value="...">
</datalist>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | none |
| Content categories | Flow content, phrasing content |
| Permitted content | Either phrasing content or zero or more <code><option></code> elements |

| Property | Value |
|-------------------|---|
| Permitted parents | Any element that accepts phrasing content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLDataListElement |

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<datalist>: worked examples

Browser suggestions

```
<label for="browser">Favourite browser</label>
<input id="browser" name="browser" list="browsers">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

Labelled values

```
<input list="codes" name="country">
<datalist id="codes">
  <option value="EG">Egypt</option>
  <option value="SA">Saudi Arabia</option>
</datalist>
```

Some browsers show the label next to the value.

Tick marks on a range slider

```
<input type="range" min="0" max="100" list="ticks">
<datalist id="ticks">
  <option value="0">
  <option value="50">
  <option value="100">
</datalist>
```

Default browser CSS

```
datalist { display: none; }
```

Accessibility

- Support in screen readers is inconsistent; consider a visible hint about available values.

Common mistakes

- Mismatched id and list values.
- Expecting the user to be limited to the options (use <select> for that).

See also

<input>, <option>, <select>

Quick card: <dd>

Description details

The <dd> element provides the description, definition or value for the preceding term (<dt>) in a description list (<dl>). One term may have several <dd> elements, and several terms may share one <dd>. Browsers indent <dd> from the left by default.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5 |
| Category | Text content |
| Closing tag | Required unless noted under Tag omission |
| Display | block |
| Own attributes | nowrap |
| Related | <dl>, <dt> |

Minimal example

```
<dl>
  <dt>HTML</dt>
  <dd>The language that structures web pages.</dd>
  <dt>CSS</dt>
  <dd>The language that styles web pages.</dd>
</dl>
```

Most common mistake

Using <dd> outside <dl>.

<dd> — Description details

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<dd>` element provides the description, definition or value for the preceding term (`<dt>`) in a description list (`<dl>`). One term may have several `<dd>` elements, and several terms may share one `<dd>`. Browsers indent `<dd>` from the left by default.

Syntax

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

Technical summary

| Property | Value |
|--------------------|---|
| Void element | No |
| Default display | block |
| Content categories | None |
| Permitted content | Flow content |
| Permitted parents | <code><dl></code> or <code><div></code> inside <code><dl></code> , after a <code><dt></code> or <code><dd></code> |
| Tag omission | End tag may be omitted if followed by another <code><dd></code> or <code><dt></code> , or if there is no more content in the parent |
| DOM interface | HTMLElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|----------|---|-------|
| nowrap | yes no | Obsolete, non-standard. Prevented wrapping. | IE |

<dd>: attribute by attribute

Each attribute of <dd> explained in detail, with a minimal example you can copy.

nowrap

Values: yes | no | Since: IE

Obsolete, non-standard. Prevented wrapping.

Write nowrap="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<dd nowrap="yes">...</dd>
```


<dd>: worked examples

Glossary

```
<dl>
  <dt>HTML</dt>
  <dd>The language that structures web pages.</dd>
  <dt>CSS</dt>
  <dd>The language that styles web pages.</dd>
</dl>
```

Several descriptions for one term

```
<dl>
  <dt>Footer</dt>
  <dd>The bottom section of a page.</dd>
  <dd>An HTML element: &lt;footer&gt;.</dd>
</dl>
```

Default browser CSS

```
dd { display: block; margin-inline-start: 40px; }
```

Accessibility

- Screen readers announce the list structure so users know which description belongs to which term.

Common mistakes

- Using <dd> outside <dl>.
- Using <dd> just for indentation.

See also

<dl>, <dt>

Quick card: `<del>`

Deleted text

The `<del>` element marks text that has been deleted from a document, typically to show changes between versions ('track changes'). Browsers render it with a line through it. It is usually paired with `<ins>`, which marks inserted text.

At a glance

| | |
|----------------|---|
| Status | CURRENT |
| Versions | HTML 4.01, HTML5 |
| Category | Demarcating edits |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | cite, datetime |
| Related | <code><ins></code> , <code><s></code> , <code><strike></code> |

Minimal example

```
<p>The meeting is on <del datetime="2026-09-10">Monday</del> <ins  
datetime="2026-09-10">Tuesday</ins>.</p>
```

Most common mistake

Using `<del>` for a discounted price; use `<s>`.

 — Deleted text

Category: Demarcating edits | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<del>` element marks text that has been deleted from a document, typically to show changes between versions ('track changes'). Browsers render it with a line through it. It is usually paired with `<ins>`, which marks inserted text.

`<del>` may contain either phrasing or flow content depending on where it is placed, but it should not cross the boundaries of block elements: to delete two paragraphs, wrap each in its own `<del>`.

For text that is no longer accurate but not part of an edit history (such as an old price), use `<s>` instead.

Syntax

```
<del datetime="YYYY-MM-DD" cite="URL">deleted text</del>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | inline |
| Content categories | Flow content, phrasing content, palpable content |
| Permitted content | Transparent |
| Permitted parents | Any element that accepts phrasing content |
| Tag omission | Neither tag may be omitted |

| Property | Value |
|---------------|----------------|
| DOM interface | HTMLModElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|--------------------------|---|-----------|
| cite | URL | A URL explaining the change, such as a link to a change log or meeting minutes. | HTML 4.01 |
| datetime | date or date-time string | When the change was made, in the format YYYY-MM-DD or YYYY-MM-DDThh:mm:ssZ. | HTML 4.01 |

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

cite

Values: URL | Since: HTML 4.01

A URL explaining the change, such as a link to a change log or meeting minutes.

Write cite="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<del cite="https://example.com/page.html">...</del>
```

datetime

Values: date or date-time string | Since: HTML 4.01

When the change was made, in the format YYYY-MM-DD or YYYY-MM-DDThh:mm:ssZ.

Write datetime="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<del datetime="date">...</del>
```

: worked examples

Showing a correction

```
<p>The meeting is on <del datetime="2026-09-10">Monday</del> <ins  
datetime="2026-09-10">Tuesday</ins>.</p>
```

A completed to-do item

```
<ul>  
  <li><del>Buy milk</del></li>  
  <li>Write HTML</li>  
</ul>
```

Default browser CSS

```
del { text-decoration: line-through; }
```

Accessibility

- Most screen readers do not announce by default. If the deletion is important, add visually hidden text such as 'deleted:' or use CSS ::before content with speak.

Common mistakes

- Using for a discounted price; use <s>.
- Using <strike>, which is obsolete.

See also

<ins>, <s>, <strike>

Quick card: <details>

Disclosure widget

The <details> element creates a disclosure widget: a box that can be opened and closed by the user to show or hide its content, with no JavaScript at all. The first child should be a <summary> element, which provides the always-visible label and the clickable arrow. If there is no <summary> the browser shows the word 'Details'.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML5 (5.1) |
| Category | Interactive elements |
| Closing tag | Required unless noted under Tag omission |
| Display | block |
| Own attributes | open, name |
| Related | <summary>, <dialog> |

Minimal example

```
<details>
  <summary>How do I centre a div?</summary>
  <p>Use <code>display: flex; justify-content: center; align-items:
center;</code> on the parent.</p>
</details>
```

Most common mistake

Putting <summary> anywhere other than first.

<details> — Disclosure widget

Category: Interactive elements | Status: CURRENT | Versions: HTML5 (5.1)

Description

The `<details>` element creates a disclosure widget: a box that can be opened and closed by the user to show or hide its content, with no JavaScript at all. The first child should be a `<summary>` element, which provides the always-visible label and the clickable arrow. If there is no `<summary>` the browser shows the word 'Details'.

The `open` attribute controls whether the content is visible. Adding or removing it from script opens or closes the widget, and the element fires a `toggle` event when its state changes. Giving several `<details>` the same `name` attribute makes them exclusive, like an accordion: opening one closes the others.

Syntax

```
<details>
  <summary>Label</summary>
  content
</details>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | block |
| Content categories | Flow content, sectioning root, interactive content, palpable content |
| Permitted content | One <code><summary></code> followed by flow content |

| Property | Value |
|-------------------|---------------------------------------|
| Permitted parents | Any element that accepts flow content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLDetailsElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|---------|--|-----------------|
| open | boolean | The details are visible. Absent means collapsed. | HTML5 |
| name | text | Groups details elements so that only one in the group can be open at a time. | Living Standard |

<details>: attribute by attribute

Each attribute of <details> explained in detail, with a minimal example you can copy.

open

Values: boolean | Since: HTML5

The details are visible. Absent means collapsed.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<details open>...</details>
```

name

Values: text | Since: Living Standard

Groups details elements so that only one in the group can be open at a time.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<details name="value">...</details>
```

<details>: worked examples

FAQ item

```
<details>
  <summary>How do I centre a div?</summary>
  <p>Use <code>display: flex; justify-content: center; align-items:
center;</code> on the parent.</p>
</details>
```

Open by default

```
<details open>
  <summary>Shipping information</summary>
  <p>Delivery takes 2-4 days.</p>
</details>
```

Accordion with the name attribute

```
<details name="faq"><summary>Question 1</summary><p>Answer 1</p></
details>
<details name="faq"><summary>Question 2</summary><p>Answer 2</p></
details>
```

Styling the arrow

```
<style>
  summary { cursor: pointer; font-weight: bold; }
  details[open] summary { color: #b90000; }
</style>
```

Default browser CSS

```
details { display: block; }
```

Accessibility

- Fully keyboard accessible and announced as a button with expanded/collapsed state, out of the box.
- Do not put interactive elements inside <summary>.

Common mistakes

- Putting <summary> anywhere other than first.
- Using open="false" (which still means open).

See also

<summary>, <dialog>

Quick card: <dfn>

Definition term

The <dfn> element marks the term that is being defined in the surrounding sentence or paragraph. The nearest paragraph, description list group or section is the definition. Browsers render it in italics. If the element has a title attribute, that is the term being defined; if it contains an <abbr> with a title, the abbreviation's expansion is the term; otherwise the text content is the term.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 3.2, HTML 4.01, HTML5 |
| Category | Text-level semantics |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | title |
| Related | <abbr>, <dl>, <dt> |

Minimal example

```
<p>A <dfn>void element</dfn> is an element that cannot have any content and therefore has no closing tag.</p>
```

Most common mistake

Using <dfn> on every occurrence of a term; use it only where it is defined.

<dfn> — Definition term

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<dfn>` element marks the term that is being defined in the surrounding sentence or paragraph. The nearest paragraph, description list group or section is the definition. Browsers render it in italics. If the element has a `title` attribute, that is the term being defined; if it contains an `<abbr>` with a `title`, the abbreviation's expansion is the term; otherwise the text content is the term.

Syntax

```
<dfn>term</dfn>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content, but no <code><dfn></code> descendants
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
title	text	The term being defined, if different from the content.	HTML 4.01

<dfn>: attribute by attribute

Each attribute of <dfn> explained in detail, with a minimal example you can copy.

title

Values: text | Since: HTML 4.01

The term being defined, if different from the content.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<dfn title="value">...</dfn>
```

<dfn>: worked examples

Defining a term

<p>A <dfn>void element</dfn> is an element that cannot have any content and therefore has no closing tag.</p>

Linking to the definition

<p>A <dfn id="def-footer">footer</dfn> is the bottom section of a page.</p>

...

<p>Put the copyright in the footer.</p>

Default browser CSS

```
dfn { font-style: italic; }
```

Accessibility

- No special announcement; purely semantic.

Common mistakes

- Using <dfn> on every occurrence of a term; use it only where it is defined.

See also

<abbr>, <dl>, <dt>

Quick card: <dialog>

Dialog box

The <dialog> element represents a dialog box or other interactive component such as a confirmation prompt, an alert, a subwindow or a modal overlay. It is hidden by default and is shown with the open attribute or, preferably, with the JavaScript methods show() (non-modal) and showModal() (modal). A modal dialog blocks interaction with the rest of the page, is displayed on top of everything else, and can be closed with the Escape key.

At a glance

Status	CURRENT
Versions	HTML 5.2 / Living Standard
Category	Interactive elements
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	open, closedby
Related	<details>, <form>, <button>

Minimal example

```
<dialog id="confirm">
  <form method="dialog">
    <p>Delete this item?</p>
    <button value="cancel">Cancel</button>
    <button value="ok">Delete</button>
  </form>
</dialog>
<button onclick="document.getElementById('confirm').showModal(
)">Delete</button>
<script>
  const d = document.getElementById('confirm');
```

```
d.addEventListener('close', () => console.log('User chose', d.  
returnValue));  
</script>
```

Most common mistake

Setting the open attribute directly for modals (no backdrop, no focus trap).

<dialog> — Dialog box

Category: Interactive elements | Status: CURRENT | Versions: HTML 5.2 / Living Standard

Description

The `<dialog>` element represents a dialog box or other interactive component such as a confirmation prompt, an alert, a subwindow or a modal overlay. It is hidden by default and is shown with the `open` attribute or, preferably, with the JavaScript methods `show()` (non-modal) and `showModal()` (modal). A modal dialog blocks interaction with the rest of the page, is displayed on top of everything else, and can be closed with the Escape key.

A `<form method="dialog">` inside the dialog closes it when submitted and sets the dialog's `returnValue` to the value of the submit button. The `::backdrop` pseudo-element styles the dimmed area behind a modal dialog.

Syntax

```
<dialog id="d">...</dialog>
<script>document.getElementById('d').showModal()</script>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning root
Permitted content	Flow content

Property	Value
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLDialogElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
open	boolean	The dialog is visible. Set it with showModal() rather than by hand so that focus and the backdrop work correctly.	HTML 5.2
closedby	any closerequest none	Which user actions close the dialog: any (Escape, backdrop click, close request), closerequest (Escape only, the default for modal), or none.	Living Standard

<dialog>: attribute by attribute

Each attribute of <dialog> explained in detail, with a minimal example you can copy.

open

Values: boolean | Since: HTML 5.2

The dialog is visible. Set it with showModal() rather than by hand so that focus and the backdrop work correctly.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<dialog open>...</dialog>
```

closedby

Values: any | closerequest | none | Since: Living Standard

Which user actions close the dialog: any (Escape, backdrop click, close request), closerequest (Escape only, the default for modal), or none.

Write closedby="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<dialog closedby="any">...</dialog>
```

<dialog>: worked examples

Modal confirmation

```
<dialog id="confirm">
  <form method="dialog">
    <p>Delete this item?</p>
    <button value="cancel">Cancel</button>
    <button value="ok">Delete</button>
  </form>
</dialog>
<button onclick="document.getElementById('confirm').showModal(
)">Delete</button>
<script>
  const d = document.getElementById('confirm');
  d.addEventListener('close', () => console.log('User chose', d.
returnValue));
</script>
```

Styling the backdrop

```
<style>
  dialog::backdrop { background: rgba(0, 0, 0, 0.6); }
  dialog { border: none; border-radius: 12px; padding: 24px; }
</style>
```

Default browser CSS

```
dialog { position: absolute; inset-inline: 0; width: fit-content;
margin: auto; border: solid; padding: 1em; background: Canvas; color:
CanvasText; }
dialog:not([open]) { display: none; }
```

Accessibility

- `showModal()` traps focus inside the dialog and returns it when closed, which custom modals often get wrong.
- Give the dialog an accessible name with `aria-labelledby` pointing at its heading.
- Always provide a visible close button.

Common mistakes

- Setting the `open` attribute directly for modals (no backdrop, no focus trap).
- Nesting a dialog inside a button or link.

See also

`<details>`, `<form>`, `<button>`

Quick card: <dir>

Directory list (obsolete)

The <dir> element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like . It was deprecated in HTML 4.01 and removed in HTML5. Use instead.

At a glance

Status	OBSOLETE
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	compact
Related	, <menu>

Minimal example

```
<!-- Old -->
<dir><li>index.html</li><li>style.css</li></dir>
```

```
<!-- New -->
<ul><li>index.html</li><li>style.css</li></ul>
```

Most common mistake

Any use.

<dir> — Directory list (obsolete)

Category: Obsolete text content | Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The `<dir>` element was a list of short items, intended to be shown in columns like a file directory listing. Browsers never implemented the columns and rendered it like `<ul>`. It was deprecated in HTML 4.01 and removed in HTML5. Use `<ul>` instead.

Syntax

```
<dir><li>item</li></dir>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Historical
Permitted content	<code></code> elements
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLDirectoryElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
compact	boolean	Obsolete. Render more compactly.	HTML 2.0

<dir>: attribute by attribute

Each attribute of <dir> explained in detail, with a minimal example you can copy.

compact

Values: boolean | Since: HTML 2.0

Obsolete. Render more compactly.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<dir compact>...</dir>
```

<dir>: worked examples

Replacement

```
<!-- Old -->
<dir><li>index.html</li><li>style.css</li></dir>
```

```
<!-- New -->
<ul><li>index.html</li><li>style.css</li></ul>
```

Default browser CSS

```
dir { display: block; list-style-type: disc; margin: 1em 0; padding-
inline-start: 40px; }
```

Common mistakes

- Any use.

See also

, <menu>

Quick card: <div>

Generic block container

The <div> element (division) is a generic container for flow content. It has no meaning of its own and no effect on the content or layout until it is styled with CSS (using class or id) or manipulated with JavaScript. Use it to group elements for styling purposes, for layout (flexbox, grid), or when no other element is more appropriate.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	align
Related	, <section>, <article>, <main>, <header>, <footer>

Minimal example

```
<style>
  .card { border: 1px solid #ccc; border-radius: 8px; padding: 16px; }
</style>
<div class="card">
  <h2>Pizza</h2>
  <p>Delicious and cheesy.</p>
</div>
```

Most common mistake

Using <div> where a semantic element exists.

<div> — Generic block container

Category: Text content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<div>` element (division) is a generic container for flow content. It has no meaning of its own and no effect on the content or layout until it is styled with CSS (using class or id) or manipulated with JavaScript. Use it to group elements for styling purposes, for layout (flexbox, grid), or when no other element is more appropriate.

Because it carries no meaning, `<div>` should be a last resort. Before using it ask whether `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>` or another element describes the content better. A page built only from divs (so-called 'div soup') works visually but tells assistive technology and search engines nothing about its structure.

The inline counterpart of `<div>` is `<span>`. Since HTML5, `<div>` may also be used inside `<dl>` to wrap a `<dt>`/`<dd>` group.

Syntax

```
<div class="name">...</div>
```

Technical summary

Property	Value
Void element	No
Default display	block

Property	Value
Content categories	Flow content, palpable content
Permitted content	Flow content; inside <dl>: one or more <dt> followed by one or more <dd>
Permitted parents	Any element that accepts flow content, and <dl>
Tag omission	Neither tag may be omitted
DOM interface	HTMLDivElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right justify	Obsolete. Horizontal alignment of the content. Use CSS text-align or margin.	HTML 3.2

<div>: attribute by attribute

Each attribute of <div> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | justify | Since: HTML 3.2

Obsolete. Horizontal alignment of the content. Use CSS `text-align` or `margin`.

Write `align="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<div align="left">...</div>
```


<div>: worked examples

Grouping for styling

```
<style>
  .card { border: 1px solid #ccc; border-radius: 8px; padding: 16px; }
</style>
<div class="card">
  <h2>Pizza</h2>
  <p>Delicious and cheesy.</p>
</div>
```

Layout with flexbox

```
<style>
  .row { display: flex; gap: 16px; }
  .row > div { flex: 1; background: #eee; padding: 8px; }
</style>
<div class="row">
  <div>Column 1</div>
  <div>Column 2</div>
  <div>Column 3</div>
</div>
```

Grouping a term and description in a list

```
<dl>
  <div>
    <dt>Name</dt>
    <dd>Ahmed</dd>
  </div>
  <div>
    <dt>Age</dt>
    <dd>20</dd>
  </div>
</dl>
```

Div versus semantic elements

```
<!-- Div soup -->
<div class="header">...</div>
<div class="nav">...</div>
<div class="footer">...</div>

<!-- Semantic -->
<header>...</header>
<nav>...</nav>
<footer>...</footer>
```

Both look identical; the second is understood by screen readers and search engines.

Default browser CSS

```
div { display: block; }
```

Accessibility

- A `<div>` has no role. If you make it interactive with JavaScript you must add role, tabindex and keyboard handling yourself; usually a `<button>` is the right choice instead.

Common mistakes

- Using `<div>` where a semantic element exists.
- Using `<div>` as a button or link.
- Putting `<div>` inside `<p>` (invalid; the browser closes the paragraph).

See also

`<span>`, `<section>`, `<article>`, `<main>`, `<header>`, `<footer>`

Quick card: <dl>

Description list

The <dl> element represents a description list: a list of groups of terms (<dt>) and their descriptions (<dd>). Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	compact
Related	<dt>, <dd>, ,

Minimal example

```
<dl>
  <dt>Tag</dt>
  <dd>The markup inside angle brackets, such as &lt;p&gt;.</dd>
  <dt>Element</dt>
  <dd>The opening tag, content and closing tag together.</dd>
</dl>
```

Most common mistake

Putting anything other than <dt>, <dd>, <div>, <script>, <template> directly inside <dl>.

<dl> — Description list

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<dl>` element represents a description list: a list of groups of terms (`<dt>`) and their descriptions (`<dd>`). Common uses are glossaries, dictionaries, metadata displays (key: value pairs), FAQ lists and product specifications.

In HTML 4 it was called a 'definition list' and meant only for definitions. HTML5 renamed it and broadened the meaning to any name-value groups. Since HTML5 each group may optionally be wrapped in a `<div>` for styling.

Syntax

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content; palpable if it contains at least one group
Permitted content	Zero or more groups each consisting of one or more <code><dt></code> followed by one or more <code><dd></code> , optionally mixed with <code><script></code> and <code><template></code> ; or zero or more <code><div></code> elements each containing such a group
Permitted parents	Any element that accepts flow content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLDListElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
compact	boolean	Obsolete. Render more compactly.	HTML 2.0

<dl>: attribute by attribute

Each attribute of <dl> explained in detail, with a minimal example you can copy.

compact

Values: boolean | Since: HTML 2.0

Obsolete. Render more compactly.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<dl compact>...</dl>
```

<dl>: worked examples

Glossary

```
<dl>
  <dt>Tag</dt>
  <dd>The markup inside angle brackets, such as &lt;p&gt;.</dd>
  <dt>Element</dt>
  <dd>The opening tag, content and closing tag together.</dd>
</dl>
```

Product specifications as key-value pairs

```
<dl>
  <dt>Weight</dt>
  <dd>1.2 kg</dd>
  <dt>Colour</dt>
  <dd>Black</dd>
  <dt>Warranty</dt>
  <dd>2 years</dd>
</dl>
```

Multiple terms sharing one description

```
<dl>
  <dt>Colour</dt>
  <dt>Color</dt>
  <dd>British and American spellings of the same word.</dd>
</dl>
```

Horizontal layout with CSS grid

```
<style>
  dl { display: grid; grid-template-columns: max-content 1fr; gap:
4px 16px; }
  dt { font-weight: bold; }
  dd { margin: 0; }
</style>
```

Default browser CSS

```
dl { display: block; margin: 1em 0; }
```

Accessibility

- Screen readers announce it as a list with terms and descriptions (support varies).
- Do not use <dl> to mark up dialogue (speaker: line); it is not a name-value list.

Common mistakes

- Putting anything other than `<dt>`, `<dd>`, `<div>`, `<script>`, `<template>` directly inside `<dl>`.
- Using `<dl>` for indentation.

See also

`<dt>`, `<dd>`, `<ul>`, `<ol>`

Quick card: <dt>

Description term

The <dt> element specifies a term, name or key in a description list (<dl>). It is followed by one or more <dd> elements that describe it. Several <dt> elements in a row share the following <dd>. A <dt> may not contain headings, sections, headers or footers.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<dl>, <dd>, <dfn>

Minimal example

```
<dl>
  <dt>Footer</dt>
  <dd>The bottom section of a page.</dd>
</dl>
```

Most common mistake

Using <dt> outside <dl>.

<dt> — Description term

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<dt>` element specifies a term, name or key in a description list (`<dl>`). It is followed by one or more `<dd>` elements that describe it. Several `<dt>` elements in a row share the following `<dd>`. A `<dt>` may not contain headings, sections, headers or footers.

Syntax

```
<dt>Term</dt>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	Flow content, but no <code><header></code> , <code><footer></code> , sectioning content or heading content
Permitted parents	<code><dl></code> or <code><div></code> inside <code><dl></code> , before a <code><dd></code> or another <code><dt></code>
Tag omission	End tag may be omitted if followed by another <code><dt></code> or a <code><dd></code>
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<dt>: worked examples

Term and description

```
<dl>
  <dt>Footer</dt>
  <dd>The bottom section of a page.</dd>
</dl>
```

Default browser CSS

```
dt { display: block; }
```

Common mistakes

- Using <dt> outside <dl>.
- Putting a heading inside <dt>.

See also

<dl>, <dd>, <dfn>

Quick card: `<em>`

Emphasis

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code></code> , <code><i></code> , <code></code> , <code><mark></code> , <code><cite></code>

Minimal example

```
<p><em>I</em> did not say he stole the money.</p>
<p>I did not say he <em>stole</em> the money.</p>
<p>I did not say he stole <em>the money</em>.</p>
```

Most common mistake

Using `<em>` for every italic, such as book titles (`<cite>`) or foreign words (`<i>`).

 — Emphasis

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<em>` element marks text that has stress emphasis: the word or phrase you would say more forcefully if you were reading the sentence aloud. Changing which word is emphasised changes the meaning of the sentence. Browsers render it in italics.

`<em>` is not the same as `<i>`. `<i>` marks text in an alternative voice or mood (a technical term, a foreign phrase, a ship name) without emphasis. And it is not the same as `<strong>`, which marks importance, seriousness or urgency rather than spoken stress. Nested `<em>` elements indicate a greater degree of emphasis.

If you only want italic text for visual reasons, with no emphasis intended, use CSS `font-style: italic` on a `<span>` instead.

Syntax

```
<em>text</em>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content

Property	Value
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

: worked examples

Emphasis changes meaning

```
<p><em>I</em> did not say he stole the money.</p>
<p>I did not say he <em>stole</em> the money.</p>
<p>I did not say he stole <em>the money</em>.</p>
```

Each sentence has the same words but a different meaning because of where the stress falls.

em, i and strong compared

```
<p>You <em>must</em> read the <i>Odyssey</i>. <strong>Warning:</strong> it is long.</p>
```

Default browser CSS

```
em { font-style: italic; }
```

Accessibility

- Some screen readers change voice pitch for ; most do not by default, so do not rely on it to carry critical meaning.

Common mistakes

- Using for every italic, such as book titles (<cite>) or foreign words (<i>).
- Using for whole paragraphs.

See also

, <i>, , <mark>, <cite>

Quick card: <embed>

External content

The <embed> element embeds external content at a specified point in the document. The content is provided by an external application or plug-in, or is a natively supported type such as an image, video, PDF or HTML page. It is a void element with no fallback content.

At a glance

Status	CURRENT
Versions	Netscape extension (1995), standardised in HTML5
Category	Embedded content
Closing tag	None (void element)
Display	inline
Own attributes	src, type, width, height, Any other attribute
Related	<object>, <iframe>, <video>, <audio>, <picture>

Minimal example

```
<embed src="guide.pdf" type="application/pdf" width="100%" height="600">
```

Most common mistake

Using <embed> for video or audio.

<embed> — External content

Category: Embedded content | Status: CURRENT | Versions: Netscape extension (1995), standardised in HTML5

Description

The `<embed>` element embeds external content at a specified point in the document. The content is provided by an external application or plug-in, or is a natively supported type such as an image, video, PDF or HTML page. It is a void element with no fallback content.

It was a Netscape invention widely used for Flash movies and was finally standardised in HTML5. Today, for video use `<video>`, for audio use `<audio>`, for other pages use `<iframe>`, and for PDFs `<iframe>` or `<object>` are more flexible because they allow fallback content.

Syntax

```
<embed src="file.pdf" type="application/pdf" width="600" height="400">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Flow content, phrasing content, embedded content, interactive content, palpable content
Permitted content	None (void element)
Permitted parents	Any element that accepts embedded content
Tag omission	No end tag

Property	Value
DOM interface	HTMLEmbedElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The URL of the resource.	Netscape / HTML5
type	MIME type	The type of the resource, used to choose the plug-in or handler.	Netscape / HTML5
width	pixels	Width of the display area.	Netscape / HTML5
height	pixels	Height of the display area.	Netscape / HTML5
Any other attribute	text	Additional attributes are passed to the plug-in as parameters (for example, old Flash used quality, wmode, allowfullscreen).	Netscape

<embed>: attribute by attribute

Each attribute of <embed> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: Netscape / HTML5

The URL of the resource.

Write `src="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<embed src="https://example.com/page.html">
```

type

Values: MIME type | Since: Netscape / HTML5

The type of the resource, used to choose the plug-in or handler.

Write `type="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<embed type="text/plain">
```

width

Values: pixels | Since: Netscape / HTML5

Width of the display area.

Write `width="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<embed width="3">
```

height

Values: pixels | Since: Netscape / HTML5

Height of the display area.

Write `height="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<embed height="3">
```

Any other attribute

Values: text | Since: Netscape

Additional attributes are passed to the plug-in as parameters (for example, old Flash used `quality`, `wmode`, `allowfullscreen`).

Write `Any other attribute="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<embed Any other attribute="value">
```

<embed>: worked examples

Embedding a PDF

```
<embed src="guide.pdf" type="application/pdf" width="100%" height="600">
```

Embedding a video (prefer <video>)

```
<embed src="clip.mp4" type="video/mp4" width="640" height="360">
```

Default browser CSS

```
embed { display: inline; }
```

Accessibility

- There is no fallback content and no accessible name; add a title attribute and provide a link to the resource next to it.

Common mistakes

- Using <embed> for video or audio.
- Writing a closing tag.

See also

<object>, <iframe>, <video>, <audio>, <picture>

Quick card: <fencedframe>

Fenced frame (experimental)

The <fencedframe> element embeds another HTML page in a way that is even more isolated than an <iframe>: the embedded page cannot communicate with the embedding page, and the URL is chosen through privacy-preserving APIs rather than given directly. It is part of the Privacy Sandbox proposals for showing personalised advertisements without cross-site tracking. It is not yet a standard and is only available in Chromium browsers.

At a glance

Status	EXPERIMENTAL
Versions	Experimental (Chromium, 2024)
Category	Embedded content
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	allow, width
Related	<iframe>

Minimal example

```
<fencedframe id="ad"></fencedframe>
<script>
  // config comes from an API such as Protected Audience
  // document.getElementById('ad').config = config;
</script>
```

Most common mistake

Using it in production sites today.

<fencedframe> — Fenced frame (experimental)

Category: Embedded content | Status: EXPERIMENTAL | Versions: Experimental (Chromium, 2024)

Description

The <fencedframe> element embeds another HTML page in a way that is even more isolated than an <iframe>: the embedded page cannot communicate with the embedding page, and the URL is chosen through privacy-preserving APIs rather than given directly. It is part of the Privacy Sandbox proposals for showing personalised advertisements without cross-site tracking. It is not yet a standard and is only available in Chromium browsers.

Syntax

```
<fencedframe width="300" height="250"></fencedframe>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content, interactive content, palpable content
Permitted content	None
Permitted parents	Any element that accepts embedded content
Tag omission	Neither
DOM interface	HTMLFencedFrameElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
allow	permissions policy	Features allowed inside the frame.	Experimental
width / height	pixels	Size of the frame.	Experimental

<fencedframe>: attribute by attribute

Each attribute of <fencedframe> explained in detail, with a minimal example you can copy.

allow

Values: permissions policy | Since: Experimental

Features allowed inside the frame.

Write allow="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<fencedframe allow="permissions policy">...</fencedframe>
```

width / height

Values: pixels | Since: Experimental

Size of the frame.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<fencedframe width="3">...</fencedframe>
```

<fencedframe>: worked examples

Configured from JavaScript

```
<fencedframe id="ad"></fencedframe>
<script>
  // config comes from an API such as Protected Audience
  // document.getElementById('ad').config = config;
</script>
```

Accessibility

- Same considerations as <iframe>.

Common mistakes

- Using it in production sites today.

See also

<iframe>

Quick card: <fieldset>

Form field group

The <fieldset> element groups related controls and labels inside a form, for example all the fields of a postal address, or a set of radio buttons that answer one question. The optional <legend> element, which must be the first child, gives the group a caption. Browsers draw a border around the group with the legend placed on the border.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	disabled, form, name
Related	<legend>, <form>, <input>, <label>

Minimal example

```
<fieldset>
  <legend>Payment method</legend>
  <label><input type="radio" name="pay" value="cash" checked> Cash on
delivery</label><br>
  <label><input type="radio" name="pay" value="wallet"> Mobile
wallet</label><br>
  <label><input type="radio" name="pay" value="card"> Card</label>
</fieldset>
```

Most common mistake

Using a heading instead of <legend> (the heading is not linked to the group).

<fieldset> — Form field group

Category: Forms | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The <fieldset> element groups related controls and labels inside a form, for example all the fields of a postal address, or a set of radio buttons that answer one question. The optional <legend> element, which must be the first child, gives the group a caption. Browsers draw a border around the group with the legend placed on the border.

Setting disabled on a fieldset disables every control inside it at once, which is convenient for showing or hiding optional sections of a form.

Syntax

```
<fieldset>
  <legend>Caption</legend>
  ...controls...
</fieldset>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning root, listed, form-associated, palpable content
Permitted content	An optional <legend> followed by flow content
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLFieldSetElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
disabled	boolean	Disables all descendant form controls (except those inside the <legend>).	HTML 4.01
form	form id	Associates the fieldset with a form elsewhere in the document.	HTML5
name	text	The name of the group, accessible from the form's elements collection.	HTML5

<fieldset>: attribute by attribute

Each attribute of <fieldset> explained in detail, with a minimal example you can copy.

disabled

Values: boolean | Since: HTML 4.01

Disables all descendant form controls (except those inside the <legend>).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<fieldset disabled>...</fieldset>
```

form

Values: form id | Since: HTML5

Associates the fieldset with a form elsewhere in the document.

Write form="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<fieldset form="form id">...</fieldset>
```

name

Values: text | Since: HTML5

The name of the group, accessible from the form's elements collection.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<fieldset name="value">...</fieldset>
```

<fieldset>: worked examples

Radio button group

```
<fieldset>
  <legend>Payment method</legend>
  <label><input type="radio" name="pay" value="cash" checked> Cash on
delivery</label><br>
  <label><input type="radio" name="pay" value="wallet"> Mobile
wallet</label><br>
  <label><input type="radio" name="pay" value="card"> Card</label>
</fieldset>
```

Disabling a whole section

```
<fieldset disabled>
  <legend>Shipping address (same as billing)</legend>
  <input name="street" placeholder="Street">
  <input name="city" placeholder="City">
</fieldset>
```

Removing the default border

```
<style>
  fieldset { border: none; padding: 0; margin: 0 0 1em; }
  legend { font-weight: bold; padding: 0; }
</style>
```

Default browser CSS

```
fieldset { display: block; margin-inline: 2px; padding: 0.35em 0.75em
0.625em; border: 2px groove; min-inline-size: min-content; }
```

Accessibility

- A fieldset with a legend is the standard, well-supported way to give a group of radio buttons or checkboxes a shared accessible name.
- Screen readers announce the legend when the user enters the group.

Common mistakes

- Using a heading instead of <legend> (the heading is not linked to the group).
- Nesting fieldsets too deeply.

- Forgetting that fieldset is display: block and has odd default styling that is hard to override (for example, it ignores display: flex in some browsers).

See also

`<legend>`, `<form>`, `<input>`, `<label>`

Quick card: <figcaption>

Figure caption

The <figcaption> element provides a caption or legend for the content of its parent <figure> element. It must be either the first or the last child of the <figure>. The caption is programmatically associated with the figure, which gives the figure an accessible name.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<figure>, <caption>,

Minimal example

```
<figure>
  
  <figcaption>Figure 1: The Giza pyramids, photographed in 2026.</
figcaption>
</figure>
```

Most common mistake

Placing figcaption in the middle of the figure.

<figcaption> — Figure caption

Category: Text content | Status: CURRENT | Versions: HTML5

Description

The <figcaption> element provides a caption or legend for the content of its parent <figure> element. It must be either the first or the last child of the <figure>. The caption is programmatically associated with the figure, which gives the figure an accessible name.

Syntax

```
<figure>
  <img ...>
  <figcaption>Caption</figcaption>
</figure>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	Flow content
Permitted parents	<figure>, as its first or last child
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<figcaption>: worked examples

Image with caption

```
<figure>
  
  <figcaption>Figure 1: The Giza pyramids, photographed in 2026.</
figcaption>
</figure>
```

Caption first

```
<figure>
  <figcaption>Listing 2: A minimal page</figcaption>
  <pre><code>&lt;!DOCTYPE html&gt;...</code></pre>
</figure>
```

Default browser CSS

```
figcaption { display: block; }
```

Accessibility

- The caption becomes the figure's accessible name. The image's alt should describe the image; the caption should not merely repeat it.

Common mistakes

- Placing figcaption in the middle of the figure.
- Using figcaption outside a figure.

See also

<figure>, <caption>,

Quick card: <figure>

Figure with optional caption

The <figure> element represents self-contained content that is referenced from the main flow but could be moved elsewhere (to an appendix, a sidebar or another page) without affecting the meaning of the text: an illustration, a diagram, a photo, a code listing, a chart, a poem, a table. An optional <figcaption> gives it a caption.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<figcaption>, , <picture>, <table>

Minimal example

```
<figure>
  
  <figcaption>Feluccas on the Nile.</figcaption>
</figure>
```

Most common mistake

Wrapping every image in a figure regardless of meaning.

<figure> — Figure with optional caption

Category: Text content | Status: CURRENT | Versions: HTML5

Description

The `<figure>` element represents self-contained content that is referenced from the main flow but could be moved elsewhere (to an appendix, a sidebar or another page) without affecting the meaning of the text: an illustration, a diagram, a photo, a code listing, a chart, a poem, a table. An optional `<figcaption>` gives it a caption.

A figure is not only for images. The key idea is that the content is a unit that the text refers to ('see Figure 3'). Purely decorative images are not figures.

Syntax

```
<figure>
  content
  <figcaption>Caption</figcaption>
</figure>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning root, palpable content
Permitted content	A <code><figcaption></code> followed by flow content, or flow content followed by a <code><figcaption></code> , or flow content alone
Permitted parents	Any element that accepts flow content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<figure>: worked examples

Photo

```
<figure>
  
  <figcaption>Feluccas on the Nile.</figcaption>
</figure>
```

Code listing

```
<figure>
  <pre><code>footer { text-align: center; }</code></pre>
  <figcaption>Centring the footer text.</figcaption>
</figure>
```

Multiple images in one figure

```
<figure>
  
  
  <figcaption>Before and after.</figcaption>
</figure>
```

Removing default margins

```
<style>
  figure { margin: 0; }
  figure img { max-width: 100%; height: auto; }
</style>
```

Default browser CSS

```
figure { display: block; margin: 1em 40px; }
```

Accessibility

- Exposed with the 'figure' role; the figcaption labels it.

Common mistakes

- Wrapping every image in a figure regardless of meaning.
- Forgetting that browsers add a 40px left and right margin by default.

See also

<figcaption>, , <picture>, <table>

Quick card:

Font (obsolete)

The element set the size, colour and typeface of its text. It was the main way to style text before CSS, was deprecated in HTML 4.01 and removed in HTML5. Browsers still render it for old pages. Every use of can be replaced by CSS: font-size, color and font-family.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	size, color, face
Related	<basefont>,

Minimal example

```
<!-- Old -->
<font face="Arial" size="5" color="#e60000">Sale!</font>

<!-- New -->
<span style="font-family: Arial, sans-serif; font-size: x-large;
color: #e60000">Sale!</span>
```

Most common mistake

Any use in new pages.

 — Font (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The `<font>` element set the size, colour and typeface of its text. It was the main way to style text before CSS, was deprecated in HTML 4.01 and removed in HTML5. Browsers still render it for old pages. Every use of `<font>` can be replaced by CSS: `font-size`, `color` and `font-family`.

The size attribute used a scale from 1 to 7 where 3 was the default; relative values such as +1 or -2 were also allowed.

Syntax

```
<font size="4" color="red" face="Arial">text</font>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Phrasing content (historical)
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLFontElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
size	1-7 or +n / -n	Font size on the HTML scale: 1 = x-small, 2 = small, 3 = medium (default), 4 = large, 5 = x-large, 6 = xx-large, 7 = xxx-large.	HTML 3.2
color	colour name or #rrggbb	Text colour.	HTML 3.2
face	comma-separated font names	Typeface, tried in order.	HTML 3.2

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

size

Values: 1-7 or +n / -n | Since: HTML 3.2

Font size on the HTML scale: 1 = x-small, 2 = small, 3 = medium (default), 4 = large, 5 = x-large, 6 = xx-large, 7 = xxx-large.

Write size="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<font size="1-7">...</font>
```

color

Values: colour name or #rrggbb | Since: HTML 3.2

Text colour.

Write color="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<font color="#336699">...</font>
```

face

Values: comma-separated font names | Since: HTML 3.2

Typeface, tried in order.

Write face="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<font face="value">...</font>
```

: worked examples

Replacement

```
<!-- Old -->
<font face="Arial" size="5" color="#e60000">Sale!</font>
```

```
<!-- New -->
<span style="font-family: Arial, sans-serif; font-size: x-large;
color: #e60000">Sale!</span>
```

Accessibility

- Fixed font sizes set with ignore user preferences.

Common mistakes

- Any use in new pages.

See also

<basefont>,

Quick card: <footer>

Footer

The <footer> element represents the footer of its nearest ancestor sectioning content or sectioning root: the page (<body>), an <article>, a <section>, an <aside> or a <nav>. A footer typically contains information about the section it belongs to: who wrote it, copyright data, links to related documents, contact details, a site map, or navigation back to the top.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<header>, <address>, <nav>, <article>, <section>, <body>

Minimal example

```
<body>
  <header><h1>My Blog</h1></header>
  <main>
    <p>Today I learned <em>a lot</em> about HTML.</p>
  </main>
  <footer>
    <p>&copy; 2026 My Blog. All rights reserved.</p>
    <nav>
      <a href="about.html">About</a> |
      <a href="privacy.html">Privacy</a> |
      <a href="#top">Back to top</a>
    </nav>
```

```
<address>Contact: <a href="mailto:me@example.com">me@example.com</a></address>
</footer>
</body>
```

Most common mistake

Nesting a `<footer>` inside another `<footer>` or a `<header>`.

<footer> — Footer

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The <footer> element represents the footer of its nearest ancestor sectioning content or sectioning root: the page (<body>), an <article>, a <section>, an <aside> or a <nav>. A footer typically contains information about the section it belongs to: who wrote it, copyright data, links to related documents, contact details, a site map, or navigation back to the top.

The most common use is the page footer at the bottom of <body>, but you can have as many footers as you have sections. A footer inside an <article> gives the article's metadata (author, date, tags); a footer inside a <blockquote> gives the source of the quotation.

Visually, a <footer> is nothing more than a block like a <div>. Its value is semantic: it tells browsers, assistive technology and search engines what the content is. When a footer is a direct child of <body> it is exposed as a 'contentinfo' landmark that screen reader users can jump to. Contact information inside a footer should be wrapped in <address>. A footer may not contain another <footer> or a <header>.

Syntax

```
<footer>...</footer>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, palpable content
Permitted content	Flow content, but no <footer> or <header> descendants
Permitted parents	Any element that accepts flow content, but not <address>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<footer>: worked examples

Page footer

```
<body>
  <header><h1>My Blog</h1></header>
  <main>
    <p>Today I learned <em>a lot</em> about HTML.</p>
  </main>
  <footer>
    <p>&copy; 2026 My Blog. All rights reserved.</p>
    <nav>
      <a href="about.html">About</a> |
      <a href="privacy.html">Privacy</a> |
      <a href="#top">Back to top</a>
    </nav>
    <address>Contact: <a href="mailto:me@example.com">me@example.com</a></address>
  </footer>
</body>
```

Styling a footer with CSS

```
<style>
  footer {
    background-color: #222;
    color: #fff;
    text-align: center;
    padding: 20px;
    font-family: Arial, sans-serif;
  }
  footer a { color: #ffd166; }
</style>
```

font-family sets the letter style for all text in the footer; text-align centres it.

Article footer with metadata

```
<article>
  <h2>Learning the footer element</h2>
  <p>...</p>
  <footer>
    <p>Posted by Ahmed on <time datetime="2026-09-11">11 September 2026</time></p>
    <p>Tags: html, semantics</p>
  </footer>
</article>
```

Multi-column site footer

```
<style>
  .cols { display: flex; flex-wrap: wrap; gap: 32px; justify-content:
center; }
</style>
<footer>
  <div class="cols">
    <section>
      <h3>Shop</h3>
      <ul><li><a href="/books">Books</a></li><li><a href="/
deals">Deals</a></li></ul>
    </section>
    <section>
      <h3>Help</h3>
      <ul><li><a href="/faq">FAQ</a></li><li><a href="/
returns">Returns</a></li></ul>
    </section>
  </div>
  <p>© 2026 Book Store</p>
</footer>
```

Sticky footer at the bottom of short pages

```
<style>
  html, body { height: 100%; margin: 0; }
  body { display: flex; flex-direction: column; }
  main { flex: 1; }
</style>
<body>
  <main>Short content</main>
  <footer>Always at the bottom</footer>
</body>
```

Default browser CSS

```
footer { display: block; }
```

Accessibility

- A body-level footer is announced as 'content information' landmark.
- Only one body-level footer should exist; footers inside articles and sections are not landmarks.
- Keep 'back to top' links inside the footer so keyboard users can find them.

Common mistakes

- Nesting a `<footer>` inside another `<footer>` or a `<header>`.
- Using `<footer>` for a sidebar or unrelated content.
- Using `<div class="footer">` when `<footer>` is available.
- Putting the main navigation only in the footer.

See also

`<header>`, `<address>`, `<nav>`, `<article>`, `<section>`,
`<body>`

Quick card: <form>

Form

The <form> element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the action attribute using the HTTP method in the method attribute.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	action, method, enctype, name, target, autocomplete, novalidate, accept-charset, rel, accept
Related	<input>, <button>, <label>, <select>, <textarea>, <fieldset>, <output>

Minimal example

```
<form action="/contact" method="post">
  <p>
    <label for="name">Name</label>
    <input id="name" name="name" required>
  </p>
  <p>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>
  </p>
```

```
<p>
  <label for="msg">Message</label>
  <textarea id="msg" name="message" rows="5" required></textarea>
</p>
<button type="submit">Send</button>
</form>
```

Most common mistake

Nesting forms.

<form> — Form

Category: Forms | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <form> element represents a section of the document containing interactive controls (text boxes, check boxes, radio buttons, drop-down lists, buttons and so on) for collecting information from the user and sending it to a server. When the form is submitted, the browser gathers the name and value of every control that has a name, encodes them, and sends them to the URL in the action attribute using the HTTP method in the method attribute.

With method="get" the data is appended to the URL as a query string (?name=value&...); this is suitable for searches and filters that can be bookmarked. With method="post" the data is sent in the request body; use it for anything that changes data on the server, sends passwords or uploads files.

HTML5 added built-in validation: attributes such as required, pattern, min, max and type="email" are checked before submission and the browser shows an error message. The novalidate attribute turns this off. Forms may not be nested inside other forms.

Syntax

```
<form action="URL" method="post">
  ...controls...
  <button>Submit</button>
</form>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, palpable content
Permitted content	Flow content, but no <form> descendants
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLFormElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
action	URL	Where to send the data. If omitted, the data is sent to the current page URL.	HTML 2.0
method	get post dialog	The HTTP method. get puts data in the URL; post sends it in the body; dialog closes the enclosing <dialog>.	HTML 2.0
enctype	application/x-www-form-urlencoded multipart/form-data text/plain	How the data is encoded for post. Use multipart/form-data when the form contains <input type="file">.	HTML 2.0
name	text	A name for the form, used by scripts (document.forms.name). Must be unique among forms.	HTML 2.0

Attribute	Values	Description	Since
target	_self _blank _parent _top name	Where to display the response.	HTML 4.01
autocomplete	on off	Whether the browser may autofill controls in this form. Default on.	HTML5
novalidate	boolean	Do not validate the form on submission.	HTML5
accept-charset	space-separated encodings	The character encodings the server accepts. Use utf-8.	HTML 4.01
rel	link types	Relationship of the target resource, for example noopener or noreferrer.	Living Standard
accept	MIME types	Obsolete. Use accept on <input type="file"> instead.	HTML 4.01

<form>: attribute by attribute

Each attribute of <form> explained in detail, with a minimal example you can copy.

action

Values: URL | Since: HTML 2.0

Where to send the data. If omitted, the data is sent to the current page URL.

Write `action="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form action="https://example.com/page.html">...</form>
```

method

Values: get | post | dialog | Since: HTML 2.0

The HTTP method. `get` puts data in the URL; `post` sends it in the body; `dialog` closes the enclosing <dialog>.

Write `method="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form method="get">...</form>
```

enctype

Values: application/x-www-form-urlencoded | multipart/form-data | text/plain | Since: HTML 2.0

How the data is encoded for post. Use `multipart/form-data` when the form contains `<input type="file">`.

Write `enctype="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form enctype="application">...</form>
```

name

Values: text | Since: HTML 2.0

A name for the form, used by scripts (document.forms.name).
Must be unique among forms.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form name="value">...</form>
```

target

Values: _self | _blank | _parent | _top | name | Since: HTML 4.01

Where to display the response.

Write target="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form target="_self">...</form>
```

autocomplete

Values: on | off | Since: HTML5

Whether the browser may autofill controls in this form.
Default on.

Write autocomplete="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form autocomplete="on">...</form>
```

novalidate

Values: boolean | Since: HTML5

Do not validate the form on submission.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<form novalidate>...</form>
```

accept-charset

Values: space-separated encodings | Since: HTML 4.01

The character encodings the server accepts. Use utf-8.

Write `accept-charset="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form accept-charset="value">...</form>
```

rel

Values: link types | Since: Living Standard

Relationship of the target resource, for example `noopener` or `noreferrer`.

Write `rel="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form rel="link types">...</form>
```

accept

Values: MIME types | Since: HTML 4.01

Obsolete. Use `accept` on `<input type="file">` instead.

Write `accept="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<form accept="text/plain">...</form>
```

<form>: worked examples

Contact form

```
<form action="/contact" method="post">
  <p>
    <label for="name">Name</label>
    <input id="name" name="name" required>
  </p>
  <p>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>
  </p>
  <p>
    <label for="msg">Message</label>
    <textarea id="msg" name="message" rows="5" required></textarea>
  </p>
  <button type="submit">Send</button>
</form>
```

Search form using GET

```
<form action="/search" method="get" role="search">
  <label for="q">Search</label>
  <input id="q" name="q" type="search">
  <button>Go</button>
</form>
```

Submitting with 'html' typed produces the URL `/search?q=html`.

File upload

```
<form action="/upload" method="post" enctype="multipart/form-data">
  <input type="file" name="photo" accept="image/*">
  <button>Upload</button>
</form>
```

Handling submission with JavaScript

```
<form id="f">
  <input name="city">
  <button>Save</button>
</form>
<script>
  document.getElementById('f').addEventListener('submit', e => {
    e.preventDefault();
    const data = new FormData(e.target);
    console.log(data.get('city'));
  });
</script>
```

Default browser CSS

```
form { display: block; margin-top: 0; }
```

Accessibility

- Every control needs a `<label>`.
- Group related controls with `<fieldset>` and `<legend>`.
- Make error messages visible and associate them with the field using `aria-describedby`.
- Do not rely on placeholder text as the only label.

Common mistakes

- Nesting forms.
- Forgetting name attributes, so nothing is submitted.
- Forgetting `enctype` for file uploads.
- Using GET for passwords or large data.
- Having no submit button and relying on Enter alone.

See also

`<input>`, `<button>`, `<label>`, `<select>`, `<textarea>`,
`<fieldset>`, `<output>`

Quick card: <frame>

Frame (obsolete)

The <frame> element defined one rectangular sub-window inside a <frameset>, each displaying a separate HTML document. Frames were used in the late 1990s to keep a navigation menu fixed while the content scrolled. They caused many problems: bookmarks and the back button did not work as expected, search engines could not index the pages properly, printing was unreliable and screen readers struggled. HTML5 removed <frame> and <frameset>. Use CSS for layout and <iframe> when you truly need to embed another document.

At a glance

Status	OBSOLETE
Versions	Netscape 2 (1996), HTML 4.01 Frameset; obsolete in HTML5
Category	Obsolete frames
Closing tag	None (void element)
Display	block
Own attributes	src, name, frameborder, marginwidth, noresize, scrolling, longdesc
Related	<frameset>, <noframes>, <iframe>

Minimal example

```
<style>
  body { display: grid; grid-template-columns: 200px 1fr; height:
100vh; margin: 0; }
  nav { position: sticky; top: 0; }
</style>
<body>
```

```
<nav>Menu</nav>  
<main>Content</main>  
</body>
```

Most common mistake

Any use.

<frame> — Frame (obsolete)

Category: Obsolete frames | Status: OBSOLETE | Versions: Netscape 2 (1996), HTML 4.01 Frameset; obsolete in HTML5

Description

The <frame> element defined one rectangular sub-window inside a <frameset>, each displaying a separate HTML document. Frames were used in the late 1990s to keep a navigation menu fixed while the content scrolled. They caused many problems: bookmarks and the back button did not work as expected, search engines could not index the pages properly, printing was unreliable and screen readers struggled. HTML5 removed <frame> and <frameset>. Use CSS for layout and <iframe> when you truly need to embed another document.

Syntax

```
<frameset cols="200,*">
  <frame src="menu.html" name="menu">
  <frame src="main.html" name="main">
</frameset>
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	block
Content categories	Historical
Permitted content	None
Permitted parents	<frameset>

Property	Value
Tag omission	No end tag
DOM interface	HTMLFrameElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The document to display.	HTML 4.01
name	text	Name used as a link target.	HTML 4.01
frameborder	0 1	Whether to draw a border.	HTML 4.01
marginwidth / margin height	pixels	Margins inside the frame.	HTML 4.01
noresize	boolean	User cannot resize the frame.	HTML 4.01
scrolling	yes no auto	Scrollbar behaviour.	HTML 4.01
longdesc	URL	Link to a long description.	HTML 4.01

<frame>: attribute by attribute

Each attribute of <frame> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML 4.01

The document to display.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame src="https://example.com/page.html">
```

name

Values: text | Since: HTML 4.01

Name used as a link target.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame name="value">
```

frameborder

Values: 0 | 1 | Since: HTML 4.01

Whether to draw a border.

Write frameborder="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame frameborder="0">
```

marginwidth / marginheight

Values: pixels | Since: HTML 4.01

Margins inside the frame.

Write marginwidth="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame marginwidth="3">
```

noresize

Values: boolean | Since: HTML 4.01

User cannot resize the frame.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<frame noresize>
```

scrolling

Values: yes | no | auto | Since: HTML 4.01

Scrollbar behaviour.

Write scrolling="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame scrolling="yes">
```

longdesc

Values: URL | Since: HTML 4.01

Link to a long description.

Write longdesc="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frame longdesc="https://example.com/page.html">
```

<frame>: worked examples

Modern replacement

```
<style>
  body { display: grid; grid-template-columns: 200px 1fr; height:
100vh; margin: 0; }
  nav { position: sticky; top: 0; }
</style>
<body>
  <nav>Menu</nav>
  <main>Content</main>
</body>
```

Accessibility

- Frames were a major accessibility barrier.

Common mistakes

- Any use.

See also

<frameset>, <noframes>, <iframe>

Quick card: <frameset>

Frameset (obsolete)

The <frameset> element replaced <body> in a frames document and divided the window into rows and columns, each filled by a <frame> or a nested <frameset>. It required the HTML 4.01 Frameset DOCTYPE. It is obsolete in HTML5. See <frame> for the reasons and the modern replacement.

At a glance

Status	OBSOLETE
Versions	Netscape 2, HTML 4.01 Frameset; obsolete in HTML5
Category	Obsolete frames
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	cols, rows, border, onload
Related	<frame>, <noframes>, <iframe>

Minimal example

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN">
<html>
<head><title>Frames</title></head>
<frameset cols="25%,75%">
  <frame src="nav.html">
  <frame src="content.html">
  <noframes><body>Your browser does not support frames.</body></
noframes>
</frameset>
</html>
```

Most common mistake

Any use.

<frameset> — Frameset (obsolete)

Category: Obsolete frames | Status: OBSOLETE | Versions: Netscape 2, HTML 4.01 Frameset; obsolete in HTML5

Description

The <frameset> element replaced <body> in a frames document and divided the window into rows and columns, each filled by a <frame> or a nested <frameset>. It required the HTML 4.01 Frameset DOCTYPE. It is obsolete in HTML5. See <frame> for the reasons and the modern replacement.

Syntax

```
<frameset rows="80,*">...</frameset>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Historical
Permitted content	<frame>, <frameset>, <noframes>
Permitted parents	<html>, instead of <body>
Tag omission	Neither
DOM interface	HTMLFrameSetElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
cols	comma-separated sizes	Column widths in pixels, percentages or * (remaining space).	HTML 4.01
rows	comma-separated sizes	Row heights.	HTML 4.01
border, frameborder, framespacing	pixels / 0 1	Border appearance.	HTML 4.01 / non-standard
onload, onunload, and other window events	script	Window event handlers, as on <body>.	HTML 4.01

<frameset>: attribute by attribute

Each attribute of <frameset> explained in detail, with a minimal example you can copy.

cols

Values: comma-separated sizes | Since: HTML 4.01

Column widths in pixels, percentages or * (remaining space).

Write cols="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frameset cols="comma-separated sizes">...</frameset>
```

rows

Values: comma-separated sizes | Since: HTML 4.01

Row heights.

Write rows="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frameset rows="comma-separated sizes">...</frameset>
```

border, frameborder, framespacing

Values: pixels / 0 | 1 | Since: HTML 4.01 / non-standard

Border appearance.

Write border="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frameset border="3">...</frameset>
```

onload, onunload, and other window events

Values: script | Since: HTML 4.01

Window event handlers, as on <body>.

Write onload="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<frameset onload="script">...</frameset>
```

<frameset>: worked examples

Historical example

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN">
<html>
<head><title>Frames</title></head>
<frameset cols="25%,75%">
  <frame src="nav.html">
  <frame src="content.html">
  <noframes><body>Your browser does not support frames.</body></
noframes>
</frameset>
</html>
```

Common mistakes

- Any use.

See also

<frame>, <noframes>, <iframe>

Quick card: <h1>

Headings h1 to h6

The <h1> to <h6> elements represent six levels of section headings. <h1> is the highest (most important) level and <h6> the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	align
Related	<hgroup>, <section>, <article>, <header>, <p>

Minimal example

```
<h1>Cooking basics</h1>
  <h2>Equipment</h2>
    <h3>Knives</h3>
    <h3>Pans</h3>
  <h2>Techniques</h2>
    <h3>Boiling</h3>
    <h3>Frying</h3>
  <h4>Deep frying</h4>
```

Most common mistake

Choosing the level by size.

<h1> — Headings h1 to h6

Category: Content sectioning | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

This entry also covers: <h2>, <h3>, <h4>, <h5>, <h6>

Description

The <h1> to <h6> elements represent six levels of section headings. <h1> is the highest (most important) level and <h6> the lowest. Headings describe the topic of the section that follows them, and together they form the outline of the document, like the table of contents of a book.

Use heading levels to express structure, not to choose a font size. A page should normally have a single <h1> describing the whole page, <h2> for its major sections, <h3> for sub-sections inside those, and so on. Do not skip levels (for example jumping from <h2> to <h4>) because it confuses the outline. If a heading looks too big or too small, change it with CSS, not by choosing a different level.

Headings are one of the most important elements for accessibility: screen reader users navigate pages by jumping from heading to heading, and search engines give heading text extra weight.

Syntax

```
<h1>Main title</h1>
<h2>Section</h2>
<h3>Sub-section</h3>
```

Technical summary

Property	Value
Void element	No

Property	Value
Default display	block
Content categories	Flow content, heading content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts flow content; also directly inside <hgroup>
Tag omission	Neither tag may be omitted
DOM interface	HTMLHeadingElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right justify	Obsolete. Use CSS text-align.	HTML 3.2

<h1>: attribute by attribute

Each attribute of <h1> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | justify | Since: HTML 3.2

Obsolete. Use CSS text-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<h1 align="left">...</h1>
```

<h1>: worked examples

A correct outline

```
<h1>Cooking basics</h1>
  <h2>Equipment</h2>
    <h3>Knives</h3>
    <h3>Pans</h3>
  <h2>Techniques</h2>
    <h3>Boiling</h3>
    <h3>Frying</h3>
    <h4>Deep frying</h4>
```

Indentation is only for illustration; headings are written at the same indentation as their content.

Default sizes

```
<h1>Heading 1 (2em)</h1>
<h2>Heading 2 (1.5em)</h2>
<h3>Heading 3 (1.17em)</h3>
<h4>Heading 4 (1em)</h4>
<h5>Heading 5 (0.83em)</h5>
<h6>Heading 6 (0.67em)</h6>
```

Changing the look without changing the level

```
<style>
  h2 { font-size: 1.2rem; text-transform: uppercase; letter-spacing:
0.1em; color: #555; }
</style>
<h2>Small-looking but still a level-2 heading</h2>
```

Heading with a subtitle

```
<hgroup>
  <h1>HTML Reference</h1>
  <p>Every tag from 1991 to today</p>
</hgroup>
```

Default browser CSS

```
h1 { display: block; font-size: 2em; margin-block: 0.67em; font-
weight: bold; }
```

Accessibility

- Screen reader users can list all headings and jump between them; a good heading structure is the single biggest navigation aid.

- Do not use headings for text that is merely large or bold.
- Do not leave headings empty.
- Each page should have one `<h1>`.

Common mistakes

- Choosing the level by size.
- Skipping levels.
- Several `<h1>` elements on a page (valid but unhelpful).
- Using `<b>` or `<font size>` instead of a heading.
- Placing a heading inside `<p>` or `<a>` incorrectly (a link may go inside a heading, not the reverse).

See also

`<hgroup>`, `<section>`, `<article>`, `<header>`, `<p>`

Quick card: <head>

Document head

The <head> element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and <meta> tags for search engines and social networks. Nothing inside <head> is displayed on the page itself, except the title, which appears in the browser tab.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Document metadata
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	profile
Related	<html>, <title>, <meta>, <link>, <style>, <script>, <base>

Minimal example

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Book Store - Home</title>
  <meta name="description" content="Buy PDF books and pay with mobile
wallet.">
  <link rel="icon" href="favicon.ico">
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
```

Most common mistake

Putting visible content (<p>, <div>,) in the head; the browser moves it to the body.

<head> — Document head

Category: Document metadata | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <head> element contains machine-readable information (metadata) about the document: its title, character encoding, links to style sheets and icons, scripts, and <meta> tags for search engines and social networks. Nothing inside <head> is displayed on the page itself, except the title, which appears in the browser tab.

<head> is the first child of <html>. Every document must have exactly one <title> inside its head (unless the title is supplied by a higher-level protocol, such as an email subject). The tags may be omitted and the browser will create the element automatically, but always write them.

Syntax

```
<head>
  <meta charset="utf-8">
  <title>Title</title>
</head>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	None
Permitted content	Metadata content: exactly one <title>, at most one <base>, any number of <link>, <meta>, <script>, <style>, <noscript>, <template>

Property	Value
Permitted parents	<html>, as its first child
Tag omission	Start tag may be omitted if the first thing inside is an element; end tag may be omitted if not followed by whitespace or a comment
DOM interface	HTMLHeadElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
profile	URLs	Obsolete. Meta data profiles.	HTML 4.01

<head>: attribute by attribute

Each attribute of <head> explained in detail, with a minimal example you can copy.

profile

Values: URLs | Since: HTML 4.01

Obsolete. Meta data profiles.

Write profile="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<head profile="https://example.com/page.html">...</head>
```

<head>: worked examples

A complete, well-formed head

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Book Store - Home</title>
  <meta name="description" content="Buy PDF books and pay with mobile
wallet.">
  <link rel="icon" href="favicon.ico">
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
```

Order matters a little: charset first, then viewport, then title, then styles, then scripts.

Default browser CSS

```
head { display: none; }
```

Accessibility

- The <title> is the first thing a screen reader announces; make it descriptive and unique per page.

Common mistakes

- Putting visible content (<p>, <div>,) in the head; the browser moves it to the body.
- Omitting <title>.
- Placing <meta charset> after 1024 bytes of content.

See also

<html>, <title>, <meta>, <link>, <style>, <script>,
<base>

Quick card: <header>

Header

The <header> element represents introductory content for its nearest sectioning ancestor or for the page: typically a group of headings, a logo, a search form, the author's name, and the main navigation. A page usually has one header at the top of <body>, but articles and sections may have their own headers too.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<footer>, <nav>, <h1>, <hgroup>, <main>

Minimal example

```
<header>
  
  <nav>
    <a href="/">Home</a>
    <a href="/books">Books</a>
    <a href="/contact">Contact</a>
  </nav>
  <form role="search" action="/search"><input name="q" type="search"
aria-label="Search"></form>
</header>
```

Most common mistake

Nesting header inside header or footer.

<header> — Header

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The `<header>` element represents introductory content for its nearest sectioning ancestor or for the page: typically a group of headings, a logo, a search form, the author's name, and the main navigation. A page usually has one header at the top of `<body>`, but articles and sections may have their own headers too.

When a `<header>` is a direct child of `<body>` (not inside `<article>`, `<aside>`, `<main>`, `<nav>` or `<section>`) it is exposed as the 'banner' landmark. A header may not contain another `<header>` or a `<footer>`.

Syntax

```
<header>...</header>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, palpable content
Permitted content	Flow content, but no <code><header></code> or <code><footer></code> descendants
Permitted parents	Any element that accepts flow content, but not <code><address></code>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<header>: worked examples

Site header

```
<header>
  
  <nav>
    <a href="/">Home</a>
    <a href="/books">Books</a>
    <a href="/contact">Contact</a>
  </nav>
  <form role="search" action="/search"><input name="q" type="search"
aria-label="Search"></form>
</header>
```

Article header

```
<article>
  <header>
    <h2>Ten HTML tips</h2>
    <p>By Sara, <time datetime="2026-09-01">1 September 2026</time></p>
  </header>
  <p>...</p>
</article>
```

Sticky header

```
<style>
  header { position: sticky; top: 0; background: white; box-shadow: 0
2px 4px rgba(0,0,0,.1); }
</style>
```

Default browser CSS

```
header { display: block; }
```

Accessibility

- A body-level header is the 'banner' landmark; there should be only one.
- Put the site's main <nav> inside the header so it is found quickly.

Common mistakes

- Nesting header inside header or footer.

- Using `<header>` for the heading text alone; a lone `<h1>` does not need a header wrapper.

See also

`<footer>`, `<nav>`, `<h1>`, `<hgroup>`, `<main>`

Quick card: <hgroup>

Heading group

The <hgroup> element groups a heading (<h1>-<h6>) with one or more <p> elements that contain a subtitle, alternative title, tagline or other secondary text. It tells the browser that the paragraphs belong to the heading rather than being ordinary content.

At a glance

Status	CURRENT
Versions	HTML5 (removed from W3C HTML 5.0, kept in the Living Standard with a new definition)
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<h1>, <header>, <p>

Minimal example

```
<hgroup>
  <h1>The Complete HTML Reference</h1>
  <p>From HTML 1 to the Living Standard</p>
</hgroup>
```

Most common mistake

Placing two headings inside one hgroup (old HTML5 style).

<hgroup> — Heading group

Category: Content sectioning | Status: CURRENT | Versions: HTML5 (removed from W3C HTML 5.0, kept in the Living Standard with a new definition)

Description

The `<hgroup>` element groups a heading (`<h1>`-`<h6>`) with one or more `<p>` elements that contain a subtitle, alternative title, tagline or other secondary text. It tells the browser that the paragraphs belong to the heading rather than being ordinary content.

Its history is unusual: in early HTML5 drafts it grouped several headings of different levels and only the highest counted in the outline. The W3C dropped it in 2013, but the WHATWG kept it and in 2022 redefined it as described above. It must contain exactly one heading element.

Syntax

```
<hgroup>
  <h1>Title</h1>
  <p>Subtitle</p>
</hgroup>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, heading content, palpable content
Permitted content	Zero or more <code><p></code> , then one <code><h1></code> - <code><h6></code> , then zero or more <code><p></code> (plus <code><script></code> and <code><template></code>)
Permitted parents	Any element that accepts flow content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<hgroup>: worked examples

Title with tagline

```
<hgroup>
  <h1>The Complete HTML Reference</h1>
  <p>From HTML 1 to the Living Standard</p>
</hgroup>
```

Kicker above the headline

```
<hgroup>
  <p>Breaking news</p>
  <h2>New HTML element announced</h2>
</hgroup>
```

Default browser CSS

```
hgroup { display: block; }
```

Accessibility

- Exposed with role 'group'; the subtitle paragraphs are not read as headings, which is the desired behaviour.

Common mistakes

- Placing two headings inside one hgroup (old HTML5 style).
- Using a lower-level heading such as <h2> as the subtitle.

See also

<h1>, <header>, <p>

Quick card: <hr>

Thematic break (horizontal rule)

The <hr> element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	None (void element)
Display	block
Own attributes	align, color, noshade, size, width
Related	<p>, , <section>

Minimal example

```
<p>The first chapter ends here.</p>
<hr>
<p>The story continues years later.</p>
```

Most common mistake

Using <hr> as a layout spacer.

<hr> — Thematic break (horizontal rule)

Category: Text content | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<hr>` element represents a thematic break between paragraph-level content: a change of scene in a story, a shift of topic within a section, or a separator between groups of options in a menu. Browsers render it as a horizontal line, which is where its name (horizontal rule) comes from.

In HTML 4 it was purely a visual line with attributes for size, width and colour. HTML5 kept the element but gave it the semantic meaning above and made the presentational attributes obsolete. If you only want a decorative line, use a CSS border instead of `<hr>`. It is a void element.

Syntax

`<hr>`

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	block
Content categories	Flow content
Permitted content	None (void element)
Permitted parents	Any element that accepts flow content, and <code><select></code>

Property	Value
Tag omission	No end tag
DOM interface	HTMLHRElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right	Obsolete. Alignment of the rule. Use CSS margin.	HTML 3.2
color	colour	Obsolete, non-standard. Colour of the rule. Use CSS border-color.	IE
noshade	boolean	Obsolete. Draw a solid line instead of a 3D groove. Use CSS.	HTML 3.2
size	pixels	Obsolete. Height of the rule. Use CSS height or border-width.	HTML 3.2
width	pixels or percentage	Obsolete. Width of the rule. Use CSS width.	HTML 3.2

<hr>: attribute by attribute

Each attribute of <hr> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | Since: HTML 3.2

Obsolete. Alignment of the rule. Use CSS margin.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<hr align="left">
```

color

Values: colour | Since: IE

Obsolete, non-standard. Colour of the rule. Use CSS border-color.

Write color="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<hr color="#336699">
```

noshade

Values: boolean | Since: HTML 3.2

Obsolete. Draw a solid line instead of a 3D groove. Use CSS.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<hr noshade>
```

size

Values: pixels | Since: HTML 3.2

Obsolete. Height of the rule. Use CSS height or border-width.

Write size="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<hr size="3">
```

width

Values: pixels or percentage | Since: HTML 3.2

Obsolete. Width of the rule. Use CSS width.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<hr width="3">
```

<hr>: worked examples

Topic change

```
<p>The first chapter ends here.</p>
<hr>
<p>The story continues years later.</p>
```

Styled rule

```
<style>
  hr { border: none; border-top: 2px dashed #c8a24a; margin: 2em auto;
    width: 60%; }
</style>
<hr>
```

Separator inside a select list

```
<select>
  <option>Egypt</option>
  <option>Saudi Arabia</option>
  <hr>
  <option>Other</option>
</select>
```

Newer browsers draw a separator line between groups of options.

Default browser CSS

```
hr { display: block; margin: 0.5em auto; border: 1px inset; color:
gray; overflow: hidden; }
```

Accessibility

- Announced as 'separator' by screen readers. If the line is only decorative, use a CSS border so it is not announced.

Common mistakes

- Using <hr> as a layout spacer.
- Writing </hr>.

See also

<p>,
, <section>

Quick card: <html>

Document root

The <html> element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: <head> followed by <body>. It comes immediately after the <!DOCTYPE html> declaration.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Main root
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	lang, dir, xmlns, manifest, version
Related	<head>, <body>

Minimal example

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Hello</title></head>
<body><p>Hello</p></body>
</html>
```

Most common mistake

Omitting lang.

<html> — Document root

Category: Main root | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<html>` element is the root element of an HTML document: every other element is a descendant of it. It has exactly two children: `<head>` followed by `<body>`. It comes immediately after the `<!DOCTYPE html>` declaration.

The most important attribute on `<html>` is `lang`, which declares the language of the page. It affects hyphenation, spell-checking, font selection, quotation marks, the voice used by screen readers and the behaviour of translation tools. For right-to-left languages add `dir="rtl"`. Both tags may technically be omitted, but always write them.

Syntax

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body>...</body>
</html>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	One <code><head></code> followed by one <code><body></code>

Property	Value
Permitted parents	None (root element)
Tag omission	Start tag may be omitted if the first thing inside is not a comment; end tag may be omitted if not followed by a comment
DOM interface	HTMLHtmlElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
lang	BCP 47 language tag	The primary language of the document: en, en-GB, ar, ar-EG, fr, de, zh-Hans...	HTML 4.01
dir	ltr rtl auto	Text direction of the document.	HTML 4.01
xmlns	http://www.w3.org/1999/xhtml	XML namespace; required in XHTML, ignored in HTML.	XHTML 1.0
manifest	URL	Obsolete. Application cache manifest for offline pages. Replaced by service workers.	HTML5 (removed)
version	text	Obsolete. The HTML DTD version.	HTML 2.0

<html>: attribute by attribute

Each attribute of <html> explained in detail, with a minimal example you can copy.

lang

Values: BCP 47 language tag | Since: HTML 4.01

The primary language of the document: en, en-GB, ar, ar-EG, fr, de, zh-Hans...

Write lang="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<html lang="en">...</html>
```

dir

Values: ltr | rtl | auto | Since: HTML 4.01

Text direction of the document.

Write dir="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<html dir="ltr">...</html>
```

xmlns

Values: <http://www.w3.org/1999/xhtml> | Since: XHTML 1.0

XML namespace; required in XHTML, ignored in HTML.

Write xmlns="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<html xmlns="http:">...</html>
```

manifest

Values: URL | Since: HTML5 (removed)

Obsolete. Application cache manifest for offline pages.

Replaced by service workers.

Write manifest="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<html manifest="https://example.com/page.html">...</html>
```

version

Values: text | Since: HTML 2.0

Obsolete. The HTML DTD version.

Write version="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<html version="value">...</html>
```

<html>: worked examples

English page

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Hello</title></head>
<body><p>Hello</p></body>
</html>
```

Arabic page, right to left

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head><meta charset="utf-8"><title>أبجـد</title></head>
<body><p>أبـجـد</p></body>
</html>
```

Old DOCTYPEs for comparison

```
<!-- HTML 2.0 -->
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">

<!-- HTML 3.2 -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">

<!-- HTML 4.01 Strict -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">

<!-- XHTML 1.0 Strict -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<!-- HTML5 -->
<!DOCTYPE html>
```

Default browser CSS

```
html { display: block; }
```

Accessibility

- Always set lang. Without it screen readers may read the page in the wrong language with a heavy accent.

Common mistakes

- Omitting lang.

- Placing content outside `<html>`.
- Forgetting the DOCTYPE before it.

See also

`<head>`, `<body>`

Quick card: `<i>`

Idiomatic text (italic)

The `<i>` element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code></code> , <code></code> , <code><cite></code> , <code><dfn></code>

Minimal example

```
<p>The Egyptians say <i lang="ar">inshallah</i> often.</p>
<p>The domestic cat is <i>Felis catus</i>.</p>
```

Most common mistake

Using `<i>` for icons with no text (`<i class="fa fa-home"></i>`) is common in icon libraries but semantically wrong; add `aria-hidden` and a text label.

<i> — Idiomatic text (italic)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<i>` element marks a span of text that is set off from the normal prose for some reason other than emphasis or importance: an alternative voice or mood, a technical term, a taxonomic name (*Homo sapiens*), a phrase in another language, a thought, a ship's name, or a transliteration. Browsers render it in italics.

In HTML 4 `<i>` simply meant 'italic'. HTML5 gave it the meaning above so that it stays valid. Choose the most specific element first: `<em>` for emphasis, `<cite>` for titles of works, `<dfn>` for defined terms, `<strong>` for importance. Use `<i>` only when none of those apply, and consider adding a `class` or `lang` attribute to say why the text is different.

Syntax

```
<i>text</i>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<i>: worked examples

Foreign phrase and scientific name

```
<p>The Egyptians say <i lang="ar">inshallah</i> often.</p>
<p>The domestic cat is <i>Felis catus</i>.</p>
```

A thought

```
<p><i>I should have used a footer</i>, she thought.</p>
```

Default browser CSS

```
i { font-style: italic; }
```

Accessibility

- No announcement in screen readers; use `<em>` when the italic carries stress.

Common mistakes

- Using `<i>` for icons with no text (`<i class="fa fa-home"></i>`) is common in icon libraries but semantically wrong; add `aria-hidden` and a text label.
- Using `<i>` instead of `<em>` or `<cite>`.

See also

`<em>`, `<b>`, `<cite>`, `<dfn>`

Quick card: <iframe>

Inline frame

The <iframe> element embeds another HTML page inside the current page, in a rectangular area called a nested browsing context. Each iframe has its own document, its own window object and its own history. Common uses are embedding videos from YouTube, maps, advertisements, payment forms, social media widgets and third-party tools.

At a glance

Status	CURRENT
Versions	Internet Explorer 3 (1996), HTML 4.01, HTML5
Category	Embedded content
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	src, srcdoc, title, name, width, height, sandbox, allow, allowfullscreen, loading, referrerpolicy, credentialless, csp, align, frameborder, longdesc, marginheight, scrolling, allowpaymentrequest
Related	<embed>, <object>, <frame>, <fencedframe>, <portal>

Minimal example

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/qfPUMV9J5yw"
  title="HTML tutorial video"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media;
gyroscope; picture-in-picture"
  allowfullscreen loading="lazy"></iframe>
```

Most common mistake

No title.

<iframe> — Inline frame

Category: Embedded content | Status: CURRENT | Versions: Internet Explorer 3 (1996), HTML 4.01, HTML5

Description

The <iframe> element embeds another HTML page inside the current page, in a rectangular area called a nested browsing context. Each iframe has its own document, its own window object and its own history. Common uses are embedding videos from YouTube, maps, advertisements, payment forms, social media widgets and third-party tools.

Because the embedded page may come from another site, iframes have security controls. The sandbox attribute restricts what the embedded page may do (run scripts, submit forms, open popups, navigate the top page). The allow attribute grants specific features such as camera or fullscreen. Sites can refuse to be framed with the X-Frame-Options or Content-Security-Policy headers.

Every iframe costs memory and processing time. Use loading="lazy" for iframes far down the page, and always give the iframe a title describing its content.

Syntax

```
<iframe src="URL" title="description" width="600" height="400"></iframe>
```

Technical summary

Property	Value
Void element	No
Default display	inline

Property	Value
Content categories	Flow content, phrasing content, embedded content, interactive content, palpable content
Permitted content	None (the element must be empty; older browsers displayed content inside it as fallback)
Permitted parents	Any element that accepts embedded content
Tag omission	Neither tag may be omitted
DOM interface	HTMLIFrameElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The page to embed. about:blank shows an empty page.	HTML 4.01
srcdoc	HTML text	Inline HTML to display instead of loading src. Overrides src if present.	HTML5
title	text	A description of the embedded content for assistive technology. Strongly recommended.	HTML 4.01
name	text	A name that can be used as the target of links and forms.	HTML 4.01
width	pixels	Width of the frame (default 300).	HTML 4.01
height	pixels	Height of the frame (default 150).	HTML 4.01

Attribute	Values	Description	Since
sandbox	empty or space-separated tokens	Applies extra restrictions. Empty value = all restrictions. Tokens that lift restrictions: allow-downloads, allow-forms, allow-modals, allow-orientation-lock, allow-pointer-lock, allow-popups, allow-popups-to-escape-sandbox, allow-presentation, allow-same-origin, allow-scripts, allow-top-navigation, allow-top-navigation-by-user-activation, allow-top-navigation-to-custom-protocols.	HTML5
allow	permissions policy	Features the frame may use, for example allow="camera; microphone; fullscreen; geolocation; autoplay; picture-in-picture".	HTML5
allowfullscreen	boolean	Legacy form of allow="fullscreen".	HTML5
loading	eager lazy	lazy defers loading until the frame is near the viewport.	Living Standard
referrerpolicy	referrer policy keyword	Which referrer to send when loading the frame.	HTML5
credentials	boolean	Experimental: load the frame in a new, empty cookie/storage context.	Experimental
csp	policy string	Experimental: Content Security Policy to enforce on the embedded document.	Experimental

Attribute	Values	Description	Since
align	left right top middle bottom	Obsolete. Use CSS.	HTML 4.01
frameborder	0 1	Obsolete. Use CSS border.	HTML 4.01
longdesc	URL	Obsolete. Link to a long description.	HTML 4.01
marginheight / marginwidth	pixels	Obsolete. Use CSS in the framed document.	HTML 4.01
scrolling	yes no auto	Obsolete. Use CSS overflow in the framed document.	HTML 4.01
allowpaymentrequest	boolean	Obsolete. Use allow="payment".	HTML5 (removed)

<iframe>: attribute by attribute

Each attribute of <iframe> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML 4.01

The page to embed. about:blank shows an empty page.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe src="https://example.com/page.html">...</iframe>
```

srcdoc

Values: HTML text | Since: HTML5

Inline HTML to display instead of loading src. Overrides src if present.

Write srcdoc="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe srcdoc="HTML text">...</iframe>
```

title

Values: text | Since: HTML 4.01

A description of the embedded content for assistive technology. Strongly recommended.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe title="value">...</iframe>
```

name

Values: text | Since: HTML 4.01

A name that can be used as the target of links and forms.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe name="value">...</iframe>
```

width

Values: pixels | Since: HTML 4.01

Width of the frame (default 300).

Write `width="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe width="3">...</iframe>
```

height

Values: pixels | Since: HTML 4.01

Height of the frame (default 150).

Write `height="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe height="3">...</iframe>
```

sandbox

Values: empty or space-separated tokens | Since: HTML5

Applies extra restrictions. Empty value = all restrictions.

Tokens that lift restrictions: `allow-downloads`, `allow-forms`, `allow-modals`, `allow-orientation-lock`, `allow-pointer-lock`, `allow-popups`, `allow-popups-to-escape-sandbox`, `allow-presentation`, `allow-same-origin`, `allow-scripts`, `allow-top-navigation`, `allow-top-navigation-by-user-activation`, `allow-top-navigation-to-custom-protocols`.

Write `sandbox="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe sandbox="empty">...</iframe>
```


allow

Values: permissions policy | Since: HTML5

Features the frame may use, for example `allow="camera; microphone; fullscreen; geolocation; autoplay; picture-in-picture"`.

Write `allow="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe allow="permissions policy">...</iframe>
```

allowfullscreen

Values: boolean | Since: HTML5

Legacy form of `allow="fullscreen"`.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<iframe allowfullscreen>...</iframe>
```

loading

Values: eager | lazy | Since: Living Standard

lazy defers loading until the frame is near the viewport.

Write `loading="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe loading="eager">...</iframe>
```

referrerpolicy

Values: referrer policy keyword | Since: HTML5

Which referrer to send when loading the frame.

Write `referrerpolicy="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe referrerpolicy="referrer policy keyword">...</iframe>
```

credentialless

Values: boolean | Since: Experimental

Experimental: load the frame in a new, empty cookie/storage context.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<iframe credentialless>...</iframe>
```

csp

Values: policy string | Since: Experimental

Experimental: Content Security Policy to enforce on the embedded document.

Write `csp="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe csp="policy string">...</iframe>
```

align

Values: left | right | top | middle | bottom | Since: HTML 4.01

Obsolete. Use CSS.

Write `align="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe align="left">...</iframe>
```

frameborder

Values: 0 | 1 | Since: HTML 4.01

Obsolete. Use CSS border.

Write `frameborder="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe frameborder="0">...</iframe>
```

longdesc

Values: URL | Since: HTML 4.01

Obsolete. Link to a long description.

Write `longdesc="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe longdesc="https://example.com/page.html">...</iframe>
```

marginheight / marginwidth

Values: pixels | Since: HTML 4.01

Obsolete. Use CSS in the framed document.

Write `marginheight="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe marginheight="3">...</iframe>
```

scrolling

Values: yes | no | auto | Since: HTML 4.01

Obsolete. Use CSS overflow in the framed document.

Write `scrolling="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<iframe scrolling="yes">...</iframe>
```

allowpaymentrequest

Values: boolean | Since: HTML5 (removed)

Obsolete. Use `allow="payment"`.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<iframe allowpaymentrequest>...</iframe>
```

<iframe>: worked examples

Embedding a YouTube video

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/qfPUMV9J5yw"
  title="HTML tutorial video"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media;
  gyroscope; picture-in-picture"
  allowfullscreen loading="lazy"></iframe>
```

Sandboxed untrusted content

```
<iframe src="https://untrusted.example.com/widget" title="Widget"
  sandbox="allow-scripts"></iframe>
```

The widget can run scripts but cannot submit forms, open popups, or access cookies of its origin.

Inline document with srcdoc

```
<iframe title="Preview" srcdoc="<h1>Hello</h1><p>Rendered from srcdoc.
</p>"></iframe>
```

Responsive 16:9 video frame

```
<style>
  .video { aspect-ratio: 16 / 9; width: 100%; border: 0; }
</style>
<iframe class="video" src="..." title="Video"></iframe>
```

Targeting an iframe from a link

```
<iframe name="preview" title="Preview" src="about:blank"></iframe>
<a href="page1.html" target="preview">Load page 1 in the frame</a>
```

Default browser CSS

```
iframe { border: 2px inset; }
```

Accessibility

- Always provide a meaningful title; screen readers announce it when entering the frame.
- Content inside the iframe must itself be accessible; you cannot fix it from outside.
- Avoid iframes that steal keyboard focus or auto-play media.

Common mistakes

- No title.
- Never combining allow-scripts and allow-same-origin in sandbox for untrusted content (it lets the frame remove its own sandbox).
- Sizing with width="100%" attribute (percent is not valid; use CSS).
- Embedding sites that forbid framing and getting a blank box.

See also

<embed>, <object>, <frame>, <fencedframe>, <portal>

Quick card: <image>

Image (non-standard alias, obsolete)

<image> is an ancient, non-standard synonym for . For compatibility, the HTML parser silently converts a start tag named image into img. Never write it; always use .

At a glance

Status	OBSOLETE
Versions	Never standard
Category	Obsolete embedded content
Closing tag	None (void element)
Display	inline
Own attributes	src
Related	

Minimal example

```

```

Most common mistake

Any use.

<image> — Image (non-standard alias, obsolete)

Category: Obsolete embedded content | Status: OBSOLETE | Versions: Never standard

Description

<image> is an ancient, non-standard synonym for . For compatibility, the HTML parser silently converts a start tag named image into img. Never write it; always use .

Syntax

<image src="x.png"> <!-- becomes -->

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Historical
Permitted content	None
Permitted parents	Phrasing content
Tag omission	No end tag
DOM interface	HTMLImageElement (after parsing)

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src, alt, etc.	as 	Whatever the parser transfers to the it creates.	-

<image>: attribute by attribute

Each attribute of <image> explained in detail, with a minimal example you can copy.

src, alt, etc.

Values: as | Since: -

Whatever the parser transfers to the it creates.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<image src="as <img>">
```

: worked examples

**Use **

```

```

Common mistakes

- Any use.

See also

Quick card:

Image

The element embeds an image in the document. It is a void element; the image file is identified by the src attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The alt attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

At a glance

Status	CURRENT
Versions	Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Image and multimedia
Closing tag	None (void element)
Display	inline
Own attributes	src, alt, width, height, srcset, sizes, loading, decoding, fetchpriority, crossorigin, referrerpolicy, usemap, ismap, elementtiming, attributionsrc, align, border, hspace, longdesc, name, lowsrc
Related	<picture>, <source>, <figure>, <map>, <area>, <canvas>, <svg>

Minimal example

```

```

Most common mistake

Omitting alt or writing alt="image".

 — Image

Category: Image and multimedia | Status: CURRENT | Versions: Mosaic (1993), HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<img>` element embeds an image in the document. It is a void element; the image file is identified by the `src` attribute and is loaded separately. Supported formats include JPEG, PNG, GIF, WebP, AVIF, SVG, APNG, BMP and ICO. The `alt` attribute holds a text alternative that is displayed if the image cannot load and is read aloud by screen readers; it is required for accessibility.

HTML5 added responsive-image attributes: `srcset` lists several versions of the image at different sizes or pixel densities and `sizes` tells the browser how wide the image will be displayed, so the browser can download the smallest adequate file. The `loading="lazy"` attribute defers loading until the image is near the viewport, and `decoding="async"` lets the browser decode it off the main thread.

Always give width and height attributes (the intrinsic size in pixels). The browser uses them to reserve space before the image loads, which prevents the page from jumping (layout shift). With CSS `max-width: 100%; height: auto` the image still scales down on small screens.

The `<img>` element was invented by Marc Andreessen for the Mosaic browser in 1993 and was the first element to bring graphics to the web. In HTML 3.2 and 4.01 it carried presentational attributes (`align`, `border`, `hspace`, `vspace`) that are obsolete today.

Syntax

```

```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Flow content, phrasing content, embedded content, palpable content; form-associated and interactive if it has usemap
Permitted content	None (void element)
Permitted parents	Any element that accepts embedded content
Tag omission	No end tag
DOM interface	HTMLImageElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The image URL. Required (unless srcset is used with a browser that supports it).	HTML 2.0
alt	text	Text alternative. Describe the content or function of the image. Use alt="" (empty) for purely decorative images so that screen readers skip them. Omitting alt entirely is an error.	HTML 2.0
width	pixels	Intrinsic width. Helps the browser reserve space.	HTML 3.2

Attribute	Values	Description	Since
height	pixels	Intrinsic height.	HTML 3.2
srcset	comma-separated candidates	Alternative image files: 'small.jpg 480w, large.jpg 1200w' (width descriptors) or 'img.jpg 1x, img@2x.jpg 2x' (density descriptors).	HTML5
sizes	media conditions and lengths	How wide the image will be rendered, for example '(max-width: 600px) 100vw, 50vw'. Required when srcset uses width descriptors.	HTML5
loading	eager lazy	lazy delays loading until the image is near the viewport. Do not lazy-load images visible at page load.	Living Standard
decoding	sync async auto	Hint about how to decode the image.	Living Standard
fetchpriority	high low auto	Hint about the download priority; set high on the hero image.	Living Standard
crossorigin	anonymous use-credentials	Load with CORS so the image can be used in canvas without tainting it.	HTML5
referrerpolicy	referrer policy keyword	Which referrer to send.	HTML5
usemap	#map-name	Links the image to a <map> element to make it an image map.	HTML 3.2
ismap	boolean	Makes the image a server-side image map; the click coordinates are appended to the link URL. Only valid inside <a href>.	HTML 2.0
elementtiming	text	Marks the image for the Element Timing performance API.	Living Standard

Attribute	Values	Description	Since
attribution src	empty or URL	Experimental attribution reporting.	Experimental
align	top middle bottom left right	Obsolete. Use CSS float or vertical-align.	HTML 2.0
border	pixels	Obsolete. Border width. Use CSS border.	HTML 3.2
hspace / vspace	pixels	Obsolete. Horizontal and vertical margins. Use CSS margin.	HTML 3.2
longdesc	URL	Obsolete. Link to a long description. Use a normal link or aria-describedby.	HTML 4.01
name	text	Obsolete. Use id.	HTML 4.01
lowsrc	URL	Obsolete, non-standard. Low-quality placeholder loaded first.	Netscape

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML 2.0

The image URL. Required (unless srcset is used with a browser that supports it).

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```

```

alt

Values: text | Since: HTML 2.0

Text alternative. Describe the content or function of the image. Use alt="" (empty) for purely decorative images so that screen readers skip them. Omitting alt entirely is an error.

Write alt="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img alt="value">
```

width

Values: pixels | Since: HTML 3.2

Intrinsic width. Helps the browser reserve space.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img width="3">
```


height

Values: pixels | Since: HTML 3.2

Intrinsic height.

Write `height="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img height="3">
```

srcset

Values: comma-separated candidates | Since: HTML5

Alternative image files: 'small.jpg 480w, large.jpg 1200w' (width descriptors) or 'img.jpg 1x, img@2x.jpg 2x' (density descriptors).

Write `srcset="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img srcset="value">
```

sizes

Values: media conditions and lengths | Since: HTML5

How wide the image will be rendered, for example '(max-width: 600px) 100vw, 50vw'. Required when `srcset` uses width descriptors.

Write `sizes="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img sizes="value">
```

loading

Values: eager | lazy | Since: Living Standard

lazy delays loading until the image is near the viewport. Do not lazy-load images visible at page load.

Write `loading="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img loading="eager">
```

decoding

Values: sync | async | auto | Since: Living Standard

Hint about how to decode the image.

Write decoding="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img decoding="sync">
```

fetchpriority

Values: high | low | auto | Since: Living Standard

Hint about the download priority; set high on the hero image.

Write fetchpriority="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img fetchpriority="high">
```

crossorigin

Values: anonymous | use-credentials | Since: HTML5

Load with CORS so the image can be used in canvas without tainting it.

Write crossorigin="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img crossorigin="anonymous">
```

referrerpolicy

Values: referrer policy keyword | Since: HTML5

Which referrer to send.

Write referrerpolicy="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img referrerpolicy="referrer policy keyword">
```

usemap

Values: #map-name | Since: HTML 3.2

Links the image to a <map> element to make it an image map.

Write usemap="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img usemap="#map-name">
```

ismap

Values: boolean | Since: HTML 2.0

Makes the image a server-side image map; the click coordinates are appended to the link URL. Only valid inside <a href>.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<img ismap>
```

elementtiming

Values: text | Since: Living Standard

Marks the image for the Element Timing performance API.

Write elementtiming="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img elementtiming="value">
```

attributionsrc

Values: empty or URL | Since: Experimental

Experimental attribution reporting.

Write attributionsrc="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```

```

align

Values: top | middle | bottom | left | right | Since: HTML 2.0

Obsolete. Use CSS float or vertical-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img align="top">
```

border

Values: pixels | Since: HTML 3.2

Obsolete. Border width. Use CSS border.

Write border="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img border="3">
```

hspace / vspace

Values: pixels | Since: HTML 3.2

Obsolete. Horizontal and vertical margins. Use CSS margin.

Write hspace="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img hspace="3">
```

longdesc

Values: URL | Since: HTML 4.01

Obsolete. Link to a long description. Use a normal link or aria-describedby.

Write longdesc="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img longdesc="https://example.com/page.html">
```

name

Values: text | Since: HTML 4.01

Obsolete. Use id.

Write `name="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<img name="value">
```

lowsrc

Values: URL | Since: Netscape

Obsolete, non-standard. Low-quality placeholder loaded first.

Write `lowsrc="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```

```

: worked examples

Basic image with good alt text

```

```

Decorative image

```

```

Empty alt tells screen readers to ignore it.

Responsive images with srcset and sizes

```

```

On a phone the image fills the screen, so the browser picks the 400w or 800w file; on a desktop it is shown at 600px wide.

High-DPI (retina) logo

```

```

Lazy loading images below the fold

```

```

Image as a link

```
<a href="index.html"></a>
```

The alt describes where the link goes.

Fluid image CSS

```
<style>
img { max-width: 100%; height: auto; }
</style>
```

Keeps images inside their container on small screens while width/height attributes still prevent layout shift.

HTML 3.2 style and its replacement

```
<!-- Old -->


<!-- New -->

```

Default browser CSS

```
img { display: inline; }
```

Accessibility

- alt is required. Write what the image conveys, not 'image of'. Keep it under about 150 characters; for complex images give a longer description in the text.
- Text inside images cannot be read by screen readers or resized; use real text.
- Animated GIFs that flash can trigger seizures; avoid or provide a pause control.

Common mistakes

- Omitting alt or writing alt="image".
- Omitting width and height, causing layout shift.
- Using a huge original file where a resized one would do.
- Lazy-loading the hero image (it delays the largest contentful paint).
- Using the align attribute.

See also

<picture>, <source>, <figure>, <map>, <area>, <canvas>, <svg>

Quick card: <input>

Input control

The <input> element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the type attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Forms
Closing tag	None (void element)
Display	inline-block
Own attributes	type, name, value, id, disabled, readonly, required, placeholder, autocomplete, autofocus, checked, maxlength, minlength, size, min, max, step, pattern, list, multiple, accept, capture, src, alt, width, form, formaction, formenctype, formmethod, formnovalidate, formtarget, dirname, inputmode, popovertarget, alpha, colorspace, align, usemap, incremental
Related	<form>, <label>, <button>, <select>, <textarea>, <datalist>, <output>, <fieldset>

Minimal example

```
<label for="user">Username</label>
<input id="user" name="user" type="text" placeholder="e.g. ahmed99"
required minlength="3" maxlength="20" autocomplete="username">
```

Most common mistake

Missing name (nothing submitted).

<input> — Input control

Category: Forms | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<input>` element creates an interactive control for accepting data from the user. It is the most versatile element in HTML: the `type` attribute selects one of more than twenty different controls, from a plain text box to a colour picker, a file chooser, a range slider or a hidden field. It is a void element.

HTML 2.0 defined `text`, `password`, `checkbox`, `radio`, `submit`, `reset`, `hidden` and `image`; HTML 3.2 and 4.01 added `file` and `button`; HTML5 added `email`, `url`, `tel`, `search`, `number`, `range`, `color`, `date`, `month`, `week`, `time` and `datetime-local`. Browsers fall back to `type="text"` for any type they do not recognise, so new types are safe to use.

Each type accepts a different subset of attributes; the complete table below lists every attribute and Part 6 of this book describes every type in its own entry with examples.

Syntax

```
<input type="text" name="field" id="field">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline-block

Property	Value
Content categories	Flow content, phrasing content, listed, submittable, resettable, form-associated, palpable; interactive and labelable unless type=hidden
Permitted content	None (void element)
Permitted parents	Any element that accepts phrasing content
Tag omission	No end tag
DOM interface	HTMLInputElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
type	see Part 6	The kind of control. Default text. Values: button, checkbox, color, date, datetime-local, email, file, hidden, image, month, number, password, radio, range, reset, search, submit, tel, text, time, url, week.	HTML 2.0 / HTML5
name	text	The name under which the value is submitted. Without it the control is not submitted. Radio buttons with the same name form a group.	HTML 2.0
value	text	The initial value (text types), the value submitted when checked (checkbox, radio), or the button label (submit, reset, button).	HTML 2.0
id	identifier	Links the control to a <label for="id">.	HTML 4.01

Attribute	Values	Description	Since
disabled	boolean	Cannot be used or focused and is not submitted.	HTML 4.01
readonly	boolean	Cannot be edited but is focusable and submitted. Text-like types only.	HTML 4.01
required	boolean	Must have a value before the form can be submitted.	HTML5
placeholder	text	Hint shown inside an empty text-like control. Not a substitute for a label.	HTML5
autocomplete	on off token list	Autofill hint: name, email, tel, street-address, postal-code, cc-number, new-password, current-password, one-time-code and more (see Part 9).	HTML5
autofocus	boolean	Focus this control on page load. Only one per page.	HTML5
checked	boolean	Initially selected (checkbox, radio).	HTML 2.0
maxlength	integer	Maximum number of characters (text-like types).	HTML 2.0
minlength	integer	Minimum number of characters.	HTML5
size	integer	Visible width in characters (text-like types). Prefer CSS width.	HTML 2.0
min	number or date	Minimum value (number, range, date/time types).	HTML5
max	number or date	Maximum value.	HTML5
step	number or any	Granularity of allowed values; 'any' allows any value.	HTML5

Attribute	Values	Description	Since
pattern	regular expression	A JavaScript regular expression the value must match (text, search, url, tel, email, password). Describe the format in title.	HTML5
list	datalist id	Links to a <datalist> of suggestions.	HTML5
multiple	boolean	Allow several values (email: comma-separated; file: several files).	HTML5
accept	MIME types / extensions	Allowed file types for type=file, for example image/*, .pdf, audio/*.	HTML 4.01
capture	user environment	For type=file on phones: use the front (user) or back (environment) camera/microphone directly.	Media Capture spec
src	URL	The image for type=image.	HTML 2.0
alt	text	Alternative text for type=image.	HTML 2.0
width / height	pixels	Size of the image for type=image.	HTML5
form	form id	Associate with a form anywhere in the document.	HTML5
formaction	URL	Override the form action (submit, image).	HTML5
formenctype	encoding type	Override the form enctype (submit, image).	HTML5
formmethod	get post dialog	Override the form method (submit, image).	HTML5
formnovalidate	boolean	Skip validation when submitting with this button.	HTML5
formtarget	browsing context	Override the form target (submit, image).	HTML5

Attribute	Values	Description	Since
dirname	text	Submit the text direction of the value under this name.	HTML5
inputmode	none text decimal numeric tel search email url	Which virtual keyboard to show on touch devices (global attribute).	HTML5
popovertarget / popovertargetaction	id / show hide toggle	Control a popover from a button-type input.	Living Standard
alpha	boolean	Experimental: allow transparency in type=color.	Experimental
colorspace	limited-srgb display-p3	Experimental: colour space for type=color.	Experimental
align	left right top middle bottom	Obsolete. Use CSS.	HTML 3.2
usemap	#map	Obsolete on input. Use .	HTML 4.01
incremental, results, webkitdirectory, orient, mozactionhint	various	Non-standard browser-specific attributes.	Non-standard

<input>: attribute by attribute

Each attribute of <input> explained in detail, with a minimal example you can copy.

type

Values: see Part 6 | Since: HTML 2.0 / HTML5

The kind of control. Default text. Values: button, checkbox, color, date, datetime-local, email, file, hidden, image, month, number, password, radio, range, reset, search, submit, tel, text, time, url, week.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input type="see Part 6">
```

name

Values: text | Since: HTML 2.0

The name under which the value is submitted. Without it the control is not submitted. Radio buttons with the same name form a group.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input name="value">
```

value

Values: text | Since: HTML 2.0

The initial value (text types), the value submitted when checked (checkbox, radio), or the button label (submit, reset, button).

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input value="value">
```

id

Values: identifier | Since: HTML 4.01

Links the control to a `<label for="id">`.

Write `id="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input id="identifier">
```

disabled

Values: boolean | Since: HTML 4.01

Cannot be used or focused and is not submitted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input disabled>
```

readonly

Values: boolean | Since: HTML 4.01

Cannot be edited but is focusable and submitted. Text-like types only.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input readonly>
```

required

Values: boolean | Since: HTML5

Must have a value before the form can be submitted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input required>
```

placeholder

Values: text | Since: HTML5

Hint shown inside an empty text-like control. Not a substitute for a label.

Write `placeholder="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input placeholder="value">
```


autocomplete

Values: on | off | token list | Since: HTML5

Autofill hint: name, email, tel, street-address, postal-code, cc-number, new-password, current-password, one-time-code and more (see Part 9).

Write `autocomplete="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input autocomplete="on">
```

autofocus

Values: boolean | Since: HTML5

Focus this control on page load. Only one per page.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input autofocus>
```

checked

Values: boolean | Since: HTML 2.0

Initially selected (checkbox, radio).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input checked>
```

maxlength

Values: integer | Since: HTML 2.0

Maximum number of characters (text-like types).

Write `maxlength="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input maxlength="3">
```

minlength

Values: integer | Since: HTML5

Minimum number of characters.

Write `maxlength="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input minlength="3">
```

size

Values: integer | Since: HTML 2.0

Visible width in characters (text-like types). Prefer CSS width.

Write `size="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input size="3">
```

min

Values: number or date | Since: HTML5

Minimum value (number, range, date/time types).

Write `min="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input min="3">
```

max

Values: number or date | Since: HTML5

Maximum value.

Write `max="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input max="3">
```

step

Values: number or any | Since: HTML5

Granularity of allowed values; 'any' allows any value.

Write `step="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input step="3">
```

pattern

Values: regular expression | Since: HTML5

A JavaScript regular expression the value must match (text, search, url, tel, email, password). Describe the format in title.

Write `pattern="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input pattern="regular expression">
```

list

Values: datalist id | Since: HTML5

Links to a `<datalist>` of suggestions.

Write `list="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input list="datalist id">
```

multiple

Values: boolean | Since: HTML5

Allow several values (email: comma-separated; file: several files).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input multiple>
```

accept

Values: MIME types / extensions | Since: HTML 4.01

Allowed file types for `type=file`, for example `image/*`, `.pdf`, `audio/*`.

Write `accept="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input accept="text/plain">
```

capture

Values: user | environment | Since: Media Capture spec

For type=file on phones: use the front (user) or back (environment) camera/microphone directly.

Write capture="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input capture="user">
```

src

Values: URL | Since: HTML 2.0

The image for type=image.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input src="https://example.com/page.html">
```

alt

Values: text | Since: HTML 2.0

Alternative text for type=image.

Write alt="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input alt="value">
```

width / height

Values: pixels | Since: HTML5

Size of the image for type=image.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input width="3">
```

form

Values: form id | Since: HTML5

Associate with a form anywhere in the document.

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input form="form id">
```

formaction

Values: URL | Since: HTML5

Override the form action (submit, image).

Write `formaction="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input formaction="https://example.com/page.html">
```

formenctype

Values: encoding type | Since: HTML5

Override the form enctype (submit, image).

Write `formenctype="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input formenctype="encoding type">
```

formmethod

Values: get | post | dialog | Since: HTML5

Override the form method (submit, image).

Write `formmethod="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input formmethod="get">
```

formnovalidate

Values: boolean | Since: HTML5

Skip validation when submitting with this button.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input formnovalidate>
```

formtarget

Values: browsing context | Since: HTML5

Override the form target (submit, image).

Write formtarget="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input formtarget="browsing context">
```

dirname

Values: text | Since: HTML5

Submit the text direction of the value under this name.

Write dirname="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input dirname="value">
```

inputmode

Values: none | text | decimal | numeric | tel | search | email | url | Since: HTML5

Which virtual keyboard to show on touch devices (global attribute).

Write inputmode="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input inputmode="none">
```

popovertarget / popovertargetaction

Values: id / show | hide | toggle | Since: Living Standard

Control a popover from a button-type input.

Write popovertarget="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input popovertarget="my-id">
```

alpha

Values: boolean | Since: Experimental

Experimental: allow transparency in `type=color`.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<input alpha>
```

colorspace

Values: limited-srgb | display-p3 | Since: Experimental

Experimental: colour space for `type=color`.

Write `colorspace="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input colorspace="limited-srgb">
```

align

Values: left | right | top | middle | bottom | Since: HTML 3.2

Obsolete. Use CSS.

Write `align="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input align="left">
```

usemap

Values: #map | Since: HTML 4.01

Obsolete on input. Use `<img usemap>`.

Write `usemap="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input usemap="#map">
```

incremental, results, webkitdirectory, orient, mozactionhint

Values: various | Since: Non-standard

Non-standard browser-specific attributes.

Write `incremental="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<input incremental="various">
```


<input>: worked examples

Text with label, placeholder and required

```
<label for="user">Username</label>
<input id="user" name="user" type="text" placeholder="e.g. ahmed99"
required minlength="3" maxlength="20" autocomplete="username">
```

Email and password

```
<label for="e">Email</label>
<input id="e" name="email" type="email" required autocomplete="email">
<label for="p">Password</label>
<input id="p" name="password" type="password" required minlength="8"
autocomplete="current-password">
```

Checkboxes and radio buttons

```
<label><input type="checkbox" name="terms" value="yes" required> I
agree to the terms</label>

<fieldset>
  <legend>Size</legend>
  <label><input type="radio" name="size" value="s"> Small</label>
  <label><input type="radio" name="size" value="m" checked> Medium</
label>
  <label><input type="radio" name="size" value="l"> Large</label>
</fieldset>
```

Number, range and date

```
<label>Quantity <input type="number" name="qty" min="1" max="10"
step="1" value="1"></label>
<label>Volume <input type="range" name="vol" min="0" max="100"
value="50"></label>
<label>Delivery date <input type="date" name="date" min="2026-09-
12"></label>
```

Pattern validation with an explanation

```
<label>Postal code <input name="zip" pattern="[0-9]{5}" title="Five
digits, for example 12511" inputmode="numeric"></label>
```

Hidden field

```
<input type="hidden" name="product_id" value="42">
```

Sent with the form but not shown.

File upload restricted to images

```
<input type="file" name="photo" accept="image/png, image/jpeg"
multiple>
```

Styling validity

```
<style>
  input:invalid { border-color: #b90000; }
  input:valid { border-color: #0f8a4c; }
  input:focus { outline: 2px solid #1b6fd8; }
</style>
```

Default browser CSS

```
input { display: inline-block; }
```

Accessibility

- Every input needs a label (visible `<label>` or `aria-label`).
- Placeholder disappears when typing; never use it as the only label.
- Use the correct type so mobile keyboards and screen readers behave correctly.
- Group radio buttons and checkboxes in a fieldset with a legend.
- Do not disable the submit button until the form is valid; let users submit and see the errors.

Common mistakes

- Missing name (nothing submitted).
- Missing label.
- Using `type="number"` for values that are not quantities (phone numbers, postal codes); use `type="tel"` or text with `inputmode`.
- Writing a closing `</input>` tag.
- Relying only on client-side validation; always validate on the server too.

See also

<form>, <label>, <button>, <select>, <textarea>,
<datalist>, <output>, <fieldset>

Quick card: <ins>

Inserted text

The <ins> element marks text that has been added to a document, to show changes between versions. Browsers underline it by default. It is the counterpart of . Like it accepts optional cite and datetime attributes that record why and when the change was made.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Demarcating edits
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	cite, datetime
Related	, <u>

Minimal example

```
<p>Our office is open <del>9 to 5</del> <ins datetime="2026-09-01">8  
to 4</ins>.</p>
```

Most common mistake

Using <u> to show insertions.

<ins> — Inserted text

Category: Demarcating edits | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<ins>` element marks text that has been added to a document, to show changes between versions. Browsers underline it by default. It is the counterpart of `<del>`. Like `<del>` it accepts optional `cite` and `datetime` attributes that record why and when the change was made.

Syntax

```
<ins datetime="YYYY-MM-DD">inserted text</ins>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Transparent
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLModElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
cite	URL	A URL explaining the change.	HTML 4.01
datetime	date or date-time	When the change was made.	HTML 4.01

<ins>: attribute by attribute

Each attribute of <ins> explained in detail, with a minimal example you can copy.

cite

Values: URL | Since: HTML 4.01

A URL explaining the change.

Write cite="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<ins cite="https://example.com/page.html">...</ins>
```

datetime

Values: date or date-time | Since: HTML 4.01

When the change was made.

Write datetime="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<ins datetime="date">...</ins>
```

<ins>: worked examples

Tracking a change

```
<p>Our office is open <del>9 to 5</del> <ins datetime="2026-09-01">8  
to 4</ins>.</p>
```

Styling insertions

```
<style>  
  ins { background: #dffe0; text-decoration: none; }  
  del { background: #ffe0e0; }  
</style>
```

Default browser CSS

```
ins { text-decoration: underline; }
```

Accessibility

- Not announced by most screen readers by default; add hidden text if the change must be conveyed.

Common mistakes

- Using <u> to show insertions.
- Wrapping several block elements in one inline <ins>.

See also

, <u>

Quick card: <isindex>

Search index prompt (obsolete)

The <isindex> element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a <form> with an <input type="search">.

At a glance

Status	OBSOLETE
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete forms
Closing tag	None (void element)
Display	block
Own attributes	prompt, action
Related	<form>, <input>

Minimal example

```
<form action="/search" method="get">
  <label>Search: <input type="search" name="q"></label>
</form>
```

Most common mistake

Any use.

<isindex> — Search index prompt (obsolete)

Category: Obsolete forms | Status: OBSOLETE | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The <isindex> element, one of the original 1991 elements, told the browser that the document was a searchable index and made it display a single-line text field with a prompt. Submitting the field reloaded the page with ?keywords in the URL. It was made redundant by forms in HTML 2.0, deprecated in HTML 4.01 and removed in HTML5. Use a <form> with an <input type="search">.

Syntax

<isindex prompt="Search: ">

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	block
Content categories	Historical
Permitted content	None
Permitted parents	<head> or <body>
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
prompt	text	The label shown before the field.	HTML 2.0
action	URL	Where to send the query (non-standard).	Netscape

<isindex>: attribute by attribute

Each attribute of <isindex> explained in detail, with a minimal example you can copy.

prompt

Values: text | Since: HTML 2.0

The label shown before the field.

Write `prompt="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<isindex prompt="value">
```

action

Values: URL | Since: Netscape

Where to send the query (non-standard).

Write `action="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<isindex action="https://example.com/page.html">
```

<isindex>: worked examples

Replacement

```
<form action="/search" method="get">
  <label>Search: <input type="search" name="q"></label>
</form>
```

Common mistakes

- Any use.

See also

<form>, <input>

Quick card: <kbd>

Keyboard input

The <kbd> element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting <kbd> elements inside a <kbd> represents individual keys in a key combination; wrapping <kbd> in <samp> represents input echoed by the system.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code>, <samp>, <var>

Minimal example

<p>Press <kbd><kbd>Ctrl</kbd> + <kbd>S</kbd></kbd> to save.</p>

Most common mistake

Using <code> for keys.

<kbd> — Keyboard input

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<kbd>` element marks text that represents user input from a keyboard, voice command, or other input device. Browsers render it in a monospace font. Nesting `<kbd>` elements inside a `<kbd>` represents individual keys in a key combination; wrapping `<kbd>` in `<samp>` represents input echoed by the system.

Syntax

`<kbd>Ctrl</kbd> + <kbd>S</kbd>`

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<kbd>: worked examples

Keyboard shortcut

<p>Press <kbd><kbd>Ctrl</kbd> + <kbd>S</kbd></kbd> to save.</p>

Key-cap styling

```
<style>
  kbd { border: 1px solid #aaa; border-radius: 4px; padding: 0 4px;
background: #f5f5f5; font-size: 0.9em; box-shadow: 0 1px 0 #999; }
</style>
```

Menu selection

<p>Choose <kbd>File</kbd> then <kbd>Save As...</kbd>.</p>

Default browser CSS

```
kbd { font-family: monospace; }
```

Common mistakes

- Using <code> for keys.
- Using <kbd> for program output (use <samp>).

See also

<code>, <samp>, <var>

Quick card: <keygen>

Key-pair generator (obsolete)

The <keygen> element generated a public/private key pair when a form was submitted, for client certificate enrolment. It came from Netscape, was added to HTML5 and then removed from the standard in 2017 and from all browsers. There is no direct replacement; use the Web Crypto API.

At a glance

Status	OBSOLETE
Versions	Netscape, briefly in HTML5; removed
Category	Obsolete forms
Closing tag	None (void element)
Display	inline-block
Own attributes	challenge, keytype, name, autofocus
Related	<input>, <form>

Minimal example

```
<!-- No modern replacement in HTML; see crypto.subtle.generateKey() -->
```

Most common mistake

Any use.

<keygen> — Key-pair generator (obsolete)

Category: Obsolete forms | Status: OBSOLETE | Versions: Netscape, briefly in HTML5; removed

Description

The <keygen> element generated a public/private key pair when a form was submitted, for client certificate enrolment. It came from Netscape, was added to HTML5 and then removed from the standard in 2017 and from all browsers. There is no direct replacement; use the Web Crypto API.

Syntax

<keygen name="key" challenge="...">

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline-block
Content categories	Historical
Permitted content	None
Permitted parents	Phrasing content
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
challenge	text	Challenge string.	HTML5
keytype	rsa dsa ec	Key algorithm.	HTML5
name	text	Form field name.	HTML5
autofocus, disabled, form	-	As on other controls.	HTML5

<keygen>: attribute by attribute

Each attribute of <keygen> explained in detail, with a minimal example you can copy.

challenge

Values: text | Since: HTML5

Challenge string.

Write challenge="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<keygen challenge="value">
```

keytype

Values: rsa | dsa | ec | Since: HTML5

Key algorithm.

Write keytype="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<keygen keytype="rsa">
```

name

Values: text | Since: HTML5

Form field name.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<keygen name="value">
```

autofocus, disabled, form

Values: - | Since: HTML5

As on other controls.

Write autofocus="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<keygen autofocus="-">
```

<keygen>: worked examples

Do not use

```
<!-- No modern replacement in HTML; see crypto.subtle.generateKey() -->
```

Common mistakes

- Any use.

See also

<input>, <form>

Quick card: <label>

Form control label

The <label> element gives a caption to a form control. Associating a label with a control has two big benefits: clicking the label focuses or toggles the control (a much larger click target for checkboxes and radio buttons), and screen readers announce the label when the control gets focus.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	for, form
Related	<input>, <select>, <textarea>, <button>, <fieldset>, <legend>

Minimal example

```
<label for="city">City</label>
<input id="city" name="city">
```

Most common mistake

for pointing to a name instead of an id.

<label> — Form control label

Category: Forms | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<label>` element gives a caption to a form control. Associating a label with a control has two big benefits: clicking the label focuses or toggles the control (a much larger click target for checkboxes and radio buttons), and screen readers announce the label when the control gets focus.

There are two ways to associate them. Explicit: give the control an id and set the label's `for` attribute to that id. Implicit: nest the control inside the label. You may combine both. A label may be associated with only one control, but a control may have several labels.

Labelable elements are `<button>`, `<input>` (except hidden), `<meter>`, `<output>`, `<progress>`, `<select>` and `<textarea>`.

Syntax

```
<label for="id">Text</label> <input id="id">
<!-- or -->
<label>Text <input></label>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, interactive content, form-associated, palpable content
Permitted content	Phrasing content, but no other <code><label></code> and at most one labelable descendant

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLLabelElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
for	id of a labelable element	The control this label describes.	HTML 4.01
form	form id	Obsolete in the Living Standard; was meant to associate the label with a form.	HTML5 (removed)

<label>: attribute by attribute

Each attribute of <label> explained in detail, with a minimal example you can copy.

for

Values: id of a labelable element | Since: HTML 4.01

The control this label describes.

Write for="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<label for="my-id">...</label>
```

form

Values: form id | Since: HTML5 (removed)

Obsolete in the Living Standard; was meant to associate the label with a form.

Write form="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<label form="form id">...</label>
```

<label>: worked examples

Explicit association

```
<label for="city">City</label>
<input id="city" name="city">
```

Implicit association (wrapping)

```
<label>
  <input type="checkbox" name="news"> Send me the newsletter
</label>
```

Clicking anywhere on the text toggles the box.

Labels stacked above fields

```
<style>
  label { display: block; margin-bottom: 4px; font-weight: 600; }
  input, select, textarea { display: block; width: 100%; margin-
bottom: 16px; }
</style>
```

Required-field indicator

```
<label for="n">Name <span aria-hidden="true">*</span></label>
<input id="n" name="n" required aria-required="true">
```

Default browser CSS

```
label { cursor: default; }
```

Accessibility

- A label is the correct way to name a control; aria-label is a fallback when no visible text is possible.
- Do not put headings or interactive elements inside a label.
- Keep the label visible; placeholder text is not a label.

Common mistakes

- for pointing to a name instead of an id.
- Two controls inside one label.
- Using <label> for text that is not a caption for a control.

See also

<input>, <select>, <textarea>, <button>, <fieldset>,
<legend>

Quick card: <legend>

Fieldset caption

The <legend> element provides a caption for its parent <fieldset>. It must be the first child of the fieldset. Browsers draw it on top of the fieldset's border. Screen readers announce the legend together with the label of each control inside the group, which is why fieldset/legend is the recommended way to caption groups of radio buttons or checkboxes.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	align
Related	<fieldset>, <label>

Minimal example

```
<fieldset>
  <legend>Contact preference</legend>
  <label><input type="radio" name="c" value="email"> Email</label>
  <label><input type="radio" name="c" value="phone"> Phone</label>
</fieldset>
```

Most common mistake

Legend not first in the fieldset.

<legend> — Fieldset caption

Category: Forms | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<legend>` element provides a caption for its parent `<fieldset>`. It must be the first child of the fieldset. Browsers draw it on top of the fieldset's border. Screen readers announce the legend together with the label of each control inside the group, which is why fieldset/legend is the recommended way to caption groups of radio buttons or checkboxes.

Syntax

```
<fieldset>
  <legend>Caption</legend>
  ...
</fieldset>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	Phrasing content and optionally heading content
Permitted parents	<code><fieldset></code> , as its first child
Tag omission	Neither tag may be omitted
DOM interface	HTMLLegendElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right top bottom	Obsolete. Use CSS.	HTML 4.01

<legend>: attribute by attribute

Each attribute of <legend> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | top | bottom | Since: HTML 4.01

Obsolete. Use CSS.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<legend align="left">...</legend>
```

<legend>: worked examples

Captioned group

```
<fieldset>
  <legend>Contact preference</legend>
  <label><input type="radio" name="c" value="email"> Email</label>
  <label><input type="radio" name="c" value="phone"> Phone</label>
</fieldset>
```

Legend containing a heading

```
<fieldset>
  <legend><h2>Billing address</h2></legend>
  ...
</fieldset>
```

Allowed since the Living Standard; keeps the document outline while still captioning the group.

Default browser CSS

```
legend { display: block; padding-inline: 2px; }
```

Accessibility

- Keep legends short; they are repeated by some screen readers for every control in the group.

Common mistakes

- Legend not first in the fieldset.
- Using legend outside a fieldset.

See also

<fieldset>, <label>

Quick card:

List item

The element represents one item in a list. It must be contained in an ordered list (), an unordered list () or a menu (<menu>). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	list-item
Own attributes	value, type
Related	, , <menu>, <dl>

Minimal example

```
<ul>
  <li>Milk</li>
  <li>Bread</li>
  <li>Eggs</li>
</ul>
```

Most common mistake

Placing directly in <div> or <body>.

 — List item

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<li>` element represents one item in a list. It must be contained in an ordered list (`<ol>`), an unordered list (`<ul>`) or a menu (`<menu>`). In ordered lists the value attribute sets the number of the item, and subsequent items continue counting from it.

A list item may contain any flow content: paragraphs, images, nested lists, even tables. To nest a list, place the inner `<ul>` or `<ol>` inside the `<li>`, not between two `<li>` elements.

Syntax

```
<ul>
  <li>Item</li>
</ul>
```

Technical summary

Property	Value
Void element	No
Default display	list-item
Content categories	None
Permitted content	Flow content
Permitted parents	<code></code> , <code></code> , <code><menu></code>
Tag omission	End tag may be omitted if followed by another <code></code> or if there is no more content in the parent

Property	Value
DOM interface	HTMLLIElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
value	integer	In : the ordinal number of this item. Following items count from it.	HTML 3.2
type	1 a A i I disc circle square	Obsolete. Bullet or numbering style for this item. Use CSS list-style-type.	HTML 3.2

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

value

Values: integer | Since: HTML 3.2

In : the ordinal number of this item. Following items count from it.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<li value="3">...</li>
```

type

Values: 1 | a | A | i | I | disc | circle | square | Since: HTML 3.2

Obsolete. Bullet or numbering style for this item. Use CSS list-style-type.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<li type="1">...</li>
```

: worked examples

Unordered list

```
<ul>
  <li>Milk</li>
  <li>Bread</li>
  <li>Eggs</li>
</ul>
```

Displays with bullets.

Ordered list with a custom start value

```
<ol>
  <li value="10">Tenth</li>
  <li>Eleventh</li>
  <li>Twelfth</li>
</ol>
```

Nested list (correct)

```
<ul>
  <li>Fruit
    <ul>
      <li>Apples</li>
      <li>Oranges</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

The inner list is inside the first .

Rich list items

```
<ol>
  <li>
    <h3>Step one</h3>
    <p>Open your editor.</p>
    
  </li>
</ol>
```

Horizontal menu with CSS

```
<style>
  nav ul { list-style: none; padding: 0; margin: 0; display: flex;
gap: 16px; }
</style>
<nav><ul><li><a href="/">Home</a></li><li><a href="/shop">Shop</a></
li></ul></nav>
```

Default browser CSS

```
li { display: list-item; text-align: match-parent; }
```

Accessibility

- Screen readers announce the number of items in a list and the position of each; this is lost if `list-style: none` is combined with `display: contents` in some browsers, or if you fake a list with `<br>`.

Common mistakes

- Placing `<li>` directly in `<div>` or `<body>`.
- Nesting a list between `<li>` elements rather than inside one.
- Using `<br>` instead of separate items.

See also

`<ul>`, `<ol>`, `<menu>`, `<dl>`

Quick card: <link>

External resource link

The <link> element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (rel="stylesheet"), but it is also used for site icons (rel="icon"), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in <head>, though body-ok link types such as stylesheet may appear in <body>.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Document metadata
Closing tag	None (void element)
Display	none
Own attributes	rel, href, type, media, as, crossorigin, hreflang, sizes, title, integrity, referrerpolicy, fetchpriority, blocking, disabled, imagesrcset, color, charset, rev, target
Related	<style>, <a>, <script>, <meta>, <head>

Minimal example

```
<link rel="stylesheet" href="style.css">
<link rel="stylesheet" href="print.css" media="print">
<link rel="stylesheet" href="dark.css" media="(prefers-color-scheme:
dark)">
```

Most common mistake

Forgetting rel.

<link> — External resource link

Category: Document metadata | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <link> element specifies a relationship between the current document and an external resource. By far its most common use is loading a CSS style sheet (rel="stylesheet"), but it is also used for site icons (rel="icon"), web app manifests, preloading and prefetching resources, canonical URLs for search engines, alternate language versions, RSS feeds and more. It is a void element and normally lives in <head>, though body-ok link types such as stylesheet may appear in <body>.

The rel attribute is required and determines everything else. The media attribute lets a style sheet apply only to certain devices or screen sizes; the browser still downloads it but with low priority. The full list of link types is in Part 9.

Syntax

```
<link rel="stylesheet" href="style.css">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Metadata content; flow and phrasing content if itemprop or a body-ok rel is present

Property	Value
Permitted content	None (void element)
Permitted parents	<head>; <body> for body-ok types
Tag omission	No end tag
DOM interface	HTMLLinkElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
rel	space-separated link types	Required. Relationship: stylesheet, icon, apple-touch-icon, manifest, canonical, alternate, preload, prefetch, preconnect, dns-prefetch, modulepreload, prerender, author, license, help, next, prev, search, pingback, privacy-policy, terms-of-service, expect.	HTML 2.0
href	URL	The resource URL.	HTML 2.0
type	MIME type	The type of the resource, for example text/css, image/png, application/rss+xml.	HTML 2.0
media	media query	Apply the style sheet only when the query matches: screen, print, (max-width: 600px), (prefers-color-scheme: dark).	HTML 4.01

Attribute	Values	Description	Since
as	audio document embed fetch font image object script style track video worker	With rel=preload or modulepreload: the type of content being loaded, so the browser can prioritise it correctly.	Living Standard
crossorigin	anonymous use-credentials	CORS mode for the request. Required when preloading fonts.	HTML5
hreflang	language tag	Language of the linked resource (with rel=alternate).	HTML 4.01
sizes	any or WxH list	Icon sizes, for example 16x16 32x32 or any (for SVG).	HTML5
title	text	For style sheets: the name of an alternate style sheet set. For feeds: the feed title.	HTML 2.0
integrity	hash	Subresource Integrity hash; the browser refuses the file if it does not match.	SRI spec
referrerpolicy	referrer policy	Referrer to send when fetching.	HTML5
fetchpriority	high low auto	Download priority hint.	Living Standard
blocking	render	Block rendering until the resource is loaded (for stylesheets/preloads/expect).	Living Standard
disabled	boolean	For style sheets: do not apply.	HTML5
imagesrcset / imagesizes	srcset / sizes syntax	With rel=preload as=image: responsive image candidates.	Living Standard

Attribute	Values	Description	Since
color	colour	With rel=mask-icon (Safari pinned tab).	Non-standard
charset	encoding	Obsolete. Character set of the resource.	HTML 4.01
rev	link types	Obsolete. Reverse relationship.	HTML 4.01
target	browsing context	Obsolete on link.	HTML 4.01

<link>: attribute by attribute

Each attribute of <link> explained in detail, with a minimal example you can copy.

rel

Values: space-separated link types | Since: HTML 2.0

Required. Relationship: stylesheet, icon, apple-touch-icon, manifest, canonical, alternate, preload, prefetch, preconnect, dns-prefetch, modulepreload, prerender, author, license, help, next, prev, search, pingback, privacy-policy, terms-of-service, expect.

Write rel="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link rel="value">
```

href

Values: URL | Since: HTML 2.0

The resource URL.

Write href="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link href="https://example.com/page.html">
```

type

Values: MIME type | Since: HTML 2.0

The type of the resource, for example text/css, image/png, application/rss+xml.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link type="text/plain">
```

media

Values: media query | Since: HTML 4.01

Apply the style sheet only when the query matches: screen, print, (max-width: 600px), (prefers-color-scheme: dark).

Write media="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link media="media query">
```

as

Values: audio | document | embed | fetch | font | image | object | script | style | track | video | worker | Since: Living Standard

With rel=preload or modulepreload: the type of content being loaded, so the browser can prioritise it correctly.

Write as="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link as="audio">
```

crossorigin

Values: anonymous | use-credentials | Since: HTML5

CORS mode for the request. Required when preloading fonts.

Write crossorigin="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link crossorigin="anonymous">
```

hreflang

Values: language tag | Since: HTML 4.01

Language of the linked resource (with rel=alternate).

Write hreflang="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link hreflang="en">
```

sizes

Values: any or WxH list | Since: HTML5

Icon sizes, for example 16x16 32x32 or any (for SVG).

Write sizes="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link sizes="any">
```

title

Values: text | Since: HTML 2.0

For style sheets: the name of an alternate style sheet set. For feeds: the feed title.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link title="value">
```

integrity

Values: hash | Since: SRI spec

Subresource Integrity hash; the browser refuses the file if it does not match.

Write integrity="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link integrity="hash">
```

referrerpolicy

Values: referrer policy | Since: HTML5

Referrer to send when fetching.

Write referrerpolicy="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link referrerpolicy="referrer policy">
```

fetchpriority

Values: high | low | auto | Since: Living Standard

Download priority hint.

Write `fetchpriority="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link fetchpriority="high">
```

blocking

Values: render | Since: Living Standard

Block rendering until the resource is loaded (for stylesheets/preloads/expect).

Write `blocking="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link blocking="render">
```

disabled

Values: boolean | Since: HTML5

For style sheets: do not apply.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<link disabled>
```

imagesrcset / imagesizes

Values: srcset / sizes syntax | Since: Living Standard

With `rel=preload as=image`: responsive image candidates.

Write `imagesrcset="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link imagesrcset="srcset">
```


color

Values: colour | Since: Non-standard

With rel=mask-icon (Safari pinned tab).

Write color="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link color="#336699">
```

charset

Values: encoding | Since: HTML 4.01

Obsolete. Character set of the resource.

Write charset="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link charset="encoding">
```

rev

Values: link types | Since: HTML 4.01

Obsolete. Reverse relationship.

Write rev="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link rev="link types">
```

target

Values: browsing context | Since: HTML 4.01

Obsolete on link.

Write target="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<link target="browsing context">
```

<link>: worked examples

Style sheets

```
<link rel="stylesheet" href="style.css">
<link rel="stylesheet" href="print.css" media="print">
<link rel="stylesheet" href="dark.css" media="(prefers-color-scheme:
dark)">
```

Icons

```
<link rel="icon" href="favicon.ico" sizes="32x32">
<link rel="icon" href="icon.svg" type="image/svg+xml">
<link rel="apple-touch-icon" href="apple-touch-icon.png">
<link rel="manifest" href="manifest.webmanifest">
```

Performance hints

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="preload" href="hero.jpg" as="image" fetchpriority="high">
<link rel="preload" href="font.woff2" as="font" type="font/woff2"
crossorigin>
<link rel="prefetch" href="next-page.html">
```

SEO and alternates

```
<link rel="canonical" href="https://example.com/books/html">
<link rel="alternate" hreflang="ar" href="https://example.com/ar/
books/html">
<link rel="alternate" type="application/rss+xml" title="Blog feed"
href="/feed.xml">
```

Google Fonts

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Cairo:wght@400;
700&display=swap" rel="stylesheet">
```

Default browser CSS

```
link { display: none; }
```

Accessibility

- Alternate style sheets (rel="alternate stylesheet" title="High contrast") can offer accessible themes in browsers that support switching.

Common mistakes

- Forgetting rel.

- Using `<link>` to make a hyperlink (that is `<a>`).
- Wrong path to the CSS file (check the browser's network tab).
- Writing `</link>`.

See also

`<style>`, `<a>`, `<script>`, `<meta>`, `<head>`

Quick card: <listing>

Listing (obsolete)

The <listing> element, from the 1991 tag set, displayed its content in a monospace font exactly as written, like <pre> but originally at a smaller size and without interpreting markup. It was deprecated already in HTML 2.0 and removed in HTML 3.2. Use <pre>, optionally with <code>.

At a glance

Status	OBSOLETE
Versions	HTML 1, HTML 2.0 (deprecated); obsolete
Category	Obsolete text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<pre>, <xmp>, <plaintext>

Minimal example

```
<pre><code>line 1  
line 2</code></pre>
```

Most common mistake

Any use.

<listing> — Listing (obsolete)

Category: Obsolete text content | Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

Description

The <listing> element, from the 1991 tag set, displayed its content in a monospace font exactly as written, like <pre> but originally at a smaller size and without interpreting markup. It was deprecated already in HTML 2.0 and removed in HTML 3.2. Use <pre>, optionally with <code>.

Syntax

```
<listing>text</listing>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Historical
Permitted content	Text
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLPreElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<listing>: worked examples

Replacement

```
<pre><code>line 1  
line 2</code></pre>
```

Default browser CSS

```
listing { display: block; font-family: monospace; white-space: pre;  
margin: 1em 0; }
```

Common mistakes

- Any use.

See also

<pre>, <xmp>, <plaintext>

Quick card: <main>

Main content

The <main> element represents the dominant content of the <body>: the content that is directly related to, or expands upon, the central topic of the page or the central function of the application. It excludes content repeated across pages such as the site header, navigation, sidebars, search box, logo and footer.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 5.1 (2016) |
| Category | Content sectioning |
| Closing tag | Required unless noted under Tag omission |
| Display | block |
| Own attributes | None (global attributes only) |
| Related | <header>, <footer>, <nav>, <aside>, <article>, <section> |

Minimal example

```
<body>
  <a href="#content" class="skip">Skip to main content</a>
  <header>...</header>
  <nav>...</nav>
  <main id="content">
    <h1>Page title</h1>
    <p>The unique content of this page.</p>
  </main>
  <aside>...</aside>
  <footer>...</footer>
</body>
```

Most common mistake

Two visible mains.

<main> — Main content

Category: Content sectioning | Status: CURRENT | Versions: HTML 5.1 (2016)

Description

The `<main>` element represents the dominant content of the `<body>`: the content that is directly related to, or expands upon, the central topic of the page or the central function of the application. It excludes content repeated across pages such as the site header, navigation, sidebars, search box, logo and footer.

A document must not have more than one visible `<main>`; additional `<main>` elements are allowed only if all but one carry the `hidden` attribute. `<main>` must not be a descendant of `<article>`, `<aside>`, `<footer>`, `<header>` or `<nav>`. It is exposed as the 'main' landmark, which lets screen reader users skip straight to the content, and it is the natural target for a 'skip to content' link.

Syntax

```
<main>...</main>
```

Technical summary

| Property | Value |
|--------------------|--------------------------------|
| Void element | No |
| Default display | block |
| Content categories | Flow content, palpable content |
| Permitted content | Flow content |

| Property | Value |
|-------------------|--|
| Permitted parents | <html>, <body>, <div>, <form> without an accessible name, and autonomous custom elements; not inside article, aside, footer, header or nav |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLElement |

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<main>: worked examples

Page structure with skip link

```
<body>
  <a href="#content" class="skip">Skip to main content</a>
  <header>...</header>
  <nav>...</nav>
  <main id="content">
    <h1>Page title</h1>
    <p>The unique content of this page.</p>
  </main>
  <aside>...</aside>
  <footer>...</footer>
</body>
```

Visually hidden skip link that appears on focus

```
<style>
  .skip { position: absolute; left: -999px; }
  .skip:focus { left: 8px; top: 8px; background: #fff; padding: 8px; }
</style>
```

Single-page app views

```
<main id="home">Home view</main>
<main id="cart" hidden>Cart view</main>
```

Only one is visible at a time.

Default browser CSS

```
main { display: block; }
```

Accessibility

- The 'main' landmark is one of the most used shortcuts in screen readers; every page should have exactly one.

Common mistakes

- Two visible mains.
- Placing main inside article or nav.
- Putting the header and footer inside main.

See also

<header>, <footer>, <nav>, <aside>, <article>, <section>

Quick card: <map>

Image map

The <map> element, together with <area> elements, defines a client-side image map: an image with several clickable regions each linking to a different destination. The map is connected to an through the img's usemap attribute, which must be # followed by the map's name.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Image and multimedia
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	name
Related	<area>,

Minimal example

```

<map name="banner">
  <area shape="rect" coords="0,0,300,100" href="books.html"
alt="Books">
  <area shape="rect" coords="300,0,600,100" href="deals.html"
alt="Deals">
</map>
```

Most common mistake

usemap without #.

<map> — Image map

Category: Image and multimedia | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The <map> element, together with <area> elements, defines a client-side image map: an image with several clickable regions each linking to a different destination. The map is connected to an through the img's usemap attribute, which must be # followed by the map's name.

Syntax

```
<map name="n">  
  <area ...>  
</map>  
<img usemap="#n" ...>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Transparent
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLMapElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
name	text	Required. The map's name, without #, unique among maps. If id is also given it must have the same value.	HTML 3.2

<map>: attribute by attribute

Each attribute of <map> explained in detail, with a minimal example you can copy.

name

Values: text | Since: HTML 3.2

Required. The map's name, without #, unique among maps. If id is also given it must have the same value.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<map name="value">...</map>
```

<map>: worked examples

Navigation image with two regions

```

<map name="banner">
  <area shape="rect" coords="0,0,300,100" href="books.html"
alt="Books">
  <area shape="rect" coords="300,0,600,100" href="deals.html"
alt="Deals">
</map>
```

Default browser CSS

```
map { display: inline; }
```

Accessibility

- Each area needs alt text. Provide an equivalent text list of links if the map is complex.

Common mistakes

- usemap without #.
- Map name mismatching.
- Coordinates for a different image size than displayed.

See also

<area>,

Quick card: <mark>

Highlighted (marked) text

The <mark> element represents text that is marked or highlighted for reference because of its relevance in the current context. Typical uses: highlighting the search terms in a page of results, or drawing attention to a passage in a quotation that the current text discusses. Browsers render it with a yellow background, like a highlighter pen.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	, ,

Minimal example

```
<p>Results for 'footer': ... the <mark>footer</mark> element  
represents the <mark>footer</mark> of its section ...</p>
```

Most common mistake

Using <mark> for decorative colour.

<mark> — Highlighted (marked) text

Category: Text-level semantics | Status: CURRENT | Versions: HTML5

Description

The `<mark>` element represents text that is marked or highlighted for reference because of its relevance in the current context. Typical uses: highlighting the search terms in a page of results, or drawing attention to a passage in a quotation that the current text discusses. Browsers render it with a yellow background, like a highlighter pen.

Do not confuse it with `<strong>` (importance) or `<em>` (emphasis). `<mark>` is about relevance to the reader's current activity, not about the author's intent.

Syntax

```
<mark>text</mark>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<mark>: worked examples

Search results

```
<p>Results for 'footer': ... the <mark>footer</mark> element  
represents the <mark>footer</mark> of its section ...</p>
```

Highlighting part of a quotation

```
<blockquote><p>Any tag you learn is <mark>a tool you keep</mark>.</p></blockquote>  
<p>The phrase 'a tool you keep' is the key idea.</p>
```

Custom colour

```
<style>  
  mark { background: #ffe58a; color: inherit; padding: 0 2px; }  
</style>
```

Default browser CSS

```
mark { background-color: yellow; color: black; }
```

Accessibility

- Not announced by default; if the highlight matters, add visually hidden text such as '[highlight start]'.

Common mistakes

- Using <mark> for decorative colour.
- Using <mark> instead of .

See also

, ,

Quick card: <marquee>

Scrolling text (obsolete)

The <marquee> element scrolled its content horizontally or vertically across the screen. It was a Microsoft extension that became infamous alongside Netscape's <blink>. HTML5 lists it as obsolete but, because so many old pages use it, browsers still implement it. Moving text is hard to read and distracting; if you need an animation, use CSS animations with a pause control and respect the prefers-reduced-motion media query.

At a glance

Status	OBSOLETE
Versions	Internet Explorer 2 (1995), never standard; obsolete in HTML5 (but its behaviour is specified for compatibility)
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	behavior, direction, loop, scrollamount, scrolldelay, truespeed, bgcolor, width, hspace
Related	<blink>

Minimal example

```
<style>
  @keyframes slide { from { transform: translateX(100%); } to {
transform: translateX(-100%); } }
  .ticker { white-space: nowrap; overflow: hidden; }
  .ticker span { display: inline-block; animation: slide 12s linear
infinite; }
  @media (prefers-reduced-motion: reduce) { .ticker span { animation:
none; } }
</style>
<div class="ticker"><span>Breaking news...</span></div>
```

Most common mistake

Any use.

<marquee> — Scrolling text (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: Internet Explorer 2 (1995), never standard; obsolete in HTML5 (but its behaviour is specified for compatibility)

Description

The <marquee> element scrolled its content horizontally or vertically across the screen. It was a Microsoft extension that became infamous alongside Netscape's <blink>. HTML5 lists it as obsolete but, because so many old pages use it, browsers still implement it. Moving text is hard to read and distracting; if you need an animation, use CSS animations with a pause control and respect the prefers-reduced-motion media query.

Syntax

```
<marquee>text</marquee>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Historical
Permitted content	Flow content
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLMarqueeElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
behavior	scroll slide alternate	Scrolling style.	IE
direction	left right up down	Scroll direction.	IE
loop	integer -1	Repeat count.	IE
scrollamount	pixels	Distance per step.	IE
scrolldelay	ms	Delay between steps.	IE
truespeed	boolean	Allow delays under 60 ms.	IE
bgcolor	colour	Background.	IE
width / height	size	Size.	IE
hspace / vspace	pixels	Margins.	IE

<marquee>: attribute by attribute

Each attribute of <marquee> explained in detail, with a minimal example you can copy.

behavior

Values: scroll | slide | alternate | Since: IE

Scrolling style.

Write behavior="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee behavior="scroll">...</marquee>
```

direction

Values: left | right | up | down | Since: IE

Scroll direction.

Write direction="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee direction="left">...</marquee>
```

loop

Values: integer | -1 | Since: IE

Repeat count.

Write loop="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee loop="3">...</marquee>
```

scrollamount

Values: pixels | Since: IE

Distance per step.

Write scrollamount="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee scrollamount="3">...</marquee>
```

scrolldelay

Values: ms | Since: IE

Delay between steps.

Write scrolldelay="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee scrolldelay="ms">...</marquee>
```

truespeed

Values: boolean | Since: IE

Allow delays under 60 ms.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<marquee truespeed>...</marquee>
```

bgcolor

Values: colour | Since: IE

Background.

Write bgcolor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee bgcolor="#336699">...</marquee>
```

width / height

Values: size | Since: IE

Size.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee width="size">...</marquee>
```

hspace / vspace

Values: pixels | Since: IE

Margins.

Write hspace="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<marquee hspace="3">...</marquee>
```

<marquee>: worked examples

CSS replacement

```
<style>
  @keyframes slide { from { transform: translateX(100%); } to {
transform: translateX(-100%); } }
  .ticker { white-space: nowrap; overflow: hidden; }
  .ticker span { display: inline-block; animation: slide 12s linear
infinite; }
  @media (prefers-reduced-motion: reduce) { .ticker span { animation:
none; } }
</style>
<div class="ticker"><span>Breaking news...</span></div>
```

Default browser CSS

```
marquee { display: inline-block; text-align: initial; }
```

Accessibility

- Moving text violates WCAG unless it can be paused.

Common mistakes

- Any use.

See also

<blink>

Quick card: <math>

MathML root

The <math> element is the top-level element of MathML (Mathematical Markup Language), which can be written directly inside HTML5 documents to display mathematical formulas. Inside <math> you use MathML elements such as <mi> (identifier), <mn> (number), <mo> (operator), <mrow> (group), <mfrac> (fraction), <msup> (superscript), <msqrt> (square root) and <mtable>. All modern browsers support MathML Core.

At a glance

Status	CURRENT
Versions	HTML5 (inline MathML)
Category	Embedded content (MathML)
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	display, xmlns
Related	<svg>, <canvas>

Minimal example

```
<math display="block">
  <mi>x</mi><mo>=</mo>
  <mfrac>
    <mrow><mo>-</mo><mi>b</mi><mo>&#xB1;</mo><msqrt><msup><mi>b</
mi><mn>2</mn></msup><mo>-</mo><mn>4</mn><mi>a</mi><mi>c</mi></msqrt></
mrow>
    <mrow><mn>2</mn><mi>a</mi></mrow>
  </mfrac>
</math>
```

Most common mistake

Writing formulas as plain text or images.

<math> — MathML root

Category: Embedded content (MathML) | Status: CURRENT | Versions: HTML5 (inline MathML)

Description

The `<math>` element is the top-level element of MathML (Mathematical Markup Language), which can be written directly inside HTML5 documents to display mathematical formulas. Inside `<math>` you use MathML elements such as `<mi>` (identifier), `<mn>` (number), `<mo>` (operator), `<mrow>` (group), `<mfrac>` (fraction), `<msup>` (superscript), `<msqrt>` (square root) and `<mtable>`. All modern browsers support MathML Core.

Syntax

```
<math>
  <mi>x</mi><mo>=</mo><mn>2</mn>
</math>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content, palpable content
Permitted content	MathML elements
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither
DOM interface	MathMLElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
display	block inline	Whether the formula is a block on its own line or inline in text.	MathML
xmlns	http://www.w3.org/1998/Math/MathML	Namespace; optional in HTML.	MathML

<math>: attribute by attribute

Each attribute of <math> explained in detail, with a minimal example you can copy.

display

Values: block | inline | Since: MathML

Whether the formula is a block on its own line or inline in text.

Write display="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<math display="block">...</math>
```

xmlns

Values: <http://www.w3.org/1998/Math/MathML> | Since: MathML

Namespace; optional in HTML.

Write xmlns="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<math xmlns="http:">...</math>
```


<math>: worked examples

Quadratic formula

```
<math display="block">
  <mi>x</mi><mo>=</mo>
  <mfrac>
    <mrow><mo>-</mo><mi>b</mi><mo>±</mo><msqrt><msup><mi>b</
mi><mn>2</mn></msup><mo>-</mo><mn>4</mn><mi>a</mi><mi>c</mi></msqrt></
mrow>
    <mrow><mn>2</mn><mi>a</mi></mrow>
  </mfrac>
</math>
```

Inline formula

```
<p>The area of a circle is <math><mi>π</mi><msup><mi>r</
mi><mn>2</mn></msup></math>.</p>
```

Default browser CSS

```
math { display: inline; math-style: compact; }
math[display=block] { display: block math; math-style: normal; }
```

Accessibility

- MathML is read by modern screen readers as mathematics; images of formulas are not.

Common mistakes

- Writing formulas as plain text or images.
- Forgetting `display="block"` for standalone equations.

See also

<svg>, <canvas>

Quick card: <menu>

Menu (toolbar list)

The <menu> element is a semantic alternative to for a list of commands or actions, such as a toolbar of buttons. Each command is an containing a <button> or similar control. Browsers render it identically to .

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	compact, type, label
Related	, , <button>, <nav>, <menuitem>

Minimal example

```
<menu>
  <li><button onclick="copy()">Copy</button></li>
  <li><button onclick="cut()">Cut</button></li>
  <li><button onclick="paste()">Paste</button></li>
</menu>
```

Most common mistake

Expecting native context menus from type="context".

<menu> — Menu (toolbar list)

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

Description

The `<menu>` element is a semantic alternative to `<ul>` for a list of commands or actions, such as a toolbar of buttons. Each command is an `<li>` containing a `<button>` or similar control. Browsers render it identically to `<ul>`.

Its history is long: in HTML 1 and 2 it was a compact list like `<dir>`; HTML 4.01 deprecated it; early HTML5 drafts turned it into a rich context-menu system with `type="context"` and `<menuitem>` children, which no browser except Firefox implemented and which was removed; the current definition is simply 'an unordered list of interactive items'.

Syntax

```
<menu>
  <li><button>Action</button></li>
</menu>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content; palpable if it has at least one <code></code>
Permitted content	Zero or more <code></code> , <code><script></code> , <code><template></code>
Permitted parents	Any element that accepts flow content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLMenuElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
compact	boolean	Obsolete. Compact rendering.	HTML 2.0
type	context toolbar list	Obsolete. From the abandoned HTML5 context menu design.	HTML5 draft (removed)
label	text	Obsolete. Menu label (context menus).	HTML5 draft (removed)

<menu>: attribute by attribute

Each attribute of <menu> explained in detail, with a minimal example you can copy.

compact

Values: boolean | Since: HTML 2.0

Obsolete. Compact rendering.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<menu compact>...</menu>
```

type

Values: context | toolbar | list | Since: HTML5 draft (removed)

Obsolete. From the abandoned HTML5 context menu design.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menu type="context">...</menu>
```

label

Values: text | Since: HTML5 draft (removed)

Obsolete. Menu label (context menus).

Write label="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menu label="value">...</menu>
```

<menu>: worked examples

Editor toolbar

```
<menu>
  <li><button onclick="copy()">Copy</button></li>
  <li><button onclick="cut()">Cut</button></li>
  <li><button onclick="paste()">Paste</button></li>
</menu>
```

Horizontal toolbar styling

```
<style>
  menu { display: flex; gap: 8px; list-style: none; padding: 0;
margin: 0; }
</style>
```

Default browser CSS

```
menu { display: block; list-style-type: disc; margin: 1em 0; padding-
inline-start: 40px; }
```

Accessibility

- Exposed as a list, the same as . Add role="toolbar" if the list is a real toolbar with arrow-key navigation.

Common mistakes

- Expecting native context menus from type="context".
- Using <menu> for navigation links (use <nav>).

See also

, , <button>, <nav>, <menuitem>

Quick card: <menuitem>

Menu item (obsolete)

The <menuitem> element represented a command in a <menu type="context">, letting a page add custom items to the browser's right-click menu. Only Firefox implemented it; it was removed from the standard in 2017 and from Firefox in 2020. Build custom menus with lists, buttons and JavaScript (or the popover attribute) instead.

At a glance

Status	OBSOLETE
Versions	HTML5 draft (Firefox only); removed 2017
Category	Obsolete interactive
Closing tag	None (void element)
Display	none
Own attributes	label, type, icon, checked
Related	<menu>, <button>

Minimal example

```
<button popovertarget="m">Options</button>
<menu id="m" popover>
  <li><button>Refresh</button></li>
</menu>
```

Most common mistake

Any use.

<menuitem> — Menu item (obsolete)

Category: Obsolete interactive | Status: OBSOLETE | Versions: HTML5 draft (Firefox only); removed 2017

Description

The `<menuitem>` element represented a command in a `<menu type="context">`, letting a page add custom items to the browser's right-click menu. Only Firefox implemented it; it was removed from the standard in 2017 and from Firefox in 2020. Build custom menus with lists, buttons and JavaScript (or the popover attribute) instead.

Syntax

```
<menuitem label="Refresh" onclick="...">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Historical
Permitted content	None
Permitted parents	<code><menu></code>
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
label	text	Item text.	HTML5 draft
type	command checkbox radio	Kind of item.	HTML5 draft
icon	URL	Icon image.	HTML5 draft
checked, disabled, default, radiogroup, command	various	State and grouping.	HTML5 draft

<menuitem>: attribute by attribute

Each attribute of <menuitem> explained in detail, with a minimal example you can copy.

label

Values: text | Since: HTML5 draft

Item text.

Write label="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menuitem label="value">
```

type

Values: command | checkbox | radio | Since: HTML5 draft

Kind of item.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menuitem type="command">
```

icon

Values: URL | Since: HTML5 draft

Icon image.

Write icon="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menuitem icon="https://example.com/page.html">
```

checked, disabled, default, radiogroup, command

Values: various | Since: HTML5 draft

State and grouping.

Write checked="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<menuitem checked="various">
```

<menuitem>: worked examples

Replacement

```
<button popovertarget="m">Options</button>
<menu id="m" popover>
  <li><button>Refresh</button></li>
</menu>
```

Common mistakes

- Any use.

See also

<menu>, <button>

Quick card: <meta>

Metadata

The <meta> element represents metadata about the document that cannot be expressed by <title>, <base>, <link>, <style> or <script>. It is a void element placed in <head>. Depending on which attributes are present it does one of four things: declares the character encoding (charset); provides document-level metadata as a name/content pair (name); simulates an HTTP response header (http-equiv); or supplies microdata (itemprop).

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Document metadata
Closing tag	None (void element)
Display	none
Own attributes	charset, name, content, http-equiv, media, property, itemprop, scheme
Related	<head>, <title>, <link>, <base>

Minimal example

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Most common mistake

Placing charset after other content or after 1024 bytes.

<meta> — Metadata

Category: Document metadata | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<meta>` element represents metadata about the document that cannot be expressed by `<title>`, `<base>`, `<link>`, `<style>` or `<script>`. It is a void element placed in `<head>`. Depending on which attributes are present it does one of four things: declares the character encoding (`charset`); provides document-level metadata as a name/content pair (`name`); simulates an HTTP response header (`http-equiv`); or supplies microdata (`itemprop`).

The most important meta tags on every page are `charset` and `viewport`. Other frequently used names are `description` (shown in search results), `robots`, `author`, `theme-color` and the Open Graph (`property="og:..."`) and Twitter card tags used when a page is shared on social networks. Part 9 contains the full list of standard meta names.

Syntax

```
<meta charset="utf-8">
<meta name="description" content="...">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Metadata content; flow and phrasing content if <code>itemprop</code> is present

Property	Value
Permitted content	None (void element)
Permitted parents	<head>; <noscript> in head; anywhere if itemprop is present
Tag omission	No end tag
DOM interface	HTMLMetaElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
charset	utf-8	Declares the document's character encoding. The only valid value in HTML5 is utf-8 (case-insensitive). Must be in the first 1024 bytes.	HTML5
name	metadata name	The kind of metadata: application-name, author, description, generator, keywords, referrer, theme-color, color-scheme, viewport, robots, googlebot, creator, publisher, and more.	HTML 2.0
content	text	The value for name or http-equiv. Required with them.	HTML 2.0
http-equiv	content-type default-style refresh content-security-policy x-ua-compatible	Pragma directive that acts like an HTTP header.	HTML 2.0
media	media query	With name=theme-color: apply only when the query matches.	Living Standard

Attribute	Values	Description	Since
property	text	Not in the HTML standard but universally used for RDFa / Open Graph tags such as og:title.	RDFa
itemprop	text	Microdata property name.	HTML5 microdata
scheme	text	Obsolete. Interpretation scheme for content.	HTML 4.01

<meta>: attribute by attribute

Each attribute of <meta> explained in detail, with a minimal example you can copy.

charset

Values: utf-8 | Since: HTML5

Declares the document's character encoding. The only valid value in HTML5 is utf-8 (case-insensitive). Must be in the first 1024 bytes.

Write charset="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta charset="utf-8">
```

name

Values: metadata name | Since: HTML 2.0

The kind of metadata: application-name, author, description, generator, keywords, referrer, theme-color, color-scheme, viewport, robots, googlebot, creator, publisher, and more.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta name="metadata name">
```

content

Values: text | Since: HTML 2.0

The value for name or http-equiv. Required with them.

Write content="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta content="value">
```


http-equiv

Values: content-type | default-style | refresh | content-security-policy | x-ua-compatible | Since: HTML 2.0

Pragma directive that acts like an HTTP header.

Write http-equiv="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta http-equiv="content-type">
```

media

Values: media query | Since: Living Standard

With name=theme-color: apply only when the query matches.

Write media="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta media="media query">
```

property

Values: text | Since: RDFa

Not in the HTML standard but universally used for RDFa / Open Graph tags such as og:title.

Write property="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta property="value">
```

itemprop

Values: text | Since: HTML5 microdata

Microdata property name.

Write itemprop="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta itemprop="value">
```

scheme

Values: text | Since: HTML 4.01

Obsolete. Interpretation scheme for content.

Write `scheme="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meta scheme="value">
```

<meta>: worked examples

The essential pair

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Without the viewport tag, phones render the page as if it were 980px wide and shrink it.

Search engine metadata

```
<meta name="description" content="Buy PDF books in Arabic and English;
  pay with mobile wallet and download instantly.">
<meta name="robots" content="index, follow">
<meta name="author" content="Book Store team">
```

Social sharing (Open Graph and Twitter)

```
<meta property="og:title" content="Learn HTML">
<meta property="og:description" content="A complete reference.">
<meta property="og:image" content="https://example.com/cover.jpg">
<meta property="og:url" content="https://example.com/html">
<meta name="twitter:card" content="summary_large_image">
```

Theme colour and colour scheme

```
<meta name="theme-color" content="#e60000">
<meta name="theme-color" content="#111" media="(prefers-color-scheme:
dark)">
<meta name="color-scheme" content="light dark">
```

Redirect after 5 seconds (use sparingly)

```
<meta http-equiv="refresh" content="5; url=https://example.com/new-
page">
```

Content Security Policy

```
<meta http-equiv="content-security-policy" content="default-src
'self'; img-src https:">
```

Old HTML 4 encoding declaration

```
<!-- HTML 4.01 -->
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">

<!-- HTML5 -->
<meta charset="utf-8">
```

Default browser CSS

```
meta { display: none; }
```

Accessibility

- Never use `<meta name="viewport" content="user-scalable=no">` or `maximum-scale=1`; it stops users zooming and fails WCAG.
- Automatic refresh/redirect disorients users; prefer server-side redirects.

Common mistakes

- Placing `charset` after other content or after 1024 bytes.
- Using keywords meta (ignored by search engines since 2009).
- Writing `</meta>`.

See also

`<head>`, `<title>`, `<link>`, `<base>`

Quick card: <meter>

Scalar measurement (gauge)

The <meter> element represents a scalar measurement within a known range, or a fractional value: disk usage, the relevance of a search result, password strength, a score out of ten, temperature within safe limits. Browsers render it as a gauge bar whose colour changes depending on whether the value is in the optimum, suboptimal or bad region defined by low, high and optimum.

At a glance

Status	CURRENT
Versions	HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	value, min, max, low, high, optimum, form
Related	<progress>, <output>, <input>

Minimal example

```
<label for="disk">Disk usage</label>
<meter id="disk" value="70" min="0" max="100" low="60" high="85"
optimum="20">70%</meter>
```

Most common mistake

Using it as a progress bar.

<meter> — Scalar measurement (gauge)

Category: Forms | Status: CURRENT | Versions: HTML5

Description

The `<meter>` element represents a scalar measurement within a known range, or a fractional value: disk usage, the relevance of a search result, password strength, a score out of ten, temperature within safe limits. Browsers render it as a gauge bar whose colour changes depending on whether the value is in the optimum, suboptimal or bad region defined by low, high and optimum.

Do not use `<meter>` for progress of a task; that is what `<progress>` is for. And do not use it for values with no known maximum (such as a bank balance) because the gauge needs a range.

Syntax

```
<meter value="0.6">60%</meter>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Flow content, phrasing content, labelable, palpable content
Permitted content	Phrasing content, but no <code><meter></code> descendants
Permitted parents	Any element that accepts phrasing content

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLMeterElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
value	number	Required. The current value, between min and max.	HTML5
min	number	Lower bound of the range. Default 0.	HTML5
max	number	Upper bound. Default 1.	HTML5
low	number	Upper bound of the 'low' region.	HTML5
high	number	Lower bound of the 'high' region.	HTML5
optimum	number	The ideal value. Determines which region is shown as good (green) versus bad (red/yellow).	HTML5
form	form id	Associated form (for labelling purposes).	HTML5

<meter>: attribute by attribute

Each attribute of <meter> explained in detail, with a minimal example you can copy.

value

Values: number | Since: HTML5

Required. The current value, between min and max.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter value="3">...</meter>
```

min

Values: number | Since: HTML5

Lower bound of the range. Default 0.

Write min="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter min="3">...</meter>
```

max

Values: number | Since: HTML5

Upper bound. Default 1.

Write max="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter max="3">...</meter>
```

low

Values: number | Since: HTML5

Upper bound of the 'low' region.

Write low="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter low="3">...</meter>
```


high

Values: number | Since: HTML5

Lower bound of the 'high' region.

Write `high="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter high="3">...</meter>
```

optimum

Values: number | Since: HTML5

The ideal value. Determines which region is shown as good (green) versus bad (red/yellow).

Write `optimum="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter optimum="3">...</meter>
```

form

Values: form id | Since: HTML5

Associated form (for labelling purposes).

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<meter form="form id">...</meter>
```

<meter>: worked examples

Disk usage

```
<label for="disk">Disk usage</label>
<meter id="disk" value="70" min="0" max="100" low="60" high="85"
optimum="20">70%</meter>
```

Because optimum is low, 70 (above low) shows in yellow and anything above 85 in red.

Score out of ten

```
<p>Rating: <meter value="8" min="0" max="10">8 out of 10</meter></p>
```

Fraction

```
<meter value="0.25">25%</meter>
```

Default range 0 to 1.

Default browser CSS

```
meter { display: inline-block; appearance: auto; }
```

Accessibility

- Provide the value as text inside the element or in a label; the bar alone is not enough.
- Always label the meter.

Common mistakes

- Using it as a progress bar.
- Omitting value.
- Value outside min-max (it is clamped).

See also

<progress>, <output>, <input>

Quick card: <multicol>

Multi-column text (non-standard, obsolete)

The <multicol> element split its content into newspaper-style columns. It existed only in Netscape Navigator 3 and 4 and was never standardised. Use the CSS multi-column properties (column-count, column-width, column-gap) instead.

At a glance

Status	OBSOLETE
Versions	Netscape 3 only; never standard
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	cols, gutter, width
Related	<div>

Minimal example

```
<div style="column-count: 3; column-gap: 20px">Long text...</div>
```

Most common mistake

Any use.

<multicol> — Multi-column text (non-standard, obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: Netscape 3 only; never standard

Description

The <multicol> element split its content into newspaper-style columns. It existed only in Netscape Navigator 3 and 4 and was never standardised. Use the CSS multi-column properties (column-count, column-width, column-gap) instead.

Syntax

```
<multicol cols="3" gutter="20">text</multicol>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Historical
Permitted content	Flow content
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
cols	integer	Number of columns.	Netscape
gutter	pixels	Space between columns.	Netscape
width	pixels	Column width.	Netscape

<multicol>: attribute by attribute

Each attribute of <multicol> explained in detail, with a minimal example you can copy.

cols

Values: integer | Since: Netscape

Number of columns.

Write cols="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<multicol cols="3">...</multicol>
```

gutter

Values: pixels | Since: Netscape

Space between columns.

Write gutter="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<multicol gutter="3">...</multicol>
```

width

Values: pixels | Since: Netscape

Column width.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<multicol width="3">...</multicol>
```

<multicol>: worked examples

CSS replacement

```
<div style="column-count: 3; column-gap: 20px">Long text...</div>
```

Common mistakes

- Any use.

See also

<div>

Quick card: <nav>

Navigation section

The <nav> element represents a section of the page whose purpose is to provide navigation links, either within the current document or to other documents. Typical uses are the main site menu, a table of contents, breadcrumbs, pagination and 'previous/next' links.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<header>, <footer>, <a>, , <menu>, <main>

Minimal example

```
<nav aria-label="Main">
  <ul>
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/books">Books</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>
```

Most common mistake

Wrapping every link group in <nav>.

<nav> — Navigation section

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The `<nav>` element represents a section of the page whose purpose is to provide navigation links, either within the current document or to other documents. Typical uses are the main site menu, a table of contents, breadcrumbs, pagination and 'previous/next' links.

Not every group of links needs a `<nav>`; it is intended for major navigation blocks. The list of links in a page footer, for example, is usually fine as a plain list, although a `<nav>` there is also acceptable. A page can have several `<nav>` elements; give each an `aria-label` so screen reader users can tell them apart. Links inside a `<nav>` are conventionally wrapped in a `<ul>`.

Syntax

```
<nav>
  <ul>
    <li><a href="...">Link</a></li>
  </ul>
</nav>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning content, palpable content
Permitted content	Flow content

Property	Value
Permitted parents	Any element that accepts flow content, but not <address>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<nav>: worked examples

Main site navigation

```
<nav aria-label="Main">
  <ul>
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/books">Books</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>
```

Breadcrumbs

```
<nav aria-label="Breadcrumb">
  <ol>
    <li><a href="/">Home</a></li>
    <li><a href="/books">Books</a></li>
    <li aria-current="page">Learn HTML</li>
  </ol>
</nav>
```

Horizontal menu CSS

```
<style>
  nav ul { display: flex; gap: 24px; list-style: none; margin: 0;
padding: 0; }
  nav a { text-decoration: none; color: #1b2431; font-weight: 600; }
  nav a[aria-current="page"] { border-bottom: 2px solid #e60000; }
</style>
```

Table of contents

```
<nav aria-label="On this page">
  <h2>Contents</h2>
  <ol>
    <li><a href="#intro">Introduction</a></li>
    <li><a href="#tags">Tags</a></li>
  </ol>
</nav>
```

Default browser CSS

```
nav { display: block; }
```

Accessibility

- Exposed as the 'navigation' landmark.
- Label multiple navs with aria-label.

- Mark the current page with `aria-current="page"`.
- Provide a skip link so keyboard users can bypass long menus.

Common mistakes

- Wrapping every link group in `<nav>`.
- Putting non-navigation content inside.
- Not using a list for the links.

See also

`<header>`, `<footer>`, `<a>`, `<ul>`, `<menu>`, `<main>`

Quick card: <nextid>

Next identifier (HTML 1 editor hint)

The <nextid> element was used by the very first HTML editors (the NeXT-based editor written at CERN) to remember the next automatically generated anchor name, so that the editor could continue numbering anchors (z1, z2, z3...) when a document was reopened. It carried a single attribute, *n*, holding that name. It had no visible effect and was placed in the <head>.

At a glance

Status	OBSOLETE
Versions	HTML 1 / HTML 2.0 (RFC 1866); removed in HTML 3.2
Category	Obsolete
Closing tag	None (void element)
Display	none
Own attributes	<i>n</i>
Related	<head>, <isindex>, <listing>

Minimal example

```
<head>
  <title>Welcome</title>
  <nextid n="z5">
</head>
```

Most common mistake

Any use in a new document.

<nextid> — Next identifier (HTML 1 editor hint)

Category: Obsolete | Status: OBSOLETE | Versions: HTML 1 / HTML 2.0 (RFC 1866); removed in HTML 3.2

Description

The <nextid> element was used by the very first HTML editors (the NeXT-based editor written at CERN) to remember the next automatically generated anchor name, so that the editor could continue numbering anchors (z1, z2, z3...) when a document was reopened. It carried a single attribute, *n*, holding that name. It had no visible effect and was placed in the <head>.

It was already marked as an editor-only feature in HTML 2.0 and was dropped from HTML 3.2. Modern browsers ignore it entirely. It is documented here only for completeness, because it appears in the earliest HTML specifications and in archived pages from 1991 to 1995.

Syntax

```
<nextid n="z12">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	None (obsolete)
Permitted content	None (void element)

Property	Value
Permitted parents	<head> (historically)
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
n	Text (anchor name)	The next anchor name the editor should generate.	HTML 1

<nextid>: attribute by attribute

Each attribute of <nextid> explained in detail, with a minimal example you can copy.

n

Values: Text (anchor name) | Since: HTML 1

The next anchor name the editor should generate.

Write `n="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<nextid n="value">
```


<nextid>: worked examples

Historical usage

```
<head>
  <title>Welcome</title>
  <nextid n="z5">
</head>
```

Ignored by all modern browsers.

Default browser CSS

```
nextid { display: none; }
```

Common mistakes

- Any use in a new document.

See also

<head>, <isindex>, <listing>

Quick card: `<nobr>`

No line break (non-standard, obsolete)

The `<nobr>` element prevented the browser from wrapping its text onto a new line. It was never standardised. Use CSS `white-space: nowrap` instead.

At a glance

Status	OBSOLETE
Versions	Netscape / IE extension, never standard; obsolete
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><wbr></code> , <code>
</code>

Minimal example

```
<span style="white-space: nowrap">+20 100 000 0000</span>
```

Most common mistake

Any use.

<nobr> — No line break (non-standard, obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: Netscape / IE extension, never standard; obsolete

Description

The <nobr> element prevented the browser from wrapping its text onto a new line. It was never standardised. Use CSS white-space: nowrap instead.

Syntax

<nobr>text that will not wrap</nobr>

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Historical
Permitted content	Phrasing content
Permitted parents	Phrasing content
Tag omission	Neither
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<nobr>: worked examples

CSS replacement

```
<span style="white-space: nowrap">+20 100 000 0000</span>
```

Default browser CSS

```
nobr { white-space: nowrap; }
```

Common mistakes

- Any use.

See also

<wbr>,

Quick card: <noembed>

Fallback for embed (obsolete)

The <noembed> element provided fallback content for browsers that did not support <embed>. It was never standard and is obsolete. Use <object> with fallback content, or <video>/<audio> with fallback content inside them.

At a glance

Status	OBSOLETE
Versions	Netscape extension; obsolete
Category	Obsolete embedded content
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	None (global attributes only)
Related	<embed>, <object>

Minimal example

```
<object data="movie.mp4" type="video/mp4">
  <p>Your browser cannot play this video. <a href="movie.
mp4">Download it</a>.</p>
</object>
```

Most common mistake

Any use.

<noembed> — Fallback for embed (obsolete)

Category: Obsolete embedded content | Status: OBSOLETE | Versions: Netscape extension; obsolete

Description

The <noembed> element provided fallback content for browsers that did not support <embed>. It was never standard and is obsolete. Use <object> with fallback content, or <video>/<audio> with fallback content inside them.

Syntax

<noembed>fallback</noembed>

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	Historical
Permitted content	Flow content
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<noembed>: worked examples

Replacement

```
<object data="movie.mp4" type="video/mp4">
  <p>Your browser cannot play this video. <a href="movie.
mp4">Download it</a>.</p>
</object>
```

Default browser CSS

```
noembed { display: none; }
```

Common mistakes

- Any use.

See also

<embed>, <object>

Quick card: <noframes>

Fallback for frames (obsolete)

The <noframes> element contained content to display in browsers that did not support frames. It became obsolete together with <frameset> and <frame> in HTML5.

At a glance

Status	OBSOLETE
Versions	HTML 4.01 Frameset; obsolete in HTML5
Category	Obsolete frames
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	None (global attributes only)
Related	<frameset>, <frame>

Minimal example

```
<frameset cols="200,*">
  <frame src="menu.html">
  <frame src="main.html">
  <noframes><body><p>This page uses frames.</p></body></noframes>
</frameset>
```

Most common mistake

Any use.

<noframes> — Fallback for frames (obsolete)

Category: Obsolete frames | Status: OBSOLETE | Versions: HTML 4.01 Frameset; obsolete in HTML5

Description

The <noframes> element contained content to display in browsers that did not support frames. It became obsolete together with <frameset> and <frame> in HTML5.

Syntax

```
<noframes><body>fallback</body></noframes>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	Historical
Permitted content	Flow content (a <body>)
Permitted parents	<frameset>
Tag omission	Neither
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<noframes>: worked examples

Historical

```
<frameset cols="200,*">
  <frame src="menu.html">
  <frame src="main.html">
  <noframes><body><p>This page uses frames.</p></body></noframes>
</frameset>
```

Default browser CSS

```
noframes { display: none; }
```

Common mistakes

- Any use.

See also

<frameset>, <frame>

Quick card: <noscript>

Fallback when scripting is disabled

The <noscript> element defines content to be shown only if scripting is disabled in the browser or the browser does not support scripting. When scripting is enabled the content is ignored entirely (in fact the parser treats it as raw text). It may be used in <head> (containing only <link>, <style> and <meta>) and in <body>.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Scripting
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<script>, <template>

Minimal example

```
<noscript>
  <p><strong>This page works best with JavaScript enabled.</strong></
p>
</noscript>
```

Most common mistake

Nesting noscript.

<noscript> — Fallback when scripting is disabled

Category: Scripting | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<noscript>` element defines content to be shown only if scripting is disabled in the browser or the browser does not support scripting. When scripting is enabled the content is ignored entirely (in fact the parser treats it as raw text). It may be used in `<head>` (containing only `<link>`, `<style>` and `<meta>`) and in `<body>`.

Common uses: a warning that the site requires JavaScript, a non-JavaScript version of a widget, or a `<link>` to a style sheet that only applies when scripts are off. Tracking pixels from analytics services also use it as a fallback.

Syntax

`<noscript>fallback content</noscript>`

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Metadata content, flow content, phrasing content
Permitted content	In <code><head></code> : <code><link></code> , <code><style></code> , <code><meta></code> . In <code><body></code> : transparent content, but no <code><noscript></code> descendants
Permitted parents	<code><head></code> and any element that accepts phrasing content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<noscript>: worked examples

Warning message

```
<noscript>
  <p><strong>This page works best with JavaScript enabled.</strong></p>
</noscript>
```

Non-JavaScript style sheet

```
<head>
  <noscript><link rel="stylesheet" href="no-js.css"></noscript>
</head>
```

Image fallback for lazy-loaded pictures

```

<noscript></noscript>
```

Default browser CSS

```
noscript { display: inline; }
```

Accessibility

- Do not assume screen reader users have scripts disabled; almost all use JavaScript-enabled browsers.

Common mistakes

- Nesting noscript.
- Expecting noscript content to appear when a script merely fails to load (it appears only when scripting is off).

See also

<script>, <template>

Quick card: <object>

External object

The <object> element represents an external resource, treated as an image, a nested browsing context (like an iframe) or a resource handled by a plug-in, depending on its type. It was introduced in HTML 4.01 as a universal replacement for , <applet> and <embed>. Unlike <embed>, it accepts fallback content between its tags, shown if the resource cannot be displayed, and it may contain <param> elements (now obsolete) that pass parameters to plug-ins.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 4.01, HTML5 |
| Category | Embedded content |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | data, type, name, width, form, usemap, archive |
| Related | <embed>, <iframe>, , <param>, <video> |

Minimal example

```
<object data="catalogue.pdf" type="application/pdf" width="100%"
height="600">
  <p>Your browser cannot display the PDF. <a href="catalogue.
pdf">Download it</a>.</p>
</object>
```

Most common mistake

Using <object> for video.

<object> — External object

Category: Embedded content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<object>` element represents an external resource, treated as an image, a nested browsing context (like an `iframe`) or a resource handled by a plug-in, depending on its type. It was introduced in HTML 4.01 as a universal replacement for `<img>`, `<applet>` and `<embed>`. Unlike `<embed>`, it accepts fallback content between its tags, shown if the resource cannot be displayed, and it may contain `<param>` elements (now obsolete) that pass parameters to plug-ins.

Today `<object>` is mainly used to embed PDFs and SVG images with fallback. For video, audio and other pages, the dedicated elements are better.

Syntax

```
<object data="URL" type="MIME" width="W" height="H">fallback</object>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | inline |
| Content categories | Flow content, phrasing content, embedded content, listed, form-associated, palpable content; interactive if usemap |
| Permitted content | Transparent (fallback content); historically zero or more <code><param></code> first |

| Property | Value |
|-------------------|---|
| Permitted parents | Any element that accepts embedded content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|----------------|-----------|---|-----------|
| data | URL | The resource URL. Required (or classid historically). | HTML 4.01 |
| type | MIME type | The content type of the resource, for example application/pdf, image/svg+xml. | HTML 4.01 |
| name | text | Name of the browsing context (for use as a target). | HTML 4.01 |
| width / height | pixels | Display size. | HTML 4.01 |
| form | form id | Associated form. | HTML5 |
| usemap | #map | Image map to use (obsolete in the Living Standard). | HTML 4.01 |

| Attribute | Values | Description | Since |
|--|---------|--|-----------------------------|
| archive,
classid,
codebase,
codetype,
declare,
standby, ty
pemustmat
ch, border,
align,
hspace,
vspace | various | Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them. | HTML 4.01 / HTML5 (removed) |

<object>: attribute by attribute

Each attribute of <object> explained in detail, with a minimal example you can copy.

data

Values: URL | Since: HTML 4.01

The resource URL. Required (or classid historically).

Write data="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object data="https://example.com/page.html">...</object>
```

type

Values: MIME type | Since: HTML 4.01

The content type of the resource, for example application/pdf, image/svg+xml.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object type="text/plain">...</object>
```

name

Values: text | Since: HTML 4.01

Name of the browsing context (for use as a target).

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object name="value">...</object>
```

width / height

Values: pixels | Since: HTML 4.01

Display size.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object width="3">...</object>
```

form

Values: form id | Since: HTML5

Associated form.

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object form="form id">...</object>
```

usemap

Values: #map | Since: HTML 4.01

Image map to use (obsolete in the Living Standard).

Write `usemap="#map"` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object usemap="#map">...</object>
```

archive, classid, codebase, codetype, declare, standby, typemustmatch, border, align, hspace, vspace

Values: various | Since: HTML 4.01 / HTML5 (removed)

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Write `archive="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<object archive="various">...</object>
```

<object>: worked examples

PDF with fallback link

```
<object data="catalogue.pdf" type="application/pdf" width="100%"
height="600">
  <p>Your browser cannot display the PDF. <a href="catalogue.
pdf">Download it</a>.</p>
</object>
```

SVG with PNG fallback

```
<object data="chart.svg" type="image/svg+xml" width="400"
height="300">
  
</object>
```

Default browser CSS

```
object { display: inline; }
```

Accessibility

- Provide fallback content that is accessible; there is no alt attribute on object.
- Add a title for screen readers when the object is a nested page.

Common mistakes

- Using <object> for video.
- Nested objects several levels deep as a 'fallback chain' for old plug-ins.

See also

<embed>, <iframe>, , <param>, <video>

Quick card:

Ordered list

The element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an . If the order is not meaningful, use instead.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5 |
| Category | Text content |
| Closing tag | Required unless noted under Tag omission |
| Display | block |
| Own attributes | type, start, reversed, compact |
| Related | , , <menu>, <dl> |

Minimal example

```
<ol>
  <li>Open the editor.</li>
  <li>Write the HTML.</li>
  <li>Save as index.html.</li>
  <li>Open it in the browser.</li>
</ol>
```

Most common mistake

Using when order does not matter.

 — Ordered list

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<ol>` element represents an ordered list of items: a list where the order matters, such as the steps of a recipe, the ranking of a competition, or the chapters of a book. Browsers number the items automatically. Each item is an `<li>`. If the order is not meaningful, use `<ul>` instead.

The `type` attribute selects the numbering system (numbers, letters, roman numerals), `start` sets the first number and `reversed` counts downward. These attributes are not presentational and remain valid in HTML5 because changing the numbering style can change the meaning of references such as 'see item iv'. For purely visual changes use CSS `list-style-type`.

Syntax

```
<ol>
  <li>First</li>
  <li>Second</li>
</ol>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | block |
| Content categories | Flow content; palpable if it has at least one <code></code> |
| Permitted content | Zero or more <code></code> , <code><script></code> , <code><template></code> |

| Property | Value |
|-------------------|---------------------------------------|
| Permitted parents | Any element that accepts flow content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLListElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|-------------------|--|----------|
| type | 1 a A i I | Numbering type: decimal (default), lowercase letters, uppercase letters, lowercase roman, uppercase roman. | HTML 3.2 |
| start | integer | The number of the first item (always a decimal integer even if type is a or i). | HTML 3.2 |
| reversed | boolean | Number the items in descending order. | HTML5 |
| compact | boolean | Obsolete. Compact rendering. | HTML 2.0 |

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

type

Values: 1 | a | A | i | I | Since: HTML 3.2

Numbering type: decimal (default), lowercase letters, uppercase letters, lowercase roman, uppercase roman.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<ol type="1">...</ol>
```

start

Values: integer | Since: HTML 3.2

The number of the first item (always a decimal integer even if type is a or i).

Write start="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<ol start="3">...</ol>
```

reversed

Values: boolean | Since: HTML5

Number the items in descending order.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<ol reversed>...</ol>
```

compact

Values: boolean | Since: HTML 2.0

Obsolete. Compact rendering.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<ol compact>...</ol>
```

: worked examples

Steps

```
<ol>
  <li>Open the editor.</li>
  <li>Write the HTML.</li>
  <li>Save as index.html.</li>
  <li>Open it in the browser.</li>
</ol>
```

Letters and roman numerals

```
<ol type="A">
  <li>Option A</li>
  <li>Option B</li>
</ol>
<ol type="i" start="4">
  <li>iv</li>
  <li>v</li>
</ol>
```

Countdown

```
<ol reversed>
  <li>Third place</li>
  <li>Second place</li>
  <li>Winner</li>
</ol>
```

Numbers 3, 2, 1.

Nested numbering with CSS counters

```
<style>
  ol { counter-reset: item; list-style: none; }
  ol li::before { counter-increment: item; content: counters(item, '
') ' '; }
</style>
<ol>
  <li>Intro
    <ol>
      <li>History</li>
      <li>Syntax</li>
    </ol>
  </li>
</ol>
```

Produces 1, 1.1, 1.2.

Removing the numbers

```
<ol style="list-style: none; padding: 0">...</ol>
```

Default browser CSS

```
ol { display: block; list-style-type: decimal; margin: 1em 0; padding-  
inline-start: 40px; }
```

Accessibility

- Screen readers announce 'list, 4 items' and the number of each item, which is very helpful for instructions.

Common mistakes

- Using `<ol>` when order does not matter.
- Writing the numbers by hand inside the items.
- Putting text directly inside `<ol>` outside of `<li>`.

See also

`<ul>`, `<li>`, `<menu>`, `<dl>`

Quick card: <optgroup>

Option group

The <optgroup> element groups related <option> elements inside a <select> under a non-selectable heading given by the label attribute. Groups cannot be nested. Disabling an optgroup disables all its options.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	label, disabled
Related	<select>, <option>

Minimal example

```
<label for="city">City</label>
<select id="city" name="city">
  <optgroup label="Egypt">
    <option value="cai">Cairo</option>
    <option value="giz">Giza</option>
    <option value="alx">Alexandria</option>
  </optgroup>
  <optgroup label="Saudi Arabia">
    <option value="ruh">Riyadh</option>
    <option value="jed">Jeddah</option>
  </optgroup>
</select>
```

Most common mistake

Omitting label.

<optgroup> — Option group

Category: Forms | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The <optgroup> element groups related <option> elements inside a <select> under a non-selectable heading given by the label attribute. Groups cannot be nested. Disabling an optgroup disables all its options.

Syntax

```
<select>
  <optgroup label="Group">
    <option>...</option>
  </optgroup>
</select>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	Zero or more <option>, <script>, <template>; optionally a <legend> first in newer browsers with customisable select
Permitted parents	<select>
Tag omission	End tag may be omitted if followed by another <optgroup> or if there is no more content in the parent
DOM interface	HTMLOptGroupElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
label	text	Required. The group heading shown in the list.	HTML 4.01
disabled	boolean	Disables all options in the group.	HTML 4.01

<optgroup>: attribute by attribute

Each attribute of <optgroup> explained in detail, with a minimal example you can copy.

label

Values: text | Since: HTML 4.01

Required. The group heading shown in the list.

Write label="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<optgroup label="value">...</optgroup>
```

disabled

Values: boolean | Since: HTML 4.01

Disables all options in the group.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<optgroup disabled>...</optgroup>
```

<optgroup>: worked examples

Cities grouped by country

```
<label for="city">City</label>
<select id="city" name="city">
  <optgroup label="Egypt">
    <option value="cai">Cairo</option>
    <option value="giz">Giza</option>
    <option value="alx">Alexandria</option>
  </optgroup>
  <optgroup label="Saudi Arabia">
    <option value="ruh">Riyadh</option>
    <option value="jed">Jeddah</option>
  </optgroup>
</select>
```

Default browser CSS

```
optgroup { display: block; font-weight: bold; font-style: normal; }
```

Accessibility

- Screen readers announce the group label when entering the group.

Common mistakes

- Omitting label.
- Nesting optgroups.
- Trying to make the group heading selectable.

See also

<select>, <option>

Quick card: <option>

Option

The <option> element defines one item in a <select> drop-down list, an <optgroup>, or a <datalist> of suggestions. Its text content is what the user sees; the value attribute is what is submitted with the form. If value is omitted the text content is submitted instead.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	value, selected, disabled, label
Related	<select>, <optgroup>, <datalist>

Minimal example

```
<select name="size">
  <option value="">-- Choose a size --</option>
  <option value="s">Small</option>
  <option value="m" selected>Medium</option>
  <option value="l">Large</option>
</select>
```

Most common mistake

Nesting HTML inside option (only text is allowed).

<option> — Option

Category: Forms | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<option>` element defines one item in a `<select>` drop-down list, an `<optgroup>`, or a `<datalist>` of suggestions. Its text content is what the user sees; the `value` attribute is what is submitted with the form. If `value` is omitted the text content is submitted instead.

The `selected` attribute pre-selects the option. In a single-select list only one option should be selected; in a `<select multiple>` several may be. The end tag may be omitted.

Syntax

```
<option value="v">Text</option>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	None
Permitted content	Text, optionally with escaped characters
Permitted parents	<code><select></code> , <code><optgroup></code> , <code><datalist></code>
Tag omission	End tag may be omitted if followed by another <code><option></code> or <code><optgroup></code> , or if there is no more content in the parent
DOM interface	HTMLOptionElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
value	text	The value submitted when this option is selected. Defaults to the text content.	HTML 2.0
selected	boolean	Initially selected.	HTML 2.0
disabled	boolean	Cannot be selected.	HTML 4.01
label	text	Alternative label shown instead of the text content (used mainly in datalist).	HTML 4.01

<option>: attribute by attribute

Each attribute of <option> explained in detail, with a minimal example you can copy.

value

Values: text | Since: HTML 2.0

The value submitted when this option is selected. Defaults to the text content.

Write `value="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<option value="value">...</option>
```

selected

Values: boolean | Since: HTML 2.0

Initially selected.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<option selected>...</option>
```

disabled

Values: boolean | Since: HTML 4.01

Cannot be selected.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<option disabled>...</option>
```

label

Values: text | Since: HTML 4.01

Alternative label shown instead of the text content (used mainly in datalist).

Write `label="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<option label="value">...</option>
```

<option>: worked examples

Basic select

```
<select name="size">
  <option value="">-- Choose a size --</option>
  <option value="s">Small</option>
  <option value="m" selected>Medium</option>
  <option value="l">Large</option>
</select>
```

The first option with an empty value acts as a placeholder; combine with `required` to force a choice.

Disabled placeholder that cannot be re-selected

```
<select required>
  <option value="" disabled selected>Select a country</option>
  <option>Egypt</option>
</select>
```

Datalist options with labels

```
<datalist id="codes">
  <option value="EG" label="Egypt">
  <option value="SA" label="Saudi Arabia">
</datalist>
```

Default browser CSS

```
option { display: block; padding: 0 2px 1px; white-space: nowrap; min-
height: 1.2em; }
```

Accessibility

- Keep option text short and unique.
- Do not rely on colour alone to differentiate options.

Common mistakes

- Nesting HTML inside option (only text is allowed).
- Multiple selected options in a single select (the last wins).
- Forgetting the value so a long label is submitted.

See also

`<select>`, `<optgroup>`, `<datalist>`

Quick card: <output>

Calculation or user-action result

The <output> element is a container into which a site or app can inject the result of a calculation or the outcome of a user action, for example the total price in a cart, the current value of a range slider, or a status message. It is form-associated and labelable, and it is an ARIA live region by default (role="status"), which means screen readers announce changes to its content automatically.

At a glance

Status	CURRENT
Versions	HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	for, form, name
Related	<input>, <form>, <meter>, <progress>

Minimal example

```
<form oninput="total.value = Number(a.value) + Number(b.value)">
  <input type="number" id="a" name="a" value="0"> +
  <input type="number" id="b" name="b" value="0"> =
  <output name="total" for="a b">0</output>
</form>
```

Most common mistake

Using a for results that should be announced.

<output> — Calculation or user-action result

Category: Forms | Status: CURRENT | Versions: HTML5

Description

The `<output>` element is a container into which a site or app can inject the result of a calculation or the outcome of a user action, for example the total price in a cart, the current value of a range slider, or a status message. It is form-associated and labelable, and it is an ARIA live region by default (`role="status"`), which means screen readers announce changes to its content automatically.

Syntax

```
<output name="result" for="a b">0</output>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, listed, labelable, resettable, form-associated, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLOutputElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
for	space-separated ids	The elements that contributed to the calculation.	HTML5
form	form id	Associated form.	HTML5
name	text	Name of the output (not submitted with the form).	HTML5

<output>: attribute by attribute

Each attribute of <output> explained in detail, with a minimal example you can copy.

for

Values: space-separated ids | Since: HTML5

The elements that contributed to the calculation.

Write for="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<output for="space-separated ids">...</output>
```

form

Values: form id | Since: HTML5

Associated form.

Write form="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<output form="form id">...</output>
```

name

Values: text | Since: HTML5

Name of the output (not submitted with the form).

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<output name="value">...</output>
```

<output>: worked examples

Live sum of two inputs

```
<form oninput="total.value = Number(a.value) + Number(b.value)">
  <input type="number" id="a" name="a" value="0"> +
  <input type="number" id="b" name="b" value="0"> =
  <output name="total" for="a b">0</output>
</form>
```

Showing a slider's value

```
<label for="vol">Volume</label>
<input type="range" id="vol" min="0" max="100" value="50" oninput="v.
value = this.value">
<output id="v" for="vol">50</output>
```

Default browser CSS

```
output { display: inline; unicode-bidi: isolate; }
```

Accessibility

- Because it is a live region, changes are announced; do not update it dozens of times per second with noise.

Common mistakes

- Using a for results that should be announced.
- Expecting the value to be submitted.

See also

<input>, <form>, <meter>, <progress>

Quick card: <p>

Paragraph

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	align
Related	<code>
</code> , <code><div></code> , <code></code> , <code><blockquote></code> , <code><pre></code>

Minimal example

```
<p>HTML describes the structure of a page.</p>
<p>CSS describes how the page looks.</p>
```

Most common mistake

Block elements inside `<p>`.

<p> — Paragraph

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<p>` element represents a paragraph: a block of related text, usually a group of sentences on one topic. Browsers display paragraphs as blocks separated by vertical space (a margin of 1em above and below). Paragraphs are one of the most used elements in HTML and one of the original elements from 1991, where `<p>` was a separator between paragraphs rather than a container.

A paragraph may contain only phrasing content (text, links, emphasis, images, and so on). It cannot contain block-level elements such as `<div>`, `<ul>`, `<table>`, headings or another `<p>`. If the browser encounters one of these inside a `<p>` it automatically closes the paragraph first, which often produces layouts that look wrong for no obvious reason.

The end tag may be omitted because the browser knows a paragraph ends when the next block starts, but writing `</p>` makes the code much clearer. Do not use empty `<p></p>` elements to add space; use CSS margins.

Syntax

```
<p>Text of the paragraph.</p>
```

Technical summary

Property	Value
Void element	No
Default display	block

Property	Value
Content categories	Flow content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts flow content
Tag omission	End tag may be omitted if the <p> is immediately followed by address, article, aside, blockquote, details, dialog, div, dl, fieldset, figcaption, figure, footer, form, h1-h6, header, hgroup, hr, main, menu, nav, ol, p, pre, search, section, table or ul, or if there is no more content in the parent (except when the parent is <a>, <audio>, , <ins>, <map>, <noscript> or <video>)
DOM interface	HTMLParagraphElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right justify	Obsolete. Use CSS text-align.	HTML 3.2

<p>: attribute by attribute

Each attribute of <p> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | justify | Since: HTML 3.2

Obsolete. Use CSS text-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<p align="left">...</p>
```

<p>: worked examples

Two paragraphs

```
<p>HTML describes the structure of a page.</p>
<p>CSS describes how the page looks.</p>
```

Inline elements inside a paragraph

```
<p>I <em>really</em> love <a href="https://developer.mozilla.
org">MDN</a>, and I read it <strong>every day</strong>.</p>
```

Wrong: a list inside a paragraph

```
<!-- Wrong: the browser closes the <p> before the <ul> -->
<p>Shopping:
  <ul><li>Milk</li></ul>
</p>
```

```
<!-- Right -->
<p>Shopping:</p>
<ul><li>Milk</li></ul>
```

Typography with CSS

```
<style>
  p { line-height: 1.6; max-width: 65ch; margin: 0 0 1em; }
  p + p { text-indent: 1.5em; margin-top: -0.5em; }
</style>
```

65ch keeps lines at a comfortable reading length; text-indent gives book-style paragraphs.

Default browser CSS

```
p { display: block; margin-block: 1em; }
```

Accessibility

- Screen readers pause between paragraphs; use real paragraphs rather than `<br><br>`.
- Keep paragraphs reasonably short; long walls of text are hard for everyone.

Common mistakes

- Block elements inside `<p>`.
- Empty paragraphs for spacing.

- Using `<br><br>` instead of a new paragraph.
- Using `<p>` for every line in an address (use `<br>` inside one `<p>`).

See also

`<br>`, `<div>`, `<span>`, `<blockquote>`, `<pre>`

Quick card: <param>

Object parameter (deprecated)

The <param> element defined a parameter for an <object> element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

At a glance

Status	DEPRECATED
Versions	HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard
Category	Embedded content
Closing tag	None (void element)
Display	none
Own attributes	name, value, type, valuetype
Related	<object>, <embed>

Minimal example

```
<object data="movie.swf" type="application/x-shockwave-flash"
width="400" height="300">
  <param name="quality" value="high">
  <param name="wmode" value="transparent">
  <p>Flash is no longer supported.</p>
</object>
```

Most common mistake

Any use in new pages.

<param> — Object parameter (deprecated)

Category: Embedded content | Status: DEPRECATED | Versions: HTML 3.2, HTML 4.01, HTML5; deprecated in the Living Standard

Description

The `<param>` element defined a parameter for an `<object>` element, passing a name/value pair to the plug-in that displayed the object (for example the autoplay setting of a Flash movie). Since plug-ins no longer exist, the element serves no purpose; the Living Standard marks it deprecated and it is retained only for compatibility. It is a void element.

Syntax

```
<object data="movie.swf">
  <param name="quality" value="high">
</object>
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	None
Permitted content	None (void element)
Permitted parents	<code><object></code> , before any flow content
Tag omission	No end tag
DOM interface	HTMLParamElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
name	text	The parameter name.	HTML 3.2
value	text	The parameter value.	HTML 3.2
type	MIME type	Obsolete. Type of the value when valuetype=ref.	HTML 4.01
valuetype	data ref object	Obsolete. How to interpret value.	HTML 4.01

<param>: attribute by attribute

Each attribute of <param> explained in detail, with a minimal example you can copy.

name

Values: text | Since: HTML 3.2

The parameter name.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<param name="value">
```

value

Values: text | Since: HTML 3.2

The parameter value.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<param value="value">
```

type

Values: MIME type | Since: HTML 4.01

Obsolete. Type of the value when valuetype=ref.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<param type="text/plain">
```

valuetype

Values: data | ref | object | Since: HTML 4.01

Obsolete. How to interpret value.

Write valuetype="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<param valuetype="data">
```

<param>: worked examples

Historical Flash embed

```
<object data="movie.swf" type="application/x-shockwave-flash"
width="400" height="300">
  <param name="quality" value="high">
  <param name="wmode" value="transparent">
  <p>Flash is no longer supported.</p>
</object>
```

Default browser CSS

```
param { display: none; }
```

Common mistakes

- Any use in new pages.

See also

<object>, <embed>

Quick card: <picture>

Picture (responsive image container)

The <picture> element contains zero or more <source> elements and exactly one element, and lets the browser choose the most suitable image from several candidates. The browser examines each <source> in order and uses the first whose media query matches and whose type it supports; if none match it falls back to the . The element is required: it is what is actually displayed, and it carries the alt text, width, height, loading and other attributes.

At a glance

Status	CURRENT
Versions	HTML 5.1 (2016)
Category	Image and multimedia
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	, <source>, <figure>

Minimal example

```
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  
</picture>
```

Most common mistake

Omitting the `<img>`.

<picture> — Picture (responsive image container)

Category: Image and multimedia | Status: CURRENT | Versions: HTML 5.1 (2016)

Description

The `<picture>` element contains zero or more `<source>` elements and exactly one `<img>` element, and lets the browser choose the most suitable image from several candidates. The browser examines each `<source>` in order and uses the first whose media query matches and whose type it supports; if none match it falls back to the `<img>`. The `<img>` element is required: it is what is actually displayed, and it carries the alt text, width, height, loading and other attributes.

Use `<picture>` for three purposes: art direction (a cropped image on phones, a wide one on desktops), format selection (AVIF or WebP for modern browsers, JPEG for old ones), and dark-mode variants. For simply serving different sizes of the same image, `srcset` and `sizes` on a plain `<img>` are enough.

Syntax

```
<picture>
  <source srcset="..." media="..." type="...">
  
</picture>
```

Technical summary

Property	Value
Void element	No
Default display	inline

Property	Value
Content categories	Flow content, phrasing content, embedded content, palpable content
Permitted content	Zero or more <source> elements, followed by one , optionally intermixed with <script> and <template>
Permitted parents	Any element that accepts embedded content
Tag omission	Neither tag may be omitted
DOM interface	HTMLPictureElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<picture>: worked examples

Modern formats with fallback

```
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  
</picture>
```

Browsers that support AVIF use it; others try WebP; the rest use JPEG.

Art direction

```
<picture>
  <source media="(max-width: 600px)" srcset="hero-portrait.jpg">
  <source media="(min-width: 601px)" srcset="hero-wide.jpg">
  
</picture>
```

A tight portrait crop on phones, the full wide picture on larger screens.

Dark mode image

```
<picture>
  <source media="(prefers-color-scheme: dark)" srcset="logo-dark.svg">
  
</picture>
```

Combining with srcset sizes

```
<picture>
  <source type="image/webp" srcset="p-400.webp 400w, p-800.webp 800w"
sizes="(max-width: 600px) 100vw, 600px">
  
</picture>
```

Default browser CSS

```
picture { display: inline; }
```

Accessibility

- All accessibility comes from the `<img>`: alt text goes there, never on `<source>`.

Common mistakes

- Omitting the `<img>`.
- Putting `<img>` before the `<source>` elements (sources after `img` are ignored).
- Using `src` instead of `srcset` on `<source>`.
- Styling the `<picture>` instead of the `<img>` (`picture` has no box of its own in most cases).

See also

`<img>`, `<source>`, `<figure>`

Quick card: <plaintext>

Plain text (obsolete)

The <plaintext> element told the browser to treat everything after it as plain text, without interpreting any further tags, until the end of the file. It could not be closed. It was one of the 1991 elements, was deprecated in HTML 2.0 and is obsolete. Use <pre> for pre-formatted text, escaping < and & as entities, or serve the file as text/plain.

At a glance

Status	OBSOLETE
Versions	HTML 1, HTML 2.0 (deprecated); obsolete
Category	Obsolete text content
Closing tag	None (void element)
Display	block
Own attributes	None (global attributes only)
Related	<pre>, <xmp>, <listing>

Minimal example

```
<pre>&lt;p&gt;This shows the tag literally.&lt;/p&gt;</pre>
```

Most common mistake

Any use.

<plaintext> — Plain text (obsolete)

Category: Obsolete text content | Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

Description

The `<plaintext>` element told the browser to treat everything after it as plain text, without interpreting any further tags, until the end of the file. It could not be closed. It was one of the 1991 elements, was deprecated in HTML 2.0 and is obsolete. Use `<pre>` for pre-formatted text, escaping `<` and `&` as entities, or serve the file as text/plain.

Syntax

`<plaintext>`

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	block
Content categories	Historical
Permitted content	Everything after it
Permitted parents	Any
Tag omission	No end tag possible
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<plaintext>: worked examples

Replacement

```
<pre>&lt;p&gt;This shows the tag literally.&lt;/p&gt;</pre>
```

Default browser CSS

```
plaintext { display: block; font-family: monospace; white-space: pre;
}
```

Common mistakes

- Any use.

See also

<pre>, <xmp>, <listing>

Quick card: <portal>

Portal (experimental, abandoned)

The <portal> element was an experiment that embedded a preview of another page, like an iframe, with the ability to 'activate' it and navigate seamlessly into it with an animation. It was available only behind a flag in Chrome and the work has been superseded by the View Transitions API and prerendering with the Speculation Rules API. Do not use it.

At a glance

| | |
|----------------|---|
| Status | EXPERIMENTAL |
| Versions | Chrome experiment (2019-2022); not standardised |
| Category | Embedded content |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | src, referrerpolicy |
| Related | <iframe>, <fencedframe> |

Minimal example

```
<script type="speculationrules">
{ "prerender": [{ "urls": ["/next-page.html"] }] }
</script>
```

Most common mistake

Any use.

<portal> — Portal (experimental, abandoned)

Category: Embedded content | Status: EXPERIMENTAL | Versions: Chrome experiment (2019-2022); not standardised

Description

The <portal> element was an experiment that embedded a preview of another page, like an iframe, with the ability to 'activate' it and navigate seamlessly into it with an animation. It was available only behind a flag in Chrome and the work has been superseded by the View Transitions API and prerendering with the Speculation Rules API. Do not use it.

Syntax

```
<portal src="https://example.com"></portal>
```

Technical summary

| Property | Value |
|--------------------|---------------------------------|
| Void element | No |
| Default display | inline |
| Content categories | Experimental |
| Permitted content | None |
| Permitted parents | Embedded content |
| Tag omission | Neither |
| DOM interface | HTMLPortalElement (Chrome only) |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|----------------|--------|------------------|--------------|
| src | URL | Page to preview. | Experimental |
| referrerpolicy | policy | Referrer policy. | Experimental |

<portal>: attribute by attribute

Each attribute of <portal> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: Experimental

Page to preview.

Write `src="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<portal src="https://example.com/page.html">...</portal>
```

referrerpolicy

Values: policy | Since: Experimental

Referrer policy.

Write `referrerpolicy="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<portal referrerpolicy="policy">...</portal>
```

<portal>: worked examples

Modern alternative

```
<script type="speculationrules">
{ "prerender": [{ "urls": ["/next-page.html"] }] }
</script>
```

Common mistakes

- Any use.

See also

<iframe>, <fencedframe>

Quick card: <pre>

Pre-formatted text

The <pre> element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

At a glance

| | |
|----------------|---|
| Status | CURRENT |
| Versions | HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5 |
| Category | Text content |
| Closing tag | Required unless noted under Tag omission |
| Display | block |
| Own attributes | width, wrap |
| Related | <code>, <samp>, <kbd>, <xmp>, <listing>, <textarea> |

Minimal example

```
<pre><code>&lt; footer&gt;
  &lt;p&gt;&amp;copy; 2026&lt;/p&gt;
&lt;/footer&gt;</code></pre>
```

Most common mistake

Forgetting to escape < and &.

<pre> — Pre-formatted text

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<pre>` element represents pre-formatted text that is displayed exactly as written in the HTML file: every space, tab and line break is preserved, and a monospace (fixed-width) font is used. It is the standard way to show source code, ASCII art, poetry with meaningful spacing, email messages and terminal output.

Markup inside `<pre>` is still interpreted, so to show HTML source you must escape `<` as `<` and `&` as `&`. For code, wrap the content in `<code>` inside the `<pre>`. A line break immediately after the opening `<pre>` tag is ignored by the parser, which is convenient for formatting.

Long lines in `<pre>` do not wrap by default and can overflow the page horizontally; add CSS `overflow-x: auto` or `white-space: pre-wrap`.

Syntax

```
<pre>
  text with
    preserved   spacing
</pre>
```

Technical summary

| Property | Value |
|-----------------|-------|
| Void element | No |
| Default display | block |

| Property | Value |
|--------------------|---------------------------------------|
| Content categories | Flow content, palpable content |
| Permitted content | Phrasing content |
| Permitted parents | Any element that accepts flow content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLPreElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|---------|---|----------|
| width | integer | Obsolete. Number of characters per line. | HTML 2.0 |
| wrap | boolean | Non-standard. Wrap long lines. Use CSS white-space: pre-wrap. | Netscape |

<pre>: attribute by attribute

Each attribute of <pre> explained in detail, with a minimal example you can copy.

width

Values: integer | Since: HTML 2.0

Obsolete. Number of characters per line.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<pre width="3">...</pre>
```

wrap

Values: boolean | Since: Netscape

Non-standard. Wrap long lines. Use CSS white-space: pre-wrap.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<pre wrap>...</pre>
```


<pre>: worked examples

Code block

```
<pre><code>&lt;footer&gt;
  &lt;p&gt;&amp;copy; 2026&lt;/p&gt;
&lt;/footer&gt;</code></pre>
```

The entities display as the real characters.

ASCII art

```
<pre>
  /\_/\
  ( o.o )
  &gt; ^ &lt;
</pre>
```

Scrollable, styled code

```
<style>
  pre { background: #1e1e1e; color: #eee; padding: 16px; border-
radius: 8px; overflow-x: auto; tab-size: 2; }
</style>
```

Wrapping long lines

```
<pre style="white-space: pre-wrap; word-break: break-word">A very
long line ...</pre>
```

Default browser CSS

```
pre { display: block; font-family: monospace; white-space: pre;
margin: 1em 0; }
```

Accessibility

- For ASCII art or diagrams, add an explanation in text or a figcaption, because screen readers read the characters one by one.
- Use `<pre>` with `<code>` so assistive technology knows it is code.

Common mistakes

- Forgetting to escape `<` and `&`.
- Indenting the content in the source, which adds unwanted spaces to the output.

- Using `<pre>` for layout.

See also

`<code>`, `<samp>`, `<kbd>`, `<xmp>`, `<listing>`, `<textarea>`

Quick card: <progress>

Progress indicator

The <progress> element displays the completion progress of a task, such as a file upload, a download, a multi-step form or a loading operation. It is rendered as a progress bar. With a value attribute it shows a determinate bar (a known percentage); without value it shows an indeterminate, animated bar meaning 'working, but I do not know how long it will take'.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML5 |
| Category | Forms |
| Closing tag | Required unless noted under Tag omission |
| Display | inline-block |
| Own attributes | value, max |
| Related | <meter>, <output> |

Minimal example

```
<label for="up">Uploading photo</label>
<progress id="up" value="65" max="100">65%</progress>
```

Most common mistake

Using it for static measurements (use <meter>).

<progress> — Progress indicator

Category: Forms | Status: CURRENT | Versions: HTML5

Description

The `<progress>` element displays the completion progress of a task, such as a file upload, a download, a multi-step form or a loading operation. It is rendered as a progress bar. With a `value` attribute it shows a determinate bar (a known percentage); without `value` it shows an indeterminate, animated bar meaning 'working, but I do not know how long it will take'.

Use `<meter>` instead for a measurement that is not progress (disk space, a score).

Syntax

```
<progress value="30" max="100">30%</progress>
```

Technical summary

| Property | Value |
|--------------------|--|
| Void element | No |
| Default display | inline-block |
| Content categories | Flow content, phrasing content, labelable, palpable content |
| Permitted content | Phrasing content, but no <code><progress></code> descendants |
| Permitted parents | Any element that accepts phrasing content |
| Tag omission | Neither tag may be omitted |
| DOM interface | HTMLProgressElement |

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

| Attribute | Values | Description | Since |
|-----------|--------|--|-------|
| value | number | How much has been completed, from 0 to max. Omit for an indeterminate bar. | HTML5 |
| max | number | The total amount of work. Default 1. | HTML5 |

<progress>: attribute by attribute

Each attribute of <progress> explained in detail, with a minimal example you can copy.

value

Values: number | Since: HTML5

How much has been completed, from 0 to max. Omit for an indeterminate bar.

Write value="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<progress value="3">...</progress>
```

max

Values: number | Since: HTML5

The total amount of work. Default 1.

Write max="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<progress max="3">...</progress>
```

<progress>: worked examples

Determinate

```
<label for="up">Uploading photo</label>
<progress id="up" value="65" max="100">65%</progress>
```

Indeterminate

```
<progress>Loading...</progress>
```

Updating from JavaScript

```
<progress id="bar" max="100" value="0"></progress>
<script>
  let v = 0;
  const t = setInterval(() => {
    document.getElementById('bar').value = ++v;
    if (v === 100) clearInterval(t);
  }, 50);
</script>
```

Styling

```
<style>
  progress { width: 100%; height: 12px; accent-color: #0f8a4c; }
</style>
```

accent-color is the simplest cross-browser way to colour the bar.

Default browser CSS

```
progress { display: inline-block; appearance: auto; }
```

Accessibility

- Label the progress bar.
- Announce completion with a live region or by moving focus; the bar's value changes are not automatically spoken by all screen readers.

Common mistakes

- Using it for static measurements (use <meter>).
- Value greater than max.

See also

<meter>, <output>

Quick card: <q>

Inline quotation

The <q> element indicates a short inline quotation that sits within a sentence. Browsers automatically add quotation marks around it, choosing the correct style for the language declared with lang (curly double quotes in English, guillemets in French, and so on), so you should not type quotation marks yourself. For long quotations that need their own paragraph use <blockquote>.

At a glance

| | |
|----------------|--|
| Status | CURRENT |
| Versions | HTML 4.01, HTML5 |
| Category | Text-level semantics |
| Closing tag | Required unless noted under Tag omission |
| Display | inline |
| Own attributes | cite |
| Related | <blockquote>, <cite> |

Minimal example

```
<p>Tim Berners-Lee said <q>the Web is for everyone</q>.</p>
```

Most common mistake

Adding manual quotation marks inside <q>, giving double marks.

<q> — Inline quotation

Category: Text-level semantics | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<q>` element indicates a short inline quotation that sits within a sentence. Browsers automatically add quotation marks around it, choosing the correct style for the language declared with `lang` (curly double quotes in English, guillemets in French, and so on), so you should not type quotation marks yourself. For long quotations that need their own paragraph use `<blockquote>`.

Syntax

```
<q cite="URL">quoted words</q>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLQuoteElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
cite	URL	The source of the quotation. Not displayed.	HTML 4.01

<q>: attribute by attribute

Each attribute of <q> explained in detail, with a minimal example you can copy.

cite

Values: URL | Since: HTML 4.01

The source of the quotation. Not displayed.

Write `cite="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<q cite="https://example.com/page.html">...</q>
```

<q>: worked examples

Basic quote

```
<p>Tim Berners-Lee said <q>the Web is for everyone</q>.</p>
```

Renders as: Tim Berners-Lee said “the Web is for everyone”.

Language-aware quotation marks

```
<p lang="fr">Il a dit <q>bonjour</q>.</p>
```

Renders with « bonjour ».

Custom quote characters

```
<style>
  q { quotes: '„' '“'; }
</style>
```

Default browser CSS

```
q { display: inline; }
q::before { content: open-quote; }
q::after { content: close-quote; }
```

Accessibility

- Screen readers may or may not announce the quotation; the generated quote marks are usually read.

Common mistakes

- Adding manual quotation marks inside <q>, giving double marks.
- Using <q> for irony or scare quotes.

See also

<blockquote>, <cite>

Quick card: <rb>

Ruby base (deprecated)

The <rb> element explicitly marked the base text component of a ruby annotation, the text that the <rt> annotation is placed above. The Living Standard removed it because the base text can simply be written directly inside <ruby>. Browsers still support it.

At a glance

Status	DEPRECATED
Versions	XHTML 1.1 Ruby, HTML 5.1; removed from the Living Standard
Category	Text-level semantics (ruby)
Closing tag	Required unless noted under Tag omission
Display	ruby-base
Own attributes	None (global attributes only)
Related	<ruby>, <rt>, <rp>, <rtc>

Minimal example

```
<!-- With rb -->
<ruby><rb>□</rb><rt>tō</rt><rb>□</rb><rt>kyō</rt></ruby>

<!-- Without rb -->
<ruby>□<rt>tō</rt>□<rt>kyō</rt></ruby>
```

Most common mistake

Relying on it in new markup.

<rb> — Ruby base (deprecated)

Category: Text-level semantics (ruby) | Status: DEPRECATED | Versions: XHTML 1.1 Ruby, HTML 5.1; removed from the Living Standard

Description

The `<rb>` element explicitly marked the base text component of a ruby annotation, the text that the `<rt>` annotation is placed above. The Living Standard removed it because the base text can simply be written directly inside `<ruby>`. Browsers still support it.

Syntax

```
<ruby><rb>□</rb><rt>kan</rt></ruby>
```

Technical summary

Property	Value
Void element	No
Default display	ruby-base
Content categories	None
Permitted content	Phrasing content
Permitted parents	<code><ruby></code>
Tag omission	End tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<rb>: worked examples

Old and new

```
<!-- With rb -->
<ruby><rb>□</rb><rt>tō</rt><rb>□</rb><rt>kyō</rt></ruby>
```

```
<!-- Without rb -->
<ruby>□<rt>tō</rt>□<rt>kyō</rt></ruby>
```

Default browser CSS

```
rb { display: ruby-base; }
```

Common mistakes

- Relying on it in new markup.

See also

<ruby>, <rt>, <rp>, <rtc>

Quick card: <rp>

Ruby fallback parenthesis

The <rp> element provides fallback parentheses for browsers that do not support ruby annotations. Browsers that support ruby hide the <rp> content; older browsers display it, so the annotation appears in brackets after the base text instead of above it. Place one <rp> before and one after each <rt>.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text-level semantics (ruby)
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	None (global attributes only)
Related	<ruby>, <rt>

Minimal example

```
<ruby>□□<rp>(</rp><rt>□□□</rt><rp>)</rp></ruby>
```

Most common mistake

Omitting rp when supporting very old browsers.

<rp> — Ruby fallback parenthesis

Category: Text-level semantics (ruby) | Status: CURRENT | Versions: HTML5

Description

The `<rp>` element provides fallback parentheses for browsers that do not support ruby annotations. Browsers that support ruby hide the `<rp>` content; older browsers display it, so the annotation appears in brackets after the base text instead of above it. Place one `<rp>` before and one after each `<rt>`.

Syntax

```
<ruby>□<rp>(</rp><rt>kan</rt><rp>)</rp></ruby>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	None
Permitted content	Text
Permitted parents	<code><ruby></code> , immediately before or after an <code><rt></code>
Tag omission	End tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<rp>: worked examples

With fallback

```
<ruby>□□<rp>(</rp><rt>□□□</rt><rp>)</rp></ruby>
```

Modern browsers: annotation above. Old browsers: □□(□□□).

Default browser CSS

```
rp { display: none; }
```

Common mistakes

- Omitting rp when supporting very old browsers.

See also

<ruby>, <rt>

Quick card: <rt>

Ruby text (annotation)

The <rt> element specifies the ruby text component of a ruby annotation: the small text placed above (or beside) the base characters to show pronunciation, translation or transliteration. It is used for East Asian typography (furigana in Japanese, pinyin/zhuyin in Chinese) and increasingly for other languages. It must be inside a <ruby> element.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text-level semantics (ruby)
Closing tag	Required unless noted under Tag omission
Display	ruby-text
Own attributes	None (global attributes only)
Related	<ruby>, <rp>, <rb>, <rtc>

Minimal example

```
<ruby>□□<rt>□□</rt></ruby>
```

Most common mistake

Using <rt> outside <ruby>.

<rt> — Ruby text (annotation)

Category: Text-level semantics (ruby) | Status: CURRENT | Versions: HTML5

Description

The `<rt>` element specifies the ruby text component of a ruby annotation: the small text placed above (or beside) the base characters to show pronunciation, translation or transliteration. It is used for East Asian typography (furigana in Japanese, pinyin/zhuyin in Chinese) and increasingly for other languages. It must be inside a `<ruby>` element.

Syntax

```
<ruby>base<rt>annotation</rt></ruby>
```

Technical summary

Property	Value
Void element	No
Default display	ruby-text
Content categories	None
Permitted content	Phrasing content
Permitted parents	<code><ruby></code>
Tag omission	End tag may be omitted if followed by <code><rt></code> or <code><rp></code> or if there is no more content in the parent
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<rt>: worked examples

Japanese furigana

```
<ruby>〇〇<rt>〇〇</rt></ruby>
```

Arabic word with transliteration

```
<ruby lang="ar">كتاب<rt lang="en">kitāb</rt></ruby>
```

Styling

```
<style>
  rt { font-size: 0.6em; color: #666; }
</style>
```

Default browser CSS

```
rt { display: ruby-text; font-size: 50%; }
```

Accessibility

- Screen readers generally read only the base text; use <rt> for supplementary information, not essential content.

Common mistakes

- Using <rt> outside <ruby>.

See also

<ruby>, <rp>, <rb>, <rtc>

Quick card: <rtc>

Ruby text container (deprecated)

The <rtc> element grouped several <rt> annotations so that a base text could have two layers of annotation (for example pronunciation above and meaning below). Only Firefox implemented it, and the Living Standard removed it. Nest two <ruby> elements for double-sided annotations.

At a glance

Status	DEPRECATED
Versions	HTML 5.1; removed from the Living Standard
Category	Text-level semantics (ruby)
Closing tag	Required unless noted under Tag omission
Display	ruby-text-container
Own attributes	None (global attributes only)
Related	<ruby>, <rt>, <rb>

Minimal example

```
<ruby>□□□<rtc><rt>jiù jīn shān</rt></rtc><rtc><rt>San Francisco</rt></rtc></ruby>
```

Most common mistake

Relying on it.

<rtc> — Ruby text container (deprecated)

Category: Text-level semantics (ruby) | Status: DEPRECATED | Versions: HTML 5.1; removed from the Living Standard

Description

The `<rtc>` element grouped several `<rt>` annotations so that a base text could have two layers of annotation (for example pronunciation above and meaning below). Only Firefox implemented it, and the Living Standard removed it. Nest two `<ruby>` elements for double-sided annotations.

Syntax

`<ruby>base<rtc><rt>a</rt></rtc><rtc><rt>b</rt></rtc></ruby>`

Technical summary

Property	Value
Void element	No
Default display	ruby-text-container
Content categories	None
Permitted content	Phrasing content or <code><rt></code>
Permitted parents	<code><ruby></code>
Tag omission	End tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<rtc>: worked examples

Historical

```
<ruby>金<rtc><rt>jiù jīn shān</rt></rtc><rtc><rt>San Francisco</rt></rtc></ruby>
```

Default browser CSS

```
rtc { display: ruby-text-container; }
```

Common mistakes

- Relying on it.

See also

<ruby>, <rt>, <rb>

Quick card: <ruby>

Ruby annotation

The <ruby> element represents small annotations rendered above, below or beside base text, most often used to show the pronunciation of East Asian characters. The name comes from a small typographic size called 'ruby' used by British printers. The base text is written directly inside <ruby>, each annotation in an <rt>, and optional fallback parentheses in <rp>.

At a glance

Status	CURRENT
Versions	XHTML 1.1 (2001), HTML5
Category	Text-level semantics (ruby)
Closing tag	Required unless noted under Tag omission
Display	ruby
Own attributes	None (global attributes only)
Related	<rt>, <rp>, <rb>, <rtc>

Minimal example

```
<ruby>□<rp>(</rp><rt>□□</rt><rp>)</rp>□<rp>(</rp><rt>□□□</rt><rp>)</rp></ruby>
```

Most common mistake

Placing <rt> outside <ruby>.

<ruby> — Ruby annotation

Category: Text-level semantics (ruby) | Status: CURRENT | Versions: XHTML 1.1 (2001), HTML5

Description

The `<ruby>` element represents small annotations rendered above, below or beside base text, most often used to show the pronunciation of East Asian characters. The name comes from a small typographic size called 'ruby' used by British printers. The base text is written directly inside `<ruby>`, each annotation in an `<rt>`, and optional fallback parentheses in `<rp>`.

Syntax

```
<ruby>base<rp>(</rp><rt>annotation</rt><rp>)</rp></ruby>
```

Technical summary

Property	Value
Void element	No
Default display	ruby
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content, <code><rt></code> , <code><rp></code>
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<ruby>: worked examples

Furigana over each character

```
<ruby>□<rp>(</rp><rt>□□</rt><rp>)</rp>□<rp>(</rp><rt>□□□</rt><rp>)</rp></ruby>
```

Whole word annotation

```
<ruby>□□<rt>Běijīng</rt></ruby>
```

Annotation below with CSS

```
<style>
  ruby { ruby-position: under; }
</style>
```

Default browser CSS

```
ruby { display: ruby; }
```

Accessibility

- Do not put essential meaning only in the annotation.

Common mistakes

- Placing <rt> outside <ruby>.
- Forgetting <rp> if old browser support is needed.

See also

<rt>, <rp>, <rb>, <rtc>

Quick card: <s>

Strikethrough (no longer accurate)

The <s> element renders text with a line through it and represents content that is no longer accurate or no longer relevant, such as an old price next to the new one, or a sold-out item. In HTML 4 it was a purely presentational strike-through synonym for <strike> and was deprecated; HTML5 gave it this meaning and made it valid again while removing <strike>.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	, <strike>, <u>

Minimal example

```
<p>Price: <s>150 EGP</s> <strong>99 EGP</strong></p>
```

Most common mistake

Using <s> for deletions in a revision (use).

<s> — Strikethrough (no longer accurate)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

Description

The `<s>` element renders text with a line through it and represents content that is no longer accurate or no longer relevant, such as an old price next to the new one, or a sold-out item. In HTML 4 it was a purely presentational strike-through synonym for `<strike>` and was deprecated; HTML5 gave it this meaning and made it valid again while removing `<strike>`.

Do not use `<s>` to indicate document edits; that is what `<del>` is for.

Syntax

```
<s>outdated text</s>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<s>: worked examples

Old price

```
<p>Price: <s>150 EGP</s> <strong>99 EGP</strong></p>
```

Sold-out event

```
<p><s>Workshop: Saturday 10 am</s> (sold out)</p>
```

Accessible old price

```
<p>Price: <s><span class="sr-only">Was </span>150 EGP</s> <span  
class="sr-only">now </span>99 EGP</p>
```

Visually hidden words make the meaning clear to screen readers.

Default browser CSS

```
s { text-decoration: line-through; }
```

Accessibility

- The strike-through is not announced; add hidden text where the meaning matters.

Common mistakes

- Using <s> for deletions in a revision (use).
- Using <strike>.

See also

, <strike>, <u>

Quick card: `<samp>`

Sample output

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><code></code> , <code><kbd></code> , <code><pre></code> , <code><var></code>

Minimal example

`<p>If the file is missing you will see <samp>404 Not Found</samp>.</p>`

Most common mistake

Using `<code>` for output.

<samp> — Sample output

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<samp>` element marks sample or quoted output from a computer program or system: error messages, terminal output, log lines. Browsers render it in a monospace font. Nest `<kbd>` inside `<samp>` to show input echoed by the system, and `<samp>` inside `<kbd>` for input that names an on-screen item.

Syntax

```
<samp>program output</samp>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<samp>: worked examples

Error message

<p>If the file is missing you will see <samp>404 Not Found</samp>.</p>

Terminal session

```
<pre><samp>$ <kbd>ls</kbd>
index.html  style.css
$ </samp></pre>
```

Default browser CSS

```
samp { font-family: monospace; }
```

Common mistakes

- Using <code> for output.
- Using <samp> for user input.

See also

<code>, <kbd>, <pre>, <var>

Quick card: <script>

Script

The <script> element embeds executable code, almost always JavaScript, or links to an external script file with the src attribute. It can also hold data blocks (JSON, templates, import maps, speculation rules) that the browser does not execute when type is set to a non-script MIME type.

At a glance

Status	CURRENT
Versions	HTML 3.2 (reserved), HTML 4.01, HTML5
Category	Scripting
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	src, type, defer, async, nomodule, crossorigin, integrity, referrerpolicy, fetchpriority, blocking, nonce, attributionsrc, charset, language, event
Related	<noscript>, <template>, <link>, <style>

Minimal example

```
<head>
  <script src="app.js" defer></script>
</head>
```

Most common mistake

Placing a blocking script in head without defer, so the page appears slowly.

<script> — Script

Category: Scripting | Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

Description

The `<script>` element embeds executable code, almost always JavaScript, or links to an external script file with the `src` attribute. It can also hold data blocks (JSON, templates, import maps, speculation rules) that the browser does not execute when `type` is set to a non-script MIME type.

By default a script blocks HTML parsing: the browser stops, downloads and runs the script, then continues. To avoid slowing down the page, put scripts at the end of `<body>` or, better, in `<head>` with the `defer` attribute (run after parsing, in order) or `async` (run as soon as downloaded, in any order). Module scripts (`type="module"`) are deferred automatically and support `import/export`.

An element with `src` must be empty; inline code and `src` cannot be combined. Inside an inline script the sequence `</script>` ends the element even inside a string, so write it as `'<\script>'` in JavaScript strings.

Syntax

```
<script src="app.js" defer></script>
<script>
  // inline code
</script>
```

Technical summary

Property	Value
Void element	No

Property	Value
Default display	none
Content categories	Metadata content, flow content, phrasing content, script-supporting element
Permitted content	Script code or data, depending on type; nothing if src is present
Permitted parents	Any element that accepts metadata or phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLScriptElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	External script file.	HTML 4.01
type	omitted module importmap speculationrules MIME type	Omitted or text/javascript: classic script. module: ES module. importmap: JSON import map. Any other type: data block that is not executed.	HTML 4.01 / HTML5
defer	boolean	Download in parallel, execute after the document has been parsed, in document order. Only for external classic scripts.	HTML 4.01
async	boolean	Download in parallel, execute as soon as ready, possibly out of order. For independent scripts such as analytics.	HTML5

Attribute	Values	Description	Since
nomodule	boolean	Do not run in browsers that support modules; used to provide fallbacks for old browsers.	HTML5
crossorigin	anonymous use-credentials	CORS setting; needed to get full error details from cross-origin scripts.	HTML5
integrity	hash	Subresource Integrity hash.	SRI
referrerpolicy	referrer policy	Referrer to send.	HTML5
fetchpriority	high low auto	Download priority hint.	Living Standard
blocking	render	Block rendering until the script runs.	Living Standard
nonce	text	Cryptographic nonce for Content Security Policy (global attribute).	CSP
attribution src	empty or URL	Experimental attribution reporting.	Experimental
charset	encoding	Obsolete. Encoding of the script; use UTF-8 for everything.	HTML 4.01
language	text	Obsolete. Scripting language name. Use type or omit.	HTML 3.2
event, for	text	Obsolete. Internet Explorer event binding.	HTML 4.01

<script>: attribute by attribute

Each attribute of <script> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML 4.01

External script file.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script src="https://example.com/page.html">...</script>
```

type

Values: omitted | module | importmap | speculationrules | MIME type | Since: HTML 4.01 / HTML5

Omitted or text/javascript: classic script. module: ES module. importmap: JSON import map. Any other type: data block that is not executed.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script type="text/plain">...</script>
```

defer

Values: boolean | Since: HTML 4.01

Download in parallel, execute after the document has been parsed, in document order. Only for external classic scripts.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<script defer>...</script>
```

async

Values: boolean | Since: HTML5

Download in parallel, execute as soon as ready, possibly out of order. For independent scripts such as analytics.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<script async>...</script>
```

nomodule

Values: boolean | Since: HTML5

Do not run in browsers that support modules; used to provide fallbacks for old browsers.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<script nomodule>...</script>
```

crossorigin

Values: anonymous | use-credentials | Since: HTML5

CORS setting; needed to get full error details from cross-origin scripts.

Write `crossorigin="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script crossorigin="anonymous">...</script>
```

integrity

Values: hash | Since: SRI

Subresource Integrity hash.

Write `integrity="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script integrity="hash">...</script>
```

referrerpolicy

Values: referrer policy | Since: HTML5

Referrer to send.

Write `referrerpolicy="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script referrerpolicy="referrer policy">...</script>
```

fetchpriority

Values: high | low | auto | Since: Living Standard

Download priority hint.

Write `fetchpriority="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script fetchpriority="high">...</script>
```

blocking

Values: render | Since: Living Standard

Block rendering until the script runs.

Write `blocking="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script blocking="render">...</script>
```

nonce

Values: text | Since: CSP

Cryptographic nonce for Content Security Policy (global attribute).

Write `nonce="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script nonce="value">...</script>
```

attributionsrc

Values: empty or URL | Since: Experimental

Experimental attribution reporting.

Write `attributionsrc="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script attributionsrc="empty">...</script>
```

charset

Values: encoding | Since: HTML 4.01

Obsolete. Encoding of the script; use UTF-8 for everything.

Write `charset="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script charset="encoding">...</script>
```

language

Values: text | Since: HTML 3.2

Obsolete. Scripting language name. Use type or omit.

Write `language="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script language="value">...</script>
```

event, for

Values: text | Since: HTML 4.01

Obsolete. Internet Explorer event binding.

Write `event="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<script event="value">...</script>
```

<script>: worked examples

Recommended: deferred external script in head

```
<head>
  <script src="app.js" defer></script>
</head>
```

The script downloads while the HTML parses and runs when the DOM is ready.

Inline script

```
<script>
  document.querySelector('footer').textContent = '© ' + new Date().
getFullYear();
</script>
```

ES module

```
<script type="module">
  import { greet } from './greet.js';
  greet('world');
</script>
```

Async analytics

```
<script src="https://analytics.example.com/tag.js" async></script>
```

JSON data block

```
<script id="config" type="application/json">
  { "currency": "EGP", "vat": 14 }
</script>
<script>
  const cfg = JSON.parse(document.getElementById('config').
textContent);
</script>
```

Structured data for search engines

```
<script type="application/ld+json">
{ "@context": "https://schema.org", "@type": "Book", "name": "Learn
HTML" }
</script>
```

Import map

```
<script type="importmap">
{ "imports": { "lodash": "https://cdn.example.com/lodash.js" } }
</script>
```

HTML 4 style with comment hiding (no longer needed)

```
<script type="text/javascript">
<!--
  alert('old style');
//-->
</script>
```

The comment trick hid code from 1995 browsers that did not know `<script>`.

Default browser CSS

```
script { display: none; }
```

Accessibility

- Content generated by scripts must be as accessible as static content: manage focus, announce changes with live regions, keep keyboard support.

Common mistakes

- Placing a blocking script in head without `defer`, so the page appears slowly.
- Running DOM code before the elements exist (fix with `defer` or `DOMContentLoaded`).
- Writing `<script src="x.js" />` (self-closing is not allowed; everything after it becomes script).
- Putting `</script>` inside a string.

See also

`<noscript>`, `<template>`, `<link>`, `<style>`

Quick card: <search>

Search section

The <search> element represents a part of the document whose contents are search or filtering controls: a site search form, a filter panel in a product list, or the controls of a find-in-page feature. It has no visual effect; it exposes the 'search' landmark to assistive technology. It replaces the old pattern <form role="search">. The form itself still goes inside it.

At a glance

Status	CURRENT
Versions	Living Standard (2023)
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<form>, <input>, <nav>

Minimal example

```
<search>
  <form action="/search">
    <label for="q">Search books</label>
    <input id="q" name="q" type="search">
    <button>Search</button>
  </form>
</search>
```

Most common mistake

Using <search> for the search results (it is for the controls).

<search> — Search section

Category: Content sectioning | Status: CURRENT | Versions: Living Standard (2023)

Description

The <search> element represents a part of the document whose contents are search or filtering controls: a site search form, a filter panel in a product list, or the controls of a find-in-page feature. It has no visual effect; it exposes the 'search' landmark to assistive technology. It replaces the old pattern <form role="search">. The form itself still goes inside it.

Syntax

```
<search>
  <form>...</form>
</search>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, palpable content
Permitted content	Flow content
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<search>: worked examples

Site search

```
<search>
  <form action="/search">
    <label for="q">Search books</label>
    <input id="q" name="q" type="search">
    <button>Search</button>
  </form>
</search>
```

Filter panel

```
<search aria-label="Filter products">
  <label><input type="checkbox" name="instock"> In stock only</label>
  <label>Max price <input type="range" name="max" min="0" max="500"></
label>
</search>
```

Default browser CSS

```
search { display: block; }
```

Accessibility

- Exposed as the 'search' landmark; screen reader users can jump straight to it.
- In older browsers add `role="search"` to the `<search>` element for the same effect.

Common mistakes

- Using `<search>` for the search results (it is for the controls).
- Nesting several search landmarks without labels.

See also

`<form>`, `<input>`, `<nav>`

Quick card: <section>

Generic section

The <section> element represents a generic, thematic grouping of content, typically with a heading. Examples: the chapters of an article, the tabs of a tabbed interface, the 'Introduction', 'Features' and 'Pricing' areas of a landing page. Each section should be identifiable, usually by an <h2>-<h6> heading as its first child.

At a glance

Status	CURRENT
Versions	HTML5
Category	Content sectioning
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<article>, <aside>, <nav>, <div>, <h1>, <main>

Minimal example

```
<main>
  <section id="features">
    <h2>Features</h2>
    <p>...</p>
  </section>
  <section id="pricing">
    <h2>Pricing</h2>
    <p>...</p>
  </section>
</main>
```

Most common mistake

Using <section> as a styling wrapper instead of <div>.

<section> — Generic section

Category: Content sectioning | Status: CURRENT | Versions: HTML5

Description

The `<section>` element represents a generic, thematic grouping of content, typically with a heading. Examples: the chapters of an article, the tabs of a tabbed interface, the 'Introduction', 'Features' and 'Pricing' areas of a landing page. Each section should be identifiable, usually by an `<h2>`-`<h6>` heading as its first child.

Choosing between the sectioning elements: use `<article>` if the content is self-contained and could be syndicated; `<nav>` for navigation; `<aside>` for tangential content; `<section>` when the content is a distinct part of the page with its own heading; and `<div>` when the grouping is only for styling or scripting and has no meaning. A `<section>` without a heading is usually a sign that `<div>` is more appropriate.

Syntax

```
<section>
  <h2>Heading</h2>
  ...
</section>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content, sectioning content, palpable content

Property	Value
Permitted content	Flow content
Permitted parents	Any element that accepts flow content, but not <address>
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<section>: worked examples

Landing page sections

```
<main>
  <section id="features">
    <h2>Features</h2>
    <p>...</p>
  </section>
  <section id="pricing">
    <h2>Pricing</h2>
    <p>...</p>
  </section>
</main>
```

Sections inside an article

```
<article>
  <h1>Learn HTML</h1>
  <section>
    <h2>Part 1: Tags</h2>
    <p>...</p>
  </section>
  <section>
    <h2>Part 2: Attributes</h2>
    <p>...</p>
  </section>
</article>
```

Labelled section as a region landmark

```
<section aria-labelledby="faq-h">
  <h2 id="faq-h">FAQ</h2>
  ...
</section>
```

A section with an accessible name is exposed as a 'region' landmark.

Default browser CSS

```
section { display: block; }
```

Accessibility

- Sections become 'region' landmarks only when they have an accessible name (aria-labelledby or aria-label).
- Always include a heading.

Common mistakes

- Using `<section>` as a styling wrapper instead of `<div>`.
- Sections without headings.
- Assuming `<section>` resets heading levels (it does not in practice; use the correct h-level).

See also

`<article>`, `<aside>`, `<nav>`, `<div>`, `<h1>`, `<main>`

Quick card: <select>

Drop-down / list box

The <select> element creates a control for choosing one option (or, with multiple, several) from a list. It contains <option> elements, optionally grouped in <optgroup>. By default it appears as a drop-down menu; with the multiple attribute or a size greater than 1 it appears as a scrolling list box. The name attribute and the value of the chosen option(s) are submitted with the form.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	name, multiple, size, required, disabled, autofocus, autocomplete, form
Related	<option>, <optgroup>, <datalist>, <input>, <label>

Minimal example

```
<label for="pay">Payment</label>
<select id="pay" name="pay" required>
  <option value="">-- Select --</option>
  <option value="cod">Cash on delivery</option>
  <option value="wallet">Mobile wallet</option>
</select>
```

Most common mistake

No empty placeholder option with required, so the first real option is submitted silently.

<select> — Drop-down / list box

Category: Forms | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <select> element creates a control for choosing one option (or, with multiple, several) from a list. It contains <option> elements, optionally grouped in <optgroup>. By default it appears as a drop-down menu; with the multiple attribute or a size greater than 1 it appears as a scrolling list box. The name attribute and the value of the chosen option(s) are submitted with the form.

A select is best for lists of roughly 5 to 30 fixed choices. For 2 to 4 choices radio buttons are easier to see at a glance; for very long lists or free text with suggestions use <input> with a <datalist>. Newer browsers allow a customisable select (CSS appearance: base-select) with rich option content.

Syntax

```
<select name="n">
  <option value="1">One</option>
</select>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Flow content, phrasing content, interactive content, listed, labelable, resettable, submittable, form-associated, palpable

Property	Value
Permitted content	Zero or more <option>, <optgroup>, <hr>, <script>, <template>; optionally a <button> first for customisable selects
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLSelectElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
name	text	Name submitted with the form.	HTML 2.0
multiple	boolean	Allow several options to be selected (Ctrl/Cmd-click).	HTML 2.0
size	integer	Number of visible rows. 1 (default) gives a drop-down; more gives a list box.	HTML 2.0
required	boolean	The user must choose an option with a non-empty value.	HTML5
disabled	boolean	The control cannot be used.	HTML 4.01
autofocus	boolean	Focus on load.	HTML5
autocomplete	token	Autofill hint, for example country, cc-exp-month.	HTML5
form	form id	Associated form.	HTML5

<select>: attribute by attribute

Each attribute of <select> explained in detail, with a minimal example you can copy.

name

Values: text | Since: HTML 2.0

Name submitted with the form.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<select name="value">...</select>
```

multiple

Values: boolean | Since: HTML 2.0

Allow several options to be selected (Ctrl/Cmd-click).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<select multiple>...</select>
```

size

Values: integer | Since: HTML 2.0

Number of visible rows. 1 (default) gives a drop-down; more gives a list box.

Write size="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<select size="3">...</select>
```

required

Values: boolean | Since: HTML5

The user must choose an option with a non-empty value.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<select required>...</select>
```

disabled

Values: boolean | Since: HTML 4.01

The control cannot be used.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<select disabled>...</select>
```

autofocus

Values: boolean | Since: HTML5

Focus on load.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<select autofocus>...</select>
```

autocomplete

Values: token | Since: HTML5

Autofill hint, for example country, cc-exp-month.

Write `autocomplete="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<select autocomplete="token">...</select>
```

form

Values: form id | Since: HTML5

Associated form.

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<select form="form id">...</select>
```

<select>: worked examples

Drop-down with placeholder

```
<label for="pay">Payment</label>
<select id="pay" name="pay" required>
  <option value="">-- Select --</option>
  <option value="cod">Cash on delivery</option>
  <option value="wallet">Mobile wallet</option>
</select>
```

Multiple selection list box

```
<label for="langs">Languages you read</label>
<select id="langs" name="langs" multiple size="4">
  <option>Arabic</option>
  <option>English</option>
  <option>French</option>
  <option>German</option>
</select>
```

Submitted as langs=Arabic&langs=English.

Reading the value in JavaScript

```
<select id="s"><option value="a">A</option><option value="b">B</option></select>
<script>
  document.getElementById('s').addEventListener('change', e =>
    console.log(e.target.value));
</script>
```

Basic styling

```
<style>
  select { padding: 8px 12px; border: 1px solid #ccc; border-radius:
6px; font: inherit; }
</style>
```

Default browser CSS

```
select { display: inline-block; }
```

Accessibility

- Always label the select.
- Do not trigger navigation on change alone; keyboard users change the value with arrow keys and would be redirected before reaching their choice. Provide a Go button.

- Multiple selects are hard to use; consider a group of checkboxes.

Common mistakes

- No empty placeholder option with required, so the first real option is submitted silently.
- Using select for 2 choices (use radio buttons).
- Putting HTML inside <option>.

See also

<option>, <optgroup>, <datalist>, <input>, <label>

Quick card: <selectedcontent>

Selected option display (customisable select)

The <selectedcontent> element is part of the customisable <select> proposal. Placed inside the <button> that sits at the start of a <select>, it mirrors the content of the currently selected <option>, so that rich option content (icons, colour swatches, descriptions) is shown in the closed control as well as in the open list. The browser clones the selected option's children into <selectedcontent> whenever the selection changes.

At a glance

Status	EXPERIMENTAL
Versions	Living Standard (2025, experimental)
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<select>, <option>, <button>

Minimal example

```
<style>
  select, ::picker(select) { appearance: base-select; }
</style>
<select>
  <button><selectedcontent></selectedcontent></button>
  <option value="red"><span class="swatch" style="background:red"></span> Red</option>
  <option value="green"><span class="swatch" style="background:green"></span> Green</option>
</select>
```


Most common mistake

Placing it anywhere other than inside the leading `<button>` of a `<select>`.

<selectedcontent> — Selected option display (customisable select)

Category: Forms | Status: EXPERIMENTAL | Versions: Living Standard (2025, experimental)

Description

The `<selectedcontent>` element is part of the customisable `<select>` proposal. Placed inside the `<button>` that sits at the start of a `<select>`, it mirrors the content of the currently selected `<option>`, so that rich option content (icons, colour swatches, descriptions) is shown in the closed control as well as in the open list. The browser clones the selected option's children into `<selectedcontent>` whenever the selection changes.

It only has an effect when the select opts in with the CSS declaration `appearance: base-select` on both the select and its `::picker(select)` pseudo-element. Browser support is still limited, so always test the fallback: browsers that do not understand it render the plain native select.

Syntax

```
<select>
  <button><selectedcontent></selectedcontent></button>
  <option>...</option>
</select>
```

Technical summary

Property	Value
Void element	No
Default display	inline

Property	Value
Content categories	None
Permitted content	Nothing authored; the browser fills it with a clone of the selected option's content
Permitted parents	A <code><button></code> that is the first child of a <code><select></code>
Tag omission	Neither tag may be omitted
DOM interface	HTMLSelectedContentElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<selectedcontent>: worked examples

Rich select with icons

```
<style>
  select, ::picker(select) { appearance: base-select; }
</style>
<select>
  <button><selectedcontent></selectedcontent></button>
  <option value="red"><span class="swatch" style="background:red"></
span> Red</option>
  <option value="green"><span class="swatch" style="background:
green"></span> Green</option>
</select>
```

The closed control shows the swatch and text of the chosen option.

Default browser CSS

```
selectedcontent { display: contents; }
```

Accessibility

- The control remains a combobox for assistive technology; the option text is announced as usual.

Common mistakes

- Placing it anywhere other than inside the leading <button> of a <select>.
- Relying on it without the appearance: base-select opt-in.
- Assuming support in every browser.

See also

<select>, <option>, <button>

Quick card: <shadow>

Shadow root insertion point (obsolete)

The <shadow> element was part of Shadow DOM v0 and marked where an older shadow root should be rendered inside a newer one. It was removed with Shadow DOM v1, which does not support multiple shadow roots per element. Use <slot>.

At a glance

Status	OBSOLETE
Versions	Shadow DOM v0; obsolete
Category	Obsolete web components
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<slot>, <template>, <content>

Minimal example

```
<slot></slot>
```

Most common mistake

Any use.

<shadow> — Shadow root insertion point (obsolete)

Category: Obsolete web components | Status: OBSOLETE | Versions: Shadow DOM v0; obsolete

Description

The <shadow> element was part of Shadow DOM v0 and marked where an older shadow root should be rendered inside a newer one. It was removed with Shadow DOM v1, which does not support multiple shadow roots per element. Use <slot>.

Syntax

```
<shadow></shadow>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Historical
Permitted content	Any
Permitted parents	Shadow root
Tag omission	Neither
DOM interface	HTMLUnknownElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<shadow>: worked examples

Replacement

`<slot></slot>`

Common mistakes

- Any use.

See also

`<slot>`, `<template>`, `<content>`

Quick card: <slot>

Shadow DOM slot

The <slot> element is a placeholder inside a web component's shadow tree that is filled with content from the light DOM (the markup written by the user of the component). A named slot receives children that have a matching slot attribute; the unnamed default slot receives all other children. Content inside the <slot> element itself is fallback content shown when nothing is slotted.

At a glance

Status	CURRENT
Versions	HTML5 / DOM Standard (2016)
Category	Web components
Closing tag	Required unless noted under Tag omission
Display	contents
Own attributes	name
Related	<template>, <content>, <shadow>

Minimal example

```
<template id="card">
  <style>.card { border: 1px solid #ccc; padding: 12px; }</style>
  <div class="card">
    <h3><slot name="name">Anonymous</slot></h3>
    <p><slot>No description</slot></p>
  </div>
</template>

<user-card>
  <span slot="name">Ahmed</span>
  Front-end developer from Giza.
</user-card>
```

```
<script>
  customElements.define('user-card', class extends HTMLElement {
    constructor() {
      super();
      this.attachShadow({ mode: 'open' }).append(document.
getElementById('card').content.cloneNode(true));
    }
  });
</script>
```

Most common mistake

Using `<slot>` outside a shadow root (it does nothing).

<slot> — Shadow DOM slot

Category: Web components | Status: CURRENT | Versions: HTML5 / DOM Standard (2016)

Description

The `<slot>` element is a placeholder inside a web component's shadow tree that is filled with content from the light DOM (the markup written by the user of the component). A named slot receives children that have a matching `slot` attribute; the unnamed default slot receives all other children. Content inside the `<slot>` element itself is fallback content shown when nothing is slotted.

Syntax

```
<slot name="title">Default title</slot>
```

Technical summary

Property	Value
Void element	No
Default display	contents
Content categories	Flow content, phrasing content
Permitted content	Transparent (fallback content)
Permitted parents	Any element that accepts phrasing content (inside a shadow tree)
Tag omission	Neither tag may be omitted
DOM interface	HTMLSlotElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
name	text	The slot's name. Children with slot="name" are placed here. Omit for the default slot.	DOM

<slot>: attribute by attribute

Each attribute of <slot> explained in detail, with a minimal example you can copy.

name

Values: text | Since: DOM

The slot's name. Children with slot="name" are placed here.
Omit for the default slot.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<slot name="value">...</slot>
```

<slot>: worked examples

A user-card component

```
<template id="card">
  <style>.card { border: 1px solid #ccc; padding: 12px; }</style>
  <div class="card">
    <h3><slot name="name">Anonymous</slot></h3>
    <p><slot>No description</slot></p>
  </div>
</template>

<user-card>
  <span slot="name">Ahmed</span>
  Front-end developer from Giza.
</user-card>

<script>
  customElements.define('user-card', class extends HTMLElement {
    constructor() {
      super();
      this.attachShadow({ mode: 'open' }).append(document.
        getElementById('card').content.cloneNode(true));
    }
  });
</script>
```

Styling slotted content

```
<style>
  ::slotted(span) { color: #b90000; }
</style>
```

Default browser CSS

```
slot { display: contents; }
```

Accessibility

- Slotted content keeps its own semantics; the shadow boundary is transparent to the accessibility tree.

Common mistakes

- Using <slot> outside a shadow root (it does nothing).
- Slot attribute on a nested descendant rather than a direct child of the host.

See also

<template>, <content>, <shadow>

Quick card: <small>

Side comment (fine print)

The <small> element represents side comments and small print: disclaimers, caveats, legal restrictions, copyright notices, attribution. It renders the text one size smaller. In HTML 3.2 and 4 it was purely presentational ('smaller text'); HTML5 kept it and gave it this meaning, while its counterpart <big> was removed.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5 (redefined)
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<big>, <sub>, <sup>, <footer>

Minimal example

```
<footer>
  <p><small>&copy; 2026 Book Store. All rights reserved.</small></p>
</footer>
```

Most common mistake

Wrapping entire paragraphs or sections in <small> for font size.

<small> — Side comment (fine print)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5 (redefined)

Description

The `<small>` element represents side comments and small print: disclaimers, caveats, legal restrictions, copyright notices, attribution. It renders the text one size smaller. In HTML 3.2 and 4 it was purely presentational ('smaller text'); HTML5 kept it and gave it this meaning, while its counterpart `<big>` was removed.

`<small>` does not reduce the importance of text. It is intended for short runs of inline text, not for whole sections; for that, use CSS font-size.

Syntax

`<small>fine print</small>`

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<small>: worked examples

Copyright in the footer

```
<footer>
  <p><small>&copy; 2026 Book Store. All rights reserved.</small></p>
</footer>
```

Price disclaimer

```
<p>From 99 EGP <small>(excluding delivery)</small></p>
```

Default browser CSS

```
small { font-size: smaller; }
```

Accessibility

- No special announcement; the meaning is visual and semantic only.

Common mistakes

- Wrapping entire paragraphs or sections in <small> for font size.
- Using <small> for sub-headings.

See also

<big>, <sub>, <sup>, <footer>

Quick card: <source>

Media or image source

The <source> element specifies one of several alternative resources for a <picture>, <audio> or <video> element. It is a void element. The browser examines the sources in order and uses the first one it can display: for <picture> it checks the media query and image type, for <audio>/<video> it checks the type (MIME type and codecs). Because it is used inside three different parents, the set of relevant attributes differs: <picture> uses srcset, sizes, media, type, width and height; media elements use src, type and (for video) media.

At a glance

Status	CURRENT
Versions	HTML5
Category	Image and multimedia
Closing tag	None (void element)
Display	none
Own attributes	src, srcset, sizes, type, media, width
Related	<picture>, <audio>, <video>, , <track>

Minimal example

```
<video controls width="640" height="360">
  <source src="clip.webm" type="video/webm">
  <source src="clip.mp4" type="video/mp4">
  <p>Download the <a href="clip.mp4">video</a>.</p>
</video>
```

Most common mistake

Using src inside <picture> (must be srcset).

<source> — Media or image source

Category: Image and multimedia | Status: CURRENT | Versions: HTML5

Description

The <source> element specifies one of several alternative resources for a <picture>, <audio> or <video> element. It is a void element. The browser examines the sources in order and uses the first one it can display: for <picture> it checks the media query and image type, for <audio>/<video> it checks the type (MIME type and codecs). Because it is used inside three different parents, the set of relevant attributes differs: <picture> uses srcset, sizes, media, type, width and height; media elements use src, type and (for video) media.

Syntax

```
<source src="file" type="MIME"> <!-- in audio/video -->
<source srcset="file" type="MIME" media="query"> <!-- in picture -->
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	None
Permitted content	None (void element)
Permitted parents	<picture> (before the), <audio>, <video> (before any <track> or flow content)
Tag omission	No end tag

Property	Value
DOM interface	HTMLSourceElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The media file. Required in <code><audio>/<video></code> ; ignored in <code><picture></code> .	HTML5
srcset	candidate list	Image candidates. Required in <code><picture></code> ; ignored in media elements.	HTML5
sizes	size list	Display widths for width-descriptor srcset (picture only).	HTML5
type	MIME type, optionally with codecs	The resource type, for example <code>image/webp</code> , <code>video/mp4</code> , <code>video/webm</code> ; <code>codecs="vp9, opus"</code> . Lets the browser skip unsupported files without downloading them.	HTML5
media	media query	Use this source only when the query matches. Valid in picture and video (audio ignores it in most browsers).	HTML5
width / height	pixels	Intrinsic size of the image candidate (picture only).	Living Standard

<source>: attribute by attribute

Each attribute of <source> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML5

The media file. Required in <audio>/<video>; ignored in <picture>.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source src="https://example.com/page.html">
```

srcset

Values: candidate list | Since: HTML5

Image candidates. Required in <picture>; ignored in media elements.

Write srcset="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source srcset="candidate list">
```

sizes

Values: size list | Since: HTML5

Display widths for width-descriptor srcset (picture only).

Write sizes="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source sizes="size list">
```

type

Values: MIME type, optionally with codecs | Since: HTML5

The resource type, for example image/webp, video/mp4, video/webm; codecs="vp9, opus". Lets the browser skip unsupported files without downloading them.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source type="text/plain">
```

media

Values: media query | Since: HTML5

Use this source only when the query matches. Valid in picture and video (audio ignores it in most browsers).

Write media="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source media="media query">
```

width / height

Values: pixels | Since: Living Standard

Intrinsic size of the image candidate (picture only).

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<source width="3">
```


<source>: worked examples

Video with two formats

```
<video controls width="640" height="360">
  <source src="clip.webm" type="video/webm">
  <source src="clip.mp4" type="video/mp4">
  <p>Download the <a href="clip.mp4">video</a>.</p>
</video>
```

Picture with format and size choice

```
<picture>
  <source srcset="img.avif" type="image/avif">
  <source srcset="img.webp" type="image/webp">
  
</picture>
```

Different video for small screens

```
<video controls>
  <source src="small.mp4" type="video/mp4" media="(max-width: 600px)">
  <source src="large.mp4" type="video/mp4">
</video>
```

Default browser CSS

```
source { display: none; }
```

Accessibility

- Accessibility attributes (alt, captions) go on the , <video> or <track>, not on <source>.

Common mistakes

- Using src inside <picture> (must be srcset).
- Placing <source> after the or after <track>.
- Omitting type, forcing the browser to download files to test them.

See also

<picture>, <audio>, <video>, , <track>

Quick card: `<spacer>`

Spacer (non-standard, obsolete)

The `<spacer>` element inserted empty horizontal or vertical space, imitating the invisible 'spacer GIF' images used for layout in the 1990s. It existed only in Netscape and was never standardised. Use CSS margin, padding or gap.

At a glance

Status	OBSOLETE
Versions	Netscape 3-4 only
Category	Obsolete presentational
Closing tag	None (void element)
Display	none
Own attributes	type, size, width
Related	<code>
</code> , <code><hr></code>

Minimal example

```
<div style="margin-top: 20px">...</div>
```

Most common mistake

Any use.

<spacer> — Spacer (non-standard, obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: Netscape 3-4 only

Description

The <spacer> element inserted empty horizontal or vertical space, imitating the invisible 'spacer GIF' images used for layout in the 1990s. It existed only in Netscape and was never standardised. Use CSS margin, padding or gap.

Syntax

```
<spacer type="horizontal" size="20">
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	Historical
Permitted content	None
Permitted parents	Any
Tag omission	No end tag
DOM interface	HTMLUnknownElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
type	horizontal vertical block	Kind of space.	Netscape
size	pixels	Amount of space.	Netscape
width / height / align	-	For type=block.	Netscape

<spacer>: attribute by attribute

Each attribute of <spacer> explained in detail, with a minimal example you can copy.

type

Values: horizontal | vertical | block | Since: Netscape

Kind of space.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<spacer type="horizontal">
```

size

Values: pixels | Since: Netscape

Amount of space.

Write size="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<spacer size="3">
```

width / height / align

Values: - | Since: Netscape

For type=block.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<spacer width="- ">
```

<spacer>: worked examples

CSS replacement

```
<div style="margin-top: 20px">...</div>
```

Common mistakes

- Any use.

See also

, <hr>

Quick card: `<span>`

Generic inline container

The `<span>` element is a generic inline container for phrasing content. Like `<div>`, it has no meaning of its own and no visual effect; it exists to be a hook for CSS (via class or id) or JavaScript, or to carry attributes such as lang or dir on a part of a sentence. It should be used only when no other element is more appropriate: `<em>`, `<strong>`, `<mark>`, `<time>`, `<abbr>`, `<code>` and the other text-level elements all carry meaning that `<span>` does not.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><div></code> , <code></code> , <code></code> , <code><mark></code> , <code></code> , <code><i></code>

Minimal example

```
<style>
  .price { color: #e60000; font-weight: bold; }
</style>
<p>The price is <span class="price">50 EGP</span> only.</p>
```

Most common mistake

Using `<span>` where `<em>`, `<strong>` or another semantic element fits.

 — Generic inline container

Category: Text-level semantics | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<span>` element is a generic inline container for phrasing content. Like `<div>`, it has no meaning of its own and no visual effect; it exists to be a hook for CSS (via class or id) or JavaScript, or to carry attributes such as lang or dir on a part of a sentence. It should be used only when no other element is more appropriate: `<em>`, `<strong>`, `<mark>`, `<time>`, `<abbr>`, `<code>` and the other text-level elements all carry meaning that `<span>` does not.

`<span>` is inline: it does not start a new line and it takes only as much width as its content. `<div>` is its block-level counterpart.

Syntax

```
<span class="name">text</span>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLSpanElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

: worked examples

Styling part of a sentence

```
<style>
  .price { color: #e60000; font-weight: bold; }
</style>
<p>The price is <span class="price">50 EGP</span> only.</p>
```

Only '50 EGP' turns red; the sentence stays on one line.

Marking a language change

```
<p>The Arabic word for book is <span lang="ar">كتاب</span>.</p>
```

A hook for JavaScript

```
<p>Items in cart: <span id="count">0</span></p>
<script>document.getElementById('count').textContent = 3;</script>
```

Visually hidden text for screen readers

```
<style>
  .sr-only { position: absolute; width: 1px; height: 1px; overflow:
  hidden; clip: rect(0 0 0 0); white-space: nowrap; }
</style>
<button>x<span class="sr-only">Close</span></button>
```

Default browser CSS

```
span { display: inline; }
```

Accessibility

- A span has no role; never make it clickable without adding `role="button"`, `tabindex="0"` and keyboard handlers, and even then a real `<button>` is better.

Common mistakes

- Using `<span>` where `<em>`, `<strong>` or another semantic element fits.
- Putting block elements inside a span.
- Using `<span>` as a button.

See also

`<div>`, `<em>`, `<strong>`, `<mark>`, `<b>`, `<i>`

Quick card: <strike>

Strikethrough (obsolete)

The <strike> element drew a line through its text. It was purely presentational, was deprecated in HTML 4.01 and removed in HTML5. Use for removed content, <s> for content that is no longer accurate, or CSS text-decoration: line-through for pure styling.

At a glance

Status	OBSOLETE
Versions	HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<s>,

Minimal example

```
<!-- Old -->
<strike>150 EGP</strike>
```

```
<!-- New: no longer accurate -->
<s>150 EGP</s>
```

```
<!-- New: edit tracking -->
<del>150 EGP</del>
```

Most common mistake

Any use.

<strike> — Strikethrough (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 3.2, HTML 4.01 (deprecated); obsolete in HTML5

Description

The `<strike>` element drew a line through its text. It was purely presentational, was deprecated in HTML 4.01 and removed in HTML5. Use `<del>` for removed content, `<s>` for content that is no longer accurate, or CSS text-decoration: line-through for pure styling.

Syntax

```
<strike>text</strike>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Historical
Permitted content	Phrasing content
Permitted parents	Phrasing content
Tag omission	Neither
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<strike>: worked examples

Replacements

```
<!-- Old -->  
<strike>150 EGP</strike>
```

```
<!-- New: no longer accurate -->  
<s>150 EGP</s>
```

```
<!-- New: edit tracking -->  
<del>150 EGP</del>
```

Default browser CSS

```
strike { text-decoration: line-through; }
```

Common mistakes

- Any use.

See also

<s>,

Quick card: `<strong>`

Strong importance

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code></code> , <code></code> , <code><mark></code> , <code><i></code>

Minimal example

```
<p><strong>Warning:</strong> unplug the device before opening it.</p>
```

Most common mistake

Using `<strong>` just for bold styling.

 — Strong importance

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<strong>` element indicates that its content has strong importance, seriousness or urgency: a warning, a key instruction, the most important part of a sentence. Browsers render it in bold. Nesting `<strong>` inside `<strong>` increases the importance further.

In HTML 4 the guidance was 'use `<strong>` instead of `<b>`'; HTML5 refined it. `<strong>` = important; `<b>` = draw attention without importance (keywords); `<em>` = spoken stress that changes meaning; `<mark>` = relevance to the reader's current task. A whole paragraph of `<strong>` text signals nothing because nothing stands out.

Syntax

```
<strong>important text</strong>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted

Property	Value
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

: worked examples

Warning

`<p><strong>Warning:</strong> unplug the device before opening it.</p>`

Key instruction in a list

```
<ul>
  <li>Save the file as <strong>index.html</strong>.</li>
  <li>Open it in a browser.</li>
</ul>
```

strong versus em

`<p><strong>Do not</strong> touch the wire. I said <em>do not</em>.</p>`

The first is a serious instruction; the second stresses the words.

Default browser CSS

```
strong { font-weight: bold; }
```

Accessibility

- Most screen readers do not announce `<strong>` by default, but it is the correct semantic element and some users enable emphasis announcements.

Common mistakes

- Using `<strong>` just for bold styling.
- Using `<strong>` for headings.
- Wrapping large blocks in `<strong>`.

See also

`<b>`, `<em>`, `<mark>`, `<i>`

Quick card: <style>

Embedded style sheet

The <style> element contains CSS rules that apply to the document. It belongs in <head>, though browsers tolerate it in <body>. A document may have any number of <style> elements; they are applied in order together with any linked style sheets. For anything larger than a few rules, an external file loaded with <link rel="stylesheet"> is better because it can be cached and shared between pages.

At a glance

Status	CURRENT
Versions	HTML 3.2 (reserved), HTML 4.01, HTML5
Category	Document metadata
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	media, blocking, nonce, title, type, scoped
Related	<link>, <head>, <script>

Minimal example

```
<head>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; }
    footer { text-align: center; color: gray; padding: 20px; }
  </style>
</head>
```

Most common mistake

Putting CSS in <style> without the tags, or putting HTML inside <style>.

<style> — Embedded style sheet

Category: Document metadata | Status: CURRENT | Versions: HTML 3.2 (reserved), HTML 4.01, HTML5

Description

The `<style>` element contains CSS rules that apply to the document. It belongs in `<head>`, though browsers tolerate it in `<body>`. A document may have any number of `<style>` elements; they are applied in order together with any linked style sheets. For anything larger than a few rules, an external file loaded with `<link rel="stylesheet">` is better because it can be cached and shared between pages.

The media attribute restricts the rules to certain devices or conditions. The content is raw text: the sequence `</style>` ends the element, and HTML entities are not decoded inside it.

Syntax

```
<style>  
  selector { property: value; }  
</style>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	Metadata content
Permitted content	CSS text

Property	Value
Permitted parents	<head>; also <noscript> in head; tolerated in <body>
Tag omission	Neither tag may be omitted
DOM interface	HTMLStyleElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
media	media query	Apply only when the query matches, for example print or (max-width: 600px). Default all.	HTML 4.01
blocking	render	Block rendering until the style sheet is applied (it does anyway for inline styles).	Living Standard
nonce	text	CSP nonce (global).	CSP
title	text	Name of an alternate style sheet set.	HTML 4.01
type	text/css	Obsolete. The only allowed value is text/css; omit it.	HTML 4.01
scoped	boolean	Removed. Was meant to scope styles to the parent element; use Shadow DOM or @scope.	HTML5 draft (removed)

<style>: attribute by attribute

Each attribute of <style> explained in detail, with a minimal example you can copy.

media

Values: media query | Since: HTML 4.01

Apply only when the query matches, for example print or (max-width: 600px). Default all.

Write media="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<style media="media query">...</style>
```

blocking

Values: render | Since: Living Standard

Block rendering until the style sheet is applied (it does anyway for inline styles).

Write blocking="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<style blocking="render">...</style>
```

nonce

Values: text | Since: CSP

CSP nonce (global).

Write nonce="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<style nonce="value">...</style>
```

title

Values: text | Since: HTML 4.01

Name of an alternate style sheet set.

Write title="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<style title="value">...</style>
```

type

Values: text/css | Since: HTML 4.01

Obsolete. The only allowed value is text/css; omit it.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<style type="value">...</style>
```

scoped

Values: boolean | Since: HTML5 draft (removed)

Removed. Was meant to scope styles to the parent element; use Shadow DOM or @scope.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<style scoped>...</style>
```

<style>: worked examples

Basic page styles

```
<head>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; }
    footer { text-align: center; color: gray; padding: 20px; }
  </style>
</head>
```

Print-only rules

```
<style media="print">
  nav, footer { display: none; }
  a::after { content: ' (' attr(href) ')'; }
</style>
```

Dark mode

```
<style>
  @media (prefers-color-scheme: dark) {
    body { background: #111; color: #eee; }
  }
</style>
```

Default browser CSS

```
style { display: none; }
```

Accessibility

- Do not use CSS to hide content that screen readers need; `display: none` hides it from everyone.

Common mistakes

- Putting CSS in `<style>` without the tags, or putting HTML inside `<style>`.
- Typing `</style>` inside a CSS string.
- Writing `type="text/css"` (harmless but unnecessary).

See also

`<link>`, `<head>`, `<script>`

Quick card:

Subscript

The <sub> element renders text as a subscript: smaller and below the baseline. Use it only where the subscript has a typographic meaning, such as chemical formulas (H₂O), mathematical variable indices (x₁) or footnote markers in some styles. For purely visual lowering, use CSS vertical-align.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<sup>, <math>

Minimal example

```
<p>Water is H<sub>2</sub>O and carbon dioxide is CO<sub>2</sub>.</p>
```

Most common mistake

Using sub/sup for layout tweaks.

<sub> — Subscript

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<sub>` element renders text as a subscript: smaller and below the baseline. Use it only where the subscript has a typographic meaning, such as chemical formulas (H₂O), mathematical variable indices (x₁) or footnote markers in some styles. For purely visual lowering, use CSS `vertical-align`.

Syntax

```
H<sub>2</sub>0
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<sub>: worked examples

Chemistry

<p>Water is H₂O and carbon dioxide is CO₂.</p>

Maths

<p>The sequence a₁, a₂, ... a_n.</p>

Default browser CSS

```
sub { vertical-align: sub; font-size: smaller; }
```

Accessibility

- Not announced; for critical meaning consider MathML.

Common mistakes

- Using sub/sup for layout tweaks.

See also

<sup>, <math>

Quick card: <summary>

Disclosure summary

The <summary> element specifies the visible heading, label or caption of a <details> element. Clicking it (or pressing Enter/Space when it is focused) toggles the details open and closed. It must be the first child of <details>. Browsers show a small triangle marker before it, which can be styled or removed with the ::marker pseudo-element or list-style: none.

At a glance

Status	CURRENT
Versions	HTML5 (5.1)
Category	Interactive elements
Closing tag	Required unless noted under Tag omission
Display	list-item
Own attributes	None (global attributes only)
Related	<details>

Minimal example

```
<details>
  <summary>What is a footer?</summary>
  <p>The bottom section of a page or section.</p>
</details>
```

Most common mistake

Placing summary anywhere but first.

<summary> — Disclosure summary

Category: Interactive elements | Status: CURRENT | Versions: HTML5 (5.1)

Description

The <summary> element specifies the visible heading, label or caption of a <details> element. Clicking it (or pressing Enter/Space when it is focused) toggles the details open and closed. It must be the first child of <details>. Browsers show a small triangle marker before it, which can be styled or removed with the ::marker pseudo-element or list-style: none.

Syntax

```
<details>
  <summary>Label</summary>
  ...
</details>
```

Technical summary

Property	Value
Void element	No
Default display	list-item
Content categories	None
Permitted content	Phrasing content, or one heading element
Permitted parents	<details>, as its first child
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<summary>: worked examples

FAQ

```
<details>
  <summary>What is a footer?</summary>
  <p>The bottom section of a page or section.</p>
</details>
```

Heading inside summary

```
<details>
  <summary><h3>Shipping</h3></summary>
  <p>2-4 days.</p>
</details>
```

Keeps the document outline; style the h3 with display: inline.

Custom marker

```
<style>
  summary { list-style: none; cursor: pointer; }
  summary::before { content: '+'; }
  details[open] summary::before { content: '-'; }
</style>
```

Default browser CSS

```
summary { display: list-item; counter-increment: list-item 0; list-
style: disclosure-closed inside; }
details[open] > summary { list-style-type: disclosure-open; }
```

Accessibility

- Announced as a button with expanded/collapsed state.
- Do not put links or buttons inside summary; nested interactive elements confuse users.

Common mistakes

- Placing summary anywhere but first.
- Using summary outside details.

See also

```
<details>
```

Quick card: <sup>

Superscript

The `<sup>` element renders text as a superscript: smaller and above the baseline. Use it where the superscript has meaning: exponents (x^2), ordinal suffixes (1st), footnote references, and abbreviations like M^{lle}. For visual raising only, use CSS.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code><sub></code> , <code><math></code>

Minimal example

```
<p>The area is 5<sup>2</sup> = 25 m<sup>2</sup>.</p>
```

Most common mistake

Using `sup` for trademark symbols where the [™] character already is superscript.

<sup> — Superscript

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<sup>` element renders text as a superscript: smaller and above the baseline. Use it where the superscript has meaning: exponents (x^2), ordinal suffixes (1st), footnote references, and abbreviations like M^{lle}. For visual raising only, use CSS.

Syntax

```
x<sup>2</sup>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<sup>: worked examples

Exponent

```
<p>The area is 5<sup>2</sup> = 25 m<sup>2</sup>.</p>
```

Footnote reference

```
<p>HTML was created in 1991.<sup><a href="#fn1" id="ref1">[1]</a></sup></p>
```

Preventing line-height changes

```
<style>
  sup, sub { line-height: 0; }
</style>
```

Superscripts can push lines apart; zero line-height fixes it.

Default browser CSS

```
sup { vertical-align: super; font-size: smaller; }
```

Accessibility

- Not announced; complex maths should use MathML.

Common mistakes

- Using sup for trademark symbols where the ™ character already is superscript.

See also

<sub>, <math>

Quick card: <svg>

Scalable Vector Graphics root

The <svg> element is the root of an inline SVG (Scalable Vector Graphics) image written directly in the HTML. SVG describes shapes with elements such as <circle>, <rect>, <line>, <path>, <text> and <g>, which scale to any size without losing sharpness, can be styled with CSS and animated with CSS or JavaScript. Inline SVG is ideal for icons, logos, charts and diagrams. SVG files can also be used through , but then CSS from the page cannot style their insides.

At a glance

Status	CURRENT
Versions	HTML5 (inline SVG)
Category	Embedded content (SVG)
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	viewBox, width, preserveAspectRatio, xmlns, fill, role
Related	, <canvas>, <math>, <picture>

Minimal example

```
<svg viewBox="0 0 24 24" width="24" height="24" aria-hidden="true"
focusable="false">
  <path d="M12 2L2 7l10 5 10-5-10-5z" fill="currentColor"/>
</svg>
```

Most common mistake

Omitting viewBox so the graphic does not scale.

<svg> — Scalable Vector Graphics root

Category: Embedded content (SVG) | Status: CURRENT | Versions: HTML5 (inline SVG)

Description

The <svg> element is the root of an inline SVG (Scalable Vector Graphics) image written directly in the HTML. SVG describes shapes with elements such as <circle>, <rect>, <line>, <path>, <text> and <g>, which scale to any size without losing sharpness, can be styled with CSS and animated with CSS or JavaScript. Inline SVG is ideal for icons, logos, charts and diagrams. SVG files can also be used through , but then CSS from the page cannot style their insides.

The viewBox attribute defines the internal coordinate system; width and height (or CSS) define the displayed size. This book documents SVG only at the level of the <svg> root; SVG has its own large specification.

Syntax

```
<svg viewBox="0 0 100 100" width="100" height="100">
  <circle cx="50" cy="50" r="40"/>
</svg>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content, palpable content

Property	Value
Permitted content	SVG elements
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither
DOM interface	SVGSVGElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
viewBox	min-x min-y width height	The internal coordinate system; makes the graphic scalable.	SVG
width / height	length	Displayed size.	SVG
preserveAspectRatio	keyword	How to fit the viewBox into the viewport.	SVG
xmlns	http://www.w3.org/2000/svg	Namespace; optional in HTML, required in standalone .svg files.	SVG
fill, stroke, stroke-width	colour / length	Presentation attributes for shapes (can also be CSS).	SVG
role, aria-label, aria-labelledby	-	Accessibility name and role for the graphic.	ARIA

<svg>: attribute by attribute

Each attribute of <svg> explained in detail, with a minimal example you can copy.

viewBox

Values: min-x min-y width height | Since: SVG

The internal coordinate system; makes the graphic scalable.

Write viewBox="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg viewBox="min-x min-y width height">...</svg>
```

width / height

Values: length | Since: SVG

Displayed size.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg width="length">...</svg>
```

preserveAspectRatio

Values: keyword | Since: SVG

How to fit the viewBox into the viewport.

Write preserveAspectRatio="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg preserveAspectRatio="keyword">...</svg>
```

xmlns

Values: <http://www.w3.org/2000/svg> | Since: SVG

Namespace; optional in HTML, required in standalone .svg files.

Write xmlns="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg xmlns="http:">...</svg>
```

fill, stroke, stroke-width

Values: colour / length | Since: SVG

Presentation attributes for shapes (can also be CSS).

Write fill="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg fill="#336699">...</svg>
```

role, aria-label, aria-labelledby

Values: - | Since: ARIA

Accessibility name and role for the graphic.

Write role="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<svg role="-">...</svg>
```

<svg>: worked examples

Icon

```
<svg viewBox="0 0 24 24" width="24" height="24" aria-hidden="true"
focusable="false">
  <path d="M12 2L2 7l10 5 10-5-10-5z" fill="currentColor"/>
</svg>
```

currentColor makes the icon follow the text colour.

Accessible logo

```
<svg viewBox="0 0 200 60" width="200" height="60" role="img" aria-
labelledby="t">
  <title id="t">Book Store</title>
  <rect width="200" height="60" rx="8" fill="#e60000"/>
  <text x="100" y="38" text-anchor="middle" fill="#fff" font-
size="24">Book Store</text>
</svg>
```

Responsive SVG

```
<style>
  svg { width: 100%; height: auto; }
</style>
```

Default browser CSS

```
svg:not(:root) { overflow: hidden; }
```

Accessibility

- Decorative SVG: `aria-hidden="true"`. Meaningful SVG: `role="img"` with `<title>` and `aria-labelledby`.
- Text in SVG `<text>` is real text and is readable, unlike text in raster images.

Common mistakes

- Omitting `viewBox` so the graphic does not scale.
- Forgetting `focusable="false"` for inline icons in old Internet Explorer.
- Huge inline SVGs repeated many times; use `<use>` or an external sprite.

See also

`<img>`, `<canvas>`, `<math>`, `<picture>`

Quick card: <table>

Table

The <table> element represents tabular data: information arranged in a two-dimensional grid of rows and columns, where the position of each cell gives it meaning (this value belongs to this row and this column). Examples: price lists, timetables, statistics, comparison charts, spreadsheets.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table
Own attributes	align, bgcolor, border, cellpadding, cellspacing, frame, rules, summary, width, height
Related	<tr>, <td>, <th>, <thead>, <tbody>, <tfoot>, <caption>, <colgroup>, <col>

Minimal example

```
<table>
  <caption>Book prices, September 2026</caption>
  <thead>
    <tr>
      <th scope="col">Title</th>
      <th scope="col">Format</th>
      <th scope="col">Price (EGP)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Learn HTML</td>
      <td>PDF</td>
      <td>120</td>
```

```
</tr>
<tr>
  <td>Learn CSS</td>
  <td>PDF</td>
  <td>140</td>
</tr>
</tbody>
<tfoot>
  <tr>
    <th scope="row" colspan="2">Total</th>
    <td>260</td>
  </tr>
</tfoot>
</table>
```

Most common mistake

Using tables for layout.

<table> — Table

Category: Table content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<table>` element represents tabular data: information arranged in a two-dimensional grid of rows and columns, where the position of each cell gives it meaning (this value belongs to this row and this column). Examples: price lists, timetables, statistics, comparison charts, spreadsheets.

A table is built row by row. `<tr>` creates a row; inside it `<th>` creates a header cell and `<td>` a data cell. Optional `<thead>`, `<tbody>` and `<tfoot>` group the rows; `<caption>` gives the table a title; `<colgroup>` and `<col>` describe columns. The cells of each row must add up to the same number of columns, taking `colspan` and `rowspan` into account.

Tables were introduced in HTML 3.2 and were immediately misused for page layout because there was no CSS. Layout tables are wrong today: they are inaccessible (screen readers announce rows and columns that mean nothing), not responsive, and hard to maintain. Use CSS flexbox and grid for layout; use `<table>` only for data.

Browsers render tables without borders by default. Add them with CSS. `border-collapse: collapse` merges adjacent cell borders into single lines, which is almost always what you want.

Syntax

```
<table>  
  <caption>Title</caption>  
  <thead><tr><th>Head</th></tr></thead>
```

```
<tbody><tr><td>Data</td></tr></tbody>
</table>
```

Technical summary

Property	Value
Void element	No
Default display	table
Content categories	Flow content, palpable content
Permitted content	In this order: an optional <caption>, zero or more <colgroup>, an optional <thead>, either zero or more <tbody> or one or more <tr>, an optional <tfoot>; <script> and <template> may be intermixed
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLTableElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right	Obsolete. Alignment of the table on the page. Use CSS margin.	HTML 3.2
bgcolor	colour	Obsolete. Background colour. Use CSS background-color.	HTML 3.2
border	pixels	Obsolete. Border width around the table and cells. Use CSS border.	HTML 3.2
cellpadding	pixels	Obsolete. Space between cell content and cell border. Use CSS padding on td/th.	HTML 3.2

Attribute	Values	Description	Since
cellspacing	pixels	Obsolete. Space between cells. Use CSS border-spacing or border-collapse.	HTML 3.2
frame	void above below hside lhs rhs vside box border	Obsolete. Which outer borders to draw. Use CSS border.	HTML 4.01
rules	none groups rows cols all	Obsolete. Which inner borders to draw. Use CSS border on cells.	HTML 4.01
summary	text	Obsolete. Description of the table for screen readers. Use <caption> or aria-describedby.	HTML 4.01
width	pixels or percentage	Obsolete. Table width. Use CSS width.	HTML 3.2
height	pixels	Obsolete, non-standard. Use CSS.	Netscape

<table>: attribute by attribute

Each attribute of <table> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | Since: HTML 3.2

Obsolete. Alignment of the table on the page. Use CSS margin.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table align="left">...</table>
```

bgcolor

Values: colour | Since: HTML 3.2

Obsolete. Background colour. Use CSS background-color.

Write bgcolor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table bgcolor="#336699">...</table>
```

border

Values: pixels | Since: HTML 3.2

Obsolete. Border width around the table and cells. Use CSS border.

Write border="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table border="3">...</table>
```

cellpadding

Values: pixels | Since: HTML 3.2

Obsolete. Space between cell content and cell border. Use CSS padding on td/th.

Write cellpadding="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table cellpadding="3">...</table>
```

cellspacing

Values: pixels | Since: HTML 3.2

Obsolete. Space between cells. Use CSS `border-spacing` or `border-collapse`.

Write `cellspacing="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table cellspacing="3">...</table>
```

frame

Values: void | above | below | hside | lhs | rhs | vside | box | border | Since: HTML 4.01

Obsolete. Which outer borders to draw. Use CSS `border`.

Write `frame="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table frame="void">...</table>
```

rules

Values: none | groups | rows | cols | all | Since: HTML 4.01

Obsolete. Which inner borders to draw. Use CSS `border` on cells.

Write `rules="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table rules="none">...</table>
```

summary

Values: text | Since: HTML 4.01

Obsolete. Description of the table for screen readers. Use `<caption>` or `aria-describedby`.

Write `summary="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table summary="value">...</table>
```


width

Values: pixels or percentage | Since: HTML 3.2

Obsolete. Table width. Use CSS width.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table width="3">...</table>
```

height

Values: pixels | Since: Netscape

Obsolete, non-standard. Use CSS.

Write height="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<table height="3">...</table>
```

<table>: worked examples

A complete data table

```
<table>
  <caption>Book prices, September 2026</caption>
  <thead>
    <tr>
      <th scope="col">Title</th>
      <th scope="col">Format</th>
      <th scope="col">Price (EGP)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Learn HTML</td>
      <td>PDF</td>
      <td>120</td>
    </tr>
    <tr>
      <td>Learn CSS</td>
      <td>PDF</td>
      <td>140</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row" colspan="2">Total</th>
      <td>260</td>
    </tr>
  </tfoot>
</table>
```

Essential CSS for tables

```
<style>
  table { border-collapse: collapse; width: 100%; }
  th, td { border: 1px solid #ccc; padding: 8px 12px; text-align:
left; }
  th { background: #f4f4f4; }
  tbody tr:nth-child(even) { background: #fafafa; }
  tbody tr:hover { background: #fff4e0; }
  td.num { text-align: right; }
</style>
```

border-collapse removes double lines; nth-child gives zebra stripes; right-align numbers.

Row and column headers together

```
<table>
  <tr>
    <td></td>
    <th scope="col">Mon</th>
    <th scope="col">Tue</th>
  </tr>
  <tr>
    <th scope="row">Morning</th>
    <td>HTML</td>
    <td>CSS</td>
  </tr>
  <tr>
    <th scope="row">Afternoon</th>
    <td>Practice</td>
    <td>Project</td>
  </tr>
</table>
```

Spanning cells

```
<table>
  <tr>
    <th colspan="2">Name</th>
    <th rowspan="2">Age</th>
  </tr>
  <tr>
    <th>First</th>
    <th>Last</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>Ali</td>
    <td>20</td>
  </tr>
</table>
```

Each row still totals three columns.

Responsive: horizontal scroll on small screens

```
<div style="overflow-x: auto">
  <table>...</table>
</div>
```

HTML 3.2 style attributes and their CSS replacement

```
<!-- Old -->
<table border="1" cellpadding="5" cellspacing="0" width="100%"
bgcolor="#eeeeee">

<!-- New -->
<style>
  table { border-collapse: collapse; width: 100%; background: #eee; }
  td, th { border: 1px solid #000; padding: 5px; }
</style>
<table>
```

Default browser CSS

```
table { display: table; border-collapse: separate; border-spacing:
2px; box-sizing: border-box; text-indent: initial; }
```

Accessibility

- Use `<th>` with `scope` for every header so screen readers can announce 'Price, 120' instead of just '120'.
- Give every data table a `<caption>`.
- Avoid layout tables; if unavoidable, add `role="presentation"`.
- Do not nest tables.
- For complex tables with multi-level headers, use the `headers` attribute on cells.

Common mistakes

- Using tables for layout.
- Rows with different numbers of cells.
- Forgetting `border-collapse` and getting double borders.
- Using `<td>` for headers.
- Placing text directly inside `<table>` or `<tr>`.

See also

`<tr>`, `<td>`, `<th>`, `<thead>`, `<tbody>`, `<tfoot>`, `<caption>`, `<colgroup>`, `<col>`

Quick card: <tbody>

Table body

The <tbody> element groups the main data rows of a table, as distinct from the header rows in <thead> and the summary rows in <tfoot>. A table may have several <tbody> elements to group rows into sections (for example one per category), which can then be styled separately. When you write <tr> directly inside <table> without a <tbody>, the browser creates one automatically, so a tbody is always present in the DOM.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-row-group
Own attributes	align
Related	<table>, <thead>, <tfoot>, <tr>

Minimal example

```
<table>
  <thead><tr><th>Item</th><th>Price</th></tr></thead>
  <tbody>
    <tr><th colspan="2">Books</th></tr>
    <tr><td>Learn HTML</td><td>120</td></tr>
  </tbody>
  <tbody>
    <tr><th colspan="2">Courses</th></tr>
    <tr><td>CSS Basics</td><td>300</td></tr>
  </tbody>
</table>
```

Most common mistake

Putting tbody before thead.

<tbody> — Table body

Category: Table content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The `<tbody>` element groups the main data rows of a table, as distinct from the header rows in `<thead>` and the summary rows in `<tfoot>`. A table may have several `<tbody>` elements to group rows into sections (for example one per category), which can then be styled separately. When you write `<tr>` directly inside `<table>` without a `<tbody>`, the browser creates one automatically, so a `tbody` is always present in the DOM.

Using explicit `thead/tbody/tfoot` lets browsers repeat the header on every printed page of a long table and lets CSS make the body scroll independently.

Syntax

```
<tbody>
  <tr>...</tr>
</tbody>
```

Technical summary

Property	Value
Void element	No
Default display	table-row-group
Content categories	None
Permitted content	Zero or more <code><tr></code> , <code><script></code> , <code><template></code>
Permitted parents	<code><table></code> , after <code><caption></code> , <code><colgroup></code> and <code><thead></code>

Property	Value
Tag omission	Start tag may be omitted if the first thing inside is a <tr> and it is not preceded by a tbody, thead or tfoot whose end tag was omitted; end tag may be omitted if followed by <tbody> or <tfoot> or if there is no more content in the parent
DOM interface	HTMLTableSectionElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align, bgcolor, char, charoff, valign	various	Obsolete presentational attributes. Use CSS.	HTML 4.01

<tbody>: attribute by attribute

Each attribute of <tbody> explained in detail, with a minimal example you can copy.

align, bgcolor, char, charoff, valign

Values: various | Since: HTML 4.01

Obsolete presentational attributes. Use CSS.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tbody align="various">...</tbody>
```

<tbody>: worked examples

Multiple bodies for grouped rows

```
<table>
  <thead><tr><th>Item</th><th>Price</th></tr></thead>
  <tbody>
    <tr><th colspan="2">Books</th></tr>
    <tr><td>Learn HTML</td><td>120</td></tr>
  </tbody>
  <tbody>
    <tr><th colspan="2">Courses</th></tr>
    <tr><td>CSS Basics</td><td>300</td></tr>
  </tbody>
</table>
```

Scrolling body with fixed header

```
<style>
  .wrap { max-height: 300px; overflow-y: auto; }
  thead th { position: sticky; top: 0; background: #fff; }
</style>
<div class="wrap"><table>...</table></div>
```

Default browser CSS

```
tbody { display: table-row-group; vertical-align: middle; }
```

Accessibility

- No special role; it exists for structure and styling.

Common mistakes

- Putting tbody before thead.
- Placing cells directly inside tbody without tr.

See also

<table>, <thead>, <tfoot>, <tr>

Quick card: <td>

Table data cell

The <td> element defines a cell of a table that contains data. It must be inside a <tr>. Each <td> occupies one column by default; colspan makes it span several columns and rowspan several rows. The headers attribute can list the ids of the <th> cells that describe it, which is needed only for complex tables where scope is not enough.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-cell
Own attributes	colspan, rowspan, headers, abbr, align, axis, bgcolor, char, height, nowrap, scope, valign, width
Related	<th>, <tr>, <table>

Minimal example

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>20</td>
  </tr>
  <tr>
    <td>Sara</td>
    <td>22</td>
  </tr>
```

</table>

Most common mistake

Placing <td> outside <tr>.

<td> — Table data cell

Category: Table content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<td>` element defines a cell of a table that contains data. It must be inside a `<tr>`. Each `<td>` occupies one column by default; `colspan` makes it span several columns and `rowspan` several rows. The `headers` attribute can list the ids of the `<th>` cells that describe it, which is needed only for complex tables where `scope` is not enough.

Data cells are left-aligned and normal weight by default, in contrast to header cells (`<th>`) which are bold and centred. Use `<th>` for headers even if you restyle them, because the distinction is what screen readers rely on.

Syntax

```
<tr>
  <td>Data</td>
</tr>
```

Technical summary

Property	Value
Void element	No
Default display	table-cell
Content categories	Sectioning root
Permitted content	Flow content
Permitted parents	<code><tr></code>

Property	Value
Tag omission	Start tag required; end tag may be omitted if followed by <code><td></code> or <code><th></code> or if there is no more content in the parent
DOM interface	HTMLTableCellElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
colspan	positive integer	Number of columns the cell spans. Default 1, maximum 1000.	HTML 3.2
rowspan	non-negative integer	Number of rows the cell spans. Default 1; 0 means to the end of the row group. Maximum 65534.	HTML 3.2
headers	space-separated ids	The ids of <code><th></code> cells that provide headers for this cell.	HTML 4.01
abbr	text	Obsolete on td (valid on th). Abbreviated description.	HTML 4.01
align	left center right justify char	Obsolete. Use CSS text-align.	HTML 3.2
axis	text	Obsolete. Category names.	HTML 4.01
bgcolor	colour	Obsolete. Use CSS background-color.	HTML 3.2
char / charoff	character / number	Obsolete. Character alignment.	HTML 4.01
height	pixels	Obsolete. Use CSS height.	HTML 3.2
nowrap	boolean	Obsolete. Use CSS white-space: nowrap.	HTML 3.2
scope	-	Obsolete on td (use on th).	HTML 4.01

Attribute	Values	Description	Since
valign	top middle bottom baseline	Obsolete. Use CSS vertical-align.	HTML 3.2
width	pixels or percentage	Obsolete. Use CSS width.	HTML 3.2

<td>: attribute by attribute

Each attribute of <td> explained in detail, with a minimal example you can copy.

colspan

Values: positive integer | Since: HTML 3.2

Number of columns the cell spans. Default 1, maximum 1000.

Write colspan="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td colspan="3">...</td>
```

rowspan

Values: non-negative integer | Since: HTML 3.2

Number of rows the cell spans. Default 1; 0 means to the end of the row group. Maximum 65534.

Write rowspan="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td rowspan="3">...</td>
```

headers

Values: space-separated ids | Since: HTML 4.01

The ids of <th> cells that provide headers for this cell.

Write headers="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td headers="space-separated ids">...</td>
```

abbr

Values: text | Since: HTML 4.01

Obsolete on td (valid on th). Abbreviated description.

Write abbr="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td abbr="value">...</td>
```


align

Values: left | center | right | justify | char | Since: HTML 3.2

Obsolete. Use CSS text-align.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td align="left">...</td>
```

axis

Values: text | Since: HTML 4.01

Obsolete. Category names.

Write axis="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td axis="value">...</td>
```

bgcolor

Values: colour | Since: HTML 3.2

Obsolete. Use CSS background-color.

Write bgcolor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td bgcolor="#336699">...</td>
```

char / charoff

Values: character / number | Since: HTML 4.01

Obsolete. Character alignment.

Write char="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td char="3">...</td>
```

height

Values: pixels | Since: HTML 3.2

Obsolete. Use CSS height.

Write height="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td height="3">...</td>
```

nowrap

Values: boolean | Since: HTML 3.2

Obsolete. Use CSS white-space: nowrap.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<td nowrap>...</td>
```

scope

Values: - | Since: HTML 4.01

Obsolete on td (use on th).

Write scope="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td scope="-">...</td>
```

valign

Values: top | middle | bottom | baseline | Since: HTML 3.2

Obsolete. Use CSS vertical-align.

Write valign="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td valign="top">...</td>
```

width

Values: pixels or percentage | Since: HTML 3.2

Obsolete. Use CSS width.

Write width="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<td width="3">...</td>
```

<td>: worked examples

Basic cells

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>20</td>
  </tr>
  <tr>
    <td>Sara</td>
    <td>22</td>
  </tr>
</table>
```

Row 1 has header cells, rows 2 and 3 have data cells.

colspan and rowspan

```
<table>
  <tr>
    <td rowspan="2">Spans two rows</td>
    <td>Row 1</td>
  </tr>
  <tr>
    <td>Row 2</td>
  </tr>
  <tr>
    <td colspan="2">Spans two columns</td>
  </tr>
</table>
```

Complex header association

```
<table>
  <tr>
    <th id="n">Name</th>
    <th id="q1">Q1</th>
    <th id="q2">Q2</th>
  </tr>
  <tr>
    <th id="r1" headers="n">Ahmed</th>
    <td headers="r1 q1">10</td>
    <td headers="r1 q2">12</td>
  </tr>
</table>
```

Cell styling

```
<style>
  td { padding: 8px 12px; border-bottom: 1px solid #e0e0e0; vertical-align: top; }
  td.num { text-align: right; font-variant-numeric: tabular-nums; }
</style>
```

Default browser CSS

```
td { display: table-cell; vertical-align: inherit; padding: 1px; }
```

Accessibility

- Do not leave header cells as `<td>`.
- Empty cells are fine; do not fill them with ` ` unless needed for old-browser borders.

Common mistakes

- Placing `<td>` outside `<tr>`.
- Inconsistent column counts.
- Using `td` for headers.

See also

`<th>`, `<tr>`, `<table>`

Quick card: <template>

Content template

The <template> element holds HTML that is not rendered when the page loads but can be cloned and inserted into the document later by JavaScript. The content is parsed into a separate document fragment (template.content): scripts inside do not run, images do not load and the markup is invisible. It is the standard way to define reusable markup for lists, cards and web components.

At a glance

Status	CURRENT
Versions	HTML5 (2013)
Category	Scripting / web components
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	shadowrootmode, shadowrootdelegatesfocus, shadowrootclonable, shadowrootserializable
Related	<slot>, <script>, <noscript>

Minimal example

```
<table id="cart"><tbody></tbody></table>
<template id="row">
  <tr><td class="name"></td><td class="price"></td></tr>
</template>
<script>
  const items = [['Learn HTML', 120], ['Learn CSS', 140]];
  const tpl = document.getElementById('row');
  const body = document.querySelector('#cart tbody');
  for (const [name, price] of items) {
    const clone = tpl.content.cloneNode(true);
    clone.querySelector('.name').textContent = name;
    clone.querySelector('.price').textContent = price;
```

```
        body.append(clone);  
    }  
</script>
```

Most common mistake

Using innerHTML on template instead of template.content.

<template> — Content template

Category: Scripting / web components | Status: CURRENT | Versions: HTML5 (2013)

Description

The `<template>` element holds HTML that is not rendered when the page loads but can be cloned and inserted into the document later by JavaScript. The content is parsed into a separate document fragment (`template.content`): scripts inside do not run, images do not load and the markup is invisible. It is the standard way to define reusable markup for lists, cards and web components.

Declarative Shadow DOM uses `<template shadowrootmode="open">` inside a custom element to create a shadow root without JavaScript.

Syntax

```
<template id="t">
  <li class="item"></li>
</template>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	Metadata content, flow content, phrasing content, script-supporting element
Permitted content	Anything (stored in a DocumentFragment, not rendered)

Property	Value
Permitted parents	Almost anywhere: head, body, and inside table, colgroup, dl, ol, ul, select...
Tag omission	Neither tag may be omitted
DOM interface	HTMLTemplateElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
shadowrootmode	open closed	Creates a declarative shadow root on the parent element.	Living Standard
shadowrootdelegatesfocus	boolean	Sets delegatesFocus on the declarative shadow root.	Living Standard
shadowrootclonable	boolean	Sets clonable on the declarative shadow root.	Living Standard
shadowrootserializable	boolean	Sets serializable on the declarative shadow root.	Living Standard

<template>: attribute by attribute

Each attribute of <template> explained in detail, with a minimal example you can copy.

shadowrootmode

Values: open | closed | Since: Living Standard

Creates a declarative shadow root on the parent element.

Write shadowrootmode="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<template shadowrootmode="open">...</template>
```

shadowrootdelegatesfocus

Values: boolean | Since: Living Standard

Sets delegatesFocus on the declarative shadow root.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<template shadowrootdelegatesfocus>...</template>
```

shadowrootclonable

Values: boolean | Since: Living Standard

Sets clonable on the declarative shadow root.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<template shadowrootclonable>...</template>
```

shadowrootserializable

Values: boolean | Since: Living Standard

Sets serializable on the declarative shadow root.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<template shadowrootserializable>...</template>
```

<template>: worked examples

Cloning a row template

```
<table id="cart"><tbody></tbody></table>
<template id="row">
  <tr><td class="name"></td><td class="price"></td></tr>
</template>
<script>
  const items = [['Learn HTML', 120], ['Learn CSS', 140]];
  const tpl = document.getElementById('row');
  const body = document.querySelector('#cart tbody');
  for (const [name, price] of items) {
    const clone = tpl.content.cloneNode(true);
    clone.querySelector('.name').textContent = name;
    clone.querySelector('.price').textContent = price;
    body.append(clone);
  }
</script>
```

Declarative shadow DOM

```
<my-card>
  <template shadowrootmode="open">
    <style>p { color: #b90000; }</style>
    <p><slot></slot></p>
  </template>
  Hello from the light DOM
</my-card>
```

Default browser CSS

```
template { display: none; }
```

Accessibility

- Template content is not in the accessibility tree until inserted.

Common mistakes

- Using innerHTML on template instead of template.content.
- Expecting scripts inside the template to run before insertion.

See also

<slot>, <script>, <noscript>

Quick card: <textarea>

Multi-line text input

The <textarea> element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike <input>, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Forms
Closing tag	Required unless noted under Tag omission
Display	inline-block
Own attributes	name, rows, cols, placeholder, required, disabled, readonly, maxlength, minlength, wrap, autocomplete, autofocus, spellcheck, dirname, form
Related	<input>, <form>, <label>, <output>

Minimal example

```
<label for="msg">Message</label>
<textarea id="msg" name="message" rows="5" placeholder="Write your
message..." required maxlength="500"></textarea>
```

Most common mistake

Writing <textarea value="..."> (value is not an attribute here).

<textarea> — Multi-line text input

Category: Forms | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The <textarea> element creates a multi-line plain-text editing control, for comments, messages, addresses and any free-form text. Unlike <input>, it is not a void element: its initial value is the text between its tags, not a value attribute. Whitespace and line breaks inside the tags are preserved, so keep the closing tag right after the text to avoid unwanted spaces.

The rows and cols attributes set its visible size in lines and characters; CSS width/height are more flexible. Most browsers let the user drag-resize it; control this with the CSS resize property. The field-sizing: content CSS property (new) makes it grow with its content.

Syntax

```
<textarea name="n" rows="4" cols="40">initial text</textarea>
```

Technical summary

Property	Value
Void element	No
Default display	inline-block
Content categories	Flow content, phrasing content, interactive content, listed, labelable, resettable, submittable, form-associated, palpable
Permitted content	Text

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLTextAreaElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
name	text	Name submitted with the form.	HTML 2.0
rows	positive integer	Visible number of lines. Default 2.	HTML 2.0
cols	positive integer	Visible width in average characters. Default 20.	HTML 2.0
placeholder	text	Hint shown when empty. Not a label.	HTML5
required	boolean	Must not be empty.	HTML5
disabled	boolean	Cannot be used or submitted.	HTML 4.01
readonly	boolean	Cannot be edited; is submitted.	HTML 4.01
maxlength	integer	Maximum characters.	HTML5
minlength	integer	Minimum characters.	HTML5
wrap	soft hard off	soft (default): lines wrap visually but newlines are not submitted. hard: newlines are inserted at wrap points when submitting (requires cols). off: no wrapping (non-standard).	HTML5
autocomplete	on off token	Autofill hint (street-address, etc.).	HTML5

Attribute	Values	Description	Since
autofocus	boolean	Focus on load.	HTML5
spellcheck	true false	Enable or disable spell checking (global).	HTML5
dirname	text	Submit text direction.	HTML5
form	form id	Associated form.	HTML5

<textarea>: attribute by attribute

Each attribute of <textarea> explained in detail, with a minimal example you can copy.

name

Values: text | Since: HTML 2.0

Name submitted with the form.

Write name="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea name="value">...</textarea>
```

rows

Values: positive integer | Since: HTML 2.0

Visible number of lines. Default 2.

Write rows="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea rows="3">...</textarea>
```

cols

Values: positive integer | Since: HTML 2.0

Visible width in average characters. Default 20.

Write cols="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea cols="3">...</textarea>
```

placeholder

Values: text | Since: HTML5

Hint shown when empty. Not a label.

Write placeholder="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea placeholder="value">...</textarea>
```

required

Values: boolean | Since: HTML5

Must not be empty.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<textarea required>...</textarea>
```

disabled

Values: boolean | Since: HTML 4.01

Cannot be used or submitted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<textarea disabled>...</textarea>
```

readonly

Values: boolean | Since: HTML 4.01

Cannot be edited; is submitted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<textarea readonly>...</textarea>
```

maxlength

Values: integer | Since: HTML5

Maximum characters.

Write `maxlength="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea maxlength="3">...</textarea>
```

minlength

Values: integer | Since: HTML5

Minimum characters.

Write `minlength="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea minlength="3">...</textarea>
```


wrap

Values: soft | hard | off | Since: HTML5

soft (default): lines wrap visually but newlines are not submitted. hard: newlines are inserted at wrap points when submitting (requires cols). off: no wrapping (non-standard).

Write wrap="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea wrap="soft">...</textarea>
```

autocomplete

Values: on | off | token | Since: HTML5

Autofill hint (street-address, etc.).

Write autocomplete="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea autocomplete="on">...</textarea>
```

autofocus

Values: boolean | Since: HTML5

Focus on load.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<textarea autofocus>...</textarea>
```

spellcheck

Values: true | false | Since: HTML5

Enable or disable spell checking (global).

Write spellcheck="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea spellcheck="true">...</textarea>
```

dirname

Values: text | Since: HTML5

Submit text direction.

Write `dirname="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea dirname="value">...</textarea>
```

form

Values: form id | Since: HTML5

Associated form.

Write `form="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<textarea form="form id">...</textarea>
```

<textarea>: worked examples

Message field

```
<label for="msg">Message</label>
<textarea id="msg" name="message" rows="5" placeholder="Write your
message..." required maxLength="500"></textarea>
```

With initial text

```
<textarea name="bio">Write a short bio here.</textarea>
```

Styling and resize control

```
<style>
  textarea { width: 100%; min-height: 120px; padding: 8px; font:
inherit; resize: vertical; box-sizing: border-box; }
</style>
```

Character counter

```
<textarea id="t" maxLength="200"></textarea>
<output id="c" for="t">0 / 200</output>
<script>
  t.addEventListener('input', () => c.value = t.value.length + ' /
200');
</script>
```

Default browser CSS

```
textarea { display: inline-block; white-space: pre-wrap; resize: both;
}
```

Accessibility

- Label it.
- Do not remove the resize handle without good reason.
- Announce character limits in the label or description.

Common mistakes

- Writing `<textarea value="...">` (value is not an attribute here).
- Indenting the content and getting leading spaces.
- Forgetting the closing tag; everything after becomes part of the textarea.

See also

`<input>`, `<form>`, `<label>`, `<output>`

Quick card: <tfoot>

Table footer

The <tfoot> element groups the rows that summarise the columns of a table: totals, averages, counts. In HTML5 it is placed after the <tbody> elements (HTML 4.01 required it before them so that the browser could render the footer before receiving all data). When printing a long table, browsers can repeat the tfoot at the bottom of every page.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-footer-group
Own attributes	align
Related	<table>, <thead>, <tbody>

Minimal example

```
<table>
  <thead><tr><th>Item</th><th>Price</th></tr></thead>
  <tbody>
    <tr><td>Book</td><td>120</td></tr>
    <tr><td>Pen</td><td>10</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Total</th><td>130</td></tr>
  </tfoot>
</table>
```

Most common mistake

Placing tfoot in the middle of the table.

<tfoot> — Table footer

Category: Table content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The <tfoot> element groups the rows that summarise the columns of a table: totals, averages, counts. In HTML5 it is placed after the <tbody> elements (HTML 4.01 required it before them so that the browser could render the footer before receiving all data). When printing a long table, browsers can repeat the tfoot at the bottom of every page.

Syntax

```
<tfoot>
  <tr><th>Total</th><td>260</td></tr>
</tfoot>
```

Technical summary

Property	Value
Void element	No
Default display	table-footer-group
Content categories	None
Permitted content	Zero or more <tr>, <script>, <template>
Permitted parents	<table>, after caption, colgroup, thead and tbody
Tag omission	End tag may be omitted if there is no more content in the parent
DOM interface	HTMLTableSectionElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align, bgcolor, char, charoff, valign	various	Obsolete presentational attributes. Use CSS.	HTML 4.01

<tfoot>: attribute by attribute

Each attribute of <tfoot> explained in detail, with a minimal example you can copy.

align, bgcolor, char, charoff, valign

Values: various | Since: HTML 4.01

Obsolete presentational attributes. Use CSS.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tfoot align="various">...</tfoot>
```


<tfoot>: worked examples

Totals row

```
<table>
  <thead><tr><th>Item</th><th>Price</th></tr></thead>
  <tbody>
    <tr><td>Book</td><td>120</td></tr>
    <tr><td>Pen</td><td>10</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Total</th><td>130</td></tr>
  </tfoot>
</table>
```

Styling

```
<style>
  tfoot { font-weight: bold; background: #f4f4f4; border-top: 2px
solid #333; }
</style>
```

Default browser CSS

```
tfoot { display: table-footer-group; vertical-align: middle; }
```

Accessibility

- No special role, but grouping totals in tfoot helps users understand the table structure.

Common mistakes

- Placing tfoot in the middle of the table.
- Multiple tfoot elements.

See also

<table>, <thead>, <tbody>

Quick card: <th>

Table header cell

The <th> element defines a header cell in a table: a cell that labels a column or a row rather than holding data. Browsers render it bold and centred. The scope attribute states whether the header applies to its column (col), its row (row), a group of columns (colgroup) or a group of rows (rowgroup). Browsers can usually guess for simple tables, but stating it explicitly is best practice because screen readers use the header text when reading each data cell.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-cell
Own attributes	scope, colspan, rowspan, headers, abbr, align
Related	<td>, <tr>, <table>, <thead>

Minimal example

```
<table>
  <tr>
    <th scope="col">Product</th>
    <th scope="col">Price</th>
  </tr>
  <tr>
    <th scope="row">Notebook</th>
    <td>25</td>
  </tr>
</table>
```

Most common mistake

Using `<td>` with bold styling instead of `<th>`.

<th> — Table header cell

Category: Table content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<th>` element defines a header cell in a table: a cell that labels a column or a row rather than holding data. Browsers render it bold and centred. The `scope` attribute states whether the header applies to its column (`col`), its row (`row`), a group of columns (`colgroup`) or a group of rows (`rowgroup`). Browsers can usually guess for simple tables, but stating it explicitly is best practice because screen readers use the header text when reading each data cell.

Syntax

```
<th scope="col">Header</th>
```

Technical summary

Property	Value
Void element	No
Default display	table-cell
Content categories	None
Permitted content	Flow content, but no header, footer, sectioning content or heading content
Permitted parents	<code><tr></code>
Tag omission	End tag may be omitted if followed by <code><td></code> or <code><th></code> or if there is no more content in the parent
DOM interface	HTMLTableCellElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
scope	row col rowgroup colgroup	Which cells this header describes.	HTML 4.01
colspan	positive integer	Columns spanned.	HTML 3.2
rowspan	non-negative integer	Rows spanned.	HTML 3.2
headers	space-separated ids	Other header cells that apply to this one (for multi-level headers).	HTML 4.01
abbr	text	A short alternative label that screen readers may use when reading the header repeatedly.	HTML 4.01
align, axis, bgcolor, char, charoff, height, nowrap, valign, width	various	Obsolete presentational attributes. Use CSS.	HTML 3.2 / 4.01

<th>: attribute by attribute

Each attribute of <th> explained in detail, with a minimal example you can copy.

scope

Values: row | col | rowgroup | colgroup | Since: HTML 4.01

Which cells this header describes.

Write scope="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th scope="row">...</th>
```

colspan

Values: positive integer | Since: HTML 3.2

Columns spanned.

Write colspan="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th colspan="3">...</th>
```

rowspan

Values: non-negative integer | Since: HTML 3.2

Rows spanned.

Write rowspan="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th rowspan="3">...</th>
```

headers

Values: space-separated ids | Since: HTML 4.01

Other header cells that apply to this one (for multi-level headers).

Write headers="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th headers="space-separated ids">...</th>
```

abbr

Values: text | Since: HTML 4.01

A short alternative label that screen readers may use when reading the header repeatedly.

Write `abbr="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th abbr="value">...</th>
```

align, axis, bgcolor, char, charoff, height, nowrap, valign, width

Values: various | Since: HTML 3.2 / 4.01

Obsolete presentational attributes. Use CSS.

Write `align="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<th align="various">...</th>
```

<th>: worked examples

Column and row headers

```
<table>
  <tr>
    <th scope="col">Product</th>
    <th scope="col">Price</th>
  </tr>
  <tr>
    <th scope="row">Notebook</th>
    <td>25</td>
  </tr>
</table>
```

A screen reader on the '25' cell announces: 'Notebook, Price, 25'.

Group headers

```
<table>
  <tr>
    <th scope="colgroup" colspan="2">2025</th>
    <th scope="colgroup" colspan="2">2026</th>
  </tr>
  <tr>
    <th scope="col">Q1</th><th scope="col">Q2</th>
    <th scope="col">Q1</th><th scope="col">Q2</th>
  </tr>
</table>
```

Left-aligned headers

```
<style>
  th { text-align: left; background: #f4f4f4; }
  th[scope="row"] { font-weight: 600; }
</style>
```

Default browser CSS

```
th { display: table-cell; vertical-align: inherit; font-weight: bold;
text-align: center; padding: 1px; }
```

Accessibility

- Always use <th> for headers and set scope.
- Use abbr for long headers that will be repeated often.

Common mistakes

- Using <td> with bold styling instead of <th>.

- Omitting scope in tables with both row and column headers.

See also

`<td>`, `<tr>`, `<table>`, `<thead>`

Quick card: <thead>

Table header

The <thead> element groups the rows that hold the column headers of a table. It comes after any <caption> and <colgroup> and before <tbody> and <tfoot>. Browsers repeat the thead at the top of each printed page of a long table, and CSS position: sticky on its cells keeps the headers visible while scrolling. A table may have at most one thead.

At a glance

Status	CURRENT
Versions	HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-header-group
Own attributes	align
Related	<table>, <tbody>, <tfoot>, <th>

Minimal example

```
<table>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Email</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Ahmed</td><td>ahmed@example.com</td></tr>
  </tbody>
</table>
```

Most common mistake

Two thead elements.

<thead> — Table header

Category: Table content | Status: CURRENT | Versions: HTML 4.01, HTML5

Description

The <thead> element groups the rows that hold the column headers of a table. It comes after any <caption> and <colgroup> and before <tbody> and <tfoot>. Browsers repeat the thead at the top of each printed page of a long table, and CSS position: sticky on its cells keeps the headers visible while scrolling. A table may have at most one thead.

Syntax

```
<thead>
  <tr><th>Column</th></tr>
</thead>
```

Technical summary

Property	Value
Void element	No
Default display	table-header-group
Content categories	None
Permitted content	Zero or more <tr>, <script>, <template>
Permitted parents	<table>, after caption and colgroup, before tbody, tfoot and tr
Tag omission	End tag may be omitted if followed by <tbody> or <tfoot>
DOM interface	HTMLTableSectionElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align, bgcolor, char, charoff, valign	various	Obsolete presentational attributes. Use CSS.	HTML 4.01

<thead>: attribute by attribute

Each attribute of <thead> explained in detail, with a minimal example you can copy.

align, bgcolor, char, charoff, valign

Values: various | Since: HTML 4.01

Obsolete presentational attributes. Use CSS.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<thead align="various">...</thead>
```

<thead>: worked examples

Header group

```
<table>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">Email</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Ahmed</td><td>ahmed@example.com</td></tr>
  </tbody>
</table>
```

Sticky header

```
<style>
  thead th { position: sticky; top: 0; background: #fff; box-shadow:
0 1px 0 #ccc; }
</style>
```

Default browser CSS

```
thead { display: table-header-group; vertical-align: middle; }
```

Accessibility

- Helps assistive technology identify header rows; combine with `th scope="col"`.

Common mistakes

- Two `thead` elements.
- Putting data rows in `thead`.

See also

`<table>`, `<tbody>`, `<tfoot>`, `<th>`

Quick card: <time>

Date / time

The <time> element represents a specific period in time: a date, a time of day, a date and time, a duration, a week or a month. The optional datetime attribute holds the value in a machine-readable format so that browsers, search engines, calendars and scripts can understand it, while the visible content can be written in any human-friendly way ('11 September 2026', 'yesterday', '□□ □□□□□□'). If datetime is omitted, the text content itself must be in a valid format.

At a glance

Status	CURRENT
Versions	HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	datetime
Related	<data>, <article>

Minimal example

<p>Published <time datetime="2026-09-11">11 September 2026</time></p>

Most common mistake

Invalid datetime formats such as 11/09/2026.

<time> — Date / time

Category: Text-level semantics | Status: CURRENT | Versions: HTML5

Description

The `<time>` element represents a specific period in time: a date, a time of day, a date and time, a duration, a week or a month. The optional `datetime` attribute holds the value in a machine-readable format so that browsers, search engines, calendars and scripts can understand it, while the visible content can be written in any human-friendly way ('11 September 2026', 'yesterday', '📅 🗓️ 📆'). If `datetime` is omitted, the text content itself must be in a valid format.

Syntax

```
<time datetime="2026-09-11">11 September 2026</time>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLTimeElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
datetime	date/time string	Machine-readable value. Formats: 2026-09-11 (date); 14:30 or 14:30:00 (time); 2026-09-11T14:30 (local date-time); 2026-09-11T14:30Z or 2026-09-11T14:30+02:00 (with time zone); 2026-09 (month); 2026-W37 (week); 2026 (year); 09-11 (yearless date); PT2H30M or 2h 30m (duration).	HTML5

<time>: attribute by attribute

Each attribute of <time> explained in detail, with a minimal example you can copy.

datetime

Values: date/time string | Since: HTML5

Machine-readable value. Formats: 2026-09-11 (date); 14:30 or 14:30:00 (time); 2026-09-11T14:30 (local date-time); 2026-09-11T14:30Z or 2026-09-11T14:30+02:00 (with time zone); 2026-09 (month); 2026-W37 (week); 2026 (year); 09-11 (yearless date); PT2H30M or 2h 30m (duration).

Write datetime="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<time datetime="date">...</time>
```

<time>: worked examples

Publication date

<p>Published <time datetime="2026-09-11">11 September 2026</time></p>

Event with time zone

<p>The webinar starts at <time datetime="2026-10-01T19:00+02:00">7 pm
Cairo time on 1 October</time>.</p>

Duration

<p>Reading time: <time datetime="PT8M">8 minutes</time></p>

Relative text with exact value

<p>Posted <time datetime="2026-09-10T09:15Z" title="10 Sep 2026, 09:
15 UTC">yesterday</time></p>

Default browser CSS

```
time { display: inline; }
```

Accessibility

- Screen readers read the visible text, not datetime; keep the visible text understandable.

Common mistakes

- Invalid datetime formats such as 11/09/2026.
- Using <time> for vague periods ('the Middle Ages').

See also

<data>, <article>

Quick card: <title>

Document title

The <title> element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one <title>, inside <head>.

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Document metadata
Closing tag	Required unless noted under Tag omission
Display	none
Own attributes	None (global attributes only)
Related	<head>, <meta>, <h1>

Minimal example

```
<title>Learn HTML (PDF) - Book Store</title>
<title>Contact us - Book Store</title>
<title>Order #1042 confirmed - Book Store</title>
```

Most common mistake

Same title on every page.

<title> — Document title

Category: Document metadata | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<title>` element defines the title of the document, shown in the browser's title bar or tab, used as the default name for bookmarks, displayed as the clickable headline in search engine results, and announced first by screen readers when the page loads. It contains text only; any tags inside are treated as literal text. Every document must have exactly one `<title>`, inside `<head>`.

A good title is unique for each page, describes the page's content, puts the most specific information first ('Learn HTML - Book Store' rather than 'Book Store - Learn HTML'), and is under about 60 characters so search engines do not truncate it.

Syntax

```
<title>Page title</title>
```

Technical summary

Property	Value
Void element	No
Default display	none
Content categories	Metadata content
Permitted content	Text (not inter-element whitespace only)
Permitted parents	<code><head></code> (exactly one)

Property	Value
Tag omission	Neither tag may be omitted
DOM interface	HTMLTitleElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<title>: worked examples

Good titles

```
<title>Learn HTML (PDF) - Book Store</title>
<title>Contact us - Book Store</title>
<title>Order #1042 confirmed - Book Store</title>
```

Changing the title from JavaScript

```
<script>
  document.title = '(3) New messages - Chat';
</script>
```

Default browser CSS

```
title { display: none; }
```

Accessibility

- The title is the first thing announced; unique, descriptive titles are a WCAG requirement (2.4.2).

Common mistakes

- Same title on every page.
- Empty or missing title.
- Putting HTML tags inside.
- Keyword stuffing.

See also

<head>, <meta>, <h1>

Quick card: <tr>

Table row

The <tr> element defines a row of cells in a table. It contains <th> (header) and/or <td> (data) cells and is placed inside <table>, <thead>, <tbody> or <tfoot>. Rows are the fundamental unit of table construction: you build a table one row at a time, and the columns emerge from the cells in each row.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01, HTML5
Category	Table content
Closing tag	Required unless noted under Tag omission
Display	table-row
Own attributes	align, bgcolor, char, valign
Related	<table>, <td>, <th>, <thead>, <tbody>, <tfoot>

Minimal example

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>20</td>
  </tr>
  <tr>
    <td>Sara</td>
    <td>22</td>
  </tr>
</table>
```

Most common mistake

Text or elements directly inside `<tr>` that are not `td/th`.

<tr> — Table row

Category: Table content | Status: CURRENT | Versions: HTML 3.2, HTML 4.01, HTML5

Description

The `<tr>` element defines a row of cells in a table. It contains `<th>` (header) and/or `<td>` (data) cells and is placed inside `<table>`, `<thead>`, `<tbody>` or `<tfoot>`. Rows are the fundamental unit of table construction: you build a table one row at a time, and the columns emerge from the cells in each row.

Syntax

```
<tr>
  <td>Cell 1</td>
  <td>Cell 2</td>
</tr>
```

Technical summary

Property	Value
Void element	No
Default display	table-row
Content categories	None
Permitted content	Zero or more <code><td></code> , <code><th></code> , <code><script></code> , <code><template></code>
Permitted parents	<code><table></code> (directly), <code><thead></code> , <code><tbody></code> , <code><tfoot></code>
Tag omission	End tag may be omitted if followed by another <code><tr></code> or if there is no more content in the parent
DOM interface	HTMLTableRowElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
align	left center right justify char	Obsolete. Use CSS text-align on cells.	HTML 3.2
bgcolor	colour	Obsolete. Use CSS background-color.	HTML 3.2
char / charoff	character / number	Obsolete.	HTML 4.01
valign	top middle bottom baseline	Obsolete. Use CSS vertical-align.	HTML 3.2

<tr>: attribute by attribute

Each attribute of <tr> explained in detail, with a minimal example you can copy.

align

Values: left | center | right | justify | char | Since: HTML 3.2

Obsolete. Use CSS text-align on cells.

Write align="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tr align="left">...</tr>
```

bgcolor

Values: colour | Since: HTML 3.2

Obsolete. Use CSS background-color.

Write bgcolor="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tr bgcolor="#336699">...</tr>
```

char / charoff

Values: character / number | Since: HTML 4.01

Obsolete.

Write char="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tr char="3">...</tr>
```

valign

Values: top | middle | bottom | baseline | Since: HTML 3.2

Obsolete. Use CSS vertical-align.

Write valign="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<tr valign="top">...</tr>
```

<tr>: worked examples

Three rows

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Ahmed</td>
    <td>20</td>
  </tr>
  <tr>
    <td>Sara</td>
    <td>22</td>
  </tr>
</table>
```

Row styling

```
<style>
  tr:nth-child(even) { background: #f9f9f9; }
  tr:hover { background: #eef; }
  tr.selected { outline: 2px solid #1b6fd8; }
</style>
```

Adding a row with JavaScript

```
<script>
  const row = document.querySelector('table').insertRow();
  row.insertCell().textContent = 'Omar';
  row.insertCell().textContent = '25';
</script>
```

Default browser CSS

```
tr { display: table-row; vertical-align: inherit; }
```

Accessibility

- Screen readers announce row numbers; keep one logical record per row.

Common mistakes

- Text or elements directly inside <tr> that are not td/th.
- Different numbers of cells per row.
- Nesting tr inside td without a new table.

See also

`<table>`, `<td>`, `<th>`, `<thead>`, `<tbody>`, `<tfoot>`

Quick card: <track>

Text track for media

The <track> element adds timed text tracks to <audio> or <video>: subtitles (translations), captions (dialogue plus sound descriptions for deaf and hard-of-hearing viewers), descriptions (spoken description of the video for blind users), chapters (navigation) and metadata (for scripts). Tracks are WebVTT files (.vtt). It is a void element placed after any <source> elements.

At a glance

Status	CURRENT
Versions	HTML5
Category	Image and multimedia
Closing tag	None (void element)
Display	none
Own attributes	src, kind, srclang, label, default
Related	<video>, <audio>, <source>

Minimal example

```
<video controls width="640" height="360">
  <source src="lesson.mp4" type="video/mp4">
  <track kind="captions" src="lesson-en.vtt" srclang="en"
label="English captions" default>
  <track kind="subtitles" src="lesson-ar.vtt" srclang="ar"
label="ةيبرعلا">
  <track kind="chapters" src="chapters.vtt" srclang="en">
</video>
```

Most common mistake

Placing track before source.

<track> — Text track for media

Category: Image and multimedia | Status: CURRENT | Versions: HTML5

Description

The <track> element adds timed text tracks to <audio> or <video>: subtitles (translations), captions (dialogue plus sound descriptions for deaf and hard-of-hearing viewers), descriptions (spoken description of the video for blind users), chapters (navigation) and metadata (for scripts). Tracks are WebVTT files (.vtt). It is a void element placed after any <source> elements.

Only one track per kind may have the default attribute. Track files loaded from another origin require CORS headers and the crossorigin attribute on the media element.

Syntax

```
<track kind="captions" src="captions.vtt" srclang="en"
label="English" default>
```

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	none
Content categories	None
Permitted content	None (void element)
Permitted parents	<audio>, <video>, after <source> and before flow content

Property	Value
Tag omission	No end tag
DOM interface	HTMLTrackElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	Required. The WebVTT file.	HTML5
kind	subtitles captions descriptions chapters metadata	The type of track. Default subtitles.	HTML5
srclang	language tag	Language of the track. Required for subtitles.	HTML5
label	text	Human-readable title shown in the track menu.	HTML5
default	boolean	Enable this track by default (one per kind).	HTML5

<track>: attribute by attribute

Each attribute of <track> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML5

Required. The WebVTT file.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<track src="https://example.com/page.html">
```

kind

Values: subtitles | captions | descriptions | chapters | metadata | Since: HTML5

The type of track. Default subtitles.

Write kind="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<track kind="subtitles">
```

srclang

Values: language tag | Since: HTML5

Language of the track. Required for subtitles.

Write srclang="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<track srclang="en">
```

label

Values: text | Since: HTML5

Human-readable title shown in the track menu.

Write label="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<track label="value">
```

default

Values: boolean | Since: HTML5

Enable this track by default (one per kind).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<track default>
```

<track>: worked examples

Video with captions and subtitles

```
<video controls width="640" height="360">
  <source src="lesson.mp4" type="video/mp4">
  <track kind="captions" src="lesson-en.vtt" srclang="en"
label="English captions" default>
  <track kind="subtitles" src="lesson-ar.vtt" srclang="ar"
label="آية برعل ا">
  <track kind="chapters" src="chapters.vtt" srclang="en">
</video>
```

A WebVTT file

WEBVTT

```
00:00:00.000 --> 00:00:04.000
Welcome to the HTML course.
```

```
00:00:04.500 --> 00:00:08.000
Today we learn about the footer element.
```

Save as lesson-en.vtt; the first line must be WEBVTT.

Default browser CSS

```
track { display: none; }
```

Accessibility

- Captions are required for accessible video (WCAG 1.2.2). Subtitles alone are not captions: captions also describe sounds.
- Provide descriptions for visual-only information.

Common mistakes

- Placing track before source.
- Wrong MIME type on the server (should be text/vtt).
- Two default tracks of the same kind.

See also

<video>, <audio>, <source>

Quick card: <tt>

Teletype text (obsolete)

The <tt> element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements <code>, <kbd>, <samp> or <var>, or CSS font-family: monospace on a .

At a glance

Status	OBSOLETE
Versions	HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5
Category	Obsolete presentational
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code>, <kbd>, <samp>, <var>, <pre>

Minimal example

```
<!-- Old -->
<tt>ls -la</tt>
```

```
<!-- New -->
<kbd>ls -la</kbd> <!-- user input -->
<code>ls -la</code> <!-- code -->
<span style="font-family: monospace">ls -la</span> <!-- style only -->
```

Most common mistake

Any use.

<tt> — Teletype text (obsolete)

Category: Obsolete presentational | Status: OBSOLETE | Versions: HTML 2.0, HTML 3.2, HTML 4.01; obsolete in HTML5

Description

The `<tt>` element (teletype) rendered text in a monospace font, imitating a teletype or typewriter. It was purely presentational and HTML5 removed it. Use the semantic elements `<code>`, `<kbd>`, `<samp>` or `<var>`, or CSS `font-family: monospace` on a `<span>`.

Syntax

```
<tt>text</tt>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Historical
Permitted content	Phrasing content
Permitted parents	Phrasing content
Tag omission	Neither
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<tt>: worked examples

Replacements

```
<!-- Old -->
<tt>ls -la</tt>
```

```
<!-- New -->
<kbd>ls -la</kbd> <!-- user input -->
<code>ls -la</code> <!-- code -->
<span style="font-family: monospace">ls -la</span> <!-- style only -->
```

Default browser CSS

```
tt { font-family: monospace; }
```

Common mistakes

- Any use.

See also

<code>, <kbd>, <samp>, <var>, <pre>

Quick card: <u>

Unarticulated annotation (underline)

The <u> element represents a span of inline text that should be rendered in a way that indicates a non-textual annotation: most commonly, marking misspelled words (as a word processor does with a wavy red line) or labelling proper names in Chinese text. Browsers render it with a solid underline. In HTML 4 <u> simply meant 'underline' and was deprecated; HTML5 redefined it.

At a glance

Status	CURRENT
Versions	HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	, <ins>, <s>,

Minimal example

```
<p>Please chek <u class="spelling">recieve</u> before sending.</p>
<style>.spelling { text-decoration: red wavy underline; }</style>
```

Most common mistake

Using <u> for emphasis or headings.

<u> — Unarticulated annotation (underline)

Category: Text-level semantics | Status: CURRENT | Versions: HTML 3.2, HTML 4.01 (deprecated), HTML5 (redefined)

Description

The `<u>` element represents a span of inline text that should be rendered in a way that indicates a non-textual annotation: most commonly, marking misspelled words (as a word processor does with a wavy red line) or labelling proper names in Chinese text. Browsers render it with a solid underline. In HTML 4 `<u>` simply meant 'underline' and was deprecated; HTML5 redefined it.

Because underlined text looks like a link on the web, avoid `<u>` for anything that is not one of its intended uses. For emphasis use `<em>`; for titles `<cite>`; for pure decoration CSS text-decoration.

Syntax

```
<u>text</u>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content

Property	Value
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<u>: worked examples

Spelling error

```
<p>Please chek <u class="spelling">recieve</u> before sending.</p>
<style>.spelling { text-decoration: red wavy underline; }</style>
```

Wrong use

```
<!-- Do not do this: looks like a link and carries no meaning -->
<p><u>Important</u></p>
```

```
<!-- Do this -->
<p><strong>Important</strong></p>
```

Default browser CSS

```
u { text-decoration: underline; }
```

Accessibility

- Underlined non-link text confuses users; they may try to click it.

Common mistakes

- Using <u> for emphasis or headings.

See also

, <ins>, <s>,

Quick card:

Unordered list

The element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an . If the order matters, use .

At a glance

Status	CURRENT
Versions	HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	type, compact
Related	, , <menu>, <dl>, <nav>

Minimal example

```
<ul>
  <li>Milk</li>
  <li>Bread</li>
  <li>Eggs</li>
</ul>
```

Most common mistake

Text directly inside outside .

 — Unordered list

Category: Text content | Status: CURRENT | Versions: HTML 1, HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<ul>` element represents an unordered list of items: a list where changing the order would not change the meaning, such as a shopping list, a set of features, or the links of a navigation menu. Browsers show each item with a bullet. Each item is an `<li>`. If the order matters, use `<ol>`.

Lists are among the most useful elements in HTML because they carry structure: screen readers announce how many items there are and let users jump between them. Navigation menus are almost always built as a `<ul>` inside a `<nav>`, with CSS removing the bullets and arranging the items horizontally.

Syntax

```
<ul>
  <li>Item</li>
  <li>Item</li>
</ul>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Flow content; palpable if it has at least one <code></code>
Permitted content	Zero or more <code></code> , <code><script></code> , <code><template></code>

Property	Value
Permitted parents	Any element that accepts flow content
Tag omission	Neither tag may be omitted
DOM interface	HTMLULElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
type	disc circle square	Obsolete. Bullet style. Use CSS list-style-type.	HTML 3.2
compact	boolean	Obsolete. Compact rendering.	HTML 2.0

: attribute by attribute

Each attribute of explained in detail, with a minimal example you can copy.

type

Values: disc | circle | square | Since: HTML 3.2

Obsolete. Bullet style. Use CSS list-style-type.

Write type="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<ul type="disc">...</ul>
```

compact

Values: boolean | Since: HTML 2.0

Obsolete. Compact rendering.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<ul compact>...</ul>
```


: worked examples

Shopping list

```
<ul>
  <li>Milk</li>
  <li>Bread</li>
  <li>Eggs</li>
</ul>
```

Shows three bulleted lines.

Nested lists

```
<ul>
  <li>Fruit
    <ul>
      <li>Apples</li>
      <li>Bananas</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

Nested lists get a different bullet automatically (disc, then circle, then square).

Bullet styles

```
<ul style="list-style-type: square">...</ul>
<ul style="list-style-type: '✓ '">...</ul>
<ul style="list-style-image: url(star.png)">...</ul>
```

Navigation menu

```
<style>
  .menu { list-style: none; margin: 0; padding: 0; display: flex; gap:
  20px; }
</style>
<nav>
  <ul class="menu">
    <li><a href="/">Home</a></li>
    <li><a href="/shop">Shop</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>
```

Custom bullets with ::marker

```
<style>
  li::marker { color: #e60000; font-size: 1.2em; }
</style>
```

Default browser CSS

```
ul { display: block; list-style-type: disc; margin: 1em 0; padding-  
inline-start: 40px; }
```

Accessibility

- Announced as 'list, 3 items'. In Safari, `list-style: none` removes list semantics; add `role="list"` to the `ul` to keep them.

Common mistakes

- Text directly inside `<ul>` outside `<li>`.
- Using `<ul>` for ordered steps.
- Using `<br>` or paragraphs with dashes instead of a real list.

See also

`<ol>`, `<li>`, `<menu>`, `<dl>`, `<nav>`

Quick card: <var>

Variable

The <var> element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with <code>, <kbd> and <samp>.

At a glance

Status	CURRENT
Versions	HTML 2.0, HTML 3.2, HTML 4.01, HTML5
Category	Text-level semantics
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	None (global attributes only)
Related	<code>, <kbd>, <samp>, <math>

Minimal example

```
<p>The area is <var>w</var> x <var>h</var>.</p>
```

Most common mistake

Using <i> or for variables.

<var> — Variable

Category: Text-level semantics | Status: CURRENT | Versions: HTML 2.0, HTML 3.2, HTML 4.01, HTML5

Description

The `<var>` element represents the name of a variable in a mathematical expression or a programming context, or a placeholder for a value the reader must supply. Browsers render it in italics. It is one of the four 'computer' phrase elements together with `<code>`, `<kbd>` and `<samp>`.

Syntax

```
<var>x</var>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, palpable content
Permitted content	Phrasing content
Permitted parents	Any element that accepts phrasing content
Tag omission	Neither tag may be omitted
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<var>: worked examples

Maths

<p>The area is <var>w</var> × <var>h</var>.</p>

Placeholder in instructions

<p>Type <kbd>cd <var>folder-name</var></kbd> to enter a folder.</p>

Default browser CSS

```
var { font-style: italic; }
```

Common mistakes

- Using <i> or for variables.

See also

<code>, <kbd>, <samp>, <math>

Quick card: <video>

Embedded video

The <video> element embeds a video player in the document. The video file is given by src or by one or more <source> children (so that the browser can pick a supported format), followed by optional <track> elements for captions and subtitles, and finally fallback content for browsers without video support. Common formats are MP4 (H.264/AAC, universally supported), WebM (VP9/AV1, smaller) and, in Safari, HEVC.

At a glance

Status	CURRENT
Versions	HTML5
Category	Image and multimedia
Closing tag	Required unless noted under Tag omission
Display	inline
Own attributes	src, controls, autoplay, loop, muted, poster, preload, width, playsinline, crossorigin, controlslist, disablepictureinpicture, disableremoteautoplay
Related	<audio>, <source>, <track>, <iframe>, <picture>

Minimal example

```
<video controls width="640" height="360" poster="poster.jpg"
preload="metadata">
  <source src="lesson.webm" type="video/webm">
  <source src="lesson.mp4" type="video/mp4">
  <track kind="captions" src="lesson-en.vtt" srclang="en"
label="English" default>
  <p>Your browser cannot play this video. <a href="lesson.
mp4">Download it</a>.</p>
</video>
```

Most common mistake

Expecting sound autoplay to work.

<video> — Embedded video

Category: Image and multimedia | Status: CURRENT | Versions: HTML5

Description

The `<video>` element embeds a video player in the document. The video file is given by `src` or by one or more `<source>` children (so that the browser can pick a supported format), followed by optional `<track>` elements for captions and subtitles, and finally fallback content for browsers without video support. Common formats are MP4 (H.264/AAC, universally supported), WebM (VP9/AV1, smaller) and, in Safari, HEVC.

The `controls` attribute shows the built-in play/pause, seek, volume, captions, picture-in-picture and fullscreen controls. Without it you must provide your own controls through the `HTMLMediaElement` API. Autoplay with sound is blocked by browsers; autoplay muted (for background video) is allowed. Set width and height or CSS aspect-ratio to prevent layout shift, and use `preload="none"` or `"metadata"` to avoid downloading large files that may never be played.

Syntax

```
<video src="clip.mp4" controls width="640" height="360"></video>
```

Technical summary

Property	Value
Void element	No
Default display	inline
Content categories	Flow content, phrasing content, embedded content; interactive and palpable if controls is present

Property	Value
Permitted content	If src: zero or more <track> then transparent content with no media elements. Otherwise: zero or more <source>, zero or more <track>, then transparent content with no media elements
Permitted parents	Any element that accepts embedded content
Tag omission	Neither tag may be omitted
DOM interface	HTMLVideoElement

Attributes

In addition to the global attributes (Part 4) and event handler attributes (Part 5), this element supports the following:

Attribute	Values	Description	Since
src	URL	The video file. Or use <source> children.	HTML5
controls	boolean	Show built-in controls.	HTML5
autoplay	boolean	Start automatically (only reliable with muted).	HTML5
loop	boolean	Repeat when finished.	HTML5
muted	boolean	Start muted.	HTML5
poster	URL	Image shown before the video plays or while loading.	HTML5
preload	none metadata auto	How much to download before play.	HTML5
width / height	pixels	Display size; also prevents layout shift.	HTML5
playsinline	boolean	On iPhone, play inline instead of forcing fullscreen.	Living Standard

Attribute	Values	Description	Since
crossorigin	anonymous use-credentials	CORS mode for the media (needed for canvas processing and cross-origin tracks).	HTML5
controlslist	nodownload nofullscreen noremoteplayback	Hide specific native controls (Chromium).	Non-standard
disablepictureinpicture	boolean	Hide the picture-in-picture option.	Living Standard
disableremoteplayback	boolean	Disable casting.	Living Standard

<video>: attribute by attribute

Each attribute of <video> explained in detail, with a minimal example you can copy.

src

Values: URL | Since: HTML5

The video file. Or use <source> children.

Write src="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video src="https://example.com/page.html">...</video>
```

controls

Values: boolean | Since: HTML5

Show built-in controls.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video controls>...</video>
```

autoplay

Values: boolean | Since: HTML5

Start automatically (only reliable with muted).

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video autoplay>...</video>
```

loop

Values: boolean | Since: HTML5

Repeat when finished.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video loop>...</video>
```

muted

Values: boolean | Since: HTML5

Start muted.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video muted>...</video>
```

poster

Values: URL | Since: HTML5

Image shown before the video plays or while loading.

Write `poster="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video poster="https://example.com/page.html">...</video>
```

preload

Values: none | metadata | auto | Since: HTML5

How much to download before play.

Write `preload="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video preload="none">...</video>
```

width / height

Values: pixels | Since: HTML5

Display size; also prevents layout shift.

Write `width="..."` inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video width="3">...</video>
```

playsinline

Values: boolean | Since: Living Standard

On iPhone, play inline instead of forcing fullscreen.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video playsinline>...</video>
```

crossorigin

Values: anonymous | use-credentials | Since: HTML5

CORS mode for the media (needed for canvas processing and cross-origin tracks).

Write crossorigin="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video crossorigin="anonymous">...</video>
```

controlslist

Values: nodownload | nofullscreen | noremoteplayback | Since: Non-standard

Hide specific native controls (Chromium).

Write controlslist="..." inside the start tag. Quote the value; several attributes are separated by spaces.

```
<video controlslist="nodownload">...</video>
```

disablepictureinpicture

Values: boolean | Since: Living Standard

Hide the picture-in-picture option.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video disablepictureinpicture>...</video>
```

disableremoteplayback

Values: boolean | Since: Living Standard

Disable casting.

Boolean attribute: write only the name to switch it on; leave it out to switch it off.

```
<video disableremoteplayback>...</video>
```

<video>: worked examples

Complete accessible video

```
<video controls width="640" height="360" poster="poster.jpg"
preload="metadata">
  <source src="lesson.webm" type="video/webm">
  <source src="lesson.mp4" type="video/mp4">
  <track kind="captions" src="lesson-en.vtt" srclang="en"
label="English" default>
  <p>Your browser cannot play this video. <a href="lesson.
mp4">Download it</a>.</p>
</video>
```

Background hero video

```
<video autoplay muted loop playsinline poster="hero.jpg" aria-
hidden="true">
  <source src="hero.mp4" type="video/mp4">
</video>
```

Muted autoplay is allowed; aria-hidden because it is decorative. Respect prefers-reduced-motion by pausing it with JavaScript.

Responsive sizing

```
<style>
  video { width: 100%; height: auto; aspect-ratio: 16 / 9; background:
  #000; }
</style>
```

Custom play button

```
<video id="v" src="clip.mp4" width="400"></video>
<button onclick="const v = document.getElementById('v'); v.paused ? v.
play() : v.pause()">Play / Pause</button>
```

Default browser CSS

```
video { object-fit: contain; }
video:not([controls]) { /* nothing shown until play */ }
```

Accessibility

- Provide captions with `<track kind="captions">`.
- Provide a transcript for audio and visual content.
- Never autoplay with sound.
- Keep native controls unless your custom controls are fully keyboard accessible.

- Avoid flashing content.

Common mistakes

- Expecting sound autoplay to work.
- A single format only.
- Huge files with `preload="auto"`.
- No poster, showing a black box until play.
- Missing `playsinline` on iOS.

See also

`<audio>`, `<source>`, `<track>`, `<iframe>`, `<picture>`

Quick card: `<wbr>`

Word break opportunity

The `<wbr>` element marks a position inside a word where the browser may break the line if necessary, without inserting a hyphen or a visible character. It is useful for very long words, URLs and code identifiers that would otherwise overflow their container. If no break is needed, nothing is displayed. It is a void element.

At a glance

Status	CURRENT
Versions	Netscape / IE extension, standardised in HTML5
Category	Text-level semantics
Closing tag	None (void element)
Display	inline
Own attributes	None (global attributes only)
Related	<code>
</code> , <code><nobr></code>

Minimal example

```
<p>Visit https://example.com/<wbr>very/<wbr>long/<wbr>path/<wbr>to/  
<wbr>a/<wbr>resource</p>
```

Most common mistake

Using `<br>` where a soft break is wanted.

<wbr> — Word break opportunity

Category: Text-level semantics | Status: CURRENT | Versions: Netscape / IE extension, standardised in HTML5

Description

The <wbr> element marks a position inside a word where the browser may break the line if necessary, without inserting a hyphen or a visible character. It is useful for very long words, URLs and code identifiers that would otherwise overflow their container. If no break is needed, nothing is displayed. It is a void element.

Alternatives: the zero-width space character (​) has the same effect; the soft hyphen (­) breaks with a hyphen; CSS overflow-wrap: anywhere allows breaks anywhere in overflowing words.

Syntax

Super<wbr>cali<wbr>fragilistic

Technical summary

Property	Value
Void element	Yes (no closing tag)
Default display	inline
Content categories	Flow content, phrasing content
Permitted content	None (void element)
Permitted parents	Any element that accepts phrasing content

Property	Value
Tag omission	No end tag
DOM interface	HTMLElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<wbr>: worked examples

Long URL

```
<p>Visit https://example.com/<wbr>very/<wbr>long/<wbr>path/<wbr>to/
<wbr>a/<wbr>resource</p>
```

The URL can break after each slash instead of overflowing.

Long identifier

```
<code>HTMLTable<wbr>Section<wbr>Element</code>
```

Default browser CSS

```
wbr { content: '\200B'; }
```

Accessibility

- Invisible to screen readers; does not affect pronunciation.

Common mistakes

- Using
 where a soft break is wanted.
- Writing </wbr>.

See also

, <nobr>

Quick card: <xmp>

Example text (obsolete)

The <xmp> element (example) displayed its content literally, without interpreting any markup, in a monospace font, so that authors could show HTML source without escaping < and &. It was one of the 1991 elements, deprecated in HTML 2.0 and removed in HTML 3.2, although browsers still support it for old pages. Use <pre> with escaped entities.

At a glance

Status	OBSOLETE
Versions	HTML 1, HTML 2.0 (deprecated); obsolete
Category	Obsolete text content
Closing tag	Required unless noted under Tag omission
Display	block
Own attributes	None (global attributes only)
Related	<pre>, <listing>, <plaintext>, <code>

Minimal example

<pre><p>shown literally</p></pre>

Most common mistake

Any use.

<xmp> — Example text (obsolete)

Category: Obsolete text content | Status: OBSOLETE | Versions: HTML 1, HTML 2.0 (deprecated); obsolete

Description

The `<xmp>` element (example) displayed its content literally, without interpreting any markup, in a monospace font, so that authors could show HTML source without escaping `<` and `&`. It was one of the 1991 elements, deprecated in HTML 2.0 and removed in HTML 3.2, although browsers still support it for old pages. Use `<pre>` with escaped entities.

Syntax

```
<xmp><p>shown literally</p></xmp>
```

Technical summary

Property	Value
Void element	No
Default display	block
Content categories	Historical
Permitted content	Raw text
Permitted parents	Flow content
Tag omission	Neither
DOM interface	HTMLPreElement

Attributes

This element supports only the global attributes (Part 4) and event handler attributes (Part 5).

<xmp>: worked examples

Replacement

```
<pre>&lt;p&gt;shown literally&lt;/p&gt;</pre>
```

Default browser CSS

```
xmp { display: block; font-family: monospace; white-space: pre;
margin: 1em 0; }
```

Common mistakes

- Any use.

See also

<pre>, <listing>, <plaintext>, <code>

The input Element in Depth

Every value of the type attribute with its own entry.

Input types overview

| type | Control | Since |
|----------------|---------------------|----------|
| text | Single-line text | HTML 2.0 |
| password | Password | HTML 2.0 |
| checkbox | Check box | HTML 2.0 |
| radio | Radio button | HTML 2.0 |
| submit | Submit button | HTML 2.0 |
| reset | Reset button | HTML 2.0 |
| button | Generic button | HTML 3.2 |
| hidden | Hidden field | HTML 2.0 |
| image | Image submit button | HTML 2.0 |
| file | File chooser | HTML 3.2 |
| email | Email address | HTML5 |
| url | Web address | HTML5 |
| tel | Telephone number | HTML5 |
| search | Search field | HTML5 |
| number | Numeric field | HTML5 |
| range | Slider | HTML5 |
| color | Colour picker | HTML5 |
| date | Date | HTML5 |
| time | Time of day | HTML5 |
| datetime-local | Local date and time | HTML5 |
| month | Month and year | HTML5 |
| week | Week of the year | HTML5 |

<input type="text"> — Single-line text

Since: HTML 2.0

The default type. A single-line text field. Line breaks are removed from the value. Use it for names, titles, and anything that has no more specific type.

Type-specific attributes

| Attribute | Note |
|-----------------------|--|
| maxlength / minlength | Length limits. |
| pattern | Regular expression to validate. |
| placeholder | Hint text. |
| size | Visible width in characters. |
| list | Datalist of suggestions. |
| autocomplete | Autofill token such as name, given-name, organization. |
| dirname | Submit text direction. |

Example

```
<label for="n">Full name</label>
<input id="n" name="name" type="text" autocomplete="name" required>
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="password"> — Password

Since: HTML 2.0

A single-line field whose characters are obscured with dots or asterisks. The value is still sent as plain text in the form submission, so the page must be served over HTTPS. Browsers offer to save and generate passwords when autocomplete is set to current-password or new-password.

Type-specific attributes

| Attribute | Note |
|-----------------------|--|
| minlength / maxlength | Length rules. |
| pattern | Complexity rules (explain them in title). |
| autocomplete | current-password, new-password, one-time-code. |

Example

```
<label for="pw">Password</label>
<input id="pw" name="password" type="password" minlength="8"
autocomplete="new-password" required>
```

Add a 'show password' toggle for usability.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="checkbox"> — Check box

Since: HTML 2.0

An on/off box. Each checkbox is independent; several checkboxes may share a name to send multiple values. Only checked boxes are submitted (as name=value). The indeterminate visual state can be set from JavaScript for 'some children selected'.

Type-specific attributes

| Attribute | Note |
|-----------|--|
| checked | Initially ticked. |
| value | Sent when checked; default 'on'. |
| required | Must be ticked (useful for 'I agree'). |

Example

```
<label><input type="checkbox" name="topics" value="html" checked>  
HTML</label>  
<label><input type="checkbox" name="topics" value="css"> CSS</label>
```

Wrap the input in the label so the text is clickable.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="radio"> — Radio button

Since: HTML 2.0

One of a group of mutually exclusive options. All radio buttons with the same name form a group and only one can be selected. Arrow keys move the selection within the group. Always provide a default or make the group required, and caption the group with fieldset/legend.

Type-specific attributes

| Attribute | Note |
|-----------|---|
| checked | Initially selected. |
| value | Sent for the selected button. |
| required | One button in the group must be selected. |

Example

```
<fieldset>
  <legend>Delivery</legend>
  <label><input type="radio" name="d" value="standard" checked>
Standard</label>
  <label><input type="radio" name="d" value="express"> Express</label>
</fieldset>
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="submit"> — Submit button

Since: HTML 2.0

A button that submits the form. Its value is the button's label and, if it has a name, is submitted too. Prefer the <button> element, which can contain HTML content.

Type-specific attributes

| Attribute | Note |
|--|---------------------------|
| value | Label (default 'Submit'). |
| formation,
formmethod,
formenctype,
formtarget,
formnovalidate | Override form settings. |

Example

```
<input type="submit" value="Send message">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="reset"> — Reset button

Since: HTML 2.0

A button that restores all controls of the form to their initial values. Rarely useful and often clicked by mistake; most forms omit it.

Type-specific attributes

| Attribute | Note |
|-----------|--------------------------|
| value | Label (default 'Reset'). |

Example

```
<input type="reset" value="Clear form">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="button"> — Generic button

Since: HTML 3.2

A button with no default behaviour, for use with JavaScript. Prefer `<button type="button">`.

Type-specific attributes

| Attribute | Note |
|-----------|--------|
| value | Label. |

Example

```
<input type="button" value="Show help" onclick="alert('Help')">
```

Also accepted: the common input attributes `name`, `value`, `disabled`, `form`, `autofocus`, `required` (where meaningful), and all global attributes.

<input type="hidden"> — Hidden field

Since: HTML 2.0

A control that is not displayed but whose value is submitted with the form. Used for record ids, CSRF tokens and other data the user does not edit. The name `_charset_` has a special meaning: the browser fills it with the form's encoding.

Type-specific attributes

| Attribute | Note |
|-----------|----------------------|
| value | The value to submit. |

Example

```
<input type="hidden" name="csrf" value="a1b2c3">
```

Never rely on hidden fields for security; users can change them.

Also accepted: the common input attributes `name`, `value`, `disabled`, `form`, `autofocus`, `required` (where meaningful), and all global attributes.

<input type="image"> — Image submit button

Since: HTML 2.0

A graphical submit button showing the image in src. When clicked it submits the form together with the click coordinates as name.x and name.y. Requires alt.

Type-specific attributes

| Attribute | Note |
|-----------------|----------------------------|
| src | Image URL. |
| alt | Required text alternative. |
| width / height | Size. |
| formaction etc. | Override form settings. |

Example

```
<input type="image" src="go.png" alt="Search" width="32" height="32">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="file"> — File chooser

Since: HTML 3.2

Lets the user select one or more files from their device to upload. The form must use `method="post"` and `enctype="multipart/form-data"`. The selected files are available in JavaScript as `input.files`. For security the value cannot be set by script and shows only a fake path.

Type-specific attributes

| Attribute | Note |
|-----------------|---|
| accept | Allowed types: image/*, video/*, audio/*, .pdf, application/pdf, .doc,.docx |
| multiple | Allow several files. |
| capture | user environment: open camera/mic directly on phones. |
| webkitdirectory | Non-standard: choose a folder. |

Example

```
<label for="cv">Upload your CV (PDF)</label>
<input id="cv" name="cv" type="file" accept=".pdf,application/pdf">
```

Also accepted: the common input attributes `name`, `value`, `disabled`, `form`, `autofocus`, `required` (where meaningful), and all global attributes.

<input type="email"> — Email address

Since: HTML5

A text field validated as an email address (or a comma-separated list with multiple). Mobile keyboards show @ and . keys. The value is trimmed and validated as user@host; a domain without a dot is accepted.

Type-specific attributes

| Attribute | Note |
|--|---|
| multiple | Allow several addresses separated by commas. |
| pattern | Additional restriction, for example a company domain. |
| list | Suggestions. |
| maxlength /
minlength / size /
placeholder /
readonly | As text. |

Example

```
<label for="e">Email</label>  
<input id="e" name="email" type="email" autocomplete="email" required>
```

Still validate on the server.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="url"> — Web address

Since: HTML5

A text field validated as an absolute URL (it must include a scheme such as https://). Mobile keyboards show / and .com keys.

Type-specific attributes

| Attribute | Note |
|---|--|
| pattern | Extra restriction, for example https only. |
| list, maxlength, minlength, placeholder, readonly, size | As text. |

Example

```
<label for="site">Website</label>
<input id="site" name="site" type="url" placeholder="https://example.com">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="tel"> — Telephone number

Since: HTML5

A text field for telephone numbers. It is not validated (formats vary too much worldwide), but phones show a telephone keypad. Add pattern for a specific format.

Type-specific attributes

| Attribute | Note |
|---|--|
| pattern | For example <code>[0-9]{11}</code> or <code>\+20[0-9]{10}</code> . |
| list, maxlength, minlength, placeholder, readonly, size | As text. |

Example

```
<label for="t">Mobile</label>
<input id="t" name="tel" type="tel" autocomplete="tel" pattern="01[0-9]{9}" title="11 digits starting with 01">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="search"> — Search field

Since: HTML5

A text field for search terms. Functionally identical to text, but some browsers style it with rounded corners and a clear (x) button, and mobile keyboards show a Search key. Line breaks are removed.

Type-specific attributes

| Attribute | Note |
|--|---|
| list, maxlength, minlength, pattern, placeholder, readonly, size | As text. |
| incremental | Non-standard (WebKit): fire search events while typing. |

Example

```
<search>
  <label for="q">Search</label>
  <input id="q" name="q" type="search" placeholder="Search books">
</search>
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="number"> — Numeric field

Since: HTML5

A field for entering a number, usually with spinner arrows. Only digits, sign, decimal point and exponent are accepted; the value is validated against min, max and step. Use it only for actual quantities. For identifiers such as credit card numbers or postal codes use type="text" with inputmode="numeric" because they can have leading zeros and are not meant to be incremented.

Type-specific attributes

| Attribute | Note |
|-----------------------------|--|
| min / max | Range. |
| step | Increment; 'any' allows decimals of any precision. Default 1, which rejects 2.5 unless step is 0.5 or any. |
| placeholder, readonly, list | As text. |

Example

```
<label for="qty">Quantity</label>
<input id="qty" name="qty" type="number" min="1" max="99" step="1"
value="1">
```

Scrolling the mouse wheel over a focused number input changes the value in some browsers.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="range"> — Slider

Since: HTML5

A slider for choosing an approximate number where the exact value is not important, such as volume or brightness. Default range 0-100, step 1. Show the current value with an <output> because the slider itself does not display it.

Type-specific attributes

| Attribute | Note |
|-----------|-----------------------------------|
| min / max | Range. |
| step | Increment. |
| list | Datalist for tick marks. |
| orient | Non-standard (Firefox): vertical. |

Example

```
<label for="vol">Volume</label>
<input id="vol" name="vol" type="range" min="0" max="100" value="40"
oninput="out.value = this.value">
<output id="out">40</output>
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="color"> — Colour picker

Since: HTML5

A control for choosing a colour, usually opening the operating system colour picker. The value is always a lowercase hexadecimal colour such as #ff0000; alpha (transparency) is not supported unless the experimental alpha attribute is used.

Type-specific attributes

| Attribute | Note |
|-------------------|---|
| value | Initial colour in #rrggbb form (default #000000). |
| list | Datalist of suggested colours. |
| alpha, colorspace | Experimental. |

Example

```
<label for="c">Theme colour</label>
<input id="c" name="color" type="color" value="#e60000">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="date"> — Date

Since: HTML5

A control for a year, month and day, usually with a calendar pop-up. The value is always in the format YYYY-MM-DD regardless of how the browser displays it to the user, so it is safe to process on the server. Safari on macOS supports it since version 14.1.

Type-specific attributes

| Attribute | Note |
|--------------------|--|
| min / max | Earliest and latest date, as YYYY-MM-DD. |
| step | In days. |
| list | Suggested dates. |
| readonly, required | As usual. |

Example

```
<label for="d">Delivery date</label>
<input id="d" name="date" type="date" min="2026-09-12" max="2026-12-31" required>
```

Set min to today's date with JavaScript to prevent past dates.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="time"> — Time of day

Since: HTML5

A control for hours and minutes (and optionally seconds), without a time zone. The value format is HH:MM or HH:MM:SS in 24-hour form.

Type-specific attributes

| Attribute | Note |
|-----------|--|
| min / max | As HH:MM. |
| step | In seconds; 60 = minutes only (default), 1 = show seconds. |
| list | Suggested times. |

Example

```
<label for="t">Appointment time</label>
<input id="t" name="time" type="time" min="09:00" max="17:00"
step="900">
```

step 900 = 15-minute increments.

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="datetime-local"> — Local date and time

Since: HTML5

A control for a date and time with no time zone, format YYYY-MM-DDTHH:MM. Convert to UTC on the server if needed. The older type="datetime" (with time zone) was removed from the standard.

Type-specific attributes

| Attribute | Note |
|-----------|----------------------|
| min / max | As YYYY-MM-DDTHH:MM. |
| step | In seconds. |
| list | Suggestions. |

Example

```
<label for="dt">Event start</label>
<input id="dt" name="start" type="datetime-local" value="2026-10-01T19:00">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="month"> — Month and year

Since: HTML5

A control for a month in a year, value format YYYY-MM. Useful for card expiry dates and reports. Not supported in Firefox or Safari desktop, which fall back to a text field.

Type-specific attributes

| Attribute | Note |
|-----------|--------------|
| min / max | As YYYY-MM. |
| step | In months. |
| list | Suggestions. |

Example

```
<label for="exp">Card expiry</label>
<input id="exp" name="exp" type="month" autocomplete="cc-exp">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

<input type="week"> — Week of the year

Since: HTML5

A control for an ISO week, value format YYYY-Www (for example 2026-W37). Limited browser support (Chromium only); others show a text field.

Type-specific attributes

| Attribute | Note |
|-----------|--------------|
| min / max | As YYYY-Www. |
| step | In weeks. |
| list | Suggestions. |

Example

```
<label for="w">Week</label>
<input id="w" name="week" type="week" value="2026-W37">
```

Also accepted: the common input attributes name, value, disabled, form, autofocus, required (where meaningful), and all global attributes.

Obsolete and Deprecated Features

Old elements and attributes you may meet, with modern replacements.

Obsolete elements

These elements appeared in HTML 1, 2.0, 3.2, 4.01 or as browser extensions and are no longer part of the standard. Browsers still render most of them for compatibility with old pages, but they must not be used in new documents. Each has a full entry in Part 6; this table summarises the replacements.

| Element | Status | Was used for | Use instead |
|---------------|--------------|---|----------------------------------|
| <acronym> | obsolete | Acronym (obsolete) | <abbr> |
| <applet> | obsolete | Java applet (obsolete) | <object>, <embed>, <canvas> |
| <basefont> | obsolete | Base font (obsolete) | , <body> |
| <bgsound> | obsolete | Background sound (non-standard, obsolete) | <audio> |
| <big> | obsolete | Bigger text (obsolete) | <small>, |
| <blink> | obsolete | Blinking text (non-standard, obsolete) | <marquee> |
| <center> | obsolete | Centred block (obsolete) | <div> |
| <content> | obsolete | Shadow DOM insertion point (obsolete) | <slot>, <template>, <shadow> |
| <dir> | obsolete | Directory list (obsolete) | , <menu> |
| <fencedframe> | experimental | Fenced frame (experimental) | <iframe> |
| | obsolete | Font (obsolete) | <basefont>, |
| <frame> | obsolete | Frame (obsolete) | <frameset>, <noframes>, <iframe> |

| Element | Status | Was used for | Use instead |
|------------------|------------------|---|-------------------------------------|
| <frameset>
> | obsolete | Frameset (obsolete) | <frame>,
<noframes>,
<iframe> |
| <image> | obsolete | Image (non-standard alias,
obsolete) | |
| <isindex> | obsolete | Search index prompt
(obsolete) | <form>, <input> |
| <keygen> | obsolete | Key-pair generator (obsolete) | <input>, <form> |
| <listing> | obsolete | Listing (obsolete) | <pre>, <xmp>,
<plaintext> |
| <marquee>
> | obsolete | Scrolling text (obsolete) | <blink> |
| <menuitem> | obsolete | Menu item (obsolete) | <menu>, <button> |
| <multicol>
> | obsolete | Multi-column text
(non-standard, obsolete) | <div> |
| <nextid> | obsolete | Next identifier (HTML 1 editor
hint) | <head>, <isindex>,
<listing> |
| <nobr> | obsolete | No line break (non-standard,
obsolete) | <wbr>,
 |
| <noembed> | obsolete | Fallback for embed
(obsolete) | <embed>, <object> |
| <noframes> | obsolete | Fallback for frames
(obsolete) | <frameset>,
<frame> |
| <param> | deprecat
ed | Object parameter
(deprecated) | <object>, <embed> |
| <plaintext>
> | obsolete | Plain text (obsolete) | <pre>, <xmp>,
<listing> |
| <portal> | experim
ental | Portal (experimental,
abandoned) | <iframe>,
<fencedframe> |

| Element | Status | Was used for | Use instead |
|--------------------|------------------|---|---------------------------------------|
| <rb> | deprecat
ed | Ruby base (deprecated) | <ruby>, <rt>, <rp>, <rtc> |
| <rtc> | deprecat
ed | Ruby text container (deprecated) | <ruby>, <rt>, <rb> |
| <selected content> | experim
ental | Selected option display (customisable select) | <select>, <option>, <button> |
| <shadow > | obsolete | Shadow root insertion point (obsolete) | <slot>, <template>, <content> |
| <spacer> | obsolete | Spacer (non-standard, obsolete) |
, <hr> |
| <strike> | obsolete | Strikethrough (obsolete) | <s>, |
| <tt> | obsolete | Teletype text (obsolete) | <code>, <kbd>, <samp>, <var>, <pre> |
| <xmp> | obsolete | Example text (obsolete) | <pre>, <listing>, <plaintext>, <code> |

Obsolete attributes

Presentational attributes were the HTML 3.2 way to control appearance. All of them are replaced by CSS.

| Attribute | Was on | CSS replacement |
|------------|--|---|
| align | div, p, h1-h6, table, tr, td, th, img, hr, caption, legend, iframe, object | text-align, float, vertical-align, margin |
| bgcolor | body, table, tr, td, th | background-color |
| background | body, table, td | background-image |

| Attribute | Was on | CSS replacement |
|---|--|---|
| border | img, table, object | border |
| cellpadding /
cellspacing | table | padding on cells / border-spacing,
border-collapse |
| width / height | table, td, th, hr,
pre, applet | width, height |
| hspace / vspace | img, object,
applet | margin |
| valign | tr, td, th, col,
tbody, thead,
tfoot | vertical-align |
| nowrap | td, th | white-space: nowrap |
| clear | br | clear |
| color / face / size | font, basefont | color, font-family, font-size |
| size / noshade | hr | height, border |
| text / link / vlink
/ alink | body | color, a:link, a:visited, a:active |
| type | ul, li | list-style-type |
| compact | dl, ol, ul, dir,
menu | margin, line-height |
| frameborder /
marginwidth /
marginheight /
scrolling | iframe, frame | border, padding, overflow |
| language | script | type (or omit) |
| name | a, img, form (as
id) | id |
| longdesc | img, iframe | a link or aria-describedby |
| summary | table | caption, aria-describedby |
| charset | a, link, script | HTTP headers |

| Attribute | Was on | CSS replacement |
|---|---------|-----------------|
| rev | a, link | rel |
| scheme | meta | - |
| manifest | html | Service workers |
| accept | form | accept on input |
| archive, classid,
codebase,
codetype,
declare, standby | object | - |

Attribute Reference

Every element-specific attribute listed alphabetically with the elements that accept it.

Alphabetical attribute index

Global attributes are in Part 4 and event attributes in Part 5. This index covers attributes specific to particular elements. Look up the attribute, find the element, then read the full entry in Part 6.

| Attribute | Used on |
|---------------------|--|
| abbr | <td>, <th> |
| accept | <form>, <input> |
| accept-charset | <form> |
| action | <form>, <isindex> |
| align | <applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr> |
| alink | <body> |
| allow | <fencedframe>, <iframe> |
| allowfullscreen | <iframe> |
| allowpaymentrequest | <iframe> |
| alpha | <input> |
| alt | <applet>, <area>, <image>, , <input> |
| archive | <applet>, <object> |
| aria-label | <svg> |
| aria-labelledby | <svg> |
| as | <link> |
| async | <script> |
| attributionsrc | <a>, , <script> |
| autocomplete | <form>, <input>, <select>, <textarea> |

| Attribute | Used on |
|--------------|--|
| autofocus | <button>, <input>, <keygen>, <select>, <textarea> |
| autoplay | <audio>, <video> |
| axis | <td>, <th> |
| background | <body> |
| balance | <bgsound> |
| behavior | <marquee> |
| bgcolor | <body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr> |
| blocking | <link>, <script>, <style> |
| border | <frameset>, , <object>, <table> |
| bottommargin | <body> |
| capture | <input> |
| cellpadding | <table> |
| cellspacing | <table> |
| challenge | <keygen> |
| char | <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr> |
| charoff | <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr> |
| charset | <a>, <link>, <meta>, <script> |
| checked | <input>, <menuitem> |
| cite | <blockquote>, , <ins>, <q> |
| classid | <object> |
| clear |
 |
| closedby | <dialog> |
| code | <applet> |

| Attribute | Used on |
|----------------|---|
| codebase | <applet>, <object> |
| codetype | <object> |
| color | <basefont>, , <hr>, <link> |
| colorspace | <input> |
| cols | <frameset>, <multicol>, <textarea> |
| colspan | <td>, <th> |
| command | <button>, <menuitem> |
| commandfor | <button> |
| compact | <dir>, <dl>, <menu>, , |
| content | <meta> |
| controls | <audio>, <video> |
| controlslist | <audio>, <video> |
| coords | <a>, <area> |
| credentialless | <iframe> |
| crossorigin | <audio>, , <link>, <script>, <video> |
| csp | <iframe> |
| data | <object> |
| datetime | , <ins>, <time> |
| declare | <object> |
| decoding | |
| default | <menuitem>, <track> |
| defer | <script> |
| dir | <bdi>, <bdo>, <html> |
| direction | <marquee> |
| dirname | <input>, <textarea> |

| Attribute | Used on |
|-------------------------|---|
| disabled | <button>, <fieldset>, <input>, <keygen>, <link>, <menuitem>, <optgroup>, <option>, <select>, <textarea> |
| disablepictureinpicture | <video> |
| disableremoteplayback | <audio>, <video> |
| display | <math> |
| download | <a>, <area> |
| elementtiming | |
| enctype | <form> |
| etc. | <image> |
| event | <script> |
| face | <basefont>, |
| fetchpriority | , <link>, <script> |
| fill | <svg> |
| for | <label>, <output>, <script> |
| form | <button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <object>, <output>, <select>, <textarea> |
| formaction | <button>, <input> |
| formenctype | <button>, <input> |
| formmethod | <button>, <input> |
| formnovalidate | <button>, <input> |
| formtarget | <button>, <input> |
| frame | <table> |
| frameborder | <frame>, <frameset>, <iframe> |

| Attribute | Used on |
|--------------|--|
| framespacing | <frameset> |
| gutter | <multicol> |
| headers | <td>, <th> |
| height | <applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video> |
| high | <meter> |
| href | <a>, <area>, <base>, <link> |
| hreflang | <a>, <link> |
| hspace | <applet>, , <marquee>, <object> |
| http-equiv | <meta> |
| icon | <menuitem> |
| id | <input> |
| imagesizes | <link> |
| imagesrcset | <link> |
| incremental | <input> |
| inputmode | <input> |
| integrity | <link>, <script> |
| ismap | |
| itemprop | <meta> |
| keytype | <keygen> |
| kind | <track> |
| label | <menu>, <menuitem>, <optgroup>, <option>, <track> |
| lang | <html> |
| language | <script> |

| Attribute | Used on |
|---------------|---|
| leftmargin | <body> |
| link | <body> |
| list | <input> |
| loading | <iframe>, |
| longdesc | <frame>, <iframe>, |
| loop | <audio>, <bgsound>, <marquee>, <video> |
| low | <meter> |
| lowsrc | |
| manifest | <html> |
| marginheight | <frame>, <iframe> |
| marginwidth | <frame>, <iframe> |
| max | <input>, <meter>, <progress> |
| maxlength | <input>, <textarea> |
| media | <link>, <meta>, <source>, <style> |
| method | <form> |
| min | <input>, <meter> |
| minlength | <input>, <textarea> |
| moz-opaque | <canvas> |
| mozactionhint | <input> |
| multiple | <input>, <select> |
| muted | <audio>, <video> |
| n | <nextid> |
| name | <a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea> |

| Attribute | Used on |
|---------------------|---------------------|
| noreferrer | <area> |
| nomodule | <script> |
| nonce | <script>, <style> |
| noresize | <frame> |
| noshade | <hr> |
| novalidate | <form> |
| nowrap | <dd>, <td>, <th> |
| object | <applet> |
| onbeforeunload | <body> |
| onhashchange | <body> |
| onlanguagechange | <body> |
| onload | <body> |
| onpopstate | <body> |
| onresize | <body> |
| onstorage | <body> |
| onunload | <body> |
| open | <details>, <dialog> |
| optimum | <meter> |
| orient | <input> |
| pattern | <input> |
| ping | <a>, <area> |
| placeholder | <input>, <textarea> |
| playsinline | <video> |
| popovertarget | <button>, <input> |
| popovertargetaction | <button>, <input> |

| Attribute | Used on |
|---------------------|--|
| poster | <video> |
| preload | <audio>, <video> |
| preserveAspectRatio | <svg> |
| profile | <head> |
| prompt | <isindex> |
| property | <meta> |
| radiogroup | <menuitem> |
| readonly | <input>, <textarea> |
| referrerpolicy | <a>, <area>, <iframe>, , <link>, <portal>, <script> |
| rel | <a>, <area>, <form>, <link> |
| required | <input>, <select>, <textarea> |
| results | <input> |
| rev | <a>, <link> |
| reversed | |
| rightmargin | <body> |
| role | <svg> |
| rows | <frameset>, <textarea> |
| rowspan | <td>, <th> |
| rules | <table> |
| sandbox | <iframe> |
| scheme | <meta> |
| scope | <td>, <th> |
| scoped | <style> |
| scrollamount | <marquee> |
| scrolldelay | <marquee> |

| Attribute | Used on |
|------------------------------|---|
| scrolling | <frame>, <iframe> |
| select | <content> |
| selected | <option> |
| shadowrootclonable | <template> |
| shadowrootdelegates
focus | <template> |
| shadowrootmode | <template> |
| shadowrootserializable | <template> |
| shape | <a>, <area> |
| size | <basefont>, , <hr>, <input>, <select>, <spacer> |
| sizes | , <link>, <source> |
| span | <col>, <colgroup> |
| spellcheck | <textarea> |
| src | <audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video> |
| srcdoc | <iframe> |
| srclang | <track> |
| srcset | , <source> |
| standby | <object> |
| start | |
| step | <input> |
| stroke | <svg> |
| stroke-width | <svg> |
| summary | <table> |

| Attribute | Used on |
|-----------------|--|
| target | <a>, <area>, <base>, <form>, <link> |
| text | <body> |
| title | <abbr>, <acronym>, <dfn>, <iframe>, <link>, <style> |
| topmargin | <body> |
| truespeed | <marquee> |
| type | <a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, |
| typemustmatch | <object> |
| usemap | , <input>, <object> |
| valign | <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr> |
| value | <button>, <data>, <input>, , <meter>, <option>, <param>, <progress> |
| valuetype | <param> |
| version | <html> |
| viewBox | <svg> |
| vlink | <body> |
| volume | <bgsound> |
| vspace | <applet>, , <marquee>, <object> |
| webkitdirectory | <input> |
| width | <applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video> |
| wrap | <pre>, <textarea> |

| Attribute | Used on |
|-----------|-----------------------|
| xmlns | <html>, <math>, <svg> |

Attribute: abbr

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|---------|--------|-----------|
| <td> | text | HTML 4.01 |
| <th> | text | HTML 4.01 |

abbr on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

Obsolete on td (valid on th). Abbreviated description.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td abbr="value">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

abbr on <th>

Element: <th> (Table header cell) | Since: HTML 4.01

Description

A short alternative label that screen readers may use when reading the header repeatedly.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th abbr="value">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: accept

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|---------|-------------------------|-----------|
| <form> | MIME types | HTML 4.01 |
| <input> | MIME types / extensions | HTML 4.01 |

accept on <form>

Element: <form> (Form) | Since: HTML 4.01

Description

Obsolete. Use accept on <input type="file"> instead.

Accepted values

MIME types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form accept="text/plain">...</form>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

accept on <input>

Element: <input> (Input control) | Since: HTML 4.01

Description

Allowed file types for type=file, for example image/*, .pdf, audio/*.

Accepted values

MIME types / extensions

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input accept="text/plain">
```

The same attribute on other elements

<form>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: accept-charset

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|---------------------------|-----------|
| <form> | space-separated encodings | HTML 4.01 |

accept-charset on <form>

Element: <form> (Form) | Since: HTML 4.01

Description

The character encodings the server accepts. Use utf-8.

Accepted values

space-separated encodings

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form accept-charset="value">...</form>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

Attribute: action

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|-----------|--------|----------|
| <form> | URL | HTML 2.0 |
| <isindex> | URL | Netscape |

action on <form>

Element: <form> (Form) | Since: HTML 2.0

Description

Where to send the data. If omitted, the data is sent to the current page URL.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form action="https://example.com/page.html">...</form>
```

The same attribute on other elements

<isindex>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

action on <isindex>

Element: <isindex> (Search index prompt (obsolete)) | Since: Netscape

Description

Where to send the query (non-standard).

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<isindex action="https://example.com/page.html">
```

The same attribute on other elements

<form>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <isindex>. Full details of the element are in Part 6.

Attribute: align

Accepted by 21 elements. Each use is described on the following pages.

| Element | Values | Since |
|------------|--|-----------------------------------|
| <applet> | left right top middle bottom | HTML 3.2 |
| <caption> | top bottom left right | HTML 3.2 |
| <col> | left center right justify char | HTML 4.01 |
| <colgroup> | various | HTML 4.01 |
| <div> | left center right justify | HTML 3.2 |
| <h1> | left center right justify | HTML 3.2 |
| <hr> | left center right | HTML 3.2 |
| <iframe> | left right top middle bottom | HTML 4.01 |
| | top middle bottom left right | HTML 2.0 |
| <input> | left right top middle bottom | HTML 3.2 |
| <legend> | left center right top bottom | HTML 4.01 |
| <object> | various | HTML 4.01 /
HTML5
(removed) |
| <p> | left center right justify | HTML 3.2 |
| <spacer> | - | Netscape |
| <table> | left center right | HTML 3.2 |
| <tbody> | various | HTML 4.01 |
| <td> | left center right justify char | HTML 3.2 |
| <tfoot> | various | HTML 4.01 |
| <th> | various | HTML 3.2 /
4.01 |
| <thead> | various | HTML 4.01 |
| <tr> | left center right justify char | HTML 3.2 |

align on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Alignment relative to surrounding text.

Accepted values

left | right | top | middle | bottom

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet align="left">...</applet>
```

The same attribute on other elements

<caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

align on <caption>

Element: <caption> (Table caption) | Since: HTML 3.2

Description

Obsolete. Position of the caption. Use CSS caption-side and text-align.

Accepted values

top | bottom | left | right

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<caption align="top">...</caption>
```

The same attribute on other elements

<applet>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <caption>. Full details of the element are in Part 6.

align on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Horizontal alignment of the cells. Use CSS text-align.

Accepted values

left | center | right | justify | char

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col align="left">
```

The same attribute on other elements

<applet>, <caption>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <col>. Full details of the element are in Part 6.

align on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup align="various">...</colgroup>
```

The same attribute on other elements

<applet>, <caption>, <col>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

align on <div>

Element: <div> (Generic block container) | Since: HTML 3.2

Description

Obsolete. Horizontal alignment of the content. Use CSS `text-align` or `margin`.

Accepted values

left | center | right | justify

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<div align="left">...</div>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <div>. Full details of the element are in Part 6.

align on <h1>

Element: <h1> (Headings h1 to h6) | Since: HTML 3.2

Description

Obsolete. Use CSS text-align.

Accepted values

left | center | right | justify

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<h1 align="left">...</h1>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <h1>. Full details of the element are in Part 6.

align on <hr>

Element: <hr> (Thematic break (horizontal rule)) | Since: HTML 3.2

Description

Obsolete. Alignment of the rule. Use CSS margin.

Accepted values

left | center | right

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<hr align="left">
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <hr>. Full details of the element are in Part 6.

align on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. Use CSS.

Accepted values

left | right | top | middle | bottom

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe align="left">...</iframe>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

align on

Element: (Image) | Since: HTML 2.0

Description

Obsolete. Use CSS float or vertical-align.

Accepted values

top | middle | bottom | left | right

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img align="top">
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

align on <input>

Element: <input> (Input control) | Since: HTML 3.2

Description

Obsolete. Use CSS.

Accepted values

left | right | top | middle | bottom

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input align="left">
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

align on <legend>

Element: <legend> (Fieldset caption) | Since: HTML 4.01

Description

Obsolete. Use CSS.

Accepted values

left | center | right | top | bottom

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<legend align="left">...</legend>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <legend>. Full details of the element are in Part 6.

align on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object align="various">...</object>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

align on <p>

Element: <p> (Paragraph) | Since: HTML 3.2

Description

Obsolete. Use CSS text-align.

Accepted values

left | center | right | justify

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<p align="left">...</p>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <p>. Full details of the element are in Part 6.

align on <spacer>

Element: <spacer> (Spacer (non-standard, obsolete)) | Since: Netscape

Description

For type=block.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<spacer align="-">
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <spacer>. Full details of the element are in Part 6.

align on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Alignment of the table on the page. Use CSS margin.

Accepted values

left | center | right

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table align="left">...</table>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

align on <tbody>

Element: <tbody> (Table body) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tbody align="various">...</tbody>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tbody>. Full details of the element are in Part 6.

align on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS text-align.

Accepted values

left | center | right | justify | char

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td align="left">...</td>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

align on <tfoot>

Element: <tfoot> (Table footer) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tfoot align="various">...</tfoot>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tfoot>. Full details of the element are in Part 6.

align on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th align="various">...</th>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

align on <thead>

Element: <thead> (Table header) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<thead align="various">...</thead>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <thead>. Full details of the element are in Part 6.

align on <tr>

Element: <tr> (Table row) | Since: HTML 3.2

Description

Obsolete. Use CSS text-align on cells.

Accepted values

left | center | right | justify | char

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tr align="left">...</tr>
```

The same attribute on other elements

<applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tr>. Full details of the element are in Part 6.

Attribute: **alink**

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|--------|----------|
| <body> | colour | HTML 3.2 |

alink on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Colour of active (being clicked) links. Use CSS `a:active`.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body alink="#336699">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: allow

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|--------------------|--------------------|--------------|
| <fencedframe>
> | permissions policy | Experimental |
| <iframe> | permissions policy | HTML5 |

allow on <fencedframe>

Element: <fencedframe> (Fenced frame (experimental)) | Since: Experimental

Description

Features allowed inside the frame.

Accepted values

permissions policy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<fencedframe allow="permissions policy">...</fencedframe>
```

The same attribute on other elements

<iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fencedframe>. Full details of the element are in Part 6.

allow on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5

Description

Features the frame may use, for example allow="camera; microphone; fullscreen; geolocation; autoplay; picture-in-picture".

Accepted values

permissions policy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe allow="permissions policy">...</iframe>
```

The same attribute on other elements

<fencedframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: allowfullscreen

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|----------|---------|-------|
| <iframe> | boolean | HTML5 |

allowfullscreen on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5

Description

Legacy form of allow="fullscreen".

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="allowfullscreen" in XHTML style). Never write allowfullscreen="false"; that still counts as present.

```
<iframe allowfullscreen>...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: allowpaymentrequest

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|----------|---------|--------------------|
| <iframe> | boolean | HTML5
(removed) |

allowpaymentrequest on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5 (removed)

Description

Obsolete. Use `allow="payment"`.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="allowpaymentrequest"` in XHTML style). Never write `allowpaymentrequest="false"`; that still counts as present.

```
<iframe allowpaymentrequest>...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: alpha

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|---------|--------------|
| <input> | boolean | Experimental |

alpha on <input>

Element: <input> (Input control) | Since: Experimental

Description

Experimental: allow transparency in type=color.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="alpha" in XHTML style). Never write alpha="false"; that still counts as present.

```
<input alpha>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: alt

Accepted by 5 elements. Each use is described on the following pages.

| Element | Values | Since |
|----------|----------|----------|
| <applet> | text | HTML 3.2 |
| <area> | text | HTML 3.2 |
| <image> | as | - |
| | text | HTML 2.0 |
| <input> | text | HTML 2.0 |

alt on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Alternative text for browsers without Java.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet alt="value">...</applet>
```

The same attribute on other elements

<area>, <image>, , <input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full

details of the element are in Part 6.

alt on <area>

Element: <area> (Image map area) | Since: HTML 3.2

Description

Alternative text for the area. Required when href is present so that the link has an accessible name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area alt="value">
```

The same attribute on other elements

<applet>, <image>, , <input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

alt on <image>

Element: <image> (Image (non-standard alias, obsolete)) | Since: -

Description

Whatever the parser transfers to the it creates.

Accepted values

as

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<image alt="as <img>">
```

The same attribute on other elements

<applet>, <area>, , <input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <image>. Full details of the element are in Part 6.

alt on

Element: (Image) | Since: HTML 2.0

Description

Text alternative. Describe the content or function of the image. Use `alt=""` (empty) for purely decorative images so that screen readers skip them. Omitting alt entirely is an error.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img alt="value">
```

The same attribute on other elements

<applet>, <area>, <image>, <input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

alt on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

Alternative text for type=image.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input alt="value">
```

The same attribute on other elements

<applet>, <area>, <image>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: archive

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|----------|----------------------|-----------------------------------|
| <applet> | comma-separated URLs | HTML 4.01 |
| <object> | various | HTML 4.01 /
HTML5
(removed) |

archive on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 4.01

Description

JAR archives to preload.

Accepted values

comma-separated URLs

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet archive="comma-separated URLs">...</applet>
```

The same attribute on other elements

<object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

archive on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object archive="various">...</object>
```

The same attribute on other elements

<applet>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: aria-label

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|--------|-------|
| <svg> | - | ARIA |

aria-label on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: ARIA

Description

Accessibility name and role for the graphic.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg aria-label="-">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: aria-labelledby

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|--------|-------|
| <svg> | - | ARIA |

aria-labelledby on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: ARIA

Description

Accessibility name and role for the graphic.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg aria-labelledby="-">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: as

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|--|-----------------|
| <link> | audio document embed fetch font image object script style track video worker | Living Standard |

as on <link>

Element: <link> (External resource link) | Since: Living Standard

Description

With rel=preload or modulepreload: the type of content being loaded, so the browser can prioritise it correctly.

Accepted values

audio | document | embed | fetch | font | image | object | script | style | track | video | worker

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link as="audio">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: async

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|----------|---------|-------|
| <script> | boolean | HTML5 |

async on <script>

Element: <script> (Script) | Since: HTML5

Description

Download in parallel, execute as soon as ready, possibly out of order. For independent scripts such as analytics.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="async"` in XHTML style). Never write `async="false"`; that still counts as present.

```
<script async>...</script>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: attributionsrc

Accepted by 3 elements. Each use is described on the following pages.

| Element | Values | Since |
|----------|--------------|-----------------|
| <a> | empty or URL | Living Standard |
| | empty or URL | Experimental |
| <script> | empty or URL | Experimental |

attributionsrc on <a>

Element: <a> (Anchor (hyperlink)) | Since: Living Standard

Description

Experimental: registers an attribution source for the Attribution Reporting API.

Accepted values

empty or URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a attributionsrc="empty">...</a>
```

The same attribute on other elements

, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

attributionsrc on

Element: (Image) | Since: Experimental

Description

Experimental attribution reporting.

Accepted values

empty or URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```

```

The same attribute on other elements

<a>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

attributionsrc on <script>

Element: <script> (Script) | Since: Experimental

Description

Experimental attribution reporting.

Accepted values

empty or URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script attributionsrc="empty">...</script>
```

The same attribute on other elements

<a>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: autocomplete

Accepted by 4 elements. Each use is described on the following pages.

| Element | Values | Since |
|------------|-----------------------|-------|
| <form> | on off | HTML5 |
| <input> | on off token list | HTML5 |
| <select> | token | HTML5 |
| <textarea> | on off token | HTML5 |

autocomplete on <form>

Element: <form> (Form) | Since: HTML5

Description

Whether the browser may autofill controls in this form.
Default on.

Accepted values

on | off

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form autocomplete="on">...</form>
```

The same attribute on other elements

<input>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

autocomplete on <input>

Element: <input> (Input control) | Since: HTML5

Description

Autofill hint: name, email, tel, street-address, postal-code, cc-number, new-password, current-password, one-time-code and more (see Part 9).

Accepted values

on | off | token list

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input autocomplete="on">
```

The same attribute on other elements

<form>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

autocomplete on <select>

Element: <select> (Drop-down / list box) | Since: HTML5

Description

Autofill hint, for example country, cc-exp-month.

Accepted values

token

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<select autocomplete="token">...</select>
```

The same attribute on other elements

<form>, <input>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <select>. Full details of the element are in Part 6.

autocomplete on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Autofill hint (street-address, etc.).

Accepted values

on | off | token

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea autocomplete="on">...</textarea>
```

The same attribute on other elements

<form>, <input>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: autofocus

Accepted by 5 elements. Each use is described on the following pages.

Element	Values	Since
<button>	boolean	HTML5
<input>	boolean	HTML5
<keygen>	-	HTML5
<select>	boolean	HTML5
<textarea>	boolean	HTML5

autofocus on <button>

Element: <button> (Button) | Since: HTML5

Description

Focus this button when the page loads (global in HTML5).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autofocus"` in XHTML style). Never write `autofocus="false"`; that still counts as present.

```
<button autofocus>...</button>
```

The same attribute on other elements

<input>, <keygen>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<button>`. Full details of the element are in Part 6.

autofocus on <input>

Element: <input> (Input control) | Since: HTML5

Description

Focus this control on page load. Only one per page.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autofocus"` in XHTML style). Never write `autofocus="false"`; that still counts as present.

```
<input autofocus>
```

The same attribute on other elements

`<button>`, `<keygen>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<input>`. Full details of the element are in Part 6.

autofocus on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

As on other controls.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen autofocus="-">
```

The same attribute on other elements

<button>, <input>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

autofocus on <select>

Element: <select> (Drop-down / list box) | Since: HTML5

Description

Focus on load.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autofocus"` in XHTML style). Never write `autofocus="false"`; that still counts as present.

```
<select autofocus>...</select>
```

The same attribute on other elements

`<button>`, `<input>`, `<keygen>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<select>`. Full details of the element are in Part 6.

autofocus on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Focus on load.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autofocus"` in XHTML style). Never write `autofocus="false"`; that still counts as present.

```
<textarea autofocus>...</textarea>
```

The same attribute on other elements

<button>, <input>, <keygen>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: autoplay

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	boolean	HTML5
<video>	boolean	HTML5

autoplay on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

Start playing as soon as enough data has loaded. Usually blocked unless muted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autoplay"` in XHTML style). Never write `autoplay="false"`; that still counts as present.

```
<audio autoplay>...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

autoplay on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Start automatically (only reliable with muted).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="autoplay"` in XHTML style). Never write `autoplay="false"`; that still counts as present.

```
<video autoplay>...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: axis

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<td>	text	HTML 4.01
<th>	various	HTML 3.2 / 4.01

axis on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

Obsolete. Category names.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td axis="value">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

axis on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th axis="various">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: background

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	URL	HTML 3.2

background on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Background image. Use CSS background-image.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body background="https://example.com/page.html">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: balance

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<bgsound>	-10000 to 10000	IE

balance on <bgsound>

Element: <bgsound> (Background sound (non-standard, obsolete)) | Since: IE

Description

Stereo balance.

Accepted values

-10000 to 10000

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bgsound balance="-10000 to 10000">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bgsound>. Full details of the element are in Part 6.

Attribute: behavior

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<marquee>	scroll slide alternate	IE

behavior on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Scrolling style.

Accepted values

scroll | slide | alternate

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee behavior="scroll">...</marquee>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

Attribute: bgcolor

Accepted by 11 elements. Each use is described on the following pages.

Element	Values	Since
<body>	colour	HTML 3.2
<col>	colour	HTML 4.01
<colgroup>	various	HTML 4.01
<marquee>	colour	IE
<table>	colour	HTML 3.2
<tbody>	various	HTML 4.01
<td>	colour	HTML 3.2
<tfoot>	various	HTML 4.01
<th>	various	HTML 3.2 / 4.01
<thead>	various	HTML 4.01
<tr>	colour	HTML 3.2

bgcolor on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Background colour. Use CSS background-color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body bgcolor="#336699">...</body>
```

The same attribute on other elements

`<col>`, `<colgroup>`, `<marquee>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<body>`. Full details of the element are in Part 6.

bgcolor on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Background colour. Use CSS.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col bgcolor="#336699">
```

The same attribute on other elements

<body>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <col>. Full details of the element are in Part 6.

bgcolor on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup bgcolor="various">...</colgroup>
```

The same attribute on other elements

<body>, <col>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

bgcolor on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Background.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee bgcolor="#336699">...</marquee>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

bgcolor on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Background colour. Use CSS background-color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table bgcolor="#336699">...</table>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

bgcolor on <tbody>

Element: <tbody> (Table body) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tbody bgcolor="various">...</tbody>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tbody>. Full details of the element are in Part 6.

bgcolor on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS background-color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td bgcolor="#336699">...</td>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

bgcolor on <tfoot>

Element: <tfoot> (Table footer) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tfoot bgcolor="various">...</tfoot>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tfoot>. Full details of the element are in Part 6.

bgcolor on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th bgcolor="various">...</th>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

bgcolor on <thead>

Element: <thead> (Table header) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<thead bgcolor="various">...</thead>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <thead>. Full details of the element are in Part 6.

bgcolor on <tr>

Element: <tr> (Table row) | Since: HTML 3.2

Description

Obsolete. Use CSS background-color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tr bgcolor="#336699">...</tr>
```

The same attribute on other elements

<body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tr>. Full details of the element are in Part 6.

Attribute: blocking

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<link>	render	Living Standard
<script>	render	Living Standard
<style>	render	Living Standard

blocking on <link>

Element: <link> (External resource link) | Since: Living Standard

Description

Block rendering until the resource is loaded (for stylesheets/preloads/expect).

Accepted values

render

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link blocking="render">
```

The same attribute on other elements

<script>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<link>`. Full details of the element are in Part 6.

blocking on <script>

Element: <script> (Script) | Since: Living Standard

Description

Block rendering until the script runs.

Accepted values

render

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script blocking="render">...</script>
```

The same attribute on other elements

<link>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

blocking on <style>

Element: <style> (Embedded style sheet) | Since: Living Standard

Description

Block rendering until the style sheet is applied (it does anyway for inline styles).

Accepted values

render

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<style blocking="render">...</style>
```

The same attribute on other elements

<link>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

Attribute: border

Accepted by 4 elements. Each use is described on the following pages.

| Element | Values | Since |
|------------|----------------|-----------------------------|
| <frameset> | pixels / 0 1 | HTML 4.01 / non-standard |
| | pixels | HTML 3.2 |
| <object> | various | HTML 4.01 / HTML5 (removed) |
| <table> | pixels | HTML 3.2 |

border on <frameset>

Element: <frameset> (Frameset (obsolete)) | Since: HTML 4.01 / non-standard

Description

Border appearance.

Accepted values

pixels / 0 | 1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frameset border="3">...</frameset>
```

The same attribute on other elements

, <object>, <table>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<frameset>`. Full details of the element are in Part 6.

border on

Element: (Image) | Since: HTML 3.2

Description

Obsolete. Border width. Use CSS border.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img border="3">
```

The same attribute on other elements

<frameset>, <object>, <table>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

border on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object border="various">...</object>
```

The same attribute on other elements

<frameset>, , <table>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

border on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Border width around the table and cells. Use CSS border.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table border="3">...</table>
```

The same attribute on other elements

<frameset>, , <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: **bottommargin**

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	pixels	IE/Netscape

bottommargin on <body>

Element: <body> (Document body) | Since: IE/Netscape

Description

Obsolete, non-standard. Page margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body bottommargin="3">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: capture

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	user environment	Media Capture spec

capture on <input>

Element: <input> (Input control) | Since: Media Capture spec

Description

For type=file on phones: use the front (user) or back (environment) camera/microphone directly.

Accepted values

user | environment

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input capture="user">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: cellpadding

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<table>	pixels	HTML 3.2

cellpadding on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Space between cell content and cell border. Use CSS padding on td/th.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table cellpadding="3">...</table>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: cellpadding

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<table>	pixels	HTML 3.2

cellpadding on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Space between cells. Use CSS border-spacing or border-collapse.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table cellpadding="3">...</table>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: challenge

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<keygen>	text	HTML5

challenge on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

Challenge string.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen challenge="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

Attribute: char

Accepted by 8 elements. Each use is described on the following pages.

Element	Values	Since
<col>	character	HTML 4.01
<colgroup>	various	HTML 4.01
<tbody>	various	HTML 4.01
<td>	character / number	HTML 4.01
<tfoot>	various	HTML 4.01
<th>	various	HTML 3.2 / 4.01
<thead>	various	HTML 4.01
<tr>	character / number	HTML 4.01

char on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Alignment character (for align=char).

Accepted values

character

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col char="character">
```

The same attribute on other elements

<colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because

accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<col>`. Full details of the element are in Part 6.

char on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup char="various">...</colgroup>
```

The same attribute on other elements

<col>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

char on <tbody>

Element: <tbody> (Table body) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tbody char="various">...</tbody>
```

The same attribute on other elements

<col>, <colgroup>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tbody>. Full details of the element are in Part 6.

char on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

Obsolete. Character alignment.

Accepted values

character / number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td char="3">...</td>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

char on <tfoot>

Element: <tfoot> (Table footer) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tfoot char="various">...</tfoot>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tfoot>. Full details of the element are in Part 6.

char on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th char="various">...</th>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

char on <thead>

Element: <thead> (Table header) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<thead char="various">...</thead>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <thead>. Full details of the element are in Part 6.

char on <tr>

Element: <tr> (Table row) | Since: HTML 4.01

Description

Obsolete.

Accepted values

character / number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tr char="3">...</tr>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tr>. Full details of the element are in Part 6.

Attribute: charoff

Accepted by 8 elements. Each use is described on the following pages.

Element	Values	Since
<col>	number	HTML 4.01
<colgroup>	various	HTML 4.01
<tbody>	various	HTML 4.01
<td>	character / number	HTML 4.01
<tfoot>	various	HTML 4.01
<th>	various	HTML 3.2 / 4.01
<thead>	various	HTML 4.01
<tr>	character / number	HTML 4.01

charoff on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Offset from the alignment character.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col charoff="3">
```

The same attribute on other elements

<colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because

accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<col>`. Full details of the element are in Part 6.

charoff on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup charoff="various">...</colgroup>
```

The same attribute on other elements

<col>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

charoff on <tbody>

Element: <tbody> (Table body) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tbody charoff="various">...</tbody>
```

The same attribute on other elements

<col>, <colgroup>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tbody>. Full details of the element are in Part 6.

charoff on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

Obsolete. Character alignment.

Accepted values

character / number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td charoff="3">...</td>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

charoff on <tfoot>

Element: <tfoot> (Table footer) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tfoot charoff="various">...</tfoot>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tfoot>. Full details of the element are in Part 6.

charoff on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th charoff="various">...</th>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

charoff on <thead>

Element: <thead> (Table header) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<thead charoff="various">...</thead>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <thead>. Full details of the element are in Part 6.

charoff on <tr>

Element: <tr> (Table row) | Since: HTML 4.01

Description

Obsolete.

Accepted values

character / number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tr charoff="3">...</tr>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tr>. Full details of the element are in Part 6.

Attribute: charset

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<a>	character encoding	HTML 4.01 (obsolete in HTML5)
<link>	encoding	HTML 4.01
<meta>	utf-8	HTML5
<script>	encoding	HTML 4.01

charset on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01 (obsolete in HTML5)

Description

Obsolete. The character encoding of the linked resource. Use the HTTP Content-Type header instead.

Accepted values

character encoding

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a charset="character encoding">...</a>
```

The same attribute on other elements

<link>, <meta>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<a>`. Full details of the element are in Part 6.

charset on <link>

Element: <link> (External resource link) | Since: HTML 4.01

Description

Obsolete. Character set of the resource.

Accepted values

encoding

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link charset="encoding">
```

The same attribute on other elements

<a>, <meta>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

charset on <meta>

Element: <meta> (Metadata) | Since: HTML5

Description

Declares the document's character encoding. The only valid value in HTML5 is utf-8 (case-insensitive). Must be in the first 1024 bytes.

Accepted values

utf-8

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta charset="utf-8">
```

The same attribute on other elements

<a>, <link>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

charset on <script>

Element: <script> (Script) | Since: HTML 4.01

Description

Obsolete. Encoding of the script; use UTF-8 for everything.

Accepted values

encoding

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script charset="encoding">...</script>
```

The same attribute on other elements

<a>, <link>, <meta>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: checked

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	boolean	HTML 2.0
<menuitem>	various	HTML5 draft

checked on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

Initially selected (checkbox, radio).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="checked"` in XHTML style). Never write `checked="false"`; that still counts as present.

```
<input checked>
```

The same attribute on other elements

<menuitem>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

checked on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

State and grouping.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem checked="various">
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

Attribute: cite

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<blockquote>	URL	HTML 4.01
	URL	HTML 4.01
<ins>	URL	HTML 4.01
<q>	URL	HTML 4.01

cite on <blockquote>

Element: <blockquote> (Block quotation) | Since: HTML 4.01

Description

The URL of the source document or message. Not displayed by browsers; readable by scripts and tools.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<blockquote cite="https://example.com/page.html">...</blockquote>
```

The same attribute on other elements

, <ins>, <q>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <blockquote>. Full details of the element are in Part 6.

**cite on **

Element: (Deleted text) | Since: HTML 4.01

Description

A URL explaining the change, such as a link to a change log or meeting minutes.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<del cite="https://example.com/page.html">...</del>
```

The same attribute on other elements

<blockquote>, <ins>, <q>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

cite on <ins>

Element: <ins> (Inserted text) | Since: HTML 4.01

Description

A URL explaining the change.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<ins cite="https://example.com/page.html">...</ins>
```

The same attribute on other elements

<blockquote>, , <q>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <ins>. Full details of the element are in Part 6.

cite on <q>

Element: <q> (Inline quotation) | Since: HTML 4.01

Description

The source of the quotation. Not displayed.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<q cite="https://example.com/page.html">...</q>
```

The same attribute on other elements

<blockquote>, , <ins>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <q>. Full details of the element are in Part 6.

Attribute: classid

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	various	HTML 4.01 / HTML5 (removed)

classid on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object classid="various">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: clear

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
 	left right all none	HTML 3.2

clear on

Element:
 (Line break) | Since: HTML 3.2

Description

Obsolete. Moved the following text below any floated images. Use CSS clear.

Accepted values

left | right | all | none

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<br clear="left">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on
. Full details of the element are in Part 6.

Attribute: closedby

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<dialog>	any closerequest none	Living Standard

closedby on <dialog>

Element: <dialog> (Dialog box) | Since: Living Standard

Description

Which user actions close the dialog: any (Escape, backdrop click, close request), closerequest (Escape only, the default for modal), or none.

Accepted values

any | closerequest | none

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<dialog closedby="any">...</dialog>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <dialog>. Full details of the element are in Part 6.

Attribute: code

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<applet>	class file name	HTML 3.2

code on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

The name of the compiled applet class file.

Accepted values

class file name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet code="class file name">...</applet>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

Attribute: codebase

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<applet>	URL	HTML 3.2
<object>	various	HTML 4.01 / HTML5 (removed)

codebase on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

The base URL for the applet's class files.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet codebase="https://example.com/page.html">...</applet>
```

The same attribute on other elements

<object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

codebase on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object codebase="various">...</object>
```

The same attribute on other elements

<applet>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: codetype

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	various	HTML 4.01 / HTML5 (removed)

codetype on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object codetype="various">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: color

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<basefont>	colour	HTML 4.01
	colour name or #rrggbb	HTML 3.2
<hr>	colour	IE
<link>	colour	Non-standard

color on <basefont>

Element: <basefont> (Base font (obsolete)) | Since: HTML 4.01

Description

Default text colour.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<basefont color="#336699">
```

The same attribute on other elements

, <hr>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <basefont>. Full details of the element are in Part 6.

color on

Element: (Font (obsolete)) | Since: HTML 3.2

Description

Text colour.

Accepted values

colour name or #rrggbb

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<font color="#336699">...</font>
```

The same attribute on other elements

<basefont>, <hr>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

color on <hr>

Element: <hr> (Thematic break (horizontal rule)) | Since: IE

Description

Obsolete, non-standard. Colour of the rule. Use CSS border-color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<hr color="#336699">
```

The same attribute on other elements

<basefont>, , <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <hr>. Full details of the element are in Part 6.

color on <link>

Element: <link> (External resource link) | Since: Non-standard

Description

With rel=mask-icon (Safari pinned tab).

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link color="#336699">
```

The same attribute on other elements

<basefont>, , <hr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: colorspace

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	limited-srgb display-p3	Experimental

colorspace on <input>

Element: <input> (Input control) | Since: Experimental

Description

Experimental: colour space for type=color.

Accepted values

limited-srgb | display-p3

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input colorspace="limited-srgb">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: cols

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<frameset>	comma-separated sizes	HTML 4.01
<multicol>	integer	Netscape
<textarea>	positive integer	HTML 2.0

cols on <frameset>

Element: <frameset> (Frameset (obsolete)) | Since: HTML 4.01

Description

Column widths in pixels, percentages or * (remaining space).

Accepted values

comma-separated sizes

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frameset cols="comma-separated sizes">...</frameset>
```

The same attribute on other elements

<multicol>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frameset>. Full details of the element are in Part 6.

cols on <multicol>

Element: <multicol> (Multi-column text (non-standard, obsolete)) | Since: Netscape

Description

Number of columns.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<multicol cols="3">...</multicol>
```

The same attribute on other elements

<frameset>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <multicol>. Full details of the element are in Part 6.

cols on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML 2.0

Description

Visible width in average characters. Default 20.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea cols="3">...</textarea>
```

The same attribute on other elements

<frameset>, <multicol>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: colspan

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<td>	positive integer	HTML 3.2
<th>	positive integer	HTML 3.2

colspan on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Number of columns the cell spans. Default 1, maximum 1000.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td colspan="3">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

colspan on <th>

Element: <th> (Table header cell) | Since: HTML 3.2

Description

Columns spanned.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th colspan="3">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: command

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	show-modal close request-close show-popover hide-popover toggle-popover custom	Living Standard
<menuitem>	various	HTML5 draft

command on <button>

Element: <button> (Button) | Since: Living Standard

Description

Experimental: declarative action to perform on commandfor's target.

Accepted values

show-modal | close | request-close | show-popover | hide-popover | toggle-popover | custom

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button command="show-modal">...</button>
```

The same attribute on other elements

<menuitem>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full

details of the element are in Part 6.

command on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

State and grouping.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem command="various">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

Attribute: commandfor

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<button>	element id	Living Standard

commandfor on <button>

Element: <button> (Button) | Since: Living Standard

Description

Experimental: the element the command applies to.

Accepted values

element id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button commandfor="element id">...</button>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

Attribute: compact

Accepted by 5 elements. Each use is described on the following pages.

Element	Values	Since
<dir>	boolean	HTML 2.0
<dl>	boolean	HTML 2.0
<menu>	boolean	HTML 2.0
	boolean	HTML 2.0
	boolean	HTML 2.0

compact on <dir>

Element: <dir> (Directory list (obsolete)) | Since: HTML 2.0

Description

Obsolete. Render more compactly.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="compact"` in XHTML style). Never write `compact="false"`; that still counts as present.

```
<dir compact>...</dir>
```

The same attribute on other elements

<dl>, <menu>, , . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<dir>`. Full details of the element are in Part 6.

compact on <dl>

Element: <dl> (Description list) | Since: HTML 2.0

Description

Obsolete. Render more compactly.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="compact"` in XHTML style). Never write `compact="false"`; that still counts as present.

```
<dl compact>...</dl>
```

The same attribute on other elements

<dir>, <menu>, , . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <dl>. Full details of the element are in Part 6.

compact on <menu>

Element: <menu> (Menu (toolbar list)) | Since: HTML 2.0

Description

Obsolete. Compact rendering.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="compact"` in XHTML style). Never write `compact="false"`; that still counts as present.

```
<menu compact>...</menu>
```

The same attribute on other elements

`<dir>`, `<dl>`, `<ol>`, `<ul>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<menu>`. Full details of the element are in Part 6.

compact on

Element: (Ordered list) | Since: HTML 2.0

Description

Obsolete. Compact rendering.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="compact"` in XHTML style). Never write `compact="false"`; that still counts as present.

```
<ol compact>...</ol>
```

The same attribute on other elements

`<dir>`, `<dl>`, `<menu>`, `<ul>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<ol>`. Full details of the element are in Part 6.

compact on

Element: (Unordered list) | Since: HTML 2.0

Description

Obsolete. Compact rendering.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="compact"` in XHTML style). Never write `compact="false"`; that still counts as present.

```
<ul compact>...</ul>
```

The same attribute on other elements

`<dir>`, `<dl>`, `<menu>`, `<ol>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<ul>`. Full details of the element are in Part 6.

Attribute: content

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meta>	text	HTML 2.0

content on <meta>

Element: <meta> (Metadata) | Since: HTML 2.0

Description

The value for name or http-equiv. Required with them.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta content="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

Attribute: controls

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	boolean	HTML5
<video>	boolean	HTML5

controls on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

Show the browser's playback controls.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="controls"` in XHTML style). Never write `controls="false"`; that still counts as present.

```
<audio controls>...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

controls on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Show built-in controls.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="controls"` in XHTML style). Never write `controls="false"`; that still counts as present.

```
<video controls>...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: controlslist

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	nodownload nofullscreen noremoteplayback	Living Standard
<video>	nodownload nofullscreen noremoteplayback	Non-standard

controlslist on <audio>

Element: <audio> (Embedded audio) | Since: Living Standard

Description

Hides specific built-in controls (non-standard, Chromium).

Accepted values

nodownload | nofullscreen | noremoteplayback

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<audio controlslist="nodownload">...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

controlslist on <video>

Element: <video> (Embedded video) | Since: Non-standard

Description

Hide specific native controls (Chromium).

Accepted values

nodownload | nofullscreen | noremotep playback

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video controlslist="nodownload">...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: coords

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<code><a></code>	comma-separated numbers	HTML 4.01 (obsolete in HTML5)
<code><area></code>	comma-separated numbers	HTML 3.2

coords on `<a>`

Element: `<a>` (Anchor (hyperlink)) | Since: HTML 4.01 (obsolete in HTML5)

Description

Obsolete. Coordinates for use with `shape` in old image maps. Use `<area>` instead.

Accepted values

comma-separated numbers

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a coords="3">...</a>
```

The same attribute on other elements

`<area>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<a>`. Full details of the element are in Part 6.

coords on <area>

Element: <area> (Image map area) | Since: HTML 3.2

Description

The coordinates of the shape in CSS pixels measured from the top-left corner of the image. rect: x1,y1,x2,y2 (top-left and bottom-right). circle: x,y,radius. poly: x1,y1,x2,y2,... (any number of points).

Accepted values

comma-separated numbers

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area coords="3">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

Attribute: credentialless

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<iframe>	boolean	Experimental

credentialless on <iframe>

Element: <iframe> (Inline frame) | Since: Experimental

Description

Experimental: load the frame in a new, empty cookie/storage context.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="credentialless"` in XHTML style). Never write `credentialless="false"`; that still counts as present.

```
<iframe credentialless>...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: crossorigin

Accepted by 5 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	anonymous use-credentials	HTML5
	anonymous use-credentials	HTML5
<link>	anonymous use-credentials	HTML5
<script>	anonymous use-credentials	HTML5
<video>	anonymous use-credentials	HTML5

crossorigin on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

How to handle cross-origin requests, needed when the audio will be processed with the Web Audio API or canvas.

Accepted values

anonymous | use-credentials

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<audio crossorigin="anonymous">...</audio>
```

The same attribute on other elements

, <link>, <script>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<audio>`. Full details of the element are in Part 6.

crossorigin on

Element: (Image) | Since: HTML5

Description

Load with CORS so the image can be used in canvas without tainting it.

Accepted values

anonymous | use-credentials

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img crossorigin="anonymous">
```

The same attribute on other elements

<audio>, <link>, <script>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

crossorigin on <link>

Element: <link> (External resource link) | Since: HTML5

Description

CORS mode for the request. Required when preloading fonts.

Accepted values

anonymous | use-credentials

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link crossorigin="anonymous">
```

The same attribute on other elements

<audio>, , <script>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

crossorigin on <script>

Element: <script> (Script) | Since: HTML5

Description

CORS setting; needed to get full error details from cross-origin scripts.

Accepted values

anonymous | use-credentials

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script crossorigin="anonymous">...</script>
```

The same attribute on other elements

<audio>, , <link>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

crossorigin on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

CORS mode for the media (needed for canvas processing and cross-origin tracks).

Accepted values

anonymous | use-credentials

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video crossorigin="anonymous">...</video>
```

The same attribute on other elements

<audio>, , <link>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: csp

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<iframe>	policy string	Experimental

csp on <iframe>

Element: <iframe> (Inline frame) | Since: Experimental

Description

Experimental: Content Security Policy to enforce on the embedded document.

Accepted values

policy string

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe csp="policy string">...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: data

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	URL	HTML 4.01

data on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

The resource URL. Required (or classid historically).

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object data="https://example.com/page.html">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: datetime

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
	date or date-time string	HTML 4.01
<ins>	date or date-time	HTML 4.01
<time>	date/time string	HTML5

datetime on

Element: (Deleted text) | Since: HTML 4.01

Description

When the change was made, in the format YYYY-MM-DD or YYYY-MM-DDThh:mm:ssZ.

Accepted values

date or date-time string

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<del datetime="date">...</del>
```

The same attribute on other elements

<ins>, <time>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

datetime on <ins>

Element: <ins> (Inserted text) | Since: HTML 4.01

Description

When the change was made.

Accepted values

date or date-time

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<ins datetime="date">...</ins>
```

The same attribute on other elements

, <time>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <ins>. Full details of the element are in Part 6.

datetime on <time>

Element: <time> (Date / time) | Since: HTML5

Description

Machine-readable value. Formats: 2026-09-11 (date); 14:30 or 14:30:00 (time); 2026-09-11T14:30 (local date-time); 2026-09-11T14:30Z or 2026-09-11T14:30+02:00 (with time zone); 2026-09 (month); 2026-W37 (week); 2026 (year); 09-11 (yearless date); PT2H30M or 2h 30m (duration).

Accepted values

date/time string

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<time datetime="date">...</time>
```

The same attribute on other elements

, <ins>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <time>. Full details of the element are in Part 6.

Attribute: declare

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	various	HTML 4.01 / HTML5 (removed)

declare on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object declare="various">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: decoding

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
	sync async auto	Living Standard

decoding on

Element: (Image) | Since: Living Standard

Description

Hint about how to decode the image.

Accepted values

sync | async | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img decoding="sync">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: default

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<menuitem>	various	HTML5 draft
<track>	boolean	HTML5

default on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

State and grouping.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem default="various">
```

The same attribute on other elements

<track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

default on <track>

Element: <track> (Text track for media) | Since: HTML5

Description

Enable this track by default (one per kind).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="default"` in XHTML style). Never write `default="false"`; that still counts as present.

```
<track default>
```

The same attribute on other elements

<menuitem>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <track>. Full details of the element are in Part 6.

Attribute: defer

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<script>	boolean	HTML 4.01

defer on <script>

Element: <script> (Script) | Since: HTML 4.01

Description

Download in parallel, execute after the document has been parsed, in document order. Only for external classic scripts.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="defer"` in XHTML style). Never write `defer="false"`; that still counts as present.

```
<script defer>...</script>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: dir

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<bdi>	ltr rtl auto	HTML5
<bdo>	ltr rtl	HTML 4.01
<html>	ltr rtl auto	HTML 4.01

dir on <bdi>

Element: <bdi> (Bidirectional isolate) | Since: HTML5

Description

On <bdi> the default is auto, unlike other elements where dir is inherited.

Accepted values

ltr | rtl | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bdi dir="ltr">...</bdi>
```

The same attribute on other elements

<bdo>, <html>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bdi>. Full details of the element are in Part 6.

dir on <bdo>

Element: <bdo> (Bidirectional text override) | Since: HTML 4.01

Description

Required. The direction to force.

Accepted values

ltr | rtl

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bdo dir="ltr">...</bdo>
```

The same attribute on other elements

<bdi>, <html>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bdo>. Full details of the element are in Part 6.

dir on <html>

Element: <html> (Document root) | Since: HTML 4.01

Description

Text direction of the document.

Accepted values

ltr | rtl | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<html dir="ltr">...</html>
```

The same attribute on other elements

<bdi>, <bdo>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <html>. Full details of the element are in Part 6.

Attribute: direction

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<marquee>	left right up down	IE

direction on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Scroll direction.

Accepted values

left | right | up | down

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee direction="left">...</marquee>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

Attribute: dirname

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	text	HTML5
<textarea>	text	HTML5

dirname on <input>

Element: <input> (Input control) | Since: HTML5

Description

Submit the text direction of the value under this name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input dirname="value">
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

dirname on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Submit text direction.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea dirname="value">...</textarea>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: disabled

Accepted by 10 elements. Each use is described on the following pages.

| Element | Values | Since |
|------------|---------|-------------|
| <button> | boolean | HTML 4.01 |
| <fieldset> | boolean | HTML 4.01 |
| <input> | boolean | HTML 4.01 |
| <keygen> | - | HTML5 |
| <link> | boolean | HTML5 |
| <menuitem> | various | HTML5 draft |
| <optgroup> | boolean | HTML 4.01 |
| <option> | boolean | HTML 4.01 |
| <select> | boolean | HTML 4.01 |
| <textarea> | boolean | HTML 4.01 |

disabled on <button>

Element: <button> (Button) | Since: HTML 4.01

Description

The button cannot be clicked or focused and is not submitted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<button disabled>...</button>
```

The same attribute on other elements

`<fieldset>`, `<input>`, `<keygen>`, `<link>`, `<menuitem>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<button>`. Full details of the element are in Part 6.

disabled on <fieldset>

Element: <fieldset> (Form field group) | Since: HTML 4.01

Description

Disables all descendant form controls (except those inside the <legend>).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<fieldset disabled>...</fieldset>
```

The same attribute on other elements

<button>, <input>, <keygen>, <link>, <menuitem>, <optgroup>, <option>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fieldset>. Full details of the element are in Part 6.

disabled on <input>

Element: <input> (Input control) | Since: HTML 4.01

Description

Cannot be used or focused and is not submitted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<input disabled>
```

The same attribute on other elements

`<button>`, `<fieldset>`, `<keygen>`, `<link>`, `<menuitem>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<input>`. Full details of the element are in Part 6.

disabled on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

As on other controls.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen disabled="-">
```

The same attribute on other elements

<button>, <fieldset>, <input>, <link>, <menuitem>, <optgroup>, <option>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

disabled on <link>

Element: <link> (External resource link) | Since: HTML5

Description

For style sheets: do not apply.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<link disabled>
```

The same attribute on other elements

`<button>`, `<fieldset>`, `<input>`, `<keygen>`, `<menuitem>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<link>`. Full details of the element are in Part 6.

disabled on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

State and grouping.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem disabled="various">
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <link>, <optgroup>, <option>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

disabled on <optgroup>

Element: <optgroup> (Option group) | Since: HTML 4.01

Description

Disables all options in the group.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<optgroup disabled>...</optgroup>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <link>, <menuitem>, <option>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <optgroup>. Full details of the element are in Part 6.

disabled on <option>

Element: <option> (Option) | Since: HTML 4.01

Description

Cannot be selected.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<option disabled>...</option>
```

The same attribute on other elements

`<button>`, `<fieldset>`, `<input>`, `<keygen>`, `<link>`, `<menuitem>`, `<optgroup>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<option>`. Full details of the element are in Part 6.

disabled on <select>

Element: <select> (Drop-down / list box) | Since: HTML 4.01

Description

The control cannot be used.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<select disabled>...</select>
```

The same attribute on other elements

`<button>`, `<fieldset>`, `<input>`, `<keygen>`, `<link>`, `<menuitem>`, `<optgroup>`, `<option>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<select>`. Full details of the element are in Part 6.

disabled on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML 4.01

Description

Cannot be used or submitted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disabled"` in XHTML style). Never write `disabled="false"`; that still counts as present.

```
<textarea disabled>...</textarea>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <link>, <menuitem>, <optgroup>, <option>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: disablepictureinpicture

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<video>	boolean	Living Standard

disablepictureinpicture on <video>

Element: <video> (Embedded video) | Since: Living Standard

Description

Hide the picture-in-picture option.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disablepictureinpicture"` in XHTML style). Never write `disablepictureinpicture="false"`; that still counts as present.

```
<video disablepictureinpicture>...</video>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: disableremoteautoplay

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	boolean	Living Standard
<video>	boolean	Living Standard

disableremoteautoplay on <audio>

Element: <audio> (Embedded audio) | Since: Living Standard

Description

Disables casting to remote devices.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disableremoteautoplay"` in XHTML style). Never write `disableremoteautoplay="false"`; that still counts as present.

```
<audio disableremoteautoplay>...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<audio>`. Full details of the element are in Part 6.

disableremoteautoplay on <video>

Element: <video> (Embedded video) | Since: Living Standard

Description

Disable casting.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="disableremoteautoplay"` in XHTML style). Never write `disableremoteautoplay="false"`; that still counts as present.

```
<video disableremoteautoplay>...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: display

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<code><math></code>	block inline	MathML

display on `<math>`

Element: `<math>` (MathML root) | Since: MathML

Description

Whether the formula is a block on its own line or inline in text.

Accepted values

block | inline

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<math display="block">...</math>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<math>`. Full details of the element are in Part 6.

Attribute: download

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<a>	empty or filename	HTML5
<area>	empty or filename	HTML5

download on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML5

Description

Instructs the browser to download the resource instead of navigating to it. If a value is given it is used as the suggested file name. Only works for same-origin URLs and blob:/data: URLs.

Accepted values

empty or filename

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a download="empty">...</a>
```

The same attribute on other elements

<area>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

download on <area>

Element: <area> (Image map area) | Since: HTML5

Description

Download the resource instead of navigating.

Accepted values

empty or filename

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area download="empty">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

Attribute: elementtiming

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
	text	Living Standard

elementtiming on

Element: (Image) | Since: Living Standard

Description

Marks the image for the Element Timing performance API.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img elementtiming="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: enctype

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<form>	application/x-www-form-urlencoded multipart/form-data text/plain	HTML 2.0

enctype on <form>

Element: <form> (Form) | Since: HTML 2.0

Description

How the data is encoded for post. Use multipart/form-data when the form contains <input type="file">.

Accepted values

application/x-www-form-urlencoded | multipart/form-data | text/plain

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form enctype="application">...</form>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

Attribute: etc.

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<image>	as 	-

etc. on <image>

Element: <image> (Image (non-standard alias, obsolete)) | Since: -

Description

Whatever the parser transfers to the it creates.

Accepted values

as

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<image etc.="as <img>">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <image>. Full details of the element are in Part 6.

Attribute: event

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<script>	text	HTML 4.01

event on <script>

Element: <script> (Script) | Since: HTML 4.01

Description

Obsolete. Internet Explorer event binding.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script event="value">...</script>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: face

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<basefont>	comma-separated font names	HTML 4.01
	comma-separated font names	HTML 3.2

face on <basefont>

Element: <basefont> (Base font (obsolete)) | Since: HTML 4.01

Description

Default font family.

Accepted values

comma-separated font names

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<basefont face="value">
```

The same attribute on other elements

. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <basefont>. Full details of the element are in Part 6.

face on

Element: (Font (obsolete)) | Since: HTML 3.2

Description

Typeface, tried in order.

Accepted values

comma-separated font names

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<font face="value">...</font>
```

The same attribute on other elements

<basefont>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: fetchpriority

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
	high low auto	Living Standard
<link>	high low auto	Living Standard
<script>	high low auto	Living Standard

fetchpriority on

Element: (Image) | Since: Living Standard

Description

Hint about the download priority; set high on the hero image.

Accepted values

high | low | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img fetchpriority="high">
```

The same attribute on other elements

<link>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details

of the element are in Part 6.

fetchpriority on <link>

Element: <link> (External resource link) | Since: Living Standard

Description

Download priority hint.

Accepted values

high | low | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link fetchpriority="high">
```

The same attribute on other elements

, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

fetchpriority on <script>

Element: <script> (Script) | Since: Living Standard

Description

Download priority hint.

Accepted values

high | low | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script fetchpriority="high">...</script>
```

The same attribute on other elements

, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: fill

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	colour / length	SVG

fill on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Presentation attributes for shapes (can also be CSS).

Accepted values

colour / length

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg fill="#336699">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: for

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<label>	id of a labelable element	HTML 4.01
<output>	space-separated ids	HTML5
<script>	text	HTML 4.01

for on <label>

Element: <label> (Form control label) | Since: HTML 4.01

Description

The control this label describes.

Accepted values

id of a labelable element

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<label for="my-id">...</label>
```

The same attribute on other elements

<output>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <label>. Full details of the element are in Part 6.

for on <output>

Element: <output> (Calculation or user-action result) | Since: HTML5

Description

The elements that contributed to the calculation.

Accepted values

space-separated ids

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<output for="space-separated ids">...</output>
```

The same attribute on other elements

<label>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <output>. Full details of the element are in Part 6.

for on <script>

Element: <script> (Script) | Since: HTML 4.01

Description

Obsolete. Internet Explorer event binding.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script for="value">...</script>
```

The same attribute on other elements

<label>, <output>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: form

Accepted by 10 elements. Each use is described on the following pages.

Element	Values	Since
<button>	form id	HTML5
<fieldset>	form id	HTML5
<input>	form id	HTML5
<keygen>	-	HTML5
<label>	form id	HTML5 (removed)
<meter>	form id	HTML5
<object>	form id	HTML5
<output>	form id	HTML5
<select>	form id	HTML5
<textarea>	form id	HTML5

form on <button>

Element: <button> (Button) | Since: HTML5

Description

Associates the button with a form elsewhere in the document.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button form="form id">...</button>
```

The same attribute on other elements

`<fieldset>`, `<input>`, `<keygen>`, `<label>`, `<meter>`, `<object>`, `<output>`, `<select>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<button>`. Full details of the element are in Part 6.

form on <fieldset>

Element: <fieldset> (Form field group) | Since: HTML5

Description

Associates the fieldset with a form elsewhere in the document.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<fieldset form="form id">...</fieldset>
```

The same attribute on other elements

<button>, <input>, <keygen>, <label>, <meter>, <object>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fieldset>. Full details of the element are in Part 6.

form on <input>

Element: <input> (Input control) | Since: HTML5

Description

Associate with a form anywhere in the document.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input form="form id">
```

The same attribute on other elements

<button>, <fieldset>, <keygen>, <label>, <meter>, <object>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

form on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

As on other controls.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen form="-">
```

The same attribute on other elements

<button>, <fieldset>, <input>, <label>, <meter>, <object>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

form on <label>

Element: <label> (Form control label) | Since: HTML5 (removed)

Description

Obsolete in the Living Standard; was meant to associate the label with a form.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<label form="form id">...</label>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <meter>, <object>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <label>. Full details of the element are in Part 6.

form on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Associated form (for labelling purposes).

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter form="form id">...</meter>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <label>, <object>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

form on <object>

Element: <object> (External object) | Since: HTML5

Description

Associated form.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object form="form id">...</object>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <output>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

form on <output>

Element: <output> (Calculation or user-action result) | Since: HTML5

Description

Associated form.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<output form="form id">...</output>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <object>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <output>. Full details of the element are in Part 6.

form on <select>

Element: <select> (Drop-down / list box) | Since: HTML5

Description

Associated form.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<select form="form id">...</select>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <object>, <output>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <select>. Full details of the element are in Part 6.

form on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Associated form.

Accepted values

form id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea form="form id">...</textarea>
```

The same attribute on other elements

<button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <object>, <output>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: formation

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	URL	HTML5
<input>	URL	HTML5

formation on <button>

Element: <button> (Button) | Since: HTML5

Description

Overrides the form's action for this button.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button formation="https://example.com/page.html">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

formation on <input>

Element: <input> (Input control) | Since: HTML5

Description

Override the form action (submit, image).

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input formation="https://example.com/page.html">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: formenctype

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	application/x-www-form-urlencoded multipart/form-data text/plain	HTML5
<input>	encoding type	HTML5

formenctype on <button>

Element: <button> (Button) | Since: HTML5

Description

Overrides the form's enctype.

Accepted values

application/x-www-form-urlencoded | multipart/form-data | text/plain

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button formenctype="application">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

formenctype on <input>

Element: <input> (Input control) | Since: HTML5

Description

Override the form enctype (submit, image).

Accepted values

encoding type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input formenctype="encoding type">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: formmethod

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	get post dialog	HTML5
<input>	get post dialog	HTML5

formmethod on <button>

Element: <button> (Button) | Since: HTML5

Description

Overrides the form's method.

Accepted values

get | post | dialog

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button formmethod="get">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

formmethod on <input>

Element: <input> (Input control) | Since: HTML5

Description

Override the form method (submit, image).

Accepted values

get | post | dialog

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input formmethod="get">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: formnovalidate

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	boolean	HTML5
<input>	boolean	HTML5

formnovalidate on <button>

Element: <button> (Button) | Since: HTML5

Description

Skips validation when this button submits.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="formnovalidate"` in XHTML style). Never write `formnovalidate="false"`; that still counts as present.

```
<button formnovalidate>...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

formnovalidate on <input>

Element: <input> (Input control) | Since: HTML5

Description

Skip validation when submitting with this button.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="formnovalidate"` in XHTML style). Never write `formnovalidate="false"`; that still counts as present.

```
<input formnovalidate>
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: formtarget

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	_self _blank _parent _top name	HTML5
<input>	browsing context	HTML5

formtarget on <button>

Element: <button> (Button) | Since: HTML5

Description

Overrides the form's target.

Accepted values

_self | _blank | _parent | _top | name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button formtarget="_self">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

formtarget on <input>

Element: <input> (Input control) | Since: HTML5

Description

Override the form target (submit, image).

Accepted values

browsing context

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input formtarget="browsing context">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: frame

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<table>	void above below hside lhs rhs vside box border	HTML 4.01

frame on <table>

Element: <table> (Table) | Since: HTML 4.01

Description

Obsolete. Which outer borders to draw. Use CSS border.

Accepted values

void | above | below | hside | lhs | rhs | vside | box | border

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table frame="void">...</table>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: frameborder

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<frame>	0 1	HTML 4.01
<frameset>	pixels / 0 1	HTML 4.01 / non-standard
<iframe>	0 1	HTML 4.01

frameborder on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Whether to draw a border.

Accepted values

0 | 1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame frameborder="0">
```

The same attribute on other elements

<frameset>, <iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

frameborder on <frameset>

Element: <frameset> (Frameset (obsolete)) | Since: HTML 4.01 / non-standard

Description

Border appearance.

Accepted values

pixels / 0 | 1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frameset frameborder="3">...</frameset>
```

The same attribute on other elements

<frame>, <iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frameset>. Full details of the element are in Part 6.

frameborder on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. Use CSS border.

Accepted values

0 | 1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe frameborder="0">...</iframe>
```

The same attribute on other elements

<frame>, <frameset>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: framespacing

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<frameset>	pixels / 0 1	HTML 4.01 / non-standard

framespacing on <frameset>

Element: <frameset> (Frameset (obsolete)) | Since: HTML 4.01 / non-standard

Description

Border appearance.

Accepted values

pixels / 0 | 1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frameset framespacing="3">...</frameset>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frameset>. Full details of the element are in Part 6.

Attribute: gutter

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<multicol>	pixels	Netscape

gutter on <multicol>

Element: <multicol> (Multi-column text (non-standard, obsolete)) | Since: Netscape

Description

Space between columns.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<multicol gutter="3">...</multicol>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <multicol>. Full details of the element are in Part 6.

Attribute: headers

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<td>	space-separated ids	HTML 4.01
<th>	space-separated ids	HTML 4.01

headers on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

The ids of <th> cells that provide headers for this cell.

Accepted values

space-separated ids

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td headers="space-separated ids">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

headers on <th>

Element: <th> (Table header cell) | Since: HTML 4.01

Description

Other header cells that apply to this one (for multi-level headers).

Accepted values

space-separated ids

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th headers="space-separated ids">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: height

Accepted by 16 elements. Each use is described on the following pages.

Element	Values	Since
<applet>	pixels	HTML 3.2
<canvas>	positive integer	HTML5
<embed>	pixels	Netscape / HTML5
<fencedframe>	pixels	Experimental
<iframe>	pixels	HTML 4.01
	pixels	HTML 3.2
<input>	pixels	HTML5
<marquee>	size	IE
<object>	pixels	HTML 4.01
<source>	pixels	Living Standard
<spacer>	-	Netscape
<svg>	length	SVG
<table>	pixels	Netscape
<td>	pixels	HTML 3.2
<th>	various	HTML 3.2 / 4.01
<video>	pixels	HTML5

height on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Size of the applet display area.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet height="3">...</applet>
```

The same attribute on other elements

`<canvas>`, `<embed>`, `<fencedframe>`, `<iframe>`, `<img>`, `<input>`, `<marquee>`, `<object>`, `<source>`, `<spacer>`, `<svg>`, `<table>`, `<td>`, `<th>`, `<video>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<applet>`. Full details of the element are in Part 6.

height on <canvas>

Element: <canvas> (Graphics canvas) | Since: HTML5

Description

Height of the bitmap in CSS pixels. Default 150.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<canvas height="3">...</canvas>
```

The same attribute on other elements

<applet>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <canvas>. Full details of the element are in Part 6.

height on <embed>

Element: <embed> (External content) | Since: Netscape / HTML5

Description

Height of the display area.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<embed height="3">
```

The same attribute on other elements

<applet>, <canvas>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <embed>. Full details of the element are in Part 6.

height on <fencedframe>

Element: <fencedframe> (Fenced frame (experimental)) | Since: Experimental

Description

Size of the frame.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<fencedframe height="3">...</fencedframe>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fencedframe>. Full details of the element are in Part 6.

height on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Height of the frame (default 150).

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe height="3">...</iframe>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

height on

Element: (Image) | Since: HTML 3.2

Description

Intrinsic height.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img height="3">
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

height on <input>

Element: <input> (Input control) | Since: HTML5

Description

Size of the image for type=image.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input height="3">
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

height on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Size.

Accepted values

size

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee height="size">...</marquee>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

height on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

Display size.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object height="3">...</object>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

height on <source>

Element: <source> (Media or image source) | Since: Living Standard

Description

Intrinsic size of the image candidate (picture only).

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source height="3">
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

height on <spacer>

Element: <spacer> (Spacer (non-standard, obsolete)) | Since: Netscape

Description

For type=block.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<spacer height="-">
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <spacer>. Full details of the element are in Part 6.

height on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Displayed size.

Accepted values

length

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg height="length">...</svg>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

height on <table>

Element: <table> (Table) | Since: Netscape

Description

Obsolete, non-standard. Use CSS.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table height="3">...</table>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

height on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS height.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td height="3">...</td>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

height on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th height="various">...</th>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

height on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Display size; also prevents layout shift.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video height="3">...</video>
```

The same attribute on other elements

<applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: high

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meter>	number	HTML5

high on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Lower bound of the 'high' region.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter high="3">...</meter>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

Attribute: href

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<a>	URL	HTML 1
<area>	URL	HTML 3.2
<base>	URL	HTML 2.0
<link>	URL	HTML 2.0

href on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 1

Description

The address the link points to. It may be absolute (<https://example.com/page>), relative ([about.html](#), [../images/photo.jpg](#)), a fragment (<#section-2>), or a special scheme (<mailto:>, <tel:>, <sms:>, <javascript:>). Without href the element is not a link.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a href="https://example.com/page.html">...</a>
```

The same attribute on other elements

<area>, <base>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<a>`. Full details of the element are in Part 6.

href on <area>

Element: <area> (Image map area) | Since: HTML 3.2

Description

The destination of the link.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area href="https://example.com/page.html">
```

The same attribute on other elements

<a>, <base>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

href on <base>

Element: <base> (Document base URL) | Since: HTML 2.0

Description

The base URL. Relative URLs such as images/logo.png are resolved against it.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<base href="https://example.com/page.html">
```

The same attribute on other elements

<a>, <area>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <base>. Full details of the element are in Part 6.

href on <link>

Element: <link> (External resource link) | Since: HTML 2.0

Description

The resource URL.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link href="https://example.com/page.html">
```

The same attribute on other elements

<a>, <area>, <base>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: hreflang

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<a>	BCP 47 language tag	HTML 4.01
<link>	language tag	HTML 4.01

hreflang on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01

Description

The language of the linked document, for example en, ar, fr-CA. Purely advisory.

Accepted values

BCP 47 language tag

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a hreflang="en">...</a>
```

The same attribute on other elements

<link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

hreflang on <link>

Element: <link> (External resource link) | Since: HTML 4.01

Description

Language of the linked resource (with rel=alternate).

Accepted values

language tag

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link hreflang="en">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: hspace

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<applet>	pixels	HTML 3.2
	pixels	HTML 3.2
<marquee>	pixels	IE
<object>	various	HTML 4.01 / HTML5 (removed)

hspace on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Horizontal and vertical margins.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet hspace="3">...</applet>
```

The same attribute on other elements

, <marquee>, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full

details of the element are in Part 6.

hspace on

Element: (Image) | Since: HTML 3.2

Description

Obsolete. Horizontal and vertical margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img hspace="3">
```

The same attribute on other elements

<applet>, <marquee>, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

hspace on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Margins.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee hspace="3">...</marquee>
```

The same attribute on other elements

<applet>, , <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

hspace on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object hspace="various">...</object>
```

The same attribute on other elements

<applet>, , <marquee>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: http-equiv

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meta>	content-type default-style refresh content-security-policy x-ua-compatible	HTML 2.0

http-equiv on <meta>

Element: <meta> (Metadata) | Since: HTML 2.0

Description

Pragma directive that acts like an HTTP header.

Accepted values

content-type | default-style | refresh | content-security-policy | x-ua-compatible

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta http-equiv="content-type">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

Attribute: icon

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<menuitem>	URL	HTML5 draft

icon on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

Icon image.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem icon="https://example.com/page.html">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

Attribute: id

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	identifier	HTML 4.01

id on <input>

Element: <input> (Input control) | Since: HTML 4.01

Description

Links the control to a <label for="id">.

Accepted values

identifier

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input id="identifier">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: imagesizes

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<link>	srcset / sizes syntax	Living Standard

imagesizes on <link>

Element: <link> (External resource link) | Since: Living Standard

Description

With rel=preload as=image: responsive image candidates.

Accepted values

srcset / sizes syntax

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link imagesizes="srcset">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: imagesrcset

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<link>	srcset / sizes syntax	Living Standard

imagesrcset on <link>

Element: <link> (External resource link) | Since: Living Standard

Description

With rel=preload as=image: responsive image candidates.

Accepted values

srcset / sizes syntax

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link imagesrcset="srcset">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: incremental

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	various	Non-standard

incremental on <input>

Element: <input> (Input control) | Since: Non-standard

Description

Non-standard browser-specific attributes.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input incremental="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: inputmode

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	none text decimal numeric tel search email url	HTML5

inputmode on <input>

Element: <input> (Input control) | Since: HTML5

Description

Which virtual keyboard to show on touch devices (global attribute).

Accepted values

none | text | decimal | numeric | tel | search | email | url

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input inputmode="none">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: integrity

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<link>	hash	SRI spec
<script>	hash	SRI

integrity on <link>

Element: <link> (External resource link) | Since: SRI spec

Description

Subresource Integrity hash; the browser refuses the file if it does not match.

Accepted values

hash

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link integrity="hash">
```

The same attribute on other elements

<script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

integrity on <script>

Element: <script> (Script) | Since: SRI

Description

Subresource Integrity hash.

Accepted values

hash

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script integrity="hash">...</script>
```

The same attribute on other elements

<link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: ismap

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|---------|----------|
| | boolean | HTML 2.0 |

ismap on

Element: (Image) | Since: HTML 2.0

Description

Makes the image a server-side image map; the click coordinates are appended to the link URL. Only valid inside <a href>.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="ismap" in XHTML style). Never write ismap="false"; that still counts as present.

```
<img ismap>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: itemprop

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meta>	text	HTML5 microdata

itemprop on <meta>

Element: <meta> (Metadata) | Since: HTML5 microdata

Description

Microdata property name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta itemprop="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

Attribute: keytype

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<keygen>	rsa dsa ec	HTML5

keytype on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

Key algorithm.

Accepted values

rsa | dsa | ec

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen keytype="rsa">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

Attribute: kind

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<track>	subtitles captions descriptions chapters metadata	HTML5

kind on <track>

Element: <track> (Text track for media) | Since: HTML5

Description

The type of track. Default subtitles.

Accepted values

subtitles | captions | descriptions | chapters | metadata

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<track kind="subtitles">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <track>. Full details of the element are in Part 6.

Attribute: label

Accepted by 5 elements. Each use is described on the following pages.

Element	Values	Since
<menu>	text	HTML5 draft (removed)
<menuitem>	text	HTML5 draft
<optgroup>	text	HTML 4.01
<option>	text	HTML 4.01
<track>	text	HTML5

label on <menu>

Element: <menu> (Menu (toolbar list)) | Since: HTML5 draft (removed)

Description

Obsolete. Menu label (context menus).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menu label="value">...</menu>
```

The same attribute on other elements

<menuitem>, <optgroup>, <option>, <track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<menu>`. Full details of the element are in Part 6.

label on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

Item text.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem label="value">
```

The same attribute on other elements

<menu>, <optgroup>, <option>, <track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

label on <optgroup>

Element: <optgroup> (Option group) | Since: HTML 4.01

Description

Required. The group heading shown in the list.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<optgroup label="value">...</optgroup>
```

The same attribute on other elements

<menu>, <menuitem>, <option>, <track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <optgroup>. Full details of the element are in Part 6.

label on <option>

Element: <option> (Option) | Since: HTML 4.01

Description

Alternative label shown instead of the text content (used mainly in datalist).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<option label="value">...</option>
```

The same attribute on other elements

<menu>, <menuitem>, <optgroup>, <track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <option>. Full details of the element are in Part 6.

label on <track>

Element: <track> (Text track for media) | Since: HTML5

Description

Human-readable title shown in the track menu.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<track label="value">
```

The same attribute on other elements

<menu>, <menuitem>, <optgroup>, <option>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <track>. Full details of the element are in Part 6.

Attribute: lang

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<html>	BCP 47 language tag	HTML 4.01

lang on <html>

Element: <html> (Document root) | Since: HTML 4.01

Description

The primary language of the document: en, en-GB, ar, ar-EG, fr, de, zh-Hans...

Accepted values

BCP 47 language tag

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<html lang="en">...</html>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <html>. Full details of the element are in Part 6.

Attribute: language

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<script>	text	HTML 3.2

language on <script>

Element: <script> (Script) | Since: HTML 3.2

Description

Obsolete. Scripting language name. Use type or omit.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script language="value">...</script>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: leftmargin

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	pixels	IE/Netscape

leftmargin on <body>

Element: <body> (Document body) | Since: IE/Netscape

Description

Obsolete, non-standard. Page margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body leftmargin="3">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: link

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	colour	HTML 3.2

link on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Colour of unvisited links. Use CSS a:link.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body link="#336699">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: list

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	datalist id	HTML5

list on <input>

Element: <input> (Input control) | Since: HTML5

Description

Links to a <datalist> of suggestions.

Accepted values

datalist id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input list="datalist id">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: loading

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<iframe>	eager lazy	Living Standard
	eager lazy	Living Standard

loading on <iframe>

Element: <iframe> (Inline frame) | Since: Living Standard

Description

lazy defers loading until the frame is near the viewport.

Accepted values

eager | lazy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe loading="eager">...</iframe>
```

The same attribute on other elements

. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

loading on

Element: (Image) | Since: Living Standard

Description

lazy delays loading until the image is near the viewport. Do not lazy-load images visible at page load.

Accepted values

eager | lazy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img loading="eager">
```

The same attribute on other elements

<iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: longdesc

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<frame>	URL	HTML 4.01
<iframe>	URL	HTML 4.01
	URL	HTML 4.01

longdesc on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Link to a long description.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame longdesc="https://example.com/page.html">
```

The same attribute on other elements

<iframe>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

longdesc on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. [Link to a long description.](#)

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe longdesc="https://example.com/page.html">...</iframe>
```

The same attribute on other elements

<frame>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

longdesc on

Element: (Image) | Since: HTML 4.01

Description

Obsolete. Link to a long description. Use a normal link or aria-describedby.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img longdesc="https://example.com/page.html">
```

The same attribute on other elements

<frame>, <iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: loop

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	boolean	HTML5
<bgsound>	number infinite	IE
<marquee>	integer -1	IE
<video>	boolean	HTML5

loop on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

Restart from the beginning when the end is reached.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="loop"` in XHTML style). Never write `loop="false"`; that still counts as present.

```
<audio loop>...</audio>
```

The same attribute on other elements

<bgsound>, <marquee>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full

details of the element are in Part 6.

loop on <bgsound>

Element: <bgsound> (Background sound (non-standard, obsolete)) | Since: IE

Description

Repeat count.

Accepted values

number | infinite

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bgsound loop="3">
```

The same attribute on other elements

<audio>, <marquee>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bgsound>. Full details of the element are in Part 6.

loop on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Repeat count.

Accepted values

integer | -1

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee loop="3">...</marquee>
```

The same attribute on other elements

<audio>, <bgsound>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

loop on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Repeat when finished.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="loop"` in XHTML style). Never write `loop="false"`; that still counts as present.

```
<video loop>...</video>
```

The same attribute on other elements

`<audio>`, `<bgsound>`, `<marquee>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<video>`. Full details of the element are in Part 6.

Attribute: low

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meter>	number	HTML5

low on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Upper bound of the 'low' region.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter low="3">...</meter>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

Attribute: lowsrc

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
	URL	Netscape

lowsrc on

Element: (Image) | Since: Netscape

Description

Obsolete, non-standard. Low-quality placeholder loaded first.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```

```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: manifest

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<html>	URL	HTML5 (removed)

manifest on <html>

Element: <html> (Document root) | Since: HTML5 (removed)

Description

Obsolete. Application cache manifest for offline pages.
Replaced by service workers.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<html manifest="https://example.com/page.html">...</html>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <html>. Full details of the element are in Part 6.

Attribute: marginheight

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<frame>	pixels	HTML 4.01
<iframe>	pixels	HTML 4.01

marginheight on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Margins inside the frame.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame marginheight="3">
```

The same attribute on other elements

<iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

marginheight on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. Use CSS in the framed document.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe marginheight="3">...</iframe>
```

The same attribute on other elements

<frame>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: marginwidth

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<frame>	pixels	HTML 4.01
<iframe>	pixels	HTML 4.01

marginwidth on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Margins inside the frame.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame marginwidth="3">
```

The same attribute on other elements

<iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

marginwidth on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. Use CSS in the framed document.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe marginwidth="3">...</iframe>
```

The same attribute on other elements

<frame>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: max

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<input>	number or date	HTML5
<meter>	number	HTML5
<progress>	number	HTML5

max on <input>

Element: <input> (Input control) | Since: HTML5

Description

Maximum value.

Accepted values

number or date

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input max="3">
```

The same attribute on other elements

<meter>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

max on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Upper bound. Default 1.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter max="3">...</meter>
```

The same attribute on other elements

<input>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

max on <progress>

Element: <progress> (Progress indicator) | Since: HTML5

Description

The total amount of work. Default 1.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<progress max="3">...</progress>
```

The same attribute on other elements

<input>, <meter>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <progress>. Full details of the element are in Part 6.

Attribute: maxlength

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	integer	HTML 2.0
<textarea>	integer	HTML5

maxlength on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

Maximum number of characters (text-like types).

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input maxlength="3">
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

maxlength on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Maximum characters.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea maxlength="3">...</textarea>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: media

Accepted by 4 elements. Each use is described on the following pages.

| Element | Values | Since |
|----------|-------------|-----------------|
| <link> | media query | HTML 4.01 |
| <meta> | media query | Living Standard |
| <source> | media query | HTML5 |
| <style> | media query | HTML 4.01 |

media on <link>

Element: <link> (External resource link) | Since: HTML 4.01

Description

Apply the style sheet only when the query matches: screen, print, (max-width: 600px), (prefers-color-scheme: dark).

Accepted values

media query

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link media="media query">
```

The same attribute on other elements

<meta>, <source>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details

of the element are in Part 6.

media on <meta>

Element: <meta> (Metadata) | Since: Living Standard

Description

With name=theme-color: apply only when the query matches.

Accepted values

media query

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta media="media query">
```

The same attribute on other elements

<link>, <source>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

media on <source>

Element: <source> (Media or image source) | Since: HTML5

Description

Use this source only when the query matches. Valid in picture and video (audio ignores it in most browsers).

Accepted values

media query

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source media="media query">
```

The same attribute on other elements

<link>, <meta>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

media on <style>

Element: <style> (Embedded style sheet) | Since: HTML 4.01

Description

Apply only when the query matches, for example print or (max-width: 600px). Default all.

Accepted values

media query

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<style media="media query">...</style>
```

The same attribute on other elements

<link>, <meta>, <source>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

Attribute: method

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<form>	get post dialog	HTML 2.0

method on <form>

Element: <form> (Form) | Since: HTML 2.0

Description

The HTTP method. get puts data in the URL; post sends it in the body; dialog closes the enclosing <dialog>.

Accepted values

get | post | dialog

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form method="get">...</form>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

Attribute: min

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	number or date	HTML5
<meter>	number	HTML5

min on <input>

Element: <input> (Input control) | Since: HTML5

Description

Minimum value (number, range, date/time types).

Accepted values

number or date

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input min="3">
```

The same attribute on other elements

<meter>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

min on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Lower bound of the range. Default 0.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter min="3">...</meter>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

Attribute: minlength

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	integer	HTML5
<textarea>	integer	HTML5

minlength on <input>

Element: <input> (Input control) | Since: HTML5

Description

Minimum number of characters.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input minlength="3">
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

minlength on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Minimum characters.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea minlength="3">...</textarea>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: moz-opaque

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|----------|---------|--------------|
| <canvas> | boolean | Non-standard |

moz-opaque on <canvas>

Element: <canvas> (Graphics canvas) | Since: Non-standard

Description

Firefox only: hints that there is no transparency, for faster rendering.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="moz-opaque"` in XHTML style). Never write `moz-opaque="false"`; that still counts as present.

```
<canvas moz-opaque>...</canvas>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <canvas>. Full details of the element are in Part 6.

Attribute: mozactionhint

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|---------|--------------|
| <input> | various | Non-standard |

mozactionhint on <input>

Element: <input> (Input control) | Since: Non-standard

Description

Non-standard browser-specific attributes.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input mozactionhint="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: multiple

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	boolean	HTML5
<select>	boolean	HTML 2.0

multiple on <input>

Element: <input> (Input control) | Since: HTML5

Description

Allow several values (email: comma-separated; file: several files).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="multiple"` in XHTML style). Never write `multiple="false"`; that still counts as present.

```
<input multiple>
```

The same attribute on other elements

<select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

multiple on <select>

Element: <select> (Drop-down / list box) | Since: HTML 2.0

Description

Allow several options to be selected (Ctrl/Cmd-click).

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="multiple"` in XHTML style). Never write `multiple="false"`; that still counts as present.

```
<select multiple>...</select>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <select>. Full details of the element are in Part 6.

Attribute: muted

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	boolean	HTML5
<video>	boolean	HTML5

muted on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

Start with the sound muted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="muted"` in XHTML style). Never write `muted="false"`; that still counts as present.

```
<audio muted>...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

muted on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Start muted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="muted"` in XHTML style). Never write `muted="false"`; that still counts as present.

```
<video muted>...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: n

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<nextid>	Text (anchor name)	HTML 1

n on <nextid>

Element: <nextid> (Next identifier (HTML 1 editor hint)) | Since: HTML 1

Description

The next anchor name the editor should generate.

Accepted values

Text (anchor name)

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<nextid n="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <nextid>. Full details of the element are in Part 6.

Attribute: name

Accepted by 19 elements. Each use is described on the following pages.

Element	Values	Since
<a>	text	HTML 1 (obsolete in HTML5)
<applet>	text	HTML 3.2
<button>	text	HTML 4.01
<details>	text	Living Standard
<fieldset>	text	HTML5
<form>	text	HTML 2.0
<frame>	text	HTML 4.01
<iframe>	text	HTML 4.01
	text	HTML 4.01
<input>	text	HTML 2.0
<keygen>	text	HTML5
<map>	text	HTML 3.2
<meta>	metadata name	HTML 2.0
<object>	text	HTML 4.01
<output>	text	HTML5
<param>	text	HTML 3.2
<select>	text	HTML 2.0
<slot>	text	DOM
<textarea>	text	HTML 2.0

name on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 1 (obsolete in HTML5)

Description

Obsolete. Defined a named anchor that could be linked to with #name. Use id instead.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a name="value">...</a>
```

The same attribute on other elements

<applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

name on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

A name so that applets on the same page can find each other.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet name="value">...</applet>
```

The same attribute on other elements

<a>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

name on <button>

Element: <button> (Button) | Since: HTML 4.01

Description

The name sent with the form data when this button submits the form.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button name="value">...</button>
```

The same attribute on other elements

<a>, <applet>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

name on <details>

Element: <details> (Disclosure widget) | Since: Living Standard

Description

Groups details elements so that only one in the group can be open at a time.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<details name="value">...</details>
```

The same attribute on other elements

<a>, <applet>, <button>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <details>. Full details of the element are in Part 6.

name on <fieldset>

Element: <fieldset> (Form field group) | Since: HTML5

Description

The name of the group, accessible from the form's elements collection.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<fieldset name="value">...</fieldset>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fieldset>. Full details of the element are in Part 6.

name on <form>

Element: <form> (Form) | Since: HTML 2.0

Description

A name for the form, used by scripts (document.forms.name). Must be unique among forms.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form name="value">...</form>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

name on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Name used as a link target.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame name="value">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

name on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

A name that can be used as the target of links and forms.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe name="value">...</iframe>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

name on

Element: (Image) | Since: HTML 4.01

Description

Obsolete. Use id.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img name="value">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

name on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

The name under which the value is submitted. Without it the control is not submitted. Radio buttons with the same name form a group.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input name="value">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

name on <keygen>

Element: <keygen> (Key-pair generator (obsolete)) | Since: HTML5

Description

Form field name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<keygen name="value">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <keygen>. Full details of the element are in Part 6.

name on <map>

Element: <map> (Image map) | Since: HTML 3.2

Description

Required. The map's name, without #, unique among maps. If id is also given it must have the same value.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<map name="value">...</map>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <map>. Full details of the element are in Part 6.

name on <meta>

Element: <meta> (Metadata) | Since: HTML 2.0

Description

The kind of metadata: application-name, author, description, generator, keywords, referrer, theme-color, color-scheme, viewport, robots, googlebot, creator, publisher, and more.

Accepted values

metadata name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta name="metadata name">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <object>, <output>, <param>, <select>, <slot>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

name on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

Name of the browsing context (for use as a target).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object name="value">...</object>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <output>, <param>, <select>, <slot>, <textarea>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

name on <output>

Element: <output> (Calculation or user-action result) | Since: HTML5

Description

Name of the output (not submitted with the form).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<output name="value">...</output>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <param>, <select>, <slot>, <textarea>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <output>. Full details of the element are in Part 6.

name on <param>

Element: <param> (Object parameter (deprecated)) | Since: HTML 3.2

Description

The parameter name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<param name="value">
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <select>, <slot>, <textarea>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <param>. Full details of the element are in Part 6.

name on <select>

Element: <select> (Drop-down / list box) | Since: HTML 2.0

Description

Name submitted with the form.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<select name="value">...</select>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <slot>, <textarea>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <select>. Full details of the element are in Part 6.

name on <slot>

Element: <slot> (Shadow DOM slot) | Since: DOM

Description

The slot's name. Children with slot="name" are placed here. Omit for the default slot.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<slot name="value">...</slot>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <slot>. Full details of the element are in Part 6.

name on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML 2.0

Description

Name submitted with the form.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea name="value">...</textarea>
```

The same attribute on other elements

<a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: nohref

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<area>	boolean	HTML 3.2 (obsolete)

nohref on <area>

Element: <area> (Image map area) | Since: HTML 3.2 (obsolete)

Description

Obsolete. Indicated the area had no link. Simply omit href instead.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="nohref"` in XHTML style). Never write `nohref="false"`; that still counts as present.

```
<area nohref>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

Attribute: nomodule

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<script>	boolean	HTML5

nomodule on <script>

Element: <script> (Script) | Since: HTML5

Description

Do not run in browsers that support modules; used to provide fallbacks for old browsers.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="nomodule"` in XHTML style). Never write `nomodule="false"`; that still counts as present.

```
<script nomodule>...</script>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: nonce

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<script>	text	CSP
<style>	text	CSP

nonce on <script>

Element: <script> (Script) | Since: CSP

Description

Cryptographic nonce for Content Security Policy (global attribute).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script nonce="value">...</script>
```

The same attribute on other elements

<style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

nonce on <style>

Element: <style> (Embedded style sheet) | Since: CSP

Description

CSP nonce (global).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<style nonce="value">...</style>
```

The same attribute on other elements

<script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

Attribute: noresize

Accepted by 1 element. Each use is described on the following pages.

| Element | Values | Since |
|---------|---------|-----------|
| <frame> | boolean | HTML 4.01 |

noresize on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

User cannot resize the frame.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="noresize"` in XHTML style). Never write `noresize="false"`; that still counts as present.

```
<frame noresize>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

Attribute: noshade

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<hr>	boolean	HTML 3.2

noshade on <hr>

Element: <hr> (Thematic break (horizontal rule)) | Since: HTML 3.2

Description

Obsolete. Draw a solid line instead of a 3D groove. Use CSS.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="noshade"` in XHTML style). Never write `noshade="false"`; that still counts as present.

```
<hr noshade>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <hr>. Full details of the element are in Part 6.

Attribute: novalidate

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<form>	boolean	HTML5

novalidate on <form>

Element: <form> (Form) | Since: HTML5

Description

Do not validate the form on submission.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="novalidate"` in XHTML style). Never write `novalidate="false"`; that still counts as present.

```
<form novalidate>...</form>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

Attribute: nowrap

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<dd>	yes no	IE
<td>	boolean	HTML 3.2
<th>	various	HTML 3.2 / 4.01

nowrap on <dd>

Element: <dd> (Description details) | Since: IE

Description

Obsolete, non-standard. Prevented wrapping.

Accepted values

yes | no

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<dd nowrap="yes">...</dd>
```

The same attribute on other elements

<td>, <th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <dd>. Full details of the element are in Part 6.

nowrap on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS white-space: nowrap.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="nowrap" in XHTML style). Never write nowrap="false"; that still counts as present.

```
<td nowrap>...</td>
```

The same attribute on other elements

<dd>, <th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

nowrap on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th nowrap="various">...</th>
```

The same attribute on other elements

<dd>, <td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: object

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<applet>	file name	HTML 4.01

object on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 4.01

Description

A serialised applet file.

Accepted values

file name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet object="file name">...</applet>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

Attribute: onbeforeunload

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML5

onbeforeunload on <body>

Element: <body> (Document body) | Since: HTML5

Description

Fired when the user is about to leave; can prompt for confirmation.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onbeforeunload="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onhashchange

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML5

onhashchange on <body>

Element: <body> (Document body) | Since: HTML5

Description

Fired when the URL fragment (#...) changes.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onhashchange="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onlanguagechange

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML5

onlanguagechange on <body>

Element: <body> (Document body) | Since: HTML5

Description

Fired when the preferred language changes.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onlanguagechange="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onload

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML 4.01

onload on <body>

Element: <body> (Document body) | Since: HTML 4.01

Description

Fired when the whole page, including images, has loaded.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onload="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onpopstate

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML5

onpopstate on <body>

Element: <body> (Document body) | Since: HTML5

Description

Fired when the active history entry changes.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onpopstate="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onresize

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML 4.01

onresize on <body>

Element: <body> (Document body) | Since: HTML 4.01

Description

Fired when the window is resized.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onresize="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onstorage

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML5

onstorage on <body>

Element: <body> (Document body) | Since: HTML5

Description

Fired when localStorage or sessionStorage changes in another tab.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onstorage="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: onunload

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	script	HTML 4.01

onunload on <body>

Element: <body> (Document body) | Since: HTML 4.01

Description

Fired when the document is unloaded. Unreliable; prefer pagehide.

Accepted values

script

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body onunload="script">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: open

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<details>	boolean	HTML5
<dialog>	boolean	HTML 5.2

open on <details>

Element: <details> (Disclosure widget) | Since: HTML5

Description

The details are visible. Absent means collapsed.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="open"` in XHTML style). Never write `open="false"`; that still counts as present.

```
<details open>...</details>
```

The same attribute on other elements

<dialog>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <details>. Full details of the element are in Part 6.

open on <dialog>

Element: <dialog> (Dialog box) | Since: HTML 5.2

Description

The dialog is visible. Set it with showModal() rather than by hand so that focus and the backdrop work correctly.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="open" in XHTML style). Never write open="false"; that still counts as present.

```
<dialog open>...</dialog>
```

The same attribute on other elements

<details>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <dialog>. Full details of the element are in Part 6.

Attribute: optimum

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meter>	number	HTML5

optimum on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

The ideal value. Determines which region is shown as good (green) versus bad (red/yellow).

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter optimum="3">...</meter>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

Attribute: orient

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	various	Non-standard

orient on <input>

Element: <input> (Input control) | Since: Non-standard

Description

Non-standard browser-specific attributes.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input orient="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: pattern

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	regular expression	HTML5

pattern on <input>

Element: <input> (Input control) | Since: HTML5

Description

A JavaScript regular expression the value must match (text, search, url, tel, email, password). Describe the format in title.

Accepted values

regular expression

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input pattern="regular expression">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: ping

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<a>	space-separated URLs	HTML5
<area>	URLs	HTML5

ping on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML5

Description

A list of URLs that receive a POST request with the body PING when the link is followed. Used for tracking.

Accepted values

space-separated URLs

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a ping="space-separated URLs">...</a>
```

The same attribute on other elements

<area>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

ping on <area>

Element: <area> (Image map area) | Since: HTML5

Description

URLs to ping when the link is followed.

Accepted values

URLs

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area ping="https://example.com/page.html">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

Attribute: placeholder

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<code><input></code>	text	HTML5
<code><textarea></code>	text	HTML5

placeholder on `<input>`

Element: `<input>` (Input control) | Since: HTML5

Description

Hint shown inside an empty text-like control. Not a substitute for a label.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input placeholder="value">
```

The same attribute on other elements

`<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<input>`. Full details of the element are in Part 6.

placeholder on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Hint shown when empty. Not a label.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea placeholder="value">...</textarea>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: playsinline

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<video>	boolean	Living Standard

playsinline on <video>

Element: <video> (Embedded video) | Since: Living Standard

Description

On iPhone, play inline instead of forcing fullscreen.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="playsinline"` in XHTML style). Never write `playsinline="false"`; that still counts as present.

```
<video playsinline>...</video>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: popovertarget

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	element id	Living Standard
<input>	id / show hide toggle	Living Standard

popovertarget on <button>

Element: <button> (Button) | Since: Living Standard

Description

The popover element this button shows, hides or toggles.

Accepted values

element id

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button popovertarget="element id">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

popovertarget on <input>

Element: <input> (Input control) | Since: Living Standard

Description

Control a popover from a button-type input.

Accepted values

id / show | hide | toggle

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input popovertarget="my-id">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: popovertargetaction

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<button>	show hide toggle	Living Standard
<input>	id / show hide toggle	Living Standard

popovertargetaction on <button>

Element: <button> (Button) | Since: Living Standard

Description

What to do to the popover (default toggle).

Accepted values

show | hide | toggle

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button popovertargetaction="show">...</button>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

popovertargetaction on <input>

Element: <input> (Input control) | Since: Living Standard

Description

Control a popover from a button-type input.

Accepted values

id / show | hide | toggle

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input popovertargetaction="my-id">
```

The same attribute on other elements

<button>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: poster

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<video>	URL	HTML5

poster on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Image shown before the video plays or while loading.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video poster="https://example.com/page.html">...</video>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: preload

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	none metadata auto	HTML5
<video>	none metadata auto	HTML5

preload on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

A hint about how much to download before playback: nothing, only metadata such as duration (default in most browsers), or the whole file.

Accepted values

none | metadata | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<audio preload="none">...</audio>
```

The same attribute on other elements

<video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

preload on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

How much to download before play.

Accepted values

none | metadata | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video preload="none">...</video>
```

The same attribute on other elements

<audio>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: preserveAspectRatio

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	keyword	SVG

preserveAspectRatio on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

How to fit the viewBox into the viewport.

Accepted values

keyword

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg preserveAspectRatio="keyword">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: profile

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<head>	URLs	HTML 4.01

profile on <head>

Element: <head> (Document head) | Since: HTML 4.01

Description

Obsolete. Meta data profiles.

Accepted values

URLs

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<head profile="https://example.com/page.html">...</head>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <head>. Full details of the element are in Part 6.

Attribute: prompt

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<isindex>	text	HTML 2.0

prompt on <isindex>

Element: <isindex> (Search index prompt (obsolete)) | Since: HTML 2.0

Description

The label shown before the field.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<isindex prompt="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <isindex>. Full details of the element are in Part 6.

Attribute: property

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meta>	text	RDFa

property on <meta>

Element: <meta> (Metadata) | Since: RDFa

Description

Not in the HTML standard but universally used for RDFa / Open Graph tags such as og:title.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta property="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

Attribute: radiogroup

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<menuitem>	various	HTML5 draft

radiogroup on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

State and grouping.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem radiogroup="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

Attribute: readonly

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<input>	boolean	HTML 4.01
<textarea>	boolean	HTML 4.01

readonly on <input>

Element: <input> (Input control) | Since: HTML 4.01

Description

Cannot be edited but is focusable and submitted. Text-like types only.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="readonly"` in XHTML style). Never write `readonly="false"`; that still counts as present.

```
<input readonly>
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

readonly on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML 4.01

Description

Cannot be edited; is submitted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="readonly"` in XHTML style). Never write `readonly="false"`; that still counts as present.

```
<textarea readonly>...</textarea>
```

The same attribute on other elements

<input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: referrerpolicy

Accepted by 7 elements. Each use is described on the following pages.

| Element | Values | Since |
|----------|---|--------------|
| <a> | no-referrer no-referrer-when-downgrade origin origin-when-cross-origin same-origin strict-origin strict-origin-when-cross-origin unsafe-url | HTML5 |
| <area> | referrer policy keyword | HTML5 |
| <iframe> | referrer policy keyword | HTML5 |
| | referrer policy keyword | HTML5 |
| <link> | referrer policy | HTML5 |
| <portal> | policy | Experimental |
| <script> | referrer policy | HTML5 |

referrerpolicy on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML5

Description

How much referrer information is sent when following the link.

Accepted values

no-referrer | no-referrer-when-downgrade | origin | origin-when-cross-origin | same-origin | strict-origin | strict-origin-when-cross-origin | unsafe-url

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a referrerpolicy="no-referrer">...</a>
```


The same attribute on other elements

`<area>`, `<iframe>`, `<img>`, `<link>`, `<portal>`, `<script>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<a>`. Full details of the element are in Part 6.

referrerpolicy on <area>

Element: <area> (Image map area) | Since: HTML5

Description

Referrer policy for the navigation.

Accepted values

referrer policy keyword

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area referrerpolicy="referrer policy keyword">
```

The same attribute on other elements

<a>, <iframe>, , <link>, <portal>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

referrerpolicy on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5

Description

Which referrer to send when loading the frame.

Accepted values

referrer policy keyword

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe referrerpolicy="referrer policy keyword">...</iframe>
```

The same attribute on other elements

<a>, <area>, , <link>, <portal>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

referrerpolicy on

Element: (Image) | Since: HTML5

Description

Which referrer to send.

Accepted values

referrer policy keyword

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img referrerpolicy="referrer policy keyword">
```

The same attribute on other elements

<a>, <area>, <iframe>, <link>, <portal>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

referrerpolicy on <link>

Element: <link> (External resource link) | Since: HTML5

Description

Referrer to send when fetching.

Accepted values

referrer policy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link referrerpolicy="referrer policy">
```

The same attribute on other elements

<a>, <area>, <iframe>, , <portal>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

referrerpolicy on <portal>

Element: <portal> (Portal (experimental, abandoned)) | Since: Experimental

Description

Referrer policy.

Accepted values

policy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<portal referrerpolicy="policy">...</portal>
```

The same attribute on other elements

<a>, <area>, <iframe>, , <link>, <script>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <portal>. Full details of the element are in Part 6.

referrerpolicy on <script>

Element: <script> (Script) | Since: HTML5

Description

Referrer to send.

Accepted values

referrer policy

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script referrerpolicy="referrer policy">...</script>
```

The same attribute on other elements

<a>, <area>, <iframe>, , <link>, <portal>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

Attribute: rel

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<a>	space-separated link types	HTML 4.01
<area>	link types	HTML5
<form>	link types	Living Standard
<link>	space-separated link types	HTML 2.0

rel on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01

Description

The relationship between the current document and the linked document. Common values: nofollow, noopener, norereferrer, external, alternate, author, bookmark, help, license, next, prev, search, tag. See the link types table in Part 9.

Accepted values

space-separated link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a rel="value">...</a>
```

The same attribute on other elements

<area>, <form>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<a>`. Full details of the element are in Part 6.

rel on <area>

Element: <area> (Image map area) | Since: HTML5

Description

Relationship of the linked resource.

Accepted values

link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area rel="link types">
```

The same attribute on other elements

<a>, <form>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

rel on <form>

Element: <form> (Form) | Since: Living Standard

Description

Relationship of the target resource, for example noopener or norereferrer.

Accepted values

link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form rel="link types">...</form>
```

The same attribute on other elements

<a>, <area>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

rel on <link>

Element: <link> (External resource link) | Since: HTML 2.0

Description

Required. Relationship: stylesheet, icon, apple-touch-icon, manifest, canonical, alternate, preload, prefetch, preconnect, dns-prefetch, modulepreload, prerender, author, license, help, next, prev, search, pingback, privacy-policy, terms-of-service, expect.

Accepted values

space-separated link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link rel="value">
```

The same attribute on other elements

<a>, <area>, <form>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: required

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
<input>	boolean	HTML5
<select>	boolean	HTML5
<textarea>	boolean	HTML5

required on <input>

Element: <input> (Input control) | Since: HTML5

Description

Must have a value before the form can be submitted.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="required"` in XHTML style). Never write `required="false"`; that still counts as present.

```
<input required>
```

The same attribute on other elements

<select>, <textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

required on <select>

Element: <select> (Drop-down / list box) | Since: HTML5

Description

The user must choose an option with a non-empty value.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="required"` in XHTML style). Never write `required="false"`; that still counts as present.

```
<select required>...</select>
```

The same attribute on other elements

`<input>`, `<textarea>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<select>`. Full details of the element are in Part 6.

required on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Must not be empty.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="required"` in XHTML style). Never write `required="false"`; that still counts as present.

```
<textarea required>...</textarea>
```

The same attribute on other elements

<input>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: results

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	various	Non-standard

results on <input>

Element: <input> (Input control) | Since: Non-standard

Description

Non-standard browser-specific attributes.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input results="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: rev

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<a>	link types	HTML 4.01 (obsolete in HTML5)
<link>	link types	HTML 4.01

rev on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01 (obsolete in HTML5)

Description

Obsolete. Reverse relationship. Use rel instead.

Accepted values

link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a rev="link types">...</a>
```

The same attribute on other elements

<link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

rev on <link>

Element: <link> (External resource link) | Since: HTML 4.01

Description

Obsolete. Reverse relationship.

Accepted values

link types

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link rev="link types">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: reversed

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
	boolean	HTML5

reversed on

Element: (Ordered list) | Since: HTML5

Description

Number the items in descending order.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="reversed"` in XHTML style). Never write `reversed="false"`; that still counts as present.

```
<ol reversed>...</ol>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: rightmargin

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	pixels	IE/Netscape

rightmargin on <body>

Element: <body> (Document body) | Since: IE/Netscape

Description

Obsolete, non-standard. Page margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body rightmargin="3">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: role

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	-	ARIA

role on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: ARIA

Description

Accessibility name and role for the graphic.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg role="-">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: rows

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<frameset>	comma-separated sizes	HTML 4.01
<textarea>	positive integer	HTML 2.0

rows on <frameset>

Element: <frameset> (Frameset (obsolete)) | Since: HTML 4.01

Description

Row heights.

Accepted values

comma-separated sizes

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frameset rows="comma-separated sizes">...</frameset>
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frameset>. Full details of the element are in Part 6.

rows on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML 2.0

Description

Visible number of lines. Default 2.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea rows="3">...</textarea>
```

The same attribute on other elements

<frameset>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: rowspan

Accepted by 2 elements. Each use is described on the following pages.

| Element | Values | Since |
|---------|----------------------|----------|
| <td> | non-negative integer | HTML 3.2 |
| <th> | non-negative integer | HTML 3.2 |

rowspan on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Number of rows the cell spans. Default 1; 0 means to the end of the row group. Maximum 65534.

Accepted values

non-negative integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td rowspan="3">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

rowspan on <th>

Element: <th> (Table header cell) | Since: HTML 3.2

Description

Rows spanned.

Accepted values

non-negative integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th rowspan="3">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: rules

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<table>	none groups rows cols all	HTML 4.01

rules on <table>

Element: <table> (Table) | Since: HTML 4.01

Description

Obsolete. Which inner borders to draw. Use CSS border on cells.

Accepted values

none | groups | rows | cols | all

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table rules="none">...</table>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: sandbox

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<iframe>	empty or space-separated tokens	HTML5

sandbox on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5

Description

Applies extra restrictions. Empty value = all restrictions.
Tokens that lift restrictions: allow-downloads, allow-forms, allow-modals, allow-orientation-lock, allow-pointer-lock, allow-popups, allow-popups-to-escape-sandbox, allow-presentation, allow-same-origin, allow-scripts, allow-top-navigation, allow-top-navigation-by-user-activation, allow-top-navigation-to-custom-protocols.

Accepted values

empty or space-separated tokens

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe sandbox="empty">...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: scheme

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<meta>	text	HTML 4.01

scheme on <meta>

Element: <meta> (Metadata) | Since: HTML 4.01

Description

Obsolete. Interpretation scheme for content.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meta scheme="value">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meta>. Full details of the element are in Part 6.

Attribute: scope

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<td>	-	HTML 4.01
<th>	row col rowgroup colgroup	HTML 4.01

scope on <td>

Element: <td> (Table data cell) | Since: HTML 4.01

Description

Obsolete on td (use on th).

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td scope="-">...</td>
```

The same attribute on other elements

<th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

scope on <th>

Element: <th> (Table header cell) | Since: HTML 4.01

Description

Which cells this header describes.

Accepted values

row | col | rowgroup | colgroup

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th scope="row">...</th>
```

The same attribute on other elements

<td>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

Attribute: scoped

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<style>	boolean	HTML5 draft (removed)

scoped on <style>

Element: <style> (Embedded style sheet) | Since: HTML5 draft (removed)

Description

Removed. Was meant to scope styles to the parent element; use Shadow DOM or @scope.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="scoped" in XHTML style). Never write scoped="false"; that still counts as present.

```
<style scoped>...</style>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

Attribute: scrollamount

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<marquee>	pixels	IE

scrollamount on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Distance per step.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee scrollamount="3">...</marquee>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

Attribute: scrolldelay

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<marquee>	ms	IE

scrolldelay on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Delay between steps.

Accepted values

ms

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee scrolldelay="ms">...</marquee>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

Attribute: scrolling

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<frame>	yes no auto	HTML 4.01
<iframe>	yes no auto	HTML 4.01

scrolling on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

Scrollbar behaviour.

Accepted values

yes | no | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame scrolling="yes">
```

The same attribute on other elements

<iframe>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

scrolling on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Obsolete. Use CSS overflow in the framed document.

Accepted values

yes | no | auto

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe scrolling="yes">...</iframe>
```

The same attribute on other elements

<frame>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: select

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<content>	CSS selector	Shadow DOM v0

select on <content>

Element: <content> (Shadow DOM insertion point (obsolete)) | Since: Shadow DOM v0

Description

Which light DOM children to insert.

Accepted values

CSS selector

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<content select="CSS selector">...</content>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <content>. Full details of the element are in Part 6.

Attribute: selected

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<option>	boolean	HTML 2.0

selected on <option>

Element: <option> (Option) | Since: HTML 2.0

Description

Initially selected.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="selected"` in XHTML style). Never write `selected="false"`; that still counts as present.

```
<option selected>...</option>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <option>. Full details of the element are in Part 6.

Attribute: shadowrootclonable

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<template>	boolean	Living Standard

shadowrootclonable on <template>

Element: <template> (Content template) | Since: Living Standard

Description

Sets clonable on the declarative shadow root.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="shadowrootclonable"` in XHTML style). Never write `shadowrootclonable="false"`; that still counts as present.

```
<template shadowrootclonable>...</template>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <template>. Full details of the element are in Part 6.

Attribute:

shadowrootdelegatesfocus

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<template>	boolean	Living Standard

shadowrootdelegatesfocus on <template>

Element: <template> (Content template) | Since: Living Standard

Description

Sets delegatesFocus on the declarative shadow root.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="shadowrootdelegatesfocus"` in XHTML style). Never write `shadowrootdelegatesfocus="false"`; that still counts as present.

```
<template shadowrootdelegatesfocus>...</template>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <template>. Full details of the element are in Part 6.

Attribute: shadowrootmode

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<template>	open closed	Living Standard

shadowrootmode on <template>

Element: <template> (Content template) | Since: Living Standard

Description

Creates a declarative shadow root on the parent element.

Accepted values

open | closed

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<template shadowrootmode="open">...</template>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <template>. Full details of the element are in Part 6.

Attribute:

shadowrootserializable

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<template>	boolean	Living Standard

shadowrootserializable on <template>

Element: <template> (Content template) | Since: Living Standard

Description

Sets serializable on the declarative shadow root.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="shadowrootserializable"` in XHTML style). Never write `shadowrootserializable="false"`; that still counts as present.

```
<template shadowrootserializable>...</template>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <template>. Full details of the element are in Part 6.

Attribute: shape

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<a>	rect circle poly default	HTML 4.01 (obsolete in HTML5)
<area>	rect circle poly default	HTML 3.2

shape on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01 (obsolete in HTML5)

Description

Obsolete. Shape of a clickable region. Use <area> instead.

Accepted values

rect | circle | poly | default

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a shape="rect">...</a>
```

The same attribute on other elements

<area>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

shape on <area>

Element: <area> (Image map area) | Since: HTML 3.2

Description

The shape of the region. default covers the whole image.

Accepted values

rect | circle | poly | default

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area shape="rect">
```

The same attribute on other elements

<a>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

Attribute: size

Accepted by 6 elements. Each use is described on the following pages.

Element	Values	Since
<basefont>	1 to 7	HTML 3.2
	1-7 or +n / -n	HTML 3.2
<hr>	pixels	HTML 3.2
<input>	integer	HTML 2.0
<select>	integer	HTML 2.0
<spacer>	pixels	Netscape

size on <basefont>

Element: <basefont> (Base font (obsolete)) | Since: HTML 3.2

Description

Default font size on the 1-7 HTML scale (3 = normal).

Accepted values

1 to 7

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<basefont size="1 to 7">
```

The same attribute on other elements

, <hr>, <input>, <select>, <spacer>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<basefont>`. Full details of the element are in Part 6.

size on

Element: (Font (obsolete)) | Since: HTML 3.2

Description

Font size on the HTML scale: 1 = x-small, 2 = small, 3 = medium (default), 4 = large, 5 = x-large, 6 = xx-large, 7 = xxx-large.

Accepted values

1-7 or +n / -n

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<font size="1-7">...</font>
```

The same attribute on other elements

<basefont>, <hr>, <input>, <select>, <spacer>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

size on <hr>

Element: <hr> (Thematic break (horizontal rule)) | Since: HTML 3.2

Description

Obsolete. Height of the rule. Use CSS height or border-width.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<hr size="3">
```

The same attribute on other elements

<basefont>, , <input>, <select>, <spacer>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <hr>. Full details of the element are in Part 6.

size on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

Visible width in characters (text-like types). Prefer CSS width.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input size="3">
```

The same attribute on other elements

<basefont>, , <hr>, <select>, <spacer>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

size on <select>

Element: <select> (Drop-down / list box) | Since: HTML 2.0

Description

Number of visible rows. 1 (default) gives a drop-down; more gives a list box.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<select size="3">...</select>
```

The same attribute on other elements

<basefont>, , <hr>, <input>, <spacer>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <select>. Full details of the element are in Part 6.

size on <spacer>

Element: <spacer> (Spacer (non-standard, obsolete)) | Since: Netscape

Description

Amount of space.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<spacer size="3">
```

The same attribute on other elements

<basefont>, , <hr>, <input>, <select>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <spacer>. Full details of the element are in Part 6.

Attribute: sizes

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
	media conditions and lengths	HTML5
<link>	any or WxH list	HTML5
<source>	size list	HTML5

sizes on

Element: (Image) | Since: HTML5

Description

How wide the image will be rendered, for example '(max-width: 600px) 100vw, 50vw'. Required when srcset uses width descriptors.

Accepted values

media conditions and lengths

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img sizes="value">
```

The same attribute on other elements

<link>, <source>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

sizes on <link>

Element: <link> (External resource link) | Since: HTML5

Description

Icon sizes, for example 16x16 32x32 or any (for SVG).

Accepted values

any or WxH list

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link sizes="any">
```

The same attribute on other elements

, <source>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

sizes on <source>

Element: <source> (Media or image source) | Since: HTML5

Description

Display widths for width-descriptor srcset (picture only).

Accepted values

size list

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source sizes="size list">
```

The same attribute on other elements

, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

Attribute: span

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<col>	positive integer	HTML 4.01
<colgroup>	positive integer	HTML 4.01

span on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

How many consecutive columns this element represents.
Default 1.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col span="3">
```

The same attribute on other elements

<colgroup>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <col>. Full details of the element are in Part 6.

span on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Number of columns in the group when there are no <col> children.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup span="3">...</colgroup>
```

The same attribute on other elements

<col>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

Attribute: spellcheck

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<textarea>	true false	HTML5

spellcheck on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

Enable or disable spell checking (global).

Accepted values

true | false

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea spellcheck="true">...</textarea>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: src

Accepted by 13 elements. Each use is described on the following pages.

Element	Values	Since
<audio>	URL	HTML5
<bgsound>	URL	IE
<embed>	URL	Netscape / HTML5
<frame>	URL	HTML 4.01
<iframe>	URL	HTML 4.01
<image>	as 	-
	URL	HTML 2.0
<input>	URL	HTML 2.0
<portal>	URL	Experimental
<script>	URL	HTML 4.01
<source>	URL	HTML5
<track>	URL	HTML5
<video>	URL	HTML5

src on <audio>

Element: <audio> (Embedded audio) | Since: HTML5

Description

The URL of the audio file. Alternatively use <source> children.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<audio src="https://example.com/page.html">...</audio>
```

The same attribute on other elements

<bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <audio>. Full details of the element are in Part 6.

src on <bgsound>

Element: <bgsound> (Background sound (non-standard, obsolete)) | Since: IE

Description

The sound file.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bgsound src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bgsound>. Full details of the element are in Part 6.

src on <embed>

Element: <embed> (External content) | Since: Netscape / HTML5

Description

The URL of the resource.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<embed src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <bgsound>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <embed>. Full details of the element are in Part 6.

src on <frame>

Element: <frame> (Frame (obsolete)) | Since: HTML 4.01

Description

The document to display.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<frame src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <frame>. Full details of the element are in Part 6.

src on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

The page to embed. about:blank shows an empty page.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe src="https://example.com/page.html">...</iframe>
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

src on <image>

Element: <image> (Image (non-standard alias, obsolete)) | Since: -

Description

Whatever the parser transfers to the it creates.

Accepted values

as

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<image src="as <img>">
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, , <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <image>. Full details of the element are in Part 6.

src on

Element: (Image) | Since: HTML 2.0

Description

The image URL. Required (unless srcset is used with a browser that supports it).

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```

```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, <input>, <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

src on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

The image for type=image.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <portal>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

src on <portal>

Element: <portal> (Portal (experimental, abandoned)) | Since: Experimental

Description

Page to preview.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<portal src="https://example.com/page.html">...</portal>
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <script>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <portal>. Full details of the element are in Part 6.

src on <script>

Element: <script> (Script) | Since: HTML 4.01

Description

External script file.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script src="https://example.com/page.html">...</script>
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <source>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

src on <source>

Element: <source> (Media or image source) | Since: HTML5

Description

The media file. Required in <audio>/<video>; ignored in <picture>.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <track>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

src on <track>

Element: <track> (Text track for media) | Since: HTML5

Description

Required. The WebVTT file.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<track src="https://example.com/page.html">
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <track>. Full details of the element are in Part 6.

src on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

The video file. Or use <source> children.

Accepted values

URL

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video src="https://example.com/page.html">...</video>
```

The same attribute on other elements

<audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: srcdoc

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<iframe>	HTML text	HTML5

srcdoc on <iframe>

Element: <iframe> (Inline frame) | Since: HTML5

Description

Inline HTML to display instead of loading src. Overrides src if present.

Accepted values

HTML text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe srcdoc="HTML text">...</iframe>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

Attribute: srclang

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<track>	language tag	HTML5

srclang on <track>

Element: <track> (Text track for media) | Since: HTML5

Description

Language of the track. Required for subtitles.

Accepted values

language tag

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<track srclang="en">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <track>. Full details of the element are in Part 6.

Attribute: srcset

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
	comma-separated candidates	HTML5
<source>	candidate list	HTML5

srcset on

Element: (Image) | Since: HTML5

Description

Alternative image files: 'small.jpg 480w, large.jpg 1200w' (width descriptors) or 'img.jpg 1x, img@2x.jpg 2x' (density descriptors).

Accepted values

comma-separated candidates

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img srcset="value">
```

The same attribute on other elements

<source>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

srcset on <source>

Element: <source> (Media or image source) | Since: HTML5

Description

Image candidates. Required in <picture>; ignored in media elements.

Accepted values

candidate list

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source srcset="candidate list">
```

The same attribute on other elements

. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

Attribute: standby

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	various	HTML 4.01 / HTML5 (removed)

standby on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object standby="various">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: start

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
	integer	HTML 3.2

start on

Element: (Ordered list) | Since: HTML 3.2

Description

The number of the first item (always a decimal integer even if type is a or i).

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<ol start="3">...</ol>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: step

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	number or any	HTML5

step on <input>

Element: <input> (Input control) | Since: HTML5

Description

Granularity of allowed values; 'any' allows any value.

Accepted values

number or any

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input step="3">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: stroke

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	colour / length	SVG

stroke on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Presentation attributes for shapes (can also be CSS).

Accepted values

colour / length

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg stroke="#336699">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: stroke-width

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	colour / length	SVG

stroke-width on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Presentation attributes for shapes (can also be CSS).

Accepted values

colour / length

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg stroke-width="#336699">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: summary

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<table>	text	HTML 4.01

summary on <table>

Element: <table> (Table) | Since: HTML 4.01

Description

Obsolete. Description of the table for screen readers. Use <caption> or aria-describedby.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table summary="value">...</table>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

Attribute: target

Accepted by 5 elements. Each use is described on the following pages.

Element	Values	Since
<a>	_self _blank _parent _top frame-name	HTML 4.01
<area>	_self _blank _parent _top name	HTML 4.01
<base>	_self _blank _parent _top name	HTML 4.01
<form>	_self _blank _parent _top name	HTML 4.01
<link>	browsing context	HTML 4.01

target on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01

Description

Where to open the linked document. `_self` (default) opens it in the same browsing context; `_blank` opens a new tab or window; `_parent` opens it in the parent frame; `_top` opens it in the full window, replacing all frames. A custom name targets an `<iframe>` with that name.

Accepted values

`_self` | `_blank` | `_parent` | `_top` | frame-name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a target="_self">...</a>
```

The same attribute on other elements

<area>, <base>, <form>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

target on <area>

Element: <area> (Image map area) | Since: HTML 4.01

Description

Where to open the destination.

Accepted values

`_self` | `_blank` | `_parent` | `_top` | `name`

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area target="_self">
```

The same attribute on other elements

`<a>`, `<base>`, `<form>`, `<link>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<area>`. Full details of the element are in Part 6.

target on <base>

Element: <base> (Document base URL) | Since: HTML 4.01

Description

The default browsing context for links and forms that do not specify their own target.

Accepted values

`_self` | `_blank` | `_parent` | `_top` | `name`

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<base target="_self">
```

The same attribute on other elements

`<a>`, `<area>`, `<form>`, `<link>`. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<base>`. Full details of the element are in Part 6.

target on <form>

Element: <form> (Form) | Since: HTML 4.01

Description

Where to display the response.

Accepted values

_self | _blank | _parent | _top | name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<form target="_self">...</form>
```

The same attribute on other elements

<a>, <area>, <base>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <form>. Full details of the element are in Part 6.

target on <link>

Element: <link> (External resource link) | Since: HTML 4.01

Description

Obsolete on link.

Accepted values

browsing context

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link target="browsing context">
```

The same attribute on other elements

<a>, <area>, <base>, <form>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

Attribute: text

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	colour	HTML 3.2

text on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Text colour. Use CSS color.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body text="#336699">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: title

Accepted by 6 elements. Each use is described on the following pages.

Element	Values	Since
<abbr>	text	HTML 4.01
<acronym>	text	HTML 4.01
<dfn>	text	HTML 4.01
<iframe>	text	HTML 4.01
<link>	text	HTML 2.0
<style>	text	HTML 4.01

title on <abbr>

Element: <abbr> (Abbreviation) | Since: HTML 4.01

Description

The full expansion of the abbreviation. On <abbr> this attribute has a specific meaning and should contain nothing else.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<abbr title="value">...</abbr>
```

The same attribute on other elements

<acronym>, <dfn>, <iframe>, <link>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<abbr>`. Full details of the element are in Part 6.

title on <acronym>

Element: <acronym> (Acronym (obsolete)) | Since: HTML 4.01

Description

The expansion of the acronym.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<acronym title="value">...</acronym>
```

The same attribute on other elements

<abbr>, <dfn>, <iframe>, <link>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <acronym>. Full details of the element are in Part 6.

title on <dfn>

Element: <dfn> (Definition term) | Since: HTML 4.01

Description

The term being defined, if different from the content.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<dfn title="value">...</dfn>
```

The same attribute on other elements

<abbr>, <acronym>, <iframe>, <link>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <dfn>. Full details of the element are in Part 6.

title on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

A description of the embedded content for assistive technology. Strongly recommended.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe title="value">...</iframe>
```

The same attribute on other elements

<abbr>, <acronym>, <dfn>, <link>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

title on <link>

Element: <link> (External resource link) | Since: HTML 2.0

Description

For style sheets: the name of an alternate style sheet set. For feeds: the feed title.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link title="value">
```

The same attribute on other elements

<abbr>, <acronym>, <dfn>, <iframe>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

title on <style>

Element: <style> (Embedded style sheet) | Since: HTML 4.01

Description

Name of an alternate style sheet set.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<style title="value">...</style>
```

The same attribute on other elements

<abbr>, <acronym>, <dfn>, <iframe>, <link>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

Attribute: topmargin

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	pixels	IE/Netscape

topmargin on <body>

Element: <body> (Document body) | Since: IE/Netscape

Description

Obsolete, non-standard. Page margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body topmargin="3">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: truespeed

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<marquee>	boolean	IE

truespeed on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Allow delays under 60 ms.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or `name=""` / `name="truespeed"` in XHTML style). Never write `truespeed="false"`; that still counts as present.

```
<marquee truespeed>...</marquee>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

Attribute: type

Accepted by 17 elements. Each use is described on the following pages.

Element	Values	Since
<a>	MIME type	HTML 4.01
<area>	MIME type	HTML 4.01
<button>	submit reset button	HTML 4.01
<embed>	MIME type	Netscape / HTML5
<input>	see Part 6	HTML 2.0 / HTML5
	1 a A i I disc circle square	HTML 3.2
<link>	MIME type	HTML 2.0
<menu>	context toolbar list	HTML5 draft (removed)
<menuitem>	command checkbox radio	HTML5 draft
<object>	MIME type	HTML 4.01
	1 a A i I	HTML 3.2
<param>	MIME type	HTML 4.01
<script>	omitted module importmap speculationrules MIME type	HTML 4.01 / HTML5
<source>	MIME type, optionally with codecs	HTML5
<spacer>	horizontal vertical block	Netscape
<style>	text/css	HTML 4.01
	disc circle square	HTML 3.2

type on <a>

Element: <a> (Anchor (hyperlink)) | Since: HTML 4.01

Description

The media type of the linked resource, for example application/pdf. Advisory only.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<a type="text/plain">...</a>
```

The same attribute on other elements

<area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <a>. Full details of the element are in Part 6.

type on <area>

Element: <area> (Image map area) | Since: HTML 4.01

Description

Obsolete in HTML5. Type of the linked resource.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<area type="text/plain">
```

The same attribute on other elements

<a>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <area>. Full details of the element are in Part 6.

type on <button>

Element: <button> (Button) | Since: HTML 4.01

Description

submit (default) sends the form; reset restores default values; button does nothing by itself.

Accepted values

submit | reset | button

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button type="submit">...</button>
```

The same attribute on other elements

<a>, <area>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <button>. Full details of the element are in Part 6.

type on <embed>

Element: <embed> (External content) | Since: Netscape / HTML5

Description

The type of the resource, used to choose the plug-in or handler.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<embed type="text/plain">
```

The same attribute on other elements

<a>, <area>, <button>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <embed>. Full details of the element are in Part 6.

type on <input>

Element: <input> (Input control) | Since: HTML 2.0 / HTML5

Description

The kind of control. Default text. Values: button, checkbox, color, date, datetime-local, email, file, hidden, image, month, number, password, radio, range, reset, search, submit, tel, text, time, url, week.

Accepted values

see Part 6

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input type="see Part 6">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

type on

Element: (List item) | Since: HTML 3.2

Description

Obsolete. Bullet or numbering style for this item. Use CSS `list-style-type`.

Accepted values

1 | a | A | i | I | disc | circle | square

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<li type="1">...</li>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

type on <link>

Element: <link> (External resource link) | Since: HTML 2.0

Description

The type of the resource, for example text/css, image/png, application/rss+xml.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<link type="text/plain">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <link>. Full details of the element are in Part 6.

type on <menu>

Element: <menu> (Menu (toolbar list)) | Since: HTML5 draft (removed)

Description

Obsolete. From the abandoned HTML5 context menu design.

Accepted values

context | toolbar | list

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menu type="context">...</menu>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menu>. Full details of the element are in Part 6.

type on <menuitem>

Element: <menuitem> (Menu item (obsolete)) | Since: HTML5 draft

Description

Kind of item.

Accepted values

command | checkbox | radio

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<menuitem type="command">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <object>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <menuitem>. Full details of the element are in Part 6.

type on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

The content type of the resource, for example application/pdf, image/svg+xml.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object type="text/plain">...</object>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, , <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

type on

Element: (Ordered list) | Since: HTML 3.2

Description

Numbering type: decimal (default), lowercase letters, uppercase letters, lowercase roman, uppercase roman.

Accepted values

1 | a | A | i | I

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<ol type="1">...</ol>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, <param>, <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

type on <param>

Element: <param> (Object parameter (deprecated)) | Since: HTML 4.01

Description

Obsolete. Type of the value when valuetype=ref.

Accepted values

MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<param type="text/plain">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <script>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <param>. Full details of the element are in Part 6.

type on <script>

Element: <script> (Script) | Since: HTML 4.01 / HTML5

Description

Omitted or text/javascript: classic script. module: ES module. importmap: JSON import map. Any other type: data block that is not executed.

Accepted values

omitted | module | importmap | speculationrules | MIME type

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<script type="text/plain">...</script>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <source>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <script>. Full details of the element are in Part 6.

type on <source>

Element: <source> (Media or image source) | Since: HTML5

Description

The resource type, for example image/webp, video/mp4, video/webm; codecs="vp9, opus". Lets the browser skip unsupported files without downloading them.

Accepted values

MIME type, optionally with codecs

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source type="text/plain">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <spacer>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

type on <spacer>

Element: <spacer> (Spacer (non-standard, obsolete)) | Since: Netscape

Description

Kind of space.

Accepted values

horizontal | vertical | block

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<spacer type="horizontal">
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <style>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <spacer>. Full details of the element are in Part 6.

type on <style>

Element: <style> (Embedded style sheet) | Since: HTML 4.01

Description

Obsolete. The only allowed value is text/css; omit it.

Accepted values

text/css

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<style type="value">...</style>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, . The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <style>. Full details of the element are in Part 6.

type on

Element: (Unordered list) | Since: HTML 3.2

Description

Obsolete. Bullet style. Use CSS list-style-type.

Accepted values

disc | circle | square

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<ul type="disc">...</ul>
```

The same attribute on other elements

<a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

Attribute: **typemustmatch**

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<object>	various	HTML 4.01 / HTML5 (removed)

typemustmatch on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object typemustmatch="various">...</object>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: usemap

Accepted by 3 elements. Each use is described on the following pages.

Element	Values	Since
	#map-name	HTML 3.2
<input>	#map	HTML 4.01
<object>	#map	HTML 4.01

usemap on

Element: (Image) | Since: HTML 3.2

Description

Links the image to a <map> element to make it an image map.

Accepted values

#map-name

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img usemap="#map-name">
```

The same attribute on other elements

<input>, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

usemap on <input>

Element: <input> (Input control) | Since: HTML 4.01

Description

Obsolete on input. Use .

Accepted values

#map

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input usemap="#map">
```

The same attribute on other elements

, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

usemap on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

Image map to use (obsolete in the Living Standard).

Accepted values

#map

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object usemap="#map">...</object>
```

The same attribute on other elements

, <input>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: valign

Accepted by 8 elements. Each use is described on the following pages.

Element	Values	Since
<col>	top middle bottom baseline	HTML 4.01
<colgroup>	various	HTML 4.01
<tbody>	various	HTML 4.01
<td>	top middle bottom baseline	HTML 3.2
<tfoot>	various	HTML 4.01
<th>	various	HTML 3.2 / 4.01
<thead>	various	HTML 4.01
<tr>	top middle bottom baseline	HTML 3.2

valign on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Vertical alignment. Use CSS vertical-align.

Accepted values

top | middle | bottom | baseline

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col valign="top">
```

The same attribute on other elements

<colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because

accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<col>`. Full details of the element are in Part 6.

valign on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup valign="various">...</colgroup>
```

The same attribute on other elements

<col>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

valign on <tbody>

Element: <tbody> (Table body) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tbody valign="various">...</tbody>
```

The same attribute on other elements

<col>, <colgroup>, <td>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tbody>. Full details of the element are in Part 6.

valign on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS vertical-align.

Accepted values

top | middle | bottom | baseline

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td valign="top">...</td>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <tfoot>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

valign on <tfoot>

Element: <tfoot> (Table footer) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tfoot valign="various">...</tfoot>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <th>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tfoot>. Full details of the element are in Part 6.

valign on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th valign="various">...</th>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <thead>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

valign on <thead>

Element: <thead> (Table header) | Since: HTML 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<thead valign="various">...</thead>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <tr>.

The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <thead>. Full details of the element are in Part 6.

valign on <tr>

Element: <tr> (Table row) | Since: HTML 3.2

Description

Obsolete. Use CSS vertical-align.

Accepted values

top | middle | bottom | baseline

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<tr valign="top">...</tr>
```

The same attribute on other elements

<col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <tr>. Full details of the element are in Part 6.

Attribute: value

Accepted by 8 elements. Each use is described on the following pages.

Element	Values	Since
<button>	text	HTML 4.01
<data>	text	HTML5
<input>	text	HTML 2.0
	integer	HTML 3.2
<meter>	number	HTML5
<option>	text	HTML 2.0
<param>	text	HTML 3.2
<progress>	number	HTML5

value on <button>

Element: <button> (Button) | Since: HTML 4.01

Description

The value sent with the name.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<button value="value">...</button>
```

The same attribute on other elements

<data>, <input>, , <meter>, <option>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on `<button>`. Full details of the element are in Part 6.

value on <data>

Element: <data> (Machine-readable value) | Since: HTML5

Description

Required. The machine-readable form of the content.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<data value="value">...</data>
```

The same attribute on other elements

<button>, <input>, , <meter>, <option>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <data>. Full details of the element are in Part 6.

value on <input>

Element: <input> (Input control) | Since: HTML 2.0

Description

The initial value (text types), the value submitted when checked (checkbox, radio), or the button label (submit, reset, button).

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input value="value">
```

The same attribute on other elements

<button>, <data>, , <meter>, <option>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

value on

Element: (List item) | Since: HTML 3.2

Description

In : the ordinal number of this item. Following items count from it.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<li value="3">...</li>
```

The same attribute on other elements

<button>, <data>, <input>, <meter>, <option>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

value on <meter>

Element: <meter> (Scalar measurement (gauge)) | Since: HTML5

Description

Required. The current value, between min and max.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<meter value="3">...</meter>
```

The same attribute on other elements

<button>, <data>, <input>, , <option>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <meter>. Full details of the element are in Part 6.

value on <option>

Element: <option> (Option) | Since: HTML 2.0

Description

The value submitted when this option is selected. Defaults to the text content.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<option value="value">...</option>
```

The same attribute on other elements

<button>, <data>, <input>, , <meter>, <param>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <option>. Full details of the element are in Part 6.

value on <param>

Element: <param> (Object parameter (deprecated)) | Since: HTML 3.2

Description

The parameter value.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<param value="value">
```

The same attribute on other elements

<button>, <data>, <input>, , <meter>, <option>, <progress>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <param>. Full details of the element are in Part 6.

value on <progress>

Element: <progress> (Progress indicator) | Since: HTML5

Description

How much has been completed, from 0 to max. Omit for an indeterminate bar.

Accepted values

number

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<progress value="3">...</progress>
```

The same attribute on other elements

<button>, <data>, <input>, , <meter>, <option>, <param>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <progress>. Full details of the element are in Part 6.

Attribute: valuetype

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<param>	data ref object	HTML 4.01

valuetype on <param>

Element: <param> (Object parameter (deprecated)) | Since: HTML 4.01

Description

Obsolete. How to interpret value.

Accepted values

data | ref | object

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<param valuetype="data">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <param>. Full details of the element are in Part 6.

Attribute: version

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<html>	text	HTML 2.0

version on <html>

Element: <html> (Document root) | Since: HTML 2.0

Description

Obsolete. The HTML DTD version.

Accepted values

text

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<html version="value">...</html>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <html>. Full details of the element are in Part 6.

Attribute: viewBox

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<svg>	min-x min-y width height	SVG

viewBox on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

The internal coordinate system; makes the graphic scalable.

Accepted values

min-x min-y width height

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg viewBox="min-x min-y width height">...</svg>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Attribute: vlink

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<body>	colour	HTML 3.2

vlink on <body>

Element: <body> (Document body) | Since: HTML 3.2

Description

Obsolete. Colour of visited links. Use CSS a:visited.

Accepted values

colour

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<body vlink="#336699">...</body>
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <body>. Full details of the element are in Part 6.

Attribute: volume

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<bgsound>	-10000 to 0	IE

volume on <bgsound>

Element: <bgsound> (Background sound (non-standard, obsolete)) | Since: IE

Description

Volume.

Accepted values

-10000 to 0

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<bgsound volume="-10000 to 0">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <bgsound>. Full details of the element are in Part 6.

Attribute: vspace

Accepted by 4 elements. Each use is described on the following pages.

Element	Values	Since
<applet>	pixels	HTML 3.2
	pixels	HTML 3.2
<marquee>	pixels	IE
<object>	various	HTML 4.01 / HTML5 (removed)

vspace on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Horizontal and vertical margins.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet vspace="3">...</applet>
```

The same attribute on other elements

, <marquee>, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full

details of the element are in Part 6.

**vspace on **

Element: (Image) | Since: HTML 3.2

Description

Obsolete. Horizontal and vertical margins. Use CSS margin.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img vspace="3">
```

The same attribute on other elements

<applet>, <marquee>, <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

vspace on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Margins.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee vspace="3">...</marquee>
```

The same attribute on other elements

<applet>, , <object>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

vspace on <object>

Element: <object> (External object) | Since: HTML 4.01 / HTML5 (removed)

Description

Obsolete attributes from HTML 4.01 plug-in usage or presentational attributes. Use CSS or drop them.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object vspace="various">...</object>
```

The same attribute on other elements

<applet>, , <marquee>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

Attribute: webkitdirectory

Accepted by 1 element. Each use is described on the following pages.

Element	Values	Since
<input>	various	Non-standard

webkitdirectory on <input>

Element: <input> (Input control) | Since: Non-standard

Description

Non-standard browser-specific attributes.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input webkitdirectory="various">
```

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

Attribute: width

Accepted by 21 elements. Each use is described on the following pages.

Element	Values	Since
<applet>	pixels	HTML 3.2
<canvas>	positive integer	HTML5
<col>	pixels or percentage	HTML 4.01
<colgroup>	various	HTML 4.01
<embed>	pixels	Netscape / HTML5
<fencedframe>	pixels	Experimental
<hr>	pixels or percentage	HTML 3.2
<iframe>	pixels	HTML 4.01
	pixels	HTML 3.2
<input>	pixels	HTML5
<marquee>	size	IE
<multicol>	pixels	Netscape
<object>	pixels	HTML 4.01
<pre>	integer	HTML 2.0
<source>	pixels	Living Standard
<spacer>	-	Netscape
<svg>	length	SVG
<table>	pixels or percentage	HTML 3.2
<td>	pixels or percentage	HTML 3.2
<th>	various	HTML 3.2 / 4.01

Element	Values	Since
<video>	pixels	HTML5

width on <applet>

Element: <applet> (Java applet (obsolete)) | Since: HTML 3.2

Description

Size of the applet display area.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<applet width="3">...</applet>
```

The same attribute on other elements

<canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <applet>. Full details of the element are in Part 6.

width on <canvas>

Element: <canvas> (Graphics canvas) | Since: HTML5

Description

Width of the bitmap in CSS pixels. Default 300.

Accepted values

positive integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<canvas width="3">...</canvas>
```

The same attribute on other elements

<applet>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <canvas>. Full details of the element are in Part 6.

width on <col>

Element: <col> (Table column) | Since: HTML 4.01

Description

Obsolete. Column width. Use CSS width.

Accepted values

pixels or percentage

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<col width="3">
```

The same attribute on other elements

<applet>, <canvas>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <col>. Full details of the element are in Part 6.

width on <colgroup>

Element: <colgroup> (Table column group) | Since: HTML 4.01

Description

Obsolete presentational attributes; see <col>.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<colgroup width="various">...</colgroup>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <colgroup>. Full details of the element are in Part 6.

width on <embed>

Element: <embed> (External content) | Since: Netscape / HTML5

Description

Width of the display area.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<embed width="3">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <embed>. Full details of the element are in Part 6.

width on <fencedframe>

Element: <fencedframe> (Fenced frame (experimental)) | Since: Experimental

Description

Size of the frame.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<fencedframe width="3">...</fencedframe>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <fencedframe>. Full details of the element are in Part 6.

width on <hr>

Element: <hr> (Thematic break (horizontal rule)) | Since: HTML 3.2

Description

Obsolete. Width of the rule. Use CSS width.

Accepted values

pixels or percentage

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<hr width="3">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <hr>. Full details of the element are in Part 6.

width on <iframe>

Element: <iframe> (Inline frame) | Since: HTML 4.01

Description

Width of the frame (default 300).

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<iframe width="3">...</iframe>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <iframe>. Full details of the element are in Part 6.

width on

Element: (Image) | Since: HTML 3.2

Description

Intrinsic width. Helps the browser reserve space.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<img width="3">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on . Full details of the element are in Part 6.

width on <input>

Element: <input> (Input control) | Since: HTML5

Description

Size of the image for type=image.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<input width="3">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <input>. Full details of the element are in Part 6.

width on <marquee>

Element: <marquee> (Scrolling text (obsolete)) | Since: IE

Description

Size.

Accepted values

size

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<marquee width="size">...</marquee>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <marquee>. Full details of the element are in Part 6.

width on <multicol>

Element: <multicol> (Multi-column text (non-standard, obsolete)) | Since: Netscape

Description

Column width.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<multicol width="3">...</multicol>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <multicol>. Full details of the element are in Part 6.

width on <object>

Element: <object> (External object) | Since: HTML 4.01

Description

Display size.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<object width="3">...</object>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <object>. Full details of the element are in Part 6.

width on <pre>

Element: <pre> (Pre-formatted text) | Since: HTML 2.0

Description

Obsolete. Number of characters per line.

Accepted values

integer

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<pre width="3">...</pre>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <pre>. Full details of the element are in Part 6.

width on <source>

Element: <source> (Media or image source) | Since: Living Standard

Description

Intrinsic size of the image candidate (picture only).

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<source width="3">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <spacer>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <source>. Full details of the element are in Part 6.

width on <spacer>

Element: <spacer> (Spacer (non-standard, obsolete)) | Since: Netscape

Description

For type=block.

Accepted values

-

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<spacer width="-">
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <svg>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <spacer>. Full details of the element are in Part 6.

width on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Displayed size.

Accepted values

length

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg width="length">...</svg>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <table>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

width on <table>

Element: <table> (Table) | Since: HTML 3.2

Description

Obsolete. Table width. Use CSS width.

Accepted values

pixels or percentage

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<table width="3">...</table>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <td>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <table>. Full details of the element are in Part 6.

width on <td>

Element: <td> (Table data cell) | Since: HTML 3.2

Description

Obsolete. Use CSS width.

Accepted values

pixels or percentage

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<td width="3">...</td>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <th>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <td>. Full details of the element are in Part 6.

width on <th>

Element: <th> (Table header cell) | Since: HTML 3.2 / 4.01

Description

Obsolete presentational attributes. Use CSS.

Accepted values

various

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<th width="various">...</th>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <video>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <th>. Full details of the element are in Part 6.

width on <video>

Element: <video> (Embedded video) | Since: HTML5

Description

Display size; also prevents layout shift.

Accepted values

pixels

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<video width="3">...</video>
```

The same attribute on other elements

<applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <video>. Full details of the element are in Part 6.

Attribute: wrap

Accepted by 2 elements. Each use is described on the following pages.

Element	Values	Since
<pre>	boolean	Netscape
<textarea>	soft hard off	HTML5

wrap on <pre>

Element: <pre> (Pre-formatted text) | Since: Netscape

Description

Non-standard. Wrap long lines. Use CSS white-space: pre-wrap.

Accepted values

boolean

How to write it

This is a boolean attribute: its presence switches the feature on, its absence switches it off. Write just the name (or name="" / name="wrap" in XHTML style). Never write wrap="false"; that still counts as present.

```
<pre wrap>...</pre>
```

The same attribute on other elements

<textarea>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <pre>. Full details of the element are in Part 6.

wrap on <textarea>

Element: <textarea> (Multi-line text input) | Since: HTML5

Description

soft (default): lines wrap visually but newlines are not submitted. hard: newlines are inserted at wrap points when submitting (requires cols). off: no wrapping (non-standard).

Accepted values

soft | hard | off

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<textarea wrap="soft">...</textarea>
```

The same attribute on other elements

<pre>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <textarea>. Full details of the element are in Part 6.

Attribute: xmlns

Accepted by 3 elements. Each use is described on the following pages.

| Element | Values | Since |
|---------|---|-----------|
| <html> | http://www.w3.org/1999/xhtml | XHTML 1.0 |
| <math> | http://www.w3.org/1998/Math/MathML | MathML |
| <svg> | http://www.w3.org/2000/svg | SVG |

xmlns on <html>

Element: <html> (Document root) | Since: XHTML 1.0

Description

XML namespace; required in XHTML, ignored in HTML.

Accepted values

<http://www.w3.org/1999/xhtml>

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<html xmlns="http:">...</html>
```

The same attribute on other elements

<math>, <svg>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <html>. Full details of the element are in Part 6.

xmlns on <math>

Element: <math> (MathML root) | Since: MathML

Description

Namespace; optional in HTML.

Accepted values

<http://www.w3.org/1998/Math/MathML>

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<math xmlns="http:">...</math>
```

The same attribute on other elements

<html>, <svg>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <math>. Full details of the element are in Part 6.

xmlns on <svg>

Element: <svg> (Scalable Vector Graphics root) | Since: SVG

Description

Namespace; optional in HTML, required in standalone .svg files.

Accepted values

<http://www.w3.org/2000/svg>

How to write it

Write the attribute inside the start tag, followed by an equals sign and the value in double quotes.

```
<svg xmlns="http:">...</svg>
```

The same attribute on other elements

<html>, <math>. The meaning is similar but check each entry, because accepted values can differ.

Remember

All global attributes (Part 4) and event handler attributes (Part 5) may be combined with this one on <svg>. Full details of the element are in Part 6.

Value References

Link types, meta names, autocomplete tokens, MIME types, URL schemes, colours and entities.

Link relationship types (rel)

Value	Allowed on	Meaning
alternate	link, a, area	An alternate version of the document: another language (with hreflang), another format (RSS feed, PDF), or an alternate style sheet (with stylesheet).
author	link, a, area	A page about the author of the document or article.
bookmark	a, area	A permalink to the nearest ancestor article.
canonical	link	The preferred URL of this page for search engines, avoiding duplicate content penalties.
dns-prefetch	link	Resolve the domain name of the given origin early.
expect	link	Block rendering until the element identified by href="#id" is parsed (for view transitions).
external	a, area, form	The link leads to a different site.
help	link, a, area, form	Context-sensitive help.
icon	link	The site icon (favicon). Combine with sizes and type. Also apple-touch-icon (non-standard) for iOS home screens.
license	link, a, area, form	The copyright licence covering the main content.
manifest	link	The web app manifest (JSON) for installable Progressive Web Apps.
me	link, a, area	The document represents the same person as the linked profile (IndieAuth / rel-me verification).

Value	Allowed on	Meaning
modulepreload	link	Preload a JavaScript module and its dependencies.
next / prev	link, a, area, form	The next or previous document in a sequence (pagination, chapters).
nofollow	a, area, form	The author does not endorse the linked page; search engines should not pass ranking credit.
noopener	a, area, form	Open the link without giving the new page access to window.opener. Implied by default for target=_blank in modern browsers.
noreferrer	a, area, form	Do not send the Referer header; also implies noopener.
opener	a, area, form	Explicitly allow window.opener for target=_blank.
pingback	link	The pingback server address for this document.
preconnect	link	Open a connection (DNS, TCP, TLS) to the origin early; use for fonts and APIs.
prefetch	link	Fetch a resource likely to be needed for the next navigation and store it in cache.
preload	link	Fetch a resource needed by the current page early with high priority. Requires as.
prerender	link	Deprecated; render a page in the background. Replaced by speculation rules.
privacy-policy	link, a, area	The privacy policy for the site.
search	link, a, area, form	A search interface for the site, often an OpenSearch description.

Value	Allowed on	Meaning
stylesheet	link	An external CSS style sheet. The most common link type.
tag	a, area	A tag (keyword) that applies to the current document.
terms-of-service	link, a, area	The terms of service for the site.

Standard meta names

Name	Meaning
application-name	Short name of the web application the page belongs to.
author	Name of the document's author.
description	A short summary of the page, often shown under the title in search results. Aim for 50-160 characters.
generator	Software that generated the page (for example a CMS).
keywords	Comma-separated keywords. Ignored by major search engines; historical.
referrer	Default referrer policy for the document: no-referrer, origin, strict-origin-when-cross-origin, and so on.
theme-color	Suggested colour for the browser UI (address bar on mobile). Accepts media.
color-scheme	Colour schemes the page supports: light, dark, light dark, only light.
viewport	Controls layout on mobile: width=device-width, initial-scale=1, plus optional minimum-scale, maximum-scale, user-scalable (do not disable zoom), viewport-fit=cover, interactive-widget.
robots	Indexing directives: index/noindex, follow/nofollow, noarchive, nosnippet, max-snippet:n, max-image-preview:large, noimageindex, notranslate, unavailable_after.
googlebot / bingbot	Same as robots but for one crawler.
google	Google-specific: nositelinkssearchbox, notranslate.
rating	Content rating: adult or RTA-5042-1996-1400-1577-RTA.
creator / publisher	The creator and publisher of the document.

Name	Meaning
format-detection	iOS: telephone=no stops phone numbers becoming links.
apple-mobile-web-app-capable / apple-mobile-web-app-status-bar-style / apple-mobile-web-app-title	iOS home-screen app behaviour (non-standard).
mobile-web-app-capable	Android equivalent (non-standard).
msapplication-TileColor / msapplication-config	Windows tile settings (non-standard).
twitter:card / twitter:site / twitter:title / twitter:description / twitter:image	Twitter/X card metadata.
property=og:title / og:type / og:image / og:url / og:description / og:site_name / og:locale	Open Graph metadata used by Facebook, LinkedIn, WhatsApp, Slack and others when a link is shared (uses property instead of name).
http-equiv=content-type	Legacy encoding declaration; use charset.
http-equiv=default-style	Name of the preferred style sheet set.
http-equiv=refresh	Reload after n seconds, optionally to another URL: content="5; url=...".
http-equiv=content-security-policy	Content Security Policy for the page.
http-equiv=x-ua-compatible	Legacy: content="IE=edge" for Internet Explorer document mode.

Name	Meaning
http-equiv=content-language	Obsolete; use lang on <html>.
http-equiv=set-cookie	Obsolete; not allowed.

Autocomplete tokens

Values for the autocomplete attribute on input, select and textarea. Combine a section, shipping/billing prefix, contact type and field: autocomplete="section-guest billing work tel".

Token	Meaning
off	Do not autofill (browsers may ignore for login fields).
on	Autofill allowed; browser guesses the field.
name	Full name.
honorific-prefix	Mr, Mrs, Dr...
given-name	First name.
additional-name	Middle name.
family-name	Last name.
honorific-suffix	Jr, PhD...
nickname	Nickname.
username	Account username.
new-password	A new password (browser may suggest one).
current-password	Existing password for login.
one-time-code	SMS/email verification code.
organization-title	Job title.
organization	Company name.
street-address	Full street address (multi-line).
address-line1 / address-line2 / address-line3	Address lines.
address-level1	Province / state / governorate.
address-level2	City.

Token	Meaning
address-level3 / address-level4	Finer divisions.
country	Country code.
country-name	Country name.
postal-code	Postal / ZIP code.
cc-name	Cardholder name.
cc-given-name / cc-additional-name / cc-family-name	Cardholder name parts.
cc-number	Card number.
cc-exp	Expiry MM/YY.
cc-exp-month / cc-exp-year	Expiry parts.
cc-csc	Security code.
cc-type	Card type.
transaction-currency	Currency code.
transaction-amount	Amount.
language	Preferred language.
bday	Birthday.
bday-day / bday-month / bday-year	Birthday parts.
sex	Gender identity as free text.
url	Home page.
photo	Photo URL.
tel	Full telephone number.

Token	Meaning
tel-country-code / tel-national / tel-area-code / tel-local / tel-extension	Telephone parts.
email	Email address.
impp	Instant messaging URL.
Prefixes: shipping / billing	shipping street-address, billing postal-code...
Prefixes: home / work / mobile / fax / pager	home tel, work email...
section-*	Groups fields into a named section: section-guest1 name.
webauthn	Allow passkey autofill (with username or current-password).

Common MIME types

MIME type	Extensions	Use
text/html	.html .htm	HTML documents
text/css	.css	Style sheets
text/javascript	.js .mjs	JavaScript (application/javascript is legacy)
application/json	.json	JSON data
application/ld+json	.jsonld	JSON-LD structured data
application/xml, text/xml	.xml	XML
image/svg+xml	.svg	SVG images
text/plain	.txt	Plain text
text/csv	.csv	Comma-separated values
text/vtt	.vtt	WebVTT subtitles
text/markdown	.md	Markdown
image/jpeg	.jpg .jpeg	Photos
image/png	.png	Lossless images with transparency
image/gif	.gif	Simple animations
image/webp	.webp	Modern lossy/lossless
image/avif	.avif	Newest, smallest
image/apng	.apng	Animated PNG
image/x-icon, image/vnd.microsoft.icon	.ico	Favicons
image/bmp	.bmp	Bitmap
image/tiff	.tif	TIFF (not displayed by browsers)

MIME type	Extensions	Use
audio/mpeg	.mp3	MP3 audio
audio/ogg	.ogg .oga .opus	Ogg Vorbis / Opus
audio/wav	.wav	Uncompressed audio
audio/aac	.aac	AAC
audio/flac	.flac	Lossless
audio/webm	.weba	WebM audio
audio/midi	.mid	MIDI
video/mp4	.mp4 .m4v	H.264/H.265 video
video/webm	.webm	VP8/VP9/AV1 video
video/ogg	.ogv	Theora
video/quicktime	.mov	QuickTime
video/x-msvideo	.avi	AVI
application/x-mpegURL	.m3u8	HLS streaming playlist
font/woff2	.woff2	Web fonts (best)
font/woff	.woff	Web fonts
font/ttf	.ttf	TrueType
font/otf	.otf	OpenType
application/pdf	.pdf	PDF
application/zip	.zip	ZIP
application/gzip	.gz	Gzip
application/octet-stream	any	Generic binary; forces download
application/msword	.doc	Word 97-2003

MIME type	Extensions	Use
application/vnd. openxmlformats -officedocument. wordprocessing ml.document	.docx	Word
application/vnd. ms-excel	.xls	Excel 97-2003
application/vnd. openxmlformats -officedocument. spreadsheetml.s heet	.xlsx	Excel
application/vnd. ms-powerpoint	.ppt	PowerPoint
application/vnd. openxmlformats -officedocument. presentationml. presentation	.pptx	PowerPoint
application/x-ww w-form-urlencoded	-	Default form encoding
multipart/form-d ata	-	Form encoding for file uploads
text/event-strea m	-	Server-sent events
application/was m	.wasm	WebAssembly
application/mani fest+json	.webmanifest	Web app manifest
application/rss+ xml / application /atom+xml	.rss .atom	Feeds

URL schemes used in href

Scheme	Meaning
https:	Secure web page. Always use for links and resources.
http:	Insecure web page.
mailto:	Opens the mail client: mailto:a@b.com?subject=Hi&body=Text&cc=c@d.com
tel:	Dials a telephone number: tel:+201001234567
sms:	Opens SMS: sms:+2010...?body=Hello
ftp:	File transfer (no longer supported by browsers).
file:	A local file (only works locally).
data:	Inline data: data:image/png;base64,.... or data:text/plain,Hello
blob:	An object URL created with URL.createObjectURL().
javascript:	Runs script when followed. Avoid; use a button.
about:	Browser pages: about:blank.
geo:	Geographic coordinates: geo:30.0131,31.2089
whatsapp: / tg: / skype:	App links (non-standard).
web+name:	Custom schemes registered with registerProtocolHandler.
#fragment	Not a scheme: a link to an id in the current page. #top and empty # scroll to the top.

Named colours

The 148 colour names recognised by CSS and by the legacy HTML color attributes, with their hexadecimal values.

Name	Hex	Name	Hex
aliceblue	#f0f8ff	antiquewhite	#faebd7
aqua	#00ffff	aquamarine	#7fffd4
azure	#f0ffff	beige	#f5f5dc
bisque	#ffe4c4	black	#000000
blanchedalmond	#ffebcd	blue	#0000ff
blueviolet	#8a2be2	brown	#a52a2a
burlywood	#deb887	cadetblue	#5f9ea0
chartreuse	#7fff00	chocolate	#d2691e
coral	#ff7f50	cornflowerblue	#6495ed
cornsilk	#fff8dc	crimson	#dc143c
cyan	#00ffff	darkblue	#00008b
darkcyan	#008b8b	darkgoldenrod	#b8860b
darkgray	#a9a9a9	darkgreen	#006400
darkkhaki	#bdb76b	darkmagenta	#8b008b
darkolivegreen	#556b2f	darkorange	#ff8c00
darkorchid	#9932cc	darkred	#8b0000
darksalmon	#e9967a	darkseagreen	#8fbc8f
darkslateblue	#483d8b	darkslategray	#2f4f4f
darkturquoise	#00ced1	darkviolet	#9400d3
deeppink	#ff1493	deepskyblue	#00bfff
dimgray	#696969	dodgerblue	#1e90ff
firebrick	#b22222	floralwhite	#fffaf0

Name	Hex	Name	Hex
forestgreen	#228b22	fuchsia	#ff00ff
gainsboro	#dcdcdc	ghostwhite	#f8f8ff
gold	#ffd700	goldenrod	#daa520
gray	#808080	green	#008000
greenyellow	#adff2f	honeydew	#f0fff0
hotpink	#ff69b4	indianred	#cd5c5c
indigo	#4b0082	ivory	#fffff0
khaki	#f0e68c	lavender	#e6e6fa
lavenderblush	#fff0f5	lawngreen	#7cfc00
lemonchiffon	#fffacd	lightblue	#add8e6
lightcoral	#f08080	lightcyan	#e0ffff
lightgoldenrodyellow	#fafad2	lightgray	#d3d3d3
lightgreen	#90ee90	lightpink	#ffb6c1
lightsalmon	#ffa07a	lightseagreen	#20b2aa
lightskyblue	#87cefa	lightslategray	#778899
lightsteelblue	#b0c4de	lightyellow	#ffffe0
lime	#00ff00	limegreen	#32cd32
linen	#faf0e6	magenta	#ff00ff
maroon	#800000	mediumaquamarine	#66cdaa
mediumblue	#0000cd	mediumorchid	#ba55d3
mediumpurple	#9370db	mediumseagreen	#3cb371
mediumslateblue	#7b68ee	mediumspringgreen	#00fa9a
mediumturquoise	#48d1cc	mediumvioletred	#c71585
midnightblue	#191970	mintcream	#f5fffa
mistyrose	#ffe4e1	moccasin	#ffe4b5

Name	Hex	Name	Hex
navajowhite	#ffdead	navy	#000080
oldlace	#fdf5e6	olive	#808000
olivedrab	#6b8e23	orange	#ffa500
orangered	#ff4500	orchid	#da70d6
palegoldenrod	#eee8aa	palegreen	#98fb98
paleturquoise	#afeeee	palevioletred	#db7093
papayawhip	#ffefd5	peachpuff	#ffdab9
peru	#cd853f	pink	#ffc0cb
plum	#dda0dd	powderblue	#b0e0e6
purple	#800080	rebeccapurple	#663399
red	#ff0000	rosybrown	#bc8f8f
royalblue	#4169e1	saddlebrown	#8b4513
salmon	#fa8072	sandybrown	#f4a460
seagreen	#2e8b57	seashell	#fff5ee
sienna	#a0522d	silver	#c0c0c0
skyblue	#87ceeb	slateblue	#6a5acd
slategray	#708090	snow	#ffaafa
springgreen	#00ff7f	steelblue	#4682b4
tan	#d2b48c	teal	#008080
thistle	#d8bfd8	tomato	#ff6347
turquoise	#40e0d0	violet	#ee82ee
wheat	#f5deb3	white	#ffffff
whitesmoke	#f5f5f5	yellow	#ffff00
yellowgreen	#9acd32		

Character entity reference

Named and numeric character references. Numeric references also work in hexadecimal: `€`; equals `€`;. With UTF-8 you can type most of these directly; only `<`, `>` and `&` are essential.

Named	Numeric	Character
<code>&amp;</code>	<code>&#38;</code>	& ampersand
<code>&lt;</code>	<code>&#60;</code>	< less-than
<code>&gt;</code>	<code>&#62;</code>	> greater-than
<code>&quot;</code>	<code>&#34;</code>	" double quote
<code>&apos;</code>	<code>&#39;</code>	' apostrophe
<code>&nbsp;</code>	<code>&#160;</code>	non-breaking space
<code>&ensp;</code>	<code>&#8194;</code>	en space
<code>&emsp;</code>	<code>&#8195;</code>	em space
<code>&thinsp;</code>	<code>&#8201;</code>	thin space
<code>&zwnj;</code>	<code>&#8204;</code>	zero-width non-joiner
<code>&zwj;</code>	<code>&#8205;</code>	zero-width joiner
<code>&shy;</code>	<code>&#173;</code>	soft hyphen
<code>&lrm;</code>	<code>&#8206;</code>	left-to-right mark
<code>&rlm;</code>	<code>&#8207;</code>	right-to-left mark
<code>&copy;</code>	<code>&#169;</code>	© copyright
<code>&reg;</code>	<code>&#174;</code>	® registered
<code>&trade;</code>	<code>&#8482;</code>	™ trademark
<code>&sect;</code>	<code>&#167;</code>	§ section
<code>&para;</code>	<code>&#182;</code>	¶ pilcrow
<code>&deg;</code>	<code>&#176;</code>	° degree

Named	Numeric	Character
µ	µ	μ micro
·	·	· middle dot
•	•	• bullet
…	…	... ellipsis
†	†	† dagger
‡	‡	‡ double dagger
‰	‰	‰ per mille
′	′	′ prime
″	″	″ double prime
–	–	– en dash
—	—	— em dash
‘	‘	‘ left single quote
’	’	’ right single quote
‚	‚	, single low quote
“	“	“ left double quote
”	”	” right double quote
„	„	„ double low quote
«	«	« left guillemet
»	»	» right guillemet
‹	‹	< single left guillemet
›	›	> single right guillemet
¡	¡	¡ inverted exclamation
¿	¿	¿ inverted question
¢	¢	¢ cent
£	£	£ pound

Named	Numeric	Character
¤t;	¤	¤ currency
¥	¥	¥ yen
€	€	€ euro
₺	₺	₺ lira
₹	₹	₹ rupee
₱	₱	₱ peso
±	±	± plus-minus
×	×	× multiplication
÷	÷	÷ division
−	−	– minus
½	½	½ one half
¼	¼	¼ one quarter
¾	¾	¾ three quarters
¹	¹	¹ superscript 1
²	²	² superscript 2
³	³	³ superscript 3
≠	≠	≠ not equal
≤	≤	≤ less or equal
≥	≥	≥ greater or equal
≈	≈	≈ approximately
≡	≡	≡ identical
∞	∞	∞ infinity
√	√	√ square root
∑	∑	Σ sum
∏	∏	∏ product

Named	Numeric	Character
∫	∫	∫ integral
∂	∂	∂ partial
∇	∇	∇ nabra
∈	∈	∈ element of
∉	∉	∉ not element of
⊂	⊂	⊂ subset
⊃	⊃	⊃ superset
∩	∩	∩ intersection
∪	∪	∪ union
∧	∧	∧ and
∨	∨	∨ or
¬	¬	¬ not
∀	∀	∀ for all
∃	∃	∃ exists
∅	∅	∅ empty set
⊥	⊥	⊥ perpendicular
∠	∠	∠ angle
∴	∴	∴ therefore
∝	∝	∝ proportional
ƒ	ƒ	ƒ function
∗	∗	* asterisk operator
⋅	⋅	· dot operator
⊕	⊕	⊕ circled plus
⊗	⊗	⊗ circled times
←	←	← left arrow

Named	Numeric	Character
↑	↑	↑ up arrow
→	→	→ right arrow
↓	↓	↓ down arrow
↔	↔	↔ left-right arrow
↵	↵	↵ carriage return
⇐	⇐	⇐ double left arrow
⇑	⇑	⇑ double up arrow
⇒	⇒	⇒ double right arrow
⇓	⇓	⇓ double down arrow
⇔	⇔	⇔ double left-right arrow
♠	♠	♠ spade
♣	♣	♣ club
♥	♥	♥ heart
♦	♦	♦ diamond
◊	◊	◇ lozenge
✓	✓	✓ check mark
✗	✗	✕ ballot x
★	★	★ black star
☆	☆	☆ white star
▲	▲	▲ up triangle
▼	▼	▼ down triangle
●	●	● black circle
■	■	■ black square
Α α	Α α	A α alpha
Β β	Β β	B β beta

Named	Numeric	Character
Γ γ	Γ γ	Γ γ gamma
Δ δ	Δ δ	Δ δ delta
Ε ε	Ε ε	Ε ε epsilon
Ζ ζ	Ζ ζ	Ζ ζ zeta
Η η	Η η	Η η eta
Θ θ	Θ θ	Θ θ theta
Ι ι	Ι ι	Ι ι iota
Κ κ	Κ κ	Κ κ kappa
Λ λ	Λ λ	Λ λ lambda
Μ μ	Μ μ	Μ μ mu
Ν ν	Ν ν	Ν ν nu
Ξ ξ	Ξ ξ	Ξ ξ xi
Ο ο	Ο ο	Ο ο omicron
Π π	Π π	Π π pi
Ρ ρ	Ρ ρ	Ρ ρ rho
Σ σ ς	Σ σ ς	Σ σ ς sigma
Τ τ	Τ τ	Τ τ tau
Υ υ	Υ υ	Υ υ upsilon
Φ φ	Φ φ	Φ φ phi
Χ χ	Χ χ	Χ χ chi
Ψ ψ	Ψ ψ	Ψ ψ psi
Ω ω	Ω ω	Ω ω omega
À à	À à	À à
Á á	Á á	Á á
Â â	Â â	Â â

Named	Numeric	Character
Ã ã	Ã ã	Ã ã
Ä ä	Ä ä	Ä ä
Å å	Å å	Å å
Æ æ	Æ æ	Æ æ
Ç ç	Ç ç	Ç ç
È è	È è	È è
É é	É é	É é
Ê ê	Ê ê	Ê ê
Ë ë	Ë ë	Ë ë
Ì ì	Ì ì	Ì ì
Í í	Í í	Í í
Î î	Î î	Î î
Ï ï	Ï ï	Ï ï
Ð ð	Ð ð	Ð ð
Ñ ñ	Ñ ñ	Ñ ñ
Ò ò	Ò ò	Ò ò
Ó ó	Ó ó	Ó ó
Ô ô	Ô ô	Ô ô
Õ õ	Õ õ	Õ õ
Ö ö	Ö ö	Ö ö
Ø ø	Ø ø	Ø ø
Œ œ	Œ œ	Œ œ
Š š	Š š	Š š
Ù ù	Ù ù	Ù ù
Ú ú	Ú ú	Ú ú

Named	Numeric	Character
Û û	Û û	Û û
Ü ü	Ü ü	Ü ü
Ý ý	Ý ý	Ý ý
Ÿ ÿ	Ÿ ÿ	Ÿ ŷ
Þ þ	Þ þ	Þ þ
ß	ß	ß sharp s

CSS Reference

Fundamentals, selectors, properties and a cookbook for styling HTML elements.

CSS fundamentals

CSS (Cascading Style Sheets) is the language that describes how HTML elements look: colours, fonts, spacing, size, position and layout. HTML says what something is; CSS says how it appears. The two are kept separate so that the same HTML can be shown in many ways (desktop, phone, print, dark mode) by changing only the CSS.

Three ways to add CSS

Method	How	When to use
External style sheet	<code><link rel="stylesheet" href="style.css"></code> in <code><head></code> ; rules live in <code>style.css</code>	Always, for real projects. One file styles every page and is cached by the browser.
Internal style sheet	<code><style> ... </style></code> in <code><head></code>	Small single-page demos, emails, or page-specific overrides.
Inline style	<code>style="color: red"</code> attribute on an element	Quick tests and values set by JavaScript. Avoid for general styling.

```
/* style.css */
body { font-family: Arial, sans-serif; margin: 0; }
footer { background: #222; color: #fff; text-align: center; padding: 20px; }
```

Anatomy of a rule

```
selector {
  property: value; /* one declaration */
  property: value;
}

footer {
  color: gray;
```

```
text-align: center;
}
```

- The selector chooses which elements the rule applies to.
- Each declaration is a property name, a colon, a value and a semicolon.
- Declarations are grouped in curly braces to form a rule (or rule set).
- Comments are written `/* like this */` and may span lines.

The cascade: which rule wins

When several rules apply to the same element and property, the browser decides in this order:

- Origin and importance: author styles beat browser defaults; !important declarations beat normal ones (avoid !important; it makes styles hard to override).
- Specificity: inline style (1,0,0,0) beats id selectors (0,1,0,0), which beat class/attribute/pseudo-class selectors (0,0,1,0), which beat element/pseudo-element selectors (0,0,0,1). Count each part; the higher total wins.
- Source order: if specificity is equal, the rule that appears later wins.
- Layers (@layer) and scope (@scope) add further control in modern CSS.

```
p { color: black; }           /* specificity 0,0,0,1 */
.note { color: blue; }       /* 0,0,1,0 - wins over p */
#intro { color: green; }     /* 0,1,0,0 - wins over .note */
<p id="intro" class="note" style="color: red"> /* inline wins: red */
```

Inheritance

Some properties are inherited by child elements automatically: color, font-family, font-size, line-height, text-align, visibility, and most text properties. Others are not: margin, padding, border, background, width, height, display, position. Any property can be forced with the keywords inherit, initial, unset and revert.

Units

Unit	Meaning	Typical use
px	CSS pixel (1/96 inch)	Borders, small fixed sizes, media query breakpoints
em	Relative to the element's font size	Padding and margin that scale with text
rem	Relative to the root (html) font size, 16px by default	Font sizes, spacing - predictable and respects user settings
%	Relative to the parent's corresponding dimension	Widths in fluid layouts
vw / vh	1% of the viewport width / height	Full-screen sections, fluid type
vmin / vmax / dvh / svh / lvh	Viewport-relative variants	Mobile-safe full-height layouts (dvh)
ch	Width of the '0' character	Line length: max-width: 65ch
fr	Fraction of free space (grid only)	grid-template-columns: 1fr 2fr
deg, rad, turn	Angles	Rotations, gradients

Unit	Meaning	Typical use
s, ms	Time	Transitions and animations
pt, cm, mm, in	Physical units	Print style sheets

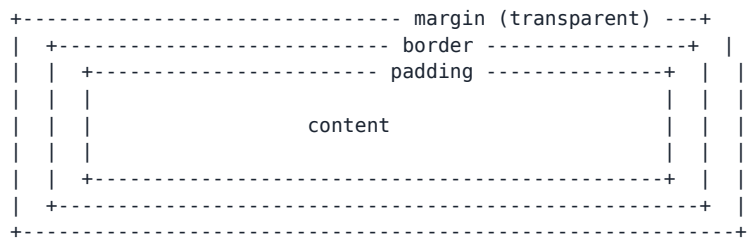
Colours

Notation	Example	Notes
Named colour	red, steelblue, transparent, currentColor	148 names; see Part 9
Hex	#e60000, #e60000cc (with alpha), #f00 (short)	Most common
rgb()	rgb(230 0 0), rgb(230 0 0 / 50%)	Modern space-separated syntax; commas still work
hsl()	hsl(0 100% 45%)	Hue 0-360, saturation, lightness; easy to adjust
oklch() / oklab()	oklch(60% 0.2 30)	Perceptually uniform; wide-gamut colours
color-mix()	color-mix(in oklab, red 30%, white)	Mixing colours
System colours	Canvas, CanvasText, LinkText, ButtonFace	Follow the operating system theme

The box model

Every element is a rectangular box made of four layers, from inside out: content, padding, border and margin. Width and height set the content box by default; with box-sizing:

border-box they include padding and border, which is far easier to reason about. Most projects begin with the reset `*`, `*::before`, `*::after` { `box-sizing: border-box;` }.



Vertical margins of adjacent blocks collapse into a single margin equal to the larger of the two. Padding and borders never collapse. Backgrounds cover the content and padding (and, by default, extend under the border).

Display and layout modes

display value	Behaviour
block	Starts on a new line, fills the available width, respects width/height. <code>div</code> , <code>p</code> , <code>h1</code> , <code>section</code> , <code>footer</code> ...
inline	Flows within text, only as wide as its content, ignores width/height and vertical margins. <code>span</code> , <code>a</code> , <code>em</code> , <code>strong</code> ...
inline-block	Flows within text but respects width, height and vertical margins/padding. <code>img</code> , <code>button</code> , <code>input</code> .
none	Removed from rendering entirely (also from the accessibility tree).
flex	The element becomes a flex container; children are laid out in a row or column with powerful alignment and distribution.
grid	The element becomes a grid container; children are placed in rows and columns defined by <code>grid-template-*</code> .

display value	Behaviour
inline-flex / inline-grid	As flex/grid, but the container itself flows inline.
table, table-row, table-cell...	Table layout without table elements.
list-item	Generates a bullet or number marker (li).
contents	The element's box disappears but its children remain.
flow-root	A block that contains its floats (clears them) and starts a new formatting context.

Positioning

position value	Behaviour
static	Default. Normal flow; top/left/etc. have no effect.
relative	Normal flow, then shifted by top/right/bottom/left; the original space is kept. Also creates a containing block for absolute children.
absolute	Removed from the flow and placed relative to the nearest positioned ancestor (or the initial containing block).
fixed	Removed from the flow and placed relative to the viewport; stays put when scrolling.
sticky	Behaves as relative until it reaches the threshold (top: 0), then sticks like fixed within its parent.

Flexbox in one page

```
.row {
  display: flex;           /* children become flex items in a row
*/
  flex-direction: row;     /* or column, row-reverse, column-
reverse */
  flex-wrap: wrap;        /* allow items to wrap onto new lines */
}
```

```

    justify-content: space-between; /* main-axis distribution: flex-
start | center | flex-end | space-around | space-evenly */
    align-items: center;           /* cross-axis alignment: stretch | flex-
start | center | flex-end | baseline */
    gap: 16px;                    /* space between items */
}
.row > .item {
    flex: 1;                      /* shorthand for flex-grow flex-shrink
flex-basis: 1 1 0 */
    order: 0;                     /* visual order */
    align-self: flex-end;         /* override align-items for one item */
}

```

Grid in one page

```

.layout {
    display: grid;
    grid-template-columns: 200px 1fr;           /* two columns */
    grid-template-rows: auto 1fr auto;         /* header, content,
footer */
    grid-template-areas:
        "header header"
        "nav    main"
        "footer footer";
    gap: 16px;
    min-height: 100vh;
}
header { grid-area: header; }
nav    { grid-area: nav; }
main   { grid-area: main; }
footer { grid-area: footer; }

/* Responsive cards without media queries */
.cards { display: grid; grid-template-columns: repeat(auto-fit,
minmax(240px, 1fr)); gap: 24px; }

```

Responsive design

```

/* Mobile first: base styles for small screens, then enhance */
.container { padding: 16px; }

@media (min-width: 600px) {
    .container { padding: 32px; }
}
@media (min-width: 1024px) {
    .container { max-width: 1180px; margin: 0 auto; }
}

/* Container queries: respond to the parent's size */

```



```
.card-wrapper { container-type: inline-size; }
@container (min-width: 400px) {
  .card { display: flex; }
}

/* User preferences */
@media (prefers-color-scheme: dark) { body { background: #111; color: #eee; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }

/* Print */
@media print { nav, footer, .no-print { display: none; } }
```

Custom properties (CSS variables)

```
:root {
  --brand: #e60000;
  --radius: 12px;
  --font: "Segoe UI", Tahoma, Arial, sans-serif;
}
.btn {
  background: var(--brand);
  border-radius: var(--radius);
  font-family: var(--font);
}
.btn.secondary { --brand: #555; } /* override locally */
```

Transitions and animations

```
.btn { transition: background-color 0.2s ease, transform 0.2s ease; }
.btn:hover { background-color: #b90000; transform: translateY(-2px); }

@keyframes fade-in {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: none; }
}
.card { animation: fade-in 0.4s ease-out both; }
```

CSS selectors

Selector	Meaning
*	Universal selector: every element.
element	Type selector: all elements with that tag name.
.class	Class selector: elements with that class.
#id	ID selector: the one element with that id.
[attr]	Attribute present.
[attr="value"]	Attribute equals value.
[attr~="word"]	Attribute contains the word in a space-separated list.
[attr ="en"]	Attribute equals en or starts with en-.
[attr^="https"]	Attribute starts with.
[attr\$=".pdf"]	Attribute ends with.
[attr*="example"]	Attribute contains substring.
[attr="v" i]	Case-insensitive match.
A B	Descendant combinator: B anywhere inside A.
A > B	Child combinator: B directly inside A.
A + B	Adjacent sibling: B immediately after A.
A ~ B	General sibling: any B after A with the same parent.
A, B	Selector list: either.
:hover	Pointer is over the element.
:focus / :focus-visible / :focus-within	Element has focus / focus shown by keyboard / a descendant has focus.
:active	Being clicked.
:visited / :link	Visited / unvisited link.
:first-child / :last-child / :only-child	Position among siblings.

Selector	Meaning
:nth-child(an+b) / :nth-last-child()	Position formula: $2n$ (even), $2n+1$ (odd), 3, $-n+3$ (first three).
:first-of-type / :last-of-type / :nth-of-type()	Position among siblings of the same type.
:not(selector)	Negation.
:is(a, b) / :where(a, b)	Matches any of the list; :where has zero specificity.
:has(selector)	Parent selector: element that contains a match.
:empty	No children (not even text).
:root	The <html> element.
:checked / :indeterminate	Checked checkbox/radio/option.
:disabled / :enabled / :read-only / :read-write	Form control state.
:required / :optional	Form control requirement.
:valid / :invalid / :user-invalid / :in-range / :out-of-range / :placeholder-shown	Validation state.
:target	Element whose id matches the URL fragment.
:lang(ar)	Element language.
:dir rtl)	Text direction.
:popover-open / :modal / :fullscreen / :open	Top-layer and open states.
::before / ::after	Generated content before/after the element's content (needs content property).

Selector	Meaning
::first-line / ::first-letter	First line / first letter of a block.
::marker	List bullet or number.
::placeholder	Placeholder text in inputs.
::selection	Text selected by the user.
::backdrop	The backdrop behind a modal dialog, popover or fullscreen element.
::part(name) / ::slotted(sel)	Shadow DOM parts and slotted content.
::file-selector-button	The button in a file input.
::cue	WebVTT captions in video.

Selector: *

What it selects

Universal selector: every element.

Example

```
* { box-sizing: border-box; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }.`
- Test any selector in the browser console with `document.querySelectorAll('*').length.`

Selector: element

What it selects

Type selector: all elements with that tag name.

Example

```
p { line-height: 1.6; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('element').length`.

Selector: .class

What it selects

Class selector: elements with that class.

Example

```
.card { border: 1px solid #ccc; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('.class').length`.

Selector: #id

What it selects

ID selector: the one element with that id.

Example

```
#main { max-width: 960px; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('#id').length`.

Selector: [attr]

What it selects

Attribute present.

Example

```
input[required] { border-left: 3px solid red; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('[attr]').length`.

Selector: [attr="value"]

What it selects

Attribute equals value.

Example

```
input[type="checkbox"] { accent-color: green; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('[attr="value"]').length`.

Selector: `[attr~="word"]`

What it selects

Attribute contains the word in a space-separated list.

Example

```
[class~="featured"] { ... }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('[attr~="word"]').length`.

Selector: [attr|="en"]

What it selects

Attribute equals en or starts with en-.

Example

```
[lang|="en"] { quotes: '"" '""'; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('[attr|="en"]').length`.

Selector: [attr^="https"]

What it selects

Attribute starts with.

Example

```
a[href^="https"]::after { content: ' 🔒'; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('[attr^="https"]').length`.

Selector: [attr\$=".pdf"]

What it selects

Attribute ends with.

Example

```
a[href$=".pdf"] { background: url(pdf.svg) no-repeat left; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('[attr$=".pdf"]').length`.

Selector: [attr*="example"]

What it selects

Attribute contains substring.

Example

```
a[href*="example.com"] { color: green; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('[attr*="example"]').length`.

Selector: `[attr="v" i]`

What it selects

Case-insensitive match.

Example

```
[type="TEXT" i] { ... }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('[attr="v" i]').length`.

Selector: A B

What it selects

Descendant combinator: B anywhere inside A.

Example

```
nav a { text-decoration: none; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('A B').length`.

Selector: A > B

What it selects

Child combinator: B directly inside A.

Example

```
ul > li { list-style: square; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('A > B').length`.

Selector: A + B

What it selects

Adjacent sibling: B immediately after A.

Example

```
h2 + p { margin-top: 0; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('A + B').length`.

Selector: A ~ B

What it selects

General sibling: any B after A with the same parent.

Example

```
h2 ~ p { color: #444; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('A ~ B').length`.

Selector: A, B

What it selects

Selector list: either.

Example

```
h1, h2, h3 { font-family: Georgia, serif; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('A, B').length`.

Selector: :hover

What it selects

Pointer is over the element.

Example

```
a:hover { text-decoration: underline; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':hover').length`.

Selector: :focus / :focus-visible / :focus-within

What it selects

Element has focus / focus shown by keyboard / a descendant has focus.

Example

```
button:focus-visible { outline: 3px solid #1b6fd8; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':focus / :focus-visible / :focus-within').length`.

Selector: :active

What it selects

Being clicked.

Example

```
button:active { transform: scale(0.98); }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':active').length`.

Selector: :visited / :link

What it selects

Visited / unvisited link.

Example

```
a:visited { color: purple; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':visited / :link').length`.

Selector: :first-child / :last-child / :only-child

What it selects

Position among siblings.

Example

```
li:first-child { font-weight: bold; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':first-child / :last-child / :only-child').length`.

Selector: **:nth-child(an+b)** / **:nth-last-child()**

What it selects

Position formula: $2n$ (even), $2n+1$ (odd), 3, $-n+3$ (first three).

Example

```
tr:nth-child(even) { background: #f7f7f7; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':nth-child(an+b) / :nth-last-child()').length`.

Selector: :first-of-type / :last-of-type / :nth-of-type()

What it selects

Position among siblings of the same type.

Example

```
p:first-of-type { font-size: 1.1em; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':first-of-type / :last-of-type / :nth-of-type()').length`.

Selector: :not(selector)

What it selects

Negation.

Example

```
input:not([type="submit"]) { width: 100%; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':not(selector)').length`.

Selector: **:is(a, b) / :where(a, b)**

What it selects

Matches any of the list; :where has zero specificity.

Example

```
:is(h1, h2, h3) a { color: inherit; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':is(a, b) / :where(a, b)').length`.

Selector: :has(selector)

What it selects

Parent selector: element that contains a match.

Example

```
article:has(img) { display: grid; }  
label:has(input:invalid) { color: red; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':has(selector)').length`.

Selector: :empty

What it selects

No children (not even text).

Example

```
p:empty { display: none; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':empty').length`.

Selector: :root

What it selects

The <html> element.

Example

```
:root { --gap: 16px; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':root').length`.

Selector: :checked / :indeterminate

What it selects

Checked checkbox/radio/option.

Example

```
input:checked + label { font-weight: bold; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':checked / :indeterminate').length`.

Selector: :disabled / :enabled / :read-only / :read-write

What it selects

Form control state.

Example

```
button:disabled { opacity: 0.5; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':disabled / :enabled / :read-only / :read-write').length`.

Selector: :required / :optional

What it selects

Form control requirement.

Example

```
input:required { border-color: #999; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':required / :optional').length`.

Selector: **:valid / :invalid / :user-invalid / :in-range / :out-of-range / :placeholder-shown**

What it selects

Validation state.

Example

```
input:user-invalid { border-color: red; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll(':valid / :invalid / :user-invalid / :in-range / :out-of-range / :placeholder-shown').length`.

Selector: :target

What it selects

Element whose id matches the URL fragment.

Example

```
:target { background: #fff9c4; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':target').length`.

Selector: :lang(ar)

What it selects

Element language.

Example

```
:lang(ar) { font-family: Cairo, sans-serif; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':lang(ar')).length`.

Selector: :dir(rtl)

What it selects

Text direction.

Example

```
:dir(rtl) .icon { transform: scaleX(-1); }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll(':dir(rtl)').length`.

Selector: **:popover-open / :modal / :fullscreen / :open**

What it selects

Top-layer and open states.

Example

```
dialog:modal { border-radius: 12px; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }.`
- Test any selector in the browser console with `document.querySelectorAll(':popover-open / :modal / :fullscreen / :open').length.`

Selector: **::before** / **::after**

What it selects

Generated content before/after the element's content (needs content property).

Example

```
q::before { content: open-quote; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::before / ::after').length`.

Selector: **::first-line / ::first-letter**

What it selects

First line / first letter of a block.

Example

```
p::first-letter { font-size: 2em; float: left; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }.`
- Test any selector in the browser console with `document.querySelectorAll('::first-line / ::first-letter').length.`

Selector: **::marker**

What it selects

List bullet or number.

Example

```
li::marker { color: red; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::marker').length`.

Selector: ::placeholder

What it selects

Placeholder text in inputs.

Example

```
input::placeholder { color: #999; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::placeholder').length`.

Selector: ::selection

What it selects

Text selected by the user.

Example

```
::selection { background: #ffd166; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: h1, h2, h3 { ... }.
- Test any selector in the browser console with `document.querySelectorAll('::selection').length`.

Selector: **::backdrop**

What it selects

The backdrop behind a modal dialog, popover or fullscreen element.

Example

```
dialog::backdrop { background: rgb(0 0 0 / 0.6); }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::backdrop').length`.

Selector: **::part(name) / ::slotted(sel)**

What it selects

Shadow DOM parts and slotted content.

Example

```
my-el::part(label) { color: red; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::part(name) / ::slotted(sel)').length`.

Selector: ::file-selector-button

What it selects

The button in a file input.

Example

```
input::file-selector-button { padding: 8px; }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::file-selector-button').length`.

Selector: **::cue**

What it selects

WebVTT captions in video.

Example

```
video::cue { background: rgb(0 0 0 / 0.8); }
```

Tips

- Selectors are case-sensitive for class names and IDs, but HTML element names are matched case-insensitively.
- Combine selectors with a comma to apply one rule to several targets: `h1, h2, h3 { ... }`.
- Test any selector in the browser console with `document.querySelectorAll('::cue').length`.

CSS property reference

Properties are grouped by purpose. Each entry gives the accepted values, an explanation and an example.

Fonts and text

font-family

Group: Fonts and text

Values

family names and generic family

Description

The typeface used for text. Give a comma-separated list, most preferred first, ending with a generic family (serif, sans-serif, monospace, cursive, fantasy, system-ui, ui-monospace, math, emoji). The browser uses the first font that is installed or loaded with @font-face. Quote names with spaces. Inherited.

Example

```
body { font-family: "Helvetica Neue", Arial, sans-serif; }
code { font-family: ui-monospace, Consolas, monospace; }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

font-size

Group: Fonts and text

Values

length | percentage | keyword

Description

The size of the text. Use rem for predictable sizes that respect user preferences (1rem = 16px by default), em for sizes relative to the parent, and clamp() for fluid type. Keywords: xx-small to xxx-large, smaller, larger.

Example

```
html { font-size: 100%; }  
h1 { font-size: 2rem; }  
p { font-size: clamp(1rem, 0.9rem + 0.5vw, 1.25rem); }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

font-weight

Group: Fonts and text

Values

normal | bold | bolder | lighter | 100-900

Description

Thickness of the strokes. 400 = normal, 700 = bold. Variable fonts support any value in between.

Example

```
strong { font-weight: 700; }  
h2 { font-weight: 600; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

font-style

Group: Fonts and text

Values

normal | italic | oblique

Description

Italic or slanted text. `<em>`, `<i>`, `<cite>`, `<var>`, `<dfn>`, `<address>` are italic by default.

Example

```
em { font-style: italic; }  
.no-italic { font-style: normal; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

font

Group: Fonts and text

Values

shorthand

Description

Shorthand for font-style, font-variant, font-weight, font-size/line-height and font-family in that order. Size and family are required.

Example

```
body { font: 16px/1.6 Arial, sans-serif; }  
h1 { font: bold 2rem Georgia, serif; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

font-variant / font-variant-numeric / font-feature-settings

Group: Fonts and text

Values

various

Description

Typographic features: small caps, tabular numbers (align digits in tables), ligatures, fractions.

Example

```
td.num { font-variant-numeric: tabular-nums; }  
.caps { font-variant: small-caps; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

@font-face

Group: Fonts and text

Values

at-rule

Description

Loads a custom font file so it can be used in font-family. Provide woff2, set font-display: swap to show fallback text while loading, and subset by unicode-range for speed.

Example

```
@font-face {  
  font-family: "Cairo";  
  src: url("cairo.woff2") format("woff2");  
  font-weight: 400 700;  
  font-display: swap;  
}  
body { font-family: "Cairo", sans-serif; }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

line-height

Group: Fonts and text

Values

normal | number | length | percentage

Description

Height of each line of text. A unitless number (1.5) is best because it scales with the font size and is inherited as a factor. 1.4-1.6 is comfortable for body text.

Example

```
body { line-height: 1.6; }  
h1 { line-height: 1.1; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

color

Group: Fonts and text

Values

colour

Description

The colour of the text (and the default for borders and `currentColor`). Inherited. Ensure at least 4.5:1 contrast with the background for body text.

Example

```
body { color: #1b2431; }  
.muted { color: #6b7280; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-align

Group: Fonts and text

Values

start | end | left | right | center | justify

Description

Horizontal alignment of inline content inside a block. start/end follow the writing direction (best for multilingual sites). Justified text can create rivers of white space; use with hyphens: auto.

Example

```
footer { text-align: center; }  
th { text-align: start; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-decoration

Group: Fonts and text

Values

none | underline | overline | line-through, style, colour, thickness

Description

Lines drawn on text. Shorthand for text-decoration-line, -style, -color, -thickness. Links are underlined by default; keep the underline or another clear indicator.

Example

```
a { text-decoration: none; }  
a:hover { text-decoration: underline; }  
.spelling { text-decoration: red wavy underline; }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-transform

Group: Fonts and text

Values

none | capitalize | uppercase | lowercase

Description

Changes letter case for display without changing the HTML. Screen readers may spell out all-caps words.

Example

```
h2 { text-transform: uppercase; letter-spacing: 0.05em; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-indent

Group: Fonts and text

Values

length | percentage

Description

Indentation of the first line of a block.

Example

```
p + p { text-indent: 1.5em; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-shadow

Group: Fonts and text

Values

x y blur colour

Description

Shadow behind text. Several shadows may be listed.

Example

```
h1 { text-shadow: 0 2px 4px rgb(0 0 0 / 0.3); }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

letter-spacing / word-spacing

Group: Fonts and text

Values

normal | length

Description

Extra space between letters or words.

Example

```
.label { letter-spacing: 0.1em; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

white-space

Group: Fonts and text

Values

normal | nowrap | pre | pre-wrap | pre-line | break-spaces

Description

How whitespace and line breaks in the source are handled. normal collapses spaces and wraps; nowrap collapses but never wraps; pre preserves everything (like <pre>); pre-wrap preserves but wraps; pre-line collapses spaces but keeps line breaks.

Example

```
pre { white-space: pre-wrap; }  
.nowrap { white-space: nowrap; }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

overflow-wrap / word-break / hyphens

Group: Fonts and text

Values

normal | break-word | anywhere / normal | break-all | keep-all / none | manual | auto

Description

Control breaking of long words. overflow-wrap: anywhere breaks only when needed to avoid overflow; word-break: break-all breaks anywhere; hyphens: auto adds hyphens based on lang.

Example

```
p { overflow-wrap: anywhere; hyphens: auto; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

text-overflow

Group: Fonts and text

Values

clip | ellipsis

Description

What to show when text is clipped by overflow: hidden on a single line. Requires white-space: nowrap and overflow: hidden.

Example

```
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

vertical-align

Group: Fonts and text

Values

baseline | sub | super | top | middle | bottom | text-top | text-bottom | length

Description

Vertical alignment of inline or table-cell content. Does not centre block elements; use flexbox for that.

Example

```
img.icon { vertical-align: middle; }  
td { vertical-align: top; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

direction / unicode-bidi / writing-mode

Group: Fonts and text

Values

ltr | rtl / normal | embed | isolate | bidi-override / horizontal-tb | vertical-rl | vertical-lr

Description

Text direction and writing mode. Prefer the HTML `dir` attribute over the CSS `direction` property. `writing-mode` enables vertical text for East Asian typography and rotated headings.

Example

```
.vertical { writing-mode: vertical-rl; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

list-style

Group: Fonts and text

Values

shorthand: type position image

Description

Bullets and numbering for lists. `list-style-type`: disc, circle, square, decimal, lower-alpha, upper-roman, none, or a string. `list-style-position`: inside | outside. Applies to ul, ol and li.

Example

```
ul { list-style: square inside; }
nav ul { list-style: none; padding: 0; }
ol { list-style-type: upper-roman; }
li::marker { color: #e60000; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

quotes

Group: Fonts and text

Values

pairs of strings

Description

Quotation marks used by `<q>` and `open-quote/close-quote`.

Example

```
q { quotes: '«' '»'; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

content

Group: Fonts and text

Values

string | attr() | counter() | url() | open-quote | none

Description

Generated content for ::before and ::after.

Example

```
a[href^="http"]::after { content: " (" attr(href) ")"; }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

counter-reset / counter-increment

Group: Fonts and text

Values

name value

Description

CSS counters for automatic numbering of headings, figures, nested lists.

Example

```
body { counter-reset: fig; }  
figcaption::before { counter-increment: fig; content: "Figure "  
counter(fig) ": "; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

tab-size

Group: Fonts and text

Values

integer | length

Description

Width of tab characters in `<pre>`.

Example

```
pre { tab-size: 2; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

user-select

Group: Fonts and text

Values

auto | none | text | all

Description

Whether the user can select the text.

Example

```
.no-copy { user-select: none; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Box model

width / height

Group: Box model

Values

auto | length | percentage | min-content | max-content | fit-content

Description

Size of the box (content box by default; border box with `box-sizing: border-box`). Percentages are relative to the containing block. Prefer max-width for fluid layouts.

Example

```
.container { width: 100%; max-width: 1180px; }  
img { width: 100%; height: auto; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

min-width / max-width / min-height / max-height

Group: Box model

Values

length | percentage | none

Description

Limits on size. max-width: 100% on images stops them overflowing; min-height: 100vh makes a section fill the screen.

Example

```
img { max-width: 100%; }  
main { min-height: 100dvh; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

aspect-ratio

Group: Box model

Values

auto | ratio

Description

Preferred width-to-height ratio when one dimension is auto. Replaces the old padding-top hack for responsive video.

Example

```
.video { aspect-ratio: 16 / 9; width: 100%; }  
.avatar { aspect-ratio: 1; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

box-sizing

Group: Box model

Values

content-box | border-box

Description

Whether width and height include padding and border.
border-box is almost always what you want.

Example

```
*, *::before, *::after { box-sizing: border-box; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

margin

Group: Box model

Values

length | percentage | auto (1-4 values)

Description

Space outside the border. One value: all sides; two: vertical horizontal; three: top horizontal bottom; four: top right bottom left (clockwise). `margin: 0 auto` centres a block with a width. Vertical margins collapse. Logical versions: `margin-block`, `margin-inline`.

Example

```
p { margin: 0 0 1em; }  
.box { margin: 20px auto; }  
.card { margin-inline: auto; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

padding

Group: Box model

Values

length | percentage (1-4 values)

Description

Space between the content and the border. Same value order as margin. Cannot be negative. Backgrounds show through padding.

Example

```
button { padding: 10px 20px; }  
.card { padding-block: 1rem; padding-inline: 1.5rem; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

border

Group: Box model

Values

width style colour

Description

Shorthand for border-width, border-style and border-color on all sides. Styles: none, solid, dashed, dotted, double, groove, ridge, inset, outset. Per-side: border-top, border-right, border-bottom, border-left, or logical border-block-start etc.

Example

```
.card { border: 1px solid #e7ddcb; }  
blockquote { border-left: 4px solid #c8a24a; }  
th, td { border-bottom: 1px solid #ddd; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

border-radius

Group: Box model

Values

length | percentage (1-4 values, optional / for elliptical)

Description

Rounds the corners. 50% on a square makes a circle.

Per-corner: border-top-left-radius etc.

Example

```
.btn { border-radius: 8px; }  
.avatar { border-radius: 50%; }  
.pill { border-radius: 999px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

border-collapse / border-spacing

Group: Box model

Values

separate | collapse / length

Description

Table borders: collapse merges adjacent cell borders into one; border-spacing sets the gap between cells in separate mode (replaces cellspacing).

Example

```
table { border-collapse: collapse; }  
table.spaced { border-collapse: separate; border-spacing: 4px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

outline

Group: Box model

Values

width style colour

Description

A line drawn outside the border that does not take space. Browsers use it for the keyboard focus ring; never remove it without a replacement. `outline-offset` moves it away from the edge.

Example

```
:focus-visible { outline: 3px solid #1b6fd8; outline-offset: 2px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

box-shadow

Group: Box model

Values

x y blur spread colour [inset]

Description

Shadows around the box. Several shadows may be listed, separated by commas.

Example

```
.card { box-shadow: 0 10px 30px rgb(27 36 49 / 0.09); }  
.inset { box-shadow: inset 0 0 0 2px #e60000; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

overflow / overflow-x / overflow-y

Group: Box model

Values

visible | hidden | clip | scroll | auto

Description

What happens when content is larger than the box. auto shows scrollbars only when needed. Setting overflow (other than visible) also clips absolutely positioned children and contains floats.

Example

```
.table-wrap { overflow-x: auto; }  
.modal-body { max-height: 70vh; overflow-y: auto; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

display

Group: Box model

Values

block | inline | inline-block | flex | grid | none | contents |
flow-root | list-item | table...

Description

How the element generates boxes and lays out its children.
See the layout table in the CSS introduction.

Example

```
nav ul { display: flex; }  
.hidden { display: none; }  
span.badge { display: inline-block; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

visibility

Group: Box model

Values

visible | hidden | collapse

Description

hidden makes the element invisible but it still takes up space (unlike `display: none`). collapse removes table rows/columns.

Example

```
.placeholder { visibility: hidden; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

opacity

Group: Box model

Values

0 to 1

Description

Transparency of the whole element including its children. 0 is invisible but still takes space and receives clicks.

Example

```
.disabled { opacity: 0.5; }  
.fade { transition: opacity 0.3s; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

object-fit / object-position

Group: Box model

Values

fill | contain | cover | none | scale-down / position

Description

How a replaced element (img, video) fits its box when the box's ratio differs from the media. cover crops to fill; contain letterboxes.

Example

```
.thumb { width: 200px; height: 150px; object-fit: cover; object-position: center top; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

resize

Group: Box model

Values

none | both | horizontal | vertical

Description

Whether the user can resize the element (textarea by default).

Example

```
textarea { resize: vertical; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

CURSOR

Group: Box model

Values

auto | pointer | default | text | move | not-allowed | grab | wait
| help | zoom-in | url()

Description

The mouse pointer shape over the element.

Example

```
button, [role="button"] { cursor: pointer; }  
.disabled { cursor: not-allowed; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Backgrounds

background-color

Group: Backgrounds

Values

colour | transparent

Description

The colour behind the content and padding.

Example

```
body { background-color: #f6f1e7; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

background-image

Group: Backgrounds

Values

url() | gradient | none

Description

One or more images or gradients layered behind the element, first listed on top. Gradients: linear-gradient(), radial-gradient(), conic-gradient(), repeating-.*.

Example

```
.hero { background-image: linear-gradient(rgb(0 0 0 / 0.5), rgb(0 0 0 / 0.5)), url(hero.jpg); }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

background-size / background-position / background-repeat / background-attachment / background-clip / background-origin

Group: Backgrounds

Values

various

Description

How the image is sized (cover, contain, lengths), placed (center, top right, 50% 50%), repeated (no-repeat, repeat-x), scrolled (fixed for parallax), and which box it fills.

Example

```
.hero { background: url(hero.jpg) center / cover no-repeat; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

background

Group: Backgrounds

Values

shorthand

Description

Shorthand for all background properties: color image position / size repeat attachment origin clip.

Example

```
footer { background: #222 url(pattern.png) repeat; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

filter / backdrop-filter

Group: Backgrounds

Values

blur() | brightness() | contrast() | grayscale() | drop-shadow() | ...

Description

Graphical effects applied to the element or to what is behind it (frosted glass).

Example

```
img:hover { filter: brightness(1.1); }  
.glass { backdrop-filter: blur(10px); background: rgb(255 255 255 / 0.6); }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

mix-blend-mode / isolation

Group: Backgrounds

Values

normal | multiply | screen | overlay | difference...

Description

How the element blends with what is behind it.

Example

```
.logo { mix-blend-mode: multiply; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

clip-path / mask

Group: Backgrounds

Values

shapes and images

Description

Clip the element to a shape (`circle()`, `polygon()`, `inset()`) or mask it with an image.

Example

```
.hex { clip-path: polygon(25% 0, 75% 0, 100% 50%, 75% 100%, 25% 100%, 0 50%); }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Layout

position / top / right / bottom / left / inset / z-index

Group: Layout

Values

static | relative | absolute | fixed | sticky

Description

Positioning scheme and offsets. z-index orders positioned (and flex/grid) siblings on the z-axis; higher is in front; it creates stacking contexts.

Example

```
.modal { position: fixed; inset: 0; z-index: 1000; }  
.badge { position: absolute; top: -8px; right: -8px; }  
header { position: sticky; top: 0; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

float / clear

Group: Layout

Values

left | right | none / left | right | both

Description

Float takes an element out of flow and lets text wrap around it (its original purpose: images in text). Clear moves an element below floats. Not for page layout; use flex/grid.

Example

```
figure.left { float: left; margin: 0 1em 1em 0; }  
.after { clear: both; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

flex-direction / flex-wrap / flex-flow

Group: Layout

Values

row | row-reverse | column | column-reverse / nowrap | wrap | wrap-reverse

Description

Direction of the main axis and whether items wrap. flex-flow is the shorthand.

Example

```
.row { display: flex; flex-flow: row wrap; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

justify-content

Group: Layout

Values

flex-start | flex-end | center | space-between | space-around | space-evenly | start | end

Description

Distribution of items along the main axis (flex) or of grid tracks along the inline axis.

Example

```
.toolbar { display: flex; justify-content: space-between; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

align-items / align-self / align-content

Group: Layout

Values

stretch | flex-start | flex-end | center | baseline | start | end

Description

Alignment on the cross axis: for all items, for one item, or for wrapped lines.

Example

```
.center { display: flex; align-items: center; justify-content: center;
}
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

justify-items / justify-self / place-items / place-content / place-self

Group: Layout

Values

start | end | center | stretch

Description

Grid inline-axis alignment and shorthands combining align and justify.

Example

```
.grid { display: grid; place-items: center; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

gap / row-gap / column-gap

Group: Layout

Values

length | percentage

Description

Space between flex items, grid tracks or multi-column columns. Replaces margin hacks.

Example

```
.cards { display: grid; gap: 24px 16px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

flex / flex-grow / flex-shrink / flex-basis

Group: Layout

Values

number number length|auto

Description

How a flex item grows and shrinks relative to siblings and its starting size. flex: 1 = fill equally; flex: 0 0 200px = fixed 200px; flex: auto = size to content then grow.

Example

```
.sidebar { flex: 0 0 240px; }  
.content { flex: 1; min-width: 0; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

order

Group: Layout

Values

integer

Description

Visual order of flex or grid items (does not change tab order; use sparingly).

Example

```
.first { order: -1; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

grid-template-columns / grid-template-rows

Group: Layout

Values

track sizes: length, %, fr, auto, min-content, max-content, minmax(), repeat(), fit-content()

Description

Define the explicit grid tracks. repeat(auto-fit, minmax(240px, 1fr)) creates as many columns as fit.

Example

```
.layout { display: grid; grid-template-columns: 240px 1fr; }  
.gallery { grid-template-columns: repeat(auto-fill, minmax(150px, 1fr)); }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

grid-template-areas / grid-area

Group: Layout

Values

strings / name or line numbers

Description

Named regions for readable layouts. Items are placed with `grid-area: name`.

Example

```
.page { grid-template-areas: "head head" "nav main" "foot foot"; }  
footer { grid-area: foot; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

grid-column / grid-row / grid-column-start etc.

Group: Layout

Values

line / span n / name

Description

Place an item by grid lines. span 2 makes it cover two tracks.

Example

```
.wide { grid-column: 1 / -1; }  
.tall { grid-row: span 2; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

grid-auto-flow / grid-auto-rows / grid-auto-columns

Group: Layout

Values

row | column | dense / track size

Description

How auto-placed items flow and how implicit tracks are sized.

Example

```
.masonry { grid-auto-flow: dense; grid-auto-rows: 120px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

columns / column-count / column-width / column-gap / column-rule / column-span / break-inside

Group: Layout

Values

various

Description

Multi-column text like a newspaper.

Example

```
.article { columns: 2 300px; column-gap: 32px; }  
figure { break-inside: avoid; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

container-type / container-name / @container

Group: Layout

Values

inline-size | size | normal

Description

Container queries: style an element based on the size of an ancestor container instead of the viewport.

Example

```
.sidebar { container-type: inline-size; }  
@container (max-width: 300px) { .card { flex-direction: column; } }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

contain / content-visibility

Group: Layout

Values

layout | paint | size | style / visible | hidden | auto

Description

Performance hints: isolate an element's rendering;
content-visibility: auto skips rendering off-screen content.

Example

```
.long-list > li { content-visibility: auto; contain-intrinsic-size:  
auto 80px; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Motion

transform / transform-origin / rotate / scale / translate

Group: Motion

Values

translate() rotate() scale() skew() matrix() perspective()

Description

Move, rotate, scale or skew the element without affecting layout; hardware accelerated. The individual properties rotate, scale and translate can be set separately.

Example

```
.card:hover { transform: translateY(-4px) scale(1.02); }  
.spin { rotate: 45deg; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

transition / transition-property / -duration / -timing-function / -delay

Group: Motion

Values

property time easing time

Description

Animate changes of properties smoothly. Timing functions: ease, linear, ease-in, ease-out, ease-in-out, cubic-bezier(), steps().

Example

```
a { transition: color 0.2s ease; }  
.panel { transition: transform 0.3s cubic-bezier(0.2, 0.8, 0.2, 1); }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

animation / @keyframes

Group: Motion

Values

name duration timing delay iteration-count direction fill-mode
play-state

Description

Keyframe animations. Respect prefers-reduced-motion.

Example

```
@keyframes pulse { 50% { opacity: 0.5; } }  
.dot { animation: pulse 1s infinite; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

scroll-behavior / scroll-snap-type / scroll-snap-align / scroll-margin / scroll-padding / overscroll-behavior

Group: Motion

Values

various

Description

Smooth scrolling, snap points for carousels, and offsets for fixed headers.

Example

```
html { scroll-behavior: smooth; scroll-padding-top: 80px; }  
.carousel { scroll-snap-type: x mandatory; }  
.carousel > * { scroll-snap-align: start; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

will-change

Group: Motion

Values

property names

Description

Hint that a property will animate so the browser can optimise. Use sparingly.

Example

```
.drawer { will-change: transform; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Interaction

pointer-events

Group: Interaction

Values

auto | none

Description

none makes the element ignore mouse/touch events (clicks pass through).

Example

```
.overlay-text { pointer-events: none; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

accent-color

Group: Interaction

Values

colour

Description

Colour of native checkboxes, radios, range sliders and progress bars.

Example

```
:root { accent-color: #e60000; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

appearance

Group: Interaction

Values

none | auto | base-select

Description

Remove or keep the native look of form controls. none lets you restyle selects and inputs fully.

Example

```
select { appearance: none; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

color-scheme

Group: Interaction

Values

light | dark | light dark

Description

Tells the browser which colour schemes the page supports so form controls and scrollbars match.

Example

```
:root { color-scheme: light dark; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

caret-color

Group: Interaction

Values

colour

Description

Colour of the text cursor.

Example

```
input { caret-color: #e60000; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

touch-action

Group: Interaction

Values

auto | none | pan-x | pan-y | manipulation

Description

Which touch gestures the browser handles; manipulation removes the 300ms tap delay on old mobile browsers.

Example

```
button { touch-action: manipulation; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

field-sizing

Group: Interaction

Values

fixed | content

Description

Make inputs and textareas grow with their content.

Example

```
textarea { field-sizing: content; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

all

Group: Interaction

Values

initial | inherit | unset | revert

Description

Reset every property at once.

Example

```
.reset { all: unset; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

@media

Group: Interaction

Values

at-rule

Description

Conditional rules based on viewport size, orientation, resolution, hover ability, user preferences (prefers-color-scheme, prefers-reduced-motion, prefers-contrast) and output (print, screen).

Example

```
@media (max-width: 600px) { nav { display: none; } }  
@media (hover: hover) { a:hover { color: red; } }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

@supports

Group: Interaction

Values

at-rule

Description

Apply rules only if the browser supports a feature.

Example

```
@supports (display: grid) { .layout { display: grid; } }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

@import / @layer / @scope / @property

Group: Interaction

Values

at-rules

Description

Import other style sheets (slow; prefer <link>), define cascade layers, scope styles to a subtree, register typed custom properties.

Example

```
@layer reset, base, components;  
@layer base { body { margin: 0; } }
```

Where to apply it

In an external style sheet (recommended), in a <style> element in the <head>, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

@page

Group: Interaction

Values

at-rule

Description

Page size and margins for printing.

Example

```
@page { size: A4; margin: 2cm; }
```

Where to apply it

In an external style sheet (recommended), in a `<style>` element in the `<head>`, or inline in a style attribute. Check the property in your browser's developer tools (Elements panel, Styles tab): declarations shown struck through are being overridden or are invalid.

Styling HTML elements: cookbook

Styling common HTML elements: a cookbook

A sensible reset

```
*, ::before, ::after { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, -apple-system, "Segoe UI",
Roboto, Arial, sans-serif; line-height: 1.6; color: #1b2431; }
img, video, svg { max-width: 100%; height: auto; display: block; }
button, input, select, textarea { font: inherit; color: inherit; }
h1, h2, h3, h4 { line-height: 1.2; margin-block: 0 0.5em; }
p, ul, ol, dl, table, figure, pre { margin-block: 0 1em; }
```

Links

```
a { color: #0b57d0; text-decoration: underline 2px; }
a:hover { color: #083b8f; }
a:focus-visible { outline: 3px solid #0b57d0; outline-offset: 2px;
border-radius: 2px; }
a[href^="http"]:not([href="mysite.com"]):after { content: "↗";
font-size: 0.8em; }
```

Headings with a fluid scale

```
h1 { font-size: clamp(1.8rem, 1.2rem + 2.5vw, 3rem); }
h2 { font-size: clamp(1.4rem, 1rem + 1.5vw, 2rem); }
h3 { font-size: 1.25rem; }
```

Lists

```
/* Default bullets tweaked */
ul { padding-left: 1.25em; }
li + li { margin-top: 0.25em; }
li::marker { color: #e60000; }

/* Bullet-less horizontal menu */
.menu { list-style: none; margin: 0; padding: 0; display: flex; gap:
1rem; }

/* Custom check bullets */
.checks { list-style: none; padding: 0; }
.checks li::before { content: "✓ "; color: #0f8a4c; font-weight: bold;
}

/* Description list as a two-column grid */
```

```

dl.specs { display: grid; grid-template-columns: max-content 1fr; gap:
  0.25rem 1rem; }
dl.specs dt { font-weight: 600; }
dl.specs dd { margin: 0; }

```

Tables

```

table { border-collapse: collapse; width: 100%; font-variant-numeric:
  tabular-nums; }
caption { caption-side: top; text-align: start; font-weight: 600;
  padding-block: 0.5rem; }
th, td { padding: 0.5rem 0.75rem; border-bottom: 1px solid #e5e5e5;
  text-align: start; vertical-align: top; }
thead th { background: #f5f5f5; position: sticky; top: 0; }
tbody tr:nth-child(even) { background: #fafafa; }
tbody tr:hover { background: #fff4e0; }
td.num, th.num { text-align: end; }
.table-wrap { overflow-x: auto; } /* wrap the table for small
  screens */

/* Stacked table on phones */
@media (max-width: 600px) {
  .stack thead { display: none; }
  .stack tr { display: block; margin-bottom: 1rem; border: 1px solid
  #ddd; }
  .stack td { display: flex; justify-content: space-between; }
  .stack td::before { content: attr(data-label); font-weight: 600; }
}

```

Forms

```

.field { display: grid; gap: 0.25rem; margin-bottom: 1rem; }
label { font-weight: 600; }
input, select, textarea { padding: 0.6rem 0.75rem; border: 1px solid
  #c9c9c9; border-radius: 8px; background: #fff; }
input:focus-visible, select:focus-visible, textarea:focus-visible {
  outline: 3px solid #0b57d0; outline-offset: 1px; border-color:
  #0b57d0; }
input:user-invalid { border-color: #b90000; }
input:disabled { background: #f0f0f0; cursor: not-allowed; }
::placeholder { color: #8a8a8a; }
fieldset { border: 1px solid #ddd; border-radius: 8px; padding: 1rem;
  }
legend { padding: 0 0.5rem; font-weight: 600; }
input[type="checkbox"], input[type="radio"] { width: 1.1em; height: 1.
  1em; accent-color: #e60000; }
button { padding: 0.65rem 1.25rem; border: 0; border-radius: 8px;
  background: #e60000; color: #fff; font-weight: 600; cursor: pointer; }
button:hover { background: #b90000; }
button:disabled { opacity: 0.5; cursor: not-allowed; }

```


Header, nav, main, footer layout

```
body { min-height: 100dvh; display: flex; flex-direction: column; }
header { position: sticky; top: 0; z-index: 10; background: #fff; box-shadow: 0 1px 0 #eee; }
header .wrap { display: flex; align-items: center; justify-content: space-between; gap: 1rem; padding: 0.75rem 1rem; }
nav ul { display: flex; gap: 1.25rem; list-style: none; margin: 0; padding: 0; }
nav a[aria-current="page"] { border-bottom: 2px solid #e60000; }
main { flex: 1; width: min(100% - 2rem, 1180px); margin-inline: auto; padding-block: 2rem; }
footer { background: #222; color: #ddd; text-align: center; padding: 2rem 1rem; margin-top: auto; }
footer a { color: #ffd166; }
```

Images and figures

```
figure { margin: 0 0 1.5rem; }
figure img { border-radius: 12px; }
figcaption { font-size: 0.9rem; color: #666; margin-top: 0.5rem; text-align: center; }
.avatar { width: 64px; aspect-ratio: 1; border-radius: 50%; object-fit: cover; }
.hero img { width: 100%; height: 50vh; object-fit: cover; }
```

Code and pre

```
code { font-family: ui-monospace, Consolas, "Courier New", monospace; font-size: 0.95em; background: #f4f4f4; padding: 0.1em 0.3em; border-radius: 4px; }
pre { background: #1e1e1e; color: #eee; padding: 1rem; border-radius: 8px; overflow-x: auto; tab-size: 2; }
pre code { background: none; padding: 0; color: inherit; }
kbd { border: 1px solid #bbb; border-bottom-width: 2px; border-radius: 4px; padding: 0 0.35em; background: #fafafa; font-size: 0.9em; }
```

Blockquote, hr, details

```
blockquote { margin: 1.5rem 0; padding: 0.75rem 1.25rem; border-inline-start: 4px solid #c8a24a; background: #fdf8f0; }
blockquote footer { font-size: 0.9rem; color: #666; }
hr { border: 0; border-top: 1px solid #e0e0e0; margin: 2rem 0; }
details { border: 1px solid #ddd; border-radius: 8px; padding: 0.5rem 1rem; }
summary { cursor: pointer; font-weight: 600; }
details[open] summary { margin-bottom: 0.5rem; }
```

Dialog and popover

```
dialog { border: 0; border-radius: 16px; padding: 1.5rem; max-width:
```

```
min(90vw, 480px); box-shadow: 0 24px 60px rgb(0 0 0 / 0.25); }
dialog::backdrop { background: rgb(0 0 0 / 0.5); backdrop-filter:
blur(2px); }
[popover] { border: 1px solid #ddd; border-radius: 12px; padding:
1rem; box-shadow: 0 10px 30px rgb(0 0 0 / 0.15); }
[popover]:popover-open { animation: fade-in 0.15s ease-out; }
```

Visually hidden (for screen readers only)

```
.sr-only {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -
1px;
  overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; border:
0;
}
```

Dark mode

```
:root { color-scheme: light dark; --bg: #fff; --ink: #1b2431; }
@media (prefers-color-scheme: dark) { :root { --bg: #111; --ink: #eee;
} }
body { background: var(--bg); color: var(--ink); }
```

Lists and tables with CSS

Lists group related items; tables display structured data (information arranged in rows and columns so that it is easy to scan). Combining the right HTML element with a little CSS makes both far more readable and accessible. This chapter walks through every common styling task for `<ul>`, `<ol>`, `<dl>` and `<table>`, step by step, with complete copy-and-paste examples.

1. Unordered and ordered lists

Use `<ul>` for unordered lists (bullets) and `<ol>` for ordered lists (numbers). Each item goes in an `<li>`. The browser adds the bullets or numbers automatically, indents the list about 40 pixels and adds a blank line before and after it.

```
<ul class="features">
  <li>Responsive layout</li>
  <li>Semantic HTML</li>
  <li>Accessible components</li>
</ul>

<ol class="steps">
  <li>Plan content</li>
  <li>Structure markup</li>
  <li>Apply typography and layout</li>
</ol>
```

Changing the marker: list-style-type

The `list-style-type` property chooses the bullet or numbering style. It replaces the obsolete HTML `type` attribute.

Value	Works on	Result
disc (default for ul)	ul	Filled circle
circle	ul	Hollow circle

Value	Works on	Result
square	ul	Filled square
none	ul, ol	No marker at all (used for menus and navigation)
decimal (default for ol)	ol	1, 2, 3
decimal-leading-zero	ol	01, 02, 03
lower-alpha / upper-alpha	ol	a, b, c / A, B, C
lower-roman / upper-roman	ol	i, ii, iii / I, II, III
lower-greek	ol	alpha, beta, gamma
arabic-indic	ol	Arabic-Indic digits used in Arabic-language pages
'\2192 ' or any string	ul	A custom character such as an arrow or a check mark

```
ul.a { list-style-type: circle; }
ul.b { list-style-type: square; }
ol.c { list-style-type: upper-roman; }
ol.d { list-style-type: lower-alpha; }
ul.arrows { list-style-type: '\2192 '; } /* a right arrow before
every item */
```

Image markers and marker position

```
ul { list-style-image: url('sqpurple.gif'); }

/* inside: the marker is part of the text and wraps with it;
   outside (default): the marker hangs in the left margin */
ul.inside { list-style-position: inside; }
ul.outside { list-style-position: outside; }
```

The list-style shorthand

list-style sets type, position and image in one declaration, in that order. Missing values return to their defaults.

```
ul { list-style: square inside url('sqpurple.gif'); }
```

Removing default settings

To use a list as a menu, remove the marker, the margin and the padding. Setting margin and padding to 0 on the `<ul>` removes the indentation that browsers add; without this step the items still sit 40 pixels from the left edge.

```
ul.menu {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
}
```

Styling the marker itself: `::marker`

The `::marker` pseudo-element styles only the bullet or number, leaving the item text alone. It accepts colour, font and content properties.

```
li::marker { color: #b90000; font-weight: bold; }  
ol li::marker { content: counter(list-item) ' '; }
```

Custom markers with `::before` (the W3Schools 'features' pattern)

For complete control replace the marker with generated content. The rules below remove the default bullets, space the items with Grid gap, give every item a card look and insert a check mark with `::before`.

```
.features {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
  display: grid;  
  gap: 12px;  
}  
.features li {  
  padding: 16px 20px;  
  background: #fff;  
  border-radius: 12px;  
  box-shadow: 0 10px 25px rgba(15, 23, 42, 0.08);  
}
```

```
.features li::before {
  content: '\2714';      /* heavy check mark */
  color: #04AA6D;
  margin-right: 12px;
}
```

Colouring lists

Anything added to the `<ul>` or `<ol>` affects the whole list; anything added to `<li>` affects one item. Padding on the list moves the markers inward as well.

```
ol {
  background: #ff9999;
  padding: 20px;
}
ul {
  background: #3399ff;
  padding: 20px;
}
ol li {
  background: #ffe5e5;
  color: darkred;
  padding: 5px;
  margin-left: 35px;
}
ul li {
  background: #cce5ff;
  color: darkblue;
  margin: 5px;
}
```

Horizontal lists: navigation bars

A navigation bar is just a list of links laid out in a row. Flexbox is the modern way; float was used before 2015.

```
<nav>
  <ul class="navbar">
    <li><a href="index.html">Home</a></li>
    <li><a href="books.html">Books</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>

<style>
.navbar {
  list-style: none;
```

```

margin: 0;
padding: 0;
display: flex;
gap: 4px;
background: #333;
}
.navbar a {
display: block;
padding: 14px 16px;
color: white;
text-decoration: none;
}
.navbar a:hover { background: #111; }
</style>

```

Numbered lists with counters

CSS counters number anything, including nested headings, and let you style the number separately.

```

ol.fancy { counter-reset: item; list-style: none; padding: 0; }
ol.fancy li { counter-increment: item; margin: 8px 0; }
ol.fancy li::before {
content: counter(item);
display: inline-block;
width: 1.6em; height: 1.6em; line-height: 1.6em;
margin-right: .6em;
text-align: center;
border-radius: 50%;
background: #b90000;
color: white;
font-weight: bold;
}

```

2. Description lists

Use `<dl>` with `<dt>` (term) and `<dd>` (definition) for glossaries or metadata. Browsers indent the `<dd>` by about 40 pixels; with CSS Grid you can lay terms and definitions out in two columns.

```

<dl class="glossary">
  <dt>HTML</dt>
  <dd>Structure of a web page.</dd>
  <dt>CSS</dt>
  <dd>Presentation and layout.</dd>
</dl>

```

```

<style>
.glossary {
  display: grid;
  grid-template-columns: max-content 1fr;
  gap: 6px 16px;
}
.glossary dt { font-weight: bold; color: #b90000; }
.glossary dd { margin: 0; }
</style>

```

3. HTML tables

`<table>` organises tabular data with rows (`<tr>`), header cells (`<th>`) and data cells (`<td>`). Tabular data is information best shown in a grid, such as price lists or schedules. Group the header row in `<thead>` and the data in `<tbody>`, and give every `<th>` a `scope` attribute so screen readers know whether it labels a column or a row.

```

<table class="pricing">
  <caption>Subscription plans</caption>
  <thead>
    <tr>
      <th scope="col">Plan</th>
      <th scope="col">Price</th>
      <th scope="col">Features</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Starter</th>
      <td>$0</td>
      <td>Basic components</td>
    </tr>
    <tr>
      <th scope="row">Pro</th>
      <td>$19</td>
      <td>Advanced layouts</td>
    </tr>
  </tbody>
</table>

```

Table borders

A table has no visible borders by default. The border property must be set on the table AND on the cells; setting it on the table alone draws only the outer frame. Because each cell has its own border you get a double line between cells until you collapse them.

```
table, th, td {  
    border: 1px solid black;  
}
```

Collapsed borders

border-collapse: collapse merges adjacent cell borders into a single line. This is almost always what you want. The old HTML cellspacing attribute is replaced by border-spacing (only in the separate model) and cellpadding by padding on the cells.

```
table {  
    border-collapse: collapse;  
}  
th, td {  
    border: 1px solid #ddd;  
    padding: 8px;  
}
```

```
/* alternative: keep separate borders but control the gap */  
table.spaced { border-collapse: separate; border-spacing: 6px 2px; }
```

Rounded or borderless tables

For a light look, add only a bottom border to each row. For rounded corners set overflow: hidden and border-radius on the table.

```
table { border-collapse: collapse; width: 100%; }  
td, th { border-bottom: 1px solid #ddd; padding: 12px; }  
  
table.rounded { border-radius: 12px; overflow: hidden; }
```

Table size

Width and height are set with CSS width and height on the table, rows or cells. width: 100% makes the table fill its container. The obsolete HTML width attribute did the same job.

```
table { width: 100%; }
th { height: 70px; }
td.narrow { width: 70px; }
table.fixed { table-layout: fixed; } /* equal columns regardless of
content */
```

Alignment

text-align sets horizontal alignment (th content is centred and bold by default, td content is left aligned); vertical-align sets vertical alignment (middle by default). These replace the align and valign attributes.

```
th { text-align: left; }
td { text-align: center; height: 50px; vertical-align: bottom; }
```

Padding and spacing

padding on th/td controls the space between the border and the content. To add space between cells themselves use border-spacing (separate border model only).

```
th, td { padding: 15px; }
th, td { padding-top: 10px; padding-bottom: 20px; padding-left: 30px;
padding-right: 40px; }
```

Horizontal dividers, hover and zebra stripes

Highlight rows on hover with :hover on <tr>, and zebra-stripe with :nth-child(even). Both make long tables easier to read.

```
tr:hover { background-color: coral; }
tr:nth-child(even) { background-color: #f2f2f2; }

/* vertical zebra stripes */
th:nth-child(even), td:nth-child(even) { background-color: #d6eeee; }

/* combining both with transparency */
tr:nth-child(even) { background-color: rgba(150, 212, 212, 0.4); }
th:nth-child(even), td:nth-child(even) { background-color: rgba(150,
```

```
212, 212, 0.4); }
```

Styling the header row

```
.pricing {
  width: 100%;
  border-collapse: collapse;
  background: #fff;
  border-radius: 12px;
  overflow: hidden;
}
.pricing th, .pricing td {
  padding: 16px 20px;
  text-align: left;
}
.pricing thead {
  background: #04AA6D;
  color: #fff;
}
.pricing tbody tr:nth-child(even) {
  background: #f9fafb;
}
```

Key rules: `border-collapse: collapse` removes gaps, header styling highlights the labels, and `:nth-child(even)` zebra-stripes the rows.

Colouring individual columns

```
<table>
  <colgroup>
    <col span="2" style="background-color: #D6EEEE">
  </colgroup>
  <tr><th>MON</th><th>TUE</th><th>WED</th></tr>
  ...
</table>
```

Only a few properties work on `<col>`: width, visibility, background and border. For anything else target `td:nth-child(n)`.

Responsive tables

Tables can overflow on small screens. Wrapping them in a scroll container preserves readability: mobile users swipe sideways instead of the table breaking the layout. The

wrapper is a `<div>` with `overflow-x: auto`.

```
<div class="table-wrapper">
  <table>...</table>
</div>

<style>
.table-wrapper { overflow-x: auto; }
</style>
```

An alternative for narrow screens is to turn each row into a block and print the column label with a data attribute:

```
@media (max-width: 600px) {
  thead { display: none; }
  tr { display: block; margin-bottom: 1em; border: 1px solid #ddd; }
  td { display: flex; justify-content: space-between; padding: 8px; }
  td::before { content: attr(data-label); font-weight: bold; }
}

<!-- HTML -->
<td data-label="Price">$19</td>
```

Sticky header and caption position

```
thead th { position: sticky; top: 0; background: #fff; }
caption { caption-side: bottom; font-style: italic; padding: 8px; }
table.no-empty { empty-cells: hide; }
```

4. Accessibility tips for tables

- Always use `<th>` for headers and add the `scope` attribute (`col` or `row`). It tells assistive technology which header belongs to which cell.
- Provide captions with `<caption>` when the table's purpose is not obvious from the surrounding text.
- Keep tables for data, not layout. Use CSS Grid or Flexbox for page layout.
- Do not rely on colour alone (for example a red cell for 'sold out'); add text or an icon as well.

- Avoid merged cells (rowspan/colspan) in data tables where possible; if you need them, add headers/id associations.
- Right-align numbers so that digits line up, and use the same number of decimals in a column.

5. Complete example: course outline with a list and a table

This demo pairs a topic list with a simple table. The list uses square bullets while the table collapses its borders for a clean grid, with coloured headers to improve readability.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Course Outline</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Course Outline</h1>
  <ul>
    <li>HTML Basics</li>
    <li>CSS Essentials</li>
    <li>Responsive Layouts</li>
  </ul>
  <table>
    <tr><th>Module</th><th>Duration</th></tr>
    <tr><td>Getting Started</td><td>2 hrs</td></tr>
    <tr><td>Flexbox & Grid</td><td>3 hrs</td></tr>
  </table>
</body>
</html>

/* styles.css */
body {
  font-family: 'Segoe UI', sans-serif;
  padding: 48px;
  background: #eef2ff;
}
ul {
  list-style: square;
}
table {
```

```

margin-top: 32px;
border-collapse: collapse;
width: 100%;
max-width: 640px;
background: #fff;
}
th, td {
border: 1px solid #d9e2ec;
padding: 12px 16px;
text-align: left;
}
th {
background: #6366f1;
color: #fff;
}

```

6. Quick reference: list and table properties

Property	Applies to	Purpose
list-style-type	ul, ol, li	Marker shape or numbering system
list-style-image	ul, ol, li	Image as marker
list-style-position	ul, ol, li	Marker inside or outside the item box
list-style	ul, ol, li	Shorthand for the three above
::marker	li	Style the marker only
counter-reset / counter-increment / counter()	any	Custom numbering
border-collapse	table	collapse or separate cell borders
border-spacing	table	Gap between cells (separate model)
caption-side	table	Caption above (top) or below (bottom)
empty-cells	table	show or hide borders of empty cells
table-layout	table	auto (content sized) or fixed (equal, faster)
vertical-align	td, th	top, middle, bottom

Property	Applies to	Purpose
text-align	td, th	left, center, right
padding	td, th	Space inside cells (replaces cellpadding)
:nth-child(even/odd)	tr, td	Zebra stripes
tr:hover	tr	Row highlight
overflow-x: auto	wrapper div	Scrollable table on small screens

7. Old HTML attributes and their CSS replacements

Old attribute (HTML 3.2 / 4)	Element	Use instead
type="square"	ul	list-style-type: square
type="A" / type="i"	ol	list-style-type: upper-alpha / lower-roman (type on ol is still allowed but CSS is preferred)
compact	ul, ol, dl	margin / line-height
border="1"	table	border on table, th, td
cellpadding	table	padding on th, td
cellspacing	table	border-spacing (or border-collapse: collapse)
width / height	table, td	width / height in CSS
bgcolor	table, tr, td	background-color
align	table, tr, td	text-align / margin: 0 auto
valign	tr, td	vertical-align
nowrap	td	white-space: nowrap
frame / rules	table	border on the relevant sides

Old attribute (HTML 3.2 / 4)	Element	Use instead
summary	table	<caption> or aria-describedby

HTML and CSS Course

Thirty-six lessons from first page to finished project, following the W3Schools HTML and CSS tutorial.

About this course

This part is a guided course rather than a reference: read the lessons in order, type the examples, and change them. Its structure follows the W3Schools HTML and CSS tutorial (foundations, styling core, layout, enhancements, quality, project). Whenever a lesson mentions an element or property, the full details are in Parts 6 and 11.

Lesson	Topic
1	Introduction and environment setup
2	Your first web page
3	Essentials - elements, attributes and nesting
4	Document structure
5	Text content
6	Lists and tables
7	Links and navigation
8	Images and media
9	Forms and validation
10	Reusable components
11	Styling essentials - the three ways to add CSS
12	Selectors and specificity
13	Cascade and inheritance
14	Typography
15	Colours and backgrounds
16	The box model
17	Layout fundamentals - normal flow
18	Positioning and z-index

Lesson	Topic
19	Flexbox
20	CSS Grid
21	Layout patterns
22	Responsive design
23	Responsive components
24	Custom properties (CSS variables)
25	calc() and CSS functions
26	Transitions
27	Transforms
28	Animations
29	Effects and filters
30	Accessibility
31	Design systems
32	Performance
33	Debugging and testing
34	Tooling and automation (optional)
35	Project - a complete landing page
36	Quick quiz and answers

Lesson 1: Introduction and environment setup

HTML gives a page its structure and meaning; CSS gives it appearance and layout. Together they are the two languages every web page is made of. HTML says 'this is a heading, this is a list of three items, this is a table of prices'; CSS says 'headings are dark red and 28 pixels, the list has no bullets and sits in a row, the table has thin grey lines'.

What you need

- A text editor. Any editor that saves plain text works: UltraEdit, Notepad++, Sublime Text or VS Code. Turn on syntax highlighting for HTML and CSS.
- A browser: Chrome, Edge or Firefox. Its developer tools (press F12) show you the HTML tree, the CSS applied to each element and the box model.
- A folder for your project, for example C:\Users\you\Documents\bookshop. Keep index.html, styles.css and an images folder inside it.
- Nothing else. No server, no installation, no account. Open the .html file in the browser by double-clicking it or dragging it into a tab.

Workflow

- Write or change a file in the editor and save it (Ctrl+S).
- Switch to the browser and reload (F5). The page shows the change immediately.
- If something looks wrong, right-click it and choose Inspect. The Elements panel shows the exact HTML and the Styles panel shows which CSS rules apply and which are crossed

out (overridden).

- Validate occasionally with <https://validator.w3.org/> (HTML) and <https://jigsaw.w3.org/css-validator/> (CSS) to catch typos such as unclosed tags or misspelled properties.

File naming rules

- Use lower-case names with no spaces: `about-us.html`, not `About Us.html`. Web servers are case-sensitive and spaces become `%20` in URLs.
- The home page is `index.html`: servers show it automatically when someone visits the folder.
- Save with UTF-8 encoding (in UltraEdit: File > Save As > Encoding: UTF-8, no BOM). Declare it with `<meta charset="utf-8">` so that Arabic, accents and symbols display correctly.

Lesson 2: Your first web page

Every page has the same skeleton. Type it once, save it as index.html and open it in the browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My first page</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Hello, world</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About</a>
    </nav>
  </header>
  <main>
    <p>This is my first web page. It has a heading, a paragraph and a
link to
      <a href="https://developer.mozilla.org">MDN</a>.</p>
  </main>
  <footer>
    <p>&copy; 2026 My Name</p>
  </footer>
</body>
</html>
```

Line	What it does
<!DOCTYPE html>	Tells the browser this is modern HTML (standards mode). Always the first line.
<html lang="en">	Root element. lang tells screen readers and translators the language (ar for Arabic, with dir="rtl").
<head>	Information about the page that is not displayed: encoding, title, links to CSS.
<meta charset="utf-8">	Character encoding; prevents garbled text.

Line	What it does
<code><meta name="viewport" ...></code>	Makes the page scale correctly on phones. Without it, mobile browsers pretend to be 980px wide.
<code><title></code>	Text in the browser tab, bookmarks and search results.
<code><link rel="stylesheet"></code>	Loads the CSS file.
<code><body></code>	Everything visible.
<code><header>, <nav>, <main>, <footer></code>	Semantic landmarks: which part of the page is which.

Add the style sheet

Create styles.css in the same folder:

```
body {
  font-family: system-ui, Arial, sans-serif;
  line-height: 1.6;
  max-width: 60rem;
  margin: 0 auto;
  padding: 1rem;
  color: #1b2431;
}
h1 { color: #b90000; }
nav a { margin-right: 1rem; }
footer { margin-top: 3rem; font-size: .9rem; color: #666; }
```

Reload the browser. If nothing changes, check that the file is really called styles.css (not styles.css.txt), that it is in the same folder as index.html, and that the href in `<link>` matches exactly.

Lesson 3: Essentials - elements, attributes and nesting

An element is a start tag, content and an end tag:

`<p>text</p>`. Attributes go inside the start tag as `name="value"` pairs. Some elements are void: they have no content and no end tag (`<br>`, `<img>`, `<input>`, `<meta>`, `<link>`, `<hr>`).

```
<a href="books.html" class="button" id="catalogue-link" title="See
all books">Catalogue</a>

```

Nesting

Elements nest like boxes: close the inner one before the outer one. `<p><em>right</em></p>` is correct;

`<p><em>wrong</p></em>` is not. Indent nested elements two spaces so the structure is visible in your editor.

Block and inline

Block-level elements (`p`, `div`, `h1`, `ul`, `table`, `section`...) start on a new line and stretch across the container. Inline elements (`a`, `em`, `strong`, `span`, `img`, `code`...) flow within the text. A block may contain inline elements; an inline element may not contain a block (except that `<a>` may wrap blocks in HTML5). CSS display can change this behaviour.

Comments and whitespace

```
<!-- This is a comment. Browsers ignore it. -->
<p>Multiple      spaces
    and line breaks collapse to a single space.</p>
```

Entities

Characters that have a meaning in HTML must be written as entities: `<` for `<`, `>` for `>`, `&` for `&`. ` ` is a

non-breaking space; © is the copyright sign. See the entity table in the Value References part.

Lesson 4: Document structure

Use semantic elements to describe the parts of the page. They mean nothing visually (all look like a `<div>`) but give screen readers, search engines and other developers a map of the page.

Element	Use for	How many per page
<code><header></code>	Introductory content of the page or of a section: logo, title, navigation	One per page plus one per article/section if needed
<code><nav></code>	Major navigation blocks	Usually one or two
<code><main></code>	The unique content of this page	Exactly one
<code><article></code>	Self-contained composition: blog post, product, comment	Any number
<code><section></code>	Thematic grouping with a heading	Any number
<code><aside></code>	Tangential content: sidebar, pull quote, related links	Any number
<code><footer></code>	Footer of the page or a section: copyright, contact, links	One per page plus per section

Element	Use for	How many per page
<code><div></code>	Generic container when no semantic element fits (mostly for styling)	Any number

```
<body>
  <header>
    <h1>Cairo Book Shop</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/books/">Books</a></li>
        <li><a href="/contact/">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <article>
      <h2>New arrivals</h2>
      <p>...</p>
    </article>
    <aside>
      <h2>Opening hours</h2>
      <p>Saturday to Thursday, 10:00 to 22:00</p>
    </aside>
  </main>
  <footer>
    <p>&copy; 2026 Cairo Book Shop &middot; <a href="/privacy/">Privacy</a></p>
  </footer>
</body>
```

Headings

Use one `<h1>` for the page title, then `<h2>` for main sections, `<h3>` inside them, and so on. Never skip levels for looks (h2 then h4); change the size with CSS instead. Screen reader users navigate by headings, so the outline must make sense on its own.

Lesson 5: Text content

Text elements carry meaning. Choose by what the text IS, not by how you want it to look; then style with CSS.

Element	Meaning	Default look
<p>	Paragraph	Block with space above and below
<h1>-<h6>	Headings	Bold, decreasing sizes
	Strong importance	Bold
	Stress emphasis	Italic
	Keywords, product names, no extra importance	Bold
<i>	Different voice: technical term, foreign phrase, thought	Italic
<mark>	Highlighted for reference	Yellow background
<small>	Side comment, legal text	Smaller
<s>	No longer accurate	Strikethrough
 / <ins>	Removed / added text	Strikethrough / underline
<sub> / <sup>	Subscript / superscript	H ₂ O, x ²
<code>	Code fragment	Monospace
<kbd>	Keyboard input	Monospace
<samp>	Program output	Monospace

Element	Meaning	Default look
<var>	Variable	Italic
<abbr title="...">	Abbreviation	Dotted underline
<time datetime="...">	Date/time	Plain
<q>	Short inline quotation	Quotation marks
<blockquote>	Long quotation	Indented block
<cite>	Title of a work	Italic
<pre>	Preformatted text (keeps spaces and line breaks)	Monospace block
 	Line break (addresses, poems only)	-
<hr>	Thematic break	Horizontal rule

```
<article>
  <h2>Learn HTML, <small>2nd edition</small></h2>
  <p>Published <time datetime="2026-03-01">1 March 2026</time>.
    <strong>Only 5 copies left.</strong> Price: <s>150 EGP</s>
<mark>120 EGP</mark>.</p>
  <p>Press <kbd>Ctrl</kbd>+<kbd>S</kbd> to save. The <abbr
title="HyperText Markup Language">HTML</abbr> file is <code>index.
html</code>.</p>
  <blockquote cite="https://example.com/review">
    <p>The clearest introduction I have read.</p>
    <footer>&mdash; <cite>Web Weekly</cite></footer>
  </blockquote>
</article>
```

Typography basics in CSS

```
body { font-family: Georgia, 'Times New Roman', serif; font-size: 1.05rem; line-height: 1.6; }
h1, h2, h3 { font-family: system-ui, sans-serif; line-height: 1.2; }
p { margin: 0 0 1em; max-width: 65ch; } /* 65 characters per line is comfortable */
```

Lesson 6: Lists and tables

Lists group related items; tables display structured data. This lesson is a summary; the chapter 'Lists and tables with CSS' in the CSS part covers every styling option in detail.

```
<ul class="features">
  <li>Responsive layout</li>
  <li>Semantic HTML</li>
  <li>Accessible components</li>
</ul>
<ol class="steps">
  <li>Plan content</li>
  <li>Structure markup</li>
  <li>Apply typography and layout</li>
</ol>
<dl class="glossary">
  <dt>HTML</dt><dd>Structure of a web page.</dd>
  <dt>CSS</dt><dd>Presentation and layout.</dd>
</dl>

/* card-style list with custom check marks */
.features { list-style: none; padding: 0; margin: 0; display: grid;
gap: 12px; }
.features li { padding: 16px 20px; background: #fff; border-radius:
12px;
                box-shadow: 0 10px 25px rgba(15, 23, 42, 0.08); }
.features li::before { content: '\2714'; color: #04AA6D; margin-right:
12px; }

<table class="pricing">
  <thead>
    <tr><th scope="col">Plan</th><th scope="col">Price</th><th
scope="col">Features</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">Starter</th><td>$0</td><td>Basic components</
td></tr>
    <tr><th scope="row">Pro</th><td>$19</td><td>Advanced layouts</
td></tr>
  </tbody>
</table>

.pricing { width: 100%; border-collapse: collapse; background: #fff;
border-radius: 12px; overflow: hidden; }
.pricing th, .pricing td { padding: 16px 20px; text-align: left; }
.pricing thead { background: #04AA6D; color: #fff; }
.pricing tbody tr:nth-child(even) { background: #f9fafb; }
```

```
.table-wrapper { overflow-x: auto; } /* wrap the table in this div  
for small screens */
```

- Always use `<th>` for headers and add `scope="col"` or `scope="row"`.
- Provide a `<caption>` when the table's purpose is not obvious.
- Keep tables for data, not layout.

Lesson 7: Links and navigation

The `<a>` element with an `href` attribute creates a hyperlink. The `href` can be absolute, relative to the current file, a fragment (`#id`) on the same page, or a special scheme such as `mailto:` or `tel:`.

```
<a href="https://example.com/">Absolute URL</a>
<a href="books/learn-html.html">Relative: file in the books folder</a>
<a href="../index.html">Relative: parent folder</a>
<a href="/contact/">Site-root relative</a>
<a href="#prices">Jump to the element with id="prices"</a>
<a href="mailto:info@example.com">Email us</a>
<a href="tel:+201001234567">Call us</a>
<a href="catalogue.pdf" download>Download the catalogue (PDF)</a>
<a href="https://partner.example" target="_blank"
rel="noopener">Opens in a new tab</a>
```

Navigation bar

```
<nav aria-label="Main">
  <ul class="nav">
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/books/">Books</a></li>
    <li><a href="/about/">About</a></li>
  </ul>
</nav>

<style>
.nav { display: flex; gap: 4px; list-style: none; margin: 0; padding:
0; background: #1b2431; }
.nav a { display: block; padding: 12px 18px; color: #fff; text-
decoration: none; }
.nav a:hover, .nav a:focus-visible { background: #b90000; }
.nav a[aria-current="page"] { background: #444; }
</style>
```

Link states

Style the five states in this order (LVHFA) or later rules will override earlier ones: `:link`, `:visited`, `:hover`, `:focus`, `:active`.

```
a:link { color: #0645ad; }
a:visited { color: #551a8b; }
a:hover { text-decoration-thickness: 2px; }
a:focus-visible { outline: 3px solid #ffbf47; outline-offset: 2px; }
a:active { color: #b90000; }
```


- Link text must make sense alone: 'Read the shipping policy', not 'click here'.
- Keep the underline or another non-colour cue so links are recognisable.
- Never remove the focus outline without providing a visible replacement.
- Use a `<button>` for actions (open menu, submit) and `<a>` for navigation.

Lesson 8: Images and media

```

```

```
<figure>
  
  <figcaption>Figure 1. Monthly sales, first half of 2026.</figcaption>
</figure>
```

```
<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
  
</picture>
```

```

```

```
<video controls width="640" poster="poster.jpg">
  <source src="tour.mp4" type="video/mp4">
  <track kind="captions" src="tour.en.vtt" srclang="en" label="English">
  Your browser does not support video.
</video>
```

```
<audio controls src="podcast.mp3"></audio>
```

Responsive images in CSS

```
img, video { max-width: 100%; height: auto; } /* never overflow the container */
.thumb { width: 200px; aspect-ratio: 3 / 4; object-fit: cover; } /* crop to a fixed shape */
.avatar { border-radius: 50%; }
```

- alt is required. Describe the content and function; use alt="" for purely decorative images.
- Always give width and height (or aspect-ratio) so the page does not jump while images load.
- Use loading="lazy" for images below the fold, never for the main hero image.

- Prefer AVIF/WebP with a JPEG fallback in `<picture>`; use SVG for logos and icons.
- Provide captions for video and a transcript for audio.

Lesson 9: Forms and validation

```
<form action="/order" method="post">
  <fieldset>
    <legend>Your details</legend>
    <p>
      <label for="name">Full name</label>
      <input id="name" name="name" type="text" required
autocomplete="name">
    </p>
    <p>
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required
autocomplete="email">
    </p>
    <p>
      <label for="qty">Quantity</label>
      <input id="qty" name="qty" type="number" min="1" max="10"
value="1">
    </p>
    <p>
      <label for="format">Format</label>
      <select id="format" name="format">
        <option value="pdf">PDF</option>
        <option value="print">Printed</option>
      </select>
    </p>
    <p>
      <label for="notes">Notes</label>
      <textarea id="notes" name="notes" rows="3"></textarea>
    </p>
    <p>
      <input id="terms" name="terms" type="checkbox" required>
      <label for="terms">I accept the terms</label>
    </p>
  </fieldset>
  <button type="submit">Order</button>
</form>
```

Built-in validation

Attribute	Checks
required	Field is not empty
type="email" / url / number / date	Value has the right format
min / max / step	Numeric or date range

Attribute	Checks
minlength / maxlength	Text length
pattern="[0-9]{5}"	Regular expression
novalidate (on form)	Turns built-in validation off

Styling forms and validation states

```
label { display: block; font-weight: 600; margin-bottom: .25rem; }
input, select, textarea, button { font: inherit; }
input, select, textarea {
  width: 100%; padding: .6rem .8rem; border: 1px solid #bbb; border-
radius: 6px;
}
input:focus-visible, select:focus-visible, textarea:focus-visible {
  outline: 3px solid #ffbf47; outline-offset: 1px; border-color:
#b90000;
}
input:user-invalid { border-color: #c00; background: #fff5f5; }
input:user-valid { border-color: #2a7; }
button[type="submit"] {
  background: #b90000; color: #fff; border: 0; padding: .7rem 1.4rem;
border-radius: 999px; cursor: pointer;
}
button:disabled { opacity: .5; cursor: not-allowed; }
```

- Every control needs a `<label>` connected by `for/id` (or wrapping it).
- Group radio buttons and checkboxes in a `<fieldset>` with a `<legend>`.
- Use the right type: it changes the mobile keyboard and adds validation for free.
- Show error messages in text next to the field, not only by colour.

Lesson 10: Reusable components

A component is a block of HTML plus the CSS that styles it, designed so you can copy it anywhere. Give it one class name on the outer element and prefix inner classes with it (card, card_title, card--featured) so styles do not leak.

```
<article class="card card--featured">
  
  <div class="card__body">
    <h3 class="card__title">Learn HTML</h3>
    <p class="card__text">From zero to a complete website.</p>
    <a class="btn" href="learn-html.html">Details</a>
  </div>
</article>

.card {
  display: flex; flex-direction: column;
  border: 1px solid #e2e8f0; border-radius: 16px; overflow: hidden;
  background: #fff;
}
.card__image { aspect-ratio: 3 / 4; object-fit: cover; }
.card__body { padding: 1rem 1.25rem; display: grid; gap: .5rem; }
.card__title { margin: 0; font-size: 1.1rem; }
.card--featured { border-color: #b90000; box-shadow: 0 10px 30px rgba(
185, 0, 0, .15); }
.btn {
  display: inline-block; padding: .5rem 1rem; border-radius: 999px;
  background: #b90000; color: #fff; text-decoration: none; justify-
self: start;
}
.btn:hover { background: #900; }
```

Design tokens with custom properties

```
:root {
  --brand: #b90000;
  --ink: #1b2431;
  --radius: 12px;
  --space-1: .5rem; --space-2: 1rem; --space-3: 2rem;
}
.card { border-radius: var(--radius); padding: var(--space-2); }
.btn { background: var(--brand); }
```

Change --brand once and every component follows. Put the variables in :root so they are available everywhere.

Lesson 11: Styling essentials - the three ways to add CSS

Method	How	Use when
External style sheet	<code><link rel="stylesheet" href="styles.css"></code> in <code><head></code>	Always, for real sites: one file shared by all pages, cached by the browser
Internal style sheet	<code><style> ... </style></code> in <code><head></code>	Single-page demos, emails, this reference's examples
Inline style	<code>style="color: red"</code> attribute on the element	Rarely: one-off values generated by a program; hard to override and maintain

Anatomy of a rule

```
selector {  
  property: value;      /* a declaration */  
  property: value;  
}  
  
h1 {  
  color: #b90000;  
  font-size: 2rem;  
}
```

Comments

```
/* CSS comments look like this and can span  
   several lines. There is no // comment in CSS. */
```

Units

Unit	Meaning	Typical use
px	CSS pixel	Borders, small fixed sizes
rem	Relative to the root font size (16px by default)	Font sizes, spacing, widths - respects user zoom settings

Unit	Meaning	Typical use
em	Relative to the element's own font size	Padding around text, icon sizes
%	Relative to the parent	Widths
vw / vh	1% of viewport width/height	Full-screen sections, fluid type
ch	Width of the digit 0	Line length (max-width: 65ch)
fr	Fraction of free space (Grid only)	Column tracks

Colours

```

color: red;                /* 148 named colours */
color: #b90000;            /* hex: rr gg bb */
color: #b90000cc;         /* hex with alpha */
color: rgb(185 0 0);
color: rgb(185 0 0 / 50%); /* 50% transparent */
color: hsl(0 100% 36%);    /* hue, saturation, lightness */
color: oklch(50% 0.2 25); /* perceptually uniform, wide gamut */
color: currentColor;       /* the element's text colour */

```


Lesson 12: Selectors and specificity

Selectors are like addresses for elements: they point CSS at the exact parts of the page you want to change.

Selector type	Example	Matches
Type	<code>p { }</code>	All <code><p></code> elements
Class	<code>.btn { }</code>	Elements with <code>class="btn"</code>
ID	<code>#hero { }</code>	The element with <code>id="hero"</code>
Universal	<code>* { }</code>	All elements
Attribute	<code>a[target="_blank"], input[type="email"], [class^="icon-"]</code>	Elements by attribute presence or value
Descendant	<code>.card p</code>	Any <code><p></code> inside <code>.card</code>
Child	<code>.card > h3</code>	<code><h3></code> directly inside <code>.card</code>
Adjacent sibling	<code>h2 + p</code>	The <code><p></code> immediately after an <code><h2></code>
General sibling	<code>h2 ~ p</code>	Every <code><p></code> after an <code><h2></code> in the same parent
Pseudo-class	<code>a:hover, input:required, li:nth-child(2n), :is(), :where(), :has()</code>	Elements in a state or position
Pseudo-element	<code>p::first-line, p::after, li::marker</code>	Parts of an element

Specificity: who wins

When two rules set the same property on the same element, the more specific selector wins. Count (IDs, classes/attributes/pseudo-classes, types/pseudo-elements). Inline styles beat everything except `!important`. If specificity

ties, the later rule wins.

Selector	Score (id, class, type)
p	0,0,1
.card p	0,1,1
.card.highlight p	0,2,1
#feature.card p	1,1,1
style="..."	1,0,0,0 (inline)
:where(.a .b)	0,0,0 (where adds nothing)

```
<article class="card"><p>Default card text.</p></article>
<article class="card highlight"><p>Highlight card text.</p></article>
<article id="feature" class="card highlight"><p>Feature card text (ID
wins).</p></article>
```

```
<style>
.card p { color: #475569; }
.card.highlight p { color: #2563eb; }
#feature.card p { color: #be123c; } /* highest specificity: wins */
</style>
```

Using the cascade strategically

- Write from general to specific: base styles first, overrides later.
- Prefer classes; avoid IDs and long descendant chains in selectors.
- Avoid !important. If you need it, the real fix is a better selector or order.
- Use @layer to put resets, base, components and utilities in explicit layers.
- Debug in DevTools: the Styles panel shows every matching rule, crossed out when overridden.

Lesson 13: Cascade and inheritance

The cascade decides which declaration wins, in this order: (1) origin and importance (user-agent < user < author; !important reverses), (2) cascade layers, (3) specificity, (4) source order. Inheritance passes some property values from parent to child automatically.

Inherited by default	Not inherited
color, font-*, line-height, text-align, letter-spacing, visibility, cursor, list-style, direction	margin, padding, border, background, width, height, display, position

```
body { color: #1b2431; font-family: Georgia, serif; } /* all text inherits these */
.box { border: 1px solid; } /* children do NOT get a border */

/* keywords that work on every property */
.reset { color: inherit; } /* take the parent's value */
.plain { all: initial; } /* specification default */
.norm { all: unset; } /* inherit if inheritable, otherwise initial */
.rev { all: revert; } /* browser default style */
```

Cascade layers

```
@layer reset, base, components, utilities;

@layer reset { * { margin: 0; box-sizing: border-box; } }
@layer base { body { font: 1rem/1.6 system-ui; } }
@layer components { .btn { padding: .5rem 1rem; } }
@layer utilities { .hidden { display: none; } }
/* later layers beat earlier ones regardless of specificity */
```

Lesson 14: Typography

```
body {
  font-family: 'Inter', system-ui, -apple-system, 'Segoe UI', Roboto,
  sans-serif;
  font-size: clamp(1rem, 0.95rem + 0.3vw, 1.125rem);
  line-height: 1.6;
}
h1 { font-size: clamp(2rem, 1.5rem + 2.5vw, 3.5rem); line-height: 1.1;
  letter-spacing: -0.02em; }
h2 { font-size: 1.75rem; }
p { max-width: 65ch; margin-block: 0 1em; }
.lead { font-size: 1.25rem; color: #475569; }
.caps { text-transform: uppercase; letter-spacing: .08em; font-size: .
8rem; }
.num { font-variant-numeric: tabular-nums; } /* aligned digits in
tables */
```

Loading a web font

```
@font-face {
  font-family: 'Inter';
  src: url('fonts/inter.woff2') format('woff2');
  font-weight: 100 900;
  font-display: swap;
}
```

Or link a font service in the `<head>` with `<link rel="preconnect">` and `<link rel="stylesheet">`. Always list fallback fonts after the web font.

Arabic and right-to-left text

```
<html lang="ar" dir="rtl">
...
body { font-family: 'Noto Naskh Arabic', 'Cairo', 'Segoe UI', Tahoma,
  sans-serif; line-height: 1.8; }
/* use logical properties so layout mirrors automatically */
.card { padding-inline-start: 1rem; margin-inline-end: auto; border-
  inline-start: 4px solid #b90000; }
```

- Font stacks: preferred font first, similar fallbacks next, generic family (serif, sans-serif, monospace) last.
- Line length 45-75 characters; line-height 1.5-1.7 for body text.

- Use rem for font sizes so users can enlarge text in their browser settings.
- Do not justify text on the web; ragged right reads better and avoids rivers.

Lesson 15: Colours and backgrounds

```
:root {
  --brand: #b90000;
  --brand-dark: color-mix(in oklab, var(--brand), black 20%);
  --surface: #fff;
  --ink: #1b2431;
}
.hero {
  background: linear-gradient(120deg, #2563eb 0%, #7c3aed 50%,
#0f172a 100%);
  color: #fff;
}
.banner {
  background-image: url('hero.jpg');
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
.tint { background-color: rgb(185 0 0 / 10%); }
.card { background: var(--surface); box-shadow: 0 14px 28px rgba(15,
23, 42, 0.08); }
```

Contrast

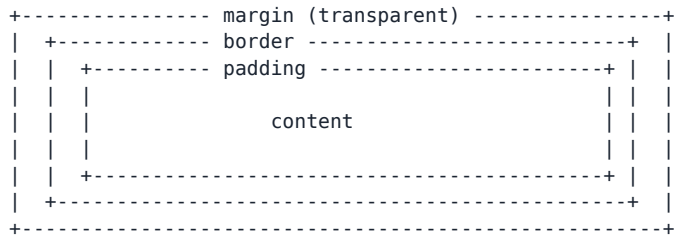
Text must have a contrast ratio of at least 4.5:1 against its background (3:1 for large text). #767676 on white is the lightest grey that passes. Check with the DevTools colour picker or a contrast checker. Never convey information by colour alone.

Dark mode

```
:root { color-scheme: light dark; }
@media (prefers-color-scheme: dark) {
  :root { --surface: #14181f; --ink: #e6e9ef; }
}
body { background: var(--surface); color: var(--ink); }
```

Lesson 16: The box model

Every element is a rectangular box: content in the middle, padding around it, the border wrapping both, and the margin as the outside buffer between neighbours.



content-box versus border-box

By default (box-sizing: content-box) width and height apply to the content only, so a 300px box with 24px padding and a 2px border is really 352px wide. With border-box the declared size includes padding and border: a 300px box stays 300px. Almost every project switches to border-box globally.

```
*, *::before, *::after { box-sizing: border-box; }

.card {
  width: 320px;
  padding: 24px;
  border: 2px solid #cbd5f5;
  margin-bottom: 24px;
}

.callout {
  margin: 32px 0;
  padding: 16px;
  border-left: 6px solid #2563eb;
  background: #eff6ff;
}
```

Shorthands

Declaration	Meaning
margin: 24px;	All four sides
margin: 24px 16px;	top/bottom 24px, left/right 16px

Declaration	Meaning
padding: 8px 12px 16px;	top 8, left/right 12, bottom 16
padding: 1px 2px 3px 4px;	top, right, bottom, left (clockwise)
border: 2px solid #1d4ed8;	width, style, colour
margin: 0 auto;	Centre a block with a width
margin-inline: auto;	Same, using logical properties

Collapsing margins

- Vertical margins between block elements collapse to the largest value (a 24px bottom margin followed by a 16px top margin gives 24px, not 40px).
- A parent and its first/last child can share margins unless padding, a border or display: flow-root intervenes.
- Margins never collapse inside Flexbox or Grid containers - one more reason to use gap.

Visualising with DevTools

Inspect an element in Chrome or Firefox and look at the Computed panel: it shows a diagram with margin, border, padding and content measurements. Hover each part to highlight it on the page.

Spacing strategies

- Create a spacing scale (base of 4 or 8 pixels) and stick to it: 4, 8, 16, 24, 32, 48.
- Store it in custom properties (--space-2: 1rem) for consistency.

- Combine gap with Flexbox or Grid to separate items instead of adding margins to each one.

Lesson 17: Layout fundamentals - normal flow

Without any layout CSS, browsers lay elements out in normal flow: block boxes stack vertically and fill the width; inline boxes flow horizontally inside lines. Every layout technique is a way of taking elements out of, or rearranging, this flow.

display value	Behaviour
block	New line, full width, respects width/height/margins
inline	Flows in text; width/height ignored; only horizontal padding/margin effective
inline-block	Flows in text but respects width/height/vertical margins
flex / inline-flex	Children laid out along one axis
grid / inline-grid	Children placed in rows and columns
flow-root	Block that contains its floats and stops margin collapsing
none	Removed from rendering and from the accessibility tree
contents	Box disappears, children remain

Floats (legacy but still useful for text wrap)

```
img.left { float: left; margin: 0 1rem 1rem 0; }  
.clearfix::after { content: ''; display: block; clear: both; }  
/* modern: */ .wrapper { display: flow-root; }
```

Overflow

```
.box { height: 200px; overflow: auto; }          /* scrollbars when  
needed */  
.clip { overflow: hidden; }                     /* cut off */  
.marquee { white-space: nowrap; overflow-x: auto; }
```

Lesson 18: Positioning and z-index

position	Behaviour
static (default)	Normal flow; top/left have no effect
relative	Normal flow, then nudged by top/left; creates a reference for absolute children
absolute	Removed from flow; placed relative to the nearest positioned ancestor
fixed	Removed from flow; placed relative to the viewport; stays while scrolling
sticky	Normal flow until it reaches the given offset, then sticks (inside its parent)

```
.card { position: relative; }
.card .badge {
  position: absolute; top: 12px; right: 12px;
  background: #b90000; color: #fff; padding: 2px 8px; border-radius:
999px;
}
.site-header { position: sticky; top: 0; z-index: 10; background:
#fff; }
.back-to-top { position: fixed; bottom: 1rem; inset-inline-end: 1rem;
}
.modal-backdrop { position: fixed; inset: 0; background: rgb(0 0 0 /
50%); z-index: 100; }
```

z-index and stacking contexts

- z-index only works on positioned elements (and flex/grid items).
- Higher values are drawn on top. Keep a scale: 10 header, 100 dropdown, 1000 modal.
- opacity < 1, transform, filter and position: fixed create a new stacking context: children cannot escape it no matter how large their z-index.

Lesson 19: Flexbox

Flexbox lays out children in one dimension - a row or a column - and distributes space between them. It is ideal for navigation bars, toolbars, card strips, media objects and form rows.

Creating a flex container

```
.toolbar {
  display: flex;           /* children become flex items in a row */
  gap: 16px;
  align-items: center;
}
.toolbar .spacer { flex: 1 1 auto; } /* pushes following items to
the right */
```

Axes and alignment

- The main axis follows flex-direction (row = horizontal, column = vertical). Distribute items along it with justify-content.
- The cross axis is perpendicular. Align items on it with align-items (all) or align-self (one).
- Use gap for consistent spacing instead of margins.

Property (container)	Values
flex-direction	row row-reverse column column-reverse
flex-wrap	nowrap wrap wrap-reverse
justify-content	flex-start center flex-end space-between space-around space-evenly
align-items	stretch flex-start center flex-end baseline
align-content	Same as justify-content, for multiple lines
gap	row-gap column-gap

Property (item)	Meaning
flex: grow shrink basis	How the item takes and gives up space (default 0 1 auto)
flex: 1	Equal widths, share all free space
flex: 0 0 300px	Fixed 300px, never grows or shrinks
flex: 1 1 220px	Ideal 220px, may grow or shrink; wraps nicely
order	Visual order (default 0); does not change tab order
align-self	Override align-items for one item

Card strip

```
<section class="cards">
  <article><h2>Prototype</h2><p>Validate ideas quickly with
interactive mockups.</p></article>
  <article><h2>Build</h2><p>Create accessible, responsive components
in code.</p></article>
  <article><h2>Ship</h2><p>Automate deployment pipelines and
monitoring.</p></article>
</section>
```

```
<style>
.cards {
  display: flex;
  gap: 24px;
  justify-content: center;
  align-items: stretch;
  flex-wrap: wrap;
}
.cards article {
  flex: 1 1 220px;
  padding: 24px;
  border-radius: 16px;
  background: #f1f5f9;
}
</style>
```

Common patterns

```
/* centre anything */
.center { display: flex; justify-content: center; align-items: center;
min-height: 100vh; }

/* split layout 2:1 */
```

```
.split { display: flex; gap: 2rem; }  
.split > main { flex: 2; }  
.split > aside { flex: 1; }  
  
/* media object that stacks on small screens */  
.media { display: flex; gap: 1rem; flex-wrap: wrap; }  
.media img { flex: 0 0 120px; }  
.media .body { flex: 1 1 240px; }  
  
/* sticky footer */  
body { min-height: 100vh; display: flex; flex-direction: column; }  
main { flex: 1; }
```

Lesson 20: CSS Grid

Grid lays out children in two dimensions at once. You declare tracks (columns and rows) on the container and place items into cells or areas. A track is one column or row - imagine drawing a table with lines; Grid lets you define those lines in CSS.

```
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;          /* two columns */
  grid-template-rows: auto 1fr auto;        /* three rows */
  gap: 24px;
  min-height: 100vh;
}
```

Track sizing

Pattern	Meaning
300px 1fr 1fr	Fixed sidebar, two equal flexible columns
repeat(12, minmax(0, 1fr))	Classic 12-column grid
repeat(auto-fit, minmax(220px, 1fr))	As many 220px+ columns as fit; collapses empty tracks
repeat(auto-fill, minmax(220px, 1fr))	Same but keeps empty tracks
fit-content(300px)	Shrink to content, max 300px
max-content / min-content	Size to the longest / shortest content

Placing items

```
.hero .copy { grid-column: span 6; }          /* auto row, six
columns wide */
.item       { grid-column: 1 / 7; grid-row: 2 / span 2; }
.full      { grid-column: 1 / -1; }          /* first to last line
*/

/* named areas */
.page {
```

```

display: grid;
grid-template-columns: 240px 1fr;
grid-template-areas:
  'header header'
  'nav    main'
  'footer footer';
}
.page > header { grid-area: header; }
.page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }
.page > footer { grid-area: footer; }
@media (max-width: 700px) {
  .page { grid-template-columns: 1fr; grid-template-areas: 'header'
    'nav' 'main' 'footer'; }
}

```

Alignment

- justify-items / align-items align content inside each cell.
- justify-content / align-content distribute the whole grid inside the container.
- place-items and place-content combine both axes:
place-items: center centres everything.

Marketing layout example

```

<section class="hero">
  <div class="copy">
    <h1>Design systems that scale</h1>
    <p>Unify design and engineering with a shared language.</p>
    <a class="cta" href="#">Get started</a>
  </div>
  <div class="visual">
    <h2>Release highlights</h2>
    <ul><li>Theme builder</li><li>Usage analytics</li>
    <li>Accessibility tracking</li></ul>
  </div>
</section>
<section class="features">
  <article><h3>Token management</h3><p>...</p></article>
  <article><h3>Component lifecycles</h3><p>...</p></article>
  <article><h3>Design reviews</h3><p>...</p></article>
</section>

.hero {
  display: grid;

```



```

    grid-template-columns: repeat(12, minmax(0, 1fr));
    gap: 24px;
    padding: 88px clamp(24px, 6vw, 80px);
    background: linear-gradient(120deg, #2563eb 0%, #7c3aed 50%,
#0f172a 100%);
    color: #fff;
}
.hero .copy { grid-column: span 6; max-width: 480px; }
.hero .visual { grid-column: span 6; background: rgba(255,255,255,0.
09); border-radius: 24px; padding: 40px; }
.features {
  display: grid;
  gap: 24px;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  padding: clamp(32px, 5vw, 80px);
}
.features article { border: 1px solid #e2e8f0; border-radius: 20px;
padding: 28px; background: #fff; }
@media (max-width: 900px) {
  .hero .copy, .hero .visual { grid-column: 1 / -1; }
}

```

The hero spans six columns per side on large screens and collapses to full width through the media query; the feature grid adapts its card count automatically.

Lesson 21: Layout patterns

A few patterns cover most pages. Copy them and adjust the numbers.

```
/* 1. Centred content column */
.container { width: min(100% - 2rem, 70rem); margin-inline: auto; }

/* 2. Holy grail: header, sidebar, content, footer */
.holy { display: grid; grid-template-rows: auto 1fr auto; grid-
template-columns: 16rem 1fr; min-height: 100vh; }
.holy > header, .holy > footer { grid-column: 1 / -1; }

/* 3. Auto grid of cards */
.grid { display: grid; gap: 1.5rem; grid-template-columns: repeat(
auto-fit, minmax(16rem, 1fr)); }

/* 4. Sidebar that stacks when narrow (no media query) */
.with-sidebar { display: flex; flex-wrap: wrap; gap: 2rem; }
.with-sidebar > aside { flex: 1 1 15rem; }
.with-sidebar > main { flex: 999 1 25rem; }

/* 5. Cluster of tags/buttons */
.cluster { display: flex; flex-wrap: wrap; gap: .5rem; align-items:
center; }

/* 6. Stack with consistent vertical rhythm */
.stack > * + * { margin-block-start: 1rem; }

/* 7. Cover: hero with centred content */
.cover { min-height: 100vh; display: grid; place-items: center; }

/* 8. Full-bleed section inside a narrow column */
.full-bleed { width: 100vw; margin-inline: calc(50% - 50vw); }
```

Lesson 22: Responsive design

- Add `<meta name="viewport" content="width=device-width, initial-scale=1">` - without it nothing else works on phones.
- Design mobile-first: write styles for narrow screens first, then add min-width media queries for tablets and desktops.
- Use relative units (rem, %, vw, ch) so elements scale naturally.
- A breakpoint is the width where your layout needs to change. Choose breakpoints from your content, not from device names.

Mobile-first media queries

```
:root { --space: clamp(1rem, 2vw, 1.5rem); }
.page { display: grid; gap: var(--space); }
@media (min-width: 768px) {
  .page { grid-template-columns: 240px 1fr; }
}
@media (min-width: 1200px) {
  .page { grid-template-columns: 280px 2fr 1fr; }
}
```

Fluid grids and typography

```
.cards { display: grid; grid-template-columns: repeat(auto-fit,
minmax(220px, 1fr)); gap: 1.5rem; }
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); } /* min,
preferred, max */
p { max-width: 65ch; }
```

Container queries

```
.card-holder { container-type: inline-size; }
@container (min-width: 500px) {
  .card { display: grid; grid-template-columns: 160px 1fr; }
}
```

Container queries watch the size of a parent element instead of the whole screen, so a component adapts wherever it is placed.

Responsive media

```
img, video, iframe { max-width: 100%; height: auto; }  
.video-wrap { aspect-ratio: 16 / 9; }  
.video-wrap iframe { width: 100%; height: 100%; }
```

Other media features

```
@media (prefers-reduced-motion: reduce) { *, *::before, *::after {  
  animation: none !important; transition: none !important; } }  
@media (prefers-color-scheme: dark) { ... }  
@media (hover: hover) { .card:hover { transform: translateY(-4px); }  
} /* only devices with a mouse */  
@media print { nav, footer { display: none; } }  
@media (orientation: landscape) { ... }
```

Testing

- Use DevTools device mode (Ctrl+Shift+M) to preview breakpoints.
- Test on a real phone when possible; check touch target size (at least 44x44px) and both orientations.
- Check text at 200% zoom: nothing should overlap or be cut off.

Lesson 23: Responsive components

```
/* Navigation that becomes a hamburger menu without JavaScript */
<nav class="nav">
  <input type="checkbox" id="menu" class="nav__toggle" hidden>
  <label for="menu" class="nav__button" aria-label="Menu">#9776;</
label>
  <ul class="nav__list">
    <li><a href="/">Home</a></li>
    <li><a href="/books/">Books</a></li>
    <li><a href="/contact/">Contact</a></li>
  </ul>
</nav>

<style>
.nav__list { list-style: none; margin: 0; padding: 0; display: none; }
.nav__toggle:checked ~ .nav__list { display: block; }
@media (min-width: 700px) {
  .nav__button { display: none; }
  .nav__list { display: flex !important; gap: 1rem; }
}
</style>

/* Responsive table (see the Lists and tables chapter) */
.table-wrapper { overflow-x: auto; }

/* Responsive card: image beside text when there is room */
.card { display: grid; gap: 1rem; }
@media (min-width: 480px) { .card { grid-template-columns: 200px
1fr; } }

/* Responsive embed */
.embed { aspect-ratio: 16 / 9; width: 100%; }

/* Responsive form: labels beside fields on wide screens */
.form-row { display: grid; gap: .5rem; }
@media (min-width: 600px) { .form-row { grid-template-columns: 10rem
1fr; align-items: center; } }
```

Lesson 24: Custom properties (CSS variables)

Custom properties store a value under a name that starts with two dashes. Read it with `var()`. They cascade and inherit like other properties, can be changed in media queries or on a class, and can be read and set from JavaScript.

```
:root {
  --brand: #b90000;
  --radius: 12px;
  --shadow: 0 10px 25px rgb(15 23 42 / 8%);
}
.btn { background: var(--brand); border-radius: var(--radius); }
.card { box-shadow: var(--shadow); padding: var(--card-padding, 1rem);
} /* fallback value */

/* theming by scope */
.theme-green { --brand: #04AA6D; }

/* change on state */
.btn:hover { --brand: #900; }

/* JavaScript */
document.documentElement.style.setProperty('--brand', '#0055aa');
```

Typed properties with `@property`

```
@property --angle {
  syntax: '<angle>';
  inherits: false;
  initial-value: 0deg;
}
.spin { background: conic-gradient(from var(--angle), red, blue);
transition: --angle 1s; }
.spin:hover { --angle: 360deg; }
```

Lesson 25: calc() and CSS functions

Function	Purpose	Example
calc()	Arithmetic mixing units	width: calc(100% - 2rem)
min()	Smallest of several values	width: min(90%, 60rem)
max()	Largest of several values	font-size: max(1rem, 2vw)
clamp()	Value between a minimum and maximum	font-size: clamp(1rem, 2vw, 1.5rem)
var()	Read a custom property	color: var(--brand, black)
attr()	Read an HTML attribute (content only)	content: attr(data-label)
url()	Reference a file	background: url('bg.png')
rgb() hsl() oklch() color-mix()	Colours	color-mix(in oklab, red, white 30%)
linear-gradient() radial-gradient() conic-gradient()	Gradient images	background: linear-gradient(90deg, red, blue)
repeat() minmax() fit-content()	Grid tracks	repeat(auto-fit, minmax(200px, 1fr))
translate() rotate() scale()	Transforms	transform: rotate(45deg)
counter()	Generated numbering	content: counter(item)

Function	Purpose	Example
env()	Environment values (phone notches)	padding-bottom: env(safe-area-inset-bottom)
round() mod() rem() abs() sign()	Maths	width: round(down, 33.3%, 1px)

```
.sidebar { width: calc(100% / 3 - 1rem); }
.container { width: min(100% - 2rem, 70rem); }
h1 { font-size: clamp(1.75rem, 1rem + 3vw, 3rem); }
.hero { min-height: calc(100vh - var(--header-height)); }
```


Lesson 26: Transitions

A transition animates a property smoothly from its old value to its new value when the value changes (on hover, focus, class change...). Set it on the element's normal state, not on the `:hover` state, so it plays both ways.

```
.btn {  
    background: #b90000;  
    transition: background-color .2s ease, transform .15s ease;  
}  
.btn:hover { background: #900; transform: translateY(-2px); }  
  
/* longhands */  
.panel {  
    transition-property: opacity, transform;  
    transition-duration: 300ms;  
    transition-timing-function: cubic-bezier(.2, .8, .2, 1);  
    transition-delay: 0s;  
}
```

Timing function	Feel
ease (default)	Fast start, slow end
linear	Constant speed
ease-in	Slow start
ease-out	Slow end (best for entering)
ease-in-out	Slow both ends
steps(4)	Jumps in 4 steps
cubic-bezier(x1,y1,x2,y2)	Custom curve

- Animate only transform and opacity when possible; they are cheap for the browser. Animating width, height or top causes layout work.
- Keep durations short: 150-300ms for interface feedback.
- Respect prefers-reduced-motion.

- display cannot be transitioned; use opacity/visibility or the newer transition-behavior: allow-discrete with @starting-style.

Lesson 27: Transforms

```
.a { transform: translate(20px, 10px); }      /* move */
.b { transform: rotate(15deg); }             /* turn */
.c { transform: scale(1.1); }                /* grow 10% */
.d { transform: skewX(10deg); }              /* slant */
.e { transform: rotate(45deg) scale(.8); }    /* combined, applied
right to left */
.f { transform-origin: top left; }            /* pivot point */

/* individual properties (no ordering problems) */
.g { translate: 0 -4px; rotate: 3deg; scale: 1.05; }

/* 3D */
.scene { perspective: 800px; }
.card3d { transform: rotateY(30deg); transform-style: preserve-3d;
backface-visibility: hidden; }
```

- Transforms do not affect the layout of other elements; the box keeps its original space.
- They create a stacking context and a containing block for fixed children.
- Combine with transition for hover lift effects: `.card:hover { transform: translateY(-4px); }`

Lesson 28: Animations

```
@keyframes fade-up {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: none; }
}
.reveal { animation: fade-up .5s ease-out both; }

@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%     { transform: scale(1.06); }
}
.badge { animation: pulse 1.5s ease-in-out infinite; }

/* longhands */
.spinner {
  animation-name: spin;
  animation-duration: 1s;
  animation-timing-function: linear;
  animation-iteration-count: infinite;
  animation-direction: normal;      /* reverse | alternate */
  animation-fill-mode: forwards;    /* keep final state */
  animation-play-state: running;    /* paused */
  animation-delay: 0s;
}
@keyframes spin { to { transform: rotate(1turn); } }

/* scroll-driven (new) */
.progress { animation: grow linear; animation-timeline: scroll(); }
@keyframes grow { from { scale: 0 1; } to { scale: 1 1; } }
```

- Use transitions for state changes and animations for things that run on their own or loop.
- Pause or remove animations for users who set `prefers-reduced-motion`.
- Avoid animating more than a few elements at once on mobile.

Lesson 29: Effects and filters

```
.card { box-shadow: 0 1px 2px rgb(0 0 0 / 8%), 0 8px 24px rgb(0 0 0 / 12%); }
.text { text-shadow: 0 1px 2px rgb(0 0 0 / 40%); }
.round { border-radius: 16px; } /* 50% for a circle */
.glass { background: rgb(255 255 255 / 60%); backdrop-filter: blur(12px); }
.grey { filter: grayscale(100%); }
.grey:hover { filter: none; }
.dim { filter: brightness(.8) contrast(1.1) saturate(1.2); }
.blurred { filter: blur(4px); }
.shadowed-png { filter: drop-shadow(0 4px 6px rgb(0 0 0 / 30%)); } /* follows transparent shape */
.fade { opacity: .6; }
.blend { mix-blend-mode: multiply; }
.gradient-text {
  background: linear-gradient(90deg, #b90000, #ff7a00);
  -webkit-background-clip: text; background-clip: text; color: transparent;
}
.clipped { clip-path: circle(50%); }
.cut { clip-path: polygon(0 0, 100% 0, 100% 80%, 0 100%); }
```

Lesson 30: Accessibility

- Use semantic HTML first: headings, lists, tables, buttons, links and landmarks give screen readers structure for free.
- Every image needs alt text; decorative images use alt="".
- Every form control needs a label. Error messages must be text, associated with the field via aria-describedby.
- Keyboard: everything clickable must be reachable with Tab and operable with Enter/Space. Never remove :focus outlines without a visible replacement (:focus-visible).
- Colour contrast at least 4.5:1 for text, 3:1 for large text and UI components. Do not use colour alone to convey meaning.
- Set lang on <html>; use dir="rtl" for Arabic and Hebrew.
- Respect user preferences: prefers-reduced-motion, prefers-color-scheme, prefers-contrast.
- Use ARIA only when no native element does the job. aria-label, aria-labelledby, aria-describedby, aria-current, aria-expanded and role are the most common.
- Hide content properly: display: none / hidden attribute hides from everyone; the .sr-only pattern hides visually but keeps it for screen readers.
- Test with the keyboard alone, with a screen reader (NVDA on Windows, VoiceOver on Mac/iOS, TalkBack on Android) and with an automatic checker (Lighthouse, axe).

```
.sr-only {  
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -  
1px;  
  overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; border:  
0;  
}  
.skip-link { position: absolute; top: -100px; }  
.skip-link:focus { top: 0; }
```

```
<a class="skip-link" href="#main">Skip to content</a>  
<button aria-expanded="false" aria-controls="menu">Menu</button>  
<ul id="menu" hidden>...</ul>
```

Lesson 31: Design systems

A design system is the set of tokens (colours, spacing, type scale), components and rules a site is built from. Even a small site benefits: decide once, reuse everywhere.

```
:root {
  /* colour tokens */
  --c-brand: #b90000; --c-ink: #1b2431; --c-muted: #5b6472; --c-line:
#d9d2c3; --c-surface: #fff; --c-bg: #faf8f3;
  /* type scale (1.25 ratio) */
  --fs-0: 1rem; --fs-1: 1.25rem; --fs-2: 1.563rem; --fs-3: 1.953rem; -
-fs-4: 2.441rem;
  /* spacing scale */
  --s-1: .25rem; --s-2: .5rem; --s-3: 1rem; --s-4: 1.5rem; --s-5:
2rem; --s-6: 3rem;
  /* shape */
  --radius: 12px; --shadow: 0 10px 25px rgb(15 23 42 / 8%);
}
```

Naming conventions

Convention	Example	Idea
BEM	.card, .card__title, .card--featured	Block, element, modifier
Utility classes	.mt-3, .flex, .text-center	One class = one declaration
State classes	.is-open, .is-active	Toggled by JavaScript
Data attributes	[data-state="open"]	State without class juggling

File organisation

```
css/
  reset.css      /* box-sizing, margins, images */
  tokens.css     /* :root variables */
  base.css       /* body, headings, links, tables, forms */
  components/    /* card.css, button.css, nav.css */
  utilities.css  /* .sr-only, .container, .stack */

/* one entry file */
@layer reset, tokens, base, components, utilities;
@import url('reset.css') layer(reset);
```



```
@import url('tokens.css') layer(tokens);  
...
```

Lesson 32: Performance

- Put CSS in `<head>` (render-blocking but needed first) and scripts at the end of `<body>` or with `defer`.
- Compress images: AVIF/WebP, correct dimensions, `loading="lazy"` below the fold, `width/height` attributes to avoid layout shift.
- Use font-display: swap and preload the main font: `<link rel="preload" href="font.woff2" as="font" type="font/woff2" crossorigin>`.
- Keep CSS small: remove unused rules, avoid deep selectors, use one style sheet per site.
- Prefer transform/opacity animations; use `will-change` sparingly.
- content-visibility: auto on long lists skips rendering off-screen sections.
- Preconnect to third-party origins you will use soon: `<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>`.
- Measure with Lighthouse (DevTools > Lighthouse) and aim for good Core Web Vitals: LCP under 2.5s, CLS under 0.1, INP under 200ms.

```
<link rel="preload" as="image" href="hero.avif" fetchpriority="high">


<script src="app.js" defer></script>
```

Lesson 33: Debugging and testing

DevTools checklist

- Elements panel: is the HTML what you expected? Unclosed tags show up as unexpected nesting here.
- Styles panel: which rules apply? Crossed-out declarations are overridden (specificity/order) or invalid (typo). A yellow warning triangle means the property or value is unknown.
- Computed panel: the final value of every property and the box model diagram.
- Toggle :hov to force :hover/:focus states; edit values live and copy them back to your file.
- Console: red messages for 404 (file not found - check paths and spelling) and script errors.
- Network panel: is styles.css actually loading? Status 200 is good; 404 means the path is wrong.

Common CSS bugs and fixes

Symptom	Likely cause	Fix
Nothing changes	Style sheet not loaded, cached, or selector does not match	Check Network panel; hard reload (Ctrl+F5); inspect the element
Property ignored	Typo, invalid value, or property does not apply to that display type	Look for the warning icon; check width on inline elements, vertical-align on blocks
Margin does not work	Margin collapse or inline element	Use padding, display: block/flow-root, or gap
Element wider than screen	100% width plus padding (content-box)	box-sizing: border-box; max-width: 100%

Symptom	Likely cause	Fix
z-index has no effect	Element not positioned or trapped in a stacking context	Add position: relative; move it out of the transformed/opacity parent
Centre does not work	margin: auto on an element without width, or on inline	Set a width or use flex/grid centring
Text overflows box	Long word/URL	overflow-wrap: anywhere; or <wbr>
Font not applied	Wrong name, quotes, or font not loaded	Check @font-face URL in Network panel; quote multi-word names

Validate and test

- HTML validator: <https://validator.w3.org/> - CSS validator: <https://jigsaw.w3.org/css-validator/>
- Test in at least Chrome and Firefox; check Safari on an iPhone if you can.
- Test keyboard navigation and zoom to 200%.
- Print preview (Ctrl+P) to check @media print styles.

Lesson 34: Tooling and automation (optional)

None of this is required to build a site, but as projects grow these tools save time.

Tool	What it does
Editor extensions (Prettier, Emmet, Live Server)	Auto-format, expand abbreviations (ul>li*3), reload the browser on save
Git	Version control: keep history, undo mistakes, collaborate
Sass / PostCSS	Pre-processors adding nesting, mixins, autoprefixing (CSS now has native nesting)
Stylelint / HTMLHint	Catch errors and enforce style rules automatically
Lighthouse	Audits for performance, accessibility, SEO
Bundlers (Vite, esbuild)	Combine and minify files for production
Static hosting (GitHub Pages, Netlify)	Publish a folder of HTML/CSS for free

Native CSS nesting

```
.card {  
  padding: 1rem;  
  & h3 { margin: 0; }  
  &:hover { box-shadow: var(--shadow); }  
  & .btn { margin-top: .5rem; }  
  @media (min-width: 600px) { padding: 2rem; }  
}
```

Lesson 35: Project - a complete landing page

This project combines everything: semantic structure, a responsive navigation, a hero built with Grid, a features section with Flexbox cards, a pricing table, a form and a footer. Copy both files, open index.html, then change the words, colours and images.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Cairo Book Shop - Learn HTML and CSS</title>
  <meta name="description" content="PDF and printed books about web
development, in Arabic and English.">
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header class="site-header">
    <a class="logo" href="/">Cairo Book Shop</a>
    <nav aria-label="Main">
      <ul class="nav">
        <li><a href="#features">Why us</a></li>
        <li><a href="#pricing">Pricing</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main id="main">
    <section class="hero">
      <div class="hero__copy">
        <h1>Learn HTML and CSS from zero</h1>
        <p class="lead">Clear books, real examples, Arabic and
English editions.</p>
        <a class="btn" href="#pricing">See the books</a>
      </div>
      
    </section>

    <section id="features" class="features">
      <h2>Why readers choose us</h2>
      <div class="cards">
        <article class="card"><h3>Step by step</h3><p>Every chapter
```

```

builds on the last.</p></article>
    <article class="card"><h3>Searchable PDF</h3><p>Find any tag
or property in seconds.</p></article>
    <article class="card"><h3>Free updates</h3><p>New editions
delivered to your inbox.</p></article>
</div>
</section>

<section id="pricing" class="pricing">
    <h2>Pricing</h2>
    <div class="table-wrapper">
        <table>
            <caption>Editions and prices</caption>
            <thead><tr><th scope="col">Edition</th><th
scope="col">Format</th><th scope="col">Price</th></tr></thead>
            <tbody>
                <tr><th scope="row">Starter</th><td>PDF</td><td>120 EGP</
td></tr>
                <tr><th scope="row">Complete</th><td>PDF + print</
td><td>350 EGP</td></tr>
            </tbody>
        </table>
    </div>
</section>

<section id="contact" class="contact">
    <h2>Contact us</h2>
    <form action="/contact" method="post" class="form">
        <div class="form-row"><label for="n">Name</label><input
id="n" name="name" required autocomplete="name"></div>
        <div class="form-row"><label for="e">Email</label><input
id="e" name="email" type="email" required autocomplete="email"></div>
        <div class="form-row"><label for="m">Message</label><textarea
id="m" name="message" rows="4" required></textarea></div>
        <button class="btn" type="submit">Send</button>
    </form>
</section>
</main>

<footer class="site-footer">
    <p>&copy; 2026 Cairo Book Shop &middot; <a href="/privacy/
">Privacy</a> &middot; <a href="mailto:info@example.com">info@example.
com</a></p>
</footer>
</body>
</html>

/* styles.css */
@layer reset, base, layout, components;

```

```

@layer reset {
  *, *::before, *::after { box-sizing: border-box; }
  body, h1, h2, h3, p, ul { margin: 0; }
  img { max-width: 100%; height: auto; display: block; }
}

@layer base {
  :root {
    --brand: #b90000; --ink: #1b2431; --muted: #5b6472; --line:
#d9d2c3; --bg: #faf8f3; --surface: #fff;
    --radius: 14px; --shadow: 0 10px 25px rgb(15 23 42 / 8%);
    --space: clamp(1rem, 2vw, 1.5rem);
  }
  body { font: 1.05rem/1.6 Georgia, serif; color: var(--ink);
background: var(--bg); }
  h1, h2, h3 { font-family: system-ui, sans-serif; line-height: 1.15;
}
  h1 { font-size: clamp(2rem, 1.4rem + 3vw, 3.25rem); }
  h2 { font-size: 1.75rem; margin-bottom: var(--space); }
  a { color: var(--brand); }
  a:focus-visible { outline: 3px solid #ffbf47; outline-offset: 2px; }
  .lead { font-size: 1.2rem; color: var(--muted); }
}

@layer layout {
  .site-header { position: sticky; top: 0; z-index: 10; background:
var(--surface); border-bottom: 1px solid var(--line);
    display: flex; flex-wrap: wrap; gap: var(--space);
align-items: center; justify-content: space-between; padding: .75rem
var(--space); }
  .nav { display: flex; gap: .25rem; list-style: none; padding: 0; }
  .nav a { display: block; padding: .5rem .9rem; border-radius: 999px;
text-decoration: none; color: var(--ink); }
  .nav a:hover { background: var(--bg); }
  main > section { padding: clamp(2rem, 6vw, 5rem) var(--space); max-
width: 70rem; margin-inline: auto; }
  .hero { display: grid; gap: var(--space); align-items: center; }
  @media (min-width: 800px) { .hero { grid-template-columns: 1.1fr
1fr; } }
  .hero__image { border-radius: var(--radius); box-shadow: var(--
shadow); }
  .cards { display: flex; flex-wrap: wrap; gap: var(--space); }
  .card { flex: 1 1 220px; }
  .site-footer { border-top: 1px solid var(--line); padding: var(--
space); text-align: center; color: var(--muted); font-size: .9rem; }
}

@layer components {
  .btn { display: inline-block; padding: .7rem 1.4rem; border-radius:

```



```

999px; border: 0; background: var(--brand); color: #fff;
    font: inherit; font-weight: 600; text-decoration: none;
cursor: pointer; transition: transform .15s, background-color .2s; }
    .btn:hover { background: #900; transform: translateY(-2px); }
    .card { background: var(--surface); border: 1px solid var(--line);
border-radius: var(--radius); padding: var(--space); box-shadow: var(-
-shadow); }
    .card h3 { margin-bottom: .5rem; }
    .table-wrapper { overflow-x: auto; }
    table { width: 100%; border-collapse: collapse; background: var(--
surface); }
    caption { text-align: left; padding: .5rem 0; color: var(--muted); }
    th, td { padding: .8rem 1rem; text-align: left; border-bottom: 1px
solid var(--line); }
    thead th { background: var(--brand); color: #fff; }
    tbody tr:nth-child(even) { background: #fcfbf8; }
    .form { display: grid; gap: 1rem; max-width: 36rem; }
    .form-row { display: grid; gap: .3rem; }
    @media (min-width: 600px) { .form-row { grid-template-columns: 8rem
1fr; align-items: start; } }
    label { font-weight: 600; }
    input, textarea { width: 100%; padding: .6rem .8rem; border: 1px
solid #bbb; border-radius: 8px; font: inherit; }
    input:focus-visible, textarea:focus-visible { outline: 3px solid
#ffbf47; border-color: var(--brand); }
    .skip-link { position: absolute; top: -100px; left: 1rem;
background: var(--brand); color: #fff; padding: .5rem 1rem; }
    .skip-link:focus { top: 1rem; }
    @media (prefers-reduced-motion: reduce) { .btn { transition: none;
} }
}

```

Exercises

- Change the colour tokens to your own brand and watch every component update.
- Add a fourth card and confirm the row wraps correctly on a narrow window.
- Add an Arabic version: set lang="ar" dir="rtl" and replace left/right properties with logical ones (inline-start / inline-end).
- Add a dark mode with prefers-color-scheme.

- Run the page through the HTML validator and Lighthouse; fix every warning.

Lesson 36: Quick quiz and answers

Question	Answer
Which element creates an ordered list?	<code></code> (with <code></code> items)
Which attribute connects a <code><label></code> to its control?	<code>for</code> , matching the control's id
What does border-collapse: collapse do?	Merges adjacent cell borders into single lines
What is the specificity of <code>#a .b p</code> ?	1,1,1
Which CSS property makes a flex container?	<code>display: flex</code>
How do you make images never overflow?	<code>img { max-width: 100%; height: auto; }</code>
What does the viewport meta tag do?	Makes the page use the real device width on phones
Which unit is relative to the root font size?	<code>rem</code>
What does gap do?	Adds space between flex/grid items without margins
How do you centre a block horizontally?	Give it a width and <code>margin-inline: auto</code> (or use flex/grid)
Where should <code><main></code> appear?	Once per page, containing the unique content
What is the difference between <code></code> and <code></code> ?	<code></code> means importance; <code></code> is only visual/keyword

Question	Answer
Which selector matches every second row?	tr:nth-child(even)
What does box-sizing: border-box change?	width/height include padding and border
What replaces the bgcolor attribute?	CSS background-color

Practical Guides

Step-by-step guidance for pages, lists, tables, forms, media, accessibility, SEO and migration.

Building a complete page step by step

This guide walks through building a small, correct web page using the elements from this book, in the order you would type them.

Step 1: The skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Book Store</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

</body>
</html>
```

Step 2: Landmarks

```
<body>
  <header>
    <h1>My Book Store</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="index.html" aria-current="page">Home</a></li>
        <li><a href="books.html">Books</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <!-- unique content goes here -->
  </main>

  <footer>
    <p><small>&copy; 2026 My Book Store</small></p>
  </footer>
</body>
```

Step 3: Content

```
<main>
  <section aria-labelledby="new">
    <h2 id="new">New this week</h2>
    <article class="card">
      
      <h3>Learn HTML</h3>
      <p>A complete beginner's guide. <span class="price">120 EGP</
span></p>
      <button type="button">Add to cart</button>
    </article>
  </section>

  <section aria-labelledby="prices">
    <h2 id="prices">Price list</h2>
    <table>
      <caption>PDF books</caption>
      <thead><tr><th scope="col">Title</th><th scope="col">Price (
EGP)</th></tr></thead>
      <tbody>
        <tr><td>Learn HTML</td><td>120</td></tr>
        <tr><td>Learn CSS</td><td>140</td></tr>
      </tbody>
    </table>
  </section>
</main>
```

Step 4: A form

```
<section aria-labelledby="contact">
  <h2 id="contact">Contact us</h2>
  <form action="/contact" method="post">
    <div class="field">
      <label for="name">Name</label>
      <input id="name" name="name" required autocomplete="name">
    </div>
    <div class="field">
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required
autocomplete="email">
    </div>
    <div class="field">
      <label for="msg">Message</label>
      <textarea id="msg" name="message" rows="4" required></textarea>
    </div>
    <button type="submit">Send</button>
  </form>
</section>
```

Step 5: The style sheet

```
/* style.css */
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, Arial, sans-serif; line-height: 1.6; color: #1b2431; background: #f6f1e7; }
header, main, footer { padding: 1rem; }
header { background: #fff; box-shadow: 0 1px 0 #e7ddcb; }
nav ul { display: flex; gap: 1rem; list-style: none; padding: 0; margin: 0; }
nav a[aria-current="page"] { font-weight: bold; }
main { max-width: 960px; margin: 0 auto; }
.card { background: #fff; border-radius: 12px; padding: 1rem; display: grid; gap: 0.5rem; max-width: 260px; }
.price { color: #e60000; font-weight: bold; }
table { border-collapse: collapse; width: 100%; background: #fff; }
th, td { padding: 0.5rem; border-bottom: 1px solid #e7ddcb; text-align: left; }
.field { display: grid; gap: 0.25rem; margin-bottom: 1rem; }
input, textarea { padding: 0.5rem; border: 1px solid #ccc; border-radius: 6px; font: inherit; }
button { background: #e60000; color: #fff; border: 0; border-radius: 6px; padding: 0.6rem 1.2rem; cursor: pointer; }
footer { text-align: center; color: #6b7280; }
```

Step 6: Check it

- Open the file in a browser and resize the window.
- Press Tab repeatedly: every link, input and button should receive a visible focus ring in a sensible order.
- Validate at validator.w3.org.
- Run Lighthouse (in Chrome DevTools) for accessibility and performance scores.

Lists in depth

HTML has three list elements: `<ul>` for unordered lists, `<ol>` for ordered lists and `<dl>` for description lists. Items of `ul` and `ol` are `<li>`; a `dl` uses `<dt>` for terms and `<dd>` for descriptions.

Choosing the right list

Question	Use
Does the order matter (steps, ranking)?	<code></code>
Is it a set of items where order is irrelevant?	<code></code>
Are they name-value pairs (term and definition, label and data)?	<code><dl></code>
Is it a toolbar of actions?	<code><menu></code>
Is it navigation links?	<code><nav></code> containing a <code></code>

Numbering control on ordered lists

Attribute / CSS	Effect	Example
<code>type="1"</code>	1, 2, 3 (default)	<code><ol type="1"></code>
<code>type="a" / "A"</code>	a, b, c / A, B, C	<code><ol type="A"></code>
<code>type="i" / "I"</code>	i, ii, iii / I, II, III	<code><ol type="I"></code>
<code>start="5"</code>	Begin at 5	<code><ol start="5"></code>
<code>reversed</code>	Count down	<code><ol reversed></code>

Attribute / CSS	Effect	Example
<code><li value="10"></code>	Set this item's number	
<code>list-style-type: lower-greek</code>	CSS: alpha, beta, gamma	<code>ol { list-style-type: lower-greek; }</code>
<code>list-style-type: arabic-indic</code>	CSS: Arabic-Indic digits	<code>ol { list-style-type: arabic-indic; }</code>

CSS list-style-type values

For `<ul>`: disc (default), circle, square, none, or any string such as '→ '. For `<ol>`: decimal, decimal-leading-zero, lower-roman, upper-roman, lower-alpha, upper-alpha, lower-greek, armenian, georgian, arabic-indic, persian, bengali, devanagari, cjk-decimal, japanese-informal, and more. `list-style-position: inside` puts the marker inside the item's box; `outside` (default) hangs it in the margin. `list-style-image: url(icon.svg)` uses an image; `::marker` is the modern way to style markers.

Nested lists

```
<ol>
  <li>Prepare
    <ul>
      <li>Install an editor</li>
      <li>Create a folder</li>
    </ul>
  </li>
  <li>Write HTML
    <ol type="a">
      <li>Skeleton</li>
      <li>Content</li>
    </ol>
  </li>
</ol>
```

Description lists for metadata

```
<dl>
```

```
<dt>Author</dt><dd>Sara Ali</dd>
<dt>Published</dt><dd><time datetime="2026-09">September 2026</
time></dd>
<dt>Pages</dt><dd>2000</dd>
</dl>
```

Common mistakes

- Text directly inside `<ul>` or `<ol>` that is not in an `<li>`.
- A nested list placed between `<li>` elements instead of inside one.
- Using `<br>` or paragraphs starting with '-' instead of a real list.
- Using `<dl>` for dialogue or for layout.

Tables in depth

Use tables for data whose meaning depends on rows and columns. A well-built table has a caption, header cells with scope, and grouped rows.

Element roles

Element	Role
<code><table></code>	The table container
<code><caption></code>	Title, first child
<code><colgroup></code> / <code><col></code>	Column groups for styling
<code><thead></code>	Header rows
<code><tbody></code>	Data rows (one or more)
<code><tfoot></code>	Summary rows
<code><tr></code>	A row
<code><th></code>	Header cell, with scope=col row colgroup rowgroup
<code><td></code>	Data cell

colspan and rowspan

`colspan="n"` makes a cell span n columns; `rowspan="n"` makes it span n rows. Every row must still add up to the same number of columns. Sketch the grid on paper for complex tables.

```
<table>
  <tr><th rowspan="2">Product</th><th colspan="2">Sales</th></tr>
  <tr><th>2025</th><th>2026</th></tr>
  <tr><td>Books</td><td>120</td><td>150</td></tr>
</table>
```

HTML 3.2/4 attributes and their CSS replacements

Old attribute	CSS replacement
<code>border="1"</code>	<code>th, td { border: 1px solid; }</code>
<code>cellpadding="5"</code>	<code>th, td { padding: 5px; }</code>
<code>cellspacing="0"</code>	<code>table { border-collapse: collapse; }</code> or <code>border-spacing: 0</code>
<code>width="100%"</code>	<code>table { width: 100%; }</code>
<code>bgcolor="#eee"</code>	<code>background-color: #eee</code>
<code>align="center" on table</code>	<code>table { margin: 0 auto; }</code>
<code>align="right" on td</code>	<code>td { text-align: right; }</code>
<code>valign="top"</code>	<code>vertical-align: top</code>
<code>nowrap</code>	<code>white-space: nowrap</code>
<code>summary="..."</code>	<code><caption></code> or <code>aria-describedby</code>

Making tables responsive

- Wrap the table in a div with `overflow-x: auto` so it scrolls horizontally on phones.
- Hide less important columns on small screens with a media query.
- Stack cells into blocks with `display: block` and `data-label` attributes (see the CSS cookbook).
- Use `font-variant-numeric: tabular-nums` so digits line up.

Sorting and filtering

HTML has no built-in sorting. Add it with JavaScript by reading rows from `tbody`, sorting the array and re-appending, and mark the sorted column header with `aria-sort="ascending"` or `"descending"`.

Forms in depth

A form collects input and sends it to a server or a script. Building a good form is mostly about labels, the right input types, validation and clear feedback.

Checklist

- Every control has a `<label>` linked by `for/id` or wrapping.
- Every control that must be submitted has a name.
- Use the most specific type: email, tel, url, number, date, and so on.
- Use autocomplete tokens so browsers can fill in addresses and cards.
- Group radio buttons and checkboxes with `<fieldset>` and `<legend>`.
- Mark required fields with `required` and say so visibly.
- Use `pattern` with a title explaining the format.
- Show errors next to the field and link them with `aria-describedby`; do not rely on colour alone.
- Use `method="post"` for anything that changes data; `enctype="multipart/form-data"` for files.
- Validate again on the server; client validation is only a convenience.

GET versus POST

	GET	POST
Data location	In the URL (<code>?a=1&b=2</code>)	In the request body
Bookmarkable	Yes	No

	GET	POST
Size limit	About 2000 characters	Practically none
Use for	Searches, filters	Logins, orders, uploads, anything with side effects
Security	Visible in history and logs	Not in URL (still needs HTTPS)

Built-in validation attributes

Attribute	Applies to	Checks
required	Most inputs, select, textarea	Not empty / selected
minlength / maxlength	Text-like inputs, textarea	Length
min / max	number, range, date, time...	Range
step	number, range, date, time...	Granularity
pattern	text, search, url, tel, email, password	Regular expression (whole value)
type=email / url	input	Format

Styling validation states

```
input:required { border-left: 3px solid #999; }
input:user-invalid { border-color: #b90000; }
input:user-valid { border-color: #0f8a4c; }
.error { color: #b90000; font-size: 0.9em; }
```

Custom validation messages

```
<input id="zip" pattern="[0-9]{5}" required>
<script>
  const zip = document.getElementById('zip');
  zip.addEventListener('invalid', () => zip.setCustomValidity('Enter
```

```
exactly five digits.'));  
    zip.addEventListener('input', () => zip.setCustomValidity(''));  
</script>
```


Semantic layout and accessibility

Semantic HTML means choosing elements for what they mean, not how they look. It is the foundation of accessibility: screen readers, keyboard navigation, reader modes and search engines all depend on it.

Landmark map

Element	Landmark role	How many per page
<header> (top level)	banner	1
<nav>	navigation	Several; label each
<main>	main	Exactly 1
<aside>	complementary	Several
<footer> (top level)	contentinfo	1
<form> with a name	form	Several
<search>	search	Several
<section> with a name	region	Several

Heading outline

One <h1> per page, then h2 for sections, h3 inside them and so on, never skipping levels. Users of screen readers navigate by headings more than by any other means.

Images

- Informative image: alt describes the content.

- Functional image (in a link or button): alt describes the action.
- Decorative image: alt="" (empty).
- Complex image (chart): short alt plus a full description in text or a linked page.

Keyboard

- Everything clickable must be reachable with Tab and operable with Enter or Space: use `<a>`, `<button>` and form controls, not `<div onclick>`.
- Never remove the focus outline without providing a visible replacement (`:focus-visible`).
- Provide a skip link to `<main>`.
- Manage focus in dialogs and after dynamic changes.

Colour and motion

- Text contrast at least 4.5:1 (3:1 for large text).
- Never convey information by colour alone.
- Respect `prefers-reduced-motion`; avoid flashing content.

ARIA in three rules

- Do not use ARIA if a native element does the job.
- Do not change native semantics unless you must.
- All interactive ARIA controls must be keyboard usable.

Media: images, audio and video

Choosing an image format

Format	Best for	Notes
AVIF	Photos, smallest files	Newest; use with fallback
WebP	Photos and graphics	Widely supported
JPEG	Photos, universal fallback	No transparency
PNG	Screenshots, logos with transparency	Large for photos
SVG	Icons, logos, diagrams	Scales perfectly; can be styled
GIF	Simple animations	Prefer video or animated WebP/AVIF

Responsive image decision

- Same image, different sizes: `<img srcset sizes>`.
- Different crops or formats: `<picture>` with `<source>`.
- High-DPI only: `srcset` with `1x/2x`.
- Always: `alt`, `width`, `height`, and `loading="lazy"` below the fold.

Audio and video checklist

- Provide MP4 (H.264) plus WebM.
- Add controls.
- Add `<track kind="captions">`.

- Set poster and preload="metadata".
- For background video: autoplay muted loop playsinline, and pause it for prefers-reduced-motion.
- Provide a transcript.

Embedding third-party media

YouTube, Vimeo and maps are embedded with `<iframe>`.

Always add title, loading="lazy" and only the allow permissions you need. Consider a click-to-load facade to save bandwidth.

SEO essentials in HTML

- Unique, descriptive `<title>` under 60 characters.
- `<meta name="description">` under 160 characters.
- One `<h1>` matching the page topic; logical heading structure.
- Semantic landmarks so crawlers understand the page.
- Descriptive link text; internal links between related pages.
- alt text on images; descriptive file names.
- `<link rel="canonical">` to avoid duplicates; `hreflang` alternates for translations.
- Structured data with JSON-LD (schema.org) for rich results.
- Fast loading: compressed images, lazy loading, deferred scripts, `preconnect` for fonts.
- Mobile friendly: the viewport meta tag and responsive CSS.
- Open Graph and Twitter tags for attractive sharing previews.

Migrating old HTML (2, 3.2, 4) to HTML5

Old	New
<!DOCTYPE HTML PUBLIC ...>	<!DOCTYPE html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">	<meta charset="utf-8">
<script type="text/javascript">	<script>
<style type="text/css">	<style>
	CSS font-size, color, font-family
<center>	CSS text-align: center / margin: auto
 for importance	
<i> for emphasis	
<u>	CSS text-decoration or <ins>
<strike>	<s> or
<tt>	<code>, <kbd>, <samp>
<big>	CSS font-size
<div id="header">	<header>
<div id="nav">	<nav>
<div id="content">	<main>
<div class="sidebar">	<aside>
<div id="footer">	<footer>

Old	New
<code><div class="post"></code>	<code><article></code>
<code><table></code> for layout	CSS grid / flexbox
<code><frameset> / <frame></code>	CSS layout; <code><iframe></code> if needed
<code><applet></code> , <code><embed></code> Flash	<code><video></code> , <code><audio></code> , <code><canvas></code> , JavaScript
<code></code>	<code>id="x"</code> on any element
<code><acronym></code>	<code><abbr></code>
<code><dir></code> , <code><menu></code> as list	<code></code>
<code>align</code> , <code>bgcolor</code> , <code>border</code> , <code>cellpadding</code> , <code>valign</code> , <code>width</code> attributes	CSS
<code><br clear="all"></code>	CSS <code>clear</code> or <code>flow-root</code>
<code></code>	<code>img { border: 0 }</code> (default now)
<code>onclick="javascript:.. ."</code>	<code>addEventListener</code> in a script

Glossary and Index

Definitions of terms and an alphabetical index of everything in the book.

Glossary

Term	Definition
Accessibility (a11y)	Designing pages so that people with disabilities can use them, including with screen readers, keyboards, magnifiers and voice control.
ARIA	Accessible Rich Internet Applications: a set of role and aria-* attributes that describe widgets and states to assistive technology.
Attribute	A name=value pair inside an opening tag that configures the element, for example href="page.html".
Block-level element	An element that starts on a new line and fills the available width, such as <div>, <p>, <h1>, <footer>.
Boolean attribute	An attribute whose presence means true and absence means false, such as disabled, checked, required.
Browser	The program that reads HTML, CSS and JavaScript and displays web pages: Chrome, Firefox, Safari, Edge.
Cascade	The CSS mechanism that decides which of several conflicting rules applies, based on importance, specificity and order.
Character reference / entity	A way to write a character by name or number, such as & or ©.
Child / parent / sibling / ancestor / descendant	Family terms describing element nesting in the document tree.
Class	A reusable label applied with the class attribute and targeted in CSS with a dot: .name.
Content model	The rules stating which children an element may contain.

Term	Definition
CSS	Cascading Style Sheets: the language that describes presentation.
Deprecated	Still supported but discouraged; may be removed in future.
DOCTYPE	The declaration at the top of a document (<code><!DOCTYPE html></code>) that switches the browser to standards mode.
DOM	Document Object Model: the tree of objects the browser builds from HTML, which JavaScript can read and modify.
Element	An opening tag, its content and its closing tag together (or a void tag alone).
Empty / void element	An element with no content and no closing tag, such as <code>
</code> , <code></code> , <code><input></code> .
Flow content	Elements allowed in the body: most elements.
Focus	The state of the element that currently receives keyboard input, moved with Tab.
Fragment	The part of a URL after <code>#</code> , pointing to an element id.
Global attribute	An attribute allowed on every element, such as <code>id</code> , <code>class</code> , <code>style</code> , <code>title</code> , <code>lang</code> , <code>hidden</code> .
HTML	HyperText Markup Language: the language that structures web pages.
HTTP / HTTPS	The protocol browsers use to fetch pages; HTTPS is the encrypted version.
Hyperlink	A reference to another resource that the user can follow, created with <code><a href></code> .
Id	A unique identifier for one element, targeted in CSS with <code>#</code> and in links with <code>#</code> .
Inline element	An element that flows within text and takes only the width of its content: <code></code> , <code><a></code> , <code></code> .

Term	Definition
JavaScript	The programming language of the web, which adds behaviour to pages.
Landmark	A region of the page (banner, navigation, main, contentinfo...) that assistive technology can jump to.
Living Standard	The continuously updated HTML specification maintained by the WHATWG.
MDN	Mozilla Developer Network (developer.mozilla.org), the most complete free web documentation.
Metadata	Data about the document, placed in <code><head></code> .
MIME type	A label such as <code>text/html</code> or <code>image/png</code> that identifies the type of a file.
Nesting	Placing elements inside other elements.
Obsolete	Removed from the standard; must not be used.
Phrasing content	Text and inline elements that can appear inside a paragraph.
Pseudo-class / pseudo-element	CSS selectors for states (<code>:hover</code>) or generated parts (<code>::before</code>).
Quirks mode	A compatibility rendering mode triggered by a missing DOCTYPE that imitates 1990s browser bugs.
Rendering	The process of turning HTML and CSS into pixels.
Responsive design	Layouts that adapt to any screen size, using fluid widths, flexible images and media queries.
Screen reader	Software that reads the page aloud or in braille for blind users (NVDA, JAWS, VoiceOver, TalkBack).
Sectioning content	<code><article></code> , <code><aside></code> , <code><nav></code> , <code><section></code> : elements that define the scope of headings and footers.
Selector	The part of a CSS rule that chooses which elements to style.
Semantic HTML	Using elements according to their meaning.

Term	Definition
Specificity	The weight of a CSS selector, used by the cascade to resolve conflicts.
Tag	The markup in angle brackets, or , that starts or ends an element.
URL	Uniform Resource Locator: the address of a resource on the web.
UTF-8	The character encoding that supports every language; always use it.
Validator	A tool that checks HTML for errors (validator.w3.org).
Viewport	The visible area of the page in the browser window.
W3C	World Wide Web Consortium, the standards body that published HTML 3.2 to HTML 5.2.
WCAG	Web Content Accessibility Guidelines, the international accessibility standard.
WHATWG	The browser vendor group that maintains the HTML Living Standard.
XHTML	HTML written with XML syntax rules.

Glossary in depth

Each term again, with the places in this book where it matters most, so you can jump from a word you do not know to the chapter that explains it.

Accessibility (a11y)

Definition

Designing pages so that people with disabilities can use them, including with screen readers, keyboards, magnifiers and voice control.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Accessibility (a11y)' in the search box to list every entry that mentions it.

ARIA

Definition

Accessible Rich Internet Applications: a set of role and aria-* attributes that describe widgets and states to assistive technology.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'ARIA' in the search box to list every entry that mentions it.

Attribute

Definition

A name=value pair inside an opening tag that configures the element, for example href="page.html".

Where to read more

- Related elements (Part 6): <html>
- Search tip: in the HTML edition type 'Attribute' in the search box to list every entry that mentions it.

Block-level element

Definition

An element that starts on a new line and fills the available width, such as `<div>`, `<p>`, `<h1>`, `<footer>`.

Where to read more

- Related elements (Part 6): `<div>`, `<footer>`, `<h1>`
- Related CSS properties (Part 11): width / height
- Search tip: in the HTML edition type 'Block-level element' in the search box to list every entry that mentions it.

Boolean attribute

Definition

An attribute whose presence means true and absence means false, such as disabled, checked, required.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Boolean attribute' in the search box to list every entry that mentions it.

Browser

Definition

The program that reads HTML, CSS and JavaScript and displays web pages: Chrome, Firefox, Safari, Edge.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'Browser' in the search box to list every entry that mentions it.

Cascade

Definition

The CSS mechanism that decides which of several conflicting rules applies, based on importance, specificity and order.

Where to read more

- Related elements (Part 6): see the index
- Related CSS properties (Part 11): order
- Search tip: in the HTML edition type 'Cascade' in the search box to list every entry that mentions it.

Character reference / entity

Definition

A way to write a character by name or number, such as `&` or `©`.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Character reference / entity' in the search box to list every entry that mentions it.

Child / parent / sibling / ancestor / descendant

Definition

Family terms describing element nesting in the document tree.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Child / parent / sibling / ancestor / descendant' in the search box to list every entry that mentions it.

Class

Definition

A reusable label applied with the class attribute and targeted in CSS with a dot: `.name`.

Where to read more

- Related elements (Part 6): `<label>`
- Search tip: in the HTML edition type 'Class' in the search box to list every entry that mentions it.

Content model

Definition

The rules stating which children an element may contain.

Where to read more

- Related elements (Part 6): `<content>`
- Related CSS properties (Part 11): `contain` / `content-visibility`, `content`
- Search tip: in the HTML edition type 'Content model' in the search box to list every entry that mentions it.

CSS

Definition

Cascading Style Sheets: the language that describes presentation.

Where to read more

- Related elements (Part 6): `<style>`
- Search tip: in the HTML edition type 'CSS' in the search box to list every entry that mentions it.

Deprecated

Definition

Still supported but discouraged; may be removed in future.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Deprecated' in the search box to list every entry that mentions it.

DOCTYPE

Definition

The declaration at the top of a document (`<!DOCTYPE html>`) that switches the browser to standards mode.

Where to read more

- Related elements (Part 6): `<html>`
- Related CSS properties (Part 11): `position` / `top` / `right` / `bottom` / `left` / `inset` / `z-index`
- Search tip: in the HTML edition type 'DOCTYPE' in the search box to list every entry that mentions it.

DOM

Definition

Document Object Model: the tree of objects the browser builds from HTML, which JavaScript can read and modify.

Where to read more

- Related elements (Part 6): `<html>`, `<object>`
- Search tip: in the HTML edition type 'DOM' in the search box to list every entry that mentions it.

Element

Definition

An opening tag, its content and its closing tag together (or a void tag alone).

Where to read more

- Related elements (Part 6): `<content>`
- Related CSS properties (Part 11): `content`
- Search tip: in the HTML edition type 'Element' in the search box to list every entry that mentions it.

Empty / void element

Definition

An element with no content and no closing tag, such as `<br>`, `<img>`, `<input>`.

Where to read more

- Related elements (Part 6): `<br>`, `<content>`, `<img>`, `<input>`
- Related CSS properties (Part 11): content
- Search tip: in the HTML edition type 'Empty / void element' in the search box to list every entry that mentions it.

Flow content

Definition

Elements allowed in the body: most elements.

Where to read more

- Related elements (Part 6): `<body>`, `<content>`
- Related CSS properties (Part 11): `content`
- Search tip: in the HTML edition type 'Flow content' in the search box to list every entry that mentions it.

Focus

Definition

The state of the element that currently receives keyboard input, moved with Tab.

Where to read more

- Related elements (Part 6): `<input>`
- Search tip: in the HTML edition type 'Focus' in the search box to list every entry that mentions it.

Fragment

Definition

The part of a URL after #, pointing to an element id.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Fragment' in the search box to list every entry that mentions it.

Global attribute

Definition

An attribute allowed on every element, such as id, class, style, title, lang, hidden.

Where to read more

- Related elements (Part 6): <style>, <title>
- Search tip: in the HTML edition type 'Global attribute' in the search box to list every entry that mentions it.

HTML

Definition

HyperText Markup Language: the language that structures web pages.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'HTML' in the search box to list every entry that mentions it.

HTTP / HTTPS

Definition

The protocol browsers use to fetch pages; HTTPS is the encrypted version.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'HTTP / HTTPS' in the search box to list every entry that mentions it.

Hyperlink

Definition

A reference to another resource that the user can follow, created with `<a href>`.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Hyperlink' in the search box to list every entry that mentions it.

Id

Definition

A unique identifier for one element, targeted in CSS with # and in links with #.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Id' in the search box to list every entry that mentions it.

Inline element

Definition

An element that flows within text and takes only the width of its content: `<span>`, `<a>`, `<em>`.

Where to read more

- Related elements (Part 6): `<content>`, `<em>`, `<span>`
- Related CSS properties (Part 11): `content`, `width` / `height`
- Search tip: in the HTML edition type 'Inline element' in the search box to list every entry that mentions it.

JavaScript

Definition

The programming language of the web, which adds behaviour to pages.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'JavaScript' in the search box to list every entry that mentions it.

Landmark

Definition

A region of the page (banner, navigation, main, contentinfo...) that assistive technology can jump to.

Where to read more

- Related elements (Part 6): `<main>`
- Search tip: in the HTML edition type 'Landmark' in the search box to list every entry that mentions it.

Living Standard

Definition

The continuously updated HTML specification maintained by the WHATWG.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'Living Standard' in the search box to list every entry that mentions it.

MDN

Definition

Mozilla Developer Network (developer.mozilla.org), the most complete free web documentation.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'MDN' in the search box to list every entry that mentions it.

Metadata

Definition

Data about the document, placed in `<head>`.

Where to read more

- Related elements (Part 6): `<data>`, `<head>`
- Search tip: in the HTML edition type 'Metadata' in the search box to list every entry that mentions it.

MIME type

Definition

A label such as text/html or image/png that identifies the type of a file.

Where to read more

- Related elements (Part 6): <html>, <image>, <label>
- Search tip: in the HTML edition type 'MIME type' in the search box to list every entry that mentions it.

Nesting

Definition

Placing elements inside other elements.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Nesting' in the search box to list every entry that mentions it.

Obsolete

Definition

Removed from the standard; must not be used.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Obsolete' in the search box to list every entry that mentions it.

Phrasing content

Definition

Text and inline elements that can appear inside a paragraph.

Where to read more

- Related elements (Part 6): `<content>`
- Related CSS properties (Part 11): `content`
- Search tip: in the HTML edition type 'Phrasing content' in the search box to list every entry that mentions it.

Pseudo-class / pseudo-element

Definition

CSS selectors for states (:hover) or generated parts (::before).

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Pseudo-class / pseudo-element' in the search box to list every entry that mentions it.

Quirks mode

Definition

A compatibility rendering mode triggered by a missing DOCTYPE that imitates 1990s browser bugs.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Quirks mode' in the search box to list every entry that mentions it.

Rendering

Definition

The process of turning HTML and CSS into pixels.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'Rendering' in the search box to list every entry that mentions it.

Responsive design

Definition

Layouts that adapt to any screen size, using fluid widths, flexible images and media queries.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Responsive design' in the search box to list every entry that mentions it.

Screen reader

Definition

Software that reads the page aloud or in braille for blind users (NVDA, JAWS, VoiceOver, TalkBack).

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Screen reader' in the search box to list every entry that mentions it.

Sectioning content

Definition

`<article>`, `<aside>`, `<nav>`, `<section>`: elements that define the scope of headings and footers.

Where to read more

- Related elements (Part 6): `<article>`, `<aside>`, `<content>`, `<nav>`, `<section>`
- Related CSS properties (Part 11): content
- Search tip: in the HTML edition type 'Sectioning content' in the search box to list every entry that mentions it.

Selector

Definition

The part of a CSS rule that chooses which elements to style.

Where to read more

- Related elements (Part 6): `<style>`
- Search tip: in the HTML edition type 'Selector' in the search box to list every entry that mentions it.

Semantic HTML

Definition

Using elements according to their meaning.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'Semantic HTML' in the search box to list every entry that mentions it.

Specificity

Definition

The weight of a CSS selector, used by the cascade to resolve conflicts.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Specificity' in the search box to list every entry that mentions it.

Tag

Definition

The markup in angle brackets, `<p>` or `</p>`, that starts or ends an element.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'Tag' in the search box to list every entry that mentions it.

URL

Definition

Uniform Resource Locator: the address of a resource on the web.

Where to read more

- Related elements (Part 6): `<address>`
- Search tip: in the HTML edition type 'URL' in the search box to list every entry that mentions it.

UTF-8

Definition

The character encoding that supports every language; always use it.

Where to read more

- Related elements (Part 6): see the index
- Search tip: in the HTML edition type 'UTF-8' in the search box to list every entry that mentions it.

Validator

Definition

A tool that checks HTML for errors (validator.w3.org).

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'Validator' in the search box to list every entry that mentions it.

Viewport

Definition

The visible area of the page in the browser window.

Where to read more

- Related elements (Part 6): `<area>`
- Search tip: in the HTML edition type 'Viewport' in the search box to list every entry that mentions it.

W3C

Definition

World Wide Web Consortium, the standards body that published HTML 3.2 to HTML 5.2.

Where to read more

- Related elements (Part 6): `<body>`, `<html>`
- Search tip: in the HTML edition type 'W3C' in the search box to list every entry that mentions it.

WCAG

Definition

Web Content Accessibility Guidelines, the international accessibility standard.

Where to read more

- Related elements (Part 6): `<content>`
- Related CSS properties (Part 11): `content`
- Search tip: in the HTML edition type 'WCAG' in the search box to list every entry that mentions it.

WHATWG

Definition

The browser vendor group that maintains the HTML Living Standard.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'WHATWG' in the search box to list every entry that mentions it.

XHTML

Definition

HTML written with XML syntax rules.

Where to read more

- Related elements (Part 6): `<html>`
- Search tip: in the HTML edition type 'XHTML' in the search box to list every entry that mentions it.

Alphabetical index

Elements are shown in angle brackets, attributes in plain type, CSS properties with a leading dot-free name marked (CSS). Use your PDF reader's search (Ctrl+F) for anything not listed here.

Term	Where to find it
@font-face (CSS)	Part 11: Fonts and text
@import / @layer / @scope / @property (CSS)	Part 11: Interaction
@media (CSS)	Part 11: Interaction
@page (CSS)	Part 11: Interaction
@supports (CSS)	Part 11: Interaction
<a>	Part 6: Anchor (hyperlink)
<abbr>	Part 6: Abbreviation
abbr	Part 9: attribute on <td>, <th>
accent-color (CSS)	Part 11: Interaction
accept	Part 9: attribute on <form>, <input>
accept-charset	Part 9: attribute on <form>
Accessibility (a11y)	Part 14: glossary
accesskey	Part 4: global attribute
<acronym>	Part 6: Acronym (obsolete)
action	Part 9: attribute on <form>, <isindex>
<address>	Part 6: Contact address

Term	Where to find it
align	Part 9: attribute on <applet>, <caption>, <col>, <colgroup>, <div>, <h1>, <hr>, <iframe>, , <input>, <legend>, <object>, <p>, <spacer>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
align-items / align-self / align-content (CSS)	Part 11: Layout
alink	Part 9: attribute on <body>
all (CSS)	Part 11: Interaction
allow	Part 9: attribute on <fencedframe>, <iframe>
allowfullscreen	Part 9: attribute on <iframe>
allowpaymentrequest	Part 9: attribute on <iframe>
alpha	Part 9: attribute on <input>
alt	Part 9: attribute on <applet>, <area>, <image>, , <input>
animation / @keyframes (CSS)	Part 11: Motion
appearance (CSS)	Part 11: Interaction
<applet>	Part 6: Java applet (obsolete)
archive	Part 9: attribute on <applet>, <object>
<area>	Part 6: Image map area
ARIA	Part 14: glossary
aria-*	Part 4: global attribute
aria-label	Part 9: attribute on <svg>
aria-labelledby	Part 9: attribute on <svg>
<article>	Part 6: Self-contained article
as	Part 9: attribute on <link>
<aside>	Part 6: Aside (sidebar / tangential content)

Term	Where to find it
aspect-ratio (CSS)	Part 11: Box model
async	Part 9: attribute on <script>
Attribute	Part 14: glossary
attributionsrc	Part 9: attribute on <a>, , <script>
<audio>	Part 6: Embedded audio
autocapitalize	Part 4: global attribute
autocomplete	Part 9: attribute on <form>, <input>, <select>, <textarea>
autocorrect	Part 4: global attribute
autofocus	Part 4: global attribute
autofocus	Part 9: attribute on <button>, <input>, <keygen>, <select>, <textarea>
autoplay	Part 9: attribute on <audio>, <video>
axis	Part 9: attribute on <td>, <th>
	Part 6: Bring attention to (bold)
background	Part 9: attribute on <body>
background (CSS)	Part 11: Backgrounds
background-color (CSS)	Part 11: Backgrounds
background-image (CSS)	Part 11: Backgrounds
background-size / background-position / background-repeat / background-attachment / background-clip / background-origin (CSS)	Part 11: Backgrounds
balance	Part 9: attribute on <bgsound>

Term	Where to find it
<base>	Part 6: Document base URL
<basefont>	Part 6: Base font (obsolete)
<bdi>	Part 6: Bidirectional isolate
<bdo>	Part 6: Bidirectional text override
behavior	Part 9: attribute on <marquee>
bgcolor	Part 9: attribute on <body>, <col>, <colgroup>, <marquee>, <table>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
<bgsound>	Part 6: Background sound (non-standard, obsolete)
<big>	Part 6: Bigger text (obsolete)
<blink>	Part 6: Blinking text (non-standard, obsolete)
Block-level element	Part 14: glossary
blocking	Part 9: attribute on <link>, <script>, <style>
<blockquote>	Part 6: Block quotation
<body>	Part 6: Document body
Boolean attribute	Part 14: glossary
border	Part 9: attribute on <frameset>, , <object>, <table>
border (CSS)	Part 11: Box model
border-collapse / border-spacing (CSS)	Part 11: Box model
border-radius (CSS)	Part 11: Box model
bottommargin	Part 9: attribute on <body>
box-shadow (CSS)	Part 11: Box model
box-sizing (CSS)	Part 11: Box model
 	Part 6: Line break
Browser	Part 14: glossary

Term	Where to find it
<button>	Part 6: Button
<canvas>	Part 6: Graphics canvas
<caption>	Part 6: Table caption
capture	Part 9: attribute on <input>
caret-color (CSS)	Part 11: Interaction
Cascade	Part 14: glossary
cellpadding	Part 9: attribute on <table>
cellspacing	Part 9: attribute on <table>
<center>	Part 6: Centred block (obsolete)
challenge	Part 9: attribute on <keygen>
char	Part 9: attribute on <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
Character reference / entity	Part 14: glossary
charoff	Part 9: attribute on <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
charset	Part 9: attribute on <a>, <link>, <meta>, <script>
checked	Part 9: attribute on <input>, <menuitem>
Child / parent / sibling / ancestor / descendant	Part 14: glossary
<cite>	Part 6: Citation (title of a work)
cite	Part 9: attribute on <blockquote>, , <ins>, <q>
class	Part 4: global attribute
Class	Part 14: glossary
classid	Part 9: attribute on <object>

Term	Where to find it
clear	Part 9: attribute on
clip-path / mask (CSS)	Part 11: Backgrounds
closedby	Part 9: attribute on <dialog>
<code>	Part 6: Inline code
code	Part 9: attribute on <applet>
codebase	Part 9: attribute on <applet>, <object>
codetype	Part 9: attribute on <object>
<col>	Part 6: Table column
<colgroup>	Part 6: Table column group
color	Part 9: attribute on <basefont>, , <hr>, <link>
color (CSS)	Part 11: Fonts and text
color-scheme (CSS)	Part 11: Interaction
colorspace	Part 9: attribute on <input>
cols	Part 9: attribute on <frameset>, <multicol>, <textarea>
colspan	Part 9: attribute on <td>, <th>
columns / column-count / column-width / column-gap / column-rule / column-span / break-inside (CSS)	Part 11: Layout
command	Part 9: attribute on <button>, <menuitem>
commandfor	Part 9: attribute on <button>
compact	Part 9: attribute on <dir>, <dl>, <menu>, ,

Term	Where to find it
contain / content-visibility (CSS)	Part 11: Layout
container-type / container-name / @container (CSS)	Part 11: Layout
<content>	Part 6: Shadow DOM insertion point (obsolete)
content	Part 9: attribute on <meta>
content (CSS)	Part 11: Fonts and text
Content model	Part 14: glossary
contenteditable	Part 4: global attribute
controls	Part 9: attribute on <audio>, <video>
controlslist	Part 9: attribute on <audio>, <video>
coords	Part 9: attribute on <a>, <area>
counter-reset / counter-increment (CSS)	Part 11: Fonts and text
credentialless	Part 9: attribute on <iframe>
crossorigin	Part 9: attribute on <audio>, , <link>, <script>, <video>
csp	Part 9: attribute on <iframe>
CSS	Part 14: glossary
cursor (CSS)	Part 11: Box model
<data>	Part 6: Machine-readable value
data	Part 9: attribute on <object>
data-*	Part 4: global attribute
<datalist>	Part 6: Predefined options for an input
datetime	Part 9: attribute on , <ins>, <time>

Term	Where to find it
<dd>	Part 6: Description details
declare	Part 9: attribute on <object>
decoding	Part 9: attribute on
default	Part 9: attribute on <menuitem>, <track>
defer	Part 9: attribute on <script>
	Part 6: Deleted text
Deprecated	Part 14: glossary
<details>	Part 6: Disclosure widget
<dfn>	Part 6: Definition term
<dialog>	Part 6: Dialog box
<dir>	Part 6: Directory list (obsolete)
dir	Part 4: global attribute
dir	Part 9: attribute on <bdi>, <bdo>, <html>
direction	Part 9: attribute on <marquee>
direction / unicode-bidi / writing-mode (CSS)	Part 11: Fonts and text
dirname	Part 9: attribute on <input>, <textarea>
disabled	Part 9: attribute on <button>, <fieldset>, <input>, <keygen>, <link>, <menuitem>, <optgroup>, <option>, <select>, <textarea>
disablepictureinpicture	Part 9: attribute on <video>
disableremoteplayback	Part 9: attribute on <audio>, <video>
display	Part 9: attribute on <math>
display (CSS)	Part 11: Box model
<div>	Part 6: Generic block container
<dl>	Part 6: Description list

Term	Where to find it
DOCTYPE	Part 14: glossary
DOM	Part 14: glossary
download	Part 9: attribute on <a>, <area>
draggable	Part 4: global attribute
<dt>	Part 6: Description term
Element	Part 14: glossary
elementtiming	Part 9: attribute on
	Part 6: Emphasis
<embed>	Part 6: External content
Empty / void element	Part 14: glossary
enctype	Part 9: attribute on <form>
enterkeyhint	Part 4: global attribute
etc.	Part 9: attribute on <image>
event	Part 9: attribute on <script>
exportparts	Part 4: global attribute
face	Part 9: attribute on <basefont>,
<fencedframe>	Part 6: Fenced frame (experimental)
fetchpriority	Part 9: attribute on , <link>, <script>
field-sizing (CSS)	Part 11: Interaction
<fieldset>	Part 6: Form field group
<figcaption>	Part 6: Figure caption
<figure>	Part 6: Figure with optional caption
fill	Part 9: attribute on <svg>
filter / backdrop-filter (CSS)	Part 11: Backgrounds

Term	Where to find it
flex / flex-grow / flex-shrink / flex-basis (CSS)	Part 11: Layout
flex-direction / flex-wrap / flex-flow (CSS)	Part 11: Layout
float / clear (CSS)	Part 11: Layout
Flow content	Part 14: glossary
Focus	Part 14: glossary
	Part 6: Font (obsolete)
font (CSS)	Part 11: Fonts and text
font-family (CSS)	Part 11: Fonts and text
font-size (CSS)	Part 11: Fonts and text
font-style (CSS)	Part 11: Fonts and text
font-variant / font-variant-numeric / font-feature-settings (CSS)	Part 11: Fonts and text
font-weight (CSS)	Part 11: Fonts and text
<footer>	Part 6: Footer
for	Part 9: attribute on <label>, <output>, <script>
<form>	Part 6: Form
form	Part 9: attribute on <button>, <fieldset>, <input>, <keygen>, <label>, <meter>, <object>, <output>, <select>, <textarea>
formaction	Part 9: attribute on <button>, <input>
formenctype	Part 9: attribute on <button>, <input>
formmethod	Part 9: attribute on <button>, <input>

Term	Where to find it
formnovalidate	Part 9: attribute on <button>, <input>
formtarget	Part 9: attribute on <button>, <input>
Fragment	Part 14: glossary
<frame>	Part 6: Frame (obsolete)
frame	Part 9: attribute on <table>
frameborder	Part 9: attribute on <frame>, <frameset>, <iframe>
<frameset>	Part 6: Frameset (obsolete)
framespacing	Part 9: attribute on <frameset>
gap / row-gap / column-gap (CSS)	Part 11: Layout
Global attribute	Part 14: glossary
grid-auto-flow / grid-auto-rows / grid-auto-columns (CSS)	Part 11: Layout
grid-column / grid-row / grid-column-start etc. (CSS)	Part 11: Layout
grid-template-areas / grid-area (CSS)	Part 11: Layout
grid-template-columns / grid-template-rows (CSS)	Part 11: Layout
gutter	Part 9: attribute on <multicol>
<h1>	Part 6: Headings h1 to h6
<h2>	Part 6: see <h1>
<h3>	Part 6: see <h1>
<h4>	Part 6: see <h1>

Term	Where to find it
<h5>	Part 6: see <h1>
<h6>	Part 6: see <h1>
<head>	Part 6: Document head
<header>	Part 6: Header
headers	Part 9: attribute on <td>, <th>
height	Part 9: attribute on <applet>, <canvas>, <embed>, <fencedframe>, <iframe>, , <input>, <marquee>, <object>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>
<hgroup>	Part 6: Heading group
hidden	Part 4: global attribute
high	Part 9: attribute on <meter>
<hr>	Part 6: Thematic break (horizontal rule)
href	Part 9: attribute on <a>, <area>, <base>, <link>
hreflang	Part 9: attribute on <a>, <link>
hspace	Part 9: attribute on <applet>, , <marquee>, <object>
<html>	Part 6: Document root
HTML	Part 14: glossary
HTTP / HTTPS	Part 14: glossary
http-equiv	Part 9: attribute on <meta>
Hyperlink	Part 14: glossary
<i>	Part 6: Idiomatic text (italic)
icon	Part 9: attribute on <menuitem>
id	Part 4: global attribute
id	Part 9: attribute on <input>
Id	Part 14: glossary

Term	Where to find it
<iframe>	Part 6: Inline frame
<image>	Part 6: Image (non-standard alias, obsolete)
imagesizes	Part 9: attribute on <link>
imagesrcset	Part 9: attribute on <link>
	Part 6: Image
incremental	Part 9: attribute on <input>
inert	Part 4: global attribute
Inline element	Part 14: glossary
<input>	Part 6: Input control
inputmode	Part 4: global attribute
inputmode	Part 9: attribute on <input>
<ins>	Part 6: Inserted text
integrity	Part 9: attribute on <link>, <script>
is	Part 4: global attribute
<isindex>	Part 6: Search index prompt (obsolete)
ismap	Part 9: attribute on
itemid, itemprop, itemref, itemscope, itemtype	Part 4: global attribute
itemprop	Part 9: attribute on <meta>
JavaScript	Part 14: glossary
justify-content (CSS)	Part 11: Layout
justify-items / justify-self / place-items / place-content / place-self (CSS)	Part 11: Layout

Term	Where to find it
<kbd>	Part 6: Keyboard input
<keygen>	Part 6: Key-pair generator (obsolete)
keytype	Part 9: attribute on <keygen>
kind	Part 9: attribute on <track>
<label>	Part 6: Form control label
label	Part 9: attribute on <menu>, <menuitem>, <optgroup>, <option>, <track>
Landmark	Part 14: glossary
lang	Part 4: global attribute
lang	Part 9: attribute on <html>
language	Part 9: attribute on <script>
leftmargin	Part 9: attribute on <body>
<legend>	Part 6: Fieldset caption
letter-spacing / word-spacing (CSS)	Part 11: Fonts and text
	Part 6: List item
line-height (CSS)	Part 11: Fonts and text
<link>	Part 6: External resource link
link	Part 9: attribute on <body>
list	Part 9: attribute on <input>
list-style (CSS)	Part 11: Fonts and text
<listing>	Part 6: Listing (obsolete)
Living Standard	Part 14: glossary
loading	Part 9: attribute on <iframe>,
longdesc	Part 9: attribute on <frame>, <iframe>,

Term	Where to find it
loop	Part 9: attribute on <audio>, <bgsound>, <marquee>, <video>
low	Part 9: attribute on <meter>
lowsrc	Part 9: attribute on
<main>	Part 6: Main content
manifest	Part 9: attribute on <html>
<map>	Part 6: Image map
margin (CSS)	Part 11: Box model
marginheight	Part 9: attribute on <frame>, <iframe>
marginwidth	Part 9: attribute on <frame>, <iframe>
<mark>	Part 6: Highlighted (marked) text
<marquee>	Part 6: Scrolling text (obsolete)
<math>	Part 6: MathML root
max	Part 9: attribute on <input>, <meter>, <progress>
maxlength	Part 9: attribute on <input>, <textarea>
MDN	Part 14: glossary
media	Part 9: attribute on <link>, <meta>, <source>, <style>
<menu>	Part 6: Menu (toolbar list)
<menuitem>	Part 6: Menu item (obsolete)
<meta>	Part 6: Metadata
Metadata	Part 14: glossary
<meter>	Part 6: Scalar measurement (gauge)
method	Part 9: attribute on <form>
MIME type	Part 14: glossary

Term	Where to find it
min	Part 9: attribute on <input>, <meter>
min-width / max-width / min-height / max-height (CSS)	Part 11: Box model
minlength	Part 9: attribute on <input>, <textarea>
mix-blend-mode / isolation (CSS)	Part 11: Backgrounds
moz-opaque	Part 9: attribute on <canvas>
mozactionhint	Part 9: attribute on <input>
<multicol>	Part 6: Multi-column text (non-standard, obsolete)
multiple	Part 9: attribute on <input>, <select>
muted	Part 9: attribute on <audio>, <video>
n	Part 9: attribute on <nextid>
name	Part 9: attribute on <a>, <applet>, <button>, <details>, <fieldset>, <form>, <frame>, <iframe>, , <input>, <keygen>, <map>, <meta>, <object>, <output>, <param>, <select>, <slot>, <textarea>
<nav>	Part 6: Navigation section
Nesting	Part 14: glossary
<nextid>	Part 6: Next identifier (HTML 1 editor hint)
<nobr>	Part 6: No line break (non-standard, obsolete)
<noembed>	Part 6: Fallback for embed (obsolete)
<noframes>	Part 6: Fallback for frames (obsolete)
nohref	Part 9: attribute on <area>
nomodule	Part 9: attribute on <script>
nonce	Part 4: global attribute
nonce	Part 9: attribute on <script>, <style>

Term	Where to find it
noresize	Part 9: attribute on <frame>
<noscript>	Part 6: Fallback when scripting is disabled
noshade	Part 9: attribute on <hr>
novalidate	Part 9: attribute on <form>
nowrap	Part 9: attribute on <dd>, <td>, <th>
<object>	Part 6: External object
object	Part 9: attribute on <applet>
object-fit / object-position (CSS)	Part 11: Box model
Obsolete	Part 14: glossary
	Part 6: Ordered list
onbeforeunload	Part 9: attribute on <body>
onhashchange	Part 9: attribute on <body>
onlanguagechange	Part 9: attribute on <body>
onload	Part 9: attribute on <body>
onpopstate	Part 9: attribute on <body>
onresize	Part 9: attribute on <body>
onstorage	Part 9: attribute on <body>
onunload	Part 9: attribute on <body>
opacity (CSS)	Part 11: Box model
open	Part 9: attribute on <details>, <dialog>
<optgroup>	Part 6: Option group
optimum	Part 9: attribute on <meter>
<option>	Part 6: Option
order (CSS)	Part 11: Layout
orient	Part 9: attribute on <input>

Term	Where to find it
outline (CSS)	Part 11: Box model
<output>	Part 6: Calculation or user-action result
overflow / overflow-x / overflow-y (CSS)	Part 11: Box model
overflow-wrap / word-break / hyphens (CSS)	Part 11: Fonts and text
<p>	Part 6: Paragraph
padding (CSS)	Part 11: Box model
<param>	Part 6: Object parameter (deprecated)
part	Part 4: global attribute
pattern	Part 9: attribute on <input>
Phrasing content	Part 14: glossary
<picture>	Part 6: Picture (responsive image container)
ping	Part 9: attribute on <a>, <area>
placeholder	Part 9: attribute on <input>, <textarea>
<plaintext>	Part 6: Plain text (obsolete)
playsinline	Part 9: attribute on <video>
pointer-events (CSS)	Part 11: Interaction
popover	Part 4: global attribute
popovertarget	Part 9: attribute on <button>, <input>
popovertargetaction	Part 9: attribute on <button>, <input>
<portal>	Part 6: Portal (experimental, abandoned)
position / top / right / bottom / left / inset / z-index (CSS)	Part 11: Layout
poster	Part 9: attribute on <video>

Term	Where to find it
<pre>	Part 6: Pre-formatted text
preload	Part 9: attribute on <audio>, <video>
preserveAspectRatio	Part 9: attribute on <svg>
profile	Part 9: attribute on <head>
<progress>	Part 6: Progress indicator
prompt	Part 9: attribute on <isindex>
property	Part 9: attribute on <meta>
Pseudo-class / pseudo-element	Part 14: glossary
<q>	Part 6: Inline quotation
Quirks mode	Part 14: glossary
quotes (CSS)	Part 11: Fonts and text
radiogroup	Part 9: attribute on <menuitem>
<rb>	Part 6: Ruby base (deprecated)
readonly	Part 9: attribute on <input>, <textarea>
referrerpolicy	Part 9: attribute on <a>, <area>, <iframe>, , <link>, <portal>, <script>
rel	Part 9: attribute on <a>, <area>, <form>, <link>
Rendering	Part 14: glossary
required	Part 9: attribute on <input>, <select>, <textarea>
resize (CSS)	Part 11: Box model
Responsive design	Part 14: glossary
results	Part 9: attribute on <input>
rev	Part 9: attribute on <a>, <link>
reversed	Part 9: attribute on

Term	Where to find it
rightmargin	Part 9: attribute on <body>
role	Part 4: global attribute
role	Part 9: attribute on <svg>
rows	Part 9: attribute on <frameset>, <textarea>
rowspan	Part 9: attribute on <td>, <th>
<rp>	Part 6: Ruby fallback parenthesis
<rt>	Part 6: Ruby text (annotation)
<rtc>	Part 6: Ruby text container (deprecated)
<ruby>	Part 6: Ruby annotation
rules	Part 9: attribute on <table>
<s>	Part 6: Strikethrough (no longer accurate)
<samp>	Part 6: Sample output
sandbox	Part 9: attribute on <iframe>
scheme	Part 9: attribute on <meta>
scope	Part 9: attribute on <td>, <th>
scoped	Part 9: attribute on <style>
Screen reader	Part 14: glossary
<script>	Part 6: Script
scroll-behavior / scroll-snap-type / scroll-snap-align / scroll-margin / scroll-padding / overscroll-behavior (CSS)	Part 11: Motion
scrollamount	Part 9: attribute on <marquee>
scrolldelay	Part 9: attribute on <marquee>

Term	Where to find it
scrolling	Part 9: attribute on <frame>, <iframe>
<search>	Part 6: Search section
<section>	Part 6: Generic section
Sectioning content	Part 14: glossary
<select>	Part 6: Drop-down / list box
select	Part 9: attribute on <content>
selected	Part 9: attribute on <option>
<selectedcontent>	Part 6: Selected option display (customisable select)
Selector	Part 14: glossary
Semantic HTML	Part 14: glossary
<shadow>	Part 6: Shadow root insertion point (obsolete)
shadowrootclonable	Part 9: attribute on <template>
shadowrootdelegatesfocus	Part 9: attribute on <template>
shadowrootmode	Part 9: attribute on <template>
shadowrootserializable	Part 9: attribute on <template>
shape	Part 9: attribute on <a>, <area>
size	Part 9: attribute on <basefont>, , <hr>, <input>, <select>, <spacer>
sizes	Part 9: attribute on , <link>, <source>
<slot>	Part 6: Shadow DOM slot
slot	Part 4: global attribute
<small>	Part 6: Side comment (fine print)
<source>	Part 6: Media or image source
<spacer>	Part 6: Spacer (non-standard, obsolete)

Term	Where to find it
	Part 6: Generic inline container
span	Part 9: attribute on <col>, <colgroup>
Specificity	Part 14: glossary
spellcheck	Part 4: global attribute
spellcheck	Part 9: attribute on <textarea>
src	Part 9: attribute on <audio>, <bgsound>, <embed>, <frame>, <iframe>, <image>, , <input>, <portal>, <script>, <source>, <track>, <video>
srcdoc	Part 9: attribute on <iframe>
srclang	Part 9: attribute on <track>
srcset	Part 9: attribute on , <source>
standby	Part 9: attribute on <object>
start	Part 9: attribute on
step	Part 9: attribute on <input>
<strike>	Part 6: Strikethrough (obsolete)
stroke	Part 9: attribute on <svg>
stroke-width	Part 9: attribute on <svg>
	Part 6: Strong importance
<style>	Part 6: Embedded style sheet
style	Part 4: global attribute
<sub>	Part 6: Subscript
<summary>	Part 6: Disclosure summary
summary	Part 9: attribute on <table>
<sup>	Part 6: Superscript
<svg>	Part 6: Scalable Vector Graphics root

Term	Where to find it
tab-size (CSS)	Part 11: Fonts and text
tabindex	Part 4: global attribute
<table>	Part 6: Table
Tag	Part 14: glossary
target	Part 9: attribute on <a>, <area>, <base>, <form>, <link>
<tbody>	Part 6: Table body
<td>	Part 6: Table data cell
<template>	Part 6: Content template
text	Part 9: attribute on <body>
text-align (CSS)	Part 11: Fonts and text
text-decoration (CSS)	Part 11: Fonts and text
text-indent (CSS)	Part 11: Fonts and text
text-overflow (CSS)	Part 11: Fonts and text
text-shadow (CSS)	Part 11: Fonts and text
text-transform (CSS)	Part 11: Fonts and text
<textarea>	Part 6: Multi-line text input
<tfoot>	Part 6: Table footer
<th>	Part 6: Table header cell
<thead>	Part 6: Table header
<time>	Part 6: Date / time
<title>	Part 6: Document title
title	Part 4: global attribute
title	Part 9: attribute on <abbr>, <acronym>, <dfn>, <iframe>, <link>, <style>
topmargin	Part 9: attribute on <body>

Term	Where to find it
touch-action (CSS)	Part 11: Interaction
<tr>	Part 6: Table row
<track>	Part 6: Text track for media
transform / transform-origin / rotate / scale / translate (CSS)	Part 11: Motion
transition / transition-property / -duration / -timing-function / -delay (CSS)	Part 11: Motion
translate	Part 4: global attribute
truespeed	Part 9: attribute on <marquee>
<tt>	Part 6: Teletype text (obsolete)
type	Part 9: attribute on <a>, <area>, <button>, <embed>, <input>, , <link>, <menu>, <menuitem>, <object>, , <param>, <script>, <source>, <spacer>, <style>,
type=button	Part 7: input type
type=checkbox	Part 7: input type
type=color	Part 7: input type
type=date	Part 7: input type
type=datetime-local	Part 7: input type
type=email	Part 7: input type
type=file	Part 7: input type
type=hidden	Part 7: input type
type=image	Part 7: input type
type=month	Part 7: input type

Term	Where to find it
type=number	Part 7: input type
type=password	Part 7: input type
type=radio	Part 7: input type
type=range	Part 7: input type
type=reset	Part 7: input type
type=search	Part 7: input type
type=submit	Part 7: input type
type=tel	Part 7: input type
type=text	Part 7: input type
type=time	Part 7: input type
type=url	Part 7: input type
type=week	Part 7: input type
typemustmatch	Part 9: attribute on <object>
<u>	Part 6: Unarticulated annotation (underline)
	Part 6: Unordered list
URL	Part 14: glossary
usemap	Part 9: attribute on , <input>, <object>
user-select (CSS)	Part 11: Fonts and text
UTF-8	Part 14: glossary
Validator	Part 14: glossary
valign	Part 9: attribute on <col>, <colgroup>, <tbody>, <td>, <tfoot>, <th>, <thead>, <tr>
value	Part 9: attribute on <button>, <data>, <input>, , <meter>, <option>, <param>, <progress>
valuetype	Part 9: attribute on <param>
<var>	Part 6: Variable

Term	Where to find it
version	Part 9: attribute on <html>
vertical-align (CSS)	Part 11: Fonts and text
<video>	Part 6: Embedded video
viewBox	Part 9: attribute on <svg>
Viewport	Part 14: glossary
virtualkeyboardpolicy	Part 4: global attribute
visibility (CSS)	Part 11: Box model
vlink	Part 9: attribute on <body>
volume	Part 9: attribute on <bgsound>
vspace	Part 9: attribute on <applet>, , <marquee>, <object>
W3C	Part 14: glossary
<wbr>	Part 6: Word break opportunity
WCAG	Part 14: glossary
webkitdirectory	Part 9: attribute on <input>
WHATWG	Part 14: glossary
white-space (CSS)	Part 11: Fonts and text
width	Part 9: attribute on <applet>, <canvas>, <col>, <colgroup>, <embed>, <fencedframe>, <hr>, <iframe>, , <input>, <marquee>, <multicol>, <object>, <pre>, <source>, <spacer>, <svg>, <table>, <td>, <th>, <video>
width / height (CSS)	Part 11: Box model
will-change (CSS)	Part 11: Motion
wrap	Part 9: attribute on <pre>, <textarea>
writingsuggestions	Part 4: global attribute
XHTML	Part 14: glossary

Term	Where to find it
xml:lang, xml:base, xml:space	Part 4: global attribute
xmlns	Part 9: attribute on <html>, <math>, <svg>
<xmp>	Part 6: Example text (obsolete)